LLD Handbook — Vol 3: Interview Approach and Case Studies I

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

5 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — The LLD Interview & How to Approach It**
   - 1.1 What this round is, and what this handbook covers
   - 1.2 How the interview runs
   - 1.3 A complete answer in five minutes: "Design a music playlist"
   - 1.4 The step-by-step answer framework
   - 1.5 From one line to a list of requirements
   - 1.6 Finding the classes
   - 1.7 Connecting the classes (preview)
   - 1.8 Keeping the design honest with SOLID
   - 1.9 The design patterns that come up in this round
   - 1.10 Code, edge cases and concurrency: how to write interview code
   - 1.11 Worked mini-example, end to end: a small parking lot
   - 1.12 Communication tips
   - 1.13 How to use this handbook, and a study plan
   - 1.14 Common mistakes
   - In the interview
   - Self-check
   - TL;DR
2. **Chapter 14 — Case Studies: Everyday Objects**
   - 14.1 Design a Parking Lot
   - 14.2 Design a Vending Machine
   - 14.3 Design a Coffee Vending Machine
   - 14.4 Design an ATM
   - 14.5 Design an Elevator System
   - 14.6 Design a Traffic Signal Control System
   - 14.7 Design a Shipping Locker System
   - Chapter summary
3. **Chapter 15 — Case Studies: Booking & Inventory (Part 1)**
   - Why booking problems are interview favourites
   - 15.1 Design a Library Management System
   - 15.2 Design a Movie Ticket Booking System (BookMyShow)
   - 15.3 Design a Concert Ticket Booking System
   - 15.4 Design a Hotel Management System
   - Chapter summary
4. **Chapter 16 — Case Studies: Booking & Inventory (Part 2)**
   - Design a Car Rental System
   - Design a Restaurant Management System
   - Design an Airline Management System
   - Design a Course Registration System
   - Chapter summary
5. **Chapter 17 — Case Studies: Games & Rules**
   - Design Blackjack and a Deck of Cards
   - Design Chess
   - Design Tic-Tac-Toe
   - Design Snake and Ladder
   - Design 2048
   - Chapter summary

Chapter 1 — The LLD Interview & How to Approach It

What you will learn in this chapter

- What the low-level design (LLD) round is, what it is called at different companies, and how it differs from the system design round.
- The difference between the **object-oriented design (OOD) round** and the **machine-coding round**.
- A complete answer, start to finish, before any theory (a music playlist in Java).
- A repeatable, step-by-step framework that turns one sentence into working code, with a time budget.
- How to turn one line into requirements, how to find classes, how to check a design with SOLID (five rules of thumb that keep classes easy to change), and which six patterns come up most.
- What interviewers actually score, how to talk while you design, and the mistakes that cost the most.
- A study plan and a map of this handbook (Chapters 1–22).
- A second worked mini-example (a small parking lot) run through every step of the framework.

1.1 What this round is, and what this handbook covers

What it is

The **low-level design** round gives you **a single application** to design while an interviewer watches, usually within 45–60 minutes (a machine-coding round stretches this to 90 minutes or 2 hours). The servers underneath are not your concern. What you design is **the set of classes inside the program**: a parking lot, a vending machine, a chess game, Splitwise (an app that splits shared expenses between friends), an elevator.

A few words we will use all the time:

Term	Meaning
Class	A named type in code. It holds some data (fields) and the methods (functions that belong to the class) that work on that data.
Object	One concrete instance of a class, with real values in its fields.
Responsibility	The one job a class does, which you should be able to state in a single sentence.
Relationship	How two classes are connected: one uses, holds, owns, or extends the other.
UML	Unified Modeling Language: the standard set of symbols for drawing classes and their relationships.
Design pattern	A named, reusable answer to a design problem that keeps coming up (for example Strategy or Observer).

Everyday analogy: system design is the **city plan** (roads, power stations, water pipes). Low-level design is the **floor plan of one house**: which rooms exist, what each room is for, and which doors connect them.

Plenty of engineers write classes daily and still struggle here, because day-to-day work seldom makes you **explain aloud** why each class exists and **justify** the way they connect. This round is exactly that.

What you should be able to do after this handbook

- Turn one sentence ("Design a parking lot") into a bounded list of requirements, with an explicit out-of-scope list.
- Pick out the classes a problem calls for, with **exactly one job** apiece.
- Decide how any pair of classes is related, and draw that link without mistakes.

- Code the handful of Java methods behind the core flows until they actually execute.
- Say which design pattern you used **and which one you rejected**, and why.

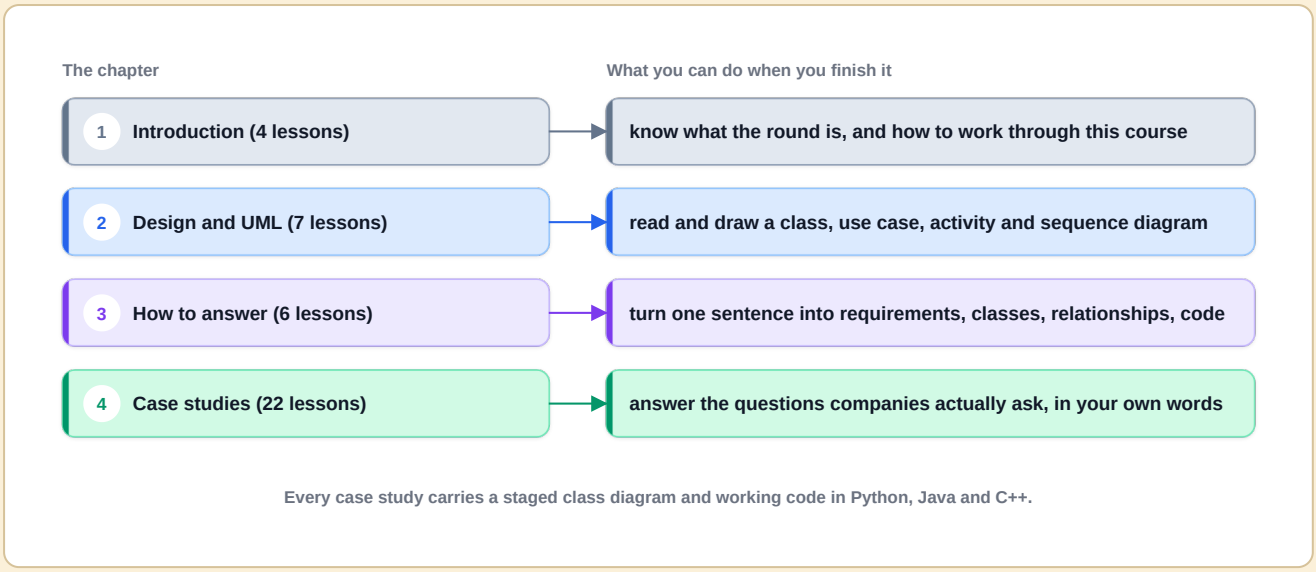


Figure — The four parts of the course material this handbook absorbs: introduction, design and UML notation, the answering method, and the case studies (course diagram)

The figure above shows the four building blocks of preparation: first understand the round (this chapter), then the notation (UML), then the **method** for answering, then many worked case studies. This handbook follows the same arc but goes wider: it adds full chapters on OOP, SOLID, the classic design principles, clean code, refactoring (improving the structure of code without changing what it does), all 23 GoF patterns (the classic catalogue from the 1994 "Gang of Four" book), and concurrency (several pieces of work running at the same time and sharing data), so that you do not need a second resource.

What every case study in this handbook contains

Once you have read one case study you always know what comes next, because each one follows the same template:

- The question, and the requirements chosen to design against (with an out-of-scope list).
- The actors and their use cases (often as a use case diagram, a picture of who uses the system and for what).
- The class diagram, built up in stages so you follow the reasoning instead of staring at a finished picture.
- Diagrams for the most important flows (activity diagrams show the steps of a process; sequence diagrams show the messages objects send each other, in time order).
- Working Java code with a driver `Main` (a small class whose `main` method runs the example) and its real output.
- The design patterns the answer uses, named, **only where it really uses one**.
- An "Extensions & follow-up questions" section: the simplifications made, and what to say when the interviewer notices them.

Who this is for

- You have an interview coming up and this round is part of it.
- You write object-oriented code at work but have never had to defend a class diagram out loud.
- You prepared for system design and discovered this is a different round with different rules.

It also pays off outside interviews: the same step-by-step method is how you split a new feature into classes at work when nobody has designed it yet.

What you need already

- **Some object-oriented language**, well enough that a class with a constructor (the special method that sets up a new object) plus a few methods reads easily to you. Every example here is in **Java 21** (a

recent long-term-support release of Java).

- You do **not** need UML (Chapter 3 teaches it from zero).
- You do **not** need system design knowledge. That round asks how a product survives millions of users (servers, databases, caches, queues; see System Design Fundamentals if those words are new). This round stays **inside one process** (one running program: classes, fields, methods, relationships). Practising one round gives you little for the other.
- You do not need data structures and algorithms beyond the basics (lists, maps, sets, queues, heaps). That is a separate round. (Quick refresher: a **list** keeps items in order, a **map** looks a value up by its key, a **set** holds no duplicates, a **queue** hands items out first-in, first-out, and a **heap** always gives you the smallest or largest item quickly.)

What this handbook will not do

It will not teach you a programming language from scratch (it assumes you can already read one). It will not give you scripts to memorise. A memorised answer is useless the moment the question is one you have not seen. **The reusable part is the method.**

TL;DR

The LLD round = design the classes inside one application, out loud, then code the main flows. Prerequisite: one OO language. The method beats memorised answers.

1.2 How the interview runs

What it is

The interviewer says one sentence, for example "Design a parking lot", and then stops talking. Nothing in that sentence tells you how many classes they want, whether to code, or how long you have. **That silence is the hard part.** This section removes it.

The round has several names, all meaning roughly the same thing:

Name	Where you hear it	What is different
Object-oriented design (OOD) interview	US big tech, many product companies	Whiteboard/doc: requirements, classes, diagram, code for main flows.
Low-level design (LLD)	Indian product companies, general usage	Same as OOD; the umbrella term.
Machine-coding round	Commonly reported at Indian product companies such as Flipkart, Swiggy and PhonePe	Same idea, but **working, runnable code is mandatory** at the end, usually in a 90-minute to 2-hour window.

LLD vs HLD (system design). Same product, different zoom level:

	Low-level design (this handbook)	High-level / system design
Scope	Inside one application or service	Many services and machines
Vocabulary	Classes, interfaces, fields, methods, relationships, patterns	Servers, load balancers, databases, caches, queues, sharding
Output	Class diagram + code	Architecture diagram + capacity numbers
Typical question	"Design the classes for a parking lot"	"Design a parking-lot booking service for 10 million users"

(For the right-hand column: a **load balancer** spreads incoming requests across several servers, and **sharding** splits one big database into pieces stored on different machines. You will not need either in this round.)

OOD round vs machine-coding round

	OOD / LLD discussion round	Machine-coding round
Duration	45–60 minutes	Usually about 90 minutes of coding (often 2 hours including evaluation)
Where	Whiteboard, shared doc, or IDE (code editor with build and run tools)	Your own IDE, on your laptop or a remote machine
What you produce	Requirements, use cases, class diagram, code for 2–3 main flows	A **complete, running program** implementing the minimum required features
Storage	Not discussed much	**In-memory** (maps and lists); no real database unless asked
Interaction	Continuous conversation with the interviewer	Problem statement up front, mostly silent coding, then a **demo and code walkthrough**
What is judged	Bounding the problem, responsibilities, relationships, trade-offs, patterns	Working code, **data models**, **service interfaces** a client could call, **extensibility** (can a new feature be added without rewriting), readability, edge cases, sometimes tests and thread safety
Common failure	Talking without converging, or coding too early	Not finishing, or finishing with one giant class that cannot be extended

The machine-coding round, as commonly described by candidates, works like this: you get a problem statement with a **minimum set of features**; within the time window you must choose the data structures, design **service interfaces** that a client (for example a command-line driver, a small program you run from a terminal) would use, and present **working code** together with the reasoning behind your design and the patterns you used. It is especially common for SDE-1 and SDE-2 hiring (Software Development Engineer job levels: entry level and mid level).

A few words from the table above: **data models** are the classes that hold the problem's data (a `Ticket`, a `Spot`); a **service interface** is the set of public methods a caller uses (`park`, `exit`); **edge cases** are unusual inputs at the boundaries (empty, full, negative, duplicate); **thread safety** means the code stays correct when several **threads** (independent lines of execution inside one program) use it at the same moment.

How it works: the five core steps

Every question in this handbook is answered with the same five core steps (Section 1.4 expands them into a finer checklist):

1. **Clarify the requirements** — turn one sentence into a short written list of things the system must do.
2. **Find the actors and use cases** — who uses the system (actors) and what each of them does (use cases).
3. **Find the classes** — turn requirements into classes; give each one responsibility.
4. **Connect the classes** — decide how each pair relates; record it in a class diagram.
5. **Write the code** — the classes plus the 2–3 methods that implement the main flows.

Getting the order right matters more than going fast. Each step feeds the next. Skip step 1 and step 3 has nothing to work from: you end up inventing classes no requirement asked for.

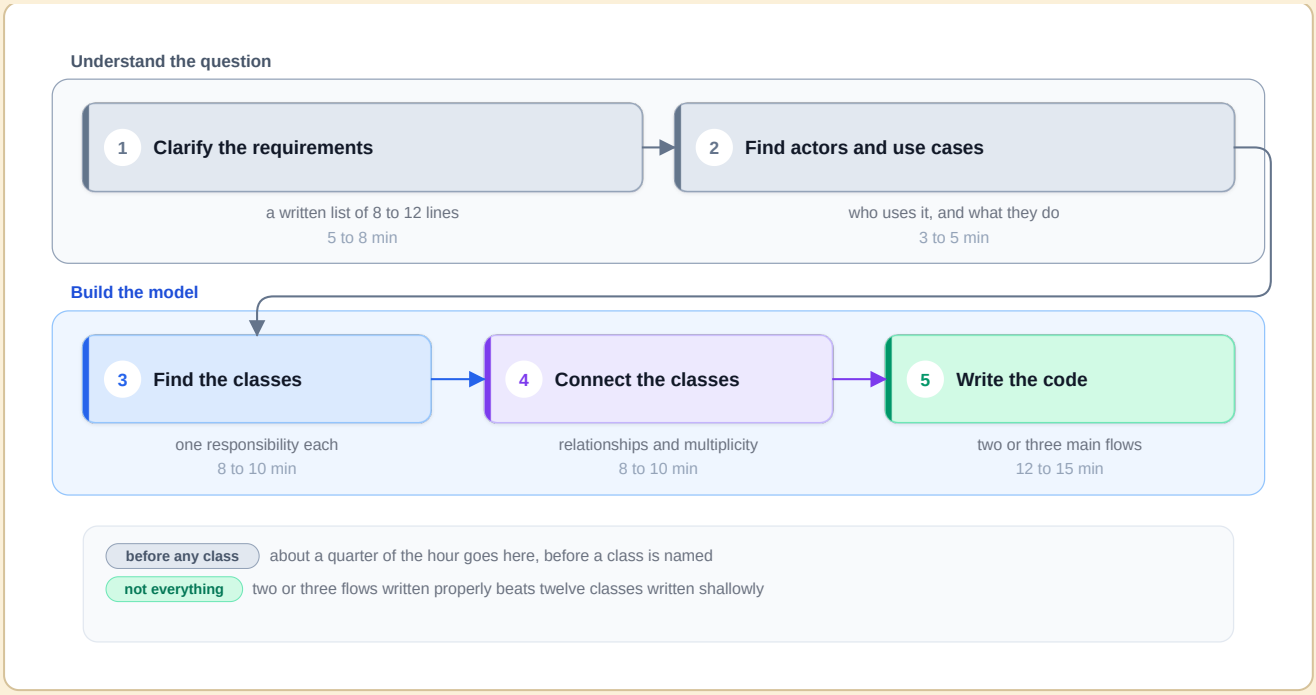


Figure — The five steps split into "understand the question" and "build the model", with a time budget for each (course diagram)

In the figure above, notice two things. First, up to **a quarter of the hour is spent before a single class is named** (requirements plus actors). Second, the code step is the largest single block, yet it covers only two or three flows.

Nobody completes the whole product in the time, and you are not expected to. The design is judged on the part you chose to build properly, not on how much of the product you touched.

Time-boxing

45-minute OOD round:

Step	Time	What exists at the end
Clarify requirements	5–8 min	A written list of 8–12 requirements + out-of-scope list
Actors and use cases	3–5 min	2–4 actors, 6–10 use cases
Find the classes	8–10 min	Class list, one responsibility each
Connect the classes	8–10 min	Class diagram with relationships and multiplicity (how many of each)
Write the code	12–15 min	2–3 methods that implement the main flows

90-minute machine-coding round (a budget that works well in practice):

Phase	Time	Output
Read the statement, list requirements and assumptions	5–10 min	Comment block at the top of `Main.java`
Model: entities, enums, service interfaces	10–15 min	Empty classes/interfaces that compile
Implement the core flow end to end	30–40 min	One happy path running from a driver `main`
Remaining features + validation + edge cases	15–20 min	All "must have" features working
Polish: names, exceptions, one extension point, demo script	5–10 min	Clean output you can demo
Buffer	5 min	Never zero

In this table, code that **compiles** is code the Java compiler accepts, so it can run; **enums** are types with a fixed list of named values (more in Section 1.6); the **happy path** is the main flow with valid input and nothing going wrong.

Rule of thumb: **get something running by the halfway mark**. A running partial solution beats a beautiful non-compiling one.

What the interviewer is actually judging

The interviewer is not counting classes. They watch for:

1. **Did you bound the problem?** A design for a clear, bounded set of requirements beats a broad design for an unbounded one.
2. **Does each class have one job you can state in one sentence?** If one sentence cannot describe a class, it is normally two classes glued together.
3. **Are the relationships right?** Holding, owning, and extending are different choices; a wrong one is visible immediately.
4. **Does the code match the diagram?** If the code has a method that the diagram has nowhere to put, either the code or the diagram is wrong.
5. **Can you explain a trade-off?** Each design decision has a road taken and a road not taken; naming both scores higher than naming only one.

Machine-coding rounds add: **does it run**, is it **modular** (can a new vehicle type / payment method / rule be added by adding a class), is it **readable** (names, small methods), does it **handle bad input** gracefully, and (for senior roles) is shared state **thread-safe**.

Notice what is **not** on the list: perfect UML notation, or using every diagram type. **What is graded is the model; the notation is just the way you present it.**

A worked example of size

Take the parking lot. About 12 requirements produce about 12 classes with about 15 relationships between them. Three classes implement the main flows: one hands out a ticket, one holds a vehicle, one prices the stay. You write those three properly. You name the other nine and describe each in a sentence. That is a complete answer in 45 minutes.

The most expensive mistake is writing all twelve classes at the same level of detail. It feels thorough, but the clock runs out before any single flow works end to end.

> **TL;DR**
>
> OOD, LLD and machine coding are one family of rounds; machine coding requires running code in ~90 minutes. Follow five steps in order, spend a quarter of the time before naming a class, and build two or three flows properly rather than twelve classes shallowly.

1.3 A complete answer in five minutes: "Design a music playlist"

Let's see a complete answer first and save the theory for later. It is small on purpose: a screenful of classes, a single diagram, a page of Java. Later case studies are roughly ten times the size, yet they follow **the identical outline**. Don't worry if some diagram notation is unfamiliar; Chapter 3 covers it.

Step 1: requirements

The interviewer says only: "Design a music playlist." Three clarifying questions produce three requirements:

- Songs sit in a playlist, and **their sequence is significant**.
- Someone can ask for **the following song**, and the player keeps track of its current position.
- A user can turn **shuffle** on, and the play order changes.

Out of scope, agreed out loud: user accounts, downloads, storage, and the audio itself. An explicit "not building" list earns as much credit as the "building" list.

Step 2: actors and use cases

Anyone or anything outside the application that interacts with it is an **actor**. This example has just one, the **listener**, who does three things: adds a song, plays the next one, switches shuffle on. For so small a question this takes ten seconds; for a parking lot it costs three minutes and repays ten, since **the actors reveal which classes have to exist**.

Step 3: classes, one sentence each

Class	Responsibility (one sentence, no "and" where possible)
`Song`	Holds a title, an artist and a length; knows about itself and nothing else.
`Playlist`	Holds songs in order; stores them, does not play them.
`PlayOrder` (interface)	Picks the position to play after the current one. It is an interface since several ordering rules exist.
`InOrder`	Walks through positions in insertion order.
`Shuffled`	Plays positions in a shuffled order decided once, up front.
`Player`	Holds a playlist and an order, and remembers the current position.

An **interface** names a set of methods without saying how they work. The key insight: **a rule that can vary becomes its own type.** Without shuffle, `Player` could just count upwards and `PlayOrder` would not exist. With shuffle, "what plays next" has two versions, so it becomes `PlayOrder`, and the player no longer needs to know which version it holds.

A useful test: if a responsibility sentence needs the word **"and"** to join two unrelated jobs, the class is probably two classes.

Step 4: connect them

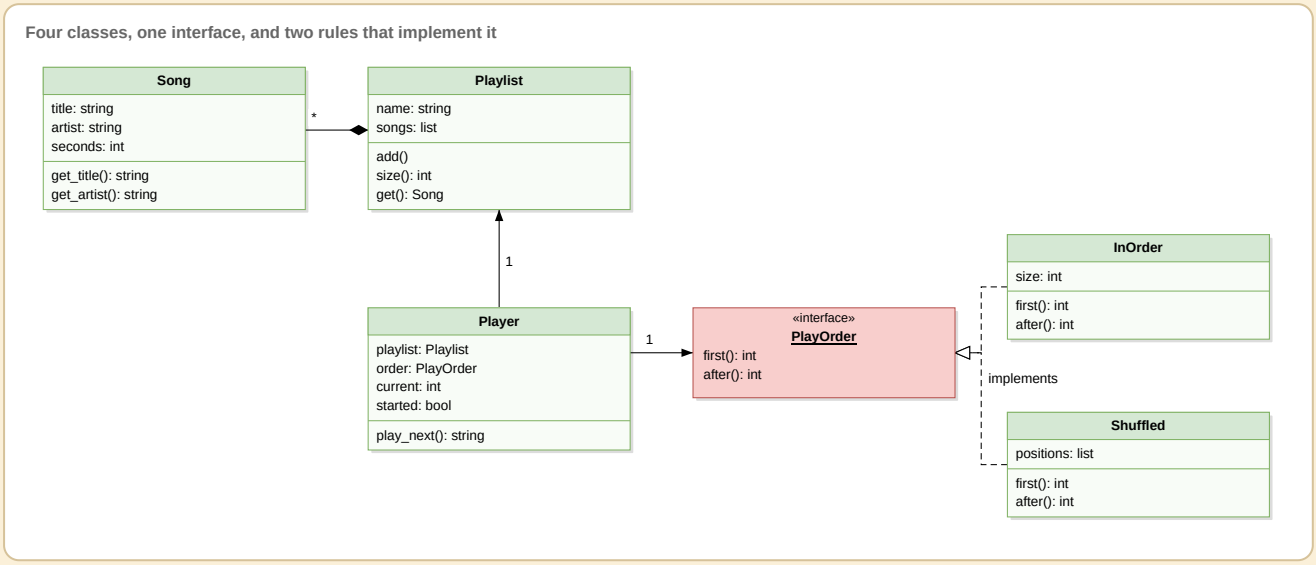
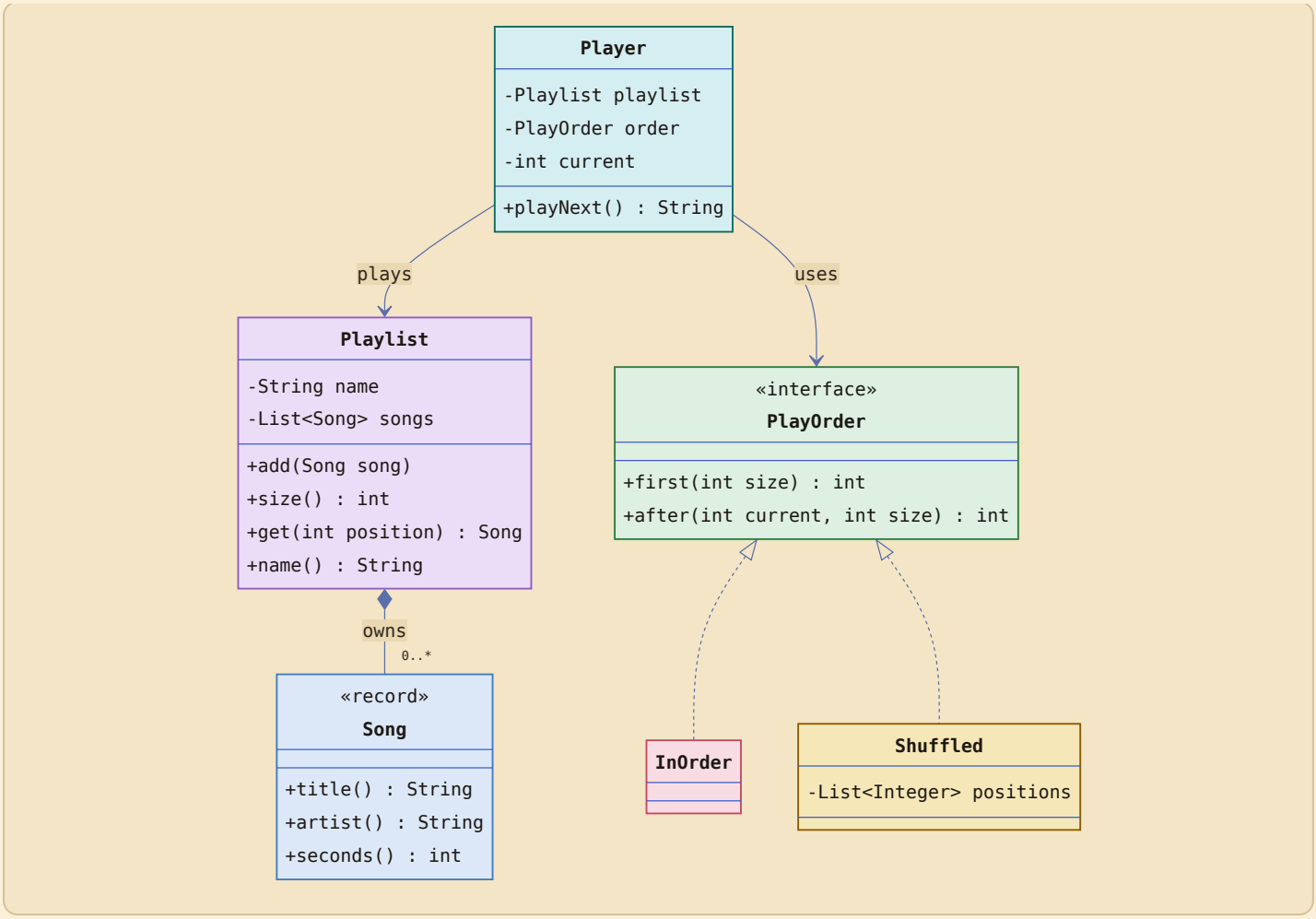


Figure — The playlist design: Playlist owns many Songs, Player uses a PlayOrder, and InOrder and Shuffled implement PlayOrder (course diagram)

Three facts from the figure above are exactly what an interviewer checks:

- The **filled diamond** between `Playlist` and `Song` means the playlist **owns** its songs (composition); the `*` means it owns **many**.
- A **plain arrow** running from `Player` to `PlayOrder` says the player **relies on** an ordering without owning one. Its head touches the **interface** only, not `InOrder` or `Shuffled`.
- The **dashed arrows with hollow triangles** mean `InOrder` and `Shuffled` **implement** `PlayOrder`. A third rule later is one extra box; nothing already drawn changes.

Here is the same design for the Java code below:



Step 5: the code

Key term — reading the Java below: A `package` line groups related classes, much like a folder. `implements PlayOrder` promises that the class provides every method of that interface, and `@Override` marks a method that fulfils such a promise (the compiler checks it). `final` on a class means no other class may extend it; on a field it means the field is set once and never reassigned. `private` hides a field or method from code outside the class.

```java
// File: code/lld/ch01/playlist/InOrder.java
package lld.ch01.playlist;

/** Plays positions in the order the songs were added. */
public final class InOrder implements PlayOrder {
    @Override public int first(int size) { return size == 0 ? NONE : 0; }
    @Override public int after(int current, int size) { return current + 1 < size ? current + 1 : NONE; }
}
```

```java
// File: code/lld/ch01/playlist/PlayOrder.java
package lld.ch01.playlist;

/** The rule that decides which position plays next. A rule that varies becomes its own type. */
public interface PlayOrder {
    int NONE = -1;                          // "there is no next position"

    int first(int size);                    // first position to play, or NONE if empty
    int after(int current, int size);       // position after 'current', or NONE at the end
}
```

```java
// File: code/lld/ch01/playlist/Player.java
package lld.ch01.playlist;

/** Holds a playlist and an order, and remembers where it is. It never asks WHICH order it holds. */
public final class Player {
    private final Playlist playlist;
    private final PlayOrder order;           // depends on the interface, not on InOrder/Shuffled
    private int current = PlayOrder.NONE;
    private boolean started = false;

    public Player(Playlist playlist, PlayOrder order) {
        this.playlist = playlist;
        this.order = order;
    }

    public String playNext() {
        if (!started) {
            current = order.first(playlist.size());
            started = true;
        } else if (current != PlayOrder.NONE) {          // once at the end, stay at the end
            current = order.after(current, playlist.size());
        }
        if (current == PlayOrder.NONE) return "end of playlist";
        Song s = playlist.get(current);
        return "playing " + s.title() + " by " + s.artist();
    }
}
```

```java
// File: code/lld/ch01/playlist/Playlist.java
package lld.ch01.playlist;

import java.util.ArrayList;
import java.util.List;

/** Holds songs in order. It stores them; it does not play them. */
public final class Playlist {
    private final String name;
    private final List<Song> songs = new ArrayList<>();   // composition: the playlist owns its list

    public Playlist(String name) { this.name = name; }

    public void add(Song song) { songs.add(song); }
    public int size()          { return songs.size(); }
    public Song get(int position) { return songs.get(position); }
    public String name()       { return name; }
}
```

```java
// File: code/lld/ch01/playlist/Shuffled.java
package lld.ch01.playlist;

import java.util.List;

/**
 * Plays positions in an order decided once, up front. The order is injected
 * (not generated inside) so the program prints the same output every run: deterministic = testable.
 */
public final class Shuffled implements PlayOrder {
    private final List<Integer> positions;

    public Shuffled(List<Integer> positions) { this.positions = List.copyOf(positions); }

    @Override public int first(int size) { return positions.isEmpty() ? NONE : positions.get(0); }

    @Override public int after(int current, int size) {
        int i = positions.indexOf(current);
        return (i >= 0 && i + 1 < positions.size()) ? positions.get(i + 1) : NONE;
    }
}
```

```java
// File: code/lld/ch01/playlist/Song.java
package lld.ch01.playlist;

/** A song knows about itself and nothing else. A record = immutable data carrier. */
public record Song(String title, String artist, int seconds) {
    public Song {
        if (title == null || title.isBlank()) throw new IllegalArgumentException("title required");
        if (seconds <= 0) throw new IllegalArgumentException("length must be positive");
    }
}
```

```java
// File: code/lld/ch01/playlist/Main.java
package lld.ch01.playlist;

import java.util.List;

public class Main {
    public static void main(String[] args) {
        Playlist mix = new Playlist("Morning mix");
        mix.add(new Song("Sunrise", "Ada Lane", 201));
        mix.add(new Song("Blue Line", "The Meridians", 187));
        mix.add(new Song("Paper Boats", "Nori Kim", 243));

        run("in order", new Player(mix, new InOrder()), mix.size());
        run("shuffled", new Player(mix, new Shuffled(List.of(2, 0, 1))), mix.size());
    }

    private static void run(String label, Player player, int songs) {
        System.out.println(label + ":");
        for (int i = 0; i <= songs; i++) {          // one extra call shows the end
            System.out.println("  " + player.playNext());
        }
    }
}
```

Run it: `cd chapters/01-lld-interview-approach/code && javac --release 21 -d /tmp/lld_01 $(find . -name '*.java') && java -cp /tmp/lld_01 lld.ch01.playlist.Main`

(`javac` is the Java compiler, which turns `.java` source files into runnable `.class` files; `java` then runs them, and `-cp` (classpath) tells it where those files are.)

Output:

```
in order:
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  playing Paper Boats by Nori Kim
  end of playlist
shuffled:
  playing Paper Boats by Nori Kim
  playing Sunrise by Ada Lane
  playing Blue Line by The Meridians
  end of playlist
```

Explaining each part:

- `Song` is a Java **record**: an immutable (cannot change after creation) data carrier whose fields, constructor, accessors (getter methods such as `title()`), `equals`, `hashCode` and `toString` (the standard methods for comparing objects, using them as map keys, and printing them) are generated. The compact constructor (a constructor with no parameter list, used just to check the values) validates input.
- `Playlist` keeps its list `private` (encapsulation: hiding a class's data so it can only be reached through the class's own methods). Callers can add and read, but cannot replace the list.
- `PlayOrder` is the variation point (the place where different behaviours can be plugged in). `NONE = -1` is an interface constant meaning "nothing more to play".

- `Shuffled` is **given** its order instead of generating one with `Random`. This makes the output identical on every run, which makes the design **testable**. (In production you would inject a `Random` with a seed, or a shuffled list produced elsewhere. To **inject** something is to hand it in from outside, usually through the constructor; a **seed** is a fixed starting number that makes `Random` produce the same sequence every time.)
- `Player` depends on the interface `PlayOrder`, not on a concrete class (a real class you can create objects from, as opposed to an interface). One `Player` gives both behaviours, and **nothing in it tests which mode is active**. (Its only `if`s track *where* it is: the first call asks `first`, later calls ask `after`, and once it has reached the end it stays there instead of asking the order again.)

What to notice

- **One requirement turned into a type.** Shuffle occupied a single line of the requirements, yet it produced an interface plus two implementations. That is the usual flow: **classes come out of requirements**, and it never runs backwards.
- **Each class has exactly one job.** A `Song` has no idea any playlist contains it. A `Playlist` is unaware that playback happens. A `Player` never learns how the next position gets picked.
- **The design is open to changes you did not build.** "Repeat one song" and "play in reverse" are just new classes implementing `PlayOrder`; nothing already written is reopened. (This is the Strategy pattern and the Open/Closed Principle, Chapters 4 and 11.)
- **Things were left out on purpose:** pause, volume, queue, time. In an interview you **announce** these gaps yourself instead of waiting to be caught. Every case study in this handbook ends with an "Extensions & follow-up questions" section for the same reason.

TL;DR

A complete answer = a short bounded requirement list, a few one-sentence classes, one diagram, and code for the main flow. A rule that varies becomes an interface, and the class that uses it never asks which version it holds.

1.4 The step-by-step answer framework

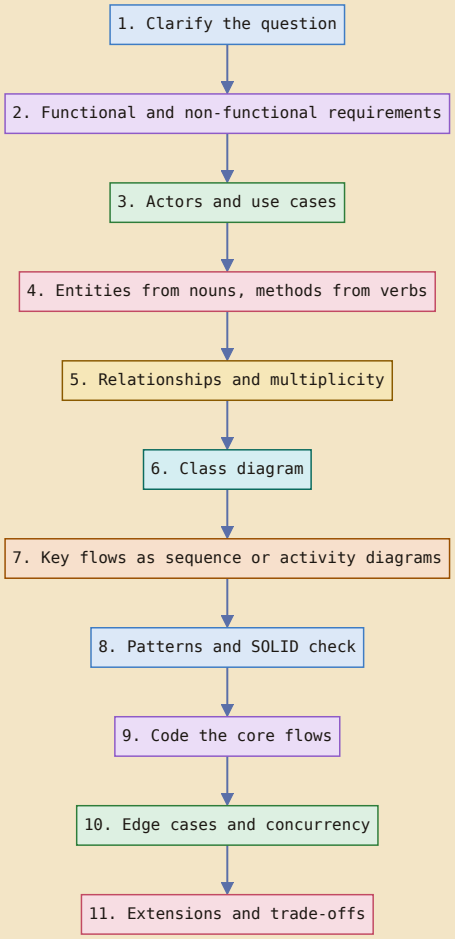
What it is

A checklist you run on **every** LLD question, so the question stops being "where do I start?" and becomes a series of small questions with known answers. It expands the five core steps into eleven finer ones.

The problem it solves

Without a framework, candidates either freeze, or jump straight into code and discover missing requirements at minute 35. The framework makes your progress **visible** to the interviewer, which is itself part of the score.

How it works


1. Clarify the question
2. Functional and non-functional requirements
3. Actors and use cases
4. Entities from nouns, methods from verbs
5. Relationships and multiplicity
6. Class diagram
7. Key flows as sequence or activity diagrams
8. Patterns and SOLID check
9. Code the core flows
10. Edge cases and concurrency
11. Extensions and trade-offs

#	Step	Questions to ask yourself	Output	Core step
1	**Clarify**	Who uses it? What exactly is "a parking lot" here? Single location or many?	Shared understanding	Requirements
2	**Requirements**	What must it do (functional)? How well (non-functional: thread-safe, extensible, in-memory, latency, i.e. response time)? What is out of scope?	8–12 numbered lines + out-of-scope list	Requirements
3	**Use cases**	For each actor, what can they do?	Actor → use case list (or use case diagram)	Actors & use cases
4	**Entities**	Which nouns hold state or behaviour? Which are just values (fields/enums)?	Class list with one-sentence responsibilities	Classes
5	**Relationships**	Uses, has, owns, or is-a? How many (1, 0..1, *)?	Relationship list	Connect
6	**Class diagram**	Fields, key methods, interfaces, enums	UML class diagram	Connect
7	**Key flows**	What happens step by step for the 2–3 main use cases? What states does the core object go through?	Sequence / activity / state diagram (states and the moves between them)	Connect
8	**Patterns & SOLID**	Is there a rule that varies (Strategy)? A lifecycle (fixed states an object moves through: State)? Many listeners (Observer)? Any class with two reasons to change?	Named patterns + rejected alternatives	Check
9	**Code**	Interfaces, enums, models, services, a driver `main`	Running code for main flows	Code
10	**Edge cases & concurrency**	Invalid input? Full capacity? Two users grabbing the same seat?	Validation, exceptions, locks	Code
11	**Extensions**	What would change if requirement X is added?	Short answers, maybe one extra class	Wrap-up

(Row 10 in plain words: an **exception** is Java's way of signalling an error that stops the normal flow, and a **lock** lets only one thread at a time run a protected piece of code.)

Trade-offs

Do this	Instead of this	Why
Spend 8–13 minutes on steps 1–3	Coding at minute 2	Unbounded scope wastes far more time later
Code 2–3 flows fully	Stub 12 classes (empty placeholders)	Interviewers judge depth on the chosen part
Name one or two patterns where the signal appears	Apply six patterns	Over-engineering (building more flexibility than the problem needs) is scored as poor judgement
Pass time into methods (`nowMinute`) and hold money as `BigDecimal` (Java's exact decimal type; or whole cents in a `long`)	Call `System.currentTimeMillis()` inside the logic, and use `double` (binary floating-point number) for money	Deterministic (same input, same output) and testable; `double` cannot represent values like 0.10 exactly, so sums drift

When not to use it (rigidly)

If the interviewer steers ("skip requirements, I care about the class model"), follow them. The framework is a default order, not a contract. In a machine-coding round, steps 5–7 are often done in

your head or as a short comment; the code **is** the diagram.

TL;DR

Clarify → requirements → use cases → entities → relationships → class diagram → key flows → patterns/SOLID → code → edge cases/concurrency → extensions. Say each step out loud as you enter it.

1.5 From one line to a list of requirements

What it is

Each **requirement** states a single capability, phrased so you can test whether the design delivers it. "Customers can pay at the exit" qualifies. "The parking lot should be good" does not.

- **Functional requirement** — a behaviour: "a customer takes a ticket at an entry panel".
- **Non-functional requirement** — a quality: "two entry panels must never assign the same spot" (thread safety), "adding a new vehicle type must not change existing classes" (extensibility), "data kept in memory" (storage).

Analogy: an architect never starts drawing before asking how many people will live in the house and whether they need a garage.

The problem it solves

This is not an interview trick: real products start the same way, with a short request someone has to turn into a specification. "Design a parking lot" is four words. The expected design has about twelve classes. Everything in between is yours to decide. Without a written list:

- **The design has no edges.** You keep adding classes (season tickets, number-plate recognition) and write no code.
- **You cannot defend a choice.** Asked "why is there no `Reservation` class?", the only honest answer is "I did not think of it". With a list, the answer is "reservations were out of scope; we agreed that at minute four".
- **You cannot tell when you are finished.** A bounded list turns an open request into a task with an end.

How it works: the four questions

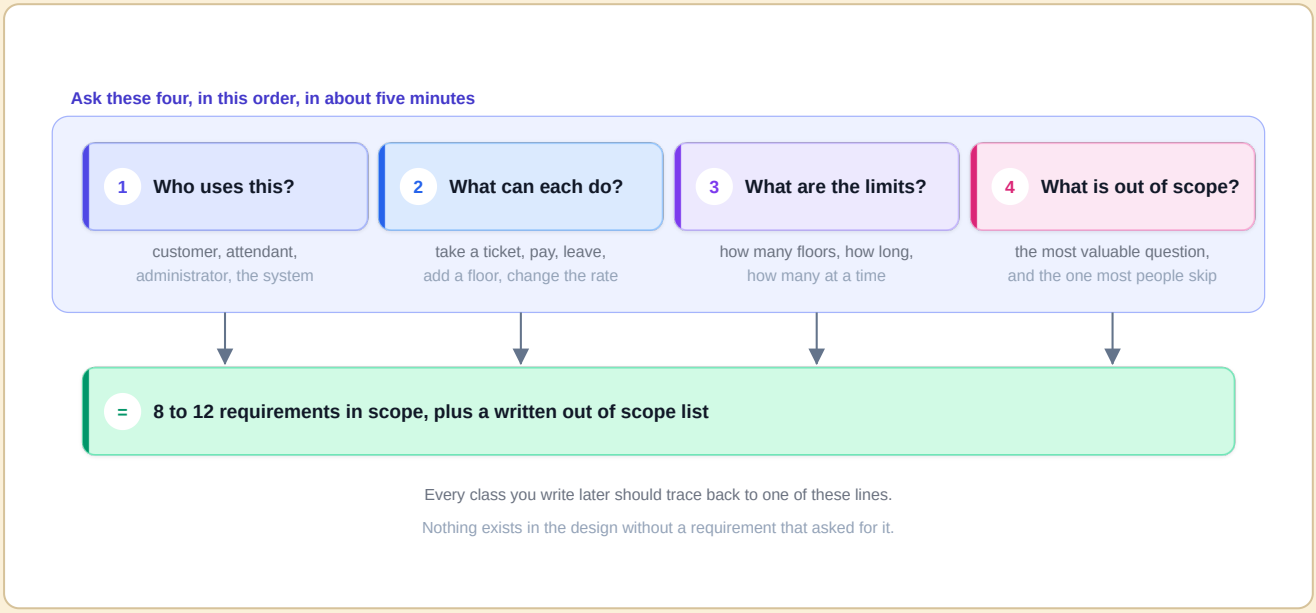


Figure — Four questions asked in order produce 8–12 in-scope requirements plus a written out-of-scope list (course diagram)

As the figure above shows, ask these in order; they take about five minutes and cover most products:

1. **Who uses this?** These are the **actors**. Parking lot: customer, attendant, administrator, and the system itself. Each brings requirements the others do not.
2. **What can each of them do?** One actor at a time, list actions. Each action is a **use case**. A customer takes a ticket, parks, pays, leaves. An admin adds floors and changes prices. Most of the list comes from here.
3. **What are the limits?** The numbers and rules: floor count, kinds of spot, maximum booking length, how many books one member can borrow. **Limits turn into constants and validation checks.** They are also the simplest requirements to gather.
4. **What is out of scope?** Hardly anyone asks this, yet it pays off most. It is the only one of the four questions that makes the problem **smaller**.

Scope questions worth asking in almost any problem:

- "Do payments belong to my design, or can I treat an external payment service as given?"
- "Is there one site, or do several sites share an account?"
- "Are users and permissions part of the model, or may I take a logged-in user for granted?"

Each answer adds or removes two or three classes, so ask early. If the interviewer says "your choice", treat it as permission, not a dodge: **choose, say it out loud, write it down**. An assumption you state clearly counts just like an answer you were given.

What good looks like: the parking lot in ten lines

In scope

1. The lot has multiple floors; each floor has many parking spots.
2. Spot types: compact, large, handicapped, motorbike, electric.
3. Vehicle types: car, van, truck, motorbike, electric car.
4. A customer takes a ticket at an entrance panel on arrival.
5. A customer pays at an exit panel, at a floor portal, or to an attendant.
6. Payment is by cash or card.
7. Charges are per hour, with the hourly rate depending on how long the stay lasts.
8. Each floor shows how many spots of each type are free.
9. The system refuses entry when no suitable spot is free.
10. Admins may add floors, spots and panels, and may update the rate.

Out of scope: season passes, licence-plate recognition, booking before arrival, refunds, reports across several sites.

Every line above becomes a class, a field, or a method. **If no line asked for it, it does not go in the design.**

Make each requirement checkable

- **Put a number on it.** Rather than "a member can borrow a few books", write "a member may have **no more than five** books out at once". That number turns into a constant, and the constant turns into a check inside one method.
- **Name the actor.** Not "books can be reserved" but "**a member** can reserve a copy that is currently on loan" (you reserve a copy that is out so you get it next). Now you know which class the method belongs to.

A line written like this is close to a test case: you can point at one method and say "requirement 5 lives here".

Use cases

A **use case** is one goal an actor achieves with the system ("customer pays for parking"). A **use case diagram** (Chapter 3) draws actors as stick figures and use cases as ovals. In a 45-minute round a bulleted list is enough:

Actor	Use cases
Customer	take ticket, park, pay (cash/card), exit
Attendant	collect cash payment, help customer
Admin	add floor, add spot, add panel, change rate
System	assign spot, update display boards, refuse entry when full

Common mistakes

- **Saying yes to every idea.** Reservations come up and you accept. Multiple sites, accepted too. Twenty minutes on, you have 22 requirements and nothing designed. Declining is allowed: "Let me park reservations for now and get ticketing and payment working end to end first; I'll come back to it if time allows." Interviewers see that as a **strong** move, because it proves you can protect a fixed time budget from an open-ended ask.
- Forgetting non-functional requirements (concurrency, extensibility).
- Writing vague lines with no number and no actor.

In the interview

> **30-second answer** (to "how do you start an LLD question?"): "I ask four questions: who uses it, what each actor does, what the limits are, and what is out of scope. I write 8 to 12 numbered requirements and an out-of-scope list, confirm them with you, and then every class I create must trace back to one of those lines."

Follow-ups:

- *"Why not just start designing?"* — Without a boundary the design grows forever and I cannot defend omissions. Five minutes here saves fifteen later.
- *"What if I say 'your choice'?"* — I pick the simpler option, say it out loud, and write it as an assumption, so the decision is visible and reversible.

Self-check

- **Q:** Which of the four questions makes the problem smaller? **A:** "What is out of scope?"
- **Q:** Rewrite "users can book some seats" as a checkable requirement. **A:** "A customer can hold at most 10 seats for one show; held seats are released after 10 minutes if unpaid."
- **Q:** Where do limits end up in code? **A:** As constants plus a validation check in the method that owns the data.

TL;DR

Four questions (who, what, limits, out of scope) → 8–12 checkable lines with numbers and actors, plus an out-of-scope list. Saying "no" to extra scope is a strength.

1.6 Finding the classes

What it is

A reliable ten-minute method to go from requirements to a class list: **collect candidates, cut the ones that are not classes, give each survivor one job.**

How it works

1. **Underline the nouns** in the requirements. For the parking lot they fall into rough groups. Structure: parking lot, floor, spot, compact spot, large spot, entrance panel, exit panel, display board, portal. Vehicles: vehicle, car, van, truck, motorbike. People: customer, attendant, administrator. Money and time: ticket, payment, cash, card, rate, hour. That is deliberately too

many: treat them as **candidates**. Since every one was taken from the requirements, none of them is made up.

2. **Underline the verbs.** Park, take a ticket, pay, leave, refuse entry, show free spots, add a floor, change the rate. **Each verb turns into a method** and has to land on some class. A verb with no home means a class is missing.
3. **Cut candidates with five tests:**

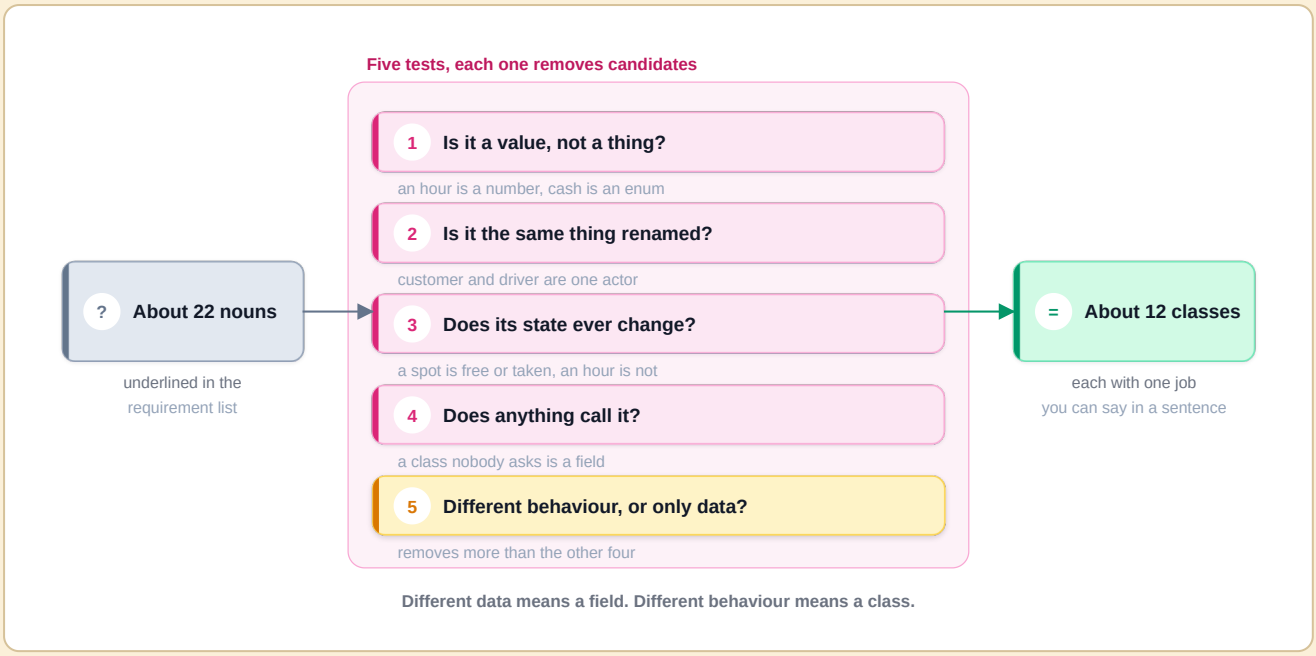


Figure — About 22 nouns pass through five tests and about 12 classes survive, each with one job (course diagram)

#	Test	Example	Result
1	Is it a **value** rather than a thing?	An hour is just a number, a licence plate just a string, and cash or card are options of a payment type	Field or **enum** constant (an enum is a fixed set of named values)
2	Is it just **another name** for a candidate already listed?	Customer and driver describe one actor; portal and panel could be a single class installed in several places	Merge, pick one name
3	Does it have **state that changes**?	A spot is free or occupied; an hour never changes	Changing state → class
4	Does anything **call** it?	Nobody ever asks an `Hour` object a question	Never called → it is a value
5	Does it differ in **behaviour** or only in **data**?	Compact vs large spot differ only in which vehicle fits; cash vs card payment behave differently	**Different data → a field. Different behaviour → a class.**

Applied to the parking-lot nouns, tests one to four cut the list to about fifteen; test five brings it to roughly twelve, removing more than the other four combined. Compact, large and handicapped spots all hold a vehicle, report free or occupied, and carry a number; they differ only in which vehicle kind is allowed. A difference like that is **data**, so a single `ParkingSpot` class gets a `type` field. Paying cash means checking enough money was given; paying by card means calling a provider and waiting for its reply. That difference is **behaviour**, so it earns two classes. Watch for this smell (a surface sign that something is probably wrong) in a diagram: five subclasses (classes that extend another class, their parent) whose only content is handing a different constant up to the parent. They are a single field pretending to be classes.

4. **Give each class one responsibility**, in one sentence with no "and":

Class	Responsibility
`ParkingLot`	Issues tickets and can tell when no space is left
`ParkingFloor`	Owns a level's spots and locates an empty one
`ParkingSpot`	Tracks if it is occupied and by which vehicle
`Vehicle`	Carries its plate number and the spot types it can use
`ParkingTicket`	Notes the vehicle, the spot it took, and the entry time
`ParkingRate`	Converts a duration in hours into a price
`Payment`	Collects the money and says if that worked
`ParkingDisplayBoard`	Shows how many spots of each type are free

`ParkingLot`'s sentence admits two jobs (issuing tickets **and** knowing fullness/assigning spots). In a longer design you would split it into a spot-assignment part and a ticketing part. In 45 minutes, **naming the tension out loud is enough**.

5. **Put each method where the data is.** "Find a free spot" touches the spot collection, which the floor owns → `ParkingFloor`. "Price the stay" touches the rate card → `ParkingRate`. "Take a ticket" touches the ticket store → `ParkingLot`. If a method needs data from three classes, it is probably in the wrong place; move it to the class that owns most of what it touches. (This is the GRASP "Information Expert" idea and the cure for the *Feature Envy* smell, Chapters 5 and 7. GRASP is a set of guidelines for deciding which class gets which job; *Feature Envy* is a method that uses another class's data more than its own.)

Common mistakes

- One subclass per value (`CompactSpot`, `LargeSpot` ... with no behaviour).
- Missing the **record-keeping** class (ticket, loan, booking) and the **rule** class (rate, strategy). These two are the classes beginners most often forget.
- "Manager" god classes that do everything (a *god class* is one huge class that owns most of the program's logic).

Self-check

- **Q:** Car, van and truck: subclasses or enum? **A:** If they differ only in size/which spots fit, an enum `VehicleType` (or a field). If they have genuinely different behaviour, subclasses.
- **Q:** A verb has no class to live on. What does that tell you? **A:** A class is missing.
- **Q:** A class has fields but no verbs. What is it? **A:** Probably a value (record/field), not a full class.

TL;DR

Nouns → candidate classes, verbs → methods. Cut values, duplicates, stateless and never-called nouns. **Different data = field; different behaviour = class.** One sentence per class, method goes where its data is. Twelve classes with one clear job each beat twenty that overlap.

1.7 Connecting the classes (preview)

Once you have the classes, decide for each pair: does one **use** the other briefly (dependency), **hold** a reference (association), **contain** parts that can live alone (aggregation), **own** parts that die with it (composition), or **is** it a kind of the other (inheritance, where a class extends another and reuses its code / realization, where a class implements an interface)? Then add **multiplicity** (1, 0..1, 1..*, *). Chapter 3 teaches every relationship and UML symbol in detail. For now, remember the most common beginner error: using **inheritance** where **composition** is right. The test is the sentence "A **is a** B". A car is not an engine, so `Car` has an `Engine`.

1.8 Keeping the design honest with SOLID

What it is

SOLID is five rules (one per letter) that catch the common ways a class diagram goes wrong. Chapter 4 teaches them in depth. In the interview, use them as **five quick checks** run over the diagram you just drew, in about five minutes.

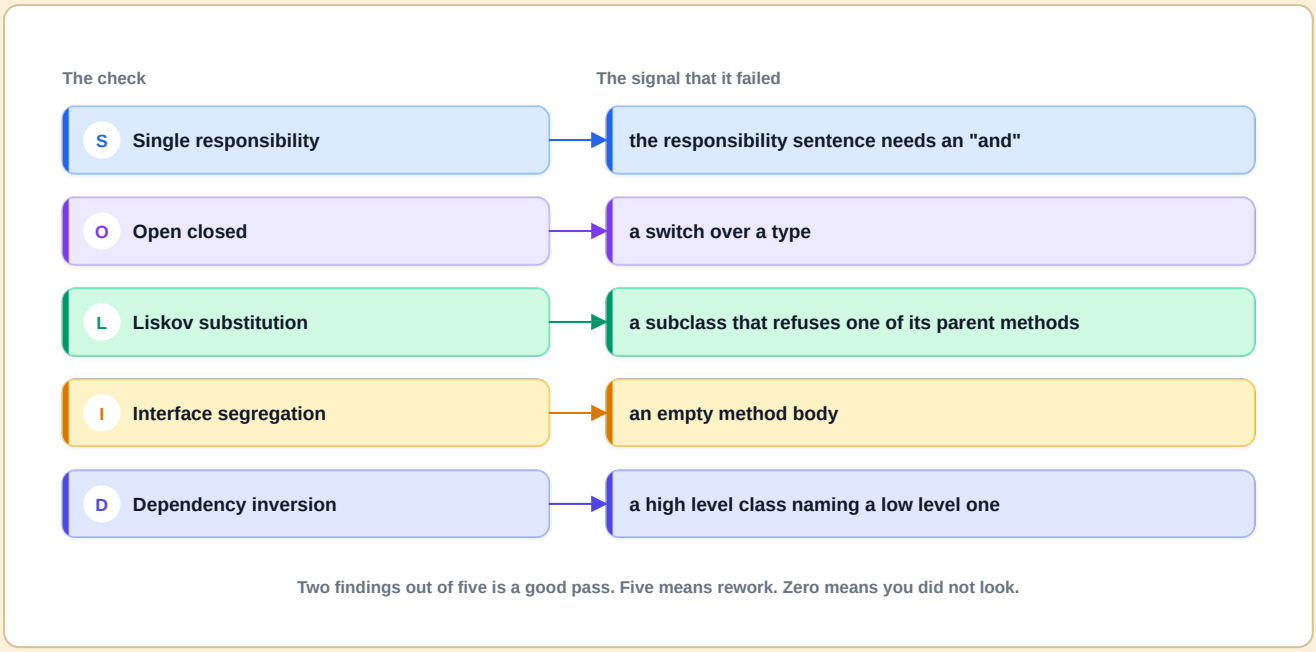


Figure — The five SOLID checks and the signal that each one has failed (course diagram)

How it works: run the five checks in this order

Check	Rule in one line	Signal that it failed (from the figure above)	Quick check
S — Single responsibility	A class should have one reason to change	The responsibility sentence needs an "and"	Name the change that would force an edit; two unrelated changes → split
O — Open/closed	Add behaviour without editing existing code	A `switch` / `if` chain over a type	Pick one likely new feature; if it edits three classes, the design is closed to the wrong thing
L — Liskov substitution	A subclass must work anywhere its parent works	A subclass that throws, does nothing, or refuses a parent method	Read each inheritance arrow: can the child be handed to code expecting the parent?
I — Interface segregation	No class should implement methods it does not use	Empty method bodies	Split the interface until every implementer uses everything
D — Dependency inversion	Depend on an interface, not a concrete class	A high-level class constructs or names a low-level one	Could you test the class without the real database/provider?

(In the **D** row, **high-level** code holds the business rules, while **low-level** code does one technical job such as talking to a database or a payment provider.)

Examples from the parking lot:

- **S:** a `ParkingLot` that both assigns spots and prints receipts changes when pricing changes **and** when the layout changes → two classes.
- **O:** pricing written as a switch over rate types means a weekend rate edits the class that prices everything. With pricing behind a small interface, the weekend rate is a new class.

- **L:** re-read every inheritance arrow at the end, because arrows are often added late when time is short. A `ReadOnlySpot extends ParkingSpot` whose `occupy` throws would fail the check: code that expects any spot to accept a vehicle breaks when handed that child.
- **I:** a library `Search` interface with four methods is fine if the catalog truly implements all four; it is not fine if half the implementers throw on two of them.
- **D:** a `ParkingLot` that does `new CreditCardTransaction()` cannot be tested without a payment provider and cannot take cash without an edit. Make it depend on `Payment`, and let something else decide which kind arrives.

The two that earn the most credit

- **Single responsibility** improves your class list: twelve well-named classes beat six classes doing three things each.
- **Open/closed** is what interviewers probe, usually as a late new requirement: "What if trucks are charged differently?" Answering "add a class" beats "add a branch to this switch".

When not to use it

- **Do not add an interface for everything.** An interface with one implementer, added "because SOLID", is indirection (an extra layer between the caller and the real code) with no benefit. Add it when a second implementation exists or is clearly coming (or when you need a test double, a fake stand-in object used in tests).
- **Do not split beyond what a reader can follow.** Thirty two-method classes are as bad as one god class. The one-sentence test is the boundary.

Each rule targets a **specific problem**. Apply it where your design shows that problem; applying it where the problem is absent only adds cost.

A five-minute pass over the parking lot

Check	Finding
S	`ParkingLot` issues tickets and finds spots: two reasons to change. Split if time allows; otherwise say it.
O	Pricing sits behind `ParkingRate`, so a new rate is a new class. Passes.
L (re-reading the inheritance arrows)	Five spot subclasses add no behaviour. Strictly this is not a Liskov violation (each would substitute fine); it is the "different data → field" test from 1.6 caught a second time while reading the arrows. Collapse them into a `type` field.
I	No wide interfaces. Passes.
D	The lot depends on `Payment`, not on the card class. Passes.

Two real findings, each explainable in a sentence: that is a good pass. **Five findings** usually means the design needs rework; **zero** usually means you did not really look.

TL;DR

Run S-O-L-I-D as five two-minute checks on your own diagram before the interviewer does. S and O earn the most credit. Do not add single-implementer interfaces or over-split.

1.9 The design patterns that come up in this round

What it is

A **design pattern** is a named solution to a design problem that keeps recurring. There are dozens (the Gang of Four book catalogues 23; Chapters 8–11 cover all of them). **Six** appear in this round often enough to know by name. The name turns a paragraph into two words and tells the interviewer you have

seen the problem before. **A misapplied pattern hurts you more than an unknown one**, so learn each pattern together with the **signal** that says it applies.

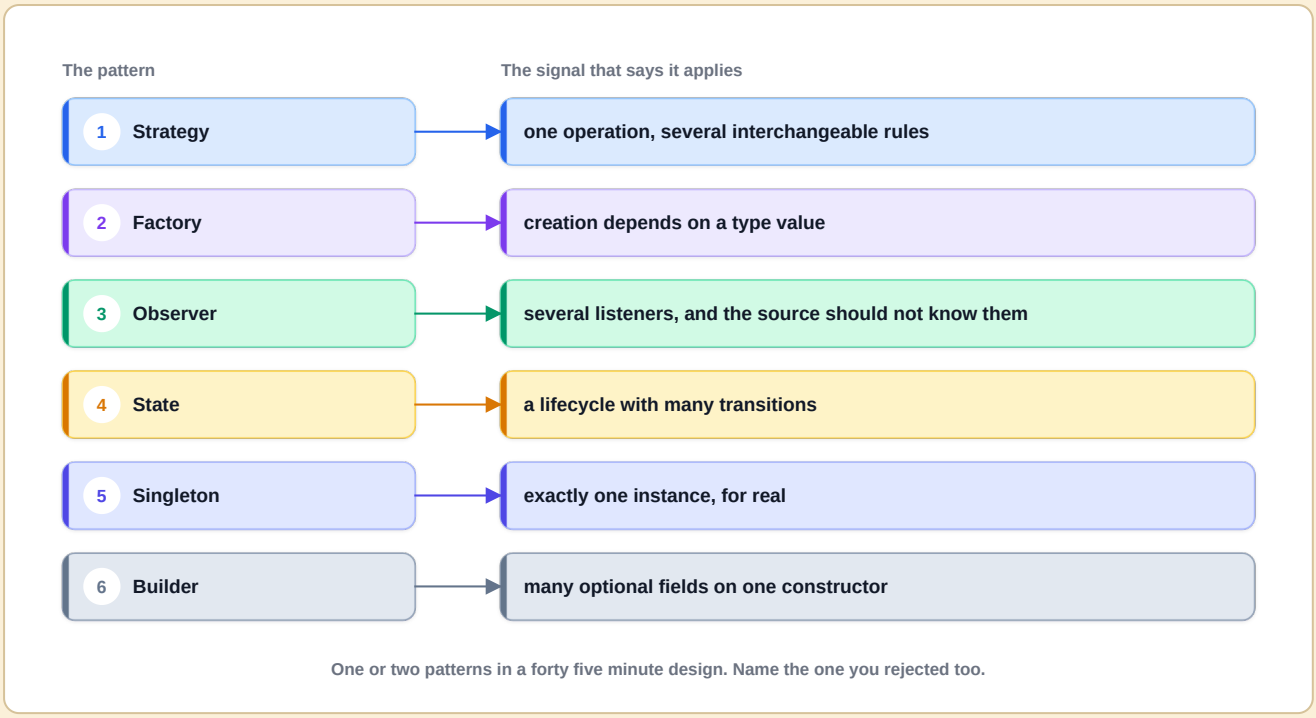


Figure — The six patterns most used in this round and the signal for each (course diagram)

Pattern	Signal (from the figure above)	Typical place	Parking lot / library example
Strategy	One operation, several interchangeable rules chosen at run time (while the program runs)	Pricing, matching, sorting, ranking, routing	`ParkingRate` interface with `HourlyRate`, `WeekendRate`, `MonthlyPass`. Answers "what if the rules change?", the most common late question
Factory	Creation depends on a type value; caller should not know the concrete class	Vehicles, spots, payments, notifications	A factory turns a type/enum into a `Vehicle`, so the entrance panel does not know every vehicle class. Keep it small: a factory **method** is usually enough
Observer	One thing changes, several others must know, and the source should not know them	Display boards, notifications, live updates, event logs	When a spot fills, all boards/panels registered on it update; adding a board changes nothing in the spot. With only **one** listener, a direct call is enough (e.g. notifying the member who reserved a returned book)
State	Behaviour depends on the current state, with a fixed set of transitions	Tickets, orders, bookings, loans, vending machines	Ticket: active → paid → closed. Book copy: available, reserved, loaned, lost. Each state class knows what is allowed, so illegal transitions become impossible. For 3–4 states, an **enum + one guarded method** is simpler: say so
Singleton	Exactly one instance must exist	The top-level object (`ParkingLot`, `Library`)	Private constructor + one accessor. Say out loud: it must be **thread-safe** (lock or eager creation, i.e. built when the class loads), and it is **global state** (data any code anywhere can reach and change), which hurts testing; many teams inject one shared instance instead
Builder	Many optional fields; the constructor is getting long	A book copy, a booking, a search query	`BookItem` with barcode, price, format, dates, rack: eight constructor args are easy to swap by mistake (two strings side by side compile fine). A builder sets fields **by name**. It is a readability fix, not a structural one

What to say about each one

- **Strategy:** without it, the pricing method grows a `switch` and every new rate edits it. With it, the lot holds one rate object and calls it; a new rate is a new class.
- **Factory:** it goes hand in hand with open/closed, since registering a new type happens in **a single spot**: the factory. Giving every product type its own factory class is heavier than a 45-minute design calls for.
- **Observer:** the simple version has the floor recompute and push to its single board. With Observer the **spot is the subject** and boards, entrance panels and floor portals register as observers; the spot never learns who is listening.
- **State:** without it, every method starts by checking the current status and the legal transitions are scattered across the class. With it, each state class knows what is allowed from there, so illegal moves cannot be expressed. Choose it only when the transition table is large, and say why.
- **Singleton:** expect the same two follow-ups (thread safety and testability) nearly every time; being able to state that trade-off counts for more than knowing the pattern.
- **Builder:** mention it when your own constructor reaches about six arguments, then move on.

Choosing patterns without overusing them

Pattern	Reach for it if	Skip it if
Strategy	A single operation has several swappable rules	Only one rule exists and a second is not in sight
Factory	Which object to build hinges on a type value	Just one concrete class exists
Observer	Several listeners; source should not know them	One listener the source already holds
State	The lifecycle has lots of transitions	A single guarded method covers three states
Singleton	There truly must be one instance	The real goal is just easy global access
Builder	Many optional fields	Three required fields

One or two patterns is the right number in a 45-minute design. Six patterns in one answer usually means the design is driven by patterns rather than by the requirements.

Nothing lands better than a **comparison**: "The rate gets Strategy because a second rate is likely. The ticket does not get State, since one guard is enough for three statuses." It proves you understand what the pattern gives and what it costs.

Real-world examples

- **Strategy:** `java.util.Comparator` passed to `List.sort`; Spring Security's pluggable `PasswordEncoder` implementations.
- **Factory** (static factory methods that hide the concrete class returned): `java.util.concurrent.Executors.newFixedThreadPool(...)`, `List.of(...)`, `Calendar.getInstance()`.
- **Observer:** Swing/AWT listeners (`ActionListener`), `java.beans.PropertyChangeListener`, Spring `ApplicationEvent` listeners.
- **State:** order lifecycle in e-commerce (placed → paid → shipped → delivered); TCP connection states.
- **Singleton:** `java.lang.Runtime.getRuntime()`; Spring beans are singleton-scoped **per container** by default (not the GoF class-enforced singleton).
- **Builder:** `StringBuilder`, `java.net.http.HttpRequest.newBuilder()`, `Stream.builder()`.

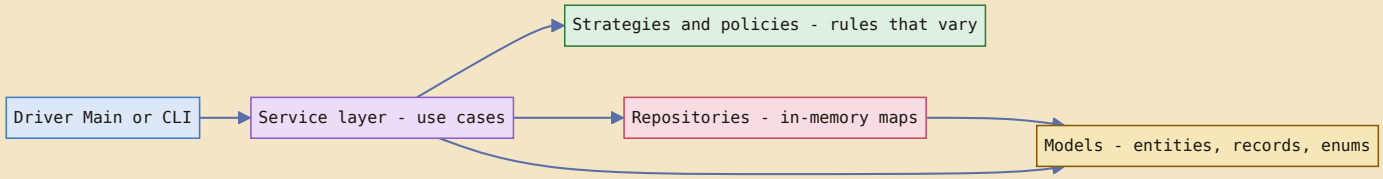
(Names in this list: **Spring** is a popular Java framework whose *container* creates and wires your application objects, called *beans*; **Swing/AWT** are Java's older desktop user-interface toolkits; **TCP** is the network protocol that keeps a reliable connection between two machines, see System Design Fundamentals; a **thread pool** is a fixed set of reusable worker threads.)

TL;DR

Know six by name: Strategy, Factory, Observer, State, Singleton, Builder. Use one or two where their signal appears, and name the one you rejected.

1.10 Code, edge cases and concurrency: how to write interview code

How it works: a machine-coding skeleton that scales



1. **Models** — entities (`ParkingSpot`; objects with an identity whose state changes over time), value objects (`record Vehicle`; plain data compared by its values), enums (`VehicleType`). Encapsulate state; validate in constructors.
2. **Repositories** — classes whose only job is storing and finding objects: an in-memory `Map<Id, Entity>` behind a small interface, so a database could replace it later.
3. **Strategies / policies** — every rule that might vary (pricing, spot selection, splitting a bill).
4. **Services** — the use cases (`park`, `exit`, `bookSeats`). This is the **service interface** a client calls.
5. **Driver** — a `Main` that runs a scripted scenario (or reads commands) and prints clear output for the demo.

Habits that interviewers notice:

- **Pass time in** (`long nowMinute`, `Instant now`, where `Instant` is Java's type for a point in time) rather than reading the clock inside the method: same input → same output → testable.
- **Hold money as** `BigDecimal` (built from strings such as `new BigDecimal("3.50")`) or as whole cents in a `long`; never `double`. This handbook uses `BigDecimal` in domain code.
- **Fail loudly on bad input**: `IllegalArgumentException` for bad arguments, `IllegalStateException` for wrong state, custom exceptions (`SpotNotAvailableException`) where they clarify.
- **Return** `Optional` (a box that either holds a value or is empty, used instead of returning `null`) when "not found" is a normal outcome.
- **Keep fields** `private final` unless they must change.
- **Name things after the domain** (`ParkingTicket`, not `Data2`).

Edge cases and concurrency checklist

Category	Questions
Input	null (no object at all), empty, negative, duplicate IDs, unknown IDs
Capacity	Full lot, empty playlist, zero seats left, max loans reached
State	Paying twice, exiting without entering, cancelling a delivered order
Time	Exit before entry, bookings expiring, part hours (round up?)
Concurrency	Two threads booking the same seat or taking the same spot: use `synchronized`, a `ReentrantLock`, or atomic structures such as `ConcurrentHashMap.computeIfAbsent`; lock the **smallest** shared thing that keeps the invariant

> **Key term — threads, locks and atomicity:** A **thread** is one independent line of execution; a server handling two panels at once runs two threads over the same objects. `synchronized` and `ReentrantLock` are Java's ways to let only one thread at a time into a block of code. An **atomic** operation happens all at once from other threads' point of view, with no half-finished moment they can see; `ConcurrentHashMap` is a thread-safe map that offers such operations. An **invariant** is a rule about the data that must always hold, such as "one spot holds at most one vehicle".

Chapters 12 and 13 cover concurrency in depth.

TL;DR

Models → repositories → strategies → services → driver. Inject time, use `BigDecimal` (or cents) for money, validate, return `Optional`, guard shared state.

1.11 Worked mini-example, end to end: a small parking lot

This runs every step of the framework on a deliberately small version of the classic question, so you can see the whole shape once. (A full parking lot with multiple panels, payments and display boards appears in the case-study chapters.)

Problem & clarifying questions

"Design a parking lot." Questions asked, with the answers we assume:

Question	Answer
Single lot or many?	Single lot, one or more floors
Which vehicles?	Motorbike, car, truck
Which spot types?	Motorbike, compact, large; a smaller vehicle may use a bigger spot
Payment?	Out of scope: we compute the fee; a payment service collects it
Pricing?	Hourly, tiered, part hours round up; trucks pay double; must be swappable (weekend flat rate)
Concurrency?	Several entry/exit panels may act at the same time

Requirements

Functional

1. The lot has floors; each floor has spots of type MOTORBIKE, COMPACT or LARGE.
2. A motorbike fits MOTORBIKE or COMPACT; a car fits COMPACT or LARGE; a truck fits LARGE only. The smallest fitting spot is tried first.
3. On entry, the system assigns a free fitting spot and issues a ticket (id, vehicle, spot, entry time).
4. If no fitting spot is free, entry is refused.
5. On exit, the ticket is closed, the spot freed and the fee returned.
6. Fee: $4.00 first hour, $3.50 for hours 2–3, $2.50 per hour after; part hours round up; minimum one hour; trucks pay double.
7. The pricing rule can be swapped at run time (e.g. a flat $10 weekend rate).
8. A floor can report how many spots of a type are free.

Non-functional: in-memory; thread-safe entry/exit; new pricing rules or vehicle types must not require editing existing services; deterministic (time passed in).

Out of scope: payment collection, reservations, display boards, number-plate recognition, multiple lots.

Use cases

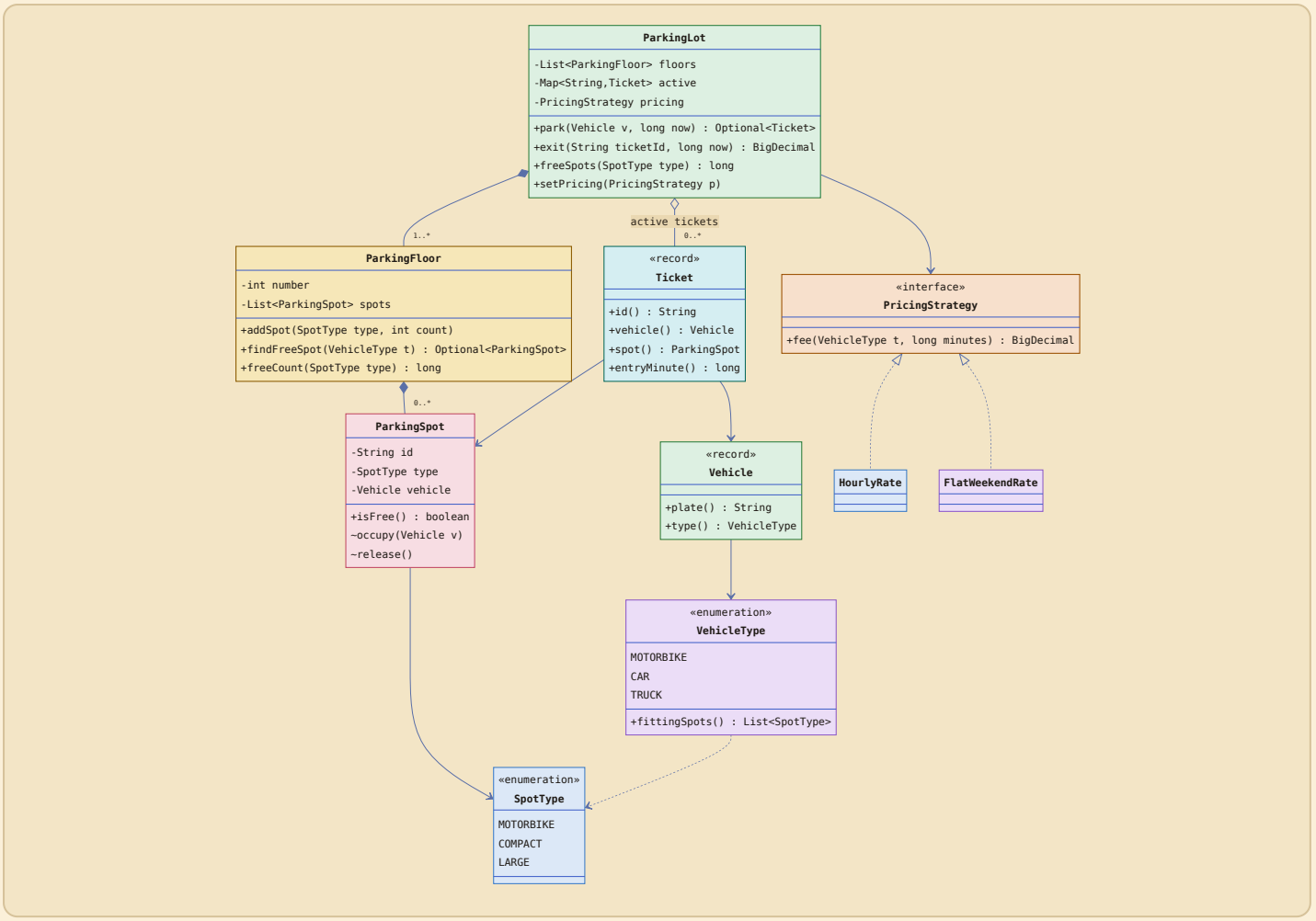
Actor	Use cases
Driver (customer)	enter and get ticket, exit and see fee
Admin	add floor/spots, change pricing rule
System	find free spot, refuse entry when full, compute fee

Core entities

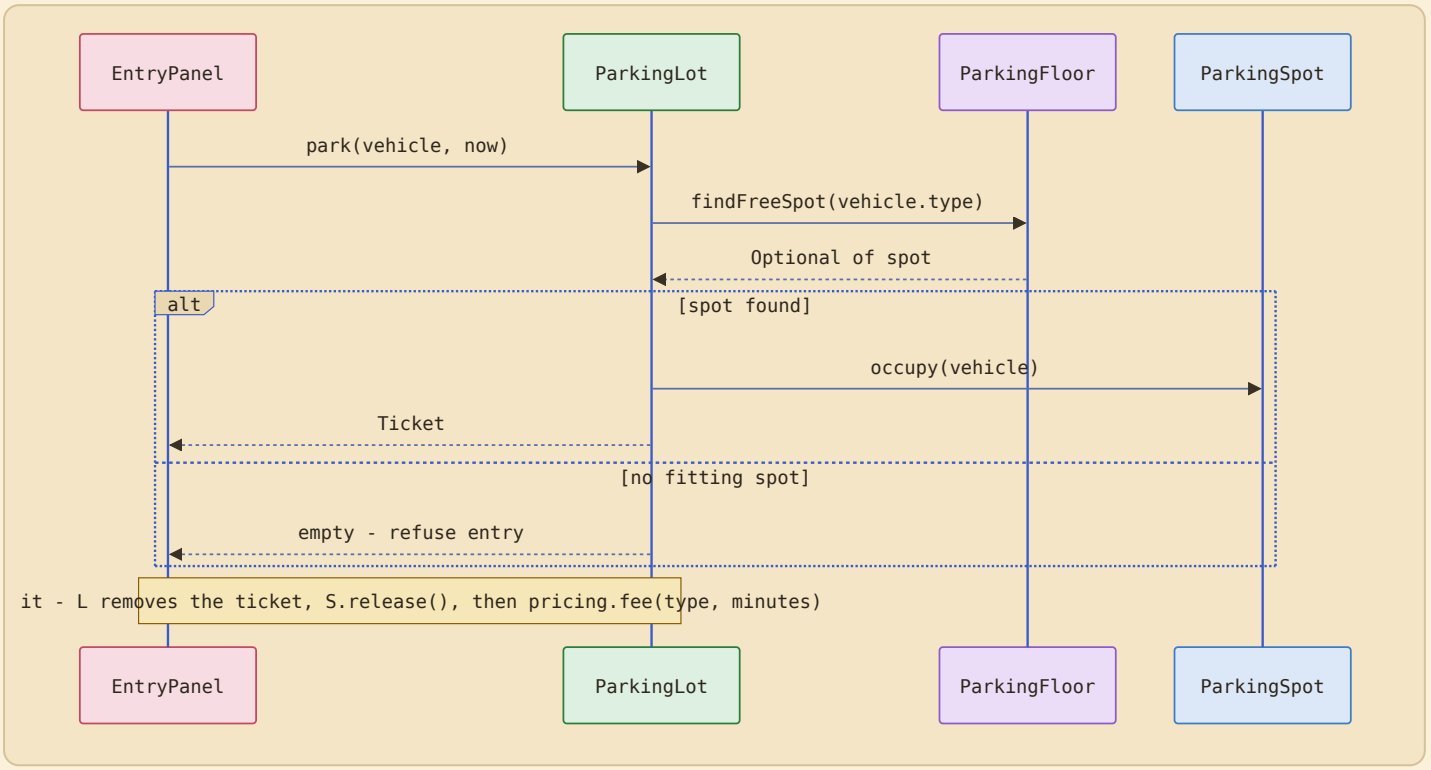
Class	Kind	Responsibility
`SpotType`	enum	The spot sizes
`VehicleType`	enum	Vehicle kinds, each knowing which spot types fit (smallest first)
`Vehicle`	record	Plate + type (pure data)
`ParkingSpot`	class	Tracks if it is occupied and by which vehicle
`ParkingFloor`	class	Holds spots on one floor and finds a free fitting one
`Ticket`	record	Records which vehicle went into which spot, and when (the "connection object")
`PricingStrategy`	interface	Turns a stay into a fee (`BigDecimal`)
`HourlyRate`, `FlatWeekendRate`	classes	Two pricing rules
`ParkingLot`	class	Issues and closes tickets; the service the panels call

Notice the **"different data → field"** rule: spot sizes are an enum, not three subclasses, because they do not behave differently.

Class diagram



Key flows



Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for pricing (`PricingStrategy`)	A second rule (weekend) is explicitly required; new rules = new classes (open/closed)	A `switch` over a rate enum inside `ParkingLot`
Spot and vehicle types as **enums**	They differ in data only	One subclass per spot type
`Ticket` as a **record**	A ticket is a fact that never changes after issue. (A record is only *shallowly* immutable: its fields cannot be reassigned, but the `ParkingSpot` it points to still changes state.)	Mutable ticket with setters (methods that change fields)
Money as `BigDecimal`	Exact decimal arithmetic; `3.50 + 3.50` is exactly `7.00`	`double` (binary fractions, rounding drift)
`synchronized` on `park` / `exit` / `freeSpots`	Two panels must never get the same spot; lot-wide lock is simple and correct for this size	Per-floor locks (better throughput, i.e. more requests handled per second; more complex; mention as an extension)
No Singleton for `ParkingLot`	Injecting one instance keeps it testable	Global `getInstance()`
No State pattern for tickets	Only active/closed; a map of active tickets is enough	State classes per ticket status

Java implementation

```java
// File: code/lld/ch01/parkinglot/FlatWeekendRate.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** A NEW rule added later: flat $10.00 per visit. Added as a class; no existing code was edited (open-closed). */
public final class FlatWeekendRate implements PricingStrategy {
    private static final BigDecimal FLAT = new BigDecimal("10.00");

    @Override public BigDecimal fee(VehicleType type, long minutesParked) { return FLAT; }
}
```

```java
// File: code/lld/ch01/parkinglot/HourlyRate.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** $4.00 for the first hour, $3.50 for hours 2 and 3, $2.50 per hour after that. Part hours round up. Trucks pay double. */
public final class HourlyRate implements PricingStrategy {
    private static final BigDecimal FIRST_HOUR = new BigDecimal("4.00");
    private static final BigDecimal HOURS_2_TO_3 = new BigDecimal("3.50");
    private static final BigDecimal LATER_HOURS = new BigDecimal("2.50");

    @Override
    public BigDecimal fee(VehicleType type, long minutesParked) {
        long hours = Math.max(1, (minutesParked + 59) / 60);    // ceiling, minimum one hour
        BigDecimal total = BigDecimal.ZERO;
        for (long h = 1; h <= hours; h++) {
            total = total.add(h == 1 ? FIRST_HOUR : (h <= 3 ? HOURS_2_TO_3 : LATER_HOURS));
        }
        return type == VehicleType.TRUCK ? total.multiply(BigDecimal.TWO) : total;
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingFloor.java
package lld.ch01.parkinglot;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Holds the spots on one floor and finds a free one. Not thread-safe on its own: ParkingLot calls it under its lock. */
public final class ParkingFloor {
    private final int number;
    private final List<ParkingSpot> spots = new ArrayList<>();

    public ParkingFloor(int number) { this.number = number; }

    public void addSpot(SpotType type, int count) {
        for (int i = 1; i <= count; i++) {
            spots.add(new ParkingSpot("F" + number + "-" + type.name().charAt(0) + i, type));
        }
    }

    /** First free spot, trying the vehicle's preferred (smallest) spot type first. */
    public Optional<ParkingSpot> findFreeSpot(VehicleType vehicleType) {
        for (SpotType wanted : vehicleType.fittingSpots()) {
            for (ParkingSpot s : spots) {
                if (s.type() == wanted && s.isFree()) return Optional.of(s);
            }
        }
        return Optional.empty();
    }

    public long freeCount(SpotType type) {
        return spots.stream().filter(s -> s.type() == type && s.isFree()).count();
    }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingLot.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/**
 * Issues tickets and closes them. Time is PASSED IN (not read from the clock) so runs are
 * repeatable. Every method that reads or changes spots, tickets or pricing is synchronized on
 * the lot, so two entrance panels cannot hand out the same spot.
 */
public final class ParkingLot {
    private final List<ParkingFloor> floors;
    private final Map<String, Ticket> active = new HashMap<>();
    private PricingStrategy pricing;
    private int nextTicket = 1;

    public ParkingLot(List<ParkingFloor> floors, PricingStrategy pricing) {
        this.floors = List.copyOf(floors);
        this.pricing = pricing;
    }

    public synchronized Optional<Ticket> park(Vehicle v, long nowMinute) {
        for (ParkingFloor floor : floors) {
            Optional<ParkingSpot> spot = floor.findFreeSpot(v.type());
            if (spot.isPresent()) {
                spot.get().occupy(v);
                Ticket t = new Ticket("T" + nextTicket++, v, spot.get(), nowMinute);
                active.put(t.id(), t);
                return Optional.of(t);
            }
        }
        return Optional.empty();              // lot full for this vehicle type -> refuse entry
    }

    /** Closes the ticket, frees the spot and returns the fee. */
    public synchronized BigDecimal exit(String ticketId, long nowMinute) {
        Ticket t = active.get(ticketId);
        if (t == null) throw new IllegalArgumentException("unknown or already closed ticket " + ticketId);
        if (nowMinute < t.entryMinute()) throw new IllegalArgumentException("exit before entry");
        active.remove(ticketId);               // validate first, then change state
        t.spot().release();
        return pricing.fee(t.vehicle().type(), nowMinute - t.entryMinute());
    }

    /** Free spots of one type across all floors, read under the same lock as park/exit. */
    public synchronized long freeSpots(SpotType type) {
        return floors.stream().mapToLong(f -> f.freeCount(type)).sum();
    }

    public synchronized void setPricing(PricingStrategy pricing) { this.pricing = pricing; }
}
```

```java
// File: code/lld/ch01/parkinglot/ParkingSpot.java
package lld.ch01.parkinglot;

/** Knows whether it is free and which vehicle is in it. */
public final class ParkingSpot {
    private final String id;
    private final SpotType type;
    private Vehicle vehicle;                // null = free

    public ParkingSpot(String id, SpotType type) { this.id = id; this.type = type; }

    public boolean isFree()      { return vehicle == null; }
    public SpotType type()       { return type; }
    public String id()           { return id; }

    void occupy(Vehicle v) {                // package-private: only the lot may change a spot
        if (!isFree()) throw new IllegalStateException(id + " is taken");
        vehicle = v;
    }
    void release() { vehicle = null; }
}
```

```java
// File: code/lld/ch01/parkinglot/PricingStrategy.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;

/** Strategy: turns a stay into an amount of money. Money is BigDecimal, never double (no binary rounding errors).
*/
public interface PricingStrategy {
    BigDecimal fee(VehicleType type, long minutesParked);
}
```

```java
// File: code/lld/ch01/parkinglot/SpotType.java
package lld.ch01.parkinglot;

/** Spot kinds differ only in data (which vehicles fit), so they are enum values, not subclasses. */
public enum SpotType { MOTORBIKE, COMPACT, LARGE }
```

```java
// File: code/lld/ch01/parkinglot/Ticket.java
package lld.ch01.parkinglot;

/** Records which vehicle went into which spot, and when. The "connection object" between Vehicle and Spot. */
public record Ticket(String id, Vehicle vehicle, ParkingSpot spot, long entryMinute) { }
```

```java
// File: code/lld/ch01/parkinglot/Vehicle.java
package lld.ch01.parkinglot;

/** A vehicle is pure data: its plate and its type. */
public record Vehicle(String plate, VehicleType type) { }
```

```java
// File: code/lld/ch01/parkinglot/VehicleType.java
package lld.ch01.parkinglot;

import java.util.List;

/** Each vehicle type knows which spot types fit it, smallest first. */
public enum VehicleType {
    MOTORBIKE(List.of(SpotType.MOTORBIKE, SpotType.COMPACT)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));

    private final List<SpotType> fits;

    VehicleType(List<SpotType> fits) { this.fits = fits; }

    public List<SpotType> fittingSpots() { return fits; }
}
```

```java
// File: code/lld/ch01/parkinglot/Main.java
package lld.ch01.parkinglot;

import java.math.BigDecimal;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        ParkingFloor ground = new ParkingFloor(0);
        ground.addSpot(SpotType.MOTORBIKE, 1);
        ground.addSpot(SpotType.COMPACT, 1);
        ground.addSpot(SpotType.LARGE, 1);
        ParkingLot lot = new ParkingLot(List.of(ground), new HourlyRate());

        Vehicle car   = new Vehicle("KA-01-1234", VehicleType.CAR);
        Vehicle truck = new Vehicle("KA-02-9999", VehicleType.TRUCK);
        Vehicle car2  = new Vehicle("KA-03-5555", VehicleType.CAR);

        Ticket t1 = lot.park(car, 0).orElseThrow();
        System.out.println(t1.id() + ": " + car.plate() + " -> spot " + t1.spot().id());
        Ticket t2 = lot.park(truck, 10).orElseThrow();
        System.out.println(t2.id() + ": " + truck.plate() + " -> spot " + t2.spot().id());
        System.out.println(car2.plate() + " -> " + lot.park(car2, 20).map(Ticket::id).orElse("refused, no suitable
spot"));

        System.out.println("free compact spots: " + lot.freeSpots(SpotType.COMPACT));
        System.out.println(t1.id() + " exits after 150 min, pays " + money(lot.exit(t1.id(), 150)));
        System.out.println(t2.id() + " exits after 50 min, pays " + money(lot.exit(t2.id(), 60)));

        lot.setPricing(new FlatWeekendRate());          // swap the strategy at run time
        Ticket t3 = lot.park(car2, 200).orElseThrow();
        System.out.println(t3.id() + ": " + car2.plate() + " -> spot " + t3.spot().id()
                + ", weekend exit pays " + money(lot.exit(t3.id(), 500)));

        try {
            lot.exit(t1.id(), 600);
        } catch (IllegalArgumentException e) {
            System.out.println("error: " + e.getMessage());
        }
    }

    private static String money(BigDecimal amount) { return "$" + amount.toPlainString(); }
}
```

Run: `java -cp /tmp/lld_01 lld.ch01.parkinglot.Main`

Output:

```
T1: KA-01-1234 -> spot F0-C1
T2: KA-02-9999 -> spot F0-L1
KA-03-5555 -> refused, no suitable spot
free compact spots: 0
T1 exits after 150 min, pays $11.00
T2 exits after 50 min, pays $8.00
T3: KA-03-5555 -> spot F0-C1, weekend exit pays $10.00
error: unknown or already closed ticket T1
```

Reading the output: the car took the compact spot, the truck the only large spot, so the third car was refused (requirement 4). 150 minutes = 3 hours = $4.00 + $3.50 + $3.50 = $11.00. The truck's 50 minutes round up to one hour, doubled = $8.00. After swapping the strategy at run time, the same service charges the flat $10. Closing a ticket twice is rejected.

Concurrency & edge cases

- `park` and `exit` are `synchronized` on the lot, so find-then-occupy is **atomic**: two panels cannot both see the same spot as free and both occupy it (a classic *check-then-act* race: a **race condition** is a bug whose result depends on which thread happens to run first).
- `freeSpots` is read under the **same** lock, so a display never sees a half-finished `park`. `ParkingFloor` itself is not thread-safe; it is only ever called through the lot.
- `exit` validates the ticket and the time **before** changing anything, so a rejected exit (for example exit before entry) leaves the ticket open.
- `ParkingSpot.occupy` also throws if the spot is taken: a second line of defence.
- Exit before entry, unknown ticket, and double exit all throw `IllegalArgumentException`.
- `occupy` / `release` are **package-private**, so code outside the package cannot change spot state directly.

Extensions & follow-up questions

Follow-up	Answer
"Add electric-vehicle spots with charging."	Add `ELECTRIC` to `SpotType`, add an `ELECTRIC_CAR` vehicle type whose fitting list starts with `ELECTRIC`. If charging has **behaviour** (meter, charging fee), then a `ChargingSpot` class or a charging component composed into the spot.
"Choose the nearest spot to the entrance instead of the first free one."	Extract a `SpotSelectionStrategy` interface (Strategy again), injected into `ParkingLot`.
"Update display boards when spots change."	Observer: spots/floors publish "spot freed/occupied" events; boards subscribe.
"Improve throughput with many panels."	Lock per floor (or per spot type) instead of the whole lot; or keep free spots in a `ConcurrentLinkedQueue` (a thread-safe queue) per type and `poll()` atomically.
"Add payment by cash or card."	`Payment` interface with `CashPayment`, `CardPayment`; `exit` returns the fee and a `PaymentService` collects it (dependency inversion).

In the interview

Spend about 5 minutes on the questions and requirement list above, 3 on actors, 8 on entities and the "enum vs subclass" decision, 8 on the diagram, and 15 on `park`, `exit` and `HourlyRate`. Close by naming Strategy (used), Singleton and State (rejected, with reasons), and the per-floor locking extension.

Self-check

- **Q:** Why is `SpotType` an enum and not a class hierarchy? **A:** Spot types differ only in data (which vehicles fit), not in behaviour.
- **Q:** What race does `synchronized` on `park` prevent? **A:** Two threads finding the same free spot and both occupying it (check-then-act).
- **Q:** How did adding a weekend rate respect open/closed? **A:** It was a new `PricingStrategy` class; no existing class was edited.

TL;DR

Small, complete, and honest: 8 requirements, 10 types, one Strategy, one lock, and a clear list of what was left out.

1.12 Communication tips

Do	Why
Think out loud, narrating the step you are in ("Now I'll list actors")	The interviewer scores your process, not only the result
Write things down (requirements, class list) where the interviewer can see them	Shared artefacts prevent misunderstandings
Confirm before moving on ("Does this scope look right?")	Cheap course corrections
State assumptions explicitly	An assumption said out loud counts like an answer from the interviewer
Offer trade-offs in pairs: "I chose X over Y because..."	Shows judgement
Welcome hints; treat the interviewer as a teammate	Collaboration is part of the evaluation
Say what you are leaving out and how you would add it	Turns a gap into a strength
Keep a clock in view; announce when you move to code	Prevents the "no code at minute 40" failure
Use domain names, not `Manager`, `Helper`, `Data`	Clear names are clear thinking
When stuck, go back to the requirements	Every class must trace back to a line

What **not** to do: go silent for minutes, argue with feedback, name `HashMap`s and threads in minute two (that is implementation, skipping analysis and design), or recite pattern definitions without a reason to use them.

1.13 How to use this handbook, and a study plan

Attempt first, then compare

There are two ways to work through case studies. The first is to **read** them. That is quick and pleasant but builds very little skill, because any finished design looks obvious after someone else has drawn it. The second is to **attempt each one yourself, then compare**. That is slower and less comfortable, and it is the approach that actually improves how you perform under pressure.

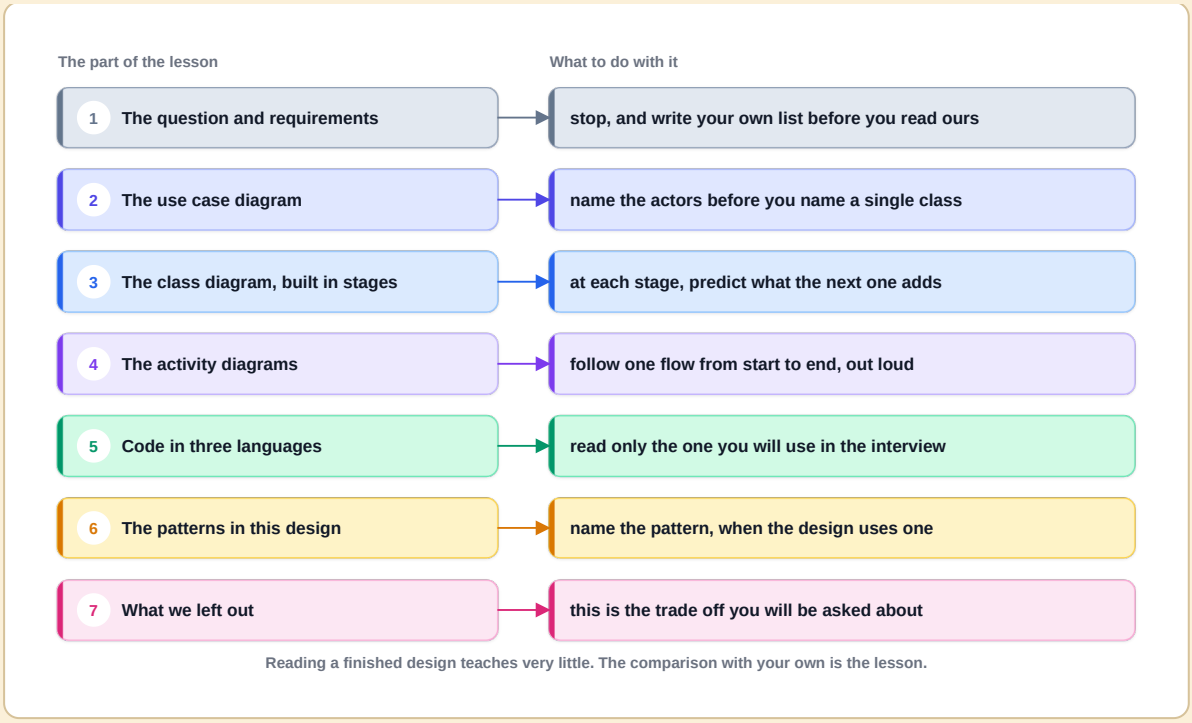


Figure — The parts of a case-study lesson and what to do at each one (course diagram)

As the figure above shows, every case study has the same parts, so you can decide in advance where to stop and think. Three stopping points matter most:

1. **Stop after the question.** Write your own requirement list (8–12 lines plus out of scope), then compare. If yours is shorter, the cause is usually not missing knowledge but **not asking enough questions**.
2. **Stop after the requirements.** Write your class list, one responsibility each, then read the class diagram. Most people are close but **two classes short**, and they are usually the same kind: **the thing that keeps a record** (ticket, loan, booking) and **the thing that decides a rule** (rate, strategy).
3. **Stop at each stage of the class diagram.** Predict what the next stage adds. A wrong guess is free, and each one shows you how an experienced designer thinks.

More habits:

- **Read the class diagram in stages.** Many case-study diagrams are built up step by step; you do not need to read the UML chapter first. Seeing a symbol inside a real design and then looking it up in Chapter 3 works well.
- **If a design decision looks wrong to you, write down why.** Arguing with a worked answer is good practice; several of these designs exist in their current form because someone disagreed with an earlier version.
- **Read one language's code** (here, Java). Reading multiple translations teaches nothing about design.
- **Do not skip the extensions / "what we left out" section.** It lists the simplifications and their trade-offs, and interviewers ask about exactly these. "I merged these two responsibilities to save time; here is how I would split them" is worth far more than hoping nobody asks.
- **Do not invent pattern names.** Where plain classes are enough, say so. Labelling a pattern the design does not use loses credibility.
- **After six or seven case studies, switch to timed runs**: a question you have already done, 45 minutes, blank page. You will not finish; nobody does. Measure two things: did you spend the first ten minutes on requirements and actors, and did you reach running code for at least one flow? These predict the real result better than the number of classes you drew.
- **Compile and run every example** (`bash tools/compile_all.sh` runs all chapters; `bash` is the Unix command shell and the `.sh` file is a script of commands).

Map of this handbook

Ch	Title	Use it for
1	The LLD interview & how to approach it	The method, time budget, study plan (this chapter)
2	OOP fundamentals	Classes, objects, the four pillars, interfaces, enums, equality, immutability, generics, OOA/OOD (object-oriented analysis / design)
3	Relationships & UML	Association, aggregation, composition, dependency; class, use case, activity, sequence, state diagrams
4	SOLID principles	The five principles in depth
5	Design principles	DRY (don't repeat yourself), KISS (keep it simple), YAGNI (you aren't gonna need it), composition over inheritance, Law of Demeter, GRASP and friends
6	Clean code	Naming, functions, comments, error handling, tests
7	Refactoring	Code smells and the named refactorings that fix them
8	Creational patterns	Singleton, Factory Method, Abstract Factory, Builder, Prototype
9	Structural patterns	Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy
10–11	Behavioural patterns	Strategy, Observer, State, Command, Template Method, Iterator, Chain of Responsibility, Mediator, Memento, Visitor, Interpreter; MVC (Model-View-Controller)
12	Concurrency concepts	Threads, locks, `java.util.concurrent`, thread-safety patterns
13	Concurrency problems	Producer–consumer, readers–writers, rate limiters and other classic problems
14	Case studies: everyday objects	Parking lot, vending machine, ATM, elevator, traffic signal, coffee machine
15–16	Case studies: booking and inventory	Library, movie tickets, concerts, hotels, car rental, restaurant, airline, course registration
17	Case studies: games	Chess, tic-tac-toe, snake and ladder, 2048, blackjack
18–19	Case studies: platforms	Social and Q&A sites, cricket scores, Amazon-style shopping, food delivery, auctions, stock brokerage and exchange, music streaming
20	Case studies: components	Caches (including LRU, which evicts the least recently used item), logger, rate limiter (caps how many requests are allowed per time window), pub-sub (publishers send messages to all subscribers of a topic), notifications, Splitwise, ride sharing and similar
21	Case studies: machine coding	90-minute style problems with runnable solutions: task manager, digital wallet, event calendar, coding contest platform, property listing, loan ledger, inventory
22	Cheat sheets & glossary	Last-day revision

Study plans

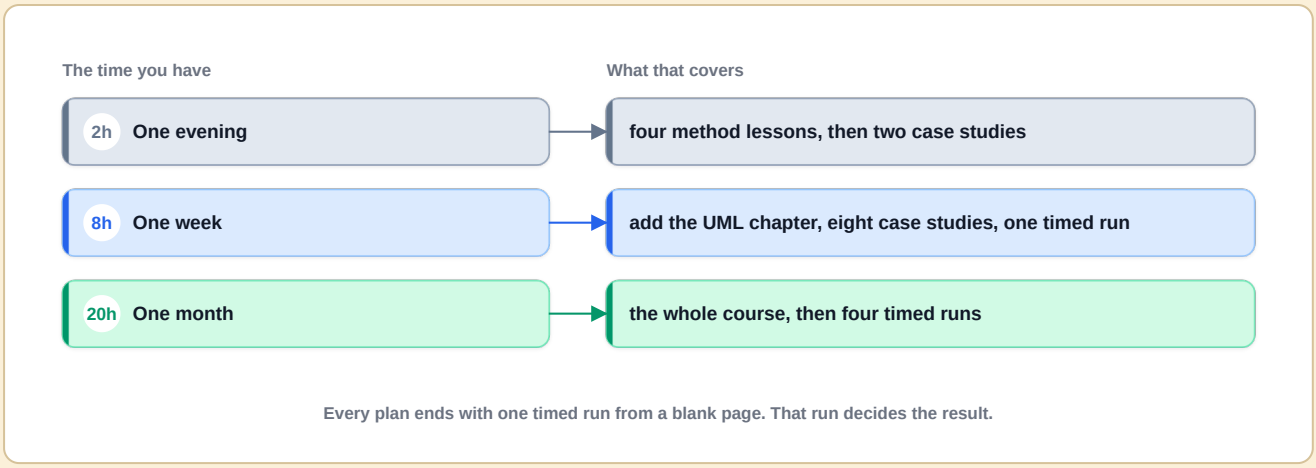


Figure — Three study plans for three amounts of time; every plan ends with one timed run from a blank page (course diagram)

Choose by the time you really have (your **calendar**), not the time you wish you had. The **method is worth more than any single case study**, and the first six case studies are worth more than the rest, because later ones repeat skills the earlier ones taught.

If the interview is tomorrow (about 2 hours)

Order	Read	Time
1	Section 1.2 (for the time budget)	10 min
2	Section 1.5 (requirements)	15 min
3	Section 1.6 (finding the classes)	15 min
4	Chapter 3: choosing the relationship between two classes	15 min
5	Chapter 3: class diagram notation	10 min
6	Parking lot case study, whole	30 min
7	Vending machine case study, whole	25 min

Two case studies show the structure twice. The parking lot is the most-asked question; the vending machine shows what to do when a design has a **lifecycle** (a fixed set of states an object moves through), the second most common kind of question. Do not start a third; write the five steps on a card and rehearse them.

If you have one week (about 8 hours, an hour a day)

Day	Do
1	Chapter 1 fully, plus Chapter 2 sections on the four pillars
2	Chapter 3: use case, class and activity diagrams (sequence diagrams later)
3	Parking lot, then library management (write your requirements first)
4	ATM, then vending machine
5	Movie ticket booking, then hotel management
6	Splitwise, then Amazon-style online shopping
7	**No new material.** Redo the parking lot from a blank page in 45 minutes

Day 7 matters most: days 1–6 fill your head with material, and day 7 checks that you can actually produce a design against the clock.

If you have one month (about 20 hours)

Week	Do
1	Chapters 1–4, then the first three case studies
2	Chapters 5, 8–11 (patterns) alongside case studies 4–10, two per session, always drawing your classes first
3	Chapters 12–13 (concurrency) and case studies 11–18; one timed run at the end of the week
4	Remaining case studies, Chapter 21 machine-coding problems, then three timed runs on questions you have read; Chapter 22 for revision

By week three the case studies feel repetitive. That means the method has been learned; keep going but read faster, checking your class list against the first diagram stages.

Beginner plan (six weeks, 5–6 hours a week) — if OOP itself is new to you:

Week	Do
1	Chapters 1–2; write and run every Chapter 2 example yourself
2	Chapters 3–5 (UML, SOLID, principles)
3	Chapters 6–7 (clean code, refactoring) and Chapter 8
4	Chapters 9–11 (remaining patterns); two small case studies
5	Chapters 12–13; four case studies including one with concurrency (ticket booking)
6	Chapter 21 machine-coding problems in 90-minute timed runs; Chapter 22

If you can only do six case studies, do these; they cover the skills the others repeat:

Case study	What it teaches
Parking lot	The canonical question, and how to bound one
Library management	Borrow, return and limits, a common family of questions
Vending machine	A lifecycle, one class per state
Movie ticket booking	Two people wanting the same seat: the concurrency question
Splitwise	Money, and a rule with several versions
Amazon online shopping	A question too big to finish, and choosing the part you build

Add a seventh, **an in-memory cache (LRU)**, if your target companies ask for a data structure rather than a product.

The night before: do not learn a new question (a half-understood design adds anxiety and will not be remembered). Instead: re-read the time budget (knowing you are *allowed* eight minutes on requirements stops you rushing), re-read the extensions section of one case study you have done, and write the five steps from memory: **requirements, actors and use cases, classes, relationships, code.**

1.14 Common mistakes

Mistake	Why it hurts	Fix
Coding in minute two	Missing requirements appear at minute 35	Spend 8–13 minutes on requirements and actors
No out-of-scope list	Scope creeps; omissions look like oversights	Write it down and agree it
Agreeing to every suggested feature	Nothing gets finished	"I'll leave X out and add it if time allows"
One subclass per value	Class explosion with no behaviour	Different data → field/enum
God class (`ParkingLotManager` does everything)	Fails single responsibility; untestable	One sentence per class, split on "and"
Missing the record class (ticket/loan/booking)	Data ends up on the wrong class (`dueDate` on `Book`)	Connection that carries data = an object
Inheritance for code reuse ("Car extends Engine")	Wrong "is-a"; fragile	Composition
Pattern stuffing	Looks like memorisation, not judgement	One or two patterns, with the signal
`switch` on type everywhere	Every new type edits many places	Polymorphism (one call, different behaviour per class) / Strategy
All twelve classes at equal depth	No flow works end to end	Two or three flows fully
Silent designing	Interviewer cannot credit your reasoning	Narrate each step
Ignoring concurrency when two users share a resource	Double booking	Lock or atomic operation on the shared state
Non-running code in machine coding	Often an automatic reject	Get a thin end-to-end path running by halfway
Using `double` for money, reading the clock inside logic	Rounding bugs, flaky demo	Cents/`BigDecimal`, inject time

---

In the interview

30-second answer (to "How do you approach a low-level design problem?"): "First I clarify and write 8 to 12 requirements with an explicit out-of-scope list. Then I list the actors and their use cases. From the nouns I pick the classes, cutting anything that is only a value, and give each class one responsibility. I connect them with the right relationships and draw a class diagram, sketch the key flow, and run a quick SOLID check and pick at most one or two patterns where the signal is clear. Then I code the two or three core flows, handle edge cases and concurrency, and finish by discussing extensions and trade-offs."

Follow-ups:

- *"How is this different from system design?"* — System design is about many machines and scale (databases, caches, queues). LLD stays inside one application: classes, interfaces, relationships and code.
- *"You won't finish; what matters?"* — Depth on the chosen part: a bounded scope, clean responsibilities, correct relationships, code that matches the diagram, and one trade-off I can

defend.

- *"What changes in a machine-coding round?"* — The code must run. I keep storage in memory, define service interfaces and a driver, get the happy path working by halfway, then add features, validation and one extension point.
- *"How do you decide whether to use a pattern?"* — I look for the signal: several interchangeable rules → Strategy, lifecycle with many transitions → State, many listeners → Observer. If there is no signal, plain classes are the better answer.

Self-check

- **Q:** Name the five core steps in order. **A:** Requirements, actors & use cases, classes, relationships, code.
- **Q:** How much of a 45-minute round goes before naming a class? **A:** Roughly a quarter (8–13 minutes).
- **Q:** What are the five things an interviewer judges? **A:** Bounded problem, one-sentence responsibilities, correct relationships, code matching the diagram, an explained trade-off.
- **Q:** What does a machine-coding round add? **A:** Working code within about 90 minutes, data models, service interfaces, extensibility, and a demo.
- **Q:** Which two classes do beginners most often miss? **A:** The record-keeping class (ticket/loan/booking) and the rule class (rate/strategy).
- **Q:** "Different data" vs "different behaviour"? **A:** Different data → field or enum. Different behaviour → separate class.
- **Q:** How many patterns in a 45-minute design? **A:** One or two, placed where the signal appears; name a rejected one too.
- **Q:** What should the last day of any study plan be? **A:** A timed run from a blank page.

TL;DR

- LLD/OOD = design the classes inside one application and code the main flows; machine coding = the same with running code in ~90 minutes.
- Clarify → requirements → use cases → entities → relationships → diagram → flows → patterns/SOLID → code → edge cases/concurrency → extensions.
- Spend a quarter of the time before naming a class; write an out-of-scope list.
- Different data → field; different behaviour → class; one sentence per class.
- Run SOLID as five quick checks; use one or two patterns where the signal appears.
- Code two or three flows properly; inject time, use `BigDecimal` (or cents) for money, guard shared state.
- Practise by attempting first, then comparing; finish every plan with a timed blank-page run.

Chapter 14 — Case Studies: Everyday Objects

> **Where this fits.** Chapters 1–13 gave you the tools: OOP (object-oriented programming: building software from objects that bundle data with the methods that act on it), UML (Unified Modeling Language, the standard way to draw classes and their links), SOLID (five principles for designing classes), clean code, the 23 GoF patterns (the "Gang of Four" design patterns, named after the four authors of the 1994 book *Design Patterns*) and concurrency (several pieces of work running at the same time). From this chapter to Chapter 21 you use those tools on real interview questions. This chapter starts with **physical machines you have used yourself**: a parking lot, a vending machine, a coffee machine, an ATM, an elevator, a traffic light and a parcel locker. You already know how they behave, so all your attention can go to the design.

What you will learn in this chapter

#	Problem	The one idea the interviewer is really testing	Main patterns
14.1	Parking Lot	**Resource allocation** under concurrency: two cars must never get the same spot	Strategy (spot finding, pricing), Factory, Observer (display boards)
14.2	Vending Machine	A **state machine**: the order in which actions are allowed	State
14.3	Coffee Vending Machine	**Atomic, all-or-nothing** deduction from a shared inventory; recipes as data	Observer (low-stock alerts), data-driven recipes, sealed result types
14.4	ATM	**Security + money correctness**: authenticate, never lose money, never double-spend	State, Chain of Responsibility (cash dispenser), Command (transactions)
14.5	Elevator System	**Scheduling** (SCAN/LOOK: sweep orders for visiting stops, see 14.5) and **dispatching** many cars	Strategy (dispatch), State-like modes
14.6	Traffic Signal Control	**Timed state machine** with a hard safety invariant and emergency override	State, Strategy (timing), Observer (safety monitor)
14.7	Shipping Locker System	**Best-fit allocation + deadlines + a secret**: reserve a door atomically, expire holds and uncollected parcels, single-use hashed pickup codes	Strategy (door choice), Observer (SMS), enum state machines, sealed results

> **Key terms — the pattern names in this table.** These are design patterns (proven, named shapes for arranging classes) from the earlier chapters. **Strategy**: a swappable rule (how to price, which car to send) hidden behind an interface. **Factory**: one place that creates objects, so callers need not know how they are built. **Observer**: an object keeps a list of listeners and tells each of them when something changes. **State**: each situation a machine can be in is its own class. **Chain of Responsibility**: a request is passed along a line of handlers, each doing what it can. **Command**: an action wrapped in an object, so it can be run, logged or queued. **Sealed result types**: a Java type whose possible variants form a fixed, known list (for example "served" or "refused"), so the compiler can check that callers handle each one.

What these problems test

These seven questions look different, but interviewers use them to check three skills:

1. **State machines.** A machine behaves differently depending on what has happened so far. A vending machine must not dispense before it is paid. An ATM must not allow a withdrawal before the PIN is checked. A traffic light must never go from green straight to green on the crossing road. A **state**

machine is a model with a fixed set of situations (states) and rules for moving between them (transitions). The **State pattern** (Chapter 11) turns each state into its own class, so every button has a defined answer in every state.

2. **Resource allocation.** A limited set of things (parking spots, notes in a cassette, millilitres of milk, elevator cars, locker doors) must be handed out correctly: never twice, never below zero, and fairly.
3. **Concurrency.** Many actors act at the same moment: several entrance gates, two ATMs on one joint account, ten customers tapping "Latte" at once, a timer thread and an emergency thread (a **thread** is one independent line of execution inside a program; many threads can run at once and touch the same objects). The design must stay correct when calls overlap (Chapters 12–13).

> **Key insight.** In every one of these problems, the hardest part is small: one method that picks a spot, one method that makes change, one method that deducts ingredients, one method that picks the next elevator stop. Find that method early, name it out loud, and spend your time there.

How each case study is laid out

Every problem follows the same template, which is also a good order to speak in during an interview:

Problem & clarifying questions → Requirements → Use cases → Core entities → Class diagram → Key flows → Design decisions & patterns → Java implementation → Concurrency & edge cases → Extensions & follow-ups → In the interview → Self-check → TL;DR.

How to read the course diagrams

The course draws its class diagrams in stages. Classes added in earlier stages are drawn in grey; the new ones are in colour. The legend below explains every arrow used in those diagrams (it is the same for all four course case studies in this chapter).

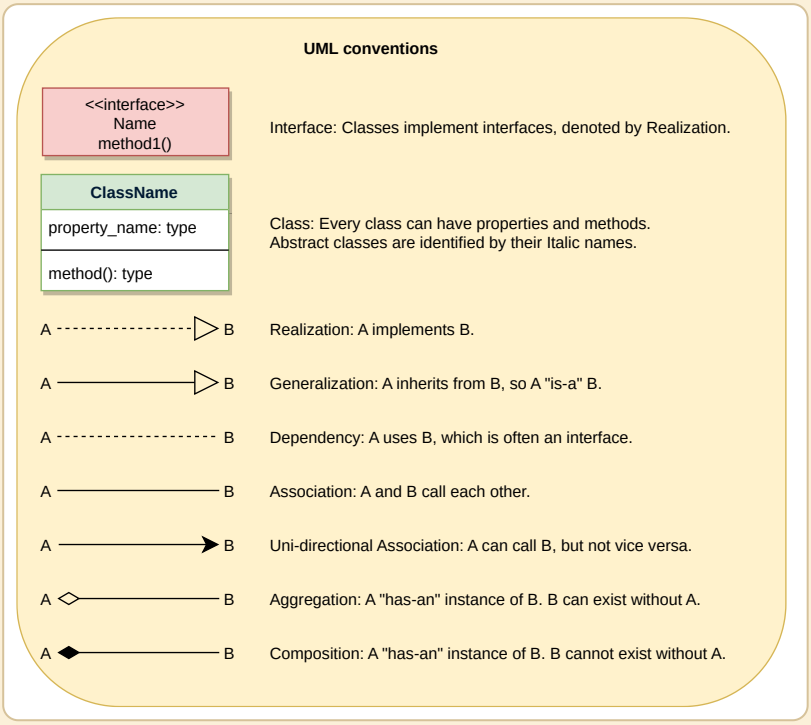


Figure — Legend for the course class diagrams: interface, class, realization, association, inheritance, composition and the other arrow kinds (course diagram)

Read the figure above once: a **hollow triangle** means inheritance ("is a"), a **dashed line with a hollow triangle** means "implements an interface" (an **interface** is a list of methods that a class promises to provide, without saying how), a **filled diamond** means composition (the part dies with the whole), and a **plain arrow** means association (one object refers to another without owning it). Chapter 3 covers these relationships in depth.

How to run every example in this chapter

```
cd chapters/14-case-everyday-objects/code
javac --release 21 -d /tmp/lld_14 $(find . -name '*.java')
java -cp /tmp/lld_14 lld.ch14.parkinglot.Main      # or vendingmachine, coffeemachine, atm, elevator,
trafficsignal, locker
```

Here `javac` is the Java compiler (it turns `.java` source files into `.class` files of bytecode), `--release 21` targets Java 21, `-d` says where to put the compiled classes, and `java -cp` runs a class using the given **classpath** (the folder where the JVM, the Java Virtual Machine that runs bytecode, looks for classes).

All seven `Main` classes compile with Java 21 (standard library only) and run. The output printed under each implementation is the real output of that program.

> **Code-style note.** To keep each design readable in one place, a file sometimes holds several small package-private types (types declared without `public`, visible only inside their own package) (for example `Vehicles.java` holds `SpotType`, `VehicleType` and `Vehicle`). That is legal Java, and the code compiles with no warnings under `-Xlint:all` (the compiler switch that turns on every optional warning, or *lint* check) except the `auxiliaryclass` lint, which exists precisely to flag this style (a type used from outside the file it is declared in). In production you would usually give each top-level type its own file.

14.1 Design a Parking Lot



Figure — A large multi-level car park, the kind of building this design models (course diagram)

A **parking lot** (car park) is an area set aside for parking vehicles. Malls, stadiums, airports and office towers have large multi-floor lots with gates, ticket machines and boards showing free spaces. The photo above is the kind of lot we are modelling: many floors, many spot sizes, and several ways in and out.

This is the most-asked LLD question (low-level design: designing the classes, their responsibilities and how they talk to each other) of all. It looks easy, which is exactly why it is useful to an interviewer: they can watch how you **scope** it, how you **model the fit rule** (which vehicle may use which spot), and whether you notice that **two gates can hand out the same spot** at the same moment.

Problem & clarifying questions

Ask these before drawing anything. The answers in the right column are the ones this design assumes.

Question	Assumed answer
One lot or a chain of lots?	One lot, many floors.
How many entrances and exits?	Several of each, all working at the same time.
Which vehicle types?	Car, truck, van, motorcycle, electric car.
Which spot types?	Compact, large, handicapped (accessible), motorcycle, electric.
How do customers pay?	Cash or card, at the exit panel, to an attendant, or earlier at an info portal on each floor.
How is the fee computed?	Per hour, with tiers: $4 for the first hour, $3.50 for hours 2 and 3, $2.50 for each hour after. A started hour counts as a full hour.
Do EVs charge?	Yes: electric spots have a panel where the driver pays for charging.
Reservations, monthly passes, number-plate recognition?	Out of scope (mentioned as extensions).
What happens when the lot is full?	The entrance panel and the ground-floor board show a "full" message; no more vehicles are admitted.

Requirements

Functional

1. The lot has **multiple floors**.
2. It has **multiple entry and exit points**.
3. A customer **collects a ticket at the entrance** and **pays at the exit**.
4. Payment at the **automated exit panel** or to a **parking attendant**.
5. Payment by **cash or credit card**.
6. Payment is also possible at a **customer info portal on each floor**; a ticket paid there is not charged again at the exit.
7. The lot **never admits more vehicles than its capacity**. When full, a message shows on the entrance panel and on the ground-floor display board.
8. Each floor has many spots of several **types**: compact, large, handicapped, motorcycle (and electric, below).
9. Some spots are for **electric cars**; each has an **electric panel** where the customer can pay for and charge the vehicle.
10. Many **vehicle types**: car, truck, van, motorcycle (and electric car).
11. Each floor has a **display board** showing free spots per spot type.
12. A **per-hour fee model** with tiers ($4 / $3.50 / $3.50 / $2.50 …).

Non-functional

- **Correctness under concurrency:** two entrance gates working at the same instant must never be given the same spot, and a ticket must never be paid twice.
- **Low latency at the gate:** finding a spot must not scan thousands of spots; taking one from a sorted set of free spots is O(log n). (**Latency** is how long one request takes from start to finish; see System Design Fundamentals. **O(log n)** is Big-O notation for cost: the work grows with the logarithm of the number of spots *n*, so even 10,000 spots need only about 14 steps.)
- **Extensibility:** a new pricing rule, a new spot-finding rule or a new vehicle type should be a small, local change.
- **Testability** (how easily automated tests can check the code)**:** time must be injectable (passed in from outside rather than read inside the code, here as a fake clock), so fee tests do not wait for hours.

Use cases

Actors (an actor is anyone or anything outside the system that uses it):

- **Admin** — adds, removes and edits floors, spots, entrance/exit panels; adds and removes attendants; sets the hourly rate.
- **Customer** — takes a ticket, pays, leaves.
- **Parking attendant** — can do everything a customer can, on the customer's behalf, and can accept cash.
- **System** — shows messages on panels and boards, assigns a vehicle to a spot and frees it again.

Top use cases: add/remove/edit a floor (each floor with its own display board); add/remove/edit a spot; add/remove an attendant; **take ticket**; **scan ticket** (to find the total charge); **credit-card payment**; **cash payment**; add/modify the parking rate.

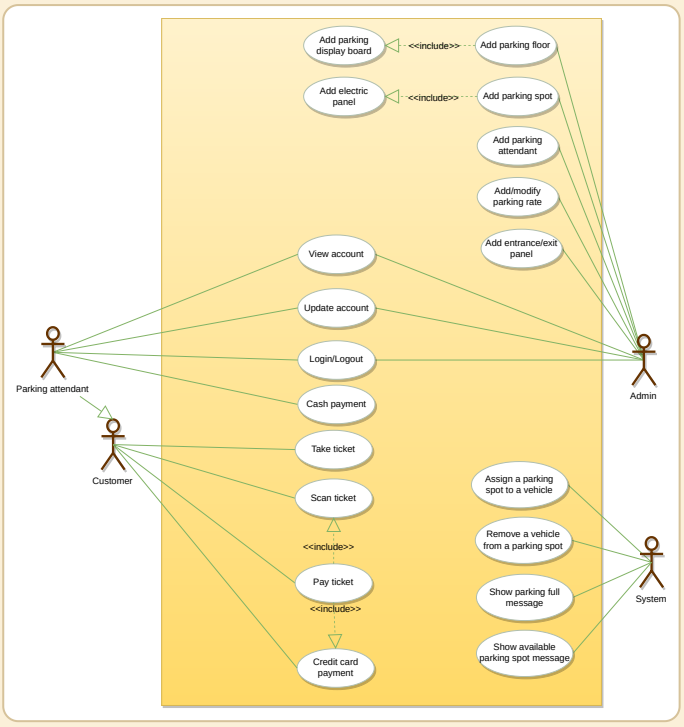


Figure — Use case diagram of the parking lot: Admin, Customer, Parking attendant and System, with their use cases (course diagram)

In the figure above, notice that the attendant's use cases overlap the customer's: the attendant is a customer "proxy" who can also take cash. The System actor is what assigns spots and updates the boards.

Core entities

Class	Responsibility
`VehicleType` (enum)	The kinds of vehicle and, for each, the spot types it fits in, **best fit first**.
`SpotType` (enum)	The kinds of spot: motorcycle, compact, large, handicapped, electric.
`Vehicle` (record)	Plate, type, accessibility permit. Knows its preferred spot types.
`ParkingSpot`	One physical spot: id, floor, type, and who occupies it (atomic).
`ChargingPanel`	The pay-and-charge panel owned by an electric spot.
`SpotFactory`	Builds spots; the only place that knows an electric spot needs a panel.
`ParkingFloor`	Holds the **free** spots per type in a thread-safe sorted set; notifies boards.
`DisplayBoard`	Shows free spots per type on one floor (an observer).
`SpotAllocationStrategy`	*How* to choose a spot (nearest floor first, least busy floor first, …).
`PricingStrategy`	*How much* a stay costs (hourly tiers, flat rate, weekend rate, …).
`Ticket`	Proof of entry: vehicle, the actual spot, entry time, status (ACTIVE → PAID → EXITED).
`PaymentMethod`	Cash or card.
`ParkingLot`	Ties it together: admit, price, pay, exit.
`EntrancePanel`, `ExitPanel`, `CustomerInfoPortal`	The devices customers touch. They hold **no rules**; they call the lot.

In the table, an **enum** is a Java type with a fixed list of named constants (such as `SpotType.COMPACT`), and a **record** is a short Java class that only carries a few values and cannot be changed after it is created. **Thread-safe** means an object stays correct even when several threads use it at the same moment.

The course also models **`Account`** with **`Admin`** and **`ParkingAttendant`** subclasses (both sign in). They carry no parking logic, so they are omitted from the code below; the attendant simply uses the same `pay` method as an exit panel.

Class diagram

The course grows the design in five steps. Every step adds a handful of classes and leaves you with a feature that runs. Use the same order in an interview: if the clock stops you early, what you have drawn already works, instead of being an unfinished picture.

Stage	What it adds	What works after it
1	Vehicle and spot types	The system can answer "may this vehicle use this spot?"
2	Floors and display boards	A floor can hand out a free spot and report counts
3	Ticket, rate card, payment	A stay can be turned into a price and settled
4	The `ParkingLot`	A full visit works: enter, park, pay, leave
5	Entrance/exit panels, info portal, electric panel	Drivers use the flow through physical devices

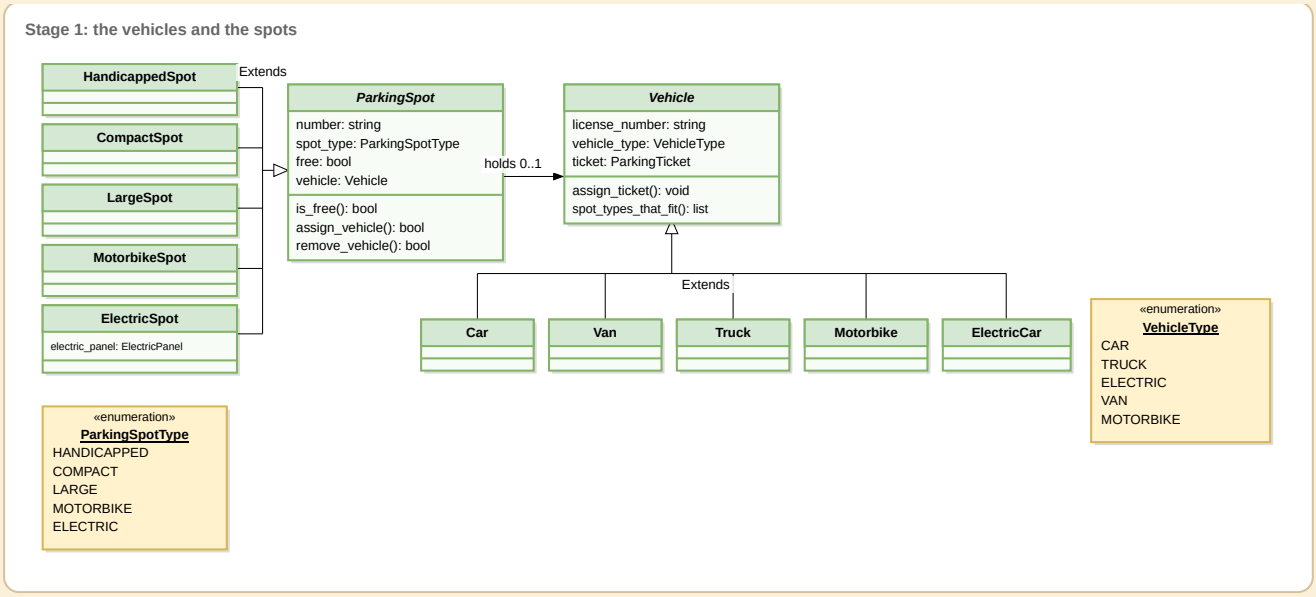


Figure — Stage 1: the vehicle and spot hierarchies, and the "holds 0..1" link from a spot to a vehicle (course diagram)

Stage 1 — vehicles and spots. Requirements 8 and 10 produce two small families of types and one link: a spot holds **at most one** vehicle (`0..1`). The most important rule of the whole design — *which spot types can this vehicle use?* — lives on the **vehicle** side. Because of that, introducing another kind of vehicle is a change to the vehicle types only; the lot code stays untouched. The fit rules: a motorcycle uses a motorcycle spot, trucks and vans use only large spots, and an electric car tries an electric spot first, then compact, then large.

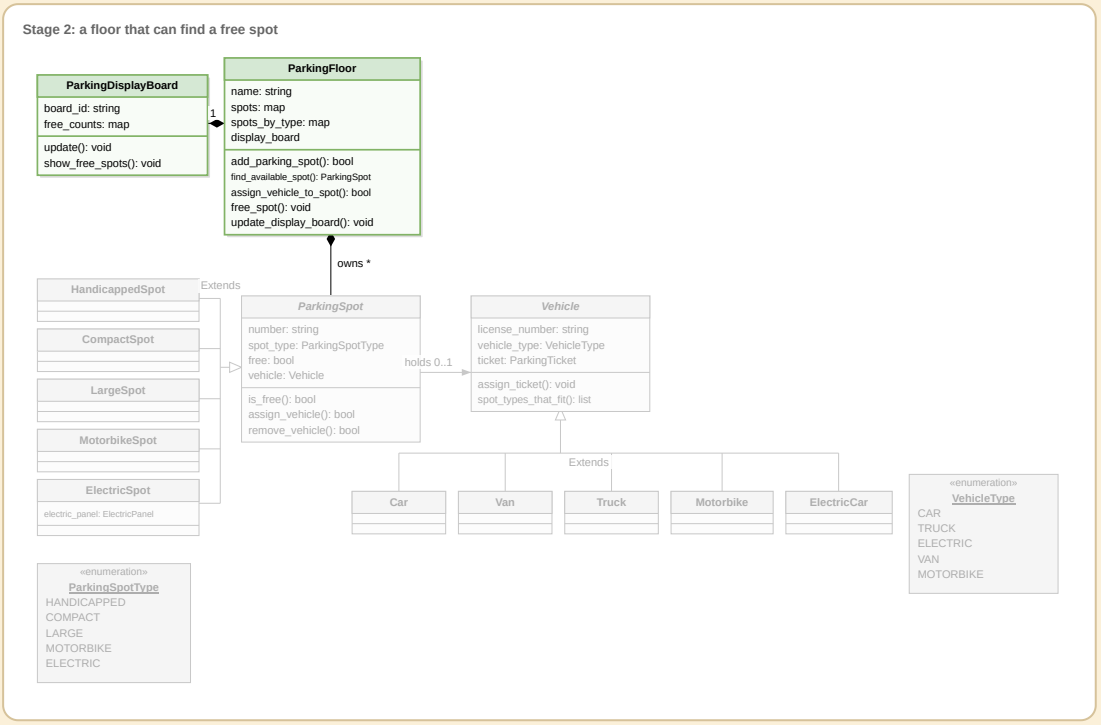


Figure — Stage 2: a floor owns its spots and its display board (composition) (course diagram)

Stage 2 — a floor that can find a free spot. Requirements 1, 8 and 11. Both new links are **composition** (filled diamond): deleting a floor deletes its spots and its board. The board stores **how many** spots of each type are free, not a pointer to "a sample free spot" of each type; such a pointer goes stale the moment that spot is taken and must be hunted for again. The course's floor keeps a map of all its spots plus an index by spot type (our code keeps a single map from spot type to its free

spots). The rejected alternative is one separate collection per type: every method would then need a five-way `if`, and adding a sixth type would touch all of them.

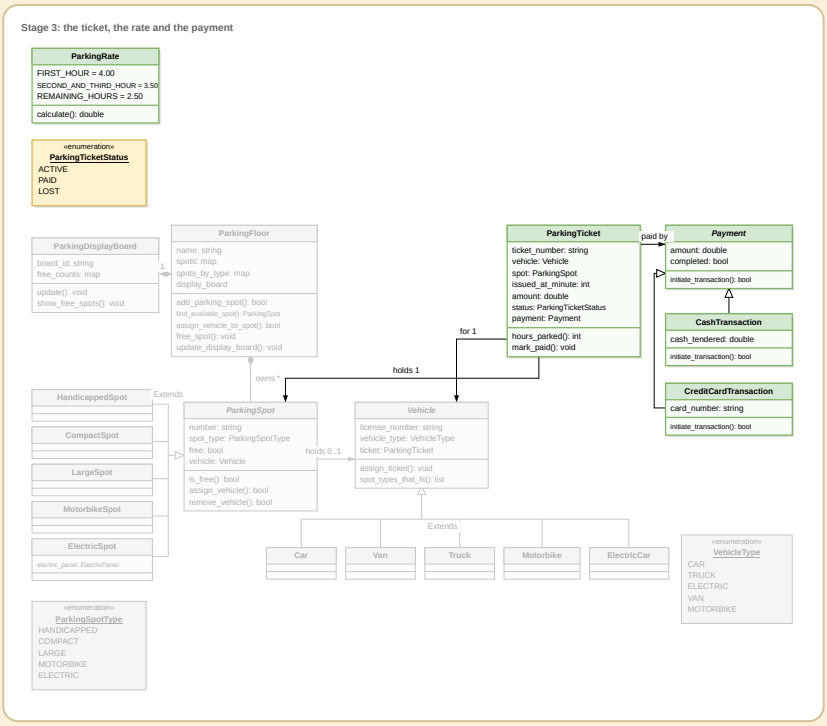


Figure — Stage 3: ticket, parking rate and the abstract payment with cash and card subclasses (course diagram)

Stage 3 — ticket, rate, payment. Requirements 3, 5 and 12. The key decision: **the ticket stores a reference to the exact spot**, not just "one more spot is taken". At the exit, that reference is how the system knows which spot to free and where the car was. A ticket merely points at the vehicle and the spot (it does not own them), so both links are plain associations. `Payment` is an abstract class (it cannot be created directly) with cash and card subclasses; this inheritance is justified because either subclass can stand in wherever "a payment" is expected (the Liskov Substitution Principle from Chapter 4).

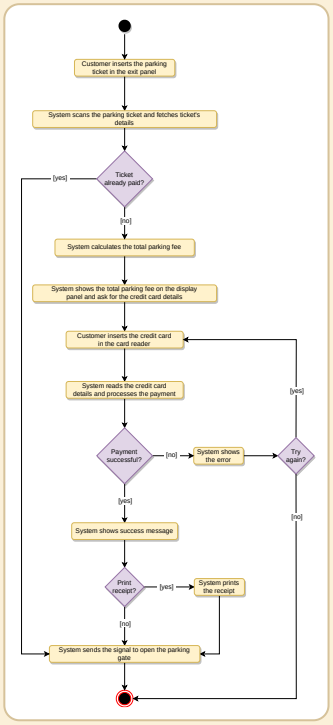


Figure — Activity diagram: the customer inserts the ticket at the exit, the system computes the fee, the customer pays by card or cash, and the barrier opens (course diagram)

The activity diagram above is the exit flow; the ticket, rate and payment classes between them handle each of its steps.

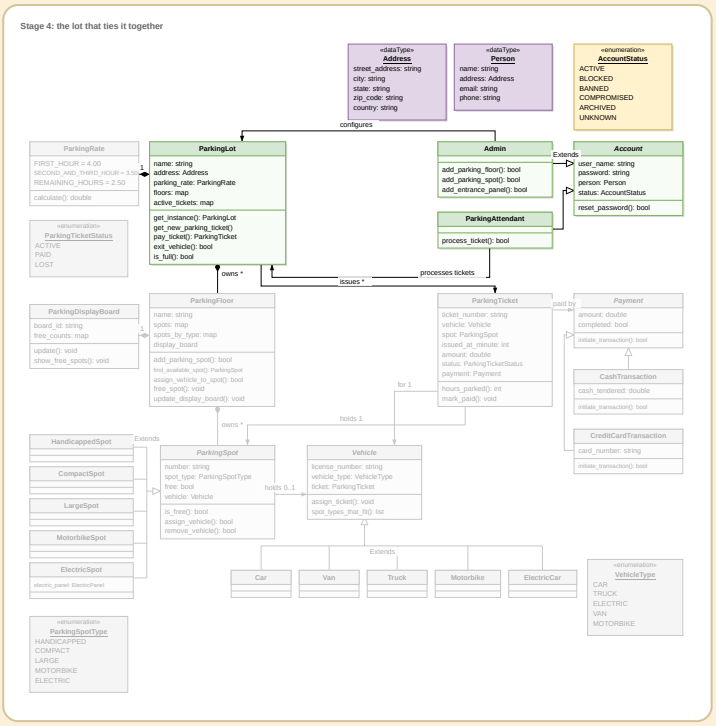


Figure — Stage 4: the parking lot owns floors and the rate; Admin and ParkingAttendant accounts connect to it (course diagram)

Stage 4 — the lot. Requirement 7 and the actor list. The lot has a **name** and an **address** (so one lot can be told apart from another) and owns its floors and its rate card (filled diamonds). The two account types, `Admin` and `ParkingAttendant`, are linked with plain arrows: they work *on* the lot (setting it up, handling tickets) without owning any part of it. The course makes the lot a **Singleton** (only one instance may exist) and serialises spot assignment (**one caller at a time**), since two entrances assigning in parallel could pick the same spot.

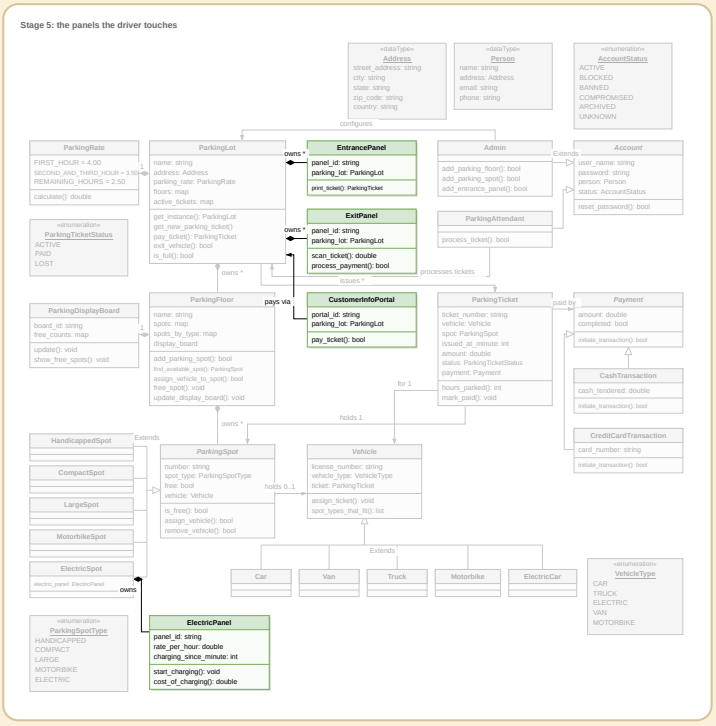


Figure — Stage 5: entrance and exit panels (owned by the lot), the customer info portal (refers to the lot) and the electric panel (owned by an electric spot) (course diagram)

Stage 5 — the panels. Requirements 2, 4, 6 and 9. The panels are thin: they forward every request to the lot, so all business rules sit in a single class. The entrance panel prints tickets, the exit panel

scans a ticket and takes payment, the info portal lets a driver settle early (marking the ticket paid), and the electric panel bills for charging. The lot owns its entrance and exit panels (composition). It keeps no list of info portals, so a portal just holds a reference to the lot (association). The electric panel is owned by its electric spot, the only composition hanging off a spot.

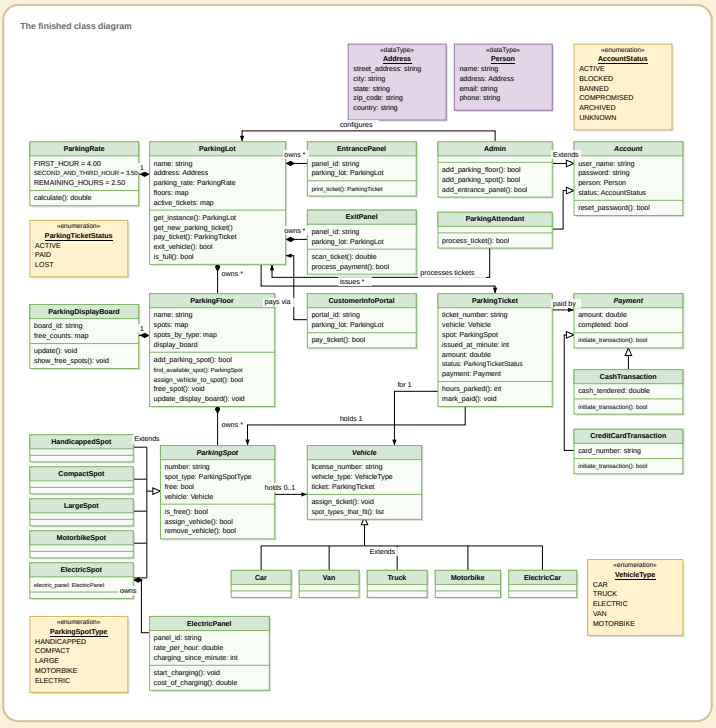
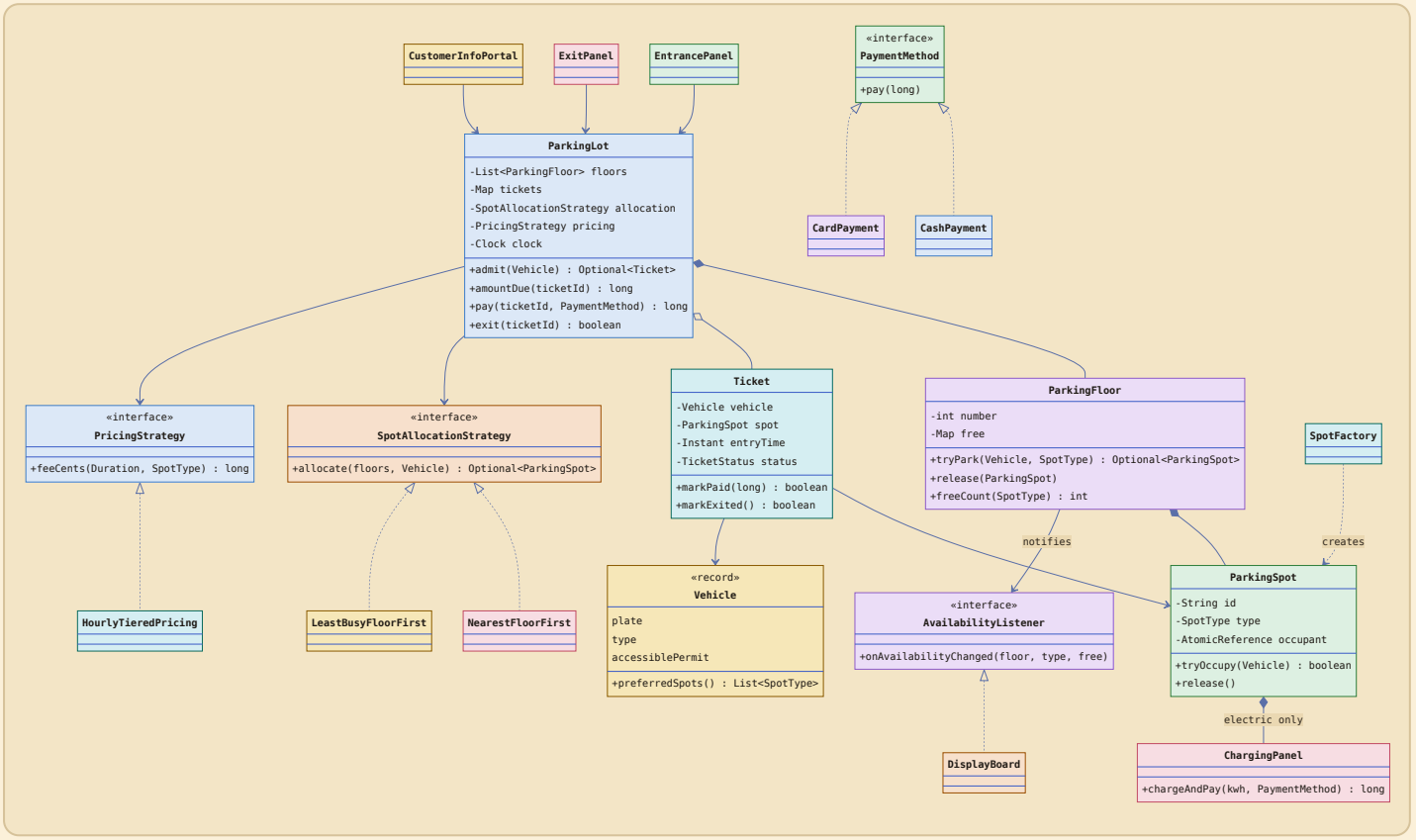


Figure — The finished class diagram of the parking lot (course diagram)

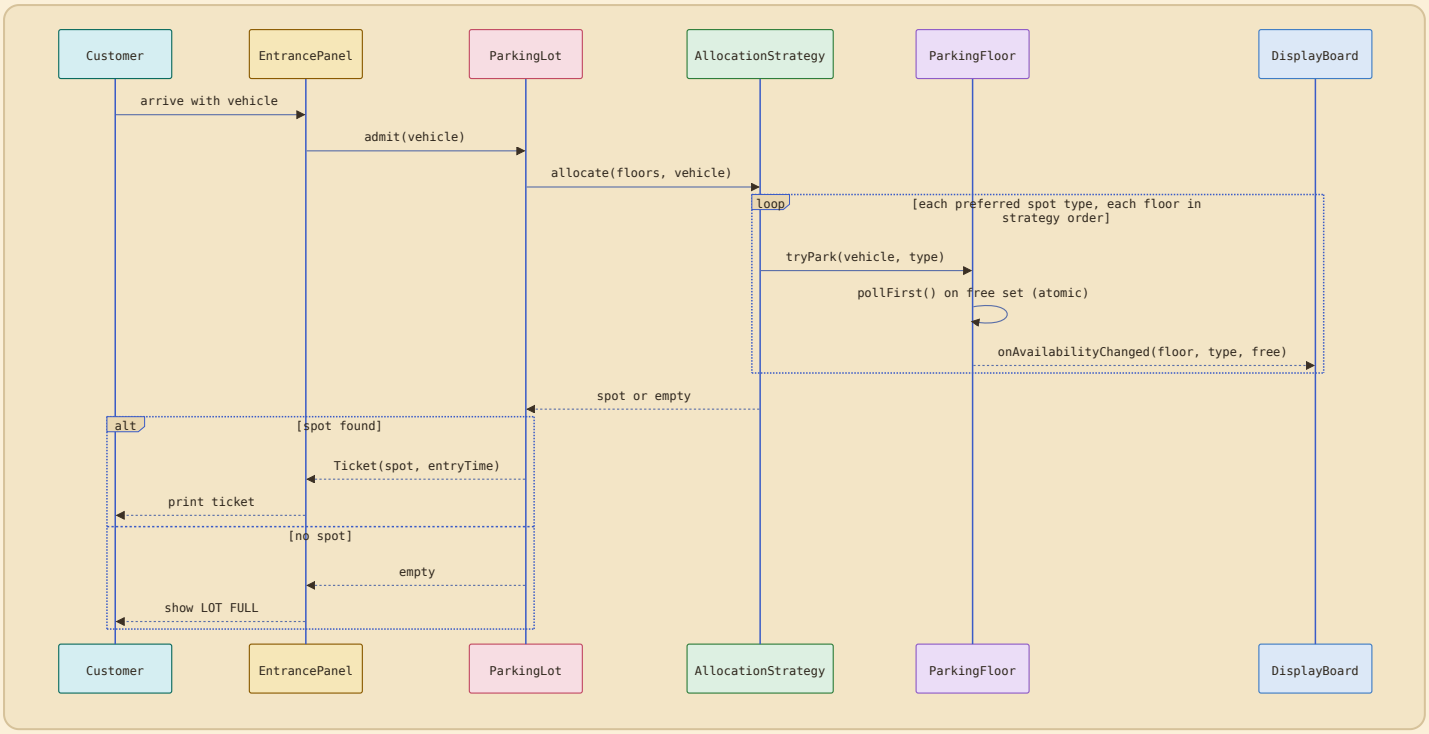
The finished diagram above is simply all five stages in colour; nothing new appears in it. Here is the class diagram of **the Java code in this chapter**. It keeps the same shape but makes three changes that interviewers usually push for: spot types are an enum instead of five empty subclasses, pricing and spot-finding are Strategies, and the display board is an Observer.



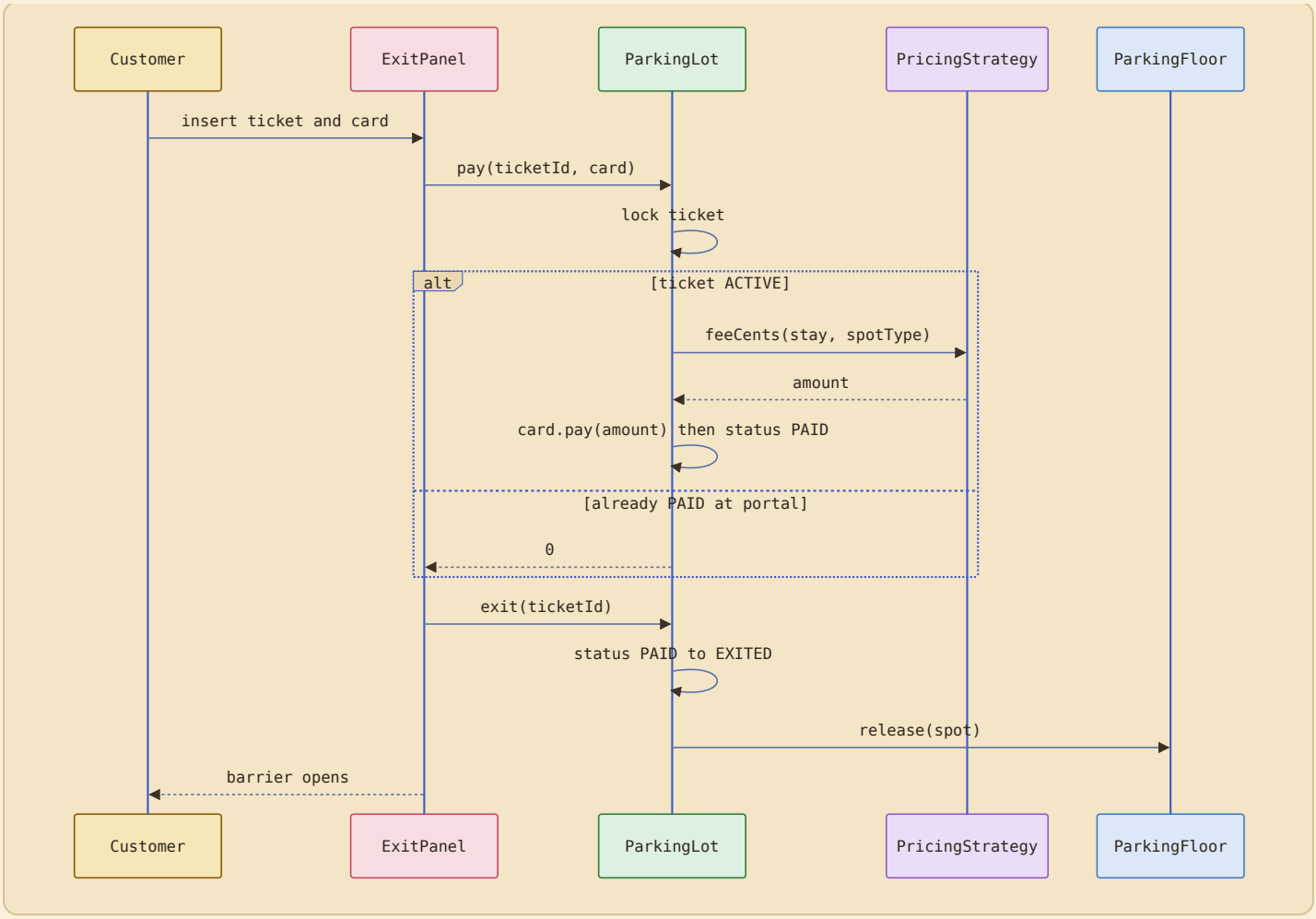
In this Mermaid diagram (Mermaid is a text format that renders as a diagram), `-` marks a private field (hidden inside its class), `+` a public method, `~T~` a generic type such as `List<ParkingFloor>`, `*--` composition, `o--` aggregation (a looser "has a" where the parts can live on their own), `-->` association, `<|..` "implements", and `..>` "uses or creates".

Key flows

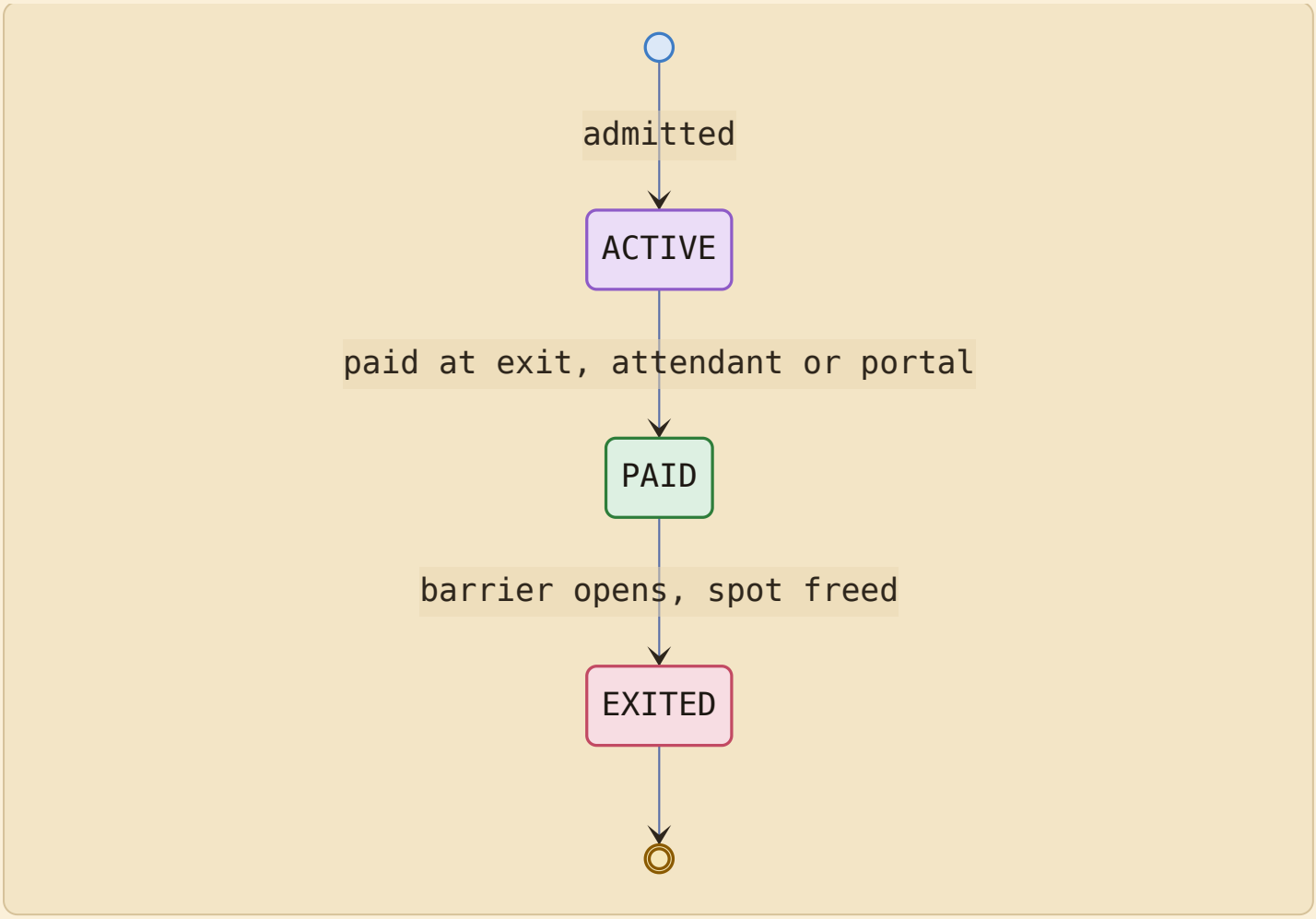
Entry (take ticket). The important step is that the spot is taken **atomically** (as one indivisible step that no other thread can interrupt halfway): `pollFirst()` on a concurrent sorted set removes the spot for exactly one caller. The **sequence diagram** below shows the messages between objects in time order, from top to bottom.



Exit (scan, pay, leave). Payment is guarded per ticket so the info portal and the exit panel cannot both charge.



Ticket life cycle.


admitted
ACTIVE
paid at exit, attendant or portal
PAID
barrier opens, spot freed
EXITED

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
The fit rule lives on `VehicleType`	Information Expert (give a job to the class that owns the data), Open/Closed (extend without editing)	Adding a vehicle type is one enum constant; the lot never changes	A big `switch(vehicleType)` inside the lot
Spot type is an **enum field**, not 5 subclasses	Composition over inheritance (hold a field instead of subclassing)	The 5 course subclasses carried no behaviour; a new type is one constant	`CompactSpot`, `LargeSpot`… (reviewers flag these as empty classes)
How to pick a spot is a `SpotAllocationStrategy`	**Strategy**	Nearest floor, least busy floor, closest to the lift, VIP floors… all swap without touching the lot	`if (mode == NEAREST) … else …` in the lot
How much to charge is a `PricingStrategy`	**Strategy**	The course's "what if pricing changes?" follow-up; admin swaps the rate at run time (the demo swaps to a flat rate)	Hard-coded tiers inside `Ticket`
Electric spots get a `ChargingPanel` in `SpotFactory`	**Factory** (simple static factory)	One place knows the construction rule "electric ⇒ has a panel"	Every caller remembers to attach a panel
Boards subscribe to floors	**Observer**	A floor change updates every board (and could update an app or a sensor light) without the floor knowing who listens	The lot pushing into boards directly
Ticket holds the **spot**, not a count	—	The exit can find and free the exact spot	Counting occupied spots
`ParkingLot` is **not** a hard-coded Singleton	Dependency injection (pass collaborators in from outside)	The course uses a Singleton ("there is one car park"). It is a global in disguise: tests cannot build two lots, and every class that calls `getInstance()` hides a dependency. We create one lot in `main` and pass it to the panels.	`ParkingLot.getInstance()`
`Clock` is injected	Testability	The demo moves time forward by hours instantly	`Instant.now()` inside the code

> **Honest note on "Strategy".** The course points out that a cash/card subclass pair is just an abstract base class, and calling it "Strategy" would be loose use of the word. Strategy means an object **holds** a behaviour and can **swap** it. In our code `ParkingLot` holds a `PricingStrategy` and a `SpotAllocationStrategy` and `setPricing(...)` swaps it at run time — that is genuinely Strategy.

The course also admits that `ParkingLot` does two jobs (finding spots and issuing tickets). Here spot-finding has moved into the strategy and the floors, so the lot is mostly a coordinator; in a longer design you would also move ticket storage into a `TicketRepository`.

Java implementation

Package `lld.ch14.parkinglot`. Amounts are in **cents** (`long`) to avoid floating-point rounding (a `double` stores numbers in binary and cannot hold values such as 0.10 exactly, so sums drift by tiny amounts).

```java
// File: lld/ch14/parkinglot/Vehicles.java
package lld.ch14.parkinglot;

import java.util.ArrayList;
import java.util.List;

/** The kinds of spot a floor can have. */
enum SpotType { MOTORCYCLE, COMPACT, LARGE, HANDICAPPED, ELECTRIC }

/** The kinds of vehicle the lot accepts, each with the spot types it fits, best fit first. */
enum VehicleType {
    MOTORCYCLE(List.of(SpotType.MOTORCYCLE)),
    CAR(List.of(SpotType.COMPACT, SpotType.LARGE)),
    ELECTRIC_CAR(List.of(SpotType.ELECTRIC, SpotType.COMPACT, SpotType.LARGE)),
    VAN(List.of(SpotType.LARGE)),
    TRUCK(List.of(SpotType.LARGE));

    private final List<SpotType> fits;

    VehicleType(List<SpotType> fits) { this.fits = fits; }

    List<SpotType> fits() { return fits; }
}

/** A vehicle is a value: plate + type + whether it carries an accessibility permit. */
record Vehicle(String plate, VehicleType type, boolean accessiblePermit) {

    Vehicle(String plate, VehicleType type) { this(plate, type, false); }

    /** Spot types this vehicle may use, most preferred first. The rule lives here, not in the lot. */
    List<SpotType> preferredSpots() {
        List<SpotType> order = new ArrayList<>();
        if (accessiblePermit && type != VehicleType.MOTORCYCLE && type != VehicleType.TRUCK) {
            order.add(SpotType.HANDICAPPED);
        }
        order.addAll(type.fits());
        return order;
    }
}
```

```java
// File: lld/ch14/parkinglot/ParkingSpot.java
package lld.ch14.parkinglot;

import java.util.Optional;
import java.util.concurrent.atomic.AtomicReference;

/** One physical spot. Occupancy is an AtomicReference so two threads can never both win it. */
class ParkingSpot implements Comparable<ParkingSpot> {
    private final String id;          // e.g. "F1-C03"
    private final int floor;
    private final SpotType type;
    private final AtomicReference<Vehicle> occupant = new AtomicReference<>();
    private final ChargingPanel chargingPanel; // only electric spots have one (composition)

    ParkingSpot(String id, int floor, SpotType type, ChargingPanel chargingPanel) {
        this.id = id;
        this.floor = floor;
        this.type = type;
        this.chargingPanel = chargingPanel;
    }

    /** Compare-and-set: succeeds only if the spot was empty. */
    boolean tryOccupy(Vehicle v) { return occupant.compareAndSet(null, v); }

    void release() { occupant.set(null); }

    boolean isFree() { return occupant.get() == null; }

    String id() { return id; }
    int floor() { return floor; }
    SpotType type() { return type; }
    Optional<ChargingPanel> chargingPanel() { return Optional.ofNullable(chargingPanel); }

    @Override public int compareTo(ParkingSpot o) { return id.compareTo(o.id); }
    @Override public String toString() { return id + "(" + type + ")"; }
}

/** The pay-and-charge panel that belongs to an electric spot. */
class ChargingPanel {
    private final long centsPerKwh;

    ChargingPanel(long centsPerKwh) { this.centsPerKwh = centsPerKwh; }

    /** Charges the car and takes payment for the energy right at the spot. */
    long chargeAndPay(double kwh, PaymentMethod method) {
        long cost = Math.round(kwh * centsPerKwh);
        method.pay(cost);
        return cost;
    }
}

/** Factory: the only place that knows an ELECTRIC spot must be built with its panel. */
final class SpotFactory {
    private static final long EV_CENTS_PER_KWH = 30;

    private SpotFactory() {}

    static ParkingSpot create(int floor, SpotType type, int number) {
        String id = "F%d-%s%02d".formatted(floor, type.name().charAt(0), number);
        ChargingPanel panel = type == SpotType.ELECTRIC ? new ChargingPanel(EV_CENTS_PER_KWH) : null;
        return new ParkingSpot(id, floor, type, panel);
    }
}
```

```java
// File: lld/ch14/parkinglot/ParkingFloor.java
package lld.ch14.parkinglot;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentSkipListSet;
import java.util.concurrent.CopyOnWriteArrayList;

/** Observer: anything that wants to know when free-spot counts change. */
interface AvailabilityListener {
    void onAvailabilityChanged(int floor, SpotType type, int freeNow);
}

/** A floor keeps one sorted, thread-safe set of FREE spots per spot type. */
class ParkingFloor {
    private final int number;
    private final Map<SpotType, ConcurrentSkipListSet<ParkingSpot>> free = new EnumMap<>(SpotType.class);
    private final List<AvailabilityListener> listeners = new CopyOnWriteArrayList<>();
    private final Set<SpotType> typesHere;

    ParkingFloor(int number, Map<SpotType, Integer> spotCounts) {
        this.number = number;
        for (SpotType t : SpotType.values()) free.put(t, new ConcurrentSkipListSet<>());
        typesHere = Set.copyOf(spotCounts.keySet());
        spotCounts.forEach((type, count) -> {
            for (int i = 1; i <= count; i++) free.get(type).add(SpotFactory.create(number, type, i));
        });
    }

    void addListener(AvailabilityListener l) {
        listeners.add(l);
        for (SpotType t : typesHere) l.onAvailabilityChanged(number, t, freeCount(t));
    }

    /** pollFirst() is atomic: two threads can never receive the same spot. Lowest id = nearest. */
    Optional<ParkingSpot> tryPark(Vehicle v, SpotType type) {
        ParkingSpot spot = free.get(type).pollFirst();
        if (spot == null) return Optional.empty();
        if (!spot.tryOccupy(v)) throw new IllegalStateException("free-set out of sync for " + spot);
        notifyListeners(type);
        return Optional.of(spot);
    }

    void release(ParkingSpot spot) {
        spot.release();
        free.get(spot.type()).add(spot);
        notifyListeners(spot.type());
    }

    int freeCount(SpotType t) { return free.get(t).size(); }
    int number() { return number; }

    private void notifyListeners(SpotType t) {
        int n = freeCount(t);
        for (AvailabilityListener l : listeners) l.onAvailabilityChanged(number, t, n);
    }
}

/** The display board on a floor: shows free spots per type. */
class DisplayBoard implements AvailabilityListener {
    private final Map<SpotType, Integer> shown = new EnumMap<>(SpotType.class);

    @Override public synchronized void onAvailabilityChanged(int floor, SpotType type, int freeNow) {
        shown.put(type, freeNow);
    }

    synchronized String render() {
```

```
        StringBuilder sb = new StringBuilder();
        shown.forEach((t, n) -> sb.append(t).append('=').append(n == 0 ? "FULL" : n).append(' '));
        return sb.toString().trim();
    }
}
```

```java
// File: lld/ch14/parkinglot/Strategies.java
package lld.ch14.parkinglot;

import java.time.Duration;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;

/** Strategy: HOW to pick a spot. */
interface SpotAllocationStrategy {
    Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v);
}

/** Fill the lowest floor first (fewest ramps to drive). */
class NearestFloorFirst implements SpotAllocationStrategy {
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots()) {
            for (ParkingFloor f : floors) {
                Optional<ParkingSpot> s = f.tryPark(v, type);
                if (s.isPresent()) return s;
            }
        }
        return Optional.empty();
    }
}

/** Spread cars out: pick the floor with the most free spots of the wanted type. */
class LeastBusyFloorFirst implements SpotAllocationStrategy {
    @Override public Optional<ParkingSpot> allocate(List<ParkingFloor> floors, Vehicle v) {
        for (SpotType type : v.preferredSpots()) {
            List<ParkingFloor> order = floors.stream()
                    .sorted(Comparator.comparingInt((ParkingFloor f) -> f.freeCount(type)).reversed())
                    .toList();
            for (ParkingFloor f : order) {
                Optional<ParkingSpot> s = f.tryPark(v, type); // may lose a race: just try the next floor
                if (s.isPresent()) return s;
            }
        }
        return Optional.empty();
    }
}

/** Strategy: HOW MUCH a stay costs. */
interface PricingStrategy {
    long feeCents(Duration stay, SpotType spotType);
}

/**
 * Per-hour tiers: $4 for hour 1, $3.50 for hours 2 and 3, $2.50 for every hour after.
 * A started hour counts as a full hour.
 */
class HourlyTieredPricing implements PricingStrategy {
    private final List<Long> firstHours;   // price of hour 1, 2, 3 ...
    private final long laterHourCents;     // price of every later hour

    HourlyTieredPricing(List<Long> firstHours, long laterHourCents) {
        this.firstHours = List.copyOf(firstHours);
        this.laterHourCents = laterHourCents;
    }

    static HourlyTieredPricing standard() { return new HourlyTieredPricing(List.of(400L, 350L, 350L), 250); }

    @Override public long feeCents(Duration stay, SpotType spotType) {
        long minutes = Math.max(1, stay.toMinutes());
        long hours = (minutes + 59) / 60;                  // round up
        long total = 0;
        for (long h = 0; h < hours; h++) {
            total += h < firstHours.size() ? firstHours.get((int) h) : laterHourCents;
        }
        return total;
```

```
    }
}
```

```java
// File: lld/ch14/parkinglot/Ticketing.java
package lld.ch14.parkinglot;

import java.time.Instant;

enum TicketStatus { ACTIVE, PAID, EXITED }

/** The ticket points at the real spot, so the exit can find and free it. */
final class Ticket {
    private final String id;
    private final Vehicle vehicle;
    private final ParkingSpot spot;
    private final Instant entryTime;
    private TicketStatus status = TicketStatus.ACTIVE;
    private long paidCents;

    Ticket(String id, Vehicle vehicle, ParkingSpot spot, Instant entryTime) {
        this.id = id;
        this.vehicle = vehicle;
        this.spot = spot;
        this.entryTime = entryTime;
    }

    /** Only one payment can ever succeed, even if the portal and the exit panel race. */
    synchronized boolean markPaid(long cents) {
        if (status != TicketStatus.ACTIVE) return false;
        status = TicketStatus.PAID;
        paidCents = cents;
        return true;
    }

    synchronized boolean markExited() {
        if (status != TicketStatus.PAID) return false;
        status = TicketStatus.EXITED;
        return true;
    }

    synchronized TicketStatus status() { return status; }
    synchronized long paidCents() { return paidCents; }
    String id() { return id; }
    Vehicle vehicle() { return vehicle; }
    ParkingSpot spot() { return spot; }
    Instant entryTime() { return entryTime; }
}

/** How a customer pays. Cash and card are two implementations. */
interface PaymentMethod {
    void pay(long cents);
    String describe();
}

record CashPayment(long tenderedCents) implements PaymentMethod {
    @Override public void pay(long cents) {
        if (tenderedCents < cents) throw new IllegalArgumentException("not enough cash");
    }
    @Override public String describe() {
        return "cash " + Money.fmt(tenderedCents);
    }
}

record CardPayment(String last4) implements PaymentMethod {
    @Override public void pay(long cents) { /* a real system calls the card gateway here */ }
    @Override public String describe() { return "card ****" + last4; }
}

final class Money {
    private Money() {}
    static String fmt(long cents) { return "$%d.%02d".formatted(cents / 100, cents % 100); }
}
```

```java
// File: lld/ch14/parkinglot/ParkingLot.java
package lld.ch14.parkinglot;

import java.time.Clock;
import java.time.Duration;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicLong;

/**
 * The lot ties floors, tickets, pricing and allocation together.
 * It is NOT a hard-coded Singleton: the application creates one and injects it
 * into the panels, which keeps tests able to build as many lots as they like.
 */
final class ParkingLot {
    private final String name;
    private final List<ParkingFloor> floors;
    private final Clock clock;
    private final Map<String, Ticket> tickets = new ConcurrentHashMap<>();
    private final Map<String, String> ticketByPlate = new ConcurrentHashMap<>();
    private final AtomicLong nextTicket = new AtomicLong(1);
    private volatile SpotAllocationStrategy allocation;
    private volatile PricingStrategy pricing;

    ParkingLot(String name, List<ParkingFloor> floors, Clock clock,
               SpotAllocationStrategy allocation, PricingStrategy pricing) {
        this.name = name;
        this.floors = List.copyOf(floors);
        this.clock = clock;
        this.allocation = allocation;
        this.pricing = pricing;
    }

    /** Entry: find a spot and issue a ticket. Empty result means "full for this vehicle". */
    Optional<Ticket> admit(Vehicle v) {
        String placeholder = "pending";
        if (ticketByPlate.putIfAbsent(v.plate(), placeholder) != null) {
            throw new IllegalStateException(v.plate() + " is already inside");
        }
        Optional<ParkingSpot> spot = allocation.allocate(floors, v);
        if (spot.isEmpty()) {
            ticketByPlate.remove(v.plate());
            return Optional.empty();
        }
        Ticket t = new Ticket("T" + nextTicket.getAndIncrement(), v, spot.get(), clock.instant());
        tickets.put(t.id(), t);
        ticketByPlate.put(v.plate(), t.id());
        return Optional.of(t);
    }

    long amountDue(String ticketId) {
        Ticket t = ticket(ticketId);
        if (t.status() != TicketStatus.ACTIVE) return 0;            // already paid (e.g. at the portal)
        Duration stay = Duration.between(t.entryTime(), clock.instant());
        return pricing.feeCents(stay, t.spot().type());
    }

    /** Pays the ticket once. Returns the amount charged (0 if it was already paid). */
    long pay(String ticketId, PaymentMethod method) {
        Ticket t = ticket(ticketId);
        synchronized (t) {                // per-ticket lock: portal and exit panel cannot both charge
            long due = amountDue(ticketId);
            if (due == 0) return 0;
            method.pay(due);              // throws if the payment fails: ticket stays ACTIVE
            t.markPaid(due);
            return due;
        }
    }
```

```
    /** Exit: only a PAID ticket opens the barrier. Frees the spot. */
    boolean exit(String ticketId) {
        Ticket t = ticket(ticketId);
        if (!t.markExited()) return false;
        floors.get(t.spot().floor() - 1).release(t.spot());
        tickets.remove(ticketId);
        ticketByPlate.remove(t.vehicle().plate());
        return true;
    }

    boolean isFullFor(VehicleType type) {
        return floors.stream().noneMatch(f -> type.fits().stream().anyMatch(s -> f.freeCount(s) > 0));
    }

    void setPricing(PricingStrategy p) { this.pricing = p; }                // admin changes the rate
    void setAllocation(SpotAllocationStrategy a) { this.allocation = a; }
    List<ParkingFloor> floors() { return floors; }
    String name() { return name; }

    Ticket ticket(String id) {
        Ticket t = tickets.get(id);
        if (t == null) throw new IllegalArgumentException("unknown ticket " + id);
        return t;
    }
}
```

```
// File: lld/ch14/parkinglot/Panels.java
package lld.ch14.parkinglot;

/** Panels hold no rules. They talk to the lot and print what happened. */
class EntrancePanel {
    private final String id;
    private final ParkingLot lot;

    EntrancePanel(String id, ParkingLot lot) { this.id = id; this.lot = lot; }

    String issueTicket(Vehicle v) {
        return lot.admit(v)
                .map(t -> "[%s] %s -> ticket %s, spot %s".formatted(id, v.plate(), t.id(), t.spot()))
                .orElse("[%s] %s -> SORRY, LOT FULL for %s".formatted(id, v.plate(), v.type()));
    }
}

class ExitPanel {
    private final String id;
    private final ParkingLot lot;

    ExitPanel(String id, ParkingLot lot) { this.id = id; this.lot = lot; }

    String scanPayAndExit(String ticketId, PaymentMethod method) {
        long charged = lot.pay(ticketId, method);
        boolean open = lot.exit(ticketId);
        String paid = charged == 0 ? "already paid" : "paid " + Money.fmt(charged) + " by " + method.describe();
        return "[%s] %s: %s, barrier %s".formatted(id, ticketId, paid, open ? "OPEN" : "CLOSED");
    }
}

/** On each floor: lets the customer pay early so the exit is free. */
class CustomerInfoPortal {
    private final ParkingLot lot;

    CustomerInfoPortal(ParkingLot lot) { this.lot = lot; }

    String pay(String ticketId, PaymentMethod method) {
        long due = lot.amountDue(ticketId);
        long charged = lot.pay(ticketId, method);
        return "[portal] %s due %s, charged %s".formatted(ticketId, Money.fmt(due), Money.fmt(charged));
    }
}
```

```java
// File: lld/ch14/parkinglot/Main.java
package lld.ch14.parkinglot;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {

    /** A clock we can move forward, so the demo is deterministic. */
    static final class TestClock extends Clock {
        private volatile Instant now = Instant.parse("2026-01-01T09:00:00Z");
        void advance(Duration d) { now = now.plus(d); }
        @Override public Instant instant() { return now; }
        @Override public ZoneId getZone() { return ZoneOffset.UTC; }
        @Override public Clock withZone(ZoneId zone) { return this; }
    }

    public static void main(String[] args) throws Exception {
        TestClock clock = new TestClock();
        ParkingFloor f1 = new ParkingFloor(1, Map.of(SpotType.COMPACT, 2, SpotType.LARGE, 1,
                SpotType.MOTORCYCLE, 1, SpotType.HANDICAPPED, 1, SpotType.ELECTRIC, 1));
        ParkingFloor f2 = new ParkingFloor(2, Map.of(SpotType.COMPACT, 3, SpotType.LARGE, 1));
        DisplayBoard board1 = new DisplayBoard(), board2 = new DisplayBoard();
        f1.addListener(board1);
        f2.addListener(board2);

        ParkingLot lot = new ParkingLot("City Mall", List.of(f1, f2), clock,
                new NearestFloorFirst(), HourlyTieredPricing.standard());
        EntrancePanel gateA = new EntrancePanel("entry-A", lot);
        ExitPanel exitA = new ExitPanel("exit-A", lot);
        CustomerInfoPortal portal = new CustomerInfoPortal(lot);

        System.out.println("== Arrivals ==");
        System.out.println(gateA.issueTicket(new Vehicle("CAR-1", VehicleType.CAR)));
        System.out.println(gateA.issueTicket(new Vehicle("EV-1", VehicleType.ELECTRIC_CAR)));
        System.out.println(gateA.issueTicket(new Vehicle("BIKE-1", VehicleType.MOTORCYCLE)));
        System.out.println(gateA.issueTicket(new Vehicle("CAR-2", VehicleType.CAR, true)));
        System.out.println(gateA.issueTicket(new Vehicle("TRUCK-1", VehicleType.TRUCK)));
        System.out.println(gateA.issueTicket(new Vehicle("VAN-1", VehicleType.VAN)));
        System.out.println(gateA.issueTicket(new Vehicle("TRUCK-2", VehicleType.TRUCK)));
        System.out.println("Floor 1 board: " + board1.render());
        System.out.println("Floor 2 board: " + board2.render());

        System.out.println("\n== EV charging at the spot ==");
        ChargingPanel panel = lot.ticket("T2").spot().chargingPanel().orElseThrow();
        System.out.println("EV-1 charged 20 kWh, paid " + Money.fmt(panel.chargeAndPay(20, new
CardPayment("4242"))));

        System.out.println("\n== Departures ==");
        clock.advance(Duration.ofMinutes(50));
        System.out.println(exitA.scanPayAndExit("T1", new CashPayment(500)));          // 1 hour
        clock.advance(Duration.ofHours(4));                                             // T2 now 4h50m
        System.out.println(portal.pay("T2", new CardPayment("1111")));                  // 5 hours
        System.out.println(exitA.scanPayAndExit("T2", new CardPayment("1111")));
        System.out.println("Unpaid exit attempt: barrier opens? " + lot.exit("T3"));

        System.out.println("\n== Admin changes the rate: flat $5/hour ==");
        lot.setPricing((stay, type) -> 500 * Math.max(1, (stay.toMinutes() + 59) / 60));
        System.out.println(exitA.scanPayAndExit("T3", new CardPayment("9999")));       // 4h50m -> 5h

        System.out.println("\n== 20 cars race for the remaining CAR spots ==");
```

```
        System.out.println("free compact before: F1=" + f1.freeCount(SpotType.COMPACT)
                + " F2=" + f2.freeCount(SpotType.COMPACT) + ", large: F1=" + f1.freeCount(SpotType.LARGE)
                + " F2=" + f2.freeCount(SpotType.LARGE));
        ConcurrentLinkedQueue<String> winners = new ConcurrentLinkedQueue<>();
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 20; i++) {
                String plate = "RACE-" + i;
                pool.submit(() -> {
                    start.await();
                    lot.admit(new Vehicle(plate, VehicleType.CAR)).ifPresent(t -> winners.add(t.spot().id()));
                    return null;
                });
            }
            start.countDown();
        } // close() waits for all tasks
        System.out.println("cars admitted: " + winners.size()
                + ", distinct spots: " + winners.stream().distinct().count());
        System.out.println("full for cars now? " + lot.isFullFor(VehicleType.CAR));
        System.out.println(gateA.issueTicket(new Vehicle("LATE-CAR", VehicleType.CAR)));
        System.out.println("Floor 2 board: " + board2.render());
    }
}
```

How the code works, part by part

- `Vehicle.preferredSpots()` returns the spot types in order of preference. An accessibility permit puts `HANDICAPPED` first; nobody else can ever get a handicapped spot.
- `ParkingFloor` keeps a `ConcurrentSkipListSet` (a thread-safe set from `java.util.concurrent` that keeps its elements sorted) of **free** spots per type, sorted by id. `pollFirst()` removes and returns the lowest id **atomically**: if 20 threads call it at once, each spot goes to exactly one of them. The lowest id stands for "nearest to the entrance".
- `ParkingSpot.tryOccupy` is a second safety net: an `AtomicReference.compareAndSet(null, v)` succeeds only if the spot was empty. This is **CAS** (compare-and-set): "change the value to *v* only if it is still *null*", done as one atomic step without any lock.
- `NearestFloorFirst` and `LeastBusyFloorFirst` are two interchangeable allocation strategies.
- `HourlyTieredPricing.standard()` encodes $4 / $3.50 / $3.50 / then $2.50, rounding a started hour up.
- `ParkingLot.admit` also reserves the **plate** with `putIfAbsent` (adds a key to a map only if it is not there yet, atomically), so the same car cannot be admitted twice.
- `ParkingLot.pay` locks **one ticket** (not the whole lot; a **lock** lets only one thread at a time run a piece of code, and Java's `synchronized` keyword is the built-in lock that every object carries), so payments for different tickets run in parallel while the portal and the exit panel cannot both charge the same ticket.
- `Main` uses a `TestClock` (a subclass of `java.time.Clock`) to jump forward in time, then races 20 threads for the last 5 car spots.

Output:

```
== Arrivals ==
[entry-A] CAR-1 -> ticket T1, spot F1-C01(COMPACT)
[entry-A] EV-1 -> ticket T2, spot F1-E01(ELECTRIC)
[entry-A] BIKE-1 -> ticket T3, spot F1-M01(MOTORCYCLE)
[entry-A] CAR-2 -> ticket T4, spot F1-H01(HANDICAPPED)
[entry-A] TRUCK-1 -> ticket T5, spot F1-L01(LARGE)
[entry-A] VAN-1 -> ticket T6, spot F2-L01(LARGE)
[entry-A] TRUCK-2 -> SORRY, LOT FULL for TRUCK
Floor 1 board: MOTORCYCLE=FULL COMPACT=1 LARGE=FULL HANDICAPPED=FULL ELECTRIC=FULL
Floor 2 board: COMPACT=3 LARGE=FULL

== EV charging at the spot ==
EV-1 charged 20 kWh, paid $6.00

== Departures ==
[exit-A] T1: paid $4.00 by cash $5.00, barrier OPEN
[portal] T2 due $16.00, charged $16.00
[exit-A] T2: already paid, barrier OPEN
Unpaid exit attempt: barrier opens? false

== Admin changes the rate: flat $5/hour ==
[exit-A] T3: paid $25.00 by card ****9999, barrier OPEN

== 20 cars race for the remaining CAR spots ==
free compact before: F1=2 F2=3, large: F1=0 F2=0
cars admitted: 5, distinct spots: 5
full for cars now? true
[entry-A] LATE-CAR -> SORRY, LOT FULL for CAR
Floor 2 board: COMPACT=FULL LARGE=FULL
```

Read the output: the truck is turned away because the only large spots are taken; T2 stayed 4 h 50 min, which rounds up to 5 hours = $4 + $3.50 + $3.50 + $2.50 + $2.50 = **$16.00**, paid at the portal, so the exit charges nothing; an unpaid ticket cannot open the barrier; after the admin switches to a flat $5/hour, the motorcycle pays $25 for 5 hours; 20 racing cars get exactly 5 distinct spots.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Two gates want the last compact spot	Both get it (double booking)	`pollFirst()` is atomic; `tryOccupy` uses CAS as a second check
Portal and exit panel pay the same ticket at once	Customer charged twice	`synchronized (ticket)` around price-charge-mark; a PAID ticket costs 0
Same plate enters twice (cloned ticket, sensor glitch)	Two tickets for one car	`ticketByPlate.putIfAbsent`
Payment fails (card declined)	Ticket marked paid anyway	`method.pay` throws **before** `markPaid`; ticket stays ACTIVE
Car leaves without paying	Spot never freed or fee lost	`exit` only succeeds from PAID; the barrier stays closed
Customer pays at the portal and then stays 3 more hours	Free parking	Real lots give a **grace period** (e.g. 15 min) after payment; add `paidAt` and re-price after the grace period
Display board shows a slightly old number	Two notifications arrive out of order	Acceptable for a display; if exactness matters, have the board **pull** `freeCount` when it renders
Lost ticket	No entry time	Charge a daily maximum, or look up the plate (the plate map already exists)
Stay of 0 minutes	Fee of $0	Minimum of one hour (`Math.max(1, minutes)`)
Lock scope	One global lock would serialise all gates	Locks are per spot (CAS) and per ticket; no global lock

`ConcurrentSkipListSet.size()` is not constant-time (it walks the whole set instead of reading a stored number) and is only an estimate while other threads change the set; that is fine for a display board, but never use it to decide whether a spot is free — use `pollFirst()`.

Extensions & follow-up questions

1. **"What if pricing changes (weekend rate, per spot type, surge)?"** — Write a new `PricingStrategy` (for example `WeekendPricing`, or a decorator (a wrapper object with the same interface that adds behaviour around another object) that multiplies another strategy for large spots) and call `setPricing`. No other class changes.
2. **"Two cars arrive at the same instant — what happens?"** — Spot handout is atomic (`pollFirst` + CAS). Show the 20-thread race in the output.
3. **"Add reservations."** — Add a `Reservation(plate, spotType, from, to)` and a reserved set per floor; the allocation strategy checks the reservation first. Hold the spot with a timeout (release it if the car does not arrive within, say, 30 minutes).
4. **"Add monthly passes / season tickets."** — A `PricingStrategy` that returns 0 for plates with a valid pass, or a `PassHolderPricing` decorator around the normal pricing.
5. **"Number-plate recognition (ticketless)."** — The entrance camera becomes another kind of entrance panel that calls the same `admit`; the ticket id is keyed by plate. No core class changes.
6. **"How would you store this in a database?"** — Tables: `floor`, `spot(id, floor_id, type, occupied_by)`, `ticket(id, plate, spot_id, entry_time, status, paid_cents)`, `payment(id, ticket_id, method, amount)`. Claim a spot with a conditional update (`UPDATE spot SET occupied_by=? WHERE id=? AND occupied_by IS NULL`) — the database equivalent of our CAS. Candidates who have been asked this report that interviewers like to see **cardinalities** (how many of one thing can link to another; `1..*` means one or more) (a floor has 1..* spots, a spot holds 0..1 vehicles, a ticket has 1 spot) and then add a new entity to see how you rewire the model.
7. **"Many lots in a city?"** — A `ParkingLot` per site plus a `ParkingNetwork` that routes drivers to the lot with free spots. This is another reason not to make the lot a Singleton.

8. **"EV idle fees?"** — Add a charging session to `ChargingPanel` (start, end, kWh) and charge a per-minute fee after charging completes.

Real-world example

Many modern malls and airports use **parking guidance systems**: a sensor above each bay drives a small green/red light, and the boards at each entrance and floor show the free count per level — exactly our `DisplayBoard` observers fed by spot changes. Airports and many city garages increasingly use **automatic number-plate recognition** instead of paper tickets, and apps such as SpotHero and ParkMobile let drivers pre-book a spot, which is the "reservations" extension. Tesla Superchargers charge **idle fees** when a car stays plugged in after charging finishes — the EV extension above.

In the interview

Plan for 45 minutes

Minutes	What you do
0–5	Clarifying questions; write the 12 requirements and an out-of-scope list
5–10	Actors and use cases (take ticket, pay, exit, admin)
10–20	Stages 1–2: vehicle/spot fit rule, floors, board. Say "the fit rule lives on the vehicle"
20–25	Stage 3–4: ticket holds the spot; lot with admit/pay/exit; Strategy for pricing
25–38	Code `admit`, `tryPark`, `pay`, `exit`, and the tiered pricing
38–45	Concurrency (atomic spot handout, per-ticket lock), extensions

> **30-second answer:** "I'll model vehicles and spot types as enums with the fit rule on the vehicle, floors that keep a concurrent sorted set of free spots per type, and a lot that admits, prices and releases. Tickets point at the real spot. Pricing and spot-finding are Strategies so they can change without touching the lot. Handing out a spot is atomic — `pollFirst` plus a CAS on the spot — so two gates can never get the same spot, and payment locks only the one ticket."

Follow-ups with answers

- *"Why not a Singleton?"* — There is one lot in production, but a Singleton is a hidden global: it makes testing two lots impossible. I create one instance and inject it. If the interviewer insists, a lazy holder-class Singleton (the single instance lives in a nested class that Java loads only on first use, which the JVM makes thread-safe for free) is thread-safe and simple.
- *"Why is the fit rule on the vehicle?"* — Because it changes when vehicle types change; putting it there keeps the lot closed for modification (OCP).
- *"How do you make finding a spot fast?"* — Keep only the free spots, indexed by type, in a sorted set; taking one is O(log n) and needs no scan.
- *"What if the display board and the real count disagree?"* — The board is informational; the atomic `pollFirst` is the source of truth. The board can pull the count on render if needed.

Self-check

1. **Q:** Why must the ticket store a reference to the spot? **A:** So the exit can free exactly that spot; a count cannot tell you which spot to release.
2. **Q:** What makes spot assignment thread-safe here? **A:** `ConcurrentSkipListSet.pollFirst()` removes a spot for exactly one caller, and `AtomicReference.compareAndSet` on the spot double-checks it.
3. **Q:** Where would you add a weekend price? **A:** A new `PricingStrategy` implementation; set it with `setPricing`. Nothing else changes.
4. **Q:** Why did we replace the five spot subclasses with an enum? **A:** They had no behaviour of their own; an enum field does the same job and a new type is one constant (composition over inheritance).
5. **Q:** A customer paid at the portal. What does the exit panel charge? **A:** Nothing: `amountDue` returns 0 for a PAID ticket, and `exit` moves it from PAID to EXITED.

6. **Q:** How does the design stop a car from being admitted twice? **A:** `ticketByPlate.putIfAbsent(plate, …)` reserves the plate before allocating.

TL;DR

- Fit rule on the **vehicle**, spot type as an **enum**, free spots per type in a **concurrent sorted set**.
- **Ticket → spot** reference makes the exit possible; status goes ACTIVE → PAID → EXITED.
- **Strategy** for pricing and spot-finding, **Factory** for electric spots, **Observer** for boards; inject the lot instead of a Singleton.
- Concurrency: atomic `pollFirst` + CAS for spots, per-ticket lock for payment, `putIfAbsent` for plates.

14.2 Design a Vending Machine



Figure — A vending machine: slots with codes, a keypad, a coin/note slot and a tray at the bottom (course diagram)

A **vending machine** is an unattended shop: you key in a slot code, pay, and pull your item out of the tray at the bottom. Because everyone has used one, the interviewer can skip the domain and look at one thing only: **which actions are legal at which moment**. Taking a coin when nothing has been chosen is a bug; handing out two items for one payment is a bug. No amount of tidy code elsewhere makes up for getting that order wrong.

Problem & clarifying questions

Question	Assumed answer
Select first, or pay first?	Select a slot code first, then pay.
Which money?	A fixed set of coins and notes (5¢, 10¢, 25¢, $1, $5). Anything else is returned.
Card or contactless?	Out of scope (an extension).
Can it give change?	Yes. If it cannot make **exact** change, it refuses the sale and returns everything.
Can the customer cancel?	Any time before the product is dispensed, with a full refund.
One customer at a time?	Yes (one physical machine).
Who restocks?	An operator: restocks slots, loads coins for change, collects the takings.

Requirements

Functional (the course's twelve)

1. The machine has many **slots**. A slot has a code (`A1`, `B2`, …), holds items of a **single** product, and tracks its remaining **count**.
2. A product has a **name** and a **price in cents**.
3. Order of use: **choose a slot, then pay**.
4. Only a **known set of coins and notes** is accepted; any other piece is given back.
5. After **every** customer action the **display** says what to do next.
6. When the amount paid covers the price, the machine **releases the item and the change**.
7. When **exact change is impossible**, the sale is **refused** and every cent paid is **returned**.
8. Until the item drops, the customer may **cancel** and receive a **full refund**.
9. A slot whose count is **zero cannot be chosen**.
10. One payment buys **exactly one** item, never two.
11. An **operator** refills slots, adds coins for change, and removes the takings.
12. After every sale, cancel and refusal, **both** the item counts and the cash counts must be **exactly right**.

Non-functional

- **No illegal transitions**: every button must have a defined answer in every situation.
- **Exact money**: integer cents only — no floating-point.
- **Extensible**: a new payment device (card reader) or a new state (maintenance) should not require editing every existing class.

Use cases

Actors: **Customer** (selects, pays, cancels, takes the product), **Operator** (restocks, loads change, collects money), **System** (decides which actions are allowed now, works out change, shows messages).

Top use cases: select a product · insert money · dispense · collect (take from the tray) · cancel · restock a slot · load change and collect takings.

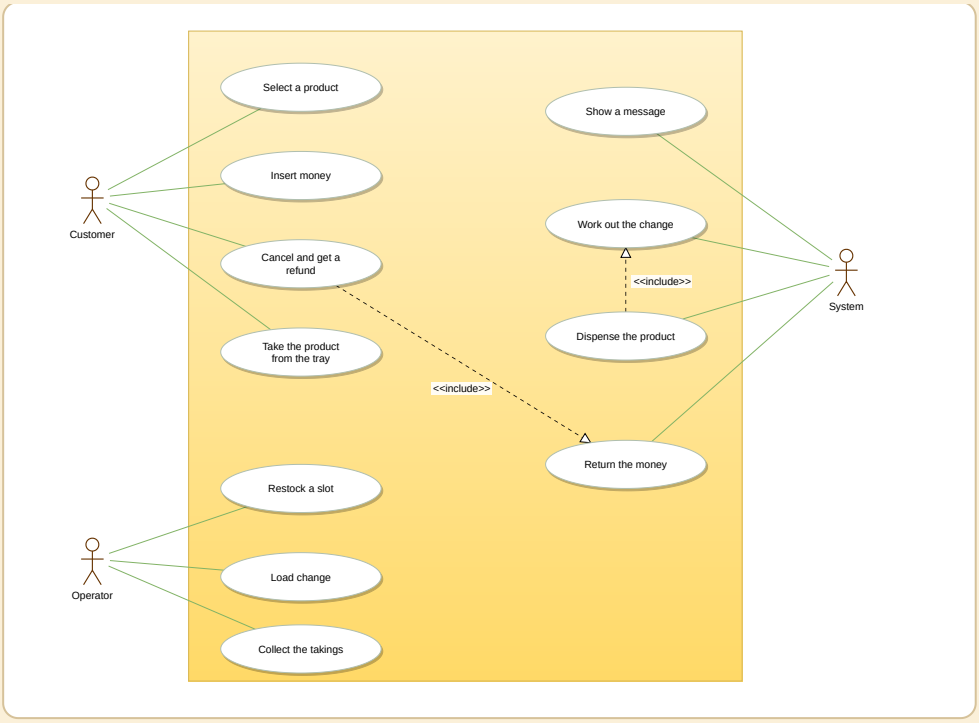


Figure — Use case diagram of the vending machine: Customer, Operator and System (course diagram)

In the figure above, "dispense" is a System use case, not a customer one: the customer only *causes* it by paying enough.

Core entities

Class	Responsibility
`Product` (record)	Name and price in cents. Describes *what* is sold.
`Slot`	One physical column: a code, one product, and the **count** left.
`Inventory`	All slots, looked up by code.
`Denomination` (enum)	The accepted coins and notes and their values in cents.
`CashBox`	Counts pieces by value; plans exact change; reports when it cannot.
`VendingState` (interface)	One method per customer button: `select`, `insert`, `cancel`, `take`.
`IdleState`, `SelectedState`, `HasMoneyState`, `DispensingState`, `OutOfStockState`	The situations the machine can be in (the course has the first four, calling `HasMoneyState` `PaidState`); each answers every button and names the next state.
`Display`	The message the customer currently sees. In our code, `VendingMachine.show(...)` plays this part.
`Keypad`, `CashSlot`, `Dispenser`	The hardware the customer touches; each calls the machine and holds no rules. In our code they are folded into the machine's public methods.
`Operator`	Restocks, loads change, collects takings. In our code: `restock`, `loadChange`, `collectTakings`.
`VendingMachine`	Holds the parts, the current state, the selection and the balance; forwards each button to the state. Also exposes operator actions.

Why is the count on the **slot**, not on the product? `Product` answers "what is it?"; `Slot` is the physical column it sits in. The same product may be loaded into two columns (the course even allows different prices), and emptying one column must leave the other untouched. That is the first real design decision in this question.

Class diagram

The course builds the machine in five stages:

Stage	What it adds	What works after it
1	`Product`, `Slot`, `Inventory`	Look up a code and learn what it holds and whether it is empty
2	`Coin`, `Note`, `CashBox`	Decide whether a given amount of change can be paid exactly
3	A class for each state	Each button press gets a defined reaction in each situation
4	`Display` and `VendingMachine`	A complete sale, from choosing to picking up the item
5	`Keypad`, `CashSlot`, `Dispenser`, `Operator`	Real hardware drives the same flow

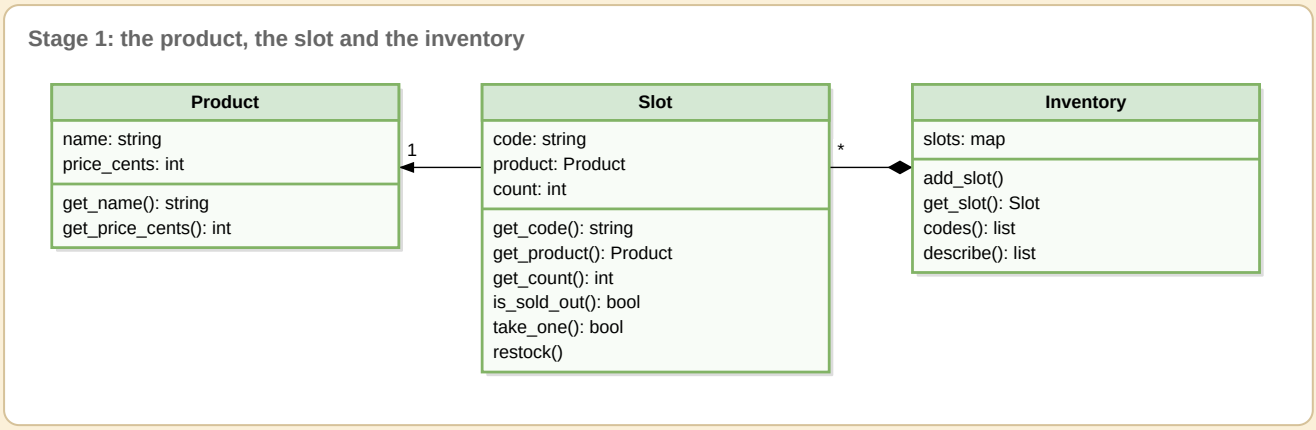


Figure — Stage 1: Product, Slot (association to Product) and Inventory (composition of Slots) (course diagram)

Stage 1. Requirements 1, 2 and 9. `Slot → Product` is an **association**: a product can appear in several slots, so removing a slot must leave the product alone. `Inventory ◆ Slot` is **composition** (filled diamond): slots are parts of the machine and do not outlive it.

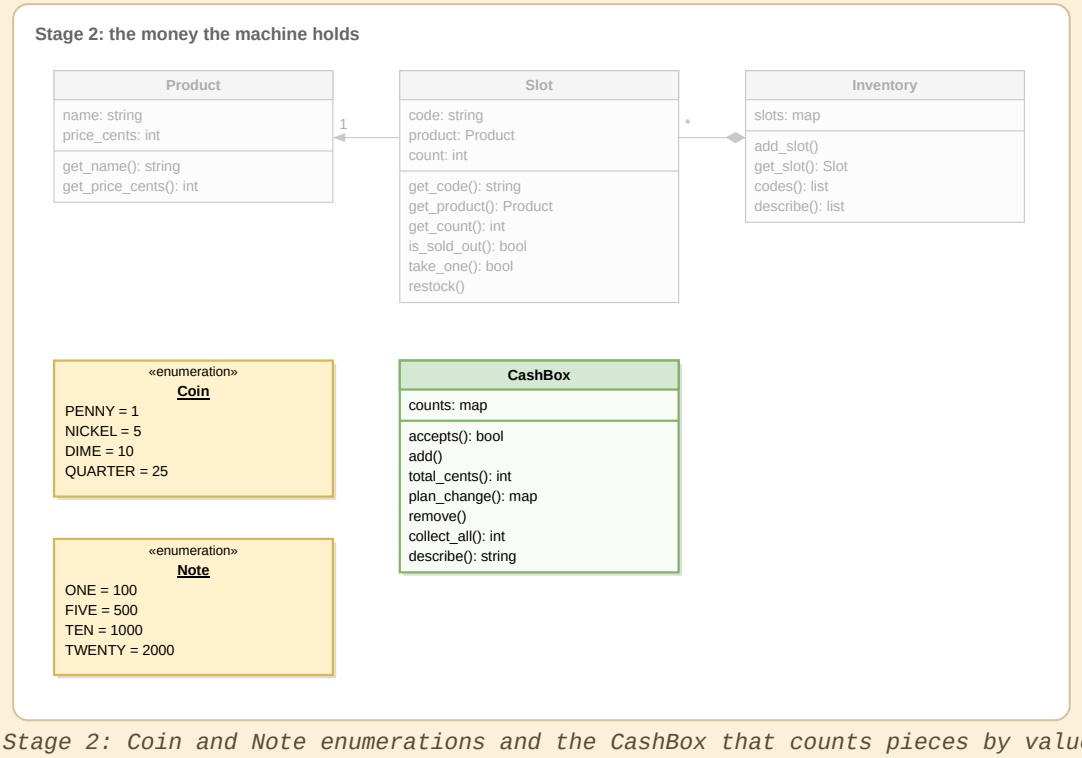


Figure — Stage 2: Coin and Note enumerations and the CashBox that counts pieces by value (course diagram)

Stage 2. Requirements 4, 6 and 7. All amounts are whole cents, so rounding errors cannot happen. `Coin` and `Note` are enumerations (fixed lists of named values; our code merges them into one `Denomination` enum), and fixed sets like these have no relationship arrows. The choice to defend here: **a piece goes into the cash box the moment it is inserted**, not into a holding area. Those pieces are then available for change, and a refund becomes nothing more than "pay out exact change equal to what was inserted". The box stores a **count per value**; which physical quarter comes back does not matter. Its change planner either returns an exact plan or says "impossible"; it never approximates, and that is what makes requirement 7 implementable.

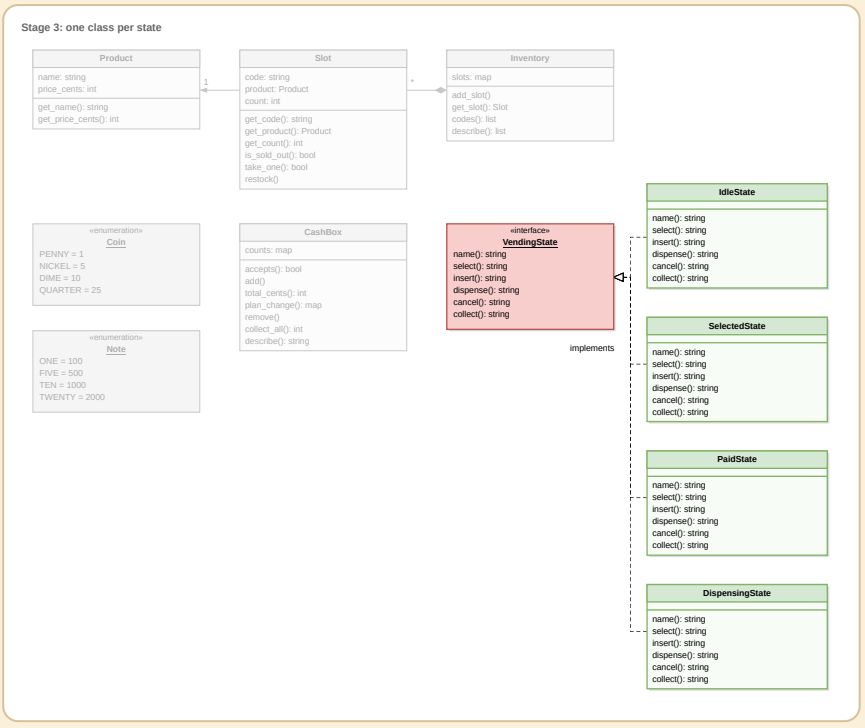


Figure — Stage 3: one class per state, all implementing the VendingState interface (course diagram)

Stage 3 — the heart of the question. Requirements 3, 8 and 10. The obvious first attempt is one class with a few boolean flags and nested `if`s; that version is exactly where double dispensing and swallowed coins come from. Use the **State pattern** instead: each situation becomes a class, and all of them implement one interface on which **each customer button is a method**. A state class that forgets a button does not compile. Picture the states as a **grid**, buttons down the side and situations across the top: the design is finished when every cell holds a response. The state classes carry no fields, and none of them checks which state is active; each handles its own buttons and hands over to the next state, so a new state can be added without editing the existing ones.

The course draws **four** states: `IdleState`, `SelectedState`, `PaidState` and `DispensingState`. Our code keeps those four (its `HasMoneyState` is the course's `PaidState`: some money is in, not yet enough) and adds a fifth, `OutOfStockState`, so that an empty machine refuses every customer button until an operator restocks it.

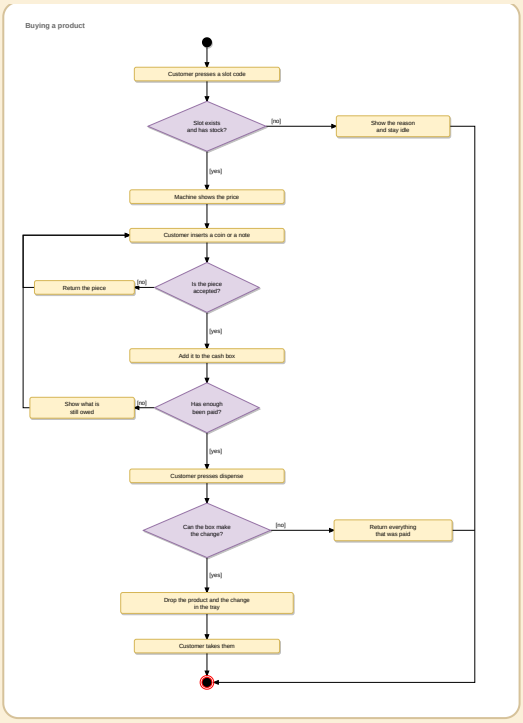


Figure — Activity diagram "Buying a product": select, check stock, insert money until enough, check change, dispense, take (course diagram)

Each decision diamond in the activity diagram above corresponds to one state choosing whether to accept an action.

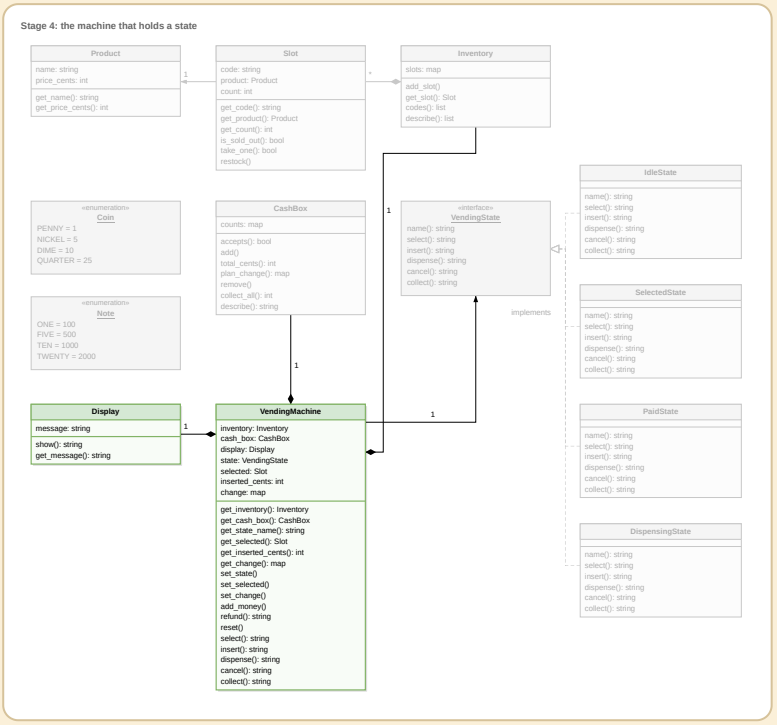


Figure — Stage 4: VendingMachine owns its display, cash box and inventory, and points at the current VendingState (course diagram)

Stage 4. Requirements 5, 6, 8 and 12. `VendingMachine` remembers the chosen slot and the money inserted so far, and passes every button press to its current state. It **owns** its `Display`, `CashBox` and `Inventory` (composition) and merely **refers to** the current state object (association). Look at what the class does *not* contain: any `if` that tests which state is active. The refund method shows why stage 2

pays off: since inserted money is already in the box, refunding is just a request for exact change equal to the balance.

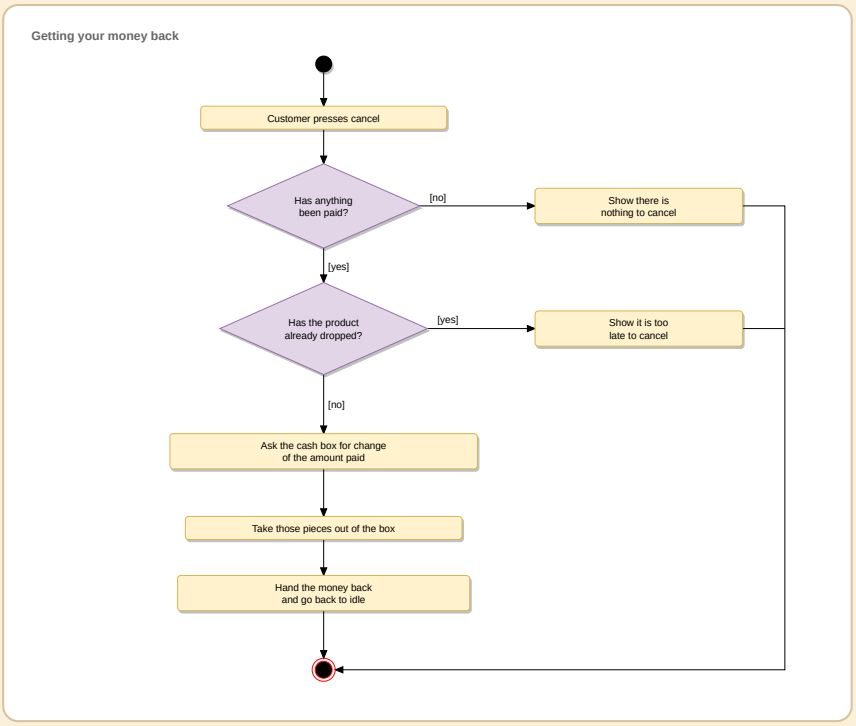


Figure — Activity diagram "Getting your money back": cancel and "cannot make change" end in the same refund path (course diagram)

As the figure above shows, a cancel and a "no exact change" refusal finish identically, so the design needs a **single** refund method.

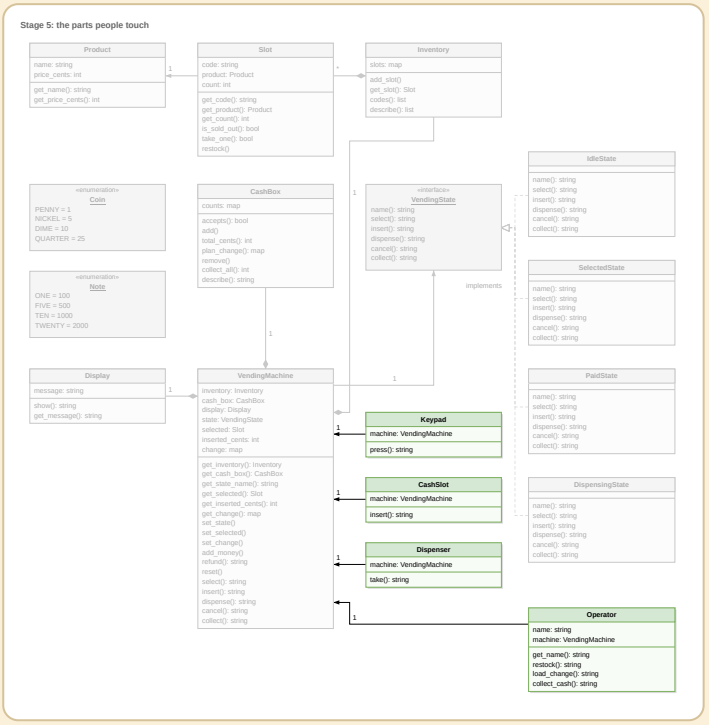


Figure — Stage 5: Keypad, CashSlot, Dispenser and Operator, all pointing at the machine (course diagram)

Stage 5. Requirements 3, 4 and 11. All four new classes hold references **to** the machine, never the reverse: the keypad calls the machine, but the machine has no idea a keypad exists. Swapping the keypad for a touch screen, or the coin slot for a card reader, therefore touches no other class. The `Dispenser` is the tray; pulling the item out of it is what finishes a sale, which is why `take` is a customer button on the state interface. `Operator` is intentionally kept **off** that interface: refilling,

loading change and emptying the box are not customer actions and are permitted whatever state the machine is in (if they were on the interface, every state class would have to answer them).

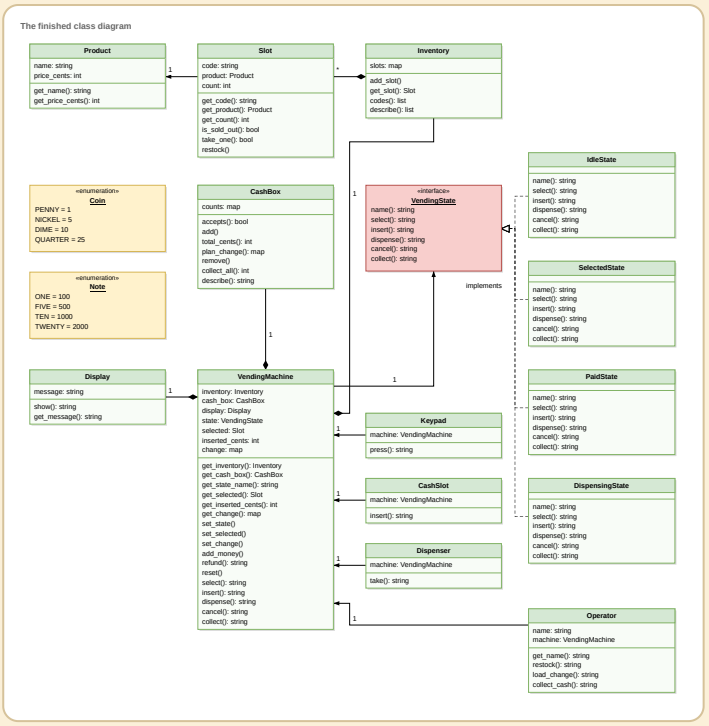
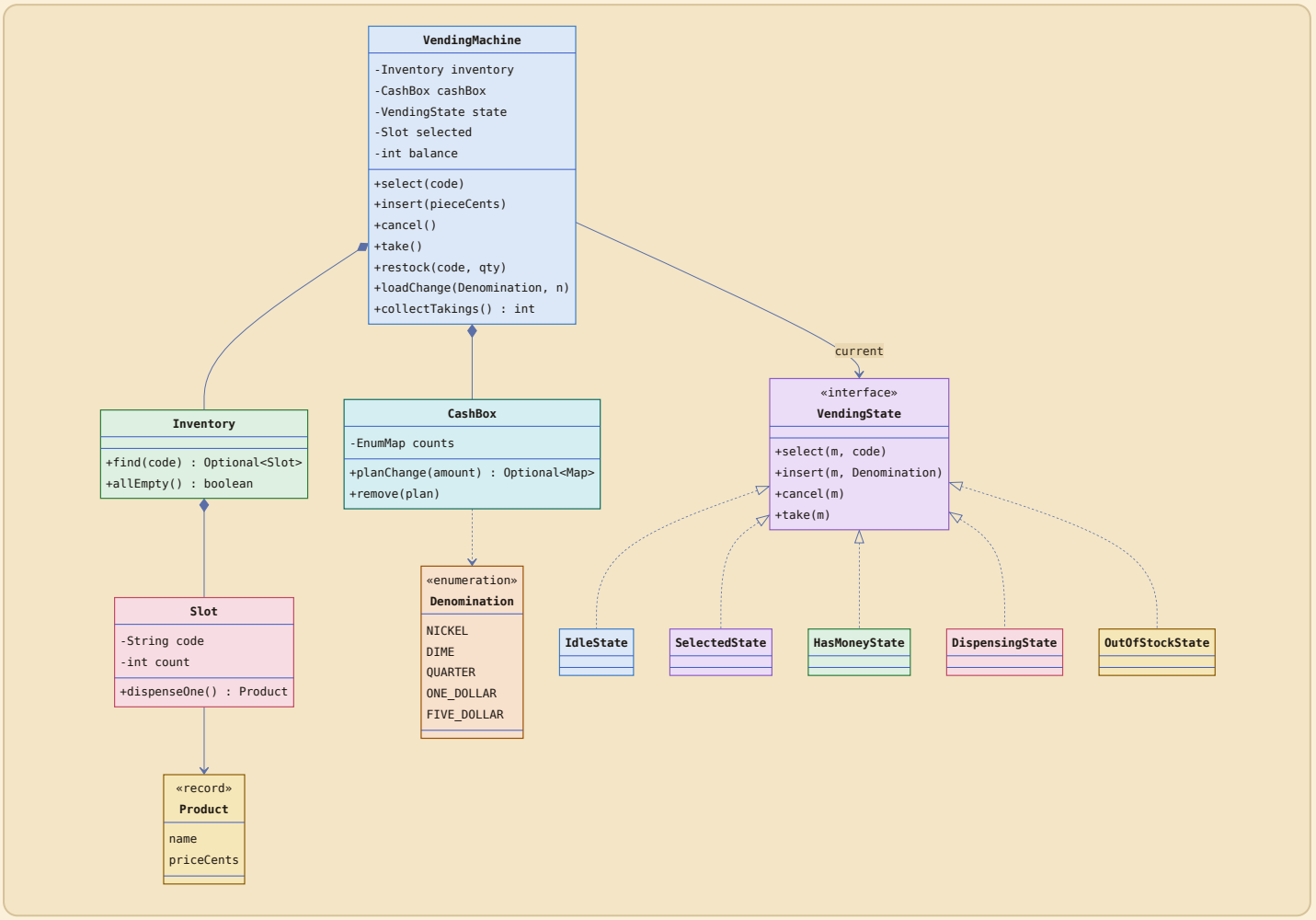


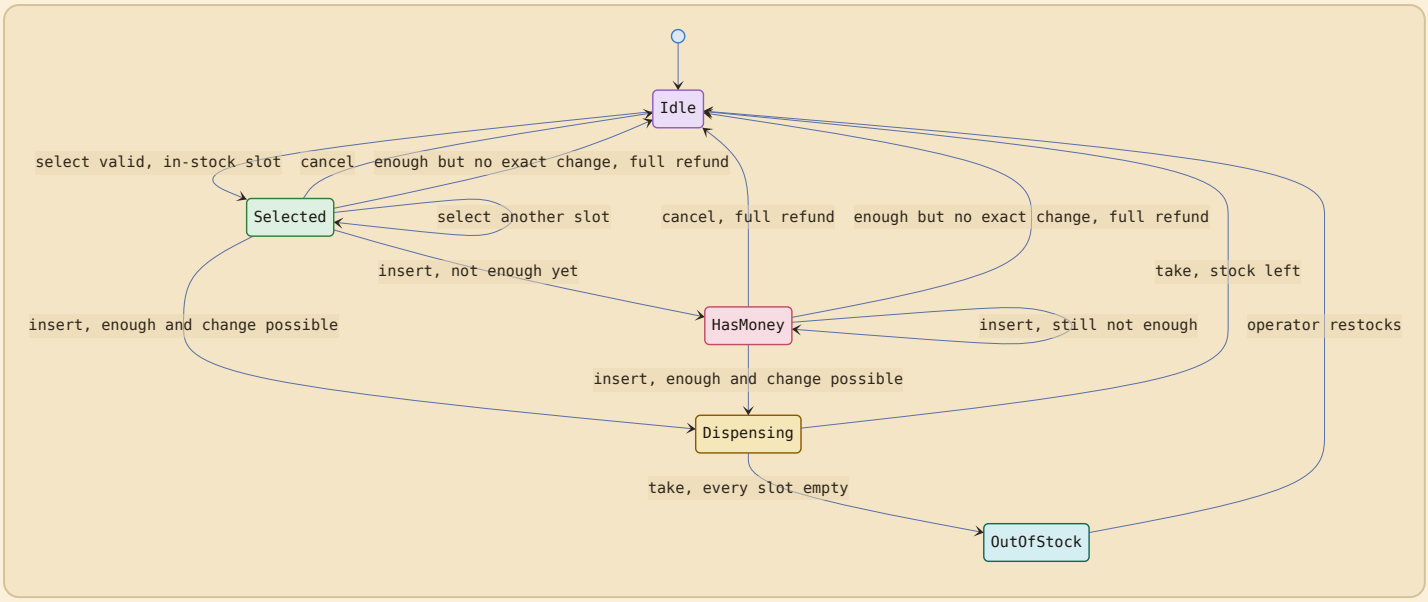
Figure — The finished class diagram of the vending machine (course diagram)

Class diagram of **our Java code** (hardware classes folded into the machine's public methods to keep the code short):



Key flows

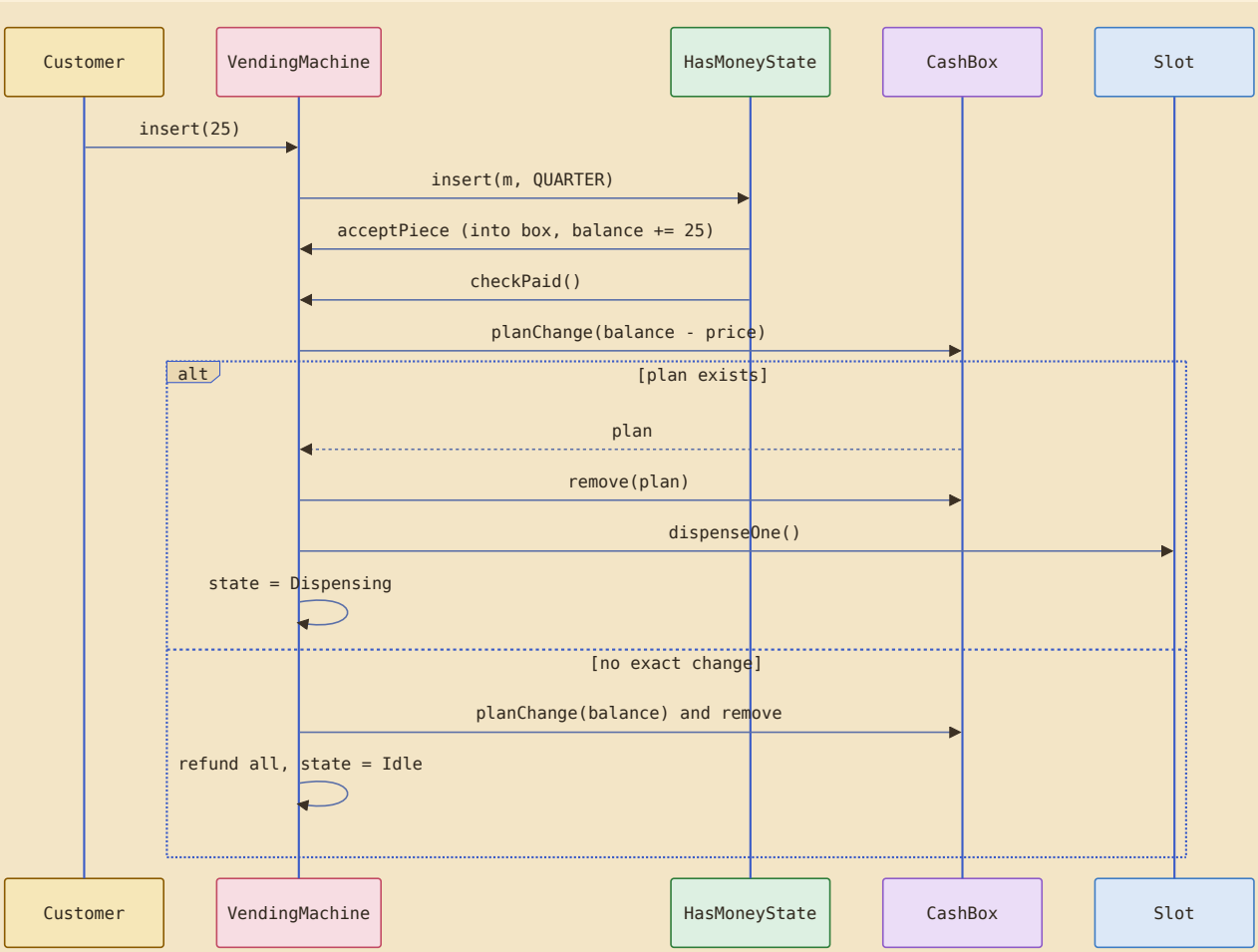
State machine. This diagram *is* the design; draw it before writing code.



The state table (rows = buttons, columns = states). Every cell has an answer:

Button ↓ / State →	Idle	Selected	HasMoney	Dispensing	OutOfStock
`select`	choose slot (reject empty/unknown)	change choice	"cancel first"	"take your product first"	"out of stock"
`insert`	return piece	accept; check paid	accept; check paid	return piece	return piece
`cancel`	"nothing to cancel"	clear choice → Idle	**refund all** → Idle	"too late"	"nothing to cancel"
`take`	"tray empty"	"pay first"	"pay the rest"	give product + change → Idle/OutOfStock	"tray empty"

Paying enough.


Customer
VendingMachine
HasMoneyState
CashBox
Slot
insert(25)
insert(m, QUARTER)
acceptPiece (into box, balance += 25)
checkPaid()
planChange(balance - price)
alt
[plan exists]
plan
remove(plan)
dispenseOne()
state = Dispensing
[no exact change]
planChange(balance) and remove
refund all, state = Idle
Customer
VendingMachine
HasMoneyState
CashBox
Slot

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
One class per situation behind `VendingState`	**State**	Every button × situation cell is answered; the compiler enforces completeness; no `if (state == …)`	Flags + `if/else` chains (the classic source of double-dispense bugs)
States are shared **stateless singletons** (`INSTANCE`; no fields, so one copy serves everyone)	Flyweight-style sharing (one shared object instead of many identical ones)	The course notes that creating a new state object on each transition is readable but wasteful; the states hold no data, so one instance each is enough	`new IdleState()` on every transition
Operator actions are **not** on the state interface	Interface Segregation (no class is forced to implement methods it does not need)	Restocking is allowed in every state; forcing five classes to answer it would be noise	Adding `restock` to `VendingState`
Money counted by value in an `EnumMap` (a fast map keyed by enum constants)	—	The machine never cares which physical quarter it returns	A list of coin objects
Inserted money goes straight into the box	—	Refund = "make change for the balance"; one refund path	A separate escrow pile (a holding area for inserted money)
Exact change with dynamic programming	Correctness	Greedy can fail when the box is short of one coin (see output part 6)	Greedy only
Refuse the sale if exact change is impossible	Requirement 7	Never short-change the customer	Give "close enough" change

> **About the change algorithm.** The course's planner is **greedy** (largest piece first, never looks back) and honestly says so: greedy is correct for standard coin sets when the box is well stocked, but when the box is short of one piece it can fail for an amount that smaller pieces could make. The exact fix is **dynamic programming** — the "coin change with limited coins" problem, where you build the answer for each amount from the answers for smaller amounts. Our `CashBox.planChange` implements that (bounded coin change, fewest pieces), and `greedyPlan` is kept only to show the difference: for 30¢ from {1 quarter, 3 dimes}, greedy takes the quarter and gets stuck at 5¢; DP returns three dimes.

Java implementation

Package `lld.ch14.vendingmachine`.

```java
// File: lld/ch14/vendingmachine/Inventory.java
package lld.ch14.vendingmachine;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

/** What is sold: a name and a price in cents. */
record Product(String name, int priceCents) {}

/** One physical column. The COUNT lives on the slot, not on the product. */
final class Slot {
    private final String code;
    private final Product product;
    private int count;

    Slot(String code, Product product, int count) {
        this.code = code;
        this.product = product;
        this.count = count;
    }

    boolean isEmpty() { return count == 0; }

    Product dispenseOne() {
        if (count == 0) throw new IllegalStateException(code + " is empty");
        count--;
        return product;
    }

    void restock(int qty) { count += qty; }
    String code() { return code; }
    Product product() { return product; }
    int count() { return count; }
}

/** All slots, looked up by code. */
final class Inventory {
    private final Map<String, Slot> slots = new LinkedHashMap<>();

    void addSlot(Slot s) { slots.put(s.code(), s); }
    Optional<Slot> find(String code) { return Optional.ofNullable(slots.get(code)); }
    boolean allEmpty() { return slots.values().stream().allMatch(Slot::isEmpty); }

    String summary() {
        StringBuilder sb = new StringBuilder();
        slots.values().forEach(s -> sb.append(s.code()).append(':').append(s.product().name())
                .append('x').append(s.count()).append(' '));
        return sb.toString().trim();
    }
}
```

```java
// File: lld/ch14/vendingmachine/CashBox.java
package lld.ch14.vendingmachine;

import java.util.Arrays;
import java.util.EnumMap;
import java.util.Map;
import java.util.Optional;

/** The fixed set of pieces the machine accepts, in cents. Anything else is returned. */
enum Denomination {
    NICKEL(5, false), DIME(10, false), QUARTER(25, false), ONE_DOLLAR(100, true), FIVE_DOLLAR(500, true);

    final int cents;
    final boolean note;

    Denomination(int cents, boolean note) { this.cents = cents; this.note = note; }

    static Optional<Denomination> of(int cents) {
        return Arrays.stream(values()).filter(d -> d.cents == cents).findFirst();
    }
}

/** Counts pieces by value. Inserted money goes in straight away, so refunds are just "make change". */
final class CashBox {
    private final EnumMap<Denomination, Integer> counts = new EnumMap<>(Denomination.class);

    CashBox() { for (Denomination d : Denomination.values()) counts.put(d, 0); }

    void add(Denomination d, int n) { counts.merge(d, n, Integer::sum); }

    /**
     * Exact change with the FEWEST pieces, respecting how many of each piece we hold
     * (bounded coin change, dynamic programming). Empty = cannot be made exactly.
     */
    Optional<Map<Denomination, Integer>> planChange(int amount) {
        Denomination[] ds = Denomination.values();
        int[][] best = new int[amount + 1][];   // best[a] = pieces used per denomination to make a
        best[0] = new int[ds.length];
        for (int i = 0; i < ds.length; i++) {
            int v = ds[i].cents;
            int copies = Math.min(counts.get(ds[i]), amount / v);
            for (int c = 0; c < copies; c++) {                // each copy may be used once (0/1)
                for (int a = amount; a >= v; a--) {
                    if (best[a - v] == null) continue;
                    int pieces = sum(best[a - v]) + 1;
                    if (best[a] == null || pieces < sum(best[a])) {
                        best[a] = best[a - v].clone();
                        best[a][i]++;
                    }
                }
            }
        }
        if (best[amount] == null) return Optional.empty();
        Map<Denomination, Integer> plan = new EnumMap<>(Denomination.class);
        for (int i = 0; i < ds.length; i++) if (best[amount][i] > 0) plan.put(ds[i], best[amount][i]);
        return Optional.of(plan);
    }

    /** Greedy (largest piece first) — shown only to compare with planChange. */
    Optional<Map<Denomination, Integer>> greedyPlan(int amount) {
        Map<Denomination, Integer> plan = new EnumMap<>(Denomination.class);
        Denomination[] ds = Denomination.values();
        for (int i = ds.length - 1; i >= 0 && amount > 0; i--) {
            int take = Math.min(counts.get(ds[i]), amount / ds[i].cents);
            if (take > 0) { plan.put(ds[i], take); amount -= take * ds[i].cents; }
        }
        return amount == 0 ? Optional.of(plan) : Optional.empty();
    }

    void remove(Map<Denomination, Integer> plan) {
```

```
        plan.forEach((d, n) -> counts.merge(d, -n, Integer::sum));
    }

    /** Operator empties the notes; coins stay behind as the float for change. */
    int collectNotes() {
        int total = 0;
        for (Denomination d : Denomination.values()) {
            if (d.note) { total += counts.get(d) * d.cents; counts.put(d, 0); }
        }
        return total;
    }

    int total() {
        return counts.entrySet().stream().mapToInt(e -> e.getKey().cents * e.getValue()).sum();
    }

    private static int sum(int[] a) { return Arrays.stream(a).sum(); }
}
```

```java
// File: lld/ch14/vendingmachine/States.java
package lld.ch14.vendingmachine;

/** State pattern: every button is on the interface, so every state must answer every button. */
interface VendingState {
    void select(VendingMachine m, String code);
    void insert(VendingMachine m, Denomination d);
    void cancel(VendingMachine m);
    void take(VendingMachine m);
    String name();
}

/** Waiting for a customer. */
final class IdleState implements VendingState {
    static final IdleState INSTANCE = new IdleState();   // states hold no data, so share one instance

    @Override public void select(VendingMachine m, String code) {
        m.inventory().find(code).ifPresentOrElse(slot -> {
            if (slot.isEmpty()) { m.show(code + " is sold out. Choose another."); return; }
            m.choose(slot);
            m.show("%s costs %d c. Insert money.".formatted(slot.product().name(), slot.product().priceCents()));
            m.setState(SelectedState.INSTANCE);
        }, () -> m.show("Unknown code " + code));
    }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "select a product first"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
    @Override public String name() { return "IDLE"; }
}

/** A product is chosen, no money yet. The customer may still change their mind freely. */
final class SelectedState implements VendingState {
    static final SelectedState INSTANCE = new SelectedState();

    @Override public void select(VendingMachine m, String code) { IdleState.INSTANCE.select(m, code); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) {
        m.clearSelection();
        m.show("Cancelled.");
        m.setState(IdleState.INSTANCE);
    }
    @Override public void take(VendingMachine m) { m.show("Please pay first."); }
    @Override public String name() { return "SELECTED"; }
}

/** Some money is in, but not enough yet. */
final class HasMoneyState implements VendingState {
    static final HasMoneyState INSTANCE = new HasMoneyState();

    @Override public void select(VendingMachine m, String code) { m.show("Cancel first to change your choice."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.acceptPiece(d); m.checkPaid(); }
    @Override public void cancel(VendingMachine m) { m.refundAll("Cancelled."); }
    @Override public void take(VendingMachine m) { m.show("Please pay the rest first."); }
    @Override public String name() { return "HAS_MONEY"; }
}

/** Product and change are in the tray. Nothing else is allowed until the customer takes them. */
final class DispensingState implements VendingState {
    static final DispensingState INSTANCE = new DispensingState();

    @Override public void select(VendingMachine m, String code) { m.show("Please take your product first."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "take your product first");
}
    @Override public void cancel(VendingMachine m) { m.show("Too late to cancel: product already dispensed."); }
    @Override public void take(VendingMachine m) { m.emptyTray(); }
    @Override public String name() { return "DISPENSING"; }
}

/** Every slot is empty. Only an operator restock leaves this state. */
final class OutOfStockState implements VendingState {
```

```
    static final OutOfStockState INSTANCE = new OutOfStockState();

    @Override public void select(VendingMachine m, String code) { m.show("Machine is out of stock."); }
    @Override public void insert(VendingMachine m, Denomination d) { m.rejectPiece(d, "out of stock"); }
    @Override public void cancel(VendingMachine m) { m.show("Nothing to cancel."); }
    @Override public void take(VendingMachine m) { m.show("The tray is empty."); }
    @Override public String name() { return "OUT_OF_STOCK"; }
}
```

```java
// File: lld/ch14/vendingmachine/VendingMachine.java
package lld.ch14.vendingmachine;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * Holds the parts and the current state and forwards every button to the state.
 * There is no "if (state == ...)" anywhere in this class.
 * Public methods are synchronized: one physical machine, but a networked
 * controller or a second input device must not interleave two actions.
 */
final class VendingMachine {
    private final Inventory inventory;
    private final CashBox cashBox;
    private VendingState state;
    private Slot selected;
    private int balance;                                    // cents inserted in this sale
    private final List<String> tray = new ArrayList<>();

    VendingMachine(Inventory inventory, CashBox cashBox) {
        this.inventory = inventory;
        this.cashBox = cashBox;
        this.state = inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE;
    }

    // ----- buttons the customer presses -----
    synchronized void select(String code) { log("select " + code); state.select(this, code); }
    synchronized void cancel() { log("cancel"); state.cancel(this); }
    synchronized void take() { log("take"); state.take(this); }

    synchronized void insert(int pieceCents) {
        log("insert " + pieceCents + "c");
        Denomination.of(pieceCents).ifPresentOrElse(
                d -> state.insert(this, d),
                () -> show("Unknown piece of " + pieceCents + "c returned."));
    }

    // ----- operator actions: allowed in every state, so NOT on the state interface -----
    synchronized void restock(String code, int qty) {
        inventory.find(code).orElseThrow().restock(qty);
        if (state == OutOfStockState.INSTANCE) setState(IdleState.INSTANCE);
        show("Operator restocked " + code + " +" + qty);
    }
    synchronized void loadChange(Denomination d, int n) { cashBox.add(d, n); }
    synchronized int collectTakings() { return cashBox.collectNotes(); }

    // ----- helpers used by the states -----
    void setState(VendingState s) { this.state = s; }
    Inventory inventory() { return inventory; }
    void choose(Slot s) { this.selected = s; }
    void clearSelection() { this.selected = null; }
    void show(String msg) { System.out.println("   [display] " + msg); }

    void acceptPiece(Denomination d) {
        cashBox.add(d, 1);
        balance += d.cents;
    }

    void rejectPiece(Denomination d, String why) { show(d + " returned: " + why + "."); }

    /** Called after every accepted piece in SELECTED / HAS_MONEY. */
    void checkPaid() {
        int price = selected.product().priceCents();
        if (balance < price) {
            show("Paid %d c, %d c to go.".formatted(balance, price - balance));
            setState(HasMoneyState.INSTANCE);
            return;
        }
```

```
        var change = cashBox.planChange(balance - price);
        if (change.isEmpty()) {                                  // refuse the sale rather than short-change
            refundAll("Cannot make exact change.");
            return;
        }
        cashBox.remove(change.get());
        tray.add(selected.dispenseOne().name());
        if (!change.get().isEmpty()) tray.add("change " + change.get());
        show("Dispensed. Please take: " + tray);
        balance = 0;
        selected = null;
        setState(DispensingState.INSTANCE);
    }

    /** One refund path for "cancel" and for "cannot make change". */
    void refundAll(String reason) {
        Map<Denomination, Integer> back = cashBox.planChange(balance)
                .orElseThrow(() -> new IllegalStateException("inserted pieces must be refundable"));
        cashBox.remove(back);
        show(reason + " Refunded " + balance + " c as " + back);
        balance = 0;
        selected = null;
        setState(IdleState.INSTANCE);
    }

    void emptyTray() {
        show("Customer took " + tray);
        tray.clear();
        setState(inventory.allEmpty() ? OutOfStockState.INSTANCE : IdleState.INSTANCE);
    }

    synchronized String status() {
        return "state=%s cash=%dc stock=[%s]".formatted(state.name(), cashBox.total(), inventory.summary());
    }

    private void log(String action) { System.out.println(" > " + action + "   (" + state.name() + ")"); }
}
```

```java
// File: lld/ch14/vendingmachine/Main.java
package lld.ch14.vendingmachine;

public class Main {
    public static void main(String[] args) {
        Inventory inv = new Inventory();
        inv.addSlot(new Slot("A1", new Product("Cola", 125), 2));
        inv.addSlot(new Slot("B1", new Product("Chips", 50), 1));
        inv.addSlot(new Slot("C1", new Product("Candy", 65), 0));
        CashBox box = new CashBox();
        VendingMachine vm = new VendingMachine(inv, box);
        vm.loadChange(Denomination.QUARTER, 2);
        vm.loadChange(Denomination.DIME, 1);
        System.out.println(vm.status());

        System.out.println("\n1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused");
        vm.select("A1");
        vm.insert(100);
        vm.insert(25);
        vm.insert(25);
        vm.insert(10);          // ignored while the product waits in the tray
        vm.cancel();            // too late
        vm.take();

        System.out.println("\n2) Wrong order and bad input");
        vm.insert(25);          // no selection yet
        vm.select("C1");        // sold out
        vm.select("Z9");        // unknown
        vm.select("B1");
        vm.insert(3);           // foreign coin

        System.out.println("\n3) Cancel with money in: full refund");
        vm.insert(25);
        vm.select("A1");        // must cancel first
        vm.cancel();

        System.out.println("\n4) Cannot make exact change -> sale refused, everything refunded");
        vm.select("B1");
        vm.insert(500);         // needs 450 c change, box can't make it
        System.out.println(vm.status());

        System.out.println("\n5) Last items sold -> OUT_OF_STOCK -> operator restocks");
        vm.select("B1");
        vm.insert(25);
        vm.insert(25);
        vm.take();
        vm.select("A1");
        vm.insert(100);
        vm.insert(25);
        vm.take();
        System.out.println(vm.status());
        vm.select("A1");
        vm.restock("C1", 5);
        vm.select("C1");
        vm.insert(100);         // 35c change: 25 + 10
        vm.take();

        System.out.println("\n6) Why the change planner is not greedy");
        CashBox demo = new CashBox();
        demo.add(Denomination.QUARTER, 1);
        demo.add(Denomination.DIME, 3);
        System.out.println("30c from {1 quarter, 3 dimes}: greedy=" + demo.greedyPlan(30)
                + " dp=" + demo.planChange(30));

        System.out.println("\nOperator collects notes: " + vm.collectTakings() + " c");
        System.out.println(vm.status());
    }
}
```

How the code works, part by part

- `VendingMachine.insert(int)` first maps the raw piece to a `Denomination`; an unknown piece (a 3¢ token) is returned without ever reaching a state.
- `SelectedState` and `HasMoneyState` both call `acceptPiece` then `checkPaid`. `checkPaid` either moves to `HasMoney`, or completes the sale (plan change → remove pieces → dispense one → `Dispensing`), or refunds everything if exact change is impossible.
- `refundAll` is the **single** refund path used by cancel and by "cannot make change". It can never fail, because the pieces just inserted are in the box, so an exact plan always exists.
- `DispensingState.take` empties the tray and moves to `Idle` or `OutOfStock` — the sale ends when the customer takes the product, which is why `take` is a customer button.
- All public methods are `synchronized`. A single physical machine has one customer, but a networked controller (remote price change, telemetry — status data sent automatically to a remote server — or a second input device) must not interleave two actions.

Output:

```
state=IDLE cash=60c stock=[A1:Colax2 B1:Chipsx1 C1:Candyx0]

1) Happy path: Cola with $1 + 25c. Extra coins and a late cancel are refused
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > insert 25c   (DISPENSING)
   [display] QUARTER returned: take your product first.
 > insert 10c   (DISPENSING)
   [display] DIME returned: take your product first.
 > cancel   (DISPENSING)
   [display] Too late to cancel: product already dispensed.
 > take   (DISPENSING)
   [display] Customer took [Cola]

2) Wrong order and bad input
 > insert 25c   (IDLE)
   [display] QUARTER returned: select a product first.
 > select C1   (IDLE)
   [display] C1 is sold out. Choose another.
 > select Z9   (IDLE)
   [display] Unknown code Z9
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 3c   (SELECTED)
   [display] Unknown piece of 3c returned.

3) Cancel with money in: full refund
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > select A1   (HAS_MONEY)
   [display] Cancel first to change your choice.
 > cancel   (HAS_MONEY)
   [display] Cancelled. Refunded 25 c as {QUARTER=1}

4) Cannot make exact change -> sale refused, everything refunded
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 500c   (SELECTED)
   [display] Cannot make exact change. Refunded 500 c as {FIVE_DOLLAR=1}
state=IDLE cash=185c stock=[A1:Colax1 B1:Chipsx1 C1:Candyx0]

5) Last items sold -> OUT_OF_STOCK -> operator restocks
 > select B1   (IDLE)
   [display] Chips costs 50 c. Insert money.
 > insert 25c   (SELECTED)
   [display] Paid 25 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Chips]
 > take   (DISPENSING)
   [display] Customer took [Chips]
 > select A1   (IDLE)
   [display] Cola costs 125 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Paid 100 c, 25 c to go.
 > insert 25c   (HAS_MONEY)
   [display] Dispensed. Please take: [Cola]
 > take   (DISPENSING)
   [display] Customer took [Cola]
state=OUT_OF_STOCK cash=360c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx0]
 > select A1   (OUT_OF_STOCK)
   [display] Machine is out of stock.
   [display] Operator restocked C1 +5
 > select C1   (IDLE)
   [display] Candy costs 65 c. Insert money.
 > insert 100c   (SELECTED)
   [display] Dispensed. Please take: [Candy, change {DIME=1, QUARTER=1}]
```

```
  > take   (DISPENSING)
    [display] Customer took [Candy, change {DIME=1, QUARTER=1}]

6) Why the change planner is not greedy
30c from {1 quarter, 3 dimes}: greedy=Optional.empty dp=Optional[{DIME=3}]

Operator collects notes: 300 c
state=IDLE cash=125c stock=[A1:Colax0 B1:Chipsx0 C1:Candyx4]
```

Read the output: in part 1 the cola is paid exactly, and coins inserted while the product waits are returned, and a late cancel is refused; part 4 shows the refusal when 450¢ of change cannot be made from the box (so the $5 note comes straight back); part 5 shows the machine going to `OUT_OF_STOCK` and back to `IDLE` after a restock; part 6 shows greedy failing where DP succeeds.

Concurrency & edge cases

Edge case	Handling
Money inserted before selecting	`IdleState.insert` returns the piece
Unknown coin or foreign note	Returned before reaching the state
Selecting an empty slot	`IdleState.select` refuses; the state stays Idle
Changing the choice after paying	`HasMoneyState.select` says "cancel first" (prevents price confusion)
Paying more than the price	Change is planned **before** dispensing; if impossible, the sale is refused
Cancel after dispensing	"Too late" — the product is already in the tray
Two products for one payment	Impossible: after dispensing, the balance is 0 and the state is Dispensing, which accepts no money
Last item sold	`take` moves to `OutOfStock`; only a restock leaves it
Two threads pressing buttons	`synchronized` public methods; states run under the machine's lock
A bank of machines sharing one stock count	The course notes the design is not safe for that; the shared count on `Slot` would then need a lock or an atomic decrement
Power failure mid-sale	Real machines keep a transaction log (credit held) in non-volatile memory (memory that keeps its contents without power); out of scope

Extensions & follow-up questions

1. **"Where would you add contactless card payment?"** — One new input device beside the cash slot that calls the same machine (for example a `payByCard(amount)` that sets the balance to the price). No state class changes if the states only look at the balance.
2. **"What if it cannot give change?"** — The planner returns empty; the sale is refused and everything is refunded (requirement 7). Mention greedy vs DP.
3. **"Add a maintenance state."** — A new `MaintenanceState` class; the operator switches it on. The existing five states are untouched (Open/Closed).
4. **"Different price per slot for the same product?"** — Move the price from `Product` to `Slot` (the slot already owns the count).
5. **"Report empty slots to headquarters."** — Make `Inventory` notify an observer when a slot reaches 0; a telemetry client sends it over the network.
6. **"What did you leave out on purpose?"** — Card payment, reporting empty slots over a network, refrigeration (chilling) control and per-customer discounts. Say so explicitly: they are scope decisions, not oversights.

7. **"Timeout if the customer walks away with money in?"** — A timer that calls `cancel()` after, say, 60 seconds of inactivity; it goes through the same state machine.

Real-world example

Modern vending machines are often retro-fitted with cashless readers such as **Nayax** or **Cantaloupe (ePort)** units, which plug into the machine's controller as just another payment device — exactly extension 1: a new input beside the coin mechanism, with the dispensing logic unchanged. Coin mechanisms in real machines keep "change tubes" of each coin and show an **"exact change only"** light when a tube runs low — the physical version of our planner reporting that it cannot make change.

In the interview

Plan for 45 minutes: 5 min questions and requirements → 5 min entities (product, slot, inventory, cash box) → **10 min: draw the state diagram and the state table** → 15 min code the interface, two or three states and `checkPaid`/`refundAll` → 10 min edge cases and follow-ups.

> **30-second answer:** "The whole question is the order of events, so I'll use the State pattern: one class per situation — Idle, Selected, HasMoney, Dispensing, OutOfStock — all implementing an interface that lists every button. The machine just forwards to its current state. Money goes into the cash box as it is inserted, so a refund is just 'make exact change for the balance'; if exact change is impossible, the sale is refused and everything is returned. Amounts are integer cents."

Follow-ups with answers

- *"Why not an enum with a switch?"* — It works for tiny machines, but every new state means editing every switch; with the State pattern a new state is a new class, and the compiler forces it to answer every button.
- *"Is greedy change correct?"* — Only for well-stocked canonical coin sets (coin systems such as 1/5/10/25 where taking the largest coin first always gives the fewest coins). With limited coins it can fail; use bounded coin-change DP.
- *"Where does the operator fit?"* — Outside the state interface; restocking is allowed in any state and can move OutOfStock → Idle.

Self-check

1. **Q:** What makes the design "complete"? **A:** Every state class implements every button method, so every (state, button) cell has a defined answer.
2. **Q:** Why does inserted money go straight into the cash box? **A:** So the same pieces can be returned as change or refund; a refund is just planning exact change for the balance.
3. **Q:** Why is `restock` not in `VendingState`? **A:** It is not a customer action and is valid in every state (Interface Segregation).
4. **Q:** Give a case where greedy change fails. **A:** 30¢ with one quarter and three dimes: greedy takes 25¢ and cannot make 5¢; three dimes work.
5. **Q:** How is "two products for one payment" prevented? **A:** After dispensing, balance is reset to 0 and the Dispensing state returns any inserted money until the product is taken.

TL;DR

- **State pattern is the whole design**: five state classes, one interface listing every button, no `if` on state in the machine.
- Count lives on the **slot**; money in **integer cents**; inserted money goes **into the box immediately**.
- Change: **exact** or refuse; greedy can fail, **DP** (bounded coin change) is the correct planner.
- One refund path for cancel and "no change"; operator actions stay off the state interface.

14.3 Design a Coffee Vending Machine

A **coffee vending machine** makes drinks on demand — espresso, americano, latte, cappuccino, mocha, hot chocolate — from a shared stock of **ingredients** (beans, water, milk, chocolate, sugar). It looks like the previous problem, but the hard part is different. A snack machine hands out a finished item from a slot. A coffee machine **consumes several ingredients at once** from shared containers, and many drinks draw on the same milk. The interviewer wants to see three things:

1. **Recipes as data** — adding a drink should be a data change, not a new class.
2. **Atomic, all-or-nothing deduction** — a latte must take its beans, water *and* milk together, or take nothing; stock must never go negative, even when ten people order at once.
3. **Low-stock alerts** — the operator hears about milk running low *before* it runs out (Observer).

Problem & clarifying questions

Question	Assumed answer
Which drinks?	Espresso, Americano, Latte, Cappuccino, Mocha, Hot Chocolate — loaded from a menu file.
Customisation?	Extra espresso shot (+50¢) and sugar (grams).
Payment?	Amount paid in cents; change returned (coin handling was covered in 14.2).
Several screens / concurrent orders?	Yes: a large machine or a kiosk with several touch screens, so orders can overlap.
Who refills?	An operator, alerted when an ingredient falls to a threshold.
One alert or one per drink?	One alert per crossing of the threshold (re-armed after a refill).

Requirements

Functional

1. Several drink types, each with a **price** and a **recipe** (amount of each ingredient).
2. Show a **menu** with prices and availability.
3. Accept a selection and a **payment**; return **change**; refund in full if the drink cannot be made.
4. Track **ingredient inventory**; deduct exactly the recipe amounts per drink.
5. Raise **low-stock notifications** when an ingredient drops to its threshold.
6. Allow **refills**.
7. Support **customisation** (extra shot, sugar) without changing the menu.

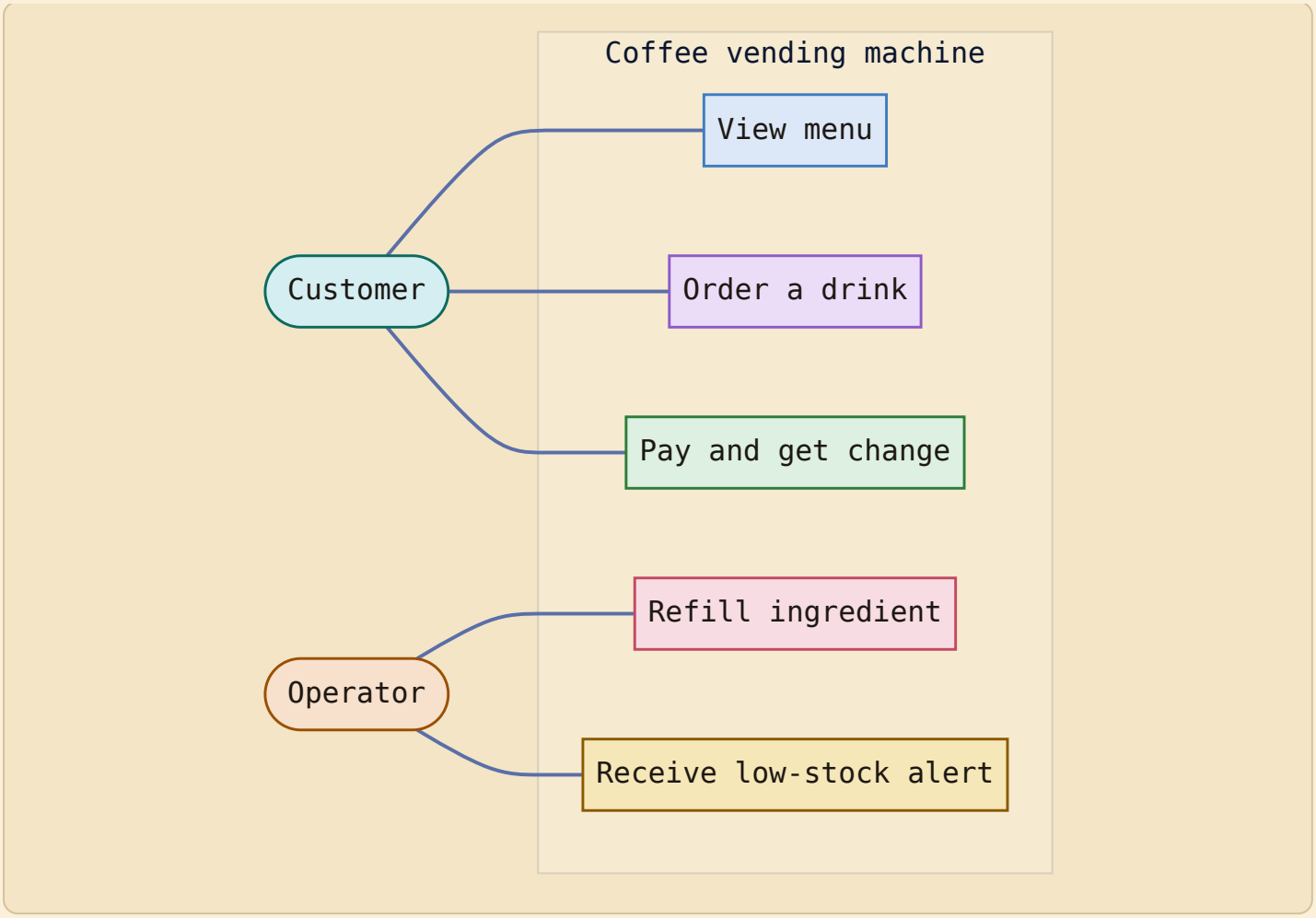
Non-functional

- **Thread-safe**: concurrent orders must never over-draw an ingredient or leave a half-deducted recipe.
- **Data-driven**: new drinks and recipe tweaks without code changes.
- **Observable**: operators, dashboards or a telemetry client can subscribe to alerts without the machine knowing about them.

Use cases

Actors: **Customer** (views menu, orders, pays, takes drink and change), **Operator** (refills, receives alerts), **System** (checks stock, deducts, brews, sends alerts).

Use cases: view menu · order drink (with customisation) · pay / get change / get refund · refill ingredient · receive low-stock alert.



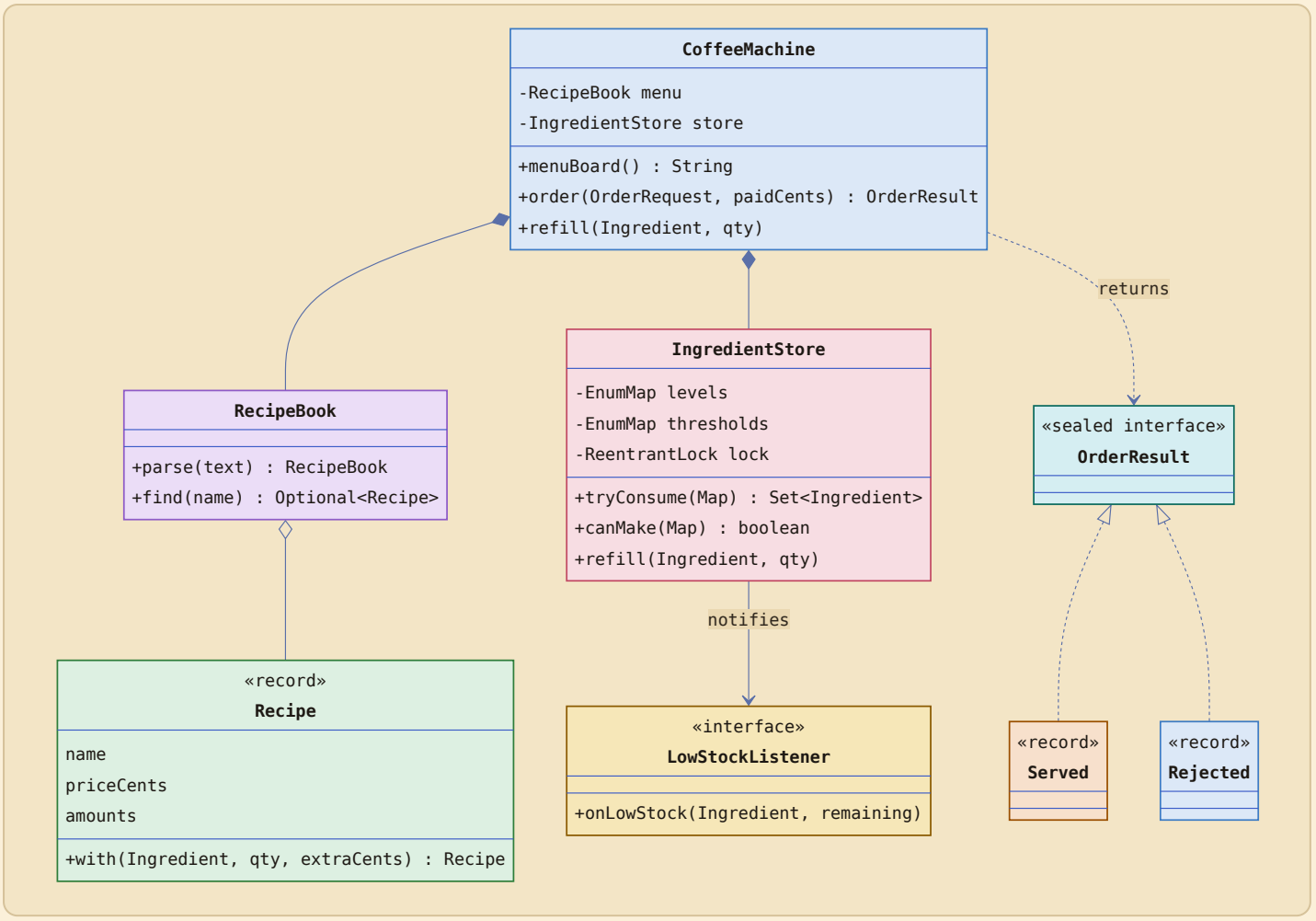
Core entities

Class	Responsibility
`Ingredient` (enum)	Beans, water, milk, chocolate, sugar — with the unit (g or ml).
`Recipe` (record)	**Data**: name, price, `Map<Ingredient, Integer>` amounts. Immutable (cannot change after creation). `with(...)` returns a customised copy.
`RecipeBook`	Parses the menu text into recipes; looks drinks up by name.
`IngredientStore`	Levels, thresholds, **one lock**; `tryConsume` = check all + deduct all atomically; alerts listeners.
`LowStockListener` (interface)	Observer called once when an ingredient crosses its threshold.
`OrderRequest` (record)	Drink name + extra shots + sugar grams.
`OrderResult` (sealed)	Either `Served(drink, price, change)` or `Rejected(reason, refund)`.
`CoffeeMachine`	Facade (one simple front object hiding the parts behind it): menu board, `order(...)`, `refill(...)`.

Mapping to the common problem statement. The widely used version of this problem (for example in the open-source *awesome-low-level-design* collection) asks for: several coffee types (espresso, cappuccino, latte) each with a price and a recipe, a menu with prices, selection and payment, dispensing with change, ingredient tracking with low-stock notifications, and thread-safe handling of concurrent requests. Its classes map onto ours as follows: `Coffee` (name, price, recipe) → our `Recipe`; `Ingredient` (name + quantity with `synchronized` update methods) → our `Ingredient` enum plus the levels inside `IngredientStore`; `Payment` (amount paid) → the `paidCents` argument and the change/refund in `OrderResult`; `CoffeeMachine` (a Singleton there) → our injected `CoffeeMachine`; concurrent users simulated with an

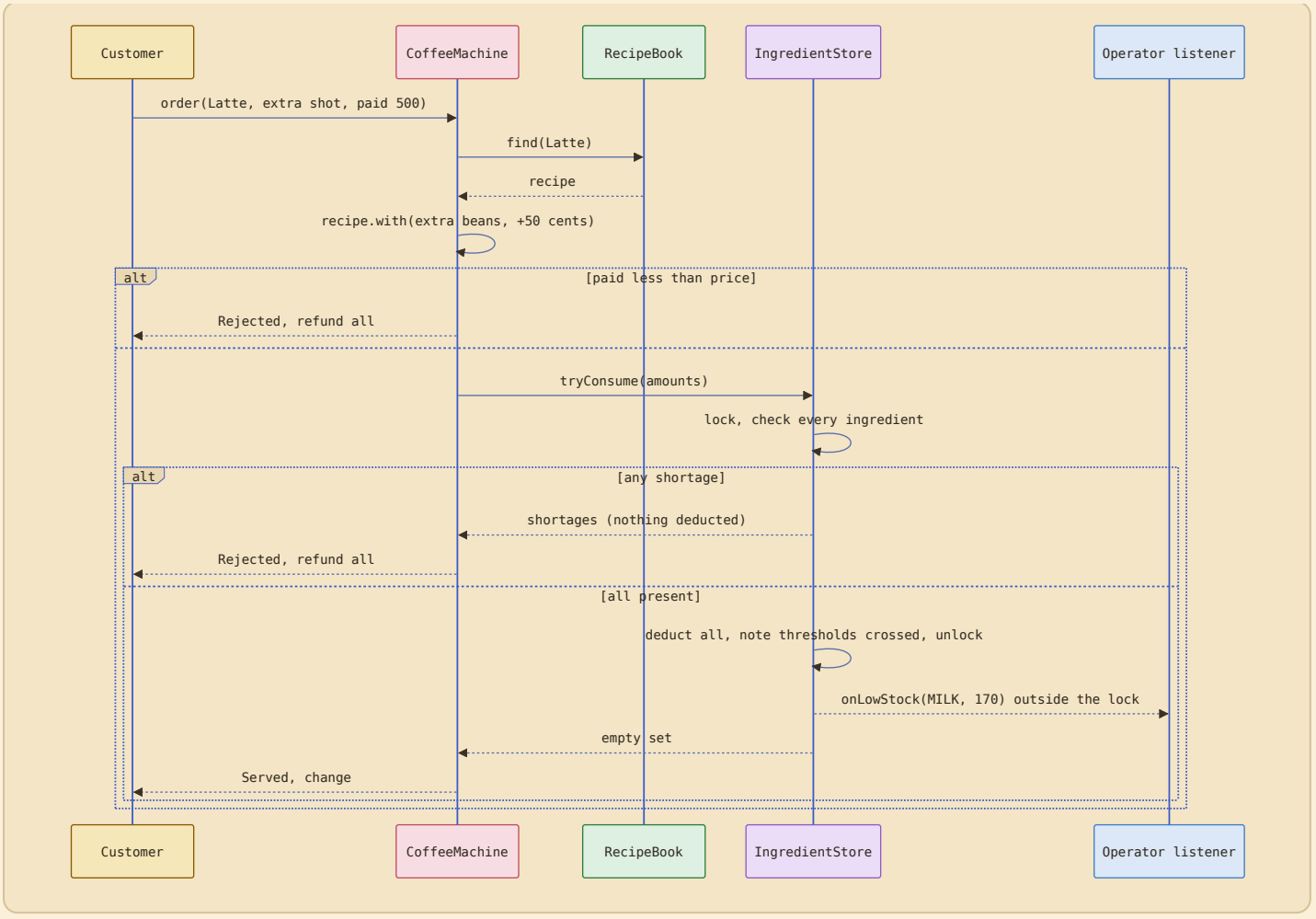
`ExecutorService` (a Java pool of worker threads that runs submitted tasks) → the 10-thread demo in our `Main`. Two deliberate differences: we lock the **whole recipe** rather than each ingredient separately (per-ingredient locking cannot make a multi-ingredient deduction all-or-nothing), and we avoid the Singleton (see the table below).

Class diagram

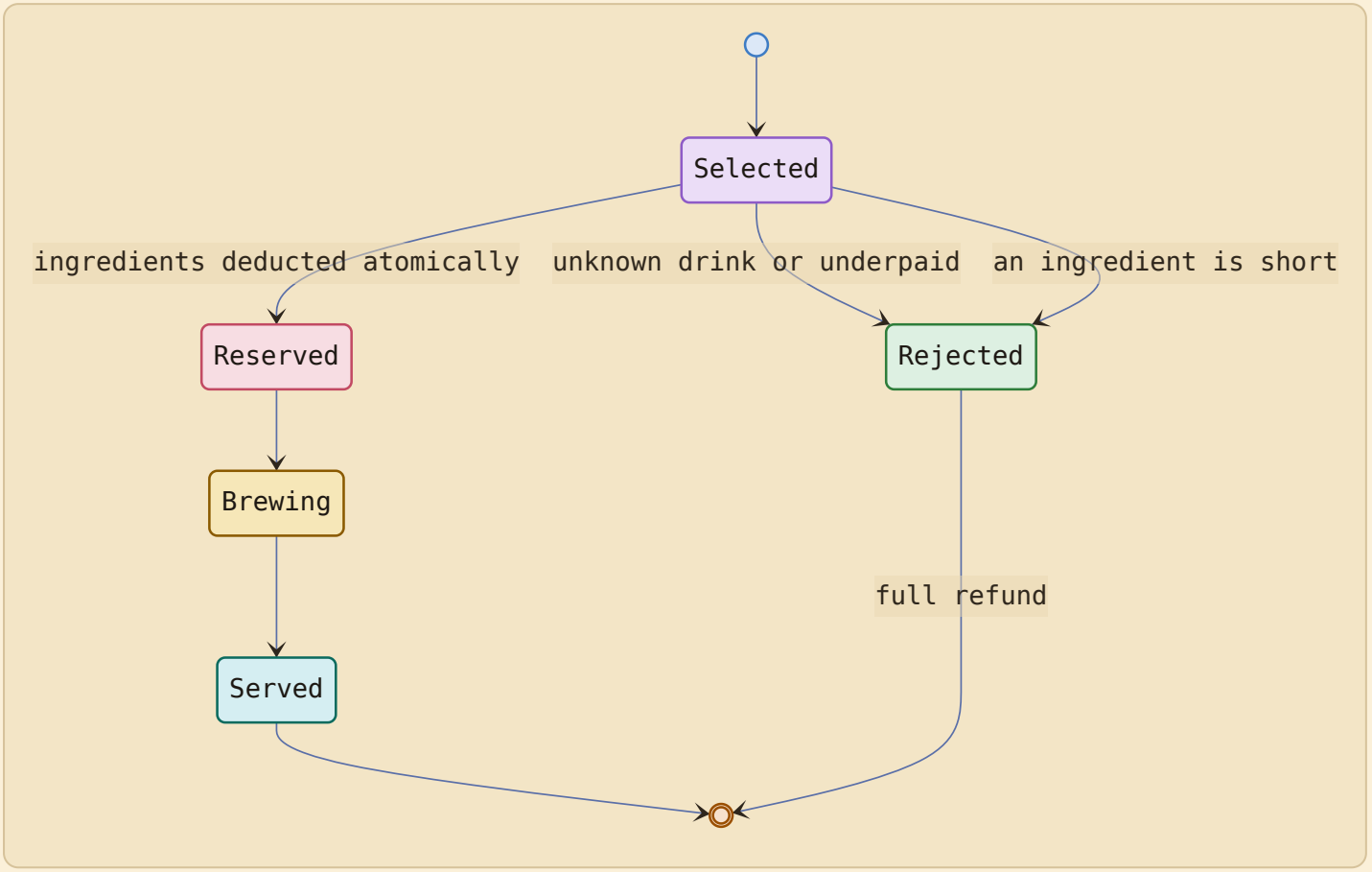


Key flows

Ordering a drink. The key point is that the stock check and the deduction happen **inside one critical section** (a piece of code that only one thread may run at a time), and alerts are sent **after** the lock is released.



Order life cycle.



Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Recipes are **data** parsed from a menu text	Data-driven design, Open/Closed	A new drink is a new line in the menu; no new class, no recompile of logic	One subclass per drink (`Latte extends Coffee`) — class explosion, and recipe tweaks need code changes
Check-and-deduct under **one lock**	Atomicity (Ch. 12)	Prevents two orders both seeing "enough milk" and both deducting (a check-then-act **race**: both check, then both act on an answer that is already stale)	Checking in one call and deducting in another; per-ingredient locks (a multi-ingredient recipe would need all of them, in order)
All-or-nothing	Transaction semantics (all steps take effect together, or none do)	Never leave a half-made recipe's ingredients missing	Deduct one by one and "roll back" on failure
Alerts via `LowStockListener`	**Observer**	Operator app, dashboard, telemetry can subscribe independently	The store calling a specific SMS class
Alert **once per crossing**, re-armed by refill	—	Prevents alert storms (one per drink)	Alerting on every order below threshold
Listeners called **outside** the lock	Concurrency hygiene	A slow listener (network call) must not block every order; it also avoids deadlocks (two threads each waiting forever for a lock the other holds) if a listener calls back into the store	Notifying inside the lock
Customisation returns a **new** immutable `Recipe`	Immutability	The menu recipe can never be corrupted by one order	Mutating the menu recipe; a Decorator chain (works, but heavier than needed for "extra X")
`OrderResult` is a **sealed** interface with records	Algebraic data type (a value that is exactly one of a fixed set of shapes; Java 17+ sealed, Java 21 pattern switch)	Callers must handle both Served and Rejected; `switch` is checked for exhaustiveness (the compiler complains if a case is missing)	Returning `null` or throwing for normal refusals
No Singleton	Dependency injection	The problem-set solutions often make `CoffeeMachine` a Singleton; we create one and pass it around, so tests can build fresh machines (the demo builds two)	`CoffeeMachine.getInstance()`

> **Decorator or data?** A classic textbook example (*Head First Design Patterns*) models coffee add-ons with the **Decorator** pattern: `new Whip(new Mocha(new DarkRoast()))`, each wrapper adding cost and description. That is a good fit when each add-on has **behaviour**. Here add-ons only change **quantities and price**, so a data change (`recipe.with(...)`) is simpler. Say both in the interview and pick one for a reason.

Java implementation

Package `lld.ch14.coffeemachine`.

```java
// File: lld/ch14/coffeemachine/Recipes.java
package lld.ch14.coffeemachine;

import java.util.Collections;
import java.util.EnumMap;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

enum Ingredient {
    COFFEE_BEANS("g"), WATER("ml"), MILK("ml"), CHOCOLATE("g"), SUGAR("g");

    final String unit;

    Ingredient(String unit) { this.unit = unit; }
}

/** A recipe is DATA: a name, a price and the amount of each ingredient. */
record Recipe(String name, int priceCents, Map<Ingredient, Integer> amounts) {

    Recipe {
        amounts = Collections.unmodifiableMap(new EnumMap<>(amounts));
    }

    /** Customisation returns a NEW recipe; the menu entry is never modified. */
    Recipe with(Ingredient extra, int qty, int extraCents) {
        if (qty == 0) return this;
        Map<Ingredient, Integer> m = new EnumMap<>(Ingredient.class);
        m.putAll(amounts);
        m.merge(extra, qty, Integer::sum);
        return new Recipe(name, priceCents + extraCents, m);
    }
}

/** Parses the menu from plain text, so adding a drink is a data change, not a code change. */
final class RecipeBook {
    private final Map<String, Recipe> byName = new LinkedHashMap<>();

    static RecipeBook parse(String text) {
        RecipeBook book = new RecipeBook();
        for (String line : text.strip().split("\n")) {
            line = line.strip();
            if (line.isEmpty() || line.startsWith("#")) continue;
            String[] parts = line.split(",");
            Map<Ingredient, Integer> amounts = new EnumMap<>(Ingredient.class);
            for (int i = 2; i < parts.length; i++) {
                String[] kv = parts[i].strip().split("=");
                amounts.put(Ingredient.valueOf(kv[0].strip()), Integer.parseInt(kv[1].strip()));
            }
            Recipe r = new Recipe(parts[0].strip(), Integer.parseInt(parts[1].strip()), amounts);
            book.byName.put(r.name().toLowerCase(), r);
        }
        return book;
    }

    Optional<Recipe> find(String name) { return Optional.ofNullable(byName.get(name.toLowerCase())); }
    Iterable<Recipe> all() { return byName.values(); }
}
```

```java
// File: lld/ch14/coffeemachine/IngredientStore.java
package lld.ch14.coffeemachine;

import java.util.ArrayList;
import java.util.EnumMap;
import java.util.EnumSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.locks.ReentrantLock;

/** Observer: told once when an ingredient drops to or below its threshold. */
interface LowStockListener {
    void onLowStock(Ingredient ingredient, int remaining);
}

/**
 * Ingredient levels. The whole check-then-deduct runs under ONE lock, so a drink either
 * takes every ingredient it needs or takes nothing (all-or-nothing, never negative).
 */
final class IngredientStore {
    private final Map<Ingredient, Integer> levels = new EnumMap<>(Ingredient.class);
    private final Map<Ingredient, Integer> thresholds = new EnumMap<>(Ingredient.class);
    private final Set<Ingredient> alerted = EnumSet.noneOf(Ingredient.class);
    private final List<LowStockListener> listeners = new CopyOnWriteArrayList<>();
    private final ReentrantLock lock = new ReentrantLock();

    IngredientStore(Map<Ingredient, Integer> initial, Map<Ingredient, Integer> lowThresholds) {
        for (Ingredient i : Ingredient.values()) {
            levels.put(i, initial.getOrDefault(i, 0));
            thresholds.put(i, lowThresholds.getOrDefault(i, 0));
        }
    }

    void addListener(LowStockListener l) { listeners.add(l); }

    /** Returns the ingredients that are short. Empty set = success, everything was deducted. */
    Set<Ingredient> tryConsume(Map<Ingredient, Integer> need) {
        Set<Ingredient> shortages = EnumSet.noneOf(Ingredient.class);
        List<Runnable> alerts = new ArrayList<>();
        lock.lock();
        try {
            need.forEach((i, q) -> { if (levels.get(i) < q) shortages.add(i); });
            if (!shortages.isEmpty()) return shortages;               // nothing touched
            need.forEach((i, q) -> levels.merge(i, -q, Integer::sum));
            for (Ingredient i : need.keySet()) {
                int left = levels.get(i);
                if (left <= thresholds.get(i) && alerted.add(i)) {    // alert once per crossing
                    alerts.add(() -> listeners.forEach(l -> l.onLowStock(i, left)));
                }
            }
        } finally {
            lock.unlock();
        }
        alerts.forEach(Runnable::run);   // call listeners OUTSIDE the lock
        return shortages;
    }

    boolean canMake(Map<Ingredient, Integer> need) {
        lock.lock();
        try {
            return need.entrySet().stream().allMatch(e -> levels.get(e.getKey()) >= e.getValue());
        } finally {
            lock.unlock();
        }
    }

    void refill(Ingredient i, int qty) {
        lock.lock();
```

```
        try {
            levels.merge(i, qty, Integer::sum);
            if (levels.get(i) > thresholds.get(i)) alerted.remove(i);  // re-arm the alert
        } finally {
            lock.unlock();
        }
    }

    int level(Ingredient i) {
        lock.lock();
        try { return levels.get(i); } finally { lock.unlock(); }
    }
}
```

```
// File: lld/ch14/coffeemachine/CoffeeMachine.java
package lld.ch14.coffeemachine;

import java.util.Set;

/** What the customer asked for. */
record OrderRequest(String drink, int extraShots, int sugarGrams) {
    OrderRequest(String drink) { this(drink, 0, 0); }
}

/** The outcome of an order: a sealed type, so callers must handle both cases. */
sealed interface OrderResult permits Served, Rejected {}
record Served(String drink, int pricedCents, int changeCents) implements OrderResult {}
record Rejected(String reason, int refundCents) implements OrderResult {}

final class CoffeeMachine {
    static final int EXTRA_SHOT_CENTS = 50;
    static final int SHOT_BEANS_G = 18;

    private final RecipeBook menu;
    private final IngredientStore store;

    CoffeeMachine(RecipeBook menu, IngredientStore store) {
        this.menu = menu;
        this.store = store;
    }

    String menuBoard() {
        StringBuilder sb = new StringBuilder();
        for (Recipe r : menu.all()) {
            String line = "  %-14s %4d c  %s".formatted(r.name(), r.priceCents(),
                    store.canMake(r.amounts()) ? "" : "(unavailable)");
            sb.append(line.stripTrailing()).append('\n');
        }
        return sb.toString();
    }

    /** Thread-safe: many touch screens may order at once. */
    OrderResult order(OrderRequest req, int paidCents) {
        var base = menu.find(req.drink());
        if (base.isEmpty()) return new Rejected("no such drink: " + req.drink(), paidCents);
        Recipe r = base.get()
                .with(Ingredient.COFFEE_BEANS, req.extraShots() * SHOT_BEANS_G, req.extraShots() *
EXTRA_SHOT_CENTS)
                .with(Ingredient.SUGAR, req.sugarGrams(), 0);
        if (paidCents < r.priceCents()) {
            return new Rejected("insufficient payment, need " + r.priceCents() + " c", paidCents);
        }
        Set<Ingredient> short_ = store.tryConsume(r.amounts());      // atomic reserve-and-deduct
        if (!short_.isEmpty()) return new Rejected("out of " + short_, paidCents);
        return new Served(r.name(), r.priceCents(), paidCents - r.priceCents());
    }

    void refill(Ingredient i, int qty) { store.refill(i, qty); }
    int level(Ingredient i) { return store.level(i); }
}
```

```java
// File: lld/ch14/coffeemachine/Main.java
package lld.ch14.coffeemachine;

import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static final String MENU = """
            # name, price_cents, INGREDIENT=qty ...
            Espresso,      250, COFFEE_BEANS=18, WATER=40
            Americano,     300, COFFEE_BEANS=18, WATER=200
            Latte,         400, COFFEE_BEANS=18, WATER=40, MILK=180
            Cappuccino,    380, COFFEE_BEANS=18, WATER=40, MILK=120
            Mocha,         450, COFFEE_BEANS=18, WATER=40, MILK=150, CHOCOLATE=25
            Hot Chocolate, 300, MILK=200, CHOCOLATE=30
            """;

    static final Map<Ingredient, Integer> LOW = Map.of(Ingredient.COFFEE_BEANS, 100, Ingredient.WATER, 500,
            Ingredient.MILK, 200, Ingredient.CHOCOLATE, 30, Ingredient.SUGAR, 20);

    public static void main(String[] args) throws Exception {
        IngredientStore store = new IngredientStore(Map.of(Ingredient.COFFEE_BEANS, 500, Ingredient.WATER, 3000,
                Ingredient.MILK, 700, Ingredient.CHOCOLATE, 60, Ingredient.SUGAR, 100), LOW);
        store.addListener((i, left) -> System.out.println("   ALERT to operator: " + i + " low, " + left + i.unit
+ " left"));
        CoffeeMachine cm = new CoffeeMachine(RecipeBook.parse(MENU), store);

        System.out.print("Menu:\n" + cm.menuBoard());
        System.out.println("\n== Orders ==");
        show(cm.order(new OrderRequest("Espresso"), 300));
        show(cm.order(new OrderRequest("Latte", 1, 10), 500));      // extra shot + sugar
        show(cm.order(new OrderRequest("Flat White"), 400));
        show(cm.order(new OrderRequest("Mocha"), 200));
        show(cm.order(new OrderRequest("Mocha"), 450));             // chocolate 60 -> 35
        show(cm.order(new OrderRequest("Hot Chocolate"), 300));     // milk 370 -> 170, chocolate 35 -> 5 : alerts
        show(cm.order(new OrderRequest("Cappuccino"), 400));        // needs 120 milk, 170 left: ok
        show(cm.order(new OrderRequest("Latte"), 400));             // 50 milk left: rejected, nothing deducted
        System.out.println("   beans left " + cm.level(Ingredient.COFFEE_BEANS) + "g (unchanged by the rejected
latte)");
        System.out.print("Menu now:\n" + cm.menuBoard());

        System.out.println("\n== Operator refills milk ==");
        cm.refill(Ingredient.MILK, 1000);
        show(cm.order(new OrderRequest("Latte"), 400));

        System.out.println("\n== 10 customers order a Latte at the same instant (milk for exactly 4) ==");
        IngredientStore store2 = new IngredientStore(Map.of(Ingredient.COFFEE_BEANS, 1000, Ingredient.WATER, 5000,
                Ingredient.MILK, 800), LOW);
        store2.addListener((i, left) -> System.out.println("   ALERT to operator: " + i + " low, " + left + i.unit
+ " left"));
        CoffeeMachine busy = new CoffeeMachine(RecipeBook.parse(MENU), store2);
        AtomicInteger served = new AtomicInteger(), rejected = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(10)) {
            for (int i = 0; i < 10; i++) {
                pool.submit(() -> {
                    go.await();
                    switch (busy.order(new OrderRequest("Latte"), 400)) {
                        case Served s -> served.incrementAndGet();
                        case Rejected r -> rejected.incrementAndGet();
                    }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("served=" + served + " rejected=" + rejected
```

```
                + " milk left=" + busy.level(Ingredient.MILK) + "ml (never negative)");
    }

    static void show(OrderResult r) {
        switch (r) {
            case Served s -> System.out.println("SERVED   %s for %d c, change %d c".formatted(s.drink(),
s.pricedCents(), s.changeCents()));
            case Rejected x -> System.out.println("REJECTED %s, refund %d c".formatted(x.reason(),
x.refundCents()));
        }
    }
}
```

How the code works, part by part

- `Main.MENU` is a Java text block (a multi-line string written between `"""` marks); `RecipeBook.parse` turns each line into a `Recipe`. Adding "Flat White" is one more line.
- `Recipe` copies the amounts into an unmodifiable `EnumMap` in its compact constructor (a record constructor written without a parameter list, used for checks and defensive copies), so a recipe can never be changed after creation.
- `IngredientStore.tryConsume` holds a `ReentrantLock` (an explicit lock object with `lock()` and `unlock()`; *re-entrant* means the thread already holding it may take it again) while it (1) collects shortages, (2) returns immediately if any, (3) deducts every ingredient, (4) records which thresholds were crossed. It releases the lock and **then** calls listeners.
- `alerted` is an `EnumSet` (a compact set of enum constants) that remembers which ingredients already raised an alert; `refill` clears the flag when the level goes back above the threshold.
- `CoffeeMachine.order` validates the drink, applies customisation, checks payment, then reserves ingredients. The `switch` in `Main.show` uses Java 21 pattern matching over the sealed `OrderResult`.

Output:

```
Menu:
  Espresso        250 c
  Americano       300 c
  Latte           400 c
  Cappuccino      380 c
  Mocha           450 c
  Hot Chocolate   300 c

== Orders ==
SERVED   Espresso for 250 c, change 50 c
SERVED   Latte for 450 c, change 50 c
REJECTED no such drink: Flat White, refund 400 c
REJECTED insufficient payment, need 450 c, refund 200 c
SERVED   Mocha for 450 c, change 0 c
   ALERT to operator: MILK low, 170ml left
   ALERT to operator: CHOCOLATE low, 5g left
SERVED   Hot Chocolate for 300 c, change 0 c
SERVED   Cappuccino for 380 c, change 20 c
REJECTED out of [MILK], refund 400 c
   beans left 410g (unchanged by the rejected latte)
Menu now:
  Espresso        250 c
  Americano       300 c
  Latte           400 c  (unavailable)
  Cappuccino      380 c  (unavailable)
  Mocha           450 c  (unavailable)
  Hot Chocolate   300 c  (unavailable)

== Operator refills milk ==
SERVED   Latte for 400 c, change 0 c

== 10 customers order a Latte at the same instant (milk for exactly 4) ==
   ALERT to operator: MILK low, 80ml left
served=4 rejected=6 milk left=80ml (never negative)
```

Read the output: the rejected latte left the beans at 410 g — nothing is deducted when any ingredient is short; the milk and chocolate alerts fire once each; in the last part ten threads order a latte at the same instant with milk for exactly four: four are served, six are refused, milk ends at 80 ml, never negative, and the alert fires exactly once.

Concurrency & edge cases

Situation	What could go wrong	How the design handles it
Ten concurrent orders, milk for four	Over-draw (negative stock) — classic check-then-act race	Check and deduct under one lock
Recipe needs several ingredients, one is short	Beans and water taken, milk missing	All-or-nothing: shortages are found before anything is deducted
Listener is slow (sends an SMS)	Every order waits on the network	Listeners run after `unlock()`
Many orders below the threshold	Alert storm	One alert per crossing; refill re-arms
Underpayment	Drink made, money short	Payment checked before reserving ingredients
Brewing fails after deduction (hardware fault)	Customer pays, gets nothing	Extension: refund and optionally `refill` the reserved amounts back (compensation: an action that undoes an earlier step)
Menu displays "available" and then the order fails	A race between display and order	The menu is advisory; `tryConsume` is the source of truth

A **single lock** is right here: the critical section is tiny (five map operations) and the machine brews one drink at a time anyway. If contention (many threads waiting for the same lock) mattered (a cloud-backed inventory shared by many machines), you would move to optimistic concurrency (work without a lock, then check at write time that nobody changed the data, and retry if they did — e.g. version numbers / compare-and-set on a snapshot) or a database transaction (a group of database changes that succeed or fail together).

Extensions & follow-up questions

1. **"Add a new drink."** — Add one line to the menu data. No code change.
2. **"Sizes (small/medium/large)?"** — A `Size` enum with a multiplier; `recipe.scaled(size)` returns a new recipe with scaled amounts and price.
3. **"Several brewing units?"** — A pool of `Brewer` workers (a `BlockingQueue` — a thread-safe queue where a taker waits while it is empty — of orders consumed by N threads). Inventory stays in the one `IngredientStore`.
4. **"Remote monitoring?"** — Another `LowStockListener` that posts to a telemetry endpoint. The store does not change.
5. **"Brewing steps differ per drink (grind, tamp, steam milk)?"** — Template Method (a base class fixes the order of the steps and subclasses fill in some of them): a `BrewProcess` with fixed steps and hooks (optional steps a subclass may override), or a list of `Step` objects in the recipe data (Command).
6. **"Card payments and change?"** — Reuse the `CashBox` from 14.2 for cash; card payment is another payment device.

Real-world example

Professional espresso machines such as the Thermoplan Mastrena used in Starbucks stores store **drink recipes as configuration**, so a chain can roll out a new beverage without new hardware. Starbucks and Microsoft publicly described (in 2019) connecting store equipment, including espresso machines, to the cloud through Microsoft Azure Sphere, so that each shot's data is collected as telemetry and new recipes are pushed from the cloud instead of being carried to stores on USB drives — the "recipes as

data" and "remote monitoring listener" ideas in this design. Many connected self-serve coffee kiosks also report stock levels remotely, so that a service visit is scheduled before the milk runs out — the low-stock Observer in this design.

In the interview

Plan for 40 minutes: 5 min questions → 5 min entities (ingredient, recipe, store, machine) → 5 min "recipes are data" and the menu format → 15 min code `tryConsume` (lock, all-or-nothing, alert outside the lock) and `order` → 10 min concurrency demo and extensions.

> **30-second answer:** "Recipes are data: a map from ingredient to amount, loaded from a menu, so a new drink is a new row. The ingredient store checks and deducts **every** ingredient of a recipe under one lock, so an order is all-or-nothing and stock never goes negative. When an ingredient crosses its threshold, the store notes it and, after releasing the lock, notifies observers once. Orders return a sealed Served/Rejected result, and a rejected order is refunded in full."

Follow-ups with answers

- *"Why not one class per drink?"* — Drinks differ only in data. Classes would multiply with every size and add-on (class explosion) and every recipe tweak would be a code change.
- *"Why not an `AtomicInteger` per ingredient?"* — Each atomic is safe alone, but a recipe updates several at once; without a common lock, two orders could each take the last milk and beans in different orders, leaving partial deductions.
- *"Why notify outside the lock?"* — A slow or re-entrant listener would block all orders or deadlock.

Self-check

1. **Q:** What does "all-or-nothing" mean for `tryConsume`? **A:** Either every ingredient of the recipe is deducted, or none is (shortages are detected first).
2. **Q:** What race does the single lock prevent? **A:** Check-then-act: two threads both see enough milk, then both deduct, going negative.
3. **Q:** How is an alert storm prevented? **A:** An `EnumSet` records ingredients already alerted; refill above the threshold re-arms them.
4. **Q:** Why is `Recipe.with` returning a new object important? **A:** Customising one order must not change the shared menu recipe.
5. **Q:** When would you use Decorator for add-ons? **A:** When add-ons carry behaviour (extra steps, special description or pricing logic), not just quantities.

TL;DR

- **Recipes are data** (`Map<Ingredient,Integer>` + price), parsed from a menu; customisation creates a new immutable recipe.
- **One lock, check-then-deduct, all-or-nothing**: stock never goes negative under concurrent orders.
- **Observer** for low-stock alerts, fired **once per crossing** and **outside** the lock.
- A **sealed** `OrderResult` makes callers handle Served and Rejected; refund in full on rejection.

14.4 Design an ATM



Figure — An automated teller machine with its card reader, keypad, screen, cash dispenser, deposit slot and printer (course diagram)

An **automated teller machine (ATM)** is a cashier-less machine, placed in a public spot, where a bank's customers can do everyday banking. It exists because branches keep limited hours and people cannot go to a branch every time they need cash or want to pay in money. The interviewer is checking whether you can keep **money correct** (never dispense without debiting, never debit without dispensing, never let two machines overdraw one account), **authenticate** before anything else, and model the machine's **session states** cleanly.

Problem & clarifying questions

Question	Assumed answer
Which transactions?	Balance inquiry, cash withdrawal, cash deposit, check deposit, transfer.
Which account types?	Checking and savings. Withdrawal is from checking (and savings within a daily cap).
How is the user identified?	ATM card + PIN (personal identification number, a short secret code). The card is the main security element.
Does the ATM talk to the bank?	Yes, on every transaction (assume the network is available).
Are deposits available at once?	A check is **not**: it waits for verification. We keep a *total* and an *available* balance.
Which notes are loaded?	$100, $50, $20, $10 cassettes.
Wrong PIN?	Three wrong attempts block the card and the ATM keeps it. The course lists PIN retry limits and card capture as deliberately left out, along with receipt-paper levels, an audit journal and other banks' networks; we add the first two and the journal.
How many customers at a time?	One per machine; the ATM must not shut down mid-session.

Requirements

Parts of the machine (from the course): **card reader** (reads the card), **keypad** (PIN, amounts), **screen** (messages; also used to pick a transaction), **cash dispenser**, **deposit slot** (cash or checks), **printer** (receipts), and a **network link** to the bank.

Functional

1. The customer inserts a card and enters a PIN; the ATM asks the **bank** to authenticate. No transaction is possible without authentication.
2. Five transactions: **balance inquiry** (each account), **deposit cash**, **deposit check**, **withdraw cash** (from checking), **transfer funds** to another account.
3. The ATM **keeps the card** for the whole session; the customer can **cancel** at any time and the card is ejected.
4. The ATM keeps an **internal log (journal)** of transactions, including hardware failures, for the operator to resolve issues.
5. A deposited **check is not added at once**: it waits for manual verification and bank approval. (The course assumes deposits in general are verified first; our code credits counted cash immediately and holds checks — see the trade-off below.)
6. The **bank manager** can see the ATM's system information (reports).

Actors' operations

- **Operator:** turn the ATM on/off with the key switch; refill cash; refill receipt paper and ink; take out deposited cash and checks.
- **Customer:** balance inquiry, withdraw, deposit (cash or check), transfer.
- **Bank manager:** reports of total deposits and total withdrawals; print them; check the cash remaining in the ATM.

Non-functional

- **Security:** PINs never stored or compared in clear on the ATM; limit PIN attempts.
- **Consistency:** debit and dispense happen together or not at all (compensate on failure).
- **Concurrency:** many ATMs act on the same account at once without overdrawing it or deadlocking.
- **Auditability** (every action can be traced afterwards)**:** every action is journaled.

Use cases

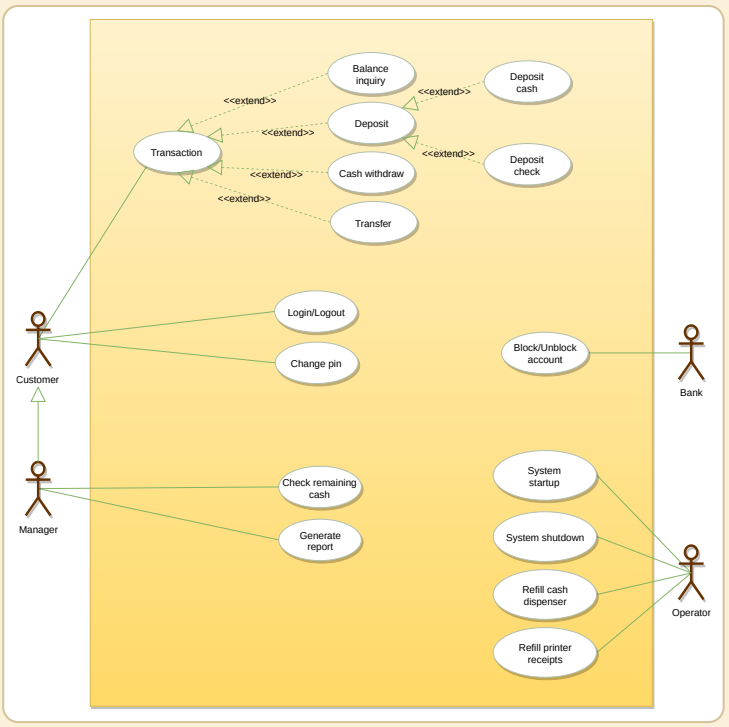


Figure — ATM use case diagram: Customer (balance, withdraw, deposit, transfer), Operator (on/off, refill cash, paper, ink, empty deposits) and Bank Manager (reports, check remaining cash) (course diagram)

The figure above shows three actors. Only the Customer's use cases go through PIN authentication; the Operator works with a physical key and the Bank Manager reads data held at the bank.

Customer authentication is the entry point of every session:

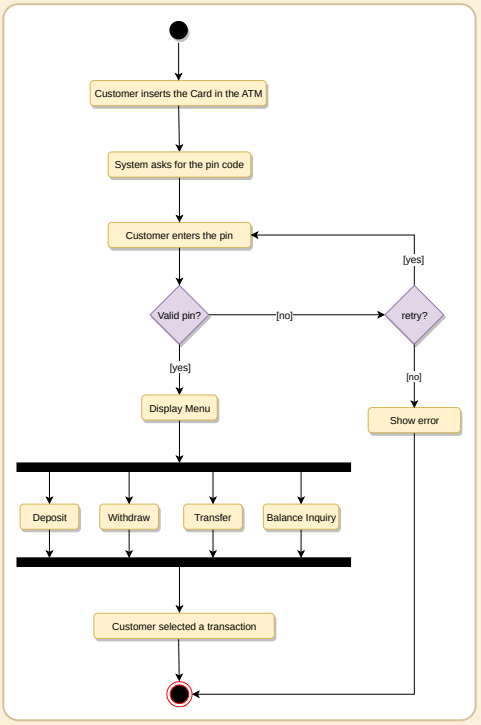


Figure — Activity diagram: insert card, the system asks for the PIN, the customer enters it; an invalid PIN shows an error and offers a retry; a valid PIN displays the menu (balance inquiry, deposit, withdraw, transfer) (course diagram)

Withdraw:

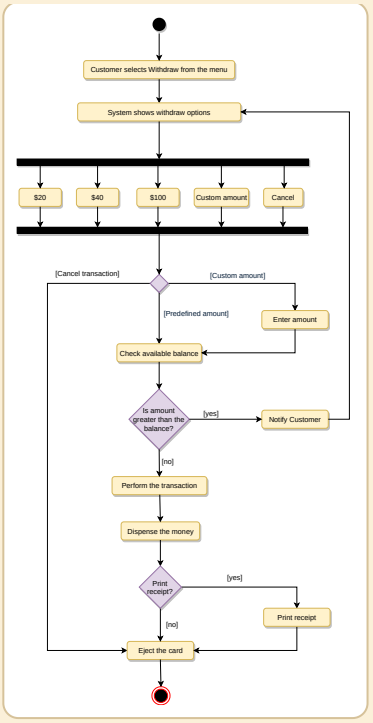


Figure — Activity diagram: choose Withdraw, pick a predefined amount ($20, $40, $100) or enter a custom one, check the available balance (notify the customer if the amount is greater), perform the transaction, dispense, optional receipt, eject the card (course diagram)

Deposit check:

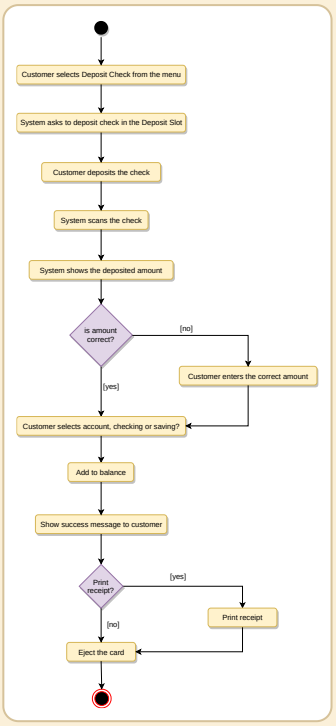


Figure — Activity diagram: choose Deposit Check, pick checking or savings, put the check in the deposit slot, the system scans it and shows the amount; if wrong the customer enters the correct amount; the amount is added, a success message and optional receipt follow, and the card is ejected (course diagram)

Transfer:

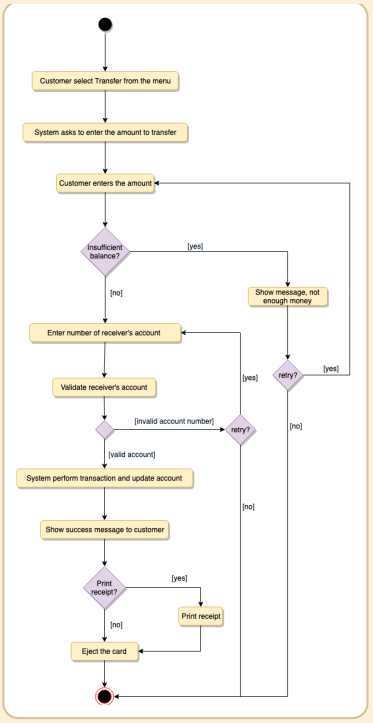


Figure — Activity diagram: enter the amount (retry if balance is insufficient), enter and validate the receiver's account (retry if invalid), perform the transfer, show success, optional receipt, eject card (course diagram)

In the transfer diagram above, note the two retry loops: one for "insufficient balance" and one for "invalid account number". If the customer declines to retry, the diagram goes straight to the end node without the "Eject the card" step; in a real design every exit path must still return the card, which is exactly the card-reader rule in stage 2 below.

Core entities

Class	Responsibility
`Card` (record)	Card number and holder name. (The course also has a `Customer` class with name and email, owning exactly one card; our code folds the holder's name into the card.)
`Account` (abstract)	Id, **total** and **available** balance, a lock; the withdrawal rule hook.
`CheckingAccount`, `SavingsAccount`	Savings overrides the rule to add a **daily withdrawal cap** — the only behavioural difference, which is why they are two classes.
`BankService` (interface)	What the ATM needs from the bank: authenticate, balances, withdraw, reverse, deposits, transfer.
`InMemoryBank`	A stand-in bank: PIN hashes (one-way fingerprints of PINs), failed-attempt counts, account locks, ordered-lock transfers.
`AuthResult` (sealed)	`Ok(accounts)`, `WrongPin(attemptsLeft)` or `Blocked`.
`NoteHandler` + 4 subclasses	**Chain of Responsibility** for $100 → $50 → $20 → $10.
`CashDispenser`	Plans a withdrawal from the chain (dry run: a trial that changes nothing), then dispenses; can jam (fault injection: a deliberately triggered failure, for testing).
`Transaction` (sealed) + 5 records	**Command**: each transaction has one `execute(ctx)`; its rules live only there.
`AtmState` + 4 states	**State**: Idle → CardInserted → Authenticated → Idle, plus OutOfService.
`Atm`	Has an id (the course also gives it a location); owns its parts, holds the session, forwards buttons to the state, keeps the journal.
`CardReader`, `Keypad`, `Screen`, `Printer`, `DepositSlot` (cash / check)	The course's hardware classes. Our code folds them into `Atm`: the `card` field is the reader's slot, `screen(...)` is the screen, the method arguments are the keypad, and printing a receipt would format a journal line.

Class diagram

The course builds the ATM in four stages:

Stage	What it adds	What works after it
1	`Customer`, `Card`, `Account` (checking, savings)	Verify a PIN and read a balance
2	Card reader, dispenser, keypad, screen, printer, deposit slots	Work out and count out a set of notes
3	`Transaction` and its five kinds	Move money, with each rule written in exactly one class
4	`Bank` and `ATM`	Run a full session for a single customer at a time

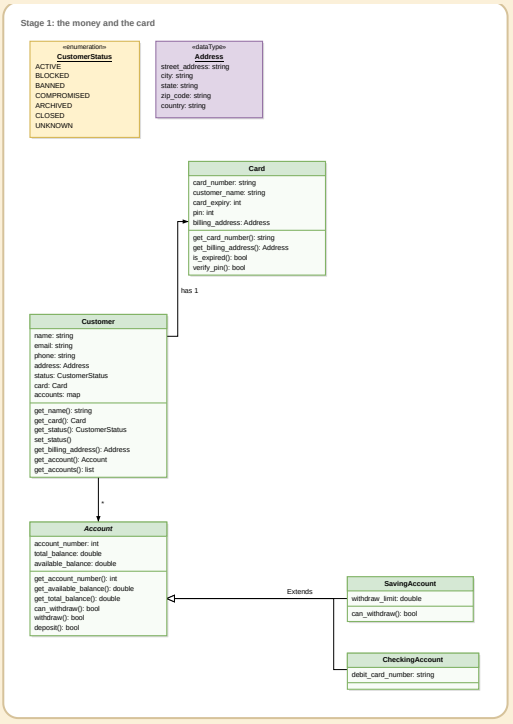


Figure — Stage 1: enumerations, Customer, Card and the abstract Account with Checking and Saving subclasses (course diagram)

Stage 1 — money and card. An account keeps **two balances**. The *total balance* is everything credited to it; the *available balance* is the part the customer may spend right now. A check deposit is the moment they diverge: the check raises the total at once, but the money becomes available only after the check clears. `Account` is abstract with `CheckingAccount` and `SavingsAccount` beneath it; savings overrides a single method to impose its withdrawal cap, and because that is a difference in **behaviour** (not just data) inheritance is justified here instead of a type field. `Customer` (name, email; each customer has **one** card) is linked to its card and accounts by plain associations, since none of them is deleted along with another.

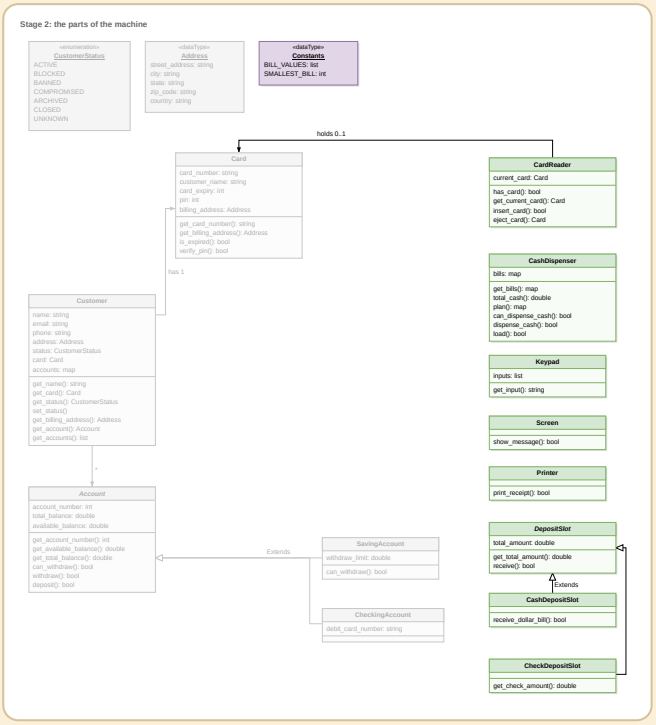


Figure — Stage 2: CardReader, CashDispenser, Keypad, Screen, Printer and the abstract DepositSlot with cash and check slots (course diagram)

Stage 2 — the parts. Six of the seven hardware parts become one class each (the seventh, the network link, is assumed rather than modelled), and none of them knows what a transaction is. Deciding whether an amount (say $85) can be paid from the notes **currently loaded** belongs to the **dispenser**, not to the

withdrawal logic; it tries the largest note first and reports failure when the exact amount is impossible. The card reader has one rule of its own: it **keeps the card for the whole session** and releases it when the session ends, so every failed sign-in path must eject (or retain) the card before it returns; the reader's link to `Card` is the only arrow leaving this stage. `DepositSlot` is abstract, with a cash slot and a check slot below it. It is the physical opening money is pushed into; do not confuse it with the deposit *transaction* of stage 3.

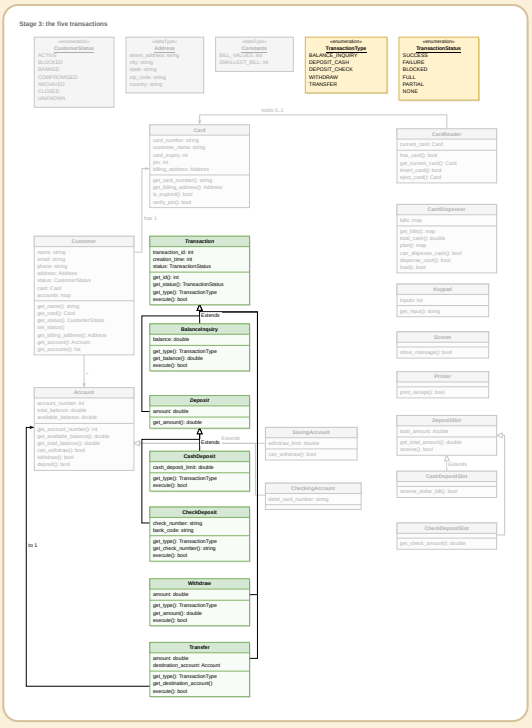


Figure — Stage 3: the abstract Transaction with BalanceInquiry, Deposit (CashDeposit, CheckDeposit), Withdraw and Transfer (course diagram)

Stage 3 — the five transactions. Every transaction type is a class with a single `execute` method, so each rule has exactly one home: nothing outside `Withdraw` needs to know the withdrawal rules. `Deposit` is an intermediate class: cash and check deposits share an amount but differ in when the money becomes spendable. `Transfer` is the only transaction with an outgoing arrow, to the destination account. Inside the course's `Withdraw`, the checks run in this order: **note denominations** first, then the **account**, then the **dispenser**, and the account is debited only once the notes have been counted out. The reason for

that order: if you take the money from the account first and the dispensing then fails, the customer has paid for cash they never received.

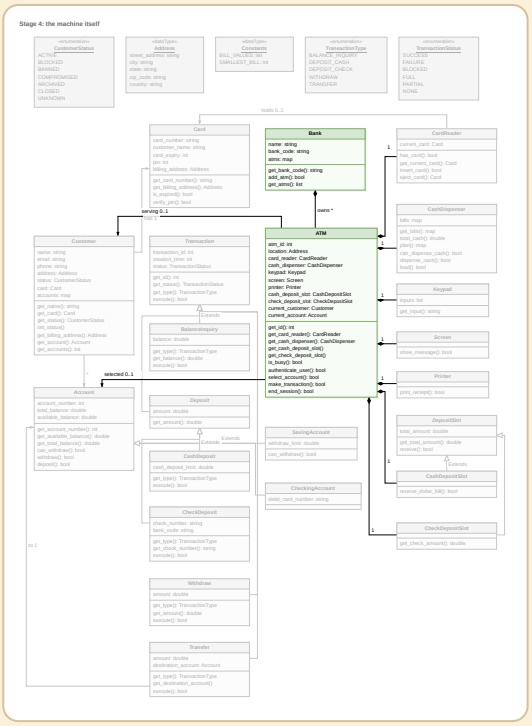


Figure — Stage 4: the ATM owns its parts (composition), refers to the current Customer and Account, and the Bank owns its ATMs (course diagram)

Stage 4 — the machine. The ATM has an **id** (`atmID`) and a **location**, and it **owns** its parts through composition: if the machine is scrapped, its reader, dispenser and screen go with it. Its links to the current `Customer` and `Account` are plain associations that change with each person at the machine. Because the ATM itself holds the current session, "one customer at a time" is enforced by the structure, not just promised in a comment. The `Bank` owns its ATMs and holds all account data; the ATM asks the bank to carry out every customer transaction.

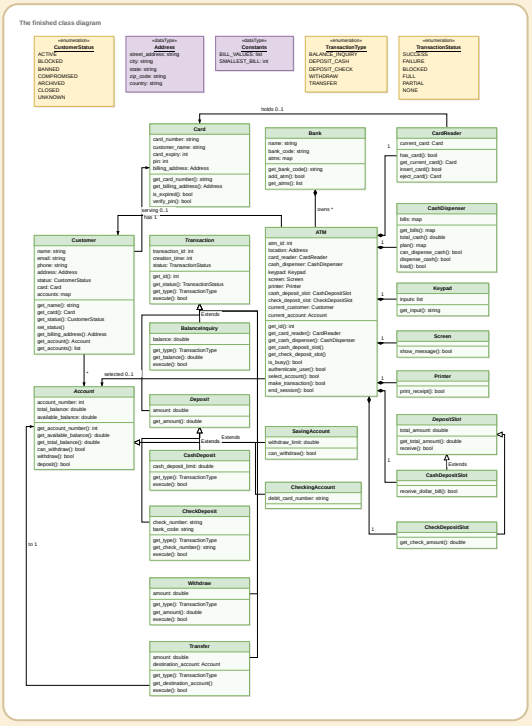
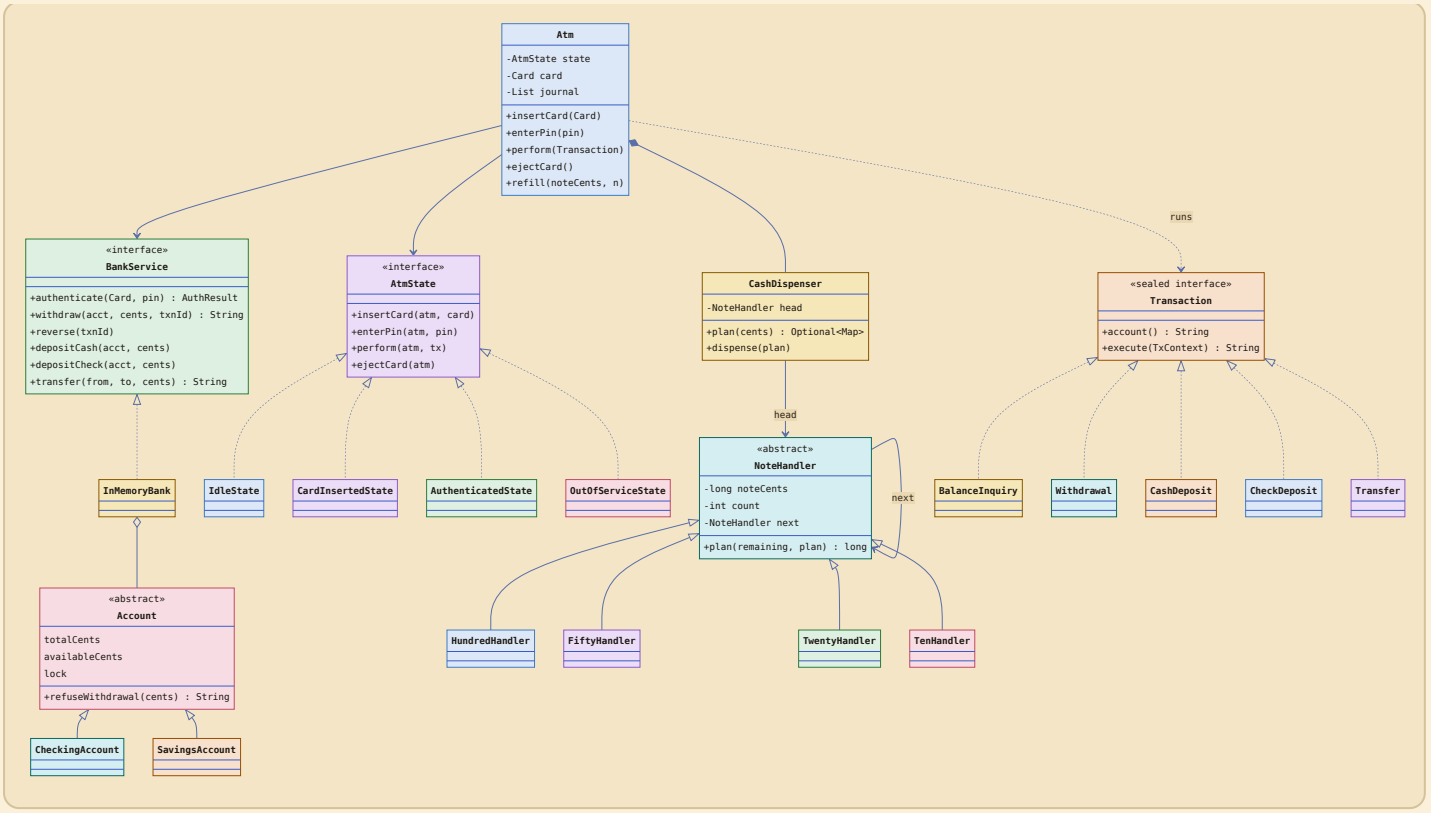


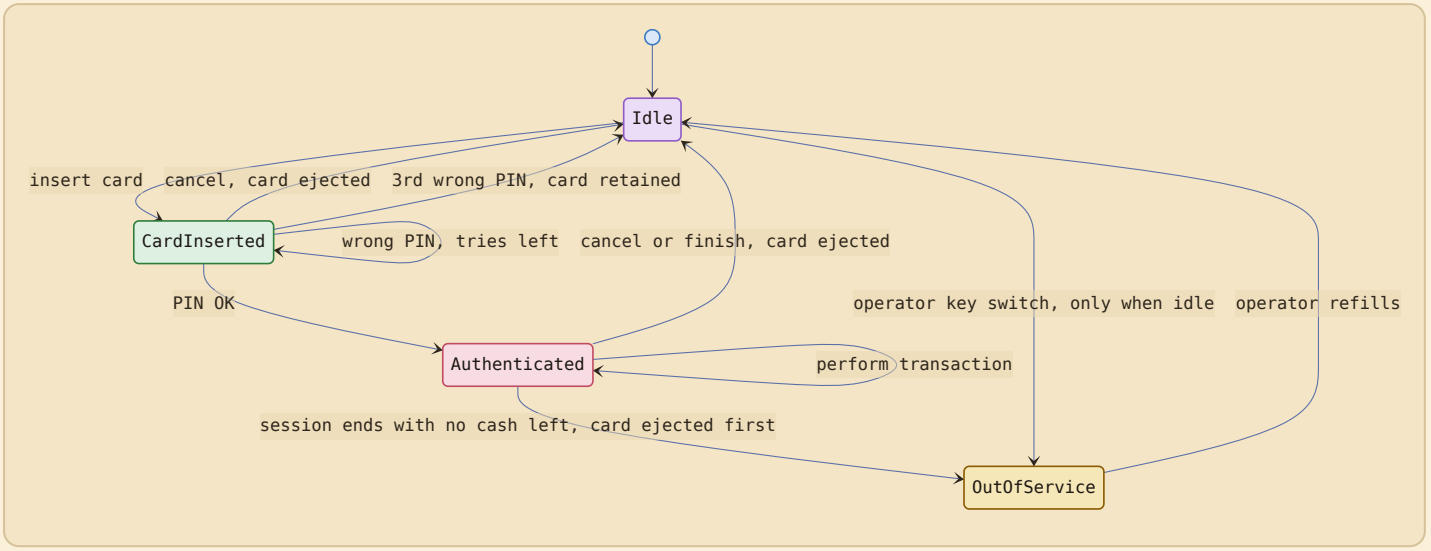
Figure — The finished class diagram of the ATM system (course diagram)

Class diagram of **our Java code**:



Key flows

Session states.



Balance inquiry (the course's sequence diagram):

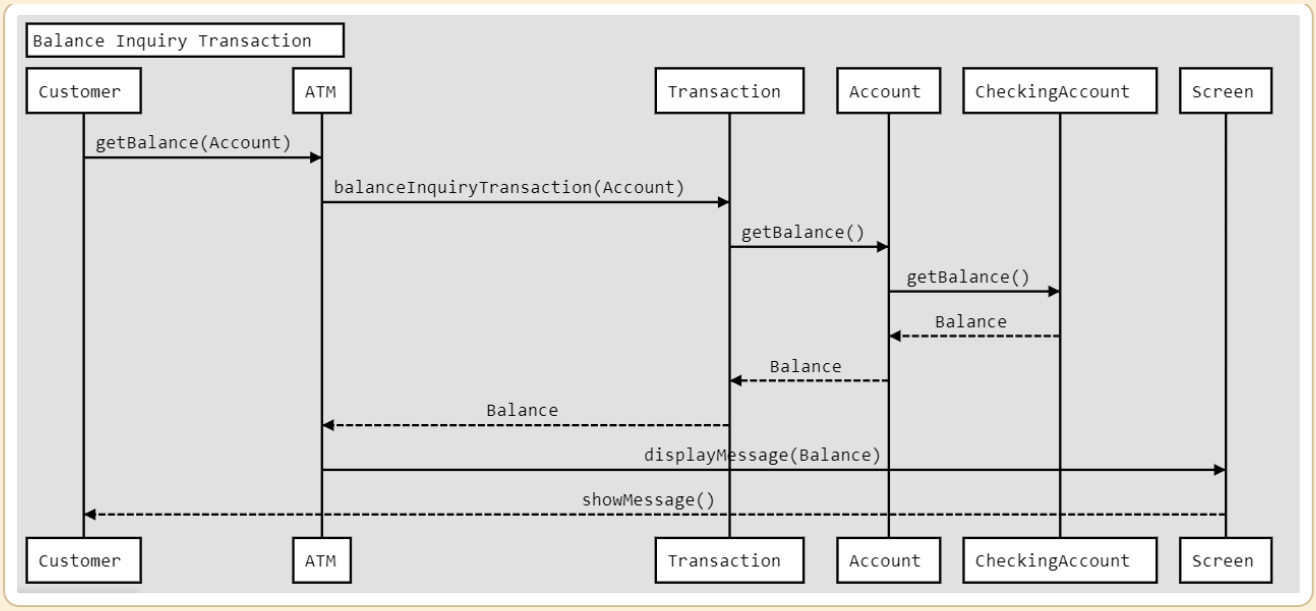
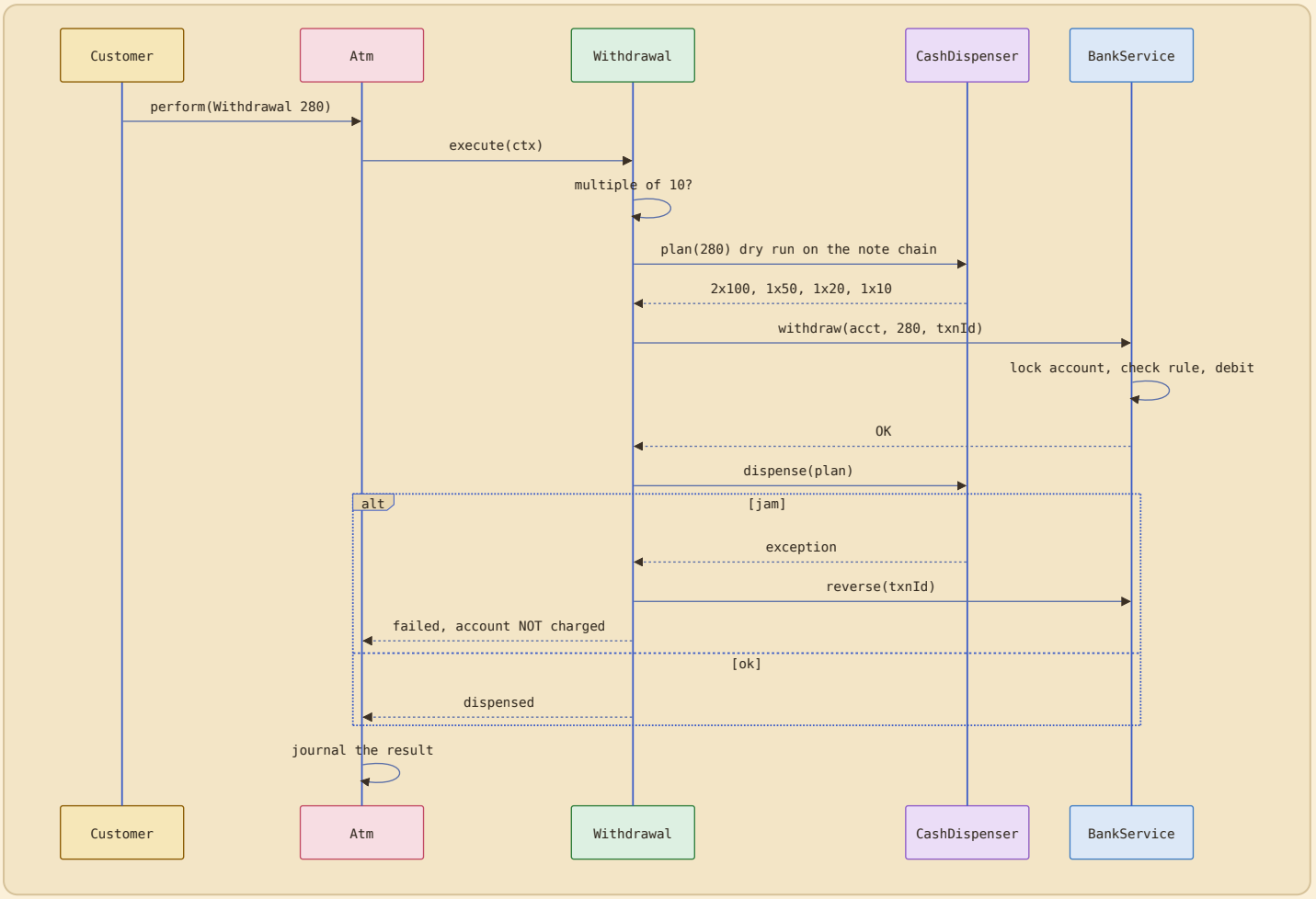


Figure — Sequence diagram of a balance inquiry: Customer → ATM → Transaction → Account → CheckingAccount, and the balance travels back to the Screen (course diagram)

In the sequence above, the ATM never reads the balance itself: it creates a transaction, the transaction asks the account, and the result is shown on the screen.

Withdrawal with compensation (our code):



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Session flow as state classes	**State**	Insert-card/enter-PIN/perform/eject each mean something different per state; e.g. `perform` in Idle is simply refused	Booleans `cardInserted`, `authenticated` and nested `if`s
Dispenser as a chain of note handlers	**Chain of Responsibility**	Each handler takes what it can and passes the rest on; adding a $5 cassette is one new handler linked into the chain	One method with a hard-coded list of notes
Plan first, then dispense	—	The chain's dry run touches nothing; a failed plan leaves the cassettes intact	Removing notes while searching
Each transaction = one class with `execute`	**Command**	The rules of each transaction live in one place; transactions can be journaled, queued or replayed	A big `switch (type)` in the ATM
`Transaction` and `AuthResult` are **sealed**	Java 17+ sealed types	The compiler knows the full list; `switch` over `AuthResult` is exhaustive	Open hierarchies with `instanceof` chains
ATM talks to a `BankService` interface	Dependency Inversion (depend on an interface, not on a concrete class)	The ATM does not care whether the bank is in-memory, ISO 8583 (the standard card-payment message format) over a network, or a test fake	ATM changes balances directly
Total vs available balance	Domain modelling	Makes the check-hold rule correct, not merely convenient	A single balance
Savings is a subclass	Inheritance for **behaviour**	Its withdrawal rule differs (daily cap), so it overrides one hook	A `type` field and `if (type == SAVINGS)`

> **Debit before or after dispensing?** The course debits **after** the notes are counted out — the safer order for the customer — and admits the weakness: a crash between the two leaves the bank short. Our code does the opposite (debit, then dispense) and adds a **compensating action**: if the dispenser jams, `reverse(txnId)` puts the money back. Both are simplifications of what real systems do: make the pair atomic (a hold/authorisation, then a confirmation after dispensing), and **reconcile** (compare two sets of records afterwards and fix any difference) from the ATM's journal when something goes wrong in between. Say this out loud — it is the "power fails halfway through" follow-up.

> **Cash deposits.** The course assumes all deposits are verified before they count. Many banks make cash that the machine itself has counted available immediately (or up to a limit) and hold checks. Our `depositCash` credits both balances and `depositCheck` credits only the total; changing the policy is a one-line change in the bank, not the ATM.

Java implementation

Package `lld.ch14.atm`. Amounts are cents (`long`); notes are $100/$50/$20/$10.

```java
// File: lld/ch14/atm/Bank.java
package lld.ch14.atm;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.HexFormat;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.locks.ReentrantLock;

final class Money {
    private Money() {}
    static String fmt(long cents) { return "$%d.%02d".formatted(cents / 100, cents % 100); }
}

/** Total = everything credited. Available = what can be spent now (a check has not cleared yet). */
abstract class Account {
    final String id;
    final ReentrantLock lock = new ReentrantLock();   // one lock per account, shared by every ATM
    long totalCents;
    long availableCents;

    Account(String id, long openingCents) {
        this.id = id;
        this.totalCents = openingCents;
        this.availableCents = openingCents;
    }

    /** Hook that subclasses refine. Called with the lock held. Returns null if allowed, else the reason. */
    String refuseWithdrawal(long cents) {
        return cents > availableCents ? "insufficient funds" : null;
    }

    void recordWithdrawal(long cents) {}
    void reverseWithdrawal(long cents) {}
}

class CheckingAccount extends Account {
    CheckingAccount(String id, long opening) { super(id, opening); }
}

/** The one behavioural difference: savings has a daily withdrawal cap. */
class SavingsAccount extends Account {
    private final long dailyLimitCents;
    private long withdrawnTodayCents;

    SavingsAccount(String id, long opening, long dailyLimitCents) {
        super(id, opening);
        this.dailyLimitCents = dailyLimitCents;
    }

    @Override String refuseWithdrawal(long cents) {
        if (withdrawnTodayCents + cents > dailyLimitCents) return "savings daily limit " +
Money.fmt(dailyLimitCents);
        return super.refuseWithdrawal(cents);
    }

    @Override void recordWithdrawal(long cents) { withdrawnTodayCents += cents; }
    @Override void reverseWithdrawal(long cents) { withdrawnTodayCents -= cents; }
}

record Card(String number, String holder) {}

sealed interface AuthResult permits AuthResult.Ok, AuthResult.WrongPin, AuthResult.Blocked {
    record Ok(List<String> accountIds) implements AuthResult {}
    record WrongPin(int attemptsLeft) implements AuthResult {}
    record Blocked() implements AuthResult {}
}
```

```java
/** What the ATM needs from the bank. The ATM never touches balances directly. */
interface BankService {
    AuthResult authenticate(Card card, String pin);
    long availableBalance(String accountId);
    long totalBalance(String accountId);
    String withdraw(String accountId, long cents, String txnId);   // null = OK, else reason
    void reverse(String txnId);
    void depositCash(String accountId, long cents);
    void depositCheck(String accountId, long cents);
    void clearCheck(String accountId, long cents);
    String transfer(String from, String to, long cents);
}

final class InMemoryBank implements BankService {
    private static final int MAX_PIN_TRIES = 3;
    private final Map<String, Account> accounts = new ConcurrentHashMap<>();
    private final Map<String, String> pinHashByCard = new ConcurrentHashMap<>();
    private final Map<String, List<String>> accountsByCard = new ConcurrentHashMap<>();
    private final Map<String, Integer> failedTries = new ConcurrentHashMap<>();
    private final Map<String, Object[]> withdrawals = new ConcurrentHashMap<>();   // txnId -> {accountId, cents}

    void open(Account a) { accounts.put(a.id, a); }

    void issueCard(Card card, String pin, List<String> accountIds) {
        pinHashByCard.put(card.number(), hash(card.number(), pin));
        accountsByCard.put(card.number(), List.copyOf(accountIds));
    }

    @Override public AuthResult authenticate(Card card, String pin) {
        String n = card.number();
        if (failedTries.getOrDefault(n, 0) >= MAX_PIN_TRIES) return new AuthResult.Blocked();
        if (hash(n, pin).equals(pinHashByCard.get(n))) {
            failedTries.remove(n);
            return new AuthResult.Ok(accountsByCard.get(n));
        }
        int tries = failedTries.merge(n, 1, Integer::sum);
        return tries >= MAX_PIN_TRIES ? new AuthResult.Blocked() : new AuthResult.WrongPin(MAX_PIN_TRIES - tries);
    }

    @Override public long availableBalance(String id) { return read(id, true); }
    @Override public long totalBalance(String id) { return read(id, false); }

    @Override public String withdraw(String id, long cents, String txnId) {
        Account a = acc(id);
        a.lock.lock();
        try {                                          // check-then-act under ONE lock: atomic
            String refusal = a.refuseWithdrawal(cents);
            if (refusal != null) return refusal;
            a.availableCents -= cents;
            a.totalCents -= cents;
            a.recordWithdrawal(cents);
            withdrawals.put(txnId, new Object[]{id, cents});
            return null;
        } finally {
            a.lock.unlock();
        }
    }

    /** Compensating action when the cash could not be handed out. */
    @Override public void reverse(String txnId) {
        Object[] w = withdrawals.remove(txnId);
        if (w == null) return;
        Account a = acc((String) w[0]);
        long cents = (Long) w[1];
        a.lock.lock();
        try {
            a.availableCents += cents;
            a.totalCents += cents;
            a.reverseWithdrawal(cents);
        } finally {
```

```
            a.lock.unlock();
        }
    }

    @Override public void depositCash(String id, long cents) { credit(id, cents, true); }
    @Override public void depositCheck(String id, long cents) { credit(id, cents, false); }   // on hold

    @Override public void clearCheck(String id, long cents) {
        Account a = acc(id);
        a.lock.lock();
        try { a.availableCents += cents; } finally { a.lock.unlock(); }
    }

    /** Locks both accounts in a fixed (id) order, so two opposite transfers cannot deadlock. */
    @Override public String transfer(String from, String to, long cents) {
        if (cents <= 0) return "amount must be positive";
        if (from.equals(to)) return "same account";
        Account a = acc(from), b = acc(to);
        Account first = from.compareTo(to) < 0 ? a : b;
        Account second = first == a ? b : a;
        first.lock.lock();
        try {
            second.lock.lock();
            try {
                if (cents > a.availableCents) return "insufficient funds";
                a.availableCents -= cents;
                a.totalCents -= cents;
                b.availableCents += cents;
                b.totalCents += cents;
                return null;
            } finally {
                second.lock.unlock();
            }
        } finally {
            first.lock.unlock();
        }
    }

    private void credit(String id, long cents, boolean available) {
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        Account a = acc(id);
        a.lock.lock();
        try {
            a.totalCents += cents;
            if (available) a.availableCents += cents;
        } finally {
            a.lock.unlock();
        }
    }

    private long read(String id, boolean available) {
        Account a = acc(id);
        a.lock.lock();
        try { return available ? a.availableCents : a.totalCents; } finally { a.lock.unlock(); }
    }

    private Account acc(String id) {
        Account a = accounts.get(id);
        if (a == null) throw new IllegalArgumentException("no account " + id);
        return a;
    }

    /** Demo only. Real banks never see the clear PIN: the PIN pad encrypts it and an HSM verifies it. */
    private static String hash(String card, String pin) {
        try {
            byte[] d = MessageDigest.getInstance("SHA-256").digest((card + ":" +
pin).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(d);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException(e);
        }
```

```
    }
}
```

```java
// File: lld/ch14/atm/CashDispenser.java
package lld.ch14.atm;

import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Optional;

/**
 * Chain of Responsibility: each handler owns one note value. It takes as many of its notes
 * as it can for the amount, then passes whatever is left to the next (smaller) handler.
 */
abstract class NoteHandler {
    private final long noteCents;
    private int count;
    private NoteHandler next;

    NoteHandler(long noteCents, int count) {
        this.noteCents = noteCents;
        this.count = count;
    }

    NoteHandler linkTo(NoteHandler next) {
        this.next = next;
        return next;
    }

    /** Dry run: fills 'plan', returns the amount that nobody in the chain could cover. */
    long plan(long remaining, Map<Long, Integer> plan) {
        int use = (int) Math.min(count, remaining / noteCents);
        if (use > 0) plan.put(noteCents, use);
        long left = remaining - use * noteCents;
        return (left == 0 || next == null) ? left : next.plan(left, plan);
    }

    void take(Map<Long, Integer> plan) {
        count -= plan.getOrDefault(noteCents, 0);
        if (next != null) next.take(plan);
    }

    long total() { return noteCents * count + (next == null ? 0 : next.total()); }
    void add(int n) { count += n; }
    long noteCents() { return noteCents; }
    NoteHandler next() { return next; }
}

final class HundredHandler extends NoteHandler { HundredHandler(int n) { super(100_00, n); } }
final class FiftyHandler extends NoteHandler { FiftyHandler(int n) { super(50_00, n); } }
final class TwentyHandler extends NoteHandler { TwentyHandler(int n) { super(20_00, n); } }
final class TenHandler extends NoteHandler { TenHandler(int n) { super(10_00, n); } }

final class CashDispenser {
    private final NoteHandler head;
    private boolean jamNext;                        // hardware fault injection for the demo

    CashDispenser(int hundreds, int fifties, int twenties, int tens) {
        head = new HundredHandler(hundreds);
        head.linkTo(new FiftyHandler(fifties)).linkTo(new TwentyHandler(twenties)).linkTo(new TenHandler(tens));
    }

    /** Plan first, touch nothing. Empty = this amount cannot be made from the notes loaded. */
    synchronized Optional<Map<Long, Integer>> plan(long cents) {
        Map<Long, Integer> plan = new LinkedHashMap<>();
        return head.plan(cents, plan) == 0 ? Optional.of(plan) : Optional.empty();
    }

    /** Physically count out the notes. May fail (jam), in which case nothing leaves the machine. */
    synchronized void dispense(Map<Long, Integer> plan) {
        if (jamNext) { jamNext = false; throw new IllegalStateException("note jam"); }
        head.take(plan);
    }
```

```
    synchronized void refill(long noteCents, int n) {
        for (NoteHandler h = head; h != null; h = h.next()) if (h.noteCents() == noteCents) h.add(n);
    }

    synchronized long total() { return head.total(); }
    synchronized void jamNextDispense() { jamNext = true; }
}
```

```java
// File: lld/ch14/atm/Transactions.java
package lld.ch14.atm;

import java.util.Map;
import java.util.stream.Collectors;

/** Everything a transaction may use. */
record TxContext(BankService bank, CashDispenser dispenser, String txnId) {}

/** Each transaction is one class with one execute(): Command pattern. Sealed = the full list is known. */
sealed interface Transaction permits BalanceInquiry, Withdrawal, CashDeposit, CheckDeposit, Transfer {
    String account();
    String execute(TxContext ctx);
}

record BalanceInquiry(String account) implements Transaction {
    public String execute(TxContext c) {
        return "Balance %s: available %s, total %s".formatted(account,
                Money.fmt(c.bank().availableBalance(account)), Money.fmt(c.bank().totalBalance(account)));
    }
}

record Withdrawal(String account, long cents) implements Transaction {
    static final long SMALLEST_NOTE = 10_00;

    public String execute(TxContext c) {
        if (cents <= 0 || cents % SMALLEST_NOTE != 0) return "Withdrawal refused: amount must be a multiple of
$10";
        var plan = c.dispenser().plan(cents);                                   // 1. can the notes make it?
        if (plan.isEmpty()) return "Withdrawal refused: ATM cannot make " + Money.fmt(cents) + " from its notes";
        String refusal = c.bank().withdraw(account, cents, c.txnId());          // 2. atomic debit at the bank
        if (refusal != null) return "Withdrawal refused: " + refusal;
        try {
            c.dispenser().dispense(plan.get());                                 // 3. hand out the cash
        } catch (IllegalStateException hardwareFault) {
            c.bank().reverse(c.txnId());                                        // compensate: undo the debit
            return "Withdrawal failed (" + hardwareFault.getMessage() + "), account NOT charged";
        }
        return "Dispensed " + Money.fmt(cents) + " as " + describe(plan.get());
    }

    private static String describe(Map<Long, Integer> plan) {
        return plan.entrySet().stream().map(e -> e.getValue() + "x$" + e.getKey() / 100)
                .collect(Collectors.joining(" + "));
    }
}

record CashDeposit(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        c.bank().depositCash(account, cents);
        return "Cash deposit " + Money.fmt(cents) + " credited";
    }
}

record CheckDeposit(String account, long cents) implements Transaction {
    public String execute(TxContext c) {
        c.bank().depositCheck(account, cents);
        return "Check " + Money.fmt(cents) + " received: on hold until the bank clears it";
    }
}

record Transfer(String account, String toAccount, long cents) implements Transaction {
    public String execute(TxContext c) {
        String refusal = c.bank().transfer(account, toAccount, cents);
        return refusal == null ? "Transferred " + Money.fmt(cents) + " to " + toAccount
                               : "Transfer refused: " + refusal;
    }
}
```

```java
// File: lld/ch14/atm/States.java
package lld.ch14.atm;

/** State pattern: what each button does depends on where the session is. */
interface AtmState {
    default void insertCard(Atm atm, Card card) { atm.screen("Card slot is not ready."); }
    default void enterPin(Atm atm, String pin) { atm.screen("Please insert your card first."); }
    default void perform(Atm atm, Transaction tx) { atm.screen("Please sign in first."); }
    default void ejectCard(Atm atm) { atm.screen("No card to eject."); }
    String name();
}

final class IdleState implements AtmState {
    static final IdleState INSTANCE = new IdleState();

    @Override public void insertCard(Atm atm, Card card) {
        atm.acceptCard(card);
        atm.screen("Hello " + card.holder() + ". Enter PIN.");
        atm.setState(CardInsertedState.INSTANCE);
    }
    @Override public String name() { return "IDLE"; }
}

final class CardInsertedState implements AtmState {
    static final CardInsertedState INSTANCE = new CardInsertedState();

    @Override public void enterPin(Atm atm, String pin) {
        switch (atm.bank().authenticate(atm.card(), pin)) {
            case AuthResult.Ok ok -> {
                atm.startSession(ok.accountIds());
                atm.screen("PIN OK. Accounts: " + ok.accountIds());
                atm.setState(AuthenticatedState.INSTANCE);
            }
            case AuthResult.WrongPin w -> atm.screen("Wrong PIN. Tries left: " + w.attemptsLeft());
            case AuthResult.Blocked b -> {
                atm.screen("Too many wrong PINs. Card retained. Contact your bank.");
                atm.retainCard();
                atm.finishSession();
            }
        }
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
    @Override public String name() { return "CARD_INSERTED"; }
}

final class AuthenticatedState implements AtmState {
    static final AuthenticatedState INSTANCE = new AuthenticatedState();

    @Override public void perform(Atm atm, Transaction tx) {
        if (!atm.sessionAccounts().contains(tx.account())) {
            atm.screen("Account " + tx.account() + " is not linked to this card.");
            return;
        }
        atm.run(tx);
    }
    @Override public void ejectCard(Atm atm) { atm.returnCard(); atm.finishSession(); }
    @Override public String name() { return "AUTHENTICATED"; }
}

final class OutOfServiceState implements AtmState {
    static final OutOfServiceState INSTANCE = new OutOfServiceState();

    @Override public void insertCard(Atm atm, Card card) { atm.screen("Out of service. Card returned."); }
    @Override public String name() { return "OUT_OF_SERVICE"; }
}
```

```java
// File: lld/ch14/atm/Atm.java
package lld.ch14.atm;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

/** One machine: owns its parts, runs one session at a time, forwards buttons to its state. */
final class Atm {
    private static final AtomicLong TXN_SEQ = new AtomicLong(1);
    private final String id;
    private final BankService bank;
    private final CashDispenser dispenser;
    private final boolean verbose;
    private final List<String> journal = new ArrayList<>();   // operator / bank-manager audit log
    private AtmState state = IdleState.INSTANCE;
    private Card card;                                        // the card reader's slot
    private List<String> sessionAccounts = List.of();

    Atm(String id, BankService bank, CashDispenser dispenser, boolean verbose) {
        this.id = id;
        this.bank = bank;
        this.dispenser = dispenser;
        this.verbose = verbose;
    }

    // ----- customer buttons (one customer at a time: synchronized) -----
    synchronized void insertCard(Card c) { state.insertCard(this, c); }
    synchronized void enterPin(String pin) { state.enterPin(this, pin); }
    synchronized void perform(Transaction tx) { state.perform(this, tx); }
    synchronized void ejectCard() { state.ejectCard(this); }

    // ----- operator -----
    /** Key switch. The ATM must not shut down while serving a customer, so this only works when idle. */
    synchronized boolean setOutOfService() {
        if (state != IdleState.INSTANCE && state != OutOfServiceState.INSTANCE) {
            journal("operator: switch-off refused, session in progress");
            return false;
        }
        state = OutOfServiceState.INSTANCE;
        journal("operator: out of service");
        return true;
    }
    synchronized void refill(long noteCents, int n) {
        dispenser.refill(noteCents, n);
        journal("operator: refilled " + n + " x " + Money.fmt(noteCents));
        if (state == OutOfServiceState.INSTANCE) state = IdleState.INSTANCE;
    }

    // ----- helpers for states -----
    void setState(AtmState s) { state = s; }
    BankService bank() { return bank; }
    Card card() { return card; }
    List<String> sessionAccounts() { return sessionAccounts; }
    void acceptCard(Card c) { card = c; journal("card in " + mask(c)); }
    void startSession(List<String> accounts) { sessionAccounts = accounts; }

    void returnCard() {
        screen("Card ejected. Goodbye.");
        journal("card out " + mask(card));
        endSession();
    }

    void retainCard() { journal("card RETAINED " + mask(card)); endSession(); }

    void run(Transaction tx) {
        String txnId = id + "-" + TXN_SEQ.getAndIncrement();
        String result = tx.execute(new TxContext(bank, dispenser, txnId));
        screen(result);
        journal(txnId + " " + tx.getClass().getSimpleName() + ": " + result);
```

```
    }

    /** End of a session: back to Idle, or OutOfService if the cash ran out (never mid-session, so the card is returned first). */
    void finishSession() {
        if (dispenser.total() == 0) {
            state = OutOfServiceState.INSTANCE;
            journal("cash empty: out of service");
        } else {
            state = IdleState.INSTANCE;
        }
    }

    void screen(String msg) { if (verbose) System.out.println("   [" + id + " screen] " + msg); }
    List<String> journal() { return List.copyOf(journal); }
    String stateName() { return state.name(); }
    long cashLeft() { return dispenser.total(); }
    void dispenserJamForDemo() { dispenser.jamNextDispense(); }

    private void journal(String line) { journal.add(line); }
    private void endSession() { card = null; sessionAccounts = List.of(); }
    private static String mask(Card c) { return "****" + c.number().substring(c.number().length() - 4); }
}
```

```java
// File: lld/ch14/atm/Main.java
package lld.ch14.atm;

import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        InMemoryBank bank = new InMemoryBank();
        bank.open(new CheckingAccount("CHK-1", 1_000_00));
        bank.open(new SavingsAccount("SAV-1", 2_000_00, 100_00));
        bank.open(new CheckingAccount("CHK-2", 50_00));
        Card alice = new Card("4000123412341111", "Alice");
        Card bob = new Card("4000999988882222", "Bob");
        bank.issueCard(alice, "1234", List.of("CHK-1", "SAV-1"));
        bank.issueCard(bob, "9999", List.of("CHK-2"));

        Atm atm = new Atm("ATM-7", bank, new CashDispenser(2, 2, 5, 3), true);
        System.out.println("Cash loaded: " + Money.fmt(atm.cashLeft()));

        System.out.println("\n== Alice: wrong PIN, then right PIN ==");
        atm.perform(new BalanceInquiry("CHK-1"));      // not signed in
        atm.insertCard(alice);
        atm.enterPin("0000");
        atm.enterPin("1234");
        atm.perform(new BalanceInquiry("CHK-1"));
        atm.perform(new Withdrawal("CHK-1", 280_00));   // 2x100 + 1x50 + 1x20 + 1x10
        atm.perform(new Withdrawal("CHK-1", 85_00));    // not a multiple of $10
        atm.perform(new Withdrawal("SAV-1", 140_00));   // over the $100 savings daily cap
        atm.perform(new CheckDeposit("CHK-1", 500_00));
        atm.perform(new BalanceInquiry("CHK-1"));       // total grows, available does not
        atm.perform(new Transfer("SAV-1", "CHK-2", 150_00));
        atm.perform(new BalanceInquiry("CHK-2"));       // not Alice's account
        System.out.println("   (dispenser jams on the next withdrawal)");
        atm.dispenserJamForDemo();
        atm.perform(new Withdrawal("CHK-1", 100_00));
        atm.perform(new BalanceInquiry("CHK-1"));
        atm.ejectCard();

        System.out.println("\n== Bank clears the check overnight ==");
        bank.clearCheck("CHK-1", 500_00);
        System.out.println("   CHK-1 available now " + Money.fmt(bank.availableBalance("CHK-1")));

        System.out.println("\n== Bob forgets his PIN three times ==");
        atm.insertCard(bob);
        atm.enterPin("1111");
        atm.enterPin("2222");
        atm.enterPin("3333");
        Atm branch = new Atm("ATM-9", bank, new CashDispenser(1, 1, 1, 1), true);
        branch.insertCard(bob);                          // same card number at another ATM
        branch.enterPin("9999");                         // blocked at the bank, even with the right PIN

        System.out.println("\n== Two ATMs, one account, same instant: $300 each from $500 ==");
        bank.open(new CheckingAccount("JOINT", 500_00));
        Card c1 = new Card("4111000000000001", "Pat"), c2 = new Card("4111000000000002", "Sam");
        bank.issueCard(c1, "1111", List.of("JOINT"));
        bank.issueCard(c2, "2222", List.of("JOINT"));
        Atm east = new Atm("EAST", bank, new CashDispenser(10, 0, 0, 0), false);
        Atm west = new Atm("WEST", bank, new CashDispenser(10, 0, 0, 0), false);
        east.insertCard(c1); east.enterPin("1111");
        west.insertCard(c2); west.enterPin("2222");
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(2)) {
            for (Atm a : List.of(east, west)) pool.submit(() -> { go.await(); a.perform(new Withdrawal("JOINT",
300_00)); return null; });
            go.countDown();
        }
```

```
        long dispensed = (10 * 100_00 - east.cashLeft()) + (10 * 100_00 - west.cashLeft());
        System.out.println("   cash handed out " + Money.fmt(dispensed) + ", JOINT left " +
Money.fmt(bank.availableBalance("JOINT")));

        System.out.println("\n== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==");
        bank.open(new CheckingAccount("A", 1_000_00));
        bank.open(new CheckingAccount("B", 1_000_00));
        AtomicInteger ok = new AtomicInteger();
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 1000; i++) {
                pool.submit(() -> { if (bank.transfer("A", "B", 1_00) == null) ok.incrementAndGet(); });
                pool.submit(() -> { if (bank.transfer("B", "A", 1_00) == null) ok.incrementAndGet(); });
            }
        }
        System.out.println("   successful=" + ok + ", A+B=" + Money.fmt(bank.totalBalance("A") +
bank.totalBalance("B")));

        System.out.println("\n== ATM-9 runs out of cash; the operator refills it ==");
        branch.insertCard(alice);
        branch.enterPin("1234");
        System.out.println("   operator key switch mid-session accepted? " + branch.setOutOfService());
        branch.perform(new Withdrawal("CHK-1", 180_00));  // takes every note in ATM-9
        System.out.println("   state after the last notes: " + branch.stateName() + " (card still inside)");
        branch.ejectCard();
        System.out.println("   state after the session: " + branch.stateName());
        branch.insertCard(alice);
        branch.refill(20_00, 10);
        System.out.println("   state after refill: " + branch.stateName() + ", cash " +
Money.fmt(branch.cashLeft()));

        System.out.println("\n== Journal of ATM-7 (for the operator and bank manager) ==");
        atm.journal().forEach(l -> System.out.println("   " + l));
    }
}
```

How the code works, part by part

- **PIN handling:** the bank stores a SHA-256 hash (a standard one-way function that turns any input into a fixed-length fingerprint that cannot practically be reversed) of card+PIN and counts failures; the third failure returns `Blocked` and the ATM **retains** the card. (The course's code compares the PIN inside `Card` and flags that as a simplification. Real banks never see the clear PIN: the PIN pad encrypts it into a PIN block and a hardware security module (HSM) verifies it. A plain hash of a 4-digit PIN is also easy to brute-force (try every possible value until one matches), which is one more reason it is demo-only.)
- **Account locks:** each `Account` has its own `ReentrantLock`, shared by every ATM. `withdraw` does check-and-debit under that lock, so two ATMs cannot both spend the same $500.
- **Transfers** lock both accounts in **id order**. Two opposite transfers (A→B and B→A) therefore always lock in the same order and cannot deadlock (Chapter 13).
- **Chain of Responsibility:** `HundredHandler → FiftyHandler → TwentyHandler → TenHandler`. `plan` fills a map and returns the uncovered remainder; `take` removes the planned notes.
- **Command:** each transaction record has `execute(TxContext)`. `AuthenticatedState.perform` first checks the transaction's account belongs to this card.
- **Journal:** the `Atm` logs card in/out, retained cards, every transaction result, operator actions and "cash empty".
- **Out of service:** `finishSession` (called after every eject or retain) chooses Idle or OutOfService, so the machine never changes to OutOfService while a card is still inside. The operator's key switch (`setOutOfService`) is refused during a session.
- **Input checks:** the bank refuses transfers and deposits of zero or negative amounts.

Output:

```
Cash loaded: $430.00

== Alice: wrong PIN, then right PIN ==
   [ATM-7 screen] Please sign in first.
   [ATM-7 screen] Hello Alice. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   [ATM-7 screen] Balance CHK-1: available $1000.00, total $1000.00
   [ATM-7 screen] Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   [ATM-7 screen] Withdrawal refused: amount must be a multiple of $10
   [ATM-7 screen] Withdrawal refused: savings daily limit $100.00
   [ATM-7 screen] Check $500.00 received: on hold until the bank clears it
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Transferred $150.00 to CHK-2
   [ATM-7 screen] Account CHK-2 is not linked to this card.
   (dispenser jams on the next withdrawal)
   [ATM-7 screen] Withdrawal failed (note jam), account NOT charged
   [ATM-7 screen] Balance CHK-1: available $720.00, total $1220.00
   [ATM-7 screen] Card ejected. Goodbye.

== Bank clears the check overnight ==
   CHK-1 available now $1220.00

== Bob forgets his PIN three times ==
   [ATM-7 screen] Hello Bob. Enter PIN.
   [ATM-7 screen] Wrong PIN. Tries left: 2
   [ATM-7 screen] Wrong PIN. Tries left: 1
   [ATM-7 screen] Too many wrong PINs. Card retained. Contact your bank.
   [ATM-9 screen] Hello Bob. Enter PIN.
   [ATM-9 screen] Too many wrong PINs. Card retained. Contact your bank.

== Two ATMs, one account, same instant: $300 each from $500 ==
   cash handed out $300.00, JOINT left $200.00

== 2,000 opposite transfers in parallel (ordered locking: no deadlock) ==
   successful=2000, A+B=$2000.00

== ATM-9 runs out of cash; the operator refills it ==
   [ATM-9 screen] Hello Alice. Enter PIN.
   [ATM-9 screen] PIN OK. Accounts: [CHK-1, SAV-1]
   operator key switch mid-session accepted? false
   [ATM-9 screen] Dispensed $180.00 as 1x$100 + 1x$50 + 1x$20 + 1x$10
   state after the last notes: AUTHENTICATED (card still inside)
   [ATM-9 screen] Card ejected. Goodbye.
   state after the session: OUT_OF_SERVICE
   [ATM-9 screen] Out of service. Card returned.
   state after refill: IDLE, cash $200.00

== Journal of ATM-7 (for the operator and bank manager) ==
   card in ****1111
   ATM-7-1 BalanceInquiry: Balance CHK-1: available $1000.00, total $1000.00
   ATM-7-2 Withdrawal: Dispensed $280.00 as 2x$100 + 1x$50 + 1x$20 + 1x$10
   ATM-7-3 Withdrawal: Withdrawal refused: amount must be a multiple of $10
   ATM-7-4 Withdrawal: Withdrawal refused: savings daily limit $100.00
   ATM-7-5 CheckDeposit: Check $500.00 received: on hold until the bank clears it
   ATM-7-6 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   ATM-7-7 Transfer: Transferred $150.00 to CHK-2
   ATM-7-8 Withdrawal: Withdrawal failed (note jam), account NOT charged
   ATM-7-9 BalanceInquiry: Balance CHK-1: available $720.00, total $1220.00
   card out ****1111
   card in ****2222
   card RETAINED ****2222
```

Read the output: $280 comes out as 2×$100 + $50 + $20 + $10; $85 is refused (not a multiple of $10); the savings cap refuses $140; after the $500 check, *total* grows but *available* does not until the bank clears it; the jam leaves the balance unchanged; Bob's card is blocked at the bank, so even another ATM refuses the right PIN; two ATMs racing on a $500 joint account (one account shared by two people) hand out only $300; 2,000 opposite transfers finish with no deadlock and the total conserved; ATM-9 refuses

the key switch mid-session, pays out its last $180, ejects the card, and only then goes out of service until the operator refills it.

Concurrency & edge cases

Situation	Risk	Handling
Two ATMs withdraw from one account at once	Overdraft (check-then-act race)	Check and debit under the **account lock** at the bank
A→B and B→A transfers at once	**Deadlock** (each holds one lock, waits for the other)	Lock in a global order (account id)
Dispenser jams after the debit	Customer charged, no cash	Compensating `reverse(txnId)`; journal entry for reconciliation
Amount not makeable from loaded notes	Partial dispense	Dry-run `plan` first; refuse before touching the bank
Greedy chain misses a combination	e.g. $60 with one $50 and three $20 but no $10: greedy takes $50, then cannot make $10, though 3×$20 works	Acceptable for standard cassettes; mention a DP planner (as in 14.2) if asked
Wrong PIN many times	Brute force	3 tries, then block at the bank (so another ATM also refuses) and retain the card
Customer walks away mid-session	Next person uses the open session	Session timeout that ejects/retains the card (extension)
Transaction on an account not linked to the card	Unauthorised access	`AuthenticatedState` checks `sessionAccounts`
Cash runs out	Customer's card trapped, or the next customer gets refused mid-session	The session finishes normally (card ejected), then `finishSession` moves the ATM to `OutOfService`; an operator refill returns it to Idle
Operator turns the key mid-session	The machine shuts down while serving	`setOutOfService` is refused unless the ATM is idle (requirement: never shut down while serving)

Extensions & follow-up questions

1. **"What if the machine cannot make the amount?"** — The dispenser's plan fails and the withdrawal is refused before the bank is called.
2. **"What if the power fails halfway through a withdrawal?"** — Use a two-phase approach: authorise (hold) at the bank, dispense, then confirm; on restart the ATM reads its journal and either confirms or reverses. The bank reconciles against the journal.
3. **"Add a daily withdrawal limit for all accounts."** — Move the cap into `Account` or into a policy object the bank consults; the ATM does not change.
4. **"Receipts and PIN change?"** — Two more `Transaction` records (`PinChange`, and a `Printer` part that formats the journal entry).
5. **"Other banks' cards?"** — `BankService` gets a routing implementation that forwards to a card network (such as Visa or Mastercard, which passes requests between banks) for foreign cards (a Proxy — a stand-in object — or an Adapter — a translator between two interfaces — in front of the network).
6. **"Cardless / mobile withdrawals?"** — A new state path: `enterOneTimeCode` instead of `insertCard` → `enterPin`. The transactions do not change.
7. **"Bank manager reports?"** — Aggregate the journal (total deposits, total withdrawals, cash left = `dispenser.total()`).

Real-world example

Real ATMs (NCR, Diebold Nixdorf and others) control their devices through the **CEN/XFS** standard, which gives the card reader, PIN pad, dispenser and printer each a standard software interface — the same "parts behind interfaces" idea as our design. Messages between ATM networks and banks commonly use the **ISO 8583** message format, and PIN blocks are defined by **ISO 9564** and verified inside HSMs. Cash sits in **cassettes** of one denomination each, and the dispenser works out a note mix from what the cassettes hold, like our handler chain. After a dispense error, ATMs routinely reverse the transaction, and banks reconcile disputed withdrawals from the machine's electronic journal.

In the interview

Plan for 45 minutes: 5 min parts, actors and transactions → 5 min total vs available balance → 10 min state diagram (Idle → CardInserted → Authenticated) → 15 min code: `Withdrawal.execute` (plan → debit → dispense → compensate) and the dispenser chain → 10 min concurrency (account lock, ordered transfer locks) and follow-ups.

> **30-second answer:** "The ATM has parts behind interfaces, a session modelled with the State pattern, and five transactions as Command objects with one `execute` each. It never touches balances: it calls a `BankService`. Accounts have total and available balances so checks can be held. Withdrawal plans the notes first through a Chain of Responsibility over the cassettes, debits the account atomically at the bank, dispenses, and reverses the debit if the hardware fails. The bank locks per account, and transfers lock in id order to avoid deadlock."

Follow-ups with answers

- *"Where is the PIN checked?"* — At the bank (in reality in an HSM, from an encrypted PIN block). The ATM never stores or compares it.
- *"How do you stop two ATMs overdrawing a joint account?"* — The check and the debit happen together under the account's lock at the bank (or in one database transaction).
- *"Why Chain of Responsibility for notes?"* — Each denomination handler decides how much it can cover and passes the rest on; adding or removing a cassette changes the chain, not the algorithm.

Self-check

1. **Q:** Why two balances? **A:** A deposited check increases the total immediately but must not be spendable until it clears; available tracks what can be withdrawn.
2. **Q:** Which order of debit and dispense is safe? **A:** Neither alone is safe against a crash; use a hold + confirm (or debit + compensating reverse) and reconcile from the journal.
3. **Q:** How does the design avoid deadlock in transfers? **A:** Both accounts are always locked in the same global order (by id).
4. **Q:** What does the dispenser's dry run protect against? **A:** Removing notes or debiting the account for an amount that cannot be made exactly.
5. **Q:** Why is `SavingsAccount` a subclass rather than a type field? **A:** It has different behaviour (a daily cap), which inheritance models by overriding one hook.
6. **Q:** What happens after the third wrong PIN? **A:** The bank blocks the card (so every ATM refuses it) and this ATM retains it.

TL;DR

- **State** for the session, **Command** for the five transactions, **Chain of Responsibility** for the note cassettes, `BankService` interface for the bank.
- **Total vs available** balance makes check deposits correct.
- Withdrawal = **plan notes → atomic debit at the bank → dispense → reverse on failure**, all journaled.
- Concurrency: **per-account lock**, transfers lock in **id order**, PIN attempts limited at the bank.

14.5 Design an Elevator System

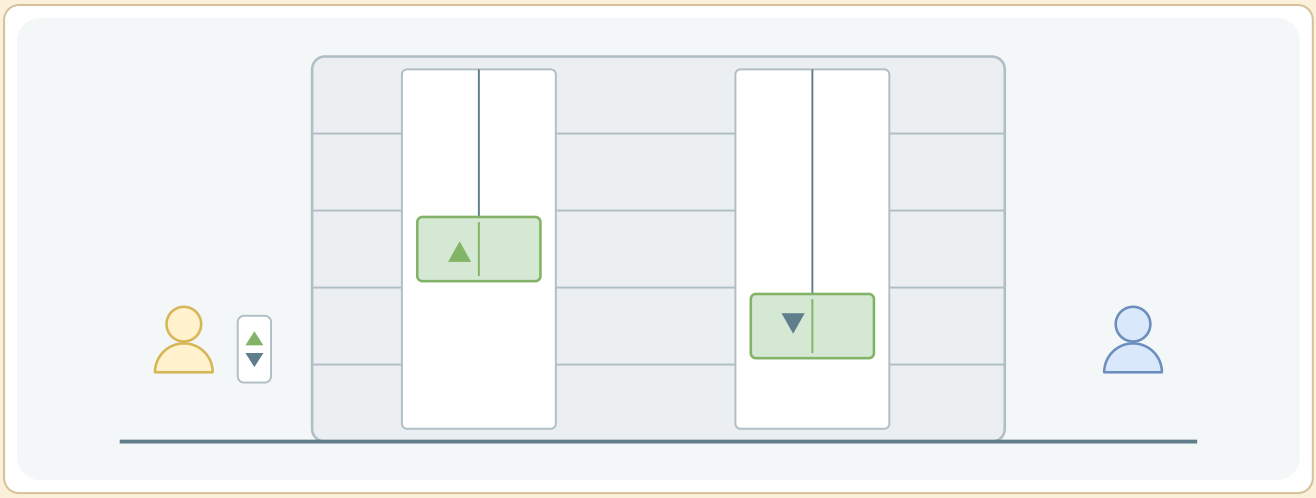


Figure — An elevator bank in a multi-floor building: several cars, hall buttons on each floor and displays (course diagram)

An **elevator** (lift) moves people up and down a building. A real building rarely has just one, and its cars are run as a **group** by a shared controller. That group control is what turns this from a small coding task into a design question. Two decisions carry the interview: **which car should answer a call from a floor** (dispatching) and **in what order a single car should visit its stops** (scheduling).

Problem & clarifying questions

Question	Assumed answer
How many floors and cars?	Many floors (0 … N-1), several cars serving all floors.
Which buttons?	Hall panel on each floor with UP and DOWN (no UP on the top floor, no DOWN on the ground floor); a car panel with one button per floor plus door-open and door-close.
Displays?	Inside each car: floor, direction, load. On each floor: one display per car showing where it is.
Scheduling rule?	A car serves every stop ahead in its direction before turning (the elevator algorithm; we use **LOOK**).
Dispatching rule?	Nearest suitable car; swappable.
Capacity?	Each car has a maximum load; an overloaded car does not move and says so.
Maintenance and emergencies?	A car can be taken out of service; fire alarm recalls all cars to the lobby.
Real time or simulated?	Simulated: one `step()` = one action. This makes the design testable without a clock.

Requirements

Functional (the course's twelve, plus emergency)

1. The building has **multiple floors** and **more than one elevator** serving all of them.
2. Each floor has a **hall panel** with UP and DOWN buttons (top floor: no UP; ground floor: no DOWN).
3. Each car has an **inside panel** with a button per floor, plus **door open** and **door close**.
4. Each car has a **display** showing its floor, direction and the weight it carries.
5. Each floor has a **display for every elevator** so a waiting rider can see where each car is.
6. When a rider presses a hall button, the system **picks one elevator** and sends it.
7. When a rider presses a floor button inside a car, the car **adds that floor** to its stops.
8. A car **serves the stops ahead of it** in its direction before turning around.

9. A car **does not move while its doors are open**.
10. Each car has a **maximum load**; an overloaded car does not move and **says so** on its display.
11. A car with no stops left **waits on the floor where it finished**.
12. An elevator can be **taken out of service** for maintenance; the system must not send riders to it.
13. *(Extension the course leaves out; we add it)* **Fire recall**: all cars go to the lobby, open their doors and ignore car calls.

Non-functional

- **Efficiency:** low average wait and ride time; no car runs to the end of the shaft for nothing.
- **Fairness / no starvation** (starvation = a request that waits forever because others keep being chosen first)**:** every accepted call is eventually served.
- **Safety:** doors, load limit and emergency modes always override scheduling.
- **Thread safety:** riders press buttons concurrently while a controller thread moves the cars.
- **Testability:** deterministic stepping (the same inputs always give the same results).

Use cases

Actors: Rider (calls a car, rides, chooses a destination), **Maintenance engineer** (takes a car out of service and back), **System** (chooses the car, moves cars, opens/closes doors, updates displays).

Top use cases: call an elevator · choose a destination · dispatch a car · move one floor · open and close the doors · report an overload · take a car out of service.

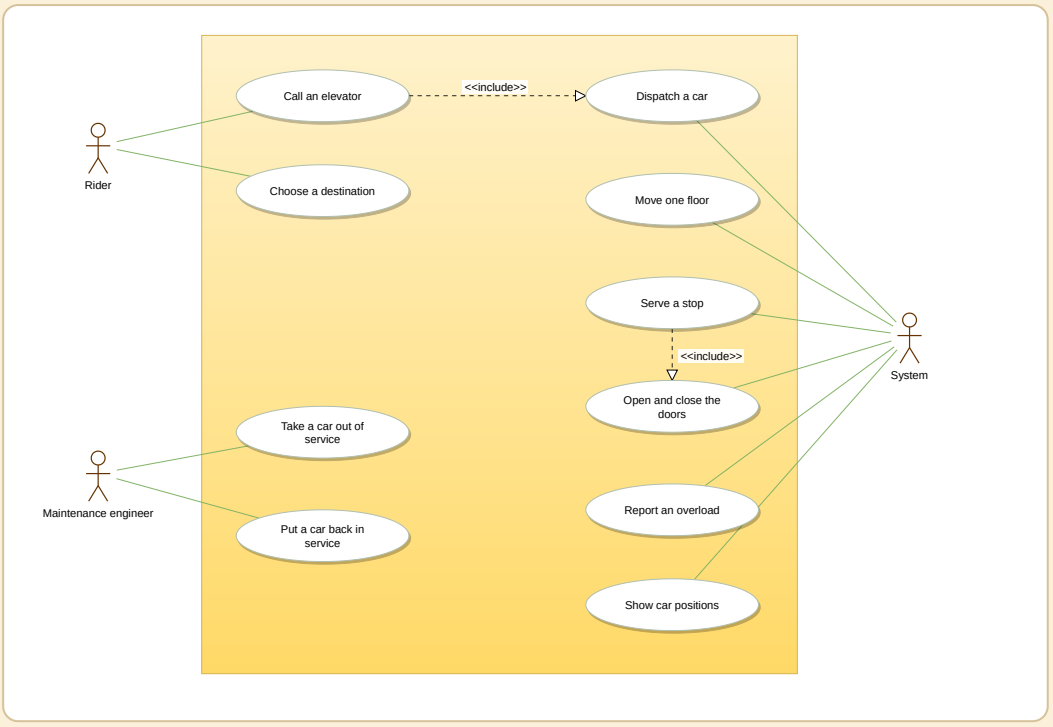


Figure — Use case diagram of the elevator system: Rider, Maintenance engineer and System (course diagram)

In the figure above, "dispatch a car" and "move one floor" belong to the System — the rider only triggers them.

Core entities

Class	Responsibility
`Direction` (enum)	UP, DOWN, IDLE.
`Stop` (record)	A floor **plus** the direction the car will travel when it leaves — two riders on floor 5 (one up, one down) are two stops.
`StopList`	The accepted stops in **two sorted sets** (`up`, `down`); `next(...)` implements LOOK.
`Door`	Open or closed.
`Elevator`	One car: floor, direction, mode, load, door, stop list. `step()` does exactly one action.
`Mode` (enum)	NORMAL, MAINTENANCE, FIRE_RECALL.
`Event` (record)	What one step did (moved, arrived, doors closed, overload, nothing).
`DispatchStrategy`	*Which car* answers a hall call. `NearestCarStrategy` is one rule.
`Dispatcher`	Holds the cars and one strategy; queues calls no car can take now.
`Building`	Owns hall-button lights, the cars and the dispatcher; ticks every car; clears hall lights on arrival; fire alarm.

The course additionally models the physical parts one-to-one: `Button` (with `HallButton`, `ElevatorButton`, `DoorButton`), `HallPanel`, `ElevatorPanel`, `Display` (with `ElevatorDisplay`, `FloorDisplay`) and `Floor`. Our code keeps the behaviour (lights, displays, door) with fewer classes; in a 45-minute interview either level of detail is fine as long as you say which you chose.

Class diagram

The course builds the design in five stages:

Stage	What it adds	What works after it
1	`Button` family and `Door`	Buttons light and reset; a door knows if it is open
2	`HallPanel`, `ElevatorPanel`, `Display` family, `Floor`	A floor accepts a call and shows each car's position
3	`Stop` and `StopList`	Given a floor and a direction, compute the next stop
4	`Elevator`	A car travels to all the stops it has accepted, one step at a time
5	`DispatchStrategy`, `Dispatcher`, `Building`	A call on floor 7 is handed to the most suitable car

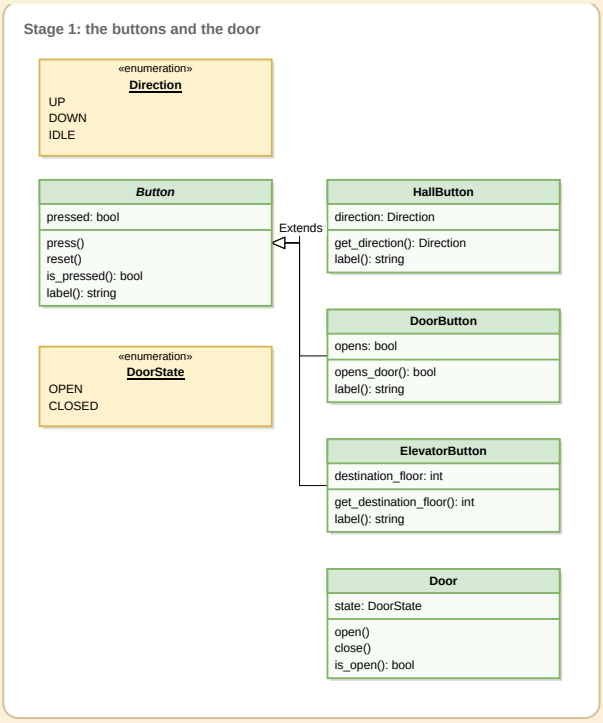


Figure — Stage 1: the Direction enum, the abstract Button with HallButton, ElevatorButton and DoorButton, and the Door (course diagram)

Stage 1. Requirements 2, 3 and 9. All buttons share one behaviour (light up on press, go dark once handled) and differ only in the value they carry: a hall button stores a **direction**, a car button a **floor number**, a door button **open or close**. That gives one abstract `Button` parent with three subclasses instead of three unrelated classes. `Door` has no owner yet; the car will own it in stage 4. Nothing can move at this point.

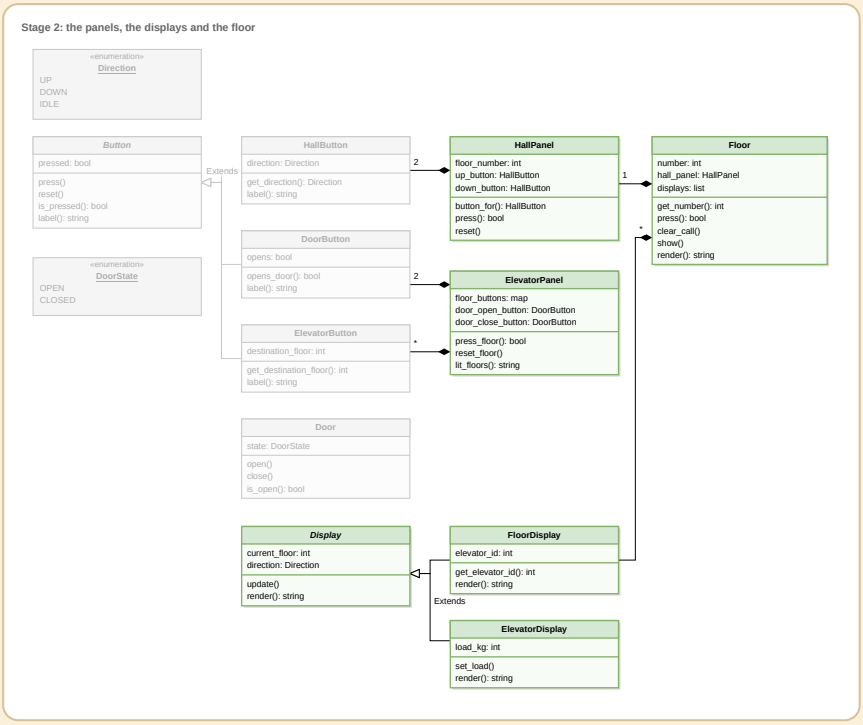


Figure — Stage 2: HallPanel, ElevatorPanel, the abstract Display with ElevatorDisplay and FloorDisplay, and Floor, all joined by composition (course diagram)

Stage 2. Requirements 2, 3, 4 and 5. Every new link is **composition**: a hall panel owns its buttons, and a floor owns its hall panel and its displays, so deleting a floor deletes them too. A hall panel has **at most two** buttons, and each is optional because the top and bottom floors lack one. The car panel creates one button per floor from the list of floors it receives, so adding a floor needs no edit here. `Display` is abstract: both subclasses show a floor and a direction; the in-car `ElevatorDisplay` adds the load, the wall-mounted `FloorDisplay` adds which car it describes. The inheritance passes the

substitution test (Chapter 4): either display can be asked to render itself wherever a `Display` is expected. After this stage a floor can register a call and show where the cars are, but nothing moves yet.

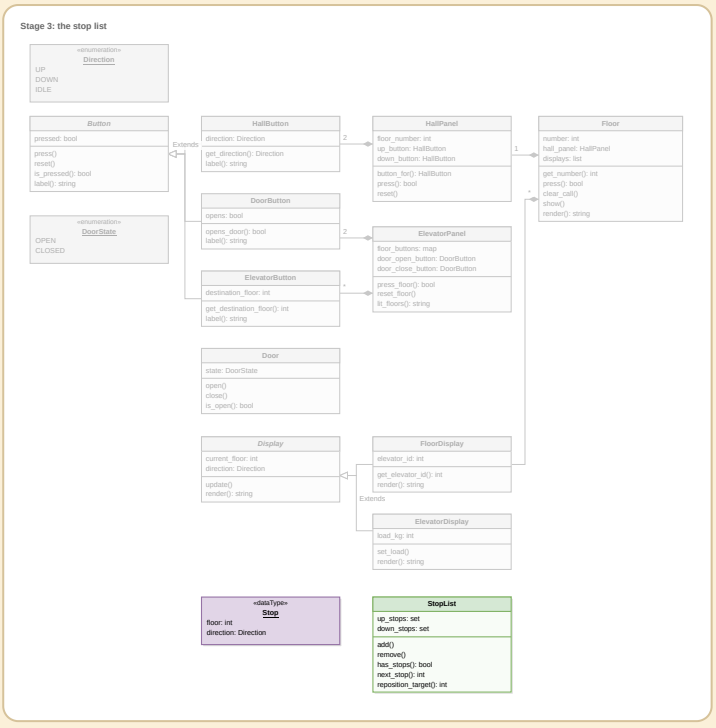


Figure — Stage 3: the Stop data type and the StopList with its up and down sets (course diagram)

Stage 3 — the most important stage. Requirement 8. The whole stage exists to answer: *knowing a car's floor and travel direction, which floor should it stop at next?* The key choice is to store stops in **two sets** (to serve going up, to serve going down) instead of one list. Picture two people waiting on floor 5, one heading up and one heading down: a plain list of floors records "5" once, so the car either stops there twice for no reason or picks up the wrong person. `Stop` is a small data type (a floor plus the direction the car will be moving when it stops there) with no relationships of its own. The next stop returned is always reachable **without reversing**; when nothing lies ahead, the car heads to the **farthest** waiting request rather than the closest one, so it sweeps everyone up on the return trip. Putting this rule in `StopList` splits responsibilities cleanly: `Elevator` **moves**, `StopList` **chooses**.

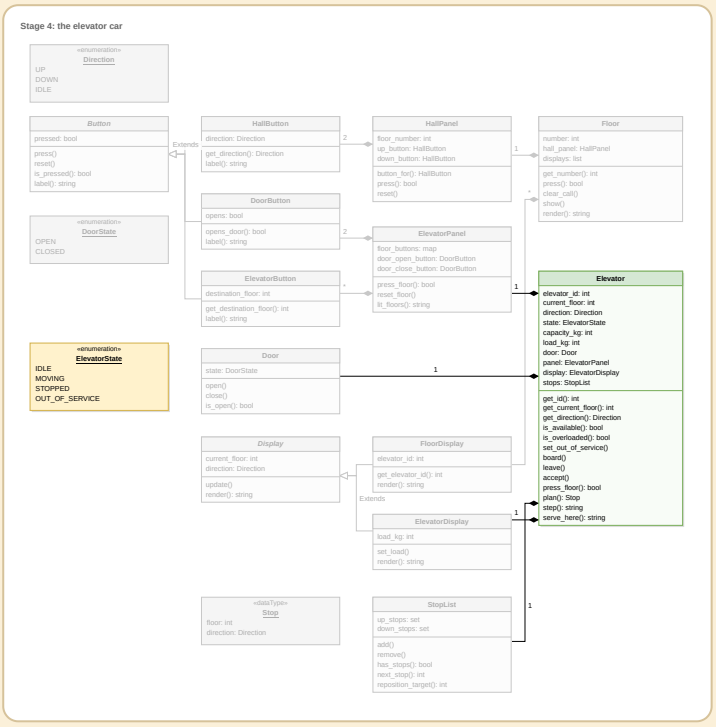


Figure — Stage 4: the Elevator owns its door, panel, display and stop list (course diagram)

Stage 4. Requirements 7, 9, 10 and 11. The car owns its door, panel, display and stop list (composition: none of them exists without the car). A single `step` does exactly **one** of four things: close the doors, refuse to move because of overload, move one floor, or stop and open the doors. Because each call does one action, a test can drive the car step by step with **no clock**. In the course, only one method reads the stop list and only `step` changes the car's position, and a car that is out of service simply does nothing on `step`, so it drops out of the building's rounds without special cases elsewhere (our `step()` returns a `NOTHING` event). Just as important is what the car does **not** know: it never touches a `Floor` and never talks to another car, which leaves the choice of car to the dispatcher.

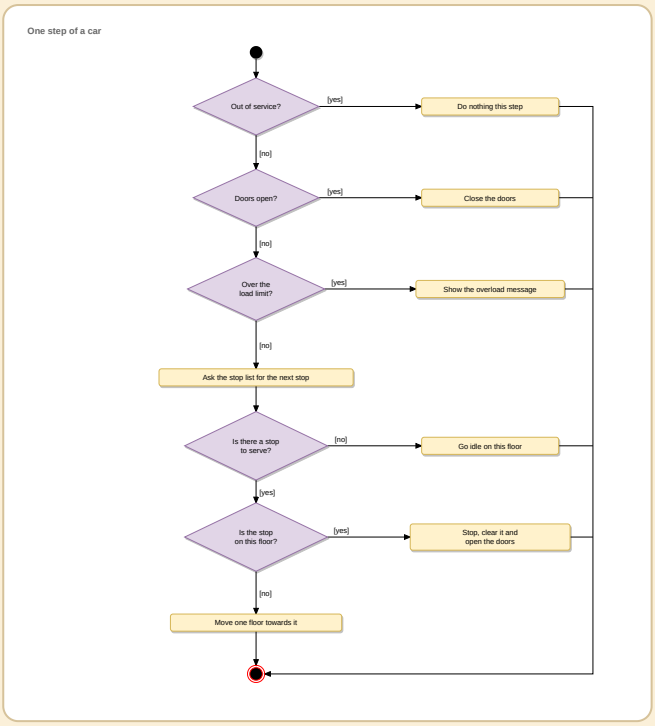


Figure — Activity diagram "One step of a car": out of service → nothing; doors open → overload check or close; otherwise ask the stop list, go idle, stop and open, or move one floor (course diagram)

The activity diagram above is our `Elevator.step()` drawn as a picture: every path ends in exactly one outcome, and only one outcome moves the car.

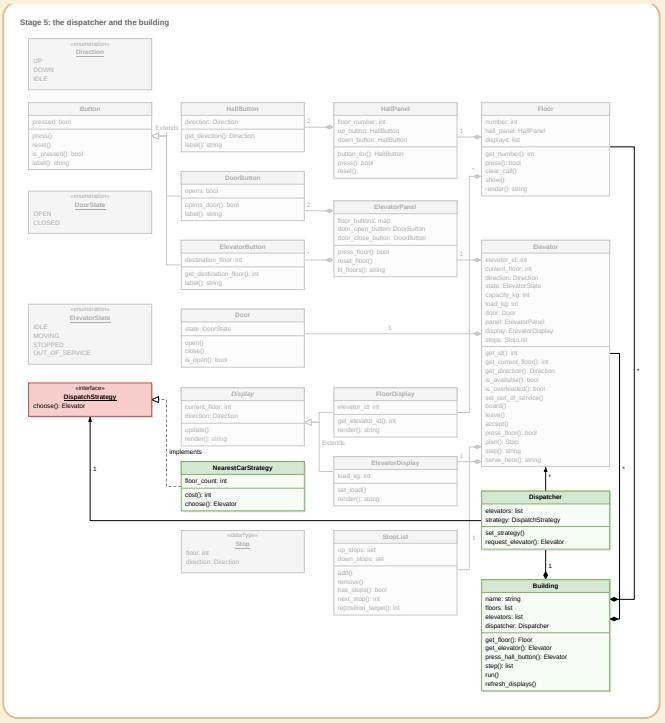


Figure — Stage 5: DispatchStrategy interface with NearestCarStrategy, the Dispatcher, and the Building that owns floors, cars and the dispatcher (course diagram)

Stage 5. Requirements 1, 6 and 12. The rule that picks a car lives **behind an interface** (`DispatchStrategy`), and the dispatcher depends only on that interface, drawn as a plain arrow to it; `NearestCarStrategy` implements it (dashed line, hollow triangle). Morning rush in an office block and a quiet night in a hotel call for different rules, and swapping the rule touches one class only. The building owns its floors, cars and dispatcher (filled diamonds). It is also the only class that knows about both cars and floors, so switching off the hall-button light when a car opens its doors is the building's job.

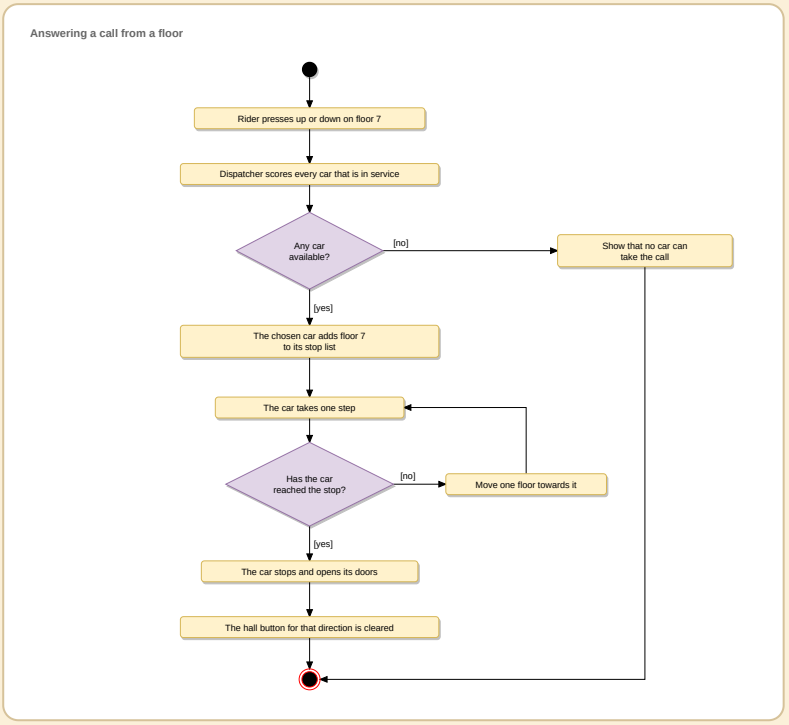


Figure — Activity diagram "Answering a call from a floor": rider presses up/down on floor 7, the dispatcher scores every in-service car, the chosen car adds floor 7, steps until it reaches the stop, opens its doors, and the hall button is cleared (course diagram)

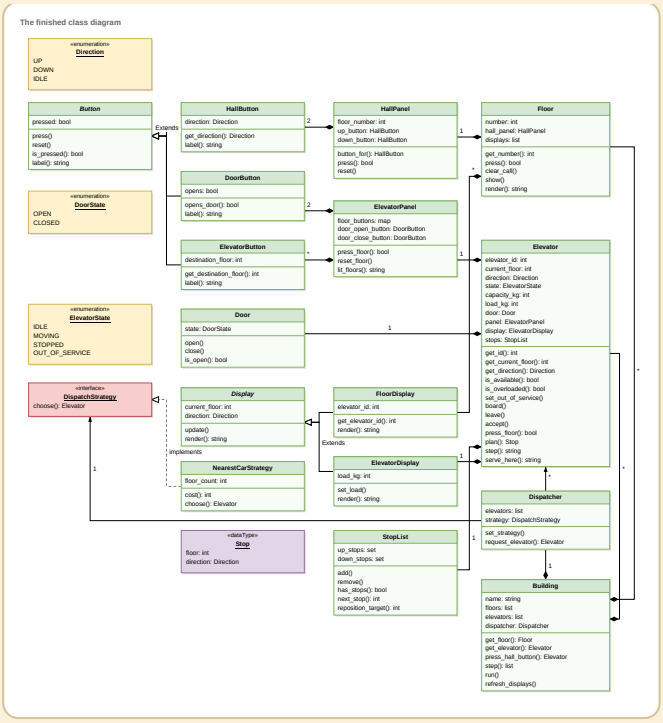
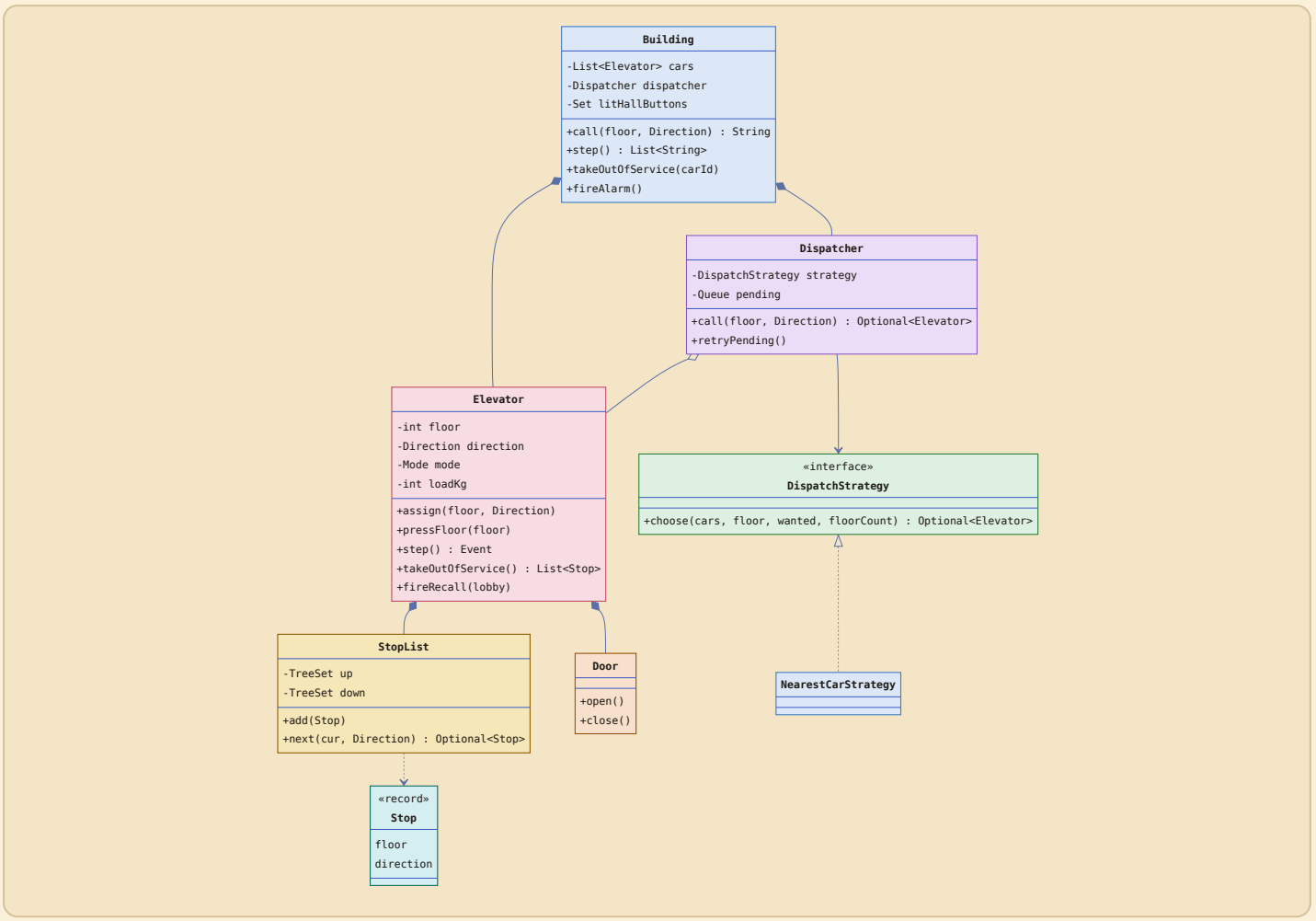


Figure — The finished class diagram of the elevator system (course diagram)

Class diagram of **our Java code**:



Key flows

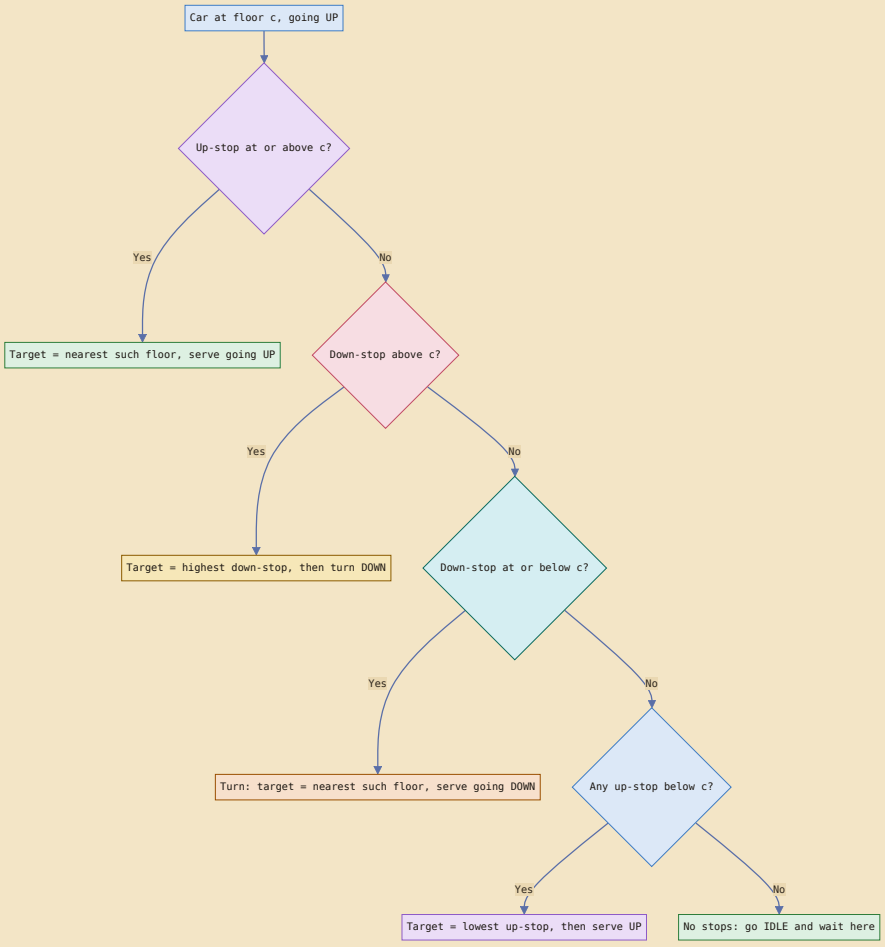
Scheduling algorithms compared. "Scheduling" means choosing the order in which one car serves its stops. These algorithms come from disk-arm scheduling in operating systems (the order in which a hard

disk's read head visits requested tracks), which has the same shape (a head moving along a line of requests).

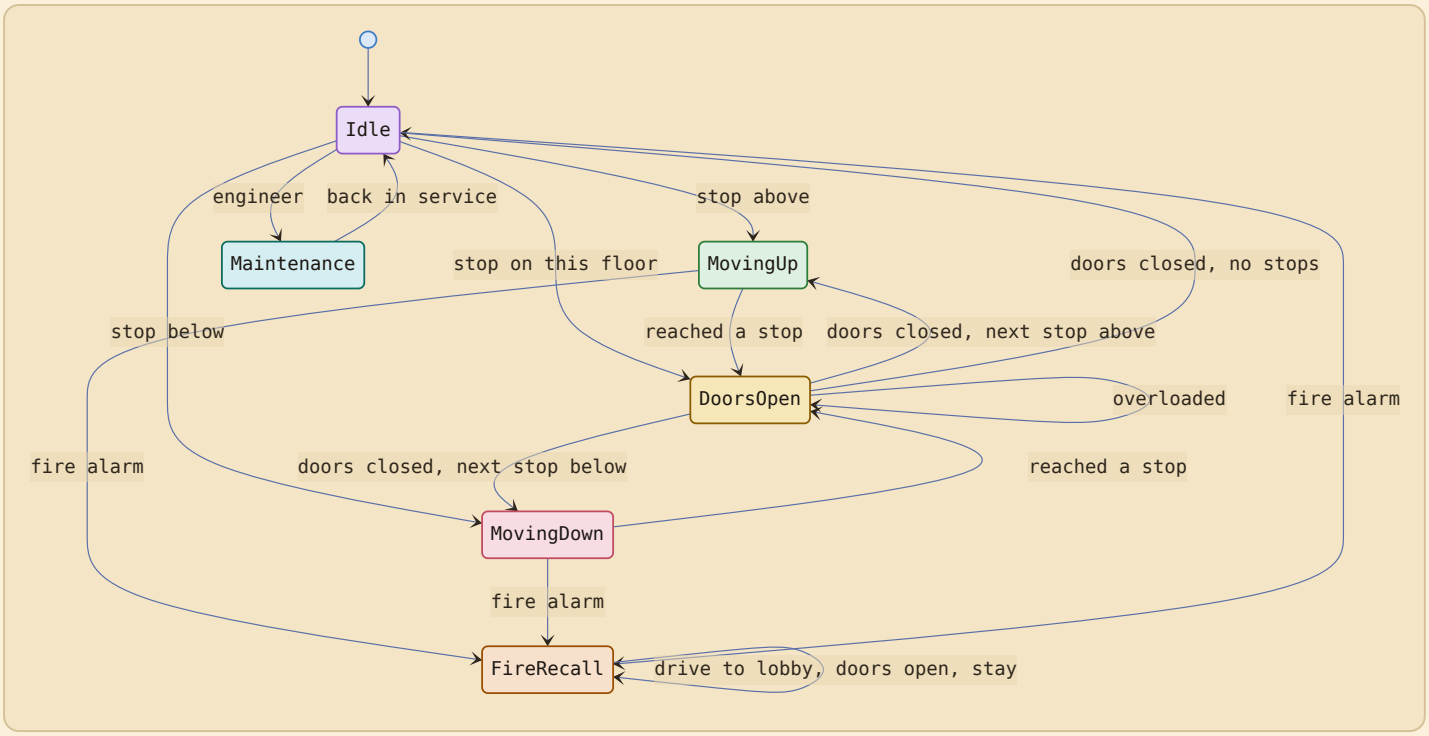
Algorithm	Rule	Good	Bad
FCFS (first come, first served)	Serve calls in arrival order	Simple, fair in order	Zig-zags up and down; very inefficient
SSTF (shortest seek time first)	Always go to the nearest stop	Short average trips	**Starvation**: a far floor may wait forever while nearby calls keep coming
SCAN ("elevator algorithm")	Sweep in one direction to the **end of the shaft**, then reverse	No starvation, predictable	Wastes travel going to the top/bottom floor even when nobody is there
LOOK	Sweep in one direction only **as far as the last request**, then reverse	No starvation, no wasted travel	Slightly more bookkeeping
C-SCAN / C-LOOK	Serve in one direction only; jump back to the start	More uniform waits	Empty return trips

Our `StopList.next` is **LOOK** with direction-aware stops:

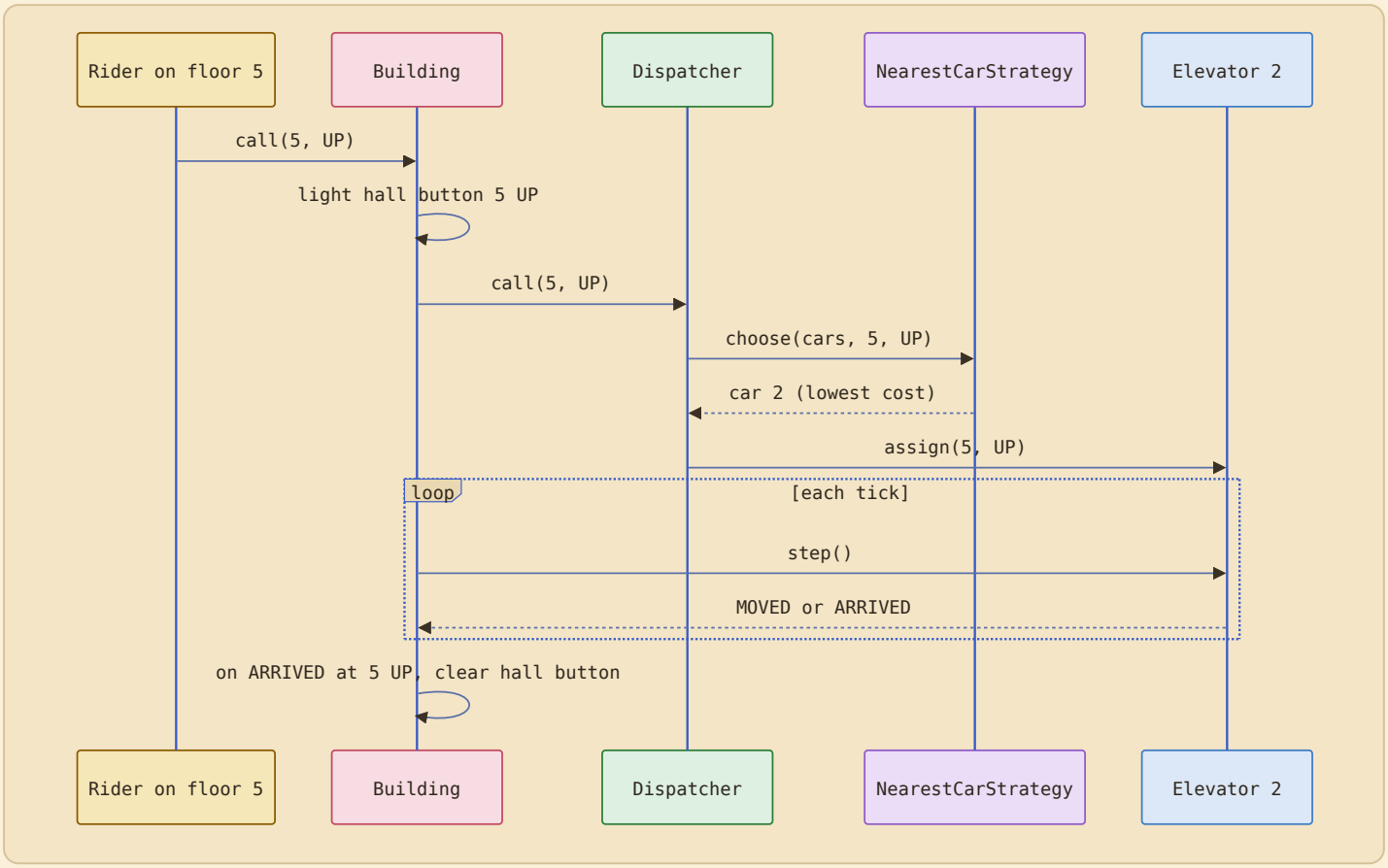
1. Going **UP**: the nearest up-stop at or above the car (`up.ceiling(cur)`).
2. None? If someone *above* wants to go **down**, go up to the **highest** such floor, then turn (collect the rest on the way down).
3. Otherwise turn: the nearest down-stop at or below the car (`down.floor(cur)`).
4. Otherwise the only work left is up-stops **below** the car: go down to the **lowest** one, then serve upward.
5. Going **DOWN** is the mirror image. An **IDLE** car picks the side of the nearest stop.



Car states.



A hall call.



Dispatch cost (`NearestCarStrategy`, from the course): an **idle** car costs its distance; a car **already coming this way in the same direction** costs its distance; any other car pays distance **+ two building heights**, so it wins only when nothing better is free. Cars out of service are skipped.

Design decisions & patterns used

Decision	Pattern / principle	Why	Alternative rejected
Choosing a car is a `DispatchStrategy`	**Strategy**	Different buildings and times of day want different rules; the course admits one implementation normally doesn't justify an interface, but here the *requirement* (swap the rule) does	`if (officeMode) … else …` in the dispatcher
Sweep rule in `StopList`, not in `Elevator`	Single Responsibility	The car drives; the stop list decides. Without this split, `Elevator` would do two jobs	Next-stop logic inside `step()`
Two sets of stops (up/down)	Data modelling	Distinguishes "5 going up" from "5 going down"	One list of floors
`step()` = one action	Deterministic simulation	Every scenario is a list of calls and an expected list of events; no clock, no flakiness (tests that pass or fail at random)	A thread per car with `sleep`
Modes as an enum inside the car	Lightweight State	Only three modes and a few checks; full State classes would add more code than they save here. If modes grew (fire phase II, independent service, VIP), switch to the State pattern	State classes from day one
Hall calls re-dispatched when a car leaves service	Robustness	A call accepted by a car must not be lost when that car is taken out	Silently clearing the car's stops
Queue calls when no car is available	—	"No car can take the call right now" is not the same as "rejected"	Dropping the call

The course lists three honest weak points, all worth saying: (1) one step per call means **no threads**; a real controller runs each car on its own thread and needs a lock around the stop list — our methods are `synchronized` for exactly that; (2) the dispatcher **decides once** and never re-evaluates, though real systems do (that is why a busy building sometimes changes the car number while you wait); (3) `Elevator` is close to doing two jobs, which is why the sweep rule lives in `StopList`. The course also names only **one** pattern here (Strategy); `Button` and `Display` are ordinary abstract base classes and `StopList` is Single Responsibility, not a pattern. Its scope list deliberately leaves out emergency calls, fire service mode, express cars that skip floors, and door sensors that reopen for a hand (we add fire recall; the others are extensions below).

Java implementation

Package `lld.ch14.elevator`.

```java
// File: lld/ch14/elevator/StopList.java
package lld.ch14.elevator;

import java.util.Optional;
import java.util.TreeSet;

enum Direction { UP, DOWN, IDLE }

/** A floor plus the direction the car will travel when it leaves that floor. */
record Stop(int floor, Direction direction) {}

/**
 * The stops a car has accepted, kept in TWO sorted sets: floors to serve while going up,
 * and floors to serve while going down. Two riders on floor 5 (one up, one down) are two stops.
 * next() implements LOOK: keep going while there is work ahead, turn at the last request,
 * never travel to the end of the shaft for nothing (that would be SCAN).
 */
final class StopList {
    private final TreeSet<Integer> up = new TreeSet<>();
    private final TreeSet<Integer> down = new TreeSet<>();

    void add(Stop s) { (s.direction() == Direction.DOWN ? down : up).add(s.floor()); }
    void remove(Stop s) { (s.direction() == Direction.DOWN ? down : up).remove(s.floor()); }
    boolean isEmpty() { return up.isEmpty() && down.isEmpty(); }
    void clear() { up.clear(); down.clear(); }

    /** Removes and returns every stop (used when a car leaves service). */
    java.util.List<Stop> drain() {
        java.util.List<Stop> all = new java.util.ArrayList<>();
        up.forEach(f -> all.add(new Stop(f, Direction.UP)));
        down.forEach(f -> all.add(new Stop(f, Direction.DOWN)));
        clear();
        return all;
    }

    Optional<Stop> next(int cur, Direction dir) {
        if (isEmpty()) return Optional.empty();
        if (dir == Direction.IDLE) dir = nearestSide(cur);
        return Optional.of(dir == Direction.UP ? goingUp(cur) : goingDown(cur));
    }

    private Stop goingUp(int cur) {
        Integer f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);                          // work ahead
        if (!down.isEmpty() && down.last() > cur) return new Stop(down.last(), Direction.DOWN); // go up, then
turn
        f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);                        // turn around now
        return new Stop(up.first(), Direction.UP);                                // go down to lowest up-call
    }

    private Stop goingDown(int cur) {
        Integer f = down.floor(cur);
        if (f != null) return new Stop(f, Direction.DOWN);
        if (!up.isEmpty() && up.first() < cur) return new Stop(up.first(), Direction.UP);
        f = up.ceiling(cur);
        if (f != null) return new Stop(f, Direction.UP);
        return new Stop(down.last(), Direction.DOWN);
    }

    private Direction nearestSide(int cur) {
        int best = Integer.MAX_VALUE;
        Direction side = Direction.UP;
        for (int f : up) if (Math.abs(f - cur) < best) { best = Math.abs(f - cur); side = f >= cur ? Direction.UP
: Direction.DOWN; }
        for (int f : down) if (Math.abs(f - cur) < best) { best = Math.abs(f - cur); side = f > cur ? Direction.UP
: Direction.DOWN; }
        return side;
    }
```

```
    @Override public String toString() { return "up" + up + " down" + down; }
}
```

```java
// File: lld/ch14/elevator/Elevator.java
package lld.ch14.elevator;

enum Mode { NORMAL, MAINTENANCE, FIRE_RECALL }

/** What one step of a car did. */
record Event(Kind kind, int car, int floor, Direction direction, String note) {
    enum Kind { MOVED, ARRIVED, DOORS_CLOSED, OVERLOAD, NOTHING }
}

final class Door {
    private boolean open;
    boolean isOpen() { return open; }
    void open() { open = true; }
    void close() { open = false; }
}

/**
 * One car. It drives; its StopList decides. One call to step() = one action, so the whole
 * system is testable without a clock. Methods are synchronized because riders press buttons
 * from other threads while the controller thread calls step().
 */
final class Elevator {
    private final int id;
    private final int capacityKg;
    private final Door door = new Door();
    private final StopList stops = new StopList();
    private int floor;
    private Direction direction = Direction.IDLE;
    private Mode mode = Mode.NORMAL;
    private int loadKg;

    Elevator(int id, int startFloor, int capacityKg) {
        this.id = id;
        this.floor = startFloor;
        this.capacityKg = capacityKg;
    }

    /** From the dispatcher: a hall call this car has been given. */
    synchronized void assign(int f, Direction wanted) { stops.add(new Stop(f, wanted)); }

    /** From the panel inside the car. */
    synchronized void pressFloor(int f) {
        if (mode != Mode.NORMAL) return;                    // car buttons are dead during fire recall
        if (f == floor && door.isOpen()) return;
        Direction d = f > floor ? Direction.UP : f < floor ? Direction.DOWN
                : (direction == Direction.IDLE ? Direction.UP : direction);
        stops.add(new Stop(f, d));
    }

    synchronized Event step() {
        if (mode == Mode.MAINTENANCE) return nothing();   // parked: never moves
        if (door.isOpen()) {
            if (mode == Mode.FIRE_RECALL && stops.isEmpty()) return nothing(); // parked at lobby, doors open
            if (loadKg > capacityKg) {
                return new Event(Event.Kind.OVERLOAD, id, floor, direction,
                        "OVERLOAD " + loadKg + "kg > " + capacityKg + "kg, doors stay open");
            }
            door.close();
            return new Event(Event.Kind.DOORS_CLOSED, id, floor, direction, "doors close");
        }
        var next = stops.next(floor, direction);
        if (next.isEmpty()) {
            direction = Direction.IDLE;
            return nothing();
        }
        Stop target = next.get();
        if (target.floor() == floor) {                        // arrived
            stops.remove(target);
            direction = stops.isEmpty() ? Direction.IDLE : target.direction();
```

```
            door.open();
            return new Event(Event.Kind.ARRIVED, id, floor, target.direction(), "doors open");
        }
        direction = target.floor() > floor ? Direction.UP : Direction.DOWN;   // never moves with doors open
        floor += direction == Direction.UP ? 1 : -1;
        return new Event(Event.Kind.MOVED, id, floor, direction, "");
    }

    synchronized void board(int kg) { loadKg += kg; }
    synchronized void alight(int kg) { loadKg = Math.max(0, loadKg - kg); }

    /** Out of service: returns the stops it had accepted, so hall calls can be re-dispatched. */
    synchronized java.util.List<Stop> takeOutOfService() {
        mode = Mode.MAINTENANCE;
        direction = Direction.IDLE;
        door.open();                        // parked with doors open so riders can leave
        return stops.drain();
    }

    synchronized void returnToService() {
        mode = Mode.NORMAL;
    }

    /** Fire service: drop every stop, go straight to the lobby, open the doors and stay there. */
    synchronized void fireRecall(int lobby) {
        if (mode == Mode.MAINTENANCE) return;
        mode = Mode.FIRE_RECALL;
        stops.clear();
        stops.add(new Stop(lobby, Direction.DOWN));
        direction = Direction.IDLE;
    }

    synchronized boolean available() { return mode == Mode.NORMAL; }
    synchronized boolean idle() { return stops.isEmpty() && (!door.isOpen() || mode != Mode.NORMAL); }
    synchronized int floor() { return floor; }
    synchronized Direction direction() { return direction; }
    int id() { return id; }

    synchronized String display() {
        return "car%d@%d %s%s %dkg %s".formatted(id, floor, direction, door.isOpen() ? " [open]" : "",
                loadKg, mode == Mode.NORMAL ? "" : mode);
    }

    private Event nothing() { return new Event(Event.Kind.NOTHING, id, floor, direction, ""); }
}
```

```java
// File: lld/ch14/elevator/Dispatch.java
package lld.ch14.elevator;

import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.Queue;

/** Strategy: which car answers a hall call. Office at 9am and hotel at midnight want different rules. */
interface DispatchStrategy {
    Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount);
}

/**
 * Idle car: cost = distance. A car already heading toward the caller in the same direction:
 * cost = distance. Anything else pays a penalty of two building heights, so it wins only
 * when nothing better exists.
 */
final class NearestCarStrategy implements DispatchStrategy {
    @Override public Optional<Elevator> choose(List<Elevator> cars, int floor, Direction wanted, int floorCount) {
        return cars.stream().filter(Elevator::available)
                .min(Comparator.comparingInt((Elevator c) -> cost(c, floor, wanted, floorCount))
                        .thenComparingInt(Elevator::id));
    }

    private int cost(Elevator c, int floor, Direction wanted, int n) {
        int distance = Math.abs(c.floor() - floor);
        Direction d = c.direction();
        if (d == Direction.IDLE) return distance;
        boolean onTheWay = (d == Direction.UP && c.floor() <= floor) || (d == Direction.DOWN && c.floor() >=
floor);
        if (onTheWay && d == wanted) return distance;
        return distance + 2 * n;
    }
}

/** Holds the cars and one strategy. Calls no car can take right now wait in a queue. */
final class Dispatcher {
    private final List<Elevator> cars;
    private final int floorCount;
    private volatile DispatchStrategy strategy;
    private final Queue<Stop> pending = new ArrayDeque<>();

    Dispatcher(List<Elevator> cars, int floorCount, DispatchStrategy strategy) {
        this.cars = cars;
        this.floorCount = floorCount;
        this.strategy = strategy;
    }

    synchronized Optional<Elevator> call(int floor, Direction wanted) {
        Optional<Elevator> car = strategy.choose(cars, floor, wanted, floorCount);
        car.ifPresentOrElse(c -> c.assign(floor, wanted), () -> pending.add(new Stop(floor, wanted)));
        return car;
    }

    /** Called every tick: retry the calls nobody could take earlier. */
    synchronized void retryPending() {
        int n = pending.size();
        for (int i = 0; i < n; i++) {
            Stop s = pending.poll();
            Optional<Elevator> car = strategy.choose(cars, s.floor(), s.direction(), floorCount);
            if (car.isPresent()) car.get().assign(s.floor(), s.direction());
            else pending.add(s);
        }
    }

    synchronized int pendingCount() { return pending.size(); }
    void setStrategy(DispatchStrategy s) { this.strategy = s; }
}
```

```java
// File: lld/ch14/elevator/Building.java
package lld.ch14.elevator;

import java.util.ArrayList;
import java.util.List;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;

/** Owns floors' hall buttons, the cars and the dispatcher. The only class that knows cars AND floors. */
final class Building {
    private final int floors;
    private final List<Elevator> cars = new ArrayList<>();
    private final Dispatcher dispatcher;
    private final Set<Stop> litHallButtons = ConcurrentHashMap.newKeySet();
    private int tick;

    Building(int floors, int[] carStartFloors, int capacityKg) {
        this.floors = floors;
        for (int i = 0; i < carStartFloors.length; i++) cars.add(new Elevator(i + 1, carStartFloors[i],
capacityKg));
        this.dispatcher = new Dispatcher(cars, floors, new NearestCarStrategy());
    }

    /** A rider presses UP or DOWN on a floor. The top floor has no UP, the ground floor no DOWN. */
    String call(int floor, Direction d) {
        if ((floor == floors - 1 && d == Direction.UP) || (floor == 0 && d == Direction.DOWN)) {
            throw new IllegalArgumentException("no " + d + " button on floor " + floor);
        }
        if (!litHallButtons.add(new Stop(floor, d))) return "hall %d %s already lit".formatted(floor, d);
        return dispatcher.call(floor, d).map(c -> "hall %d %s -> car%d".formatted(floor, d, c.id()))
                .orElse("hall %d %s -> queued (no car available)".formatted(floor, d));
    }

    /** One tick: retry queued calls, then every car takes one step. Returns what happened. */
    List<String> step() {
        tick++;
        dispatcher.retryPending();
        List<String> out = new ArrayList<>();
        for (Elevator c : cars) {
            Event e = c.step();
            if (e.kind() == Event.Kind.ARRIVED) litHallButtons.remove(new Stop(e.floor(), e.direction()));
            if (e.kind() == Event.Kind.MOVED) out.add("car%d->%d".formatted(e.car(), e.floor()));
            else if (e.kind() != Event.Kind.NOTHING) out.add("car%d@%d %s".formatted(e.car(), e.floor(),
e.note()));
        }
        return out;
    }

    /** Engineer takes a car out: its accepted hall calls go back to the dispatcher. */
    List<String> takeOutOfService(int carId) {
        List<String> out = new ArrayList<>();
        for (Stop s : car(carId).takeOutOfService()) {
            if (litHallButtons.contains(s)) {
                out.add("re-dispatch " + dispatcher.call(s.floor(), s.direction())
                        .map(c -> "hall %d %s -> car%d".formatted(s.floor(), s.direction(), c.id()))
                        .orElse("hall %d %s -> queued".formatted(s.floor(), s.direction())));
            }
        }
        return out;
    }

    void returnToService(int carId) { car(carId).returnToService(); }

    void fireAlarm() {
        litHallButtons.clear();
        cars.forEach(c -> c.fireRecall(0));
    }

    boolean allIdle() { return cars.stream().allMatch(Elevator::idle) && dispatcher.pendingCount() == 0; }
    Elevator car(int id) { return cars.get(id - 1); }
```

```
    int tick() { return tick; }

    /** What the display on every floor shows. */
    String floorDisplay() {
        StringBuilder sb = new StringBuilder();
        cars.forEach(c -> sb.append(c.display().stripTrailing()).append(" | "));
        return sb.substring(0, sb.length() - 3);
    }
}
```

```java
// File: lld/ch14/elevator/Main.java
package lld.ch14.elevator;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println("== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==");
        StopList sl = new StopList();
        sl.add(new Stop(6, Direction.UP));
        sl.add(new Stop(9, Direction.UP));
        sl.add(new Stop(2, Direction.DOWN));
        sl.add(new Stop(1, Direction.UP));
        int cur = 4;
        Direction dir = Direction.UP;
        List<Integer> order = new ArrayList<>();
        while (!sl.isEmpty()) {
            Stop s = sl.next(cur, dir).orElseThrow();
            sl.remove(s);
            order.add(s.floor());
            cur = s.floor();
            dir = s.direction();
        }
        System.out.println("service order " + order + "  (turns at 9 and at 1: never runs to 0 or to a floor above
9)");

        System.out.println("\n== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==");
        Building b = new Building(10, new int[]{0, 8}, 680);
        System.out.println(b.call(5, Direction.UP));      // car2 idle, 3 away, beats car1 (5 away)
        System.out.println(b.call(1, Direction.UP));      // car1 idle, 1 away
        steps(b, 2);
        System.out.println("rider enters car1 at floor 1, presses 7");
        b.car(1).board(80);
        b.car(1).pressFloor(7);
        steps(b, 2);
        System.out.println("rider enters car2 at floor 5, presses 9");
        b.car(2).board(75);
        b.car(2).pressFloor(9);
        System.out.println(b.call(3, Direction.UP));      // car1 is below 3 and going up: on its way
        runUntilIdle(b);
        System.out.println("display: " + b.floorDisplay());

        System.out.println("\n== 3. Overload: at floor 9, 10 more people squeeze into car2 ==");
        System.out.println(b.call(9, Direction.DOWN));
        steps(b, 1);                                      // doors open at 9
        b.car(2).board(700);
        b.car(2).pressFloor(0);
        steps(b, 2);
        System.out.println("two people step out");
        b.car(2).alight(160);
        steps(b, 3);
        runUntilIdle(b);
        b.car(2).alight(615);                             // everyone leaves at the lobby

        System.out.println("\n== 4. Maintenance: calls move to another car ==");
        System.out.println(b.call(4, Direction.DOWN));
        b.takeOutOfService(1).forEach(System.out::println);       // car1 had the call: it moves to car2
        b.takeOutOfService(2).forEach(System.out::println);  // now no car at all: call is queued
        System.out.println(b.call(2, Direction.UP));
        System.out.println("engineer returns car1 to service; queued calls are dispatched on the next tick");
        b.returnToService(1);
        runUntilIdle(b);
        b.returnToService(2);

        System.out.println("\n== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==");
        b.car(2).pressFloor(9);
```

```
        steps(b, 3);
        b.fireAlarm();
        b.car(1).pressFloor(8);
        runUntilIdle(b);
        System.out.println("display: " + b.floorDisplay());

        System.out.println("\n== 6. 8 riders press hall buttons at the same instant (new building) ==");
        Building busy = new Building(10, new int[]{0, 9, 4}, 680);
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            int[][] calls = {{1, 0}, {2, 0}, {3, 1}, {5, 0}, {6, 1}, {7, 1}, {8, 0}, {9, 1}};
            for (int[] c : calls) {
                pool.submit(() -> { go.await(); busy.call(c[0], c[1] == 0 ? Direction.UP : Direction.DOWN); return
null; });
            }
            go.countDown();
        }
        while (!busy.allIdle() && busy.tick() < 200) busy.step();
        System.out.println("all 8 calls served: " + busy.allIdle() + " (ticks=" + busy.tick() + " < 200)");
    }

    /** Exactly n ticks, printing what happened in each. */
    static void steps(Building b, int n) {
        for (int i = 0; i < n; i++) print(b, b.step());
    }

    static void runUntilIdle(Building b) {
        for (int guard = 0; guard < 100 && !b.allIdle(); guard++) print(b, b.step());
    }

    private static void print(Building b, List<String> events) {
        System.out.println("  t" + b.tick() + ": " + (events.isEmpty() ? "-" : String.join(", ", events)));
    }
}
```

How the code works, part by part

- `StopList` keeps two `TreeSet<Integer>`s (a `TreeSet` is a sorted set). `ceiling`, `floor`, `first` and `last` give O(log n) answers to "nearest stop at or above/below" — the core of LOOK.
- `Elevator.step()` checks, in order: maintenance → doors open (overload? fire parked? else close) → ask the stop list → arrive (remove the stop, open the doors) or move one floor. The car never moves with the doors open.
- `pressFloor` classifies a car call as an up- or down-stop relative to the current floor; during fire recall car buttons are ignored.
- `takeOutOfService` returns the car's accepted stops; `Building` re-dispatches the ones that are still lit hall calls.
- `Dispatcher.retryPending` runs every tick, so a queued call is dispatched as soon as any car becomes available.
- `Main` part 1 runs LOOK on one car in isolation; parts 2–5 run a two-car building; part 6 has eight threads press hall buttons at the same instant.

Output:

```
== 1. LOOK on one car: at floor 4 going up; stops 6(up) 9(car) 2(down) 1(up) ==
service order [6, 9, 2, 1]  (turns at 9 and at 1: never runs to 0 or to a floor above 9)

== 2. Building: floors 0-9, car1 at 0, car2 at 8 ==
hall 5 UP -> car2
hall 1 UP -> car1
  t1: car1->1, car2->7
  t2: car1@1 doors open, car2->6
rider enters car1 at floor 1, presses 7
  t3: car1@1 doors close, car2->5
  t4: car1->2, car2@5 doors open
rider enters car2 at floor 5, presses 9
hall 3 UP -> car1
  t5: car1->3, car2@5 doors close
  t6: car1@3 doors open, car2->6
  t7: car1@3 doors close, car2->7
  t8: car1->4, car2->8
  t9: car1->5, car2->9
  t10: car1->6, car2@9 doors open
  t11: car1->7, car2@9 doors close
  t12: car1@7 doors open
  t13: car1@7 doors close
display: car1@7 IDLE 80kg | car2@9 IDLE 75kg

== 3. Overload: at floor 9, 10 more people squeeze into car2 ==
hall 9 DOWN -> car2
  t14: car2@9 doors open
  t15: car2@9 OVERLOAD 775kg > 680kg, doors stay open
  t16: car2@9 OVERLOAD 775kg > 680kg, doors stay open
two people step out
  t17: car2@9 doors close
  t18: car2->8
  t19: car2->7
  t20: car2->6
  t21: car2->5
  t22: car2->4
  t23: car2->3
  t24: car2->2
  t25: car2->1
  t26: car2->0
  t27: car2@0 doors open
  t28: car2@0 doors close

== 4. Maintenance: calls move to another car ==
hall 4 DOWN -> car1
re-dispatch hall 4 DOWN -> car2
re-dispatch hall 4 DOWN -> queued
hall 2 UP -> queued (no car available)
engineer returns car1 to service; queued calls are dispatched on the next tick
  t29: car1@7 doors close
  t30: car1->6
  t31: car1->5
  t32: car1->4
  t33: car1@4 doors open
  t34: car1@4 doors close
  t35: car1->3
  t36: car1->2
  t37: car1@2 doors open
  t38: car1@2 doors close

== 5. Fire alarm: every car to the lobby, doors open, car buttons dead ==
  t39: car2@0 doors close
  t40: car2->1
  t41: car2->2
  t42: car1->1, car2->1
  t43: car1->0, car2->0
  t44: car1@0 doors open, car2@0 doors open
display: car1@0 IDLE [open] 80kg FIRE_RECALL | car2@0 IDLE [open] 0kg FIRE_RECALL
```

```
== 6. 8 riders press hall buttons at the same instant (new building) ==
all 8 calls served: true (ticks=11 < 200)
```

Read the output: in part 1 the car serves 6 and 9 going up, turns at 9 (the highest request, not the top of the shaft), serves 2 going down, then 1 — LOOK, not SCAN. In part 2 the hall call on 3 goes to car 1 because it is below floor 3 and already going up. In part 3 the overloaded car keeps its doors open until two people step out. In part 4 the call held by car 1 moves to car 2 when car 1 goes out of service, then is queued when no car is available, and served as soon as car 1 returns. In part 5 car 2 reverses mid-trip and both cars park at the lobby with doors open.

Concurrency & edge cases

Situation	Handling
Riders press buttons while the controller steps cars	`Elevator` methods are `synchronized`; the dispatcher is `synchronized`; hall lights are a concurrent set
Same hall button pressed twice	`litHallButtons.add` returns false: "already lit", no second dispatch
Up and down requests on the same floor	Two different `Stop`s in two sets
Car call for the floor the car is on, doors open	Ignored (doors already open)
Overload	Doors stay open, display shows OVERLOAD, car does not move
Car taken out of service with accepted hall calls	Calls re-dispatched or queued; never lost
No car available	Call queued; retried every tick
Fire alarm	All stops dropped, lobby added, car buttons ignored, doors stay open at the lobby
Starvation	LOOK serves every accepted stop within one round trip (one sweep each way)
Invalid button (UP on the top floor)	`IllegalArgumentException` — that button does not exist

Extensions & follow-up questions

1. **"How do you choose which car?"** — `DispatchStrategy`; show `NearestCarStrategy` and mention alternatives: zoning (car 1 serves floors 0–10), **up-peak** mode (send idle cars to the lobby at 9 a.m.), and energy-saving rules.
2. **"What if two riders press at the same instant?"** — The dispatcher is synchronized, so calls are dispatched one at a time; each car's stop list is guarded by the car's lock. (The course's design leaves this lock out and says so.)
3. **"How would you test this?"** — Because one call = one step, every test is: a list of button presses, N steps, and an expected list of events. No sleeps, no flakiness.
4. **"Destination dispatch?"** — Riders enter their destination in the lobby keypad; the system groups riders with nearby destinations into the same car. Implement as another `DispatchStrategy` that receives (from, to) instead of (floor, direction).
5. **"Re-evaluate assignments?"** — Each tick, for unserved hall calls, ask the strategy again and move the call if a much better car is free now.
6. **"Real time?"** — A `ScheduledExecutorService` (a Java thread pool that runs tasks on a timer) calls `building.step()` every second (or per car on its own thread, with the car lock guarding the stop list).
7. **"Emergency stop button inside the car?"** — A new mode that halts movement immediately and raises an alarm; with more modes, refactor modes into State classes.
8. **"Express cars?"** — A car with a set of floors it may serve (e.g. lobby plus 30–40); the dispatch strategy skips cars whose served set does not include the call floor.

9. **"Door sensor that reopens for a hand?"** — A `DoorSensor` input; while it reports an obstruction, `step()` reopens the doors instead of closing them (same slot as the overload check).

Real-world example

Modern high-rise buildings use **destination dispatch** systems (for example Schindler's PORT, Otis CompassPlus and KONE's destination control): riders choose their floor in the lobby and the system tells them which car to take, which cuts stops per trip — extension 4. Controllers switch between traffic modes such as **up-peak** in the morning. The fire behaviour in this design follows the idea of **Phase I emergency recall** in elevator safety codes such as ASME A17.1: on a fire alarm, cars return to the designated landing, open their doors and are removed from normal service.

In the interview

Plan for 45 minutes: 5 min requirements and scope (how many cars, which modes) → 5 min entities → **15 min on the stop list and LOOK** (two sets, the four cases) → 10 min car `step()` and dispatcher strategy → 10 min concurrency, testing and follow-ups.

> **30-second answer:** "Each car keeps its stops in two sorted sets, up-stops and down-stops, and uses LOOK: keep serving stops ahead in the current direction, turn at the last request, never run to the end of the shaft for nothing. A car's `step()` does one action — close doors, refuse to move if overloaded, move one floor, or stop and open — which makes the whole system testable without a clock. A dispatcher assigns hall calls through a `DispatchStrategy`, nearest suitable car by default, and queues calls when no car is available. Doors, load, maintenance and fire recall always override scheduling."

Follow-ups with answers

- *"Why LOOK and not SSTF?"* — SSTF can starve a distant floor; LOOK serves every accepted stop within one round trip (one sweep up, one sweep down).
- *"Why two sets and not one?"* — A floor can be requested going up and going down; one set cannot tell which rider to serve on which sweep.
- *"Where do you put the lock?"* — Around each car's stop list and state (the car's monitor — the built-in lock every Java object carries, taken by `synchronized`), and around the dispatcher's assignment. No global lock across cars.

Self-check

1. **Q:** Difference between SCAN and LOOK? **A:** SCAN runs to the end of the shaft before reversing; LOOK reverses at the last request.
2. **Q:** A car at 4 going up has stops 6(up), 9(car), 2(down), 1(up). Order? **A:** 6, 9, 2, 1.
3. **Q:** Why is the scheduling rule in `StopList` and not `Elevator`? **A:** Single Responsibility — the car drives, the stop list decides; each can be tested alone.
4. **Q:** What does the dispatcher do when every car is in maintenance? **A:** Queues the call and retries on every tick.
5. **Q:** Why does one `step()` do only one action? **A:** Deterministic, clock-free simulation: tests are sequences of calls with expected events.
6. **Q:** When would you convert modes into State classes? **A:** When there are many modes with different behaviour per action (fire phase I/II, independent service, VIP, emergency stop).

TL;DR

- **Two sorted sets** (up/down) of stops + **LOOK** = efficient, starvation-free scheduling.
- **`step()` = one action**; doors, load, maintenance and fire recall override scheduling.
- **Strategy** for dispatch (nearest car by default), with a pending queue and re-dispatch when a car leaves service.
- Thread safety: per-car lock, synchronized dispatcher, concurrent hall-light set.

14.6 Design a Traffic Signal Control System

A **traffic signal controller** decides which approaches of an intersection get green, for how long, and in what order. An **approach** is one road entering the intersection (north, south, east, west). A **phase** is a set of approaches that may be green together because their paths do not cross (for example north and south going straight). The controller cycles through phases, inserting **yellow** (warning) and **all-red** (clearance) intervals between them.

This question tests a **timed state machine** with a **hard safety rule** — *two crossing flows must never be green at the same time* — plus an **emergency override** that must interrupt the normal cycle *safely* (never by cutting a yellow short).

Problem & clarifying questions

Question	Assumed answer
How many roads?	A four-way intersection (N, S, E, W); phases are configurable.
Which signals?	Red, yellow, green; flashing red in fault mode.
Fixed or adaptive timing?	Configurable: fixed durations, or adaptive based on queue sensors.
Pedestrian signals, turn arrows?	Out of scope (extensions).
Emergency vehicles?	Yes: an approaching ambulance or fire truck gets green as soon as safely possible, held until it passes.
Many intersections?	Each intersection has its own controller; a central system can coordinate them (extension).
Real time?	A timer ticks once per second; the logic is tick-driven so it can also be simulated.

Requirements

Functional

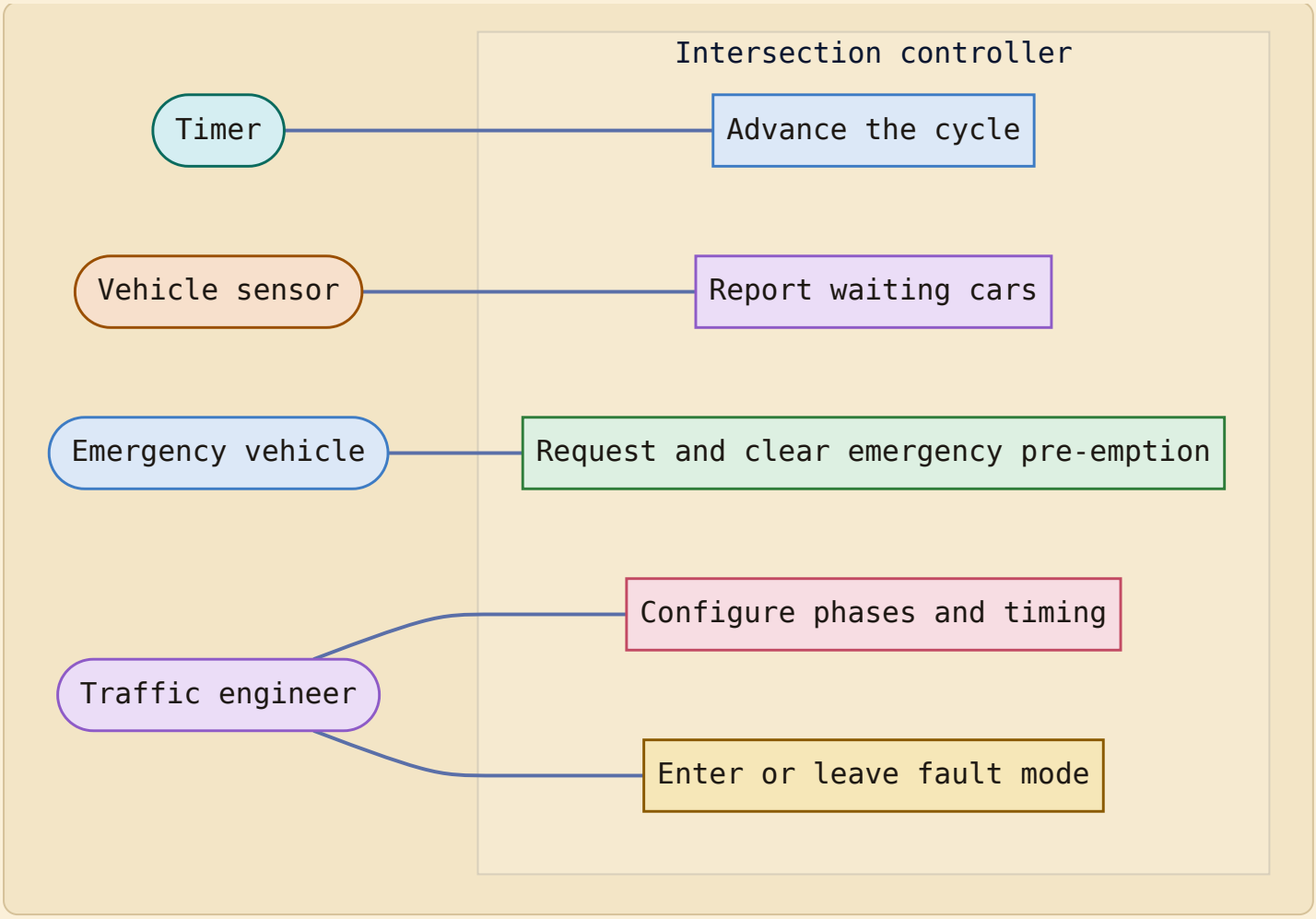
1. Control traffic at an intersection with **multiple roads**.
2. Support **red, yellow and green** signals.
3. Signal **durations are configurable** and can **adapt to traffic** conditions.
4. Transitions are **smooth and safe**: green → yellow → all-red → next green.
5. Handle **emergencies**: an approaching ambulance or fire truck gets priority.
6. Be **scalable and extensible** (more phases, more intersections, new features).
7. On a controller fault, fall back to a **safe mode** (all flashing red = treat as an all-way stop).

Non-functional

- **Safety invariant** (an invariant is a rule that must hold at every moment)**:** never conflicting greens, not even for an instant during a switch.
- **Deterministic timing:** a tick = one second; testable with a simulated clock.
- **Thread safety:** a timer thread ticks while sensor and emergency inputs arrive from other threads.
- **Observability** (the system's behaviour can be watched from outside)**:** displays, logs and monitors can subscribe to light changes.

Use cases

Actors: Timer (ticks every second), **Vehicle sensors** (report queue lengths), **Emergency vehicle / dispatch** (request and clear pre-emption — interrupting the normal cycle to serve a priority request), **Traffic engineer** (configures phases and timing), **Drivers** (see the lights).



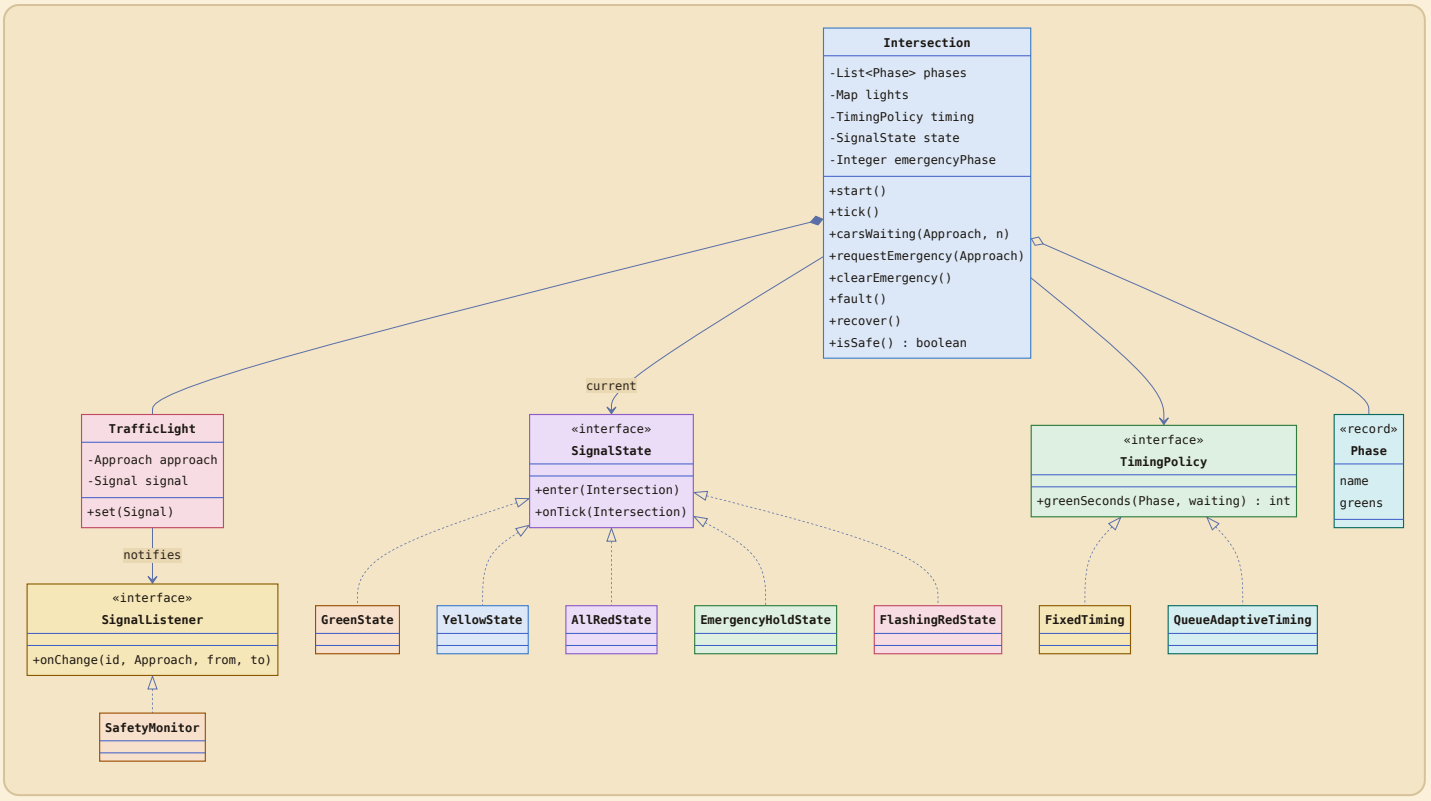
Core entities

Class	Responsibility
`Signal` (enum)	RED, YELLOW, GREEN, FLASHING_RED.
`Approach` (enum)	NORTH, SOUTH, EAST, WEST.
`Phase` (record)	A name and the set of approaches that may be green together.
`TrafficLight`	One signal head for one approach; notifies listeners on every change.
`SignalListener`	**Observer** interface for light changes.
`SafetyMonitor`	A listener that checks the safety invariant after every change to green.
`TimingPolicy`	**Strategy** for green duration: `FixedTiming`, `QueueAdaptiveTiming`.
`SignalState` + 5 states	**State**: `GreenState`, `YellowState`, `AllRedState`, `EmergencyHoldState`, `FlashingRedState`.
`Intersection`	The State context (the object that holds the current state and hands it each call): phases, lights, timing, sensors, emergency flag; `tick()`.

Mapping to the common problem statement. The widely used version of this problem (for example in the open-source *awesome-low-level-design* collection) asks for: an intersection with multiple roads, red/yellow/green signals, configurable durations that can adjust to traffic, smooth transitions, handling of emergency vehicles (ambulances, fire trucks), and a scalable, extensible design. Its classes map onto ours as follows: the `Signal` enum → our `Signal`; `Road` (id, name, and the traffic light facing it) → our `Approach` plus the `TrafficLight` stored for it; `TrafficLight` (current signal, durations, notifies observers) → our `TrafficLight` (signal + listeners) with the durations moved into `TimingPolicy` and the yellow/all-red settings; `TrafficController` (a Singleton managing all roads) → our `Intersection`, deliberately **not** a Singleton. What the reference leaves implicit, and we make explicit,

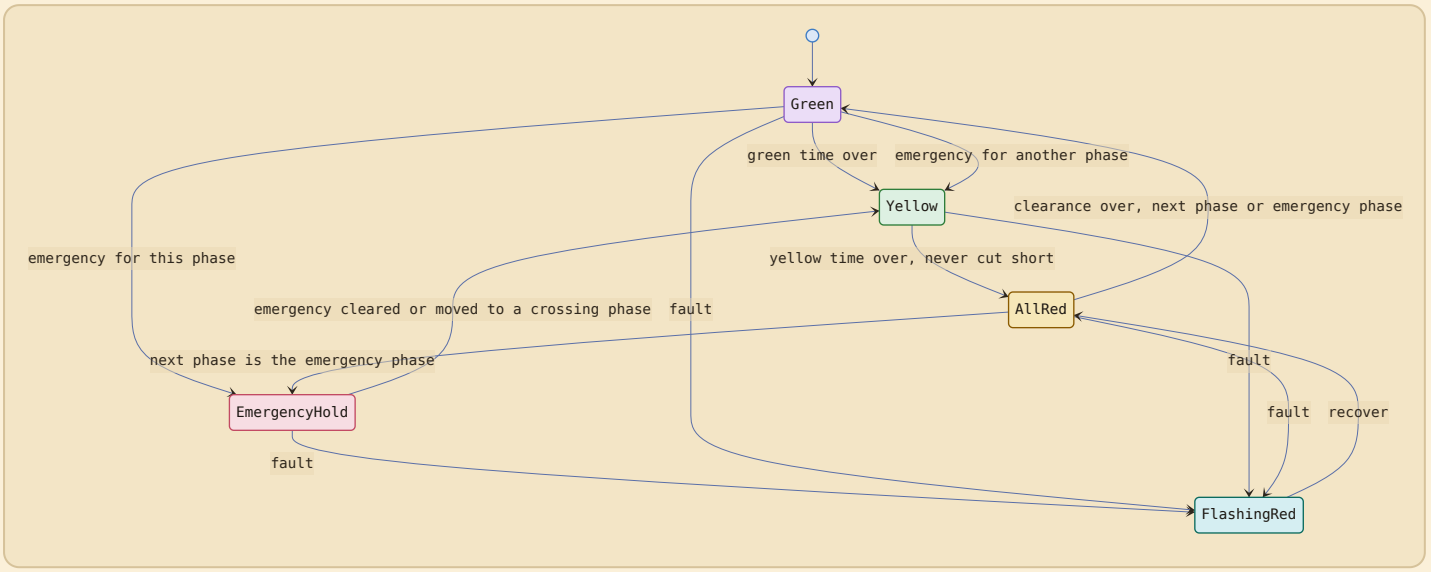
is the phase model (which roads may be green together), the all-red clearance interval, and the safety check.

Class diagram

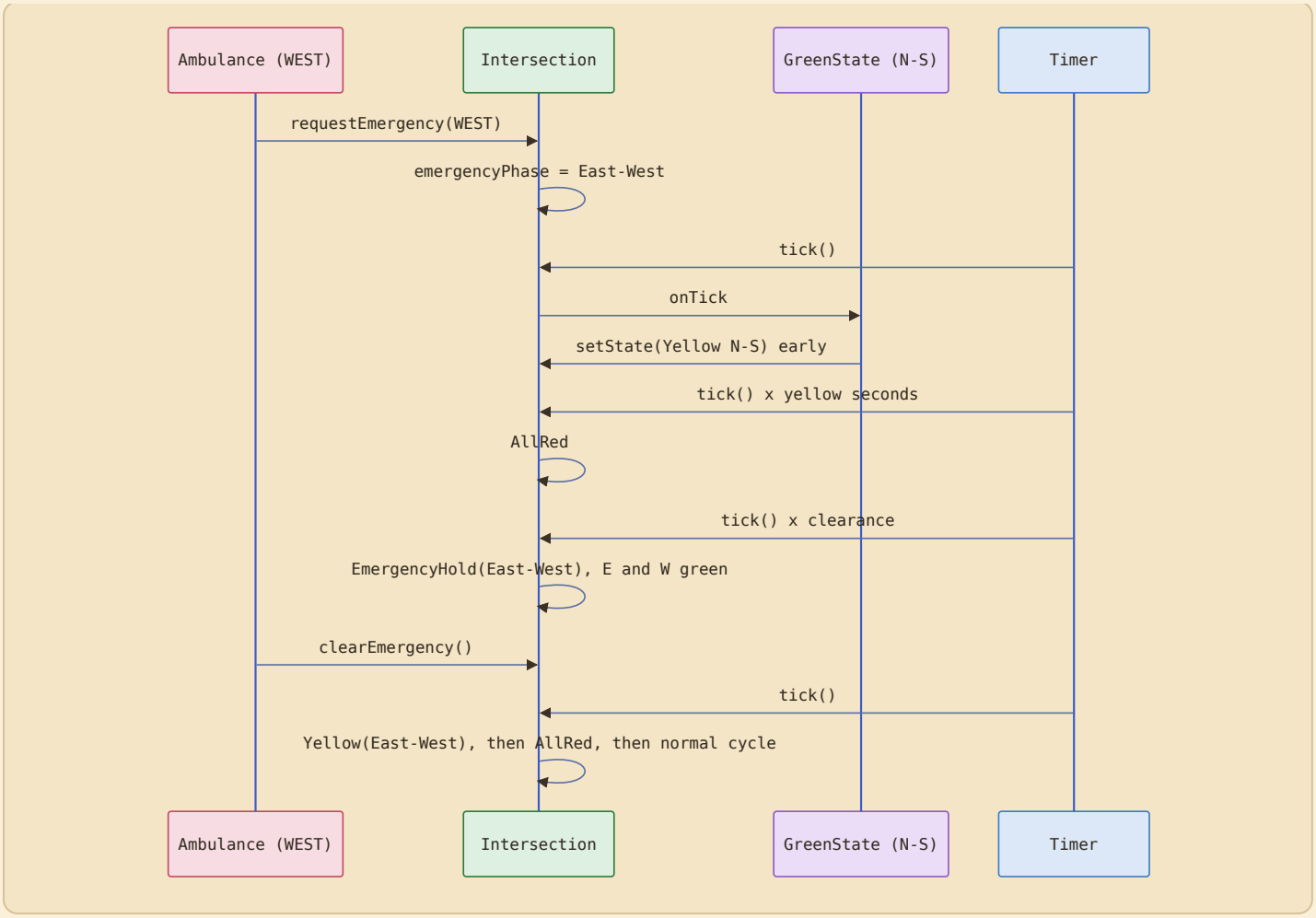


Key flows

The controller's state machine.



Emergency pre-emption.



One full fixed cycle (green 5 s, yellow 2 s, all-red 1 s): `N-S green (5) → N-S yellow (2) → all red (1) → E-W green (5) → E-W yellow (2) → all red (1) → …` — a 16-second cycle.

Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Each interval is a state class with `enter` and `onTick`	**State**	What a tick means depends on the interval; the transitions (and the "yellow is never cut short" rule) live in one small class each	One `switch` over an enum with counters for every case
States **carry data** (remaining seconds) and are created per transition	State with per-instance data	Unlike the vending machine's stateless states, these hold a countdown, so they cannot be shared	Shared singletons with counters on the context
Green duration comes from a `TimingPolicy`	**Strategy**	Fixed, adaptive (queue sensors), time-of-day plans all swap at run time	Durations hard-coded in `GreenState`
Light changes are published to listeners	**Observer**	Displays, logs, a central monitoring system and the safety monitor subscribe independently	The intersection calling each consumer directly
`showPhaseGreen` turns others **red first**, then turns the phase green	Safety by construction	There is never an instant with conflicting greens	Setting lights in arbitrary order
A separate `SafetyMonitor` checks the invariant	Defence in depth (several independent safety layers)	Mirrors real hardware conflict monitors that force flash if a bug ever causes conflicting greens	Trusting the controller logic alone
Emergency is a **flag** the states read, not a direct jump	Safety	A request only *shortens* green; yellow and all-red still run in full	Jumping straight to the emergency green (a crash risk)
The awesome-lld reference makes the controller a **Singleton**	Rejected	A city has many intersections, each with its own controller; the demo runs two. A per-city `TrafficControlCenter` can own them	`TrafficController.getInstance()`

Java implementation

Package `lld.ch14.trafficsignal`.

```java
// File: lld/ch14/trafficsignal/Model.java
package lld.ch14.trafficsignal;

import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.CopyOnWriteArrayList;

enum Signal { RED, YELLOW, GREEN, FLASHING_RED }

enum Approach { NORTH, SOUTH, EAST, WEST }

/** A phase = the set of approaches that may be green together (they never cross). */
record Phase(String name, Set<Approach> greens) {}

/** Observer: anyone who wants to hear about light changes (displays, logs, safety monitor). */
interface SignalListener {
    void onChange(String intersection, Approach approach, Signal from, Signal to);
}

/** One light head facing one approach. */
final class TrafficLight {
    private final String intersection;
    private final Approach approach;
    private final List<SignalListener> listeners;
    private Signal signal = Signal.RED;

    TrafficLight(String intersection, Approach approach, List<SignalListener> listeners) {
        this.intersection = intersection;
        this.approach = approach;
        this.listeners = listeners;
    }

    void set(Signal to) {
        if (to == signal) return;
        Signal from = signal;
        signal = to;
        listeners.forEach(l -> l.onChange(intersection, approach, from, to));
    }

    Signal signal() { return signal; }
}

/** Strategy: how long a phase stays green. */
interface TimingPolicy {
    int greenSeconds(Phase phase, Map<Approach, Integer> waitingCars);
}

record FixedTiming(int green) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) { return green; }
}

/** More waiting cars on the phase's approaches = longer green, between min and max. */
record QueueAdaptiveTiming(int min, int max, int secondsPerCar) implements TimingPolicy {
    public int greenSeconds(Phase p, Map<Approach, Integer> waiting) {
        int cars = p.greens().stream().mapToInt(a -> waiting.getOrDefault(a, 0)).sum();
        return Math.max(min, Math.min(max, cars * secondsPerCar));
    }
}

/** Observer that enforces the one rule that must never break: no two crossing flows moving at once. */
final class SafetyMonitor implements SignalListener {
    private final Intersection[] watched = new Intersection[1];
    private final List<String> violations = new CopyOnWriteArrayList<>();

    void watch(Intersection i) { watched[0] = i; }

    @Override public void onChange(String id, Approach a, Signal from, Signal to) {
        if (to != Signal.GREEN || watched[0] == null) return;
        if (!watched[0].isSafe()) violations.add(id + ": " + a + " went green while a crossing flow was moving");
```

```
    }

    List<String> violations() { return violations; }
}
```

```java
// File: lld/ch14/trafficsignal/States.java
package lld.ch14.trafficsignal;

/** State pattern: each state knows what one second of time means for it, and which state is next. */
interface SignalState {
    default void enter(Intersection x) {}
    void onTick(Intersection x);
    String name();
}

final class GreenState implements SignalState {
    private final int phase;
    private int remaining;

    GreenState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) {
        if (Integer.valueOf(phase).equals(x.emergencyPhase())) { x.setState(new EmergencyHoldState(phase));
return; }
        remaining = x.greenSecondsFor(phase);
        x.showPhaseGreen(phase);
    }

    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        if (em != null) {                                   // pre-emption: end this green early
            x.setState(em == phase ? new EmergencyHoldState(phase) : new YellowState(phase));
            return;
        }
        if (--remaining <= 0) x.setState(new YellowState(phase));
    }

    @Override public String name() { return "GREEN(" + phase + ")"; }
}

final class YellowState implements SignalState {
    private final int phase;
    private int remaining;

    YellowState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) {
        remaining = x.yellowSeconds();
        x.showPhase(phase, Signal.YELLOW);
    }

    @Override public void onTick(Intersection x) {          // yellow is never cut short: that is unsafe
        if (--remaining <= 0) x.setState(new AllRedState(x.nextPhaseAfter(phase)));
    }

    @Override public String name() { return "YELLOW(" + phase + ")"; }
}

/** Clearance interval: everyone red so the junction empties before the next phase. */
final class AllRedState implements SignalState {
    private final int next;
    private int remaining;

    AllRedState(int next) { this.next = next; }

    @Override public void enter(Intersection x) {
        remaining = x.allRedSeconds();
        x.showAllRed();
    }

    @Override public void onTick(Intersection x) {
        if (--remaining <= 0) {
            Integer em = x.emergencyPhase();
            x.setState(new GreenState(em != null ? em : next));
        }
```

```
    }

    @Override public String name() { return "ALL_RED"; }
}

/** Emergency vehicle approaching: hold its phase green until the override is cleared. */
final class EmergencyHoldState implements SignalState {
    private final int phase;

    EmergencyHoldState(int phase) { this.phase = phase; }

    @Override public void enter(Intersection x) { x.showPhaseGreen(phase); }

    @Override public void onTick(Intersection x) {
        Integer em = x.emergencyPhase();
        // Cleared, or a newer request came from a crossing phase: leave through yellow + all-red.
        if (em == null || em != phase) x.setState(new YellowState(phase));
    }

    @Override public String name() { return "EMERGENCY_HOLD(" + phase + ")"; }
}

/** Controller fault or power problem: all approaches flash red (treat as a four-way stop). */
final class FlashingRedState implements SignalState {
    @Override public void enter(Intersection x) { x.showAll(Signal.FLASHING_RED); }
    @Override public void onTick(Intersection x) {}
    @Override public String name() { return "FLASHING_RED"; }
}
```

```java
// File: lld/ch14/trafficsignal/Intersection.java
package lld.ch14.trafficsignal;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.StringJoiner;
import java.util.concurrent.CopyOnWriteArrayList;

/**
 * The context of the State pattern. tick() is called once per second by a timer thread;
 * emergency and sensor calls arrive from other threads. Every public method is
 * synchronized, so a state transition is never observed half-done.
 */
final class Intersection {
    private final String id;
    private final List<Phase> phases;
    private final Map<Approach, TrafficLight> lights = new EnumMap<>(Approach.class);
    private final Map<Approach, Integer> waitingCars = new EnumMap<>(Approach.class);
    private final List<SignalListener> listeners = new CopyOnWriteArrayList<>();
    private final int yellowSeconds;
    private final int allRedSeconds;
    private volatile TimingPolicy timing;
    private SignalState state;
    private Integer emergencyPhase;         // null = no override
    private long clock;

    Intersection(String id, List<Phase> phases, TimingPolicy timing, int yellowSeconds, int allRedSeconds) {
        this.id = id;
        this.phases = List.copyOf(phases);
        this.timing = timing;
        this.yellowSeconds = yellowSeconds;
        this.allRedSeconds = allRedSeconds;
        for (Approach a : Approach.values()) lights.put(a, new TrafficLight(id, a, listeners));
    }

    void addListener(SignalListener l) { listeners.add(l); }

    synchronized void start() { setState(new GreenState(0)); }
    synchronized void tick() { clock++; state.onTick(this); }

    // ----- inputs from other threads -----
    synchronized void carsWaiting(Approach a, int n) { waitingCars.put(a, n); }

    synchronized void requestEmergency(Approach from) {
        for (int i = 0; i < phases.size(); i++) if (phases.get(i).greens().contains(from)) emergencyPhase = i;
    }

    synchronized void clearEmergency() { emergencyPhase = null; }
    synchronized void fault() { setState(new FlashingRedState()); }
    synchronized void recover() { emergencyPhase = null; setState(new AllRedState(0)); }
    void setTiming(TimingPolicy t) { timing = t; }

    // ----- used by the states -----
    void setState(SignalState s) { state = s; s.enter(this); }
    Integer emergencyPhase() { return emergencyPhase; }
    int nextPhaseAfter(int p) { return (p + 1) % phases.size(); }
    int greenSecondsFor(int p) { return timing.greenSeconds(phases.get(p), waitingCars); }
    int yellowSeconds() { return yellowSeconds; }
    int allRedSeconds() { return allRedSeconds; }

    /** Turns red first, then green: never two crossing flows green, even for an instant. */
    void showPhaseGreen(int p) {
        Phase phase = phases.get(p);
        lights.forEach((a, l) -> { if (!phase.greens().contains(a)) l.set(Signal.RED); });
        phase.greens().forEach(a -> lights.get(a).set(Signal.GREEN));
    }

    void showPhase(int p, Signal s) { phases.get(p).greens().forEach(a -> lights.get(a).set(s)); }
    void showAllRed() { lights.values().forEach(l -> l.set(Signal.RED)); }
```

```
    void showAll(Signal s) { lights.values().forEach(l -> l.set(s)); }

    /** Safe = every approach that is green or yellow belongs to ONE phase. */
    synchronized boolean isSafe() {
        return phases.stream().anyMatch(p -> lights.entrySet().stream()
                .filter(e -> e.getValue().signal() == Signal.GREEN || e.getValue().signal() == Signal.YELLOW)
                .allMatch(e -> p.greens().contains(e.getKey())));
    }

    synchronized String snapshot() {
        StringJoiner j = new StringJoiner(" ");
        lights.forEach((a, l) -> j.add(a.name().charAt(0) + ":" + l.signal().name().charAt(0)
                + (l.signal() == Signal.FLASHING_RED ? "*" : "")));
        return j + "  " + state.name();
    }

    synchronized long clock() { return clock; }
    String id() { return id; }
}
```

```java
// File: lld/ch14/trafficsignal/Main.java
package lld.ch14.trafficsignal;

import java.util.List;
import java.util.Set;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static final List<Phase> PHASES = List.of(
            new Phase("North-South", Set.of(Approach.NORTH, Approach.SOUTH)),
            new Phase("East-West", Set.of(Approach.EAST, Approach.WEST)));

    public static void main(String[] args) throws Exception {
        SafetyMonitor safety = new SafetyMonitor();
        Intersection x = new Intersection("5th&Main", PHASES, new FixedTiming(5), 2, 1);
        safety.watch(x);
        x.addListener(safety);
        AtomicInteger changes = new AtomicInteger();
        x.addListener((id, a, from, to) -> changes.incrementAndGet());

        System.out.println("== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==");
        x.start();
        System.out.println("t=0   " + x.snapshot());
        run(x, 16);

        System.out.println("\n== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==");
        x.setTiming(new QueueAdaptiveTiming(3, 10, 1));
        x.carsWaiting(Approach.EAST, 7);
        x.carsWaiting(Approach.WEST, 5);
        x.carsWaiting(Approach.NORTH, 2);
        run(x, 20);

        System.out.println("\n== 3. Ambulance from the WEST while North-South is green ==");
        x.setTiming(new FixedTiming(5));
        run(x, 2);
        System.out.println("t=" + x.clock() + "  emergency request: WEST");
        x.requestEmergency(Approach.WEST);
        run(x, 7);
        System.out.println("t=" + x.clock() + "  ambulance has passed: clear");
        x.clearEmergency();
        run(x, 5);

        System.out.println("\n== 4. Controller fault -> flashing red, then recovery ==");
        x.fault();
        System.out.println("t=" + x.clock() + "  " + x.snapshot());
        x.recover();
        run(x, 3);

        System.out.println("\n== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==");
        Intersection y = new Intersection("Oak&Pine", PHASES, new FixedTiming(5), 2, 1);
        SafetyMonitor safety2 = new SafetyMonitor();
        safety2.watch(y);
        y.addListener(safety2);
        y.start();
        CountDownLatch done = new CountDownLatch(500);
        ScheduledExecutorService timer = Executors.newSingleThreadScheduledExecutor();
        timer.scheduleAtFixedRate(() -> { y.tick(); done.countDown(); }, 0, 1, TimeUnit.MILLISECONDS);
        Thread dispatcher = Thread.ofVirtual().start(() -> {
            for (int i = 0; i < 20; i++) {
                y.requestEmergency(i % 2 == 0 ? Approach.EAST : Approach.NORTH);
                Thread.onSpinWait();
                y.clearEmergency();
            }
        });
        done.await();
        timer.shutdown();
```

```
        timer.awaitTermination(1, TimeUnit.SECONDS);
        dispatcher.join();
        System.out.println("ticks run: " + (y.clock() >= 500) + ", safety violations: " +
safety2.violations().size());

        System.out.println("\nlight changes seen by observer: " + changes.get()
                + ", safety violations: " + safety.violations().size());
    }

    static void run(Intersection x, int seconds) {
        String last = x.snapshot();
        for (int i = 0; i < seconds; i++) {
            x.tick();
            String now = x.snapshot();
            if (!now.equals(last)) System.out.printf("t=%-3d %s%n", x.clock(), now);
            last = now;
        }
    }
}
```

How the code works, part by part

- `Intersection.tick()` is `synchronized` and simply delegates to `state.onTick(this)`. A state changes the context with `setState(next)`, which immediately calls `next.enter(this)` so the new state can set lights and load its timer.
- `GreenState.enter` asks the `TimingPolicy` for the duration, and jumps straight to `EmergencyHoldState` if its phase is the emergency phase.
- `GreenState.onTick` ends the green **early** when an emergency is pending for another phase. `YellowState` never checks the emergency flag — yellow always runs in full.
- `AllRedState` chooses the emergency phase if one is pending, else the next phase in order.
- `QueueAdaptiveTiming` gives `secondsPerCar × waiting cars`, clamped between min and max.
- `SafetyMonitor` receives every change to GREEN and checks `isSafe()`: all green/yellow approaches must belong to one phase.
- Part 5 of `Main` drives a second intersection with a real `ScheduledExecutorService` (1 ms stands for 1 s) while a virtual thread (a lightweight Java 21 thread, cheap enough to create by the thousand) fires 20 emergency requests and clears; the synchronized methods keep every transition whole, and the monitor sees zero violations.

Output:

```
== 1. Fixed cycle: green 5s, yellow 2s, all-red 1s (printed when lights change) ==
t=0   N:G S:G E:R W:R  GREEN(0)
t=5   N:Y S:Y E:R W:R  YELLOW(0)
t=7   N:R S:R E:R W:R  ALL_RED
t=8   N:R S:R E:G W:G  GREEN(1)
t=13  N:R S:R E:Y W:Y  YELLOW(1)
t=15  N:R S:R E:R W:R  ALL_RED
t=16  N:G S:G E:R W:R  GREEN(0)

== 2. Adaptive timing: 12 cars waiting East-West, 2 North-South ==
t=21  N:Y S:Y E:R W:R  YELLOW(0)
t=23  N:R S:R E:R W:R  ALL_RED
t=24  N:R S:R E:G W:G  GREEN(1)
t=34  N:R S:R E:Y W:Y  YELLOW(1)
t=36  N:R S:R E:R W:R  ALL_RED

== 3. Ambulance from the WEST while North-South is green ==
t=37  N:G S:G E:R W:R  GREEN(0)
t=38  emergency request: WEST
t=39  N:Y S:Y E:R W:R  YELLOW(0)
t=41  N:R S:R E:R W:R  ALL_RED
t=42  N:R S:R E:G W:G  EMERGENCY_HOLD(1)
t=45  ambulance has passed: clear
t=46  N:R S:R E:Y W:Y  YELLOW(1)
t=48  N:R S:R E:R W:R  ALL_RED
t=49  N:G S:G E:R W:R  GREEN(0)

== 4. Controller fault -> flashing red, then recovery ==
t=50  N:F* S:F* E:F* W:F*  FLASHING_RED
t=51  N:G S:G E:R W:R  GREEN(0)

== 5. Real timer thread (1 ms = 1 s) + emergency from another thread ==
ticks run: true, safety violations: 0

light changes seen by observer: 48, safety violations: 0
```

Read the output: the fixed cycle is 5 s green, 2 s yellow, 1 s all-red; with 12 cars waiting east-west, the adaptive policy gives E-W a 10-second green (the maximum); the ambulance request at t=38 ends N-S green at the very next tick, but yellow (2 s) and all-red (1 s) still run before E-W goes green and is **held**; after the clear, the cycle resumes with N-S; a fault switches every head to flashing red; the observer counted every change and found no safety violation.

Concurrency & edge cases

Situation	Handling
Emergency request arrives between two ticks	Stored in a field under the intersection's lock; the next tick acts on it
Emergency arrives during yellow	Yellow runs in full; `AllRedState` then picks the emergency phase
Emergency for the phase that is already green	`GreenState.onTick` switches to `EmergencyHold` — no needless yellow
Two emergency vehicles from crossing directions	The latest request wins: if it arrives during a hold, `EmergencyHoldState` leaves through yellow and all-red and the new phase is then held. A real system queues requests by priority and arrival and serves them in turn (extension)
Emergency never cleared (lost message)	Add a maximum hold time after which the controller resumes the cycle
Timer thread and sensor thread overlap	All public methods are `synchronized`; listeners are in a `CopyOnWriteArrayList` (a thread-safe list that copies itself on every change)
Adaptive timing starves a quiet road	Minimum green time in `QueueAdaptiveTiming`; every phase still gets its turn in the cycle
Controller bug causes conflicting greens	`SafetyMonitor` detects it; real controllers then force flashing red
Timer drift (small delays adding up so ticks slowly fall behind)	`scheduleAtFixedRate` keeps a fixed rate; for real deployments use the controller's hardware clock

Extensions & follow-up questions

1. **"Pedestrian crossings?"** — Add pedestrian signal heads per phase (WALK during the phase's green, flashing DON'T WALK before yellow) and a push-button input that requests the phase.
2. **"Left-turn arrows?"** — More phases (e.g. N-S protected left, then N-S through); phases are data, so the state machine does not change.
3. **"Coordinate many intersections (green wave)?"** — A `TrafficControlCenter` sets each intersection's cycle length and **offset** so that a platoon (group travelling together) of cars driving at the speed limit meets greens in sequence.
4. **"Emergency corridor?"** — The centre pre-empts the next intersections along the ambulance's route, each through `requestEmergency`.
5. **"Night mode?"** — A time-of-day `TimingPolicy`, or a flashing-yellow state for the main road with flashing red for the side road.
6. **"How to test?"** — The tick-driven design is deterministic: call `tick()` N times and assert the snapshot, as `Main` does.

Real-world example

Adaptive systems such as **SCATS** (developed in Sydney) and **SCOOT** (developed in the UK) adjust green times and cycle lengths from road sensors across whole city networks — the `QueueAdaptiveTiming` strategy scaled up. **Emergency vehicle pre-emption** systems (for example infrared or GPS-based emitters on fire trucks) request green for the vehicle's approach; the controller still times out the current yellow and red clearance before serving it, exactly like our `EmergencyHoldState`. In North American cabinets, a **conflict monitor / malfunction management unit** watches the outputs and forces the intersection into flash if it ever sees conflicting greens — our `SafetyMonitor` observer.

In the interview

Plan for 40 minutes: 5 min requirements (phases, timing, emergency, fault) → 5 min entities → 10 min draw the state machine (green → yellow → all-red → next; emergency and fault edges) → 15 min code `GreenState` / `YellowState` / `AllRedState` and `tick()` → 5 min concurrency and follow-ups.

30-second answer: "An intersection has phases — sets of approaches that can be green together — and a controller that is a timed State machine: Green, Yellow, All-Red, plus Emergency-Hold and Flashing-Red. A timer calls `tick()` once a second; each state counts down and names the next one. Green duration comes from a swappable timing Strategy, fixed or queue-adaptive. An emergency request only shortens the current green: yellow and all-red always run in full, then the emergency phase is held green until cleared. Lights publish changes to observers, including a safety monitor that checks no crossing flows are ever green together."

Follow-ups with answers

- *"Why not jump straight to the emergency green?"* — Cars already in the intersection need the yellow and all-red intervals to clear it; skipping them causes crashes.
- *"Why is the controller not a Singleton?"* — There is one per intersection, and a city has thousands.
- *"How do you make the timing adaptive?"* — Sensors report queue lengths; a `TimingPolicy` turns them into green seconds within a min/max.

Self-check

1. **Q:** What is a phase? **A:** A set of approaches that may be green together because their paths do not cross.
2. **Q:** Why the all-red interval? **A:** To let vehicles still in the intersection clear it before the crossing flow gets green.
3. **Q:** What happens to an emergency request that arrives during yellow? **A:** Yellow finishes; all-red then picks the emergency phase instead of the normal next phase.
4. **Q:** Why do these states hold data while the vending states do not? **A:** Each interval has its own countdown, so each state instance carries remaining seconds.
5. **Q:** Which pattern lets you switch from fixed to adaptive timing at run time? **A:** Strategy (`TimingPolicy`).
6. **Q:** What does the `SafetyMonitor` check? **A:** That all green/yellow approaches belong to a single phase whenever a light turns green.

TL;DR

- **Timed State machine**: Green → Yellow → All-Red → next Green; plus Emergency-Hold and Flashing-Red.
- **Safety first**: red before green, yellow never cut short, conflict-monitor observer.
- **Strategy** for timing (fixed/adaptive), **Observer** for lights, emergency as a flag the states read.
- Tick-driven and synchronized: deterministic tests, safe with a timer thread plus emergency threads.

14.7 Design a Shipping Locker System

A **parcel locker** (also called a *pickup locker* or *shipping locker*) is a wall of numbered steel doors in a shop, station or apartment lobby. A courier puts a parcel behind one door; the customer later types a **pickup code** on the station's screen and that one door opens. Amazon Hub Locker, InPost's Paczkomat network in Poland and similar networks run by couriers and online shops in many countries all work this way. Lockers save the courier a failed doorstep delivery and let the customer collect at any hour.

This question looks like the parking lot (§14.1) — hand out a limited set of spaces of different sizes — but adds three things interviewers like to probe: **a lifecycle with deadlines** (a reservation that times out, a parcel that expires if nobody collects it), **a secret** (the pickup code, which must be hard to guess and never shown to the courier), and **a network** of stations (if the nearest one is full, suggest another).

Problem & clarifying questions

Question	Assumed answer
What sizes do doors come in?	A few fixed sizes (small, medium, large). A parcel may go into a bigger door if no door of its own size is free.
When is a door chosen — at the station, or before?	Before: the carrier's system **reserves** a door when the parcel is sorted for delivery, so the courier never arrives to a full station. The hold lasts 2 hours; if nothing is deposited, the door is released.
How does the customer open the door?	A 6-digit **one-time code** sent by SMS/app when the parcel is deposited. The courier never sees it.
How long can a parcel stay?	3 days after deposit. After that it **expires** and the carrier takes it back to the sender on the next visit.
What stops someone guessing codes?	Codes are random and single-use; after 5 wrong codes in a row the keypad locks for 15 minutes.
Returns (customer drops a parcel off)?	Same flow in reverse — a carrier-side "code"; treated as an extension.
Many stations?	Yes, a city-wide network. If the chosen station is full, offer the nearest station with space.
Broken doors?	A technician can take a free door **out of service**.
Payment at the locker (cash on delivery)?	Out of scope (extension).

Requirements

Functional

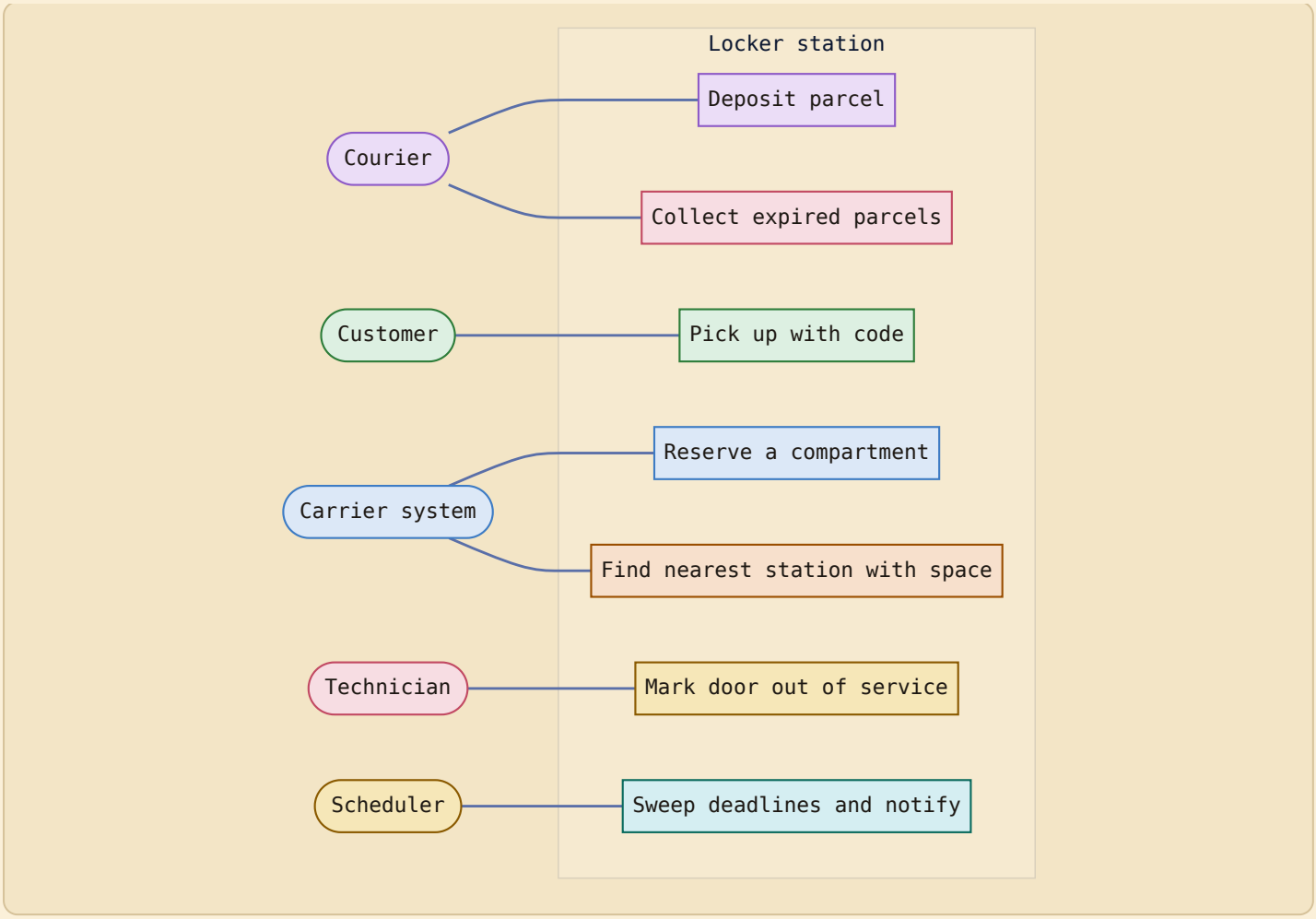
1. A station has numbered compartments of fixed sizes.
2. **Reserve** the best-fitting free compartment for a parcel (all-or-nothing); a reservation expires if the parcel is not deposited in time.
3. **Deposit**: the parcel goes in, a one-time pickup code is generated and **sent to the customer only**.
4. **Pickup**: a correct code opens exactly one door, frees the compartment and stops working.
5. **Expiry**: an uncollected parcel expires after the pickup window; the customer is told; the carrier **collects expired parcels** for return.
6. **Brute-force protection**: lock the keypad after too many wrong codes.
7. **Network search**: nearest station within a radius that has a free door this parcel fits.
8. Compartments can be taken out of service.

Non-functional

- **Never give one door to two parcels**, even when many carriers reserve at the same moment.
- **Security**: codes are unpredictable, single-use, stored only as a hash (the one-way fingerprint described in the ATM section).
- **Correct deadlines** without depending on a background timer being on time.
- **Testable**: time and randomness are injected (a fake clock and a seeded random generator in the demo — one started from a fixed seed, so it produces the same numbers every run).

Use cases

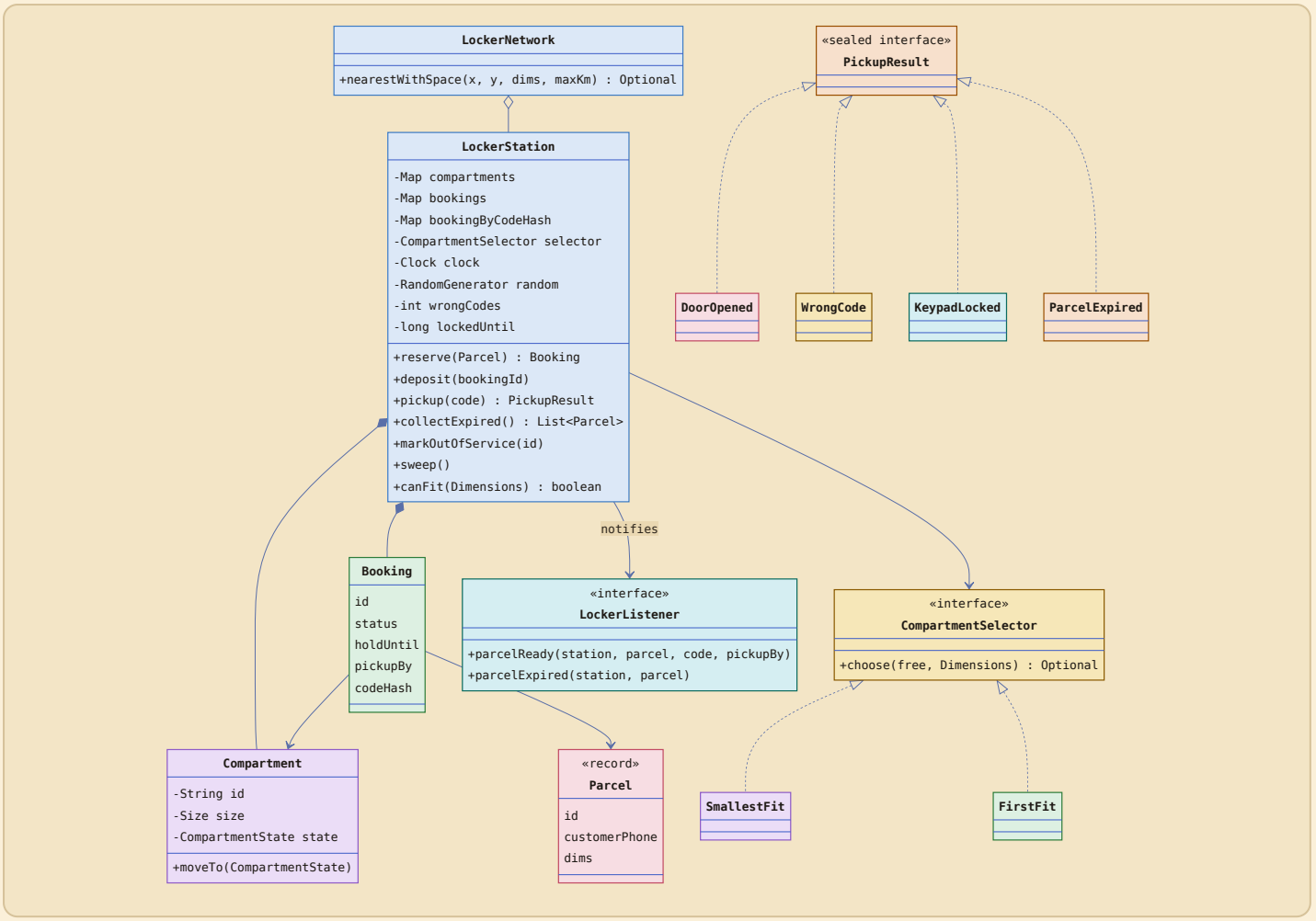
Actors: Carrier system (reserves doors), **Courier** (deposits, collects expired parcels), **Customer** (picks up), **Technician** (takes doors out of service), **Scheduler** (a component that runs a job at set times; here it runs the periodic sweep), **Notification service** (SMS/app).



Core entities

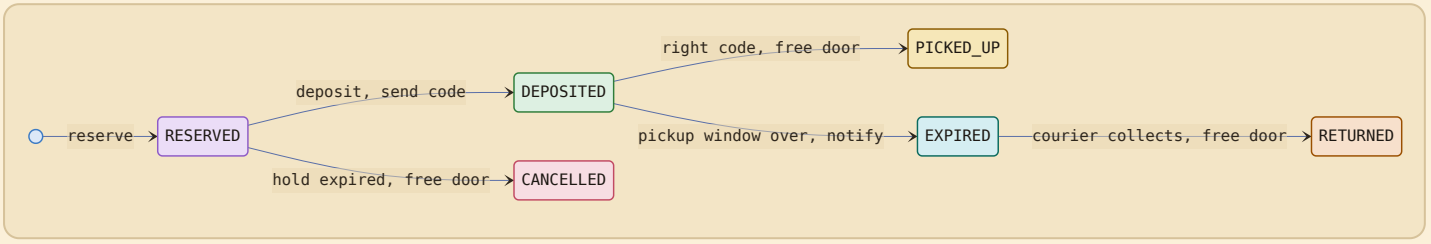
Class	Responsibility
`Size` (enum)	SMALL / MEDIUM / LARGE with inner dimensions; `fits(Dimensions)` checks a box against a door after sorting both sides.
`Dimensions` (record)	A parcel's three sides in cm (validated positive).
`Parcel` (record)	Parcel id, customer phone, dimensions.
`Compartment`	One door: id, size, `CompartmentState` (FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE) with the legal moves inside `moveTo`.
`Booking` + `BookingStatus`	One parcel's stay in one door: RESERVED → DEPOSITED → PICKED_UP, or → EXPIRED → RETURNED, or RESERVED → CANCELLED. Holds the deadlines and the code **hash**.
`PickupResult` (sealed)	`DoorOpened`, `WrongCode`, `KeypadLocked`, `ParcelExpired` — every keypad outcome is a type, not a magic string (a bare text value whose meaning is known only by convention).
`CompartmentSelector`	**Strategy**: `SmallestFit` (default), `FirstFit`.
`LockerListener`	**Observer**: parcel ready (with the code), parcel expired.
`LockerStation`	The station: doors, bookings, code index, lockout counter, clock, random generator; every state change under one lock.
`LockerNetwork`	All stations; nearest station with space.

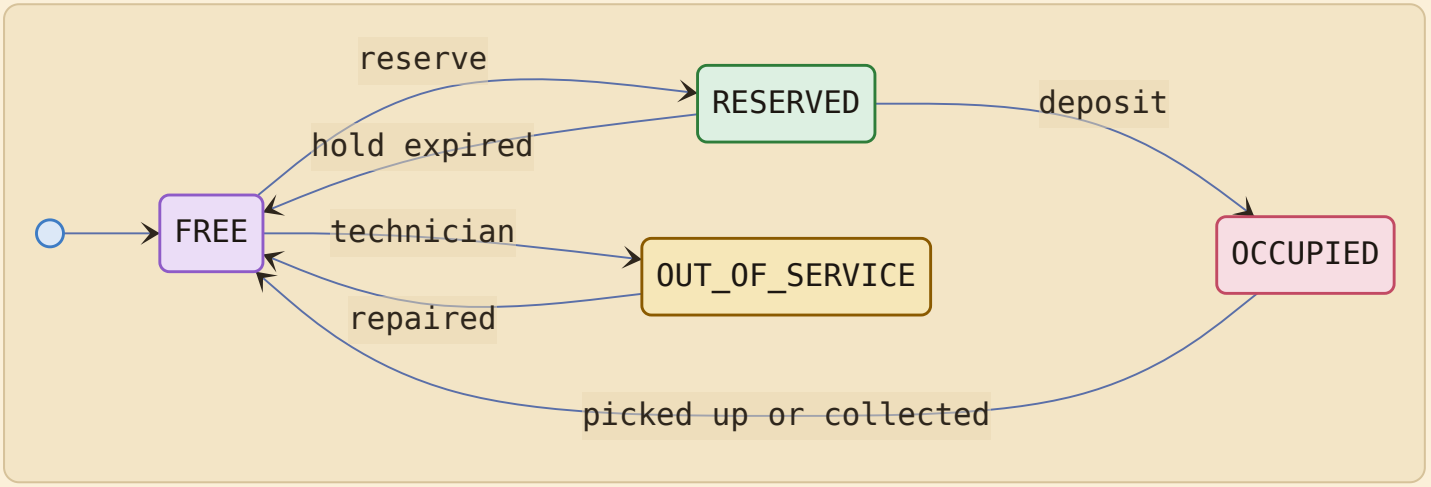
Class diagram



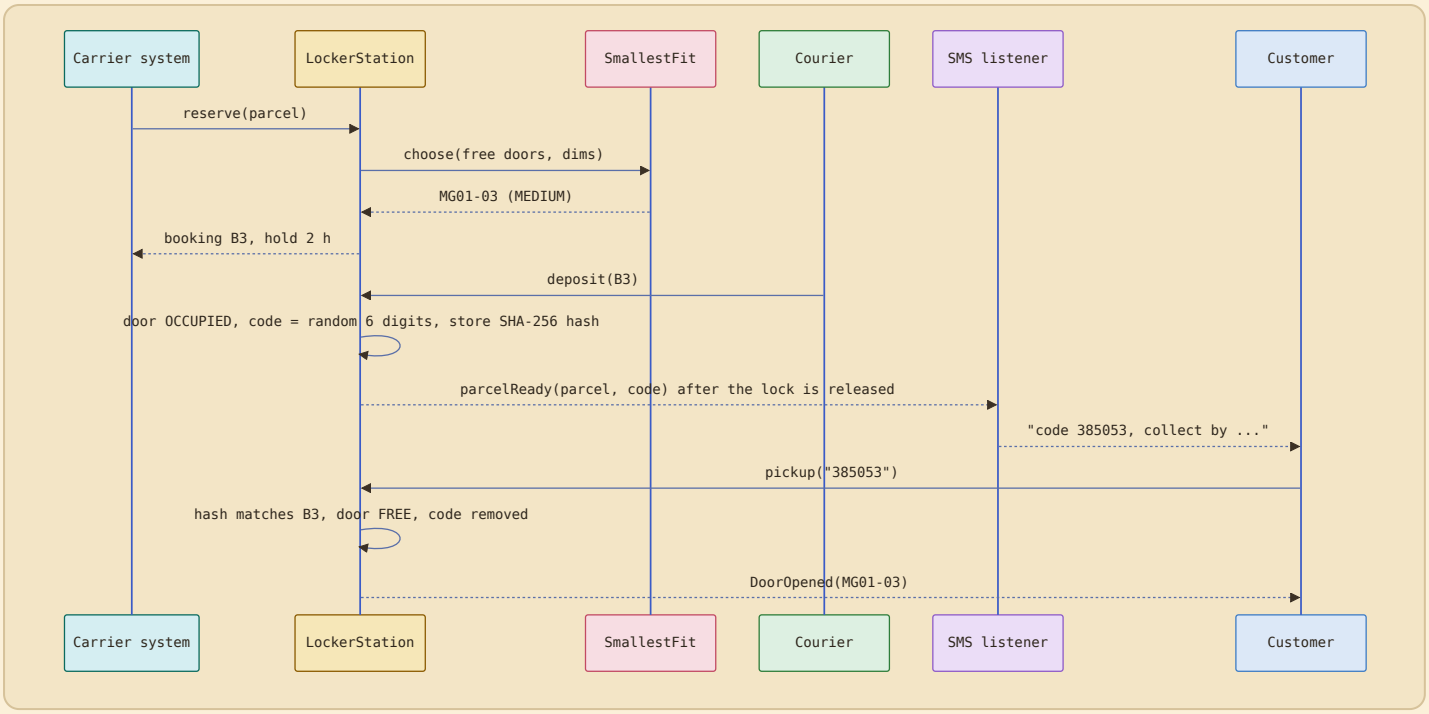
Key flows

The two state machines — first the booking, then the door. A compartment only knows whether it is free; a booking knows the parcel's story. Keeping them separate means a door can be reused while the old booking stays in the history.





Delivery and pickup.



Design decisions & patterns used

Decision	Pattern	Why	Alternative rejected
Door choice is a `CompartmentSelector`	**Strategy**	"Smallest door that fits" keeps large doors for large parcels; a station may prefer "lowest door first" (fast for the courier) or "eye-level first" (accessibility). Each is a small class.	An `if (mode == ...)` inside `reserve`
Separate `Compartment` state and `Booking` status	Two small state machines	A door is reused many times; the booking keeps each parcel's history and deadlines	One status field on the door that mixes "free" with "expired parcel inside"
Transitions checked inside `moveTo` (a `switch` over the current state)	Enum state machine	An illegal move (e.g. taking an occupied door out of service) throws at once	Setting the field anywhere
Keypad outcomes as a **sealed** `PickupResult`	Sealed result types	The UI must handle every outcome; the compiler checks the `switch`	Returning `null`, `-1` or strings
Codes: random, single-use, unique per station, stored as a salted (mixed with a random value before hashing) **SHA-256** hash	Security by design	A database leak does not reveal live codes; the courier never sees them	Storing the plain code, or deriving it from the parcel id
Notify through `LockerListener` **after** releasing the lock	**Observer**	SMS, app push and audit log are independent; a slow SMS gateway (the service that actually sends text messages) must not block the keypad	Calling the SMS gateway inside `deposit` while holding the lock
Lazy expiry (deadlines checked whenever the data is next touched) at the start of every operation, plus a `sweep()` for notifications	Deadline checking on read	Correct even if the scheduler is late or down; the scheduler only sends messages	A reaper thread (a background thread that cleans up expired items) as the *only* thing that frees doors
`Clock` and `RandomGenerator` are injected	Dependency injection	Deterministic demo and tests; production passes `Clock.systemUTC()` and a `SecureRandom` (Java's cryptographically strong random generator)	`Instant.now()` and `new Random()` inside the class
One lock per station (`synchronized`)	Monitor	A station has tens of doors and each call is microseconds, so one lock is simple and never the bottleneck (the one part that limits overall speed); stations never share a lock	Per-door CAS like the parking lot — more code for no gain at this size

Java implementation

Package `lld.ch14.locker`.

```java
// File: lld/ch14/locker/Model.java
package lld.ch14.locker;

import java.util.Arrays;

/** Inner size of a compartment door, in centimetres (height x width x depth). */
enum Size {
    SMALL(10, 40, 60), MEDIUM(20, 40, 60), LARGE(40, 40, 60);

    private final int[] sorted;

    Size(int h, int w, int d) { sorted = new int[]{h, w, d}; Arrays.sort(sorted); }

    /** A box fits if, after sorting both sides, each parcel side is no longer than the matching compartment side.
*/
    boolean fits(Dimensions p) {
        int[] q = p.sorted();
        return q[0] <= sorted[0] && q[1] <= sorted[1] && q[2] <= sorted[2];
    }
}

/** Parcel box size in centimetres. */
record Dimensions(int a, int b, int c) {
    Dimensions {
        if (a <= 0 || b <= 0 || c <= 0) throw new LockerException("Dimensions must be positive");
    }
    int[] sorted() { int[] s = {a, b, c}; Arrays.sort(s); return s; }
    @Override public String toString() { return a + "x" + b + "x" + c + "cm"; }
}

record Parcel(String id, String customerPhone, Dimensions dims) { }

enum CompartmentState { FREE, RESERVED, OCCUPIED, OUT_OF_SERVICE }

/** One door in a locker station. Its state is changed only by the station, under the station's lock. */
final class Compartment {
    private final String id;
    private final Size size;
    private CompartmentState state = CompartmentState.FREE;

    Compartment(String id, Size size) { this.id = id; this.size = size; }

    String id() { return id; }
    Size size() { return size; }
    CompartmentState state() { return state; }
    boolean isFree() { return state == CompartmentState.FREE; }

    void moveTo(CompartmentState next) {
        boolean ok = switch (state) {
            case FREE -> next == CompartmentState.RESERVED || next == CompartmentState.OUT_OF_SERVICE;
            case RESERVED -> next == CompartmentState.OCCUPIED || next == CompartmentState.FREE;
            case OCCUPIED -> next == CompartmentState.FREE;
            case OUT_OF_SERVICE -> next == CompartmentState.FREE;
        };
        if (!ok) throw new LockerException("Compartment " + id + ": illegal move " + state + " -> " + next);
        state = next;
    }

    @Override public String toString() { return id + "(" + size + ")"; }
}

/** Lifecycle of one parcel's stay in a locker. */
enum BookingStatus {
    RESERVED,      // carrier booked a door, parcel not yet inside
    DEPOSITED,     // parcel inside, pickup code sent
    PICKED_UP,     // customer opened the door with the code (final)
    EXPIRED,       // not collected in time; waiting for the carrier to take it back
    RETURNED,      // carrier removed the expired parcel (final)
    CANCELLED      // reservation hold ran out before the deposit (final)
}
```

```
/** One parcel's use of one compartment. */
final class Booking {
    final String id;
    final Parcel parcel;
    final Compartment compartment;
    BookingStatus status = BookingStatus.RESERVED;
    long holdUntil;          // epoch seconds: deposit must happen before this
    long pickupBy;           // epoch seconds: set at deposit
    String codeHash;         // only a hash of the pickup code is stored

    Booking(String id, Parcel parcel, Compartment compartment, long holdUntil) {
        this.id = id; this.parcel = parcel; this.compartment = compartment; this.holdUntil = holdUntil;
    }
}

/** What the customer's keypad entry produced. */
sealed interface PickupResult permits DoorOpened, WrongCode, KeypadLocked, ParcelExpired { }
record DoorOpened(String compartmentId, String parcelId) implements PickupResult { }
record WrongCode(int attemptsLeft) implements PickupResult { }
record KeypadLocked(long secondsLeft) implements PickupResult { }
record ParcelExpired(String parcelId) implements PickupResult { }

/** Observer: SMS/app notifications, carrier dashboards, audit logs. */
interface LockerListener {
    void parcelReady(String stationId, Parcel parcel, String pickupCode, long pickupBy);
    default void parcelExpired(String stationId, Parcel parcel) { }
}

final class LockerException extends RuntimeException {
    LockerException(String message) { super(message); }
}
```

```
// File: lld/ch14/locker/Selection.java
package lld.ch14.locker;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;

/** Strategy: which free compartment should this parcel get? */
interface CompartmentSelector {
    Optional<Compartment> choose(List<Compartment> free, Dimensions parcel);
}

/** Smallest door that fits: keeps large doors free for large parcels (the usual default). */
final class SmallestFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::size).thenComparing(Compartment::id));
    }
}

/** Lowest door number that fits, whatever its size: simple and fast, but wastes large doors. */
final class FirstFit implements CompartmentSelector {
    @Override public Optional<Compartment> choose(List<Compartment> free, Dimensions parcel) {
        return free.stream()
                .filter(c -> c.size().fits(parcel))
                .min(Comparator.comparing(Compartment::id));
    }
}
```

```java
// File: lld/ch14/locker/LockerStation.java
package lld.ch14.locker;

import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.time.Clock;
import java.time.Duration;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.HashMap;
import java.util.HexFormat;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.random.RandomGenerator;

/**
 * One locker station (a bank of doors at one address). Every state change happens under the
 * station's monitor lock: a station has tens of doors and each operation takes microseconds,
 * so one lock is simple and never the bottleneck.
 */
final class LockerStation {
    static final int MAX_WRONG_CODES = 5;

    private final String id;
    private final double x, y;                                  // location in km on a city grid
    private final Map<String, Compartment> compartments = new LinkedHashMap<>();
    private final Map<String, Booking> bookings = new HashMap<>();
    private final Map<String, String> bookingByCodeHash = new HashMap<>();
    private final List<LockerListener> listeners = new CopyOnWriteArrayList<>();
    private final CompartmentSelector selector;
    private final Clock clock;
    private final RandomGenerator random;
    private final Duration reservationHold, pickupWindow, lockout;
    private final List<Parcel> newlyExpired = new ArrayList<>();
    private int nextBooking = 1, wrongCodes = 0;
    private long lockedUntil = 0;

    /** layout = the size of each door, in door-number order (door 01 is the first element). */
    LockerStation(String id, double x, double y, List<Size> layout, CompartmentSelector selector,
                  Clock clock, RandomGenerator random) {
        this.id = id; this.x = x; this.y = y;
        this.selector = selector; this.clock = clock; this.random = random;
        this.reservationHold = Duration.ofHours(2);
        this.pickupWindow = Duration.ofDays(3);
        this.lockout = Duration.ofMinutes(15);
        for (int n = 1; n <= layout.size(); n++) {
            String cid = "%s-%02d".formatted(id, n);
            compartments.put(cid, new Compartment(cid, layout.get(n - 1)));
        }
    }

    /** Convenience layout: small doors first, then medium, then large. */
    static List<Size> layout(int small, int medium, int large) {
        List<Size> l = new ArrayList<>();
        for (int i = 0; i < small; i++) l.add(Size.SMALL);
        for (int i = 0; i < medium; i++) l.add(Size.MEDIUM);
        for (int i = 0; i < large; i++) l.add(Size.LARGE);
        return l;
    }

    String id() { return id; }
    double distanceTo(double px, double py) { return Math.hypot(x - px, y - py); }
    void addListener(LockerListener l) { listeners.add(l); }

    /** Carrier books a door before driving to the station. All-or-nothing: a door, or an exception. */
    synchronized Booking reserve(Parcel parcel) {
        expireOverdue();
```

```
        if (bookings.values().stream().anyMatch(b -> b.parcel.id().equals(parcel.id()) && isActive(b)))
            throw new LockerException("Parcel " + parcel.id() + " already has a locker");
        List<Compartment> free = compartments.values().stream().filter(Compartment::isFree).toList();
        Compartment c = selector.choose(free, parcel.dims())
                .orElseThrow(() -> new LockerException("No free compartment fits " + parcel.dims() + " at " +
id));
        c.moveTo(CompartmentState.RESERVED);
        Booking b = new Booking(id + "-B" + nextBooking++, parcel, c, now() + reservationHold.toSeconds());
        bookings.put(b.id, b);
        return b;
    }

    /** Carrier puts the parcel in and closes the door. The code goes to the customer, never to the carrier. */
    void deposit(String bookingId) {
        Booking b;
        String code;
        synchronized (this) {
            expireOverdue();
            b = booking(bookingId);
            if (b.status != BookingStatus.RESERVED) throw new LockerException("Booking " + bookingId + " is " +
b.status);
            b.compartment.moveTo(CompartmentState.OCCUPIED);
            b.status = BookingStatus.DEPOSITED;
            b.pickupBy = now() + pickupWindow.toSeconds();
            do { code = "%06d".formatted(random.nextInt(1_000_000)); }       // unique among active codes here
            while (bookingByCodeHash.containsKey(hash(code)));
            b.codeHash = hash(code);
            bookingByCodeHash.put(b.codeHash, b.id);
        }
        for (LockerListener l : listeners) l.parcelReady(id, b.parcel, code, b.pickupBy);   // outside the lock
    }

    /** Customer types a code on the station keypad. */
    synchronized PickupResult pickup(String code) {
        expireOverdue();
        long t = now();
        if (t < lockedUntil) return new KeypadLocked(lockedUntil - t);
        String bookingId = bookingByCodeHash.get(hash(code));
        if (bookingId == null) {
            if (++wrongCodes >= MAX_WRONG_CODES) {                  // slow down code guessing
                lockedUntil = t + lockout.toSeconds();
                wrongCodes = 0;
                return new KeypadLocked(lockout.toSeconds());
            }
            return new WrongCode(MAX_WRONG_CODES - wrongCodes);
        }
        wrongCodes = 0;
        Booking b = bookings.get(bookingId);
        if (b.status == BookingStatus.EXPIRED) return new ParcelExpired(b.parcel.id());
        b.status = BookingStatus.PICKED_UP;
        b.compartment.moveTo(CompartmentState.FREE);
        bookingByCodeHash.remove(b.codeHash);                       // a code works once
        return new DoorOpened(b.compartment.id(), b.parcel.id());
    }

    /** Carrier's daily visit: open every door holding an expired parcel and take them for return. */
    synchronized List<Parcel> collectExpired() {
        expireOverdue();
        List<Parcel> taken = new ArrayList<>();
        for (Booking b : bookings.values())
            if (b.status == BookingStatus.EXPIRED) {
                b.status = BookingStatus.RETURNED;
                b.compartment.moveTo(CompartmentState.FREE);
                bookingByCodeHash.remove(b.codeHash);
                taken.add(b.parcel);
            }
        return taken;
    }

    synchronized void markOutOfService(String compartmentId) {
        Compartment c = compartments.get(compartmentId);
```

```
        if (c == null) throw new LockerException("Unknown compartment " + compartmentId);
        c.moveTo(CompartmentState.OUT_OF_SERVICE);                // only a FREE door can be taken out
    }

    synchronized Map<Size, Integer> freeBySize() {
        expireOverdue();
        Map<Size, Integer> m = new EnumMap<>(Size.class);
        for (Size s : Size.values()) m.put(s, 0);
        compartments.values().stream().filter(Compartment::isFree).forEach(c -> m.merge(c.size(), 1,
Integer::sum));
        return m;
    }

    synchronized boolean canFit(Dimensions d) {
        expireOverdue();
        return compartments.values().stream().anyMatch(c -> c.isFree() && c.size().fits(d));
    }

    /** Called by a scheduler (e.g. every minute): applies deadlines and tells customers, outside the lock. */
    void sweep() {
        List<Parcel> expired;
        synchronized (this) { expireOverdue(); expired = List.copyOf(newlyExpired); newlyExpired.clear(); }
        for (Parcel p : expired) for (LockerListener l : listeners) l.parcelExpired(id, p);
    }

    synchronized BookingStatus status(String bookingId) { expireOverdue(); return booking(bookingId).status; }

    /** Lazy expiry: every operation first applies the deadlines that have passed, so correctness never waits for
a timer. */
    private void expireOverdue() {
        long t = now();
        for (Booking b : bookings.values()) {
            if (b.status == BookingStatus.RESERVED && t >= b.holdUntil) {
                b.status = BookingStatus.CANCELLED;
                b.compartment.moveTo(CompartmentState.FREE);
            } else if (b.status == BookingStatus.DEPOSITED && t >= b.pickupBy) {
                b.status = BookingStatus.EXPIRED;                  // door stays OCCUPIED until the carrier
collects
                newlyExpired.add(b.parcel);                        // notified by sweep(), outside the lock
            }
        }
    }

    private static boolean isActive(Booking b) {
        return b.status == BookingStatus.RESERVED || b.status == BookingStatus.DEPOSITED || b.status ==
BookingStatus.EXPIRED;
    }

    private Booking booking(String bookingId) {
        Booking b = bookings.get(bookingId);
        if (b == null) throw new LockerException("Unknown booking " + bookingId);
        return b;
    }

    private long now() { return clock.instant().getEpochSecond(); }

    /** Store only a salted hash of each code, so a leaked database does not reveal live codes. */
    private String hash(String code) {
        try {
            byte[] h = MessageDigest.getInstance("SHA-256").digest((id + ":" +
code).getBytes(StandardCharsets.UTF_8));
            return HexFormat.of().formatHex(h);
        } catch (NoSuchAlgorithmException e) { throw new IllegalStateException(e); }
    }
}
```

```java
// File: lld/ch14/locker/LockerNetwork.java
package lld.ch14.locker;

import java.util.Comparator;
import java.util.List;
import java.util.Optional;
import java.util.concurrent.CopyOnWriteArrayList;

/** All stations in a city. Used at checkout ("pick a locker near you") and when a station is full. */
final class LockerNetwork {
    private final List<LockerStation> stations = new CopyOnWriteArrayList<>();

    void add(LockerStation s) { stations.add(s); }

    /** Nearest station within maxKm that has a free door this parcel fits in right now. */
    Optional<LockerStation> nearestWithSpace(double x, double y, Dimensions parcel, double maxKm) {
        return stations.stream()
                .filter(s -> s.distanceTo(x, y) <= maxKm)
                .sorted(Comparator.comparingDouble(s -> s.distanceTo(x, y)))
                .filter(s -> s.canFit(parcel))
                .findFirst();
    }
}
```

```java
// File: lld/ch14/locker/Main.java
package lld.ch14.locker;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.ZoneOffset;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.SplittableRandom;

public class Main {
    /** A clock the demo can move forward; production code gets Clock.systemUTC(). */
    static final class TestClock extends Clock {
        private volatile Instant now = Instant.parse("2026-10-01T09:00:00Z");
        void advance(Duration d) { now = now.plus(d); }
        @Override public Instant instant() { return now; }
        @Override public java.time.ZoneId getZone() { return ZoneOffset.UTC; }
        @Override public Clock withZone(java.time.ZoneId z) { return this; }
    }

    /** Fake SMS gateway: remembers the last code sent to each phone so the "customer" can type it. */
    static final class SmsInbox implements LockerListener {
        final Map<String, String> lastCode = new ConcurrentHashMap<>();
        @Override public void parcelReady(String station, Parcel p, String code, long pickupBy) {
            lastCode.put(p.customerPhone(), code);
            System.out.printf("   SMS to %s: parcel %s is in locker %s, code %s, collect by %s%n",
                    p.customerPhone(), p.id(), station, code, Instant.ofEpochSecond(pickupBy));
        }
        @Override public void parcelExpired(String station, Parcel p) {
            System.out.printf("   SMS to %s: parcel %s was not collected and will be returned%n",
p.customerPhone(), p.id());
        }
    }

    public static void main(String[] args) throws Exception {
        TestClock clock = new TestClock();
        SmsInbox sms = new SmsInbox();
        LockerStation s1 = new LockerStation("MG01", 0, 0,
                LockerStation.layout(2, 2, 1), new SmallestFit(), clock, new SplittableRandom(7));
        s1.addListener(sms);

        System.out.println("== 1. Carrier reserves doors (smallest door that fits) ==");
        Parcel phoneCase = new Parcel("P1", "+91-98450-00001", new Dimensions(3, 10, 18));
        Parcel book = new Parcel("P2", "+91-98450-00002", new Dimensions(4, 16, 24));
        Parcel mug = new Parcel("P3", "+91-98450-00003", new Dimensions(9, 12, 12));
        Parcel shoes = new Parcel("P4", "+91-98450-00004", new Dimensions(14, 22, 34));
        Booking b1 = s1.reserve(phoneCase), b2 = s1.reserve(book), b3 = s1.reserve(mug), b4 = s1.reserve(shoes);
        for (Booking b : List.of(b1, b2, b3, b4))
            System.out.printf("   %s %-12s -> %s%n", b.parcel.id(), b.parcel.dims(), b.compartment);
        System.out.println("   free now: " + s1.freeBySize() + "  (the mug got a MEDIUM: both SMALL doors were
taken)");
        try { s1.reserve(phoneCase); } catch (LockerException e) { System.out.println("   error: " +
e.getMessage()); }
        try { s1.reserve(new Parcel("P9", "+91-98450-00009", new Dimensions(50, 50, 70))); }
        catch (LockerException e) { System.out.println("   error: " + e.getMessage()); }

        System.out.println("\n== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==");
        s1.deposit(b1.id);
        s1.deposit(b2.id);
        s1.deposit(b3.id);
        try { s1.deposit(b1.id); } catch (LockerException e) { System.out.println("   error: " + e.getMessage());
}
```

```
        System.out.println("\n== 3. Pickup ==");
        System.out.println("   typo 123456 -> " + s1.pickup("123456"));
        String code1 = sms.lastCode.get(phoneCase.customerPhone());
        System.out.println("   right code  -> " + s1.pickup(code1));
        System.out.println("   same code again -> " + s1.pickup(code1) + "  (a code works once)");

        System.out.println("\n== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==");
        for (int i = 1; i <= 4; i++) System.out.println("   guess 00000" + i + " -> " + s1.pickup("00000" + i));
        String code2 = sms.lastCode.get(book.customerPhone());
        System.out.println("   real owner during lockout -> " + s1.pickup(code2));
        clock.advance(Duration.ofMinutes(15));
        System.out.println("   15 min later -> " + s1.pickup(code2));

        System.out.println("\n== 5. Reservation hold runs out (carrier never came with the shoes) ==");
        clock.advance(Duration.ofHours(2));
        System.out.println("   booking " + b4.id + " is " + s1.status(b4.id) + ", free now: " + s1.freeBySize());

        System.out.println("\n== 6. Mug not collected within 3 days ==");
        clock.advance(Duration.ofDays(3));
        s1.sweep();
        System.out.println("   customer types the code -> " + s1.pickup(sms.lastCode.get(mug.customerPhone())));
        System.out.println("   carrier collects for return: " +
s1.collectExpired().stream().map(Parcel::id).toList()
                + ", free now: " + s1.freeBySize());

        System.out.println("\n== 7. Station full -> suggest the nearest station with space ==");
        LockerStation tiny = new LockerStation("KR02", 1.0, 0.5, LockerStation.layout(1, 0, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        LockerStation far = new LockerStation("WF03", 6.0, 2.0, LockerStation.layout(0, 3, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        LockerNetwork city = new LockerNetwork();
        city.add(tiny); city.add(far); city.add(s1);
        tiny.reserve(new Parcel("P5", "+91-98450-00005", new Dimensions(2, 10, 10)));
        Dimensions box = new Dimensions(8, 20, 30);
        System.out.println("   customer at (1.0, 0.4), box " + box + " -> "
                + city.nearestWithSpace(1.0, 0.4, box, 5).map(LockerStation::id).orElse("none within 5 km"));
        System.out.println("   same box, max 0.2 km -> "
                + city.nearestWithSpace(1.0, 0.4, box, 0.2).map(LockerStation::id).orElse("none within 0.2 km"));

        System.out.println("\n== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==");
        LockerStation ff = new LockerStation("F", 0, 0, List.of(Size.LARGE, Size.SMALL), new FirstFit(), clock,
new SplittableRandom(7));
        LockerStation sf = new LockerStation("S", 0, 0, List.of(Size.LARGE, Size.SMALL), new SmallestFit(), clock,
new SplittableRandom(7));
        for (LockerStation st : List.of(ff, sf)) {
            String small = st.reserve(new Parcel("A", "x", new Dimensions(2, 5, 5))).compartment.toString();
            String big;
            try { big = st.reserve(new Parcel("B", "y", new Dimensions(30, 35, 50))).compartment.toString(); }
            catch (LockerException e) { big = "REJECTED"; }
            System.out.println("   station " + st.id() + ": small parcel -> " + small + ", then large parcel -> "
+ big);
        }

        System.out.println("\n== 9. 50 carriers reserve at once on a 20-door station ==");
        LockerStation hub = new LockerStation("HUB", 0, 0, LockerStation.layout(0, 20, 0), new SmallestFit(),
clock, new SplittableRandom(7));
        Set<String> doorsGiven = ConcurrentHashMap.newKeySet();
        AtomicInteger ok = new AtomicInteger(), full = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < 50; i++) {
                int n = i;
                pool.submit(() -> {
                    go.await();
                    try {
                        doorsGiven.add(hub.reserve(new Parcel("C" + n, "z", new Dimensions(10, 20,
30))).compartment.id());
                        ok.incrementAndGet();
                    } catch (LockerException e) { full.incrementAndGet(); }
                    return null;
                });
```

```
            }
            go.countDown();
        }
        System.out.println("   reserved=" + ok + " rejected=" + full + " distinct doors=" + doorsGiven.size());
    }
}
```

How the code works, part by part

- `Size.fits` sorts the three sides of both the box and the door, then compares smallest-to-smallest. That handles a parcel lying on its side without trying all six rotations.
- `reserve` first applies overdue deadlines, rejects a parcel that already has a door, asks the selector for a free door, moves it to RESERVED and creates a `Booking` whose hold ends in 2 hours. It is `synchronized`, so "find a free door" and "mark it reserved" are one atomic step.
- `deposit` does its state change inside a `synchronized (this)` block, generates a 6-digit code (retrying on the rare clash with another live code at this station), stores only its hash, and calls the listeners **after** the block ends.
- `pickup` checks the lockout, looks the code up by hash, counts wrong attempts (5 in a row locks the keypad for 15 minutes), refuses an expired parcel, and on success frees the door and deletes the code so it cannot be used twice.
- `expireOverdue` runs at the top of every public method: RESERVED past its hold → CANCELLED (door freed); DEPOSITED past its window → EXPIRED (door stays OCCUPIED until the courier collects). `sweep()` is what a scheduler calls every minute; it runs the same check and sends the "not collected" messages outside the lock.
- `LockerNetwork.nearestWithSpace` sorts stations by distance, keeps those within the radius and returns the first that can fit the parcel right now.

Output:

```
== 1. Carrier reserves doors (smallest door that fits) ==
   P1 3x10x18cm    -> MG01-01(SMALL)
   P2 4x16x24cm    -> MG01-02(SMALL)
   P3 9x12x12cm    -> MG01-03(MEDIUM)
   P4 14x22x34cm   -> MG01-04(MEDIUM)
   free now: {SMALL=0, MEDIUM=0, LARGE=1}  (the mug got a MEDIUM: both SMALL doors were taken)
   error: Parcel P1 already has a locker
   error: No free compartment fits 50x50x70cm at MG01

== 2. Deposit: the code goes to the customer by SMS, never to the carrier ==
   SMS to +91-98450-00001: parcel P1 is in locker MG01, code 694696, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00002: parcel P2 is in locker MG01, code 607553, collect by 2026-10-04T09:00:00Z
   SMS to +91-98450-00003: parcel P3 is in locker MG01, code 385053, collect by 2026-10-04T09:00:00Z
   error: Booking MG01-B1 is DEPOSITED

== 3. Pickup ==
   typo 123456 -> WrongCode[attemptsLeft=4]
   right code  -> DoorOpened[compartmentId=MG01-01, parcelId=P1]
   same code again -> WrongCode[attemptsLeft=4]  (a code works once)

== 4. Someone guesses codes: keypad locks after 5 wrong tries in a row ==
   guess 000001 -> WrongCode[attemptsLeft=3]
   guess 000002 -> WrongCode[attemptsLeft=2]
   guess 000003 -> WrongCode[attemptsLeft=1]
   guess 000004 -> KeypadLocked[secondsLeft=900]
   real owner during lockout -> KeypadLocked[secondsLeft=900]
   15 min later -> DoorOpened[compartmentId=MG01-02, parcelId=P2]

== 5. Reservation hold runs out (carrier never came with the shoes) ==
   booking MG01-B4 is CANCELLED, free now: {SMALL=2, MEDIUM=1, LARGE=1}

== 6. Mug not collected within 3 days ==
   SMS to +91-98450-00003: parcel P3 was not collected and will be returned
   customer types the code -> ParcelExpired[parcelId=P3]
   carrier collects for return: [P3], free now: {SMALL=2, MEDIUM=2, LARGE=1}

== 7. Station full -> suggest the nearest station with space ==
   customer at (1.0, 0.4), box 8x20x30cm -> MG01
   same box, max 0.2 km -> none within 0.2 km

== 8. FirstFit vs SmallestFit: doors 01 = LARGE, 02 = SMALL ==
   station F: small parcel -> F-01(LARGE), then large parcel -> REJECTED
   station S: small parcel -> S-02(SMALL), then large parcel -> S-01(LARGE)

== 9. 50 carriers reserve at once on a 20-door station ==
   reserved=20 rejected=30 distinct doors=20
```

Reading the output: the mug (9 × 12 × 12 cm) fits a small door but both were taken, so it was upgraded to a medium one; a second reservation for the same parcel and an oversized box are refused. Codes went only to the customers' phones. The keypad counted the earlier typo, so the fourth guess in section 4 hit the limit of five and locked it — even the real owner had to wait 15 minutes. The shoes' reservation lapsed after 2 hours and the door came back. After 3 days the mug expired, the customer's code now says so, and the courier's visit freed the door. `FirstFit` put a tiny parcel in the only large door and then had to refuse a large parcel; `SmallestFit` served both. Fifty concurrent reservations on a 20-door station produced exactly 20 bookings on 20 different doors.

Concurrency & edge cases

Situation	Handling
Two carriers reserve the last medium door at once	`reserve` is `synchronized`: choose + mark RESERVED is one step; the loser gets "no free compartment" (or a larger door)
Courier arrives after the hold ran out	`deposit` sees CANCELLED and refuses; the carrier system reserves again (possibly at another station)
Same parcel reserved twice (retry after a timeout)	Rejected while it has an active booking — make the carrier's reserve call idempotent (repeating it has the same effect as calling it once) by parcel id
Two live parcels get the same random code	Generation retries until the code is unique among live codes at this station
Brute-force guessing	10^6 possible codes, 5 wrong tries then a 15-minute lockout per station; codes also expire
Scheduler down for an hour	Lazy expiry still frees doors on the next operation; only the "expired" SMS is late
Slow SMS gateway	Listeners run outside the lock; in production they hand off to a queue (a buffer that a separate worker drains)
`canFit` says yes but `reserve` then fails (someone else took the door)	Expected: `nearestWithSpace` is a hint; the caller reserves and, on failure, tries the next station
Door jammed / broken	Technician marks a **free** door OUT_OF_SERVICE; an occupied door must be emptied first (`moveTo` enforces it)
Power cut or network loss at the station	Real lockers keep a local copy of live code hashes and sync events later; the design already keeps the station self-contained

Extensions & follow-up questions

1. **"Returns (customer drops off)?"** — A `DropOffBooking`: the customer gets a drop-off code, the door opens empty, the courier collects later. Same states with the roles swapped.
2. **"Let the customer extend the pickup window?"** — A method that moves `pickupBy` once, only while DEPOSITED; a policy object can decide how many extensions are allowed.
3. **"Cash on delivery?"** — Payment at the station screen before the door opens: a `PaymentStep` inserted into `pickup` (Chain of Responsibility or a small state machine per session).
4. **"Fairness when the station is nearly full?"** — Keep a few large doors for large parcels only (a `ReservedCapacitySelector` that refuses to upgrade small parcels when fewer than k large doors are free).
5. **"Scale to thousands of stations?"** — Stations are independent, so shard by station id (sharding = splitting the data across several servers, each owning a slice; see System Design Fundamentals); the network search uses a geo index (an index for fast "what is near this point?" queries: a geohash turns a map cell into a short string, an R-tree is a tree of nested rectangles) instead of sorting every station.
6. **"How would you store this in a database?"** — `compartment(id, station_id, size, state)`, `booking(id, parcel_id, compartment_id, status, hold_until, pickup_by, code_hash)`; reserve with `UPDATE compartment SET state='RESERVED' WHERE id=? AND state='FREE'` and check that one row changed.

Real-world example

Amazon Hub Locker sends the customer a pickup code by email or app and gives a limited number of days to collect before the parcel goes back — the `DEPOSITED → EXPIRED → RETURNED` path. InPost's Paczkomat lockers in Poland let couriers load many parcels in one visit and customers open a door with a code or a phone app; the network is dense enough that "nearest locker with space" matters. In both, the door is chosen by the system, not the courier, which is our `CompartmentSelector`.

In the interview

Plan for 40 minutes: 5 min questions (sizes, holds, codes, expiry) → 5 min entities → 8 min the two state machines → 15 min code `reserve`, `deposit`, `pickup` with the lock and the listener call outside it → 7 min security, concurrency and follow-ups.

> **30-second answer:** "A station has doors of a few sizes. A carrier reserves one ahead of time; a Strategy picks the smallest free door that fits, and the choose-and-mark step runs under the station's lock, so two carriers never get the same door. On deposit I generate a random single-use code, store only its hash, and notify the customer through an observer after releasing the lock. Pickup matches the hash, frees the door and deletes the code; five wrong codes lock the keypad. Holds and pickup windows are checked lazily on every call, so doors come back even if the scheduler is late, and a network object suggests the nearest station with space."

Follow-ups with answers

- *"Why not let the courier choose any empty door?"* — Couriers would use big doors for small parcels; the system also needs to know the door before the courier arrives.
- *"Why store only a hash of the code?"* — Anyone who reads the database or logs could otherwise open doors.
- *"Why a separate Booking instead of putting the parcel on the Compartment?"* — The door is reused; the booking keeps deadlines and history per parcel.

Self-check

1. **Q:** How does `Size.fits` handle a box lying on its side? **A:** It sorts the three sides of both box and door and compares them in order.
2. **Q:** What happens to an expired parcel's door? **A:** It stays OCCUPIED until the courier runs `collectExpired`, which frees it.
3. **Q:** Why is the listener called after the `synchronized` block in `deposit`? **A:** A slow or failing SMS call must not hold the station lock.
4. **Q:** What frees a door if the scheduler never runs? **A:** Lazy expiry at the start of every public method.
5. **Q:** Which pattern lets you switch from smallest-fit to first-fit? **A:** Strategy (`CompartmentSelector`).
6. **Q:** Why is one lock per station enough here but not in a parking lot of 5,000 spots with 20 gates? **A:** A station has tens of doors and few concurrent callers; the lock is held for microseconds.

TL;DR

- Doors of fixed sizes; a **Strategy** picks the smallest free door that fits; choose-and-mark is **atomic** under the station lock.
- Two small **state machines**: door (FREE/RESERVED/OCCUPIED/OUT_OF_SERVICE) and booking (RESERVED → DEPOSITED → PICKED_UP / EXPIRED → RETURNED / CANCELLED).
- Codes are **random, single-use, hashed**, sent only to the customer via an **Observer** outside the lock; lockout after 5 wrong tries.
- Deadlines are enforced **lazily** on every call; a sweep only sends messages. Inject clock and randomness.

Chapter summary

Problem	Core difficulty	Key data structure	Key patterns	Concurrency tool
Parking lot	Allocate spots without double booking	Concurrent sorted set of free spots per type	Strategy, Factory, Observer	`pollFirst` + CAS, per-ticket lock, `putIfAbsent`
Vending machine	Legal order of actions; exact change	`EnumMap` of coin counts; DP table	State	`synchronized` machine
Coffee machine	All-or-nothing multi-ingredient deduction	`EnumMap` levels, recipes as data	Observer, sealed results	One `ReentrantLock`; notify outside the lock
ATM	Money correctness and authentication	Total vs available balance; note chain	State, Chain of Responsibility, Command	Per-account lock; ordered locking for transfers
Elevator	Scheduling and dispatching	Two `TreeSet`s (up/down stops)	Strategy (+ modes)	Per-car lock, synchronized dispatcher
Traffic signal	Timed transitions with a safety invariant	Phases as data	State, Strategy, Observer	Synchronized tick; scheduled executor
Shipping locker	Best-fit door, holds and pickup windows, secret codes	Doors per station; bookings; code-hash index	Strategy, Observer, enum state machines, sealed results	One lock per station; lazy expiry; notify outside the lock

Chapter self-check

1. **Q:** Which two problems use the State pattern with *stateless* states, and which uses states *with data*? **A:** Vending machine and ATM share stateless singletons; the traffic signal's states hold countdowns, so a new instance is created per transition.
2. **Q:** Name the check-then-act race in each of parking, coffee and ATM, and its fix. **A:** Two gates take one spot (atomic `pollFirst`/CAS); two orders take the last milk (one lock around check+deduct); two ATMs overdraw one account (check+debit under the account lock).
3. **Q:** Where did we reject a Singleton, and why? **A:** Parking lot, coffee machine, traffic controller: a Singleton is a hidden global, blocks testing several instances, and real deployments have many (lots, machines, intersections).
4. **Q:** Where does greedy fail and what replaces it? **A:** Change-making with limited coins (vending) and note selection with limited cassettes (ATM); bounded coin-change dynamic programming.
5. **Q:** Which designs are tick-driven, and why is that useful? **A:** Elevator and traffic signal: deterministic, clock-free tests; a real timer just calls `tick()`/`step()`.
6. **Q:** Which case enforces deadlines *lazily*, and why? **A:** The shipping locker: every call first expires overdue holds and parcels, so doors come back even if the scheduler is late; the scheduled sweep only sends messages.

Chapter TL;DR

- Find the **one hard method** in each problem (spot allocation, change-making, ingredient deduction, withdrawal, next stop, next phase) and spend your time there.
- **State** for machines whose buttons mean different things over time; **Strategy** for rules that change (pricing, dispatch, timing); **Observer** for boards, alerts and monitors; **Chain of Responsibility** and **Command** for the ATM.
- Make every **check-then-act** atomic, lock at the **smallest** useful scope, lock in a **fixed order**, and call listeners **outside** locks.
- Prefer injected instances over Singletons, integer cents over doubles, and tick-driven logic over sleeps.
- For anything with a **deadline** (holds, pickup windows), check it on every call and keep secrets (pickup codes) **hashed and single-use**.

Chapter 15 — Case Studies: Booking & Inventory (Part 1)

> **Where this fits.** Chapter 14 designed small, self-contained objects. This chapter is the first of two on **booking systems**: software that sells or lends a limited thing (a book copy, a cinema seat, a concert place, a hotel night) to many people who all want it at once. Chapter 16 continues with more booking problems. Everything here uses the case-study template from Chapter 1 and the concurrency tools from Chapters 12–13.

Why booking problems are interview favourites

A booking problem looks like simple CRUD ("create, read, update, delete") until two people want the same thing. Then five hard questions appear, and every problem in this chapter is a different mix of them:

Theme	The question the interviewer is really asking	Where you will see it
Inventory	What exactly is the thing being sold or lent? A title or a copy? A seat or a seat *for one show*? A room or a room *on one night*?	`Book` vs `BookItem` (15.1), `HallSeat` vs `ShowSeat` (15.2), GA (general admission) count vs numbered seat (15.3), `Room` + booked nights (15.4)
Holds / locks	How do you keep something for a customer while they pay, without keeping it forever?	Seat lock with a TTL (time-to-live) (15.2), CAS holds + reaper (15.3)
Double-booking prevention	What stops two threads (or two servers) from both succeeding?	`synchronized` per item (15.1, 15.4), one monitor per show (15.2), lock-free compare-and-set (15.3)
Payments	What happens if payment fails? If it succeeds but the hold was lost meanwhile?	Release on decline, refund as **compensation** (15.2, 15.3)
Cancellations	Who gets the thing next, and how much money comes back?	Waitlists (15.1, 15.3), refund policies (15.2, 15.4)

Three words you will meet in every section:

- **Race condition** — two threads read the same data, both decide "it is free", and both act. One result silently overwrites the other.
- **Check-then-act** — the pattern "if free, then take it". It is only safe if the check and the act happen as one indivisible (atomic) step.
- **Hold** (also called a *lock*, *soft lock* or *reservation*) — a temporary claim with an expiry time, so an abandoned checkout does not block the item forever.

> `Key terms — the other words in the table above.` A **thread** is one independent line of execution inside a program; a server runs many at once, and they can all touch the same objects. A **lock** lets only one thread at a time run a piece of code; Java's `synchronized` keyword uses the built-in lock that every object carries, called its **monitor**. **Compare-and-set (CAS)** is a single atomic step that says "change this value to *new*, but only if it still equals *expected*"; code built on CAS instead of locks is called **lock-free**, because no thread ever waits for another. **Contention** means many threads competing for the same item or lock at the same moment. A **reaper** is a background job that sweeps away expired holds. **Compensation** is an action that undoes an earlier step after the fact — for money, a refund.

What you will learn

#	Problem	The one idea that carries the design
15.1	Library Management System	A *title* (`Book`) is not a *copy* (`BookItem`). Limits live with the member; the waitlist lives with the title.
15.2	Movie Ticket Booking (BookMyShow)	You book a `ShowSeat`, not a physical seat. Hold with TTL → pay → commit; refund if the hold was lost.
15.3	Concert Ticket Booking	Extreme contention: lock-free CAS, a counter for general admission, a fair virtual queue, per-user limits, a waitlist.
15.4	Hotel Management System	A status field is not availability. Each room remembers which nights are already sold.

How to run every example in this chapter

```
cd chapters/15-case-booking-1/code
javac --release 21 -d /tmp/lld_15 $(find . -name '*.java')
java -cp /tmp/lld_15 lld.ch15.library.Main        # or moviebooking, concert, hotel
```

All four `Main` classes compile with Java 21 (standard library only) and run. The output under each implementation is the real output of that program. In the concurrency demos, *which* thread wins a race changes from run to run; the *number* of winners never does.

> **Code-style note.** To keep each design readable in one place, a file sometimes holds several small package-private types (types declared without `public`, visible only inside their own package) (for example `Model.java` holds the enums and records of the library). That is legal Java. In a production codebase you would usually give each public type its own file. (If you compile with `-Xlint:all` — the switch that turns on every optional compiler warning — javac, the Java compiler, prints `[auxiliaryclass]` warnings for exactly this: a package-private type declared in one file and used from another. They are style warnings, not errors.)

> **Money.** Every amount (prices, fines, deposits, refunds) is a `java.math.BigDecimal` (Java's exact decimal-number type) built from a string such as `new BigDecimal("0.50")`, never a `double`: binary floating point cannot store 0.10 exactly, so sums drift. Comparisons use `compareTo`/`signum()`, not `equals` (which also compares the number of decimal places), and every rounding step names its `RoundingMode` (the rounding rule, for example half-up).

> **A time trick used in every demo.** Holds expire and fines grow with time. Instead of `Thread.sleep` (pausing for real time), every service takes a `java.time.Clock`. The demos pass a tiny `SimClock` (a `Clock` subclass whose `advance(Duration)` moves time forward). This is standard practice for testable time-dependent code: production passes `Clock.systemUTC()`, tests pass a controllable clock.

15.1 Design a Library Management System



Figure — Cover image for the library case study (course diagram)

Problem & clarifying questions

A **Library Management System** (LMS) is the software a library uses for its daily work: it tracks every book and every physical copy, who borrowed which copy and when it is due, each member's account and card, reservations, and fines. New stock is entered into it, and every loan is recorded with a due date.

Always clarify scope first. Good questions, with the answers we will design for:

Question	Answer we assume
Can a title have several physical copies?	Yes. Each copy is a **book item** with its own barcode and rack position.
How many books can one member hold?	At most **5** at a time.
How long can a member keep a book?	At most **10 days**, then a fine per day late.
Can a member reserve a book that is out?	Yes — and be told when a copy comes back.
How are people and books identified at the desk?	Every book item and every library card has a **unique barcode**; the system scans both.
Do we collect the fine money?	We *calculate and record* fines. Taking payment is out of scope (we say where it would go).
E-books, inter-library loans, lost-copy replacement?	Out of scope. Name them as extensions.

Requirements

Functional

1. Search the catalog by **title, author, subject category and publication date**.
2. Each book has a unique identification number (ISBN, the International Standard Book Number printed on every published book) and details; each copy has a **rack number** to find it physically.
3. A title can have **many copies** (book items); members check out and reserve copies.
4. Answer **"who took this copy?"** and **"which books does this member have?"**.

5. Limit: **max 5 books** per member.
6. Limit: **max 10 days** per loan.
7. **Fines** for books returned after the due date.
8. **Reserve** books that are not currently available.
9. **Notify** a member when a reserved book becomes available, and when a book is overdue.
10. **Barcodes** on every book item and every library card.

Non-functional

- **Correctness under concurrency:** two members at two desks can never both borrow the last copy; a member can never exceed 5 books even if two librarians scan for them at once.
- **Fast search:** searches are by far the most common operation, so they should be index lookups, not scans (an **index** is a lookup structure, such as a map from author name to books, that jumps straight to the matches instead of checking every book).
- **Extensibility:** fine rules and notification channels change; they should be swappable without editing the lending code.

Use cases

Three actors:

- **Librarian** — adds, edits and removes books, book items and members; can also issue, reserve and return items on a member's behalf.
- **Member** — searches the catalog; checks out, reserves, renews and returns books.
- **System** — sends notifications (overdue books, a reserved book is back, cancelled reservations).

Top use cases: add/remove/edit book (and book item); search catalog; register new account / cancel membership; **check out** a book; **reserve** a book that is out; **renew** (borrow the same copy again); **return** a book.

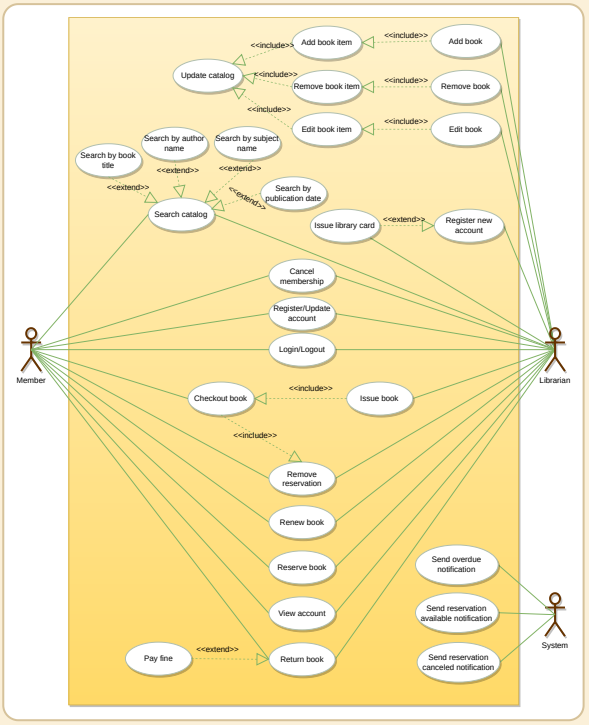


Figure — Library use cases: Member and Librarian share most of them (log in, search, check out, renew, reserve, return); the four "search by" use cases extend "Search catalog"; add/remove/edit book and book item feed "Update catalog"; "Checkout book" includes "Remove reservation"; the librarian's "Issue book" includes "Checkout book"; "Issue library card" is attached to "Register new account" (course diagram)

In the figure above, notice the two relationship kinds. `<<include>>` means a use case *always* performs another one as part of it — checking out a book always clears the member's reservation for it, if they had one. `<<extend>>` means an *optional variation* of a base use case — searching by author is one particular way a catalog search can go.

Core entities

Class	Responsibility
`Book` (record)	The **title**: ISBN, title, subject, publisher, publication date, authors. Immutable.
`BookItem`	One **physical copy**: barcode, format, rack, price, status (`AVAILABLE`, `RESERVED`, `LOANED`, `LOST`), and who it is held for.
`Author`, `Rack` (records)	An author's name; a rack's number and location.
`Account` (sealed, abstract) → `Member`, `Librarian`	People. An account **owns** its `LibraryCard` (composition). A member carries the loan count.
`LibraryCard` (record)	Card number, barcode, issue date.
`BookLending`	One loan: copy barcode, member id, creation date, **due date**, return date.
`Fine` (record)	Amount owed for one late return.
`Search` (interface) / `Catalog`	Four search methods; the catalog keeps one index per search and knows every copy of every title.
`LendingService`	The rules: check out, return, renew, reserve, overdue notices.
`FinePolicy` / `PerDayFine`	How much a late return costs (Strategy: a swappable rule behind an interface).
`Notifier` / `ConsoleNotifier`	Where messages go (email, SMS… here: console).
`Library`	Facade (one simple front object for the whole subsystem): owns the catalog and the lending service; only librarians may add stock.

In the table, a **record** is a short Java class that only carries values; **immutable** means it cannot change after it is created; an **enum** is a type with a fixed list of named constants; an **abstract** class cannot be created directly, only its subclasses can; and a **sealed** type lists exactly which subclasses it allows, so the compiler knows them all.

The course lists the same core classes: **Library** (name, address), **Book**, **BookItem**, **Account** (member or librarian), **LibraryCard**, **BookReservation**, **BookLending**, **Catalog** (searching by title, author, subject, publish date), **Fine**, **Author**, **Rack** (number + location identifier) and **Notification**. We keep every one of them; the only renames are `BookReservation` → a per-title waitlist inside `LendingService` (explained under *Design decisions*) and `Notification` → the `Notifier` interface.

Class diagram

The course builds its diagram in five stages, adding a few classes each time and greying out earlier ones. Walking through the stages is a good way to *present* a design in an interview, because every box arrives with a reason.

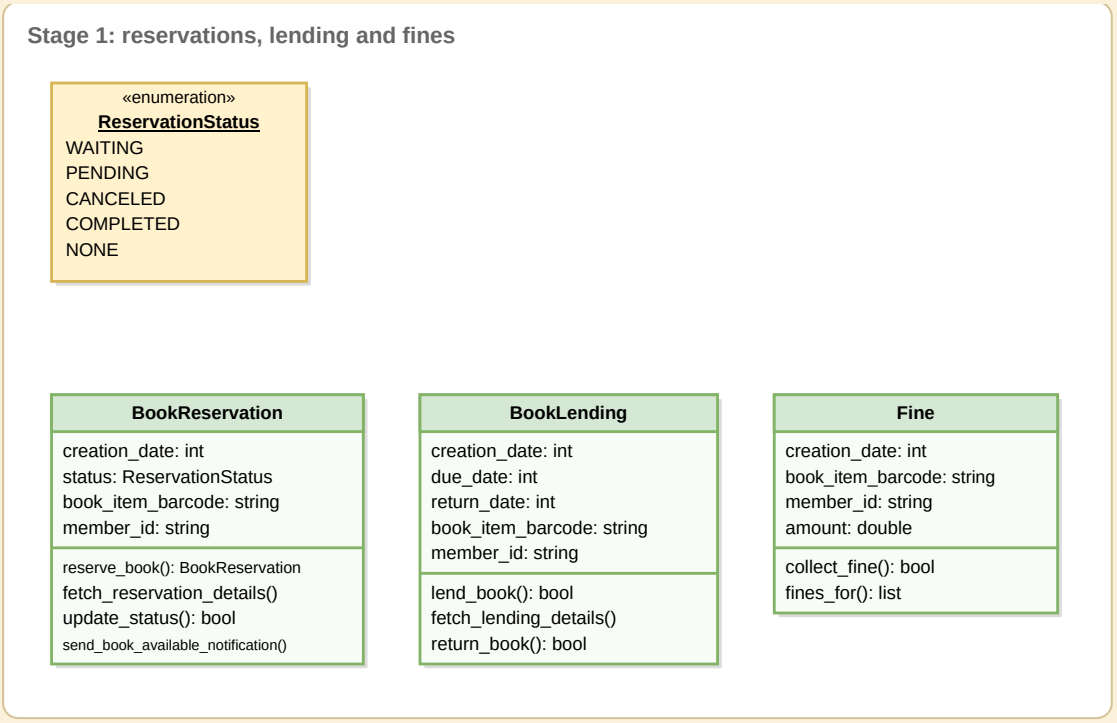


Figure — Stage 1: the three "what happened" records — `BookReservation`, `BookLending` and `Fine` — each keeping a copy's barcode and a member id (course diagram)

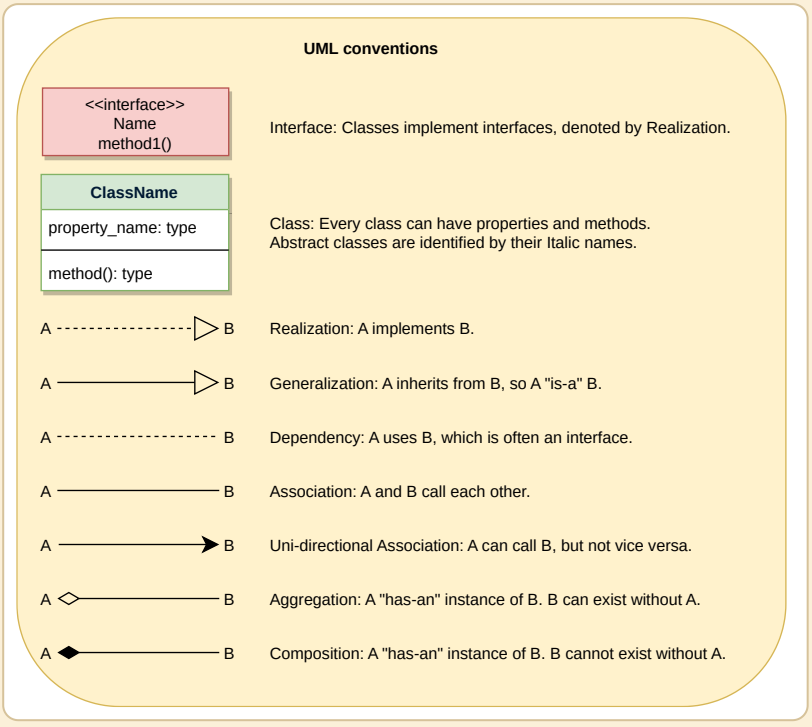


Figure — How to read a class box (name, `property: type`, `method(): type`) and the arrow styles used in these diagrams (course diagram)

Stage 1 answers requirements 4, 7 and 8, which are all about *remembering what happened*. Because each record stores the copy's barcode and the member's id, both halves of "who took this book / what does this member have" are answered by scanning these records.

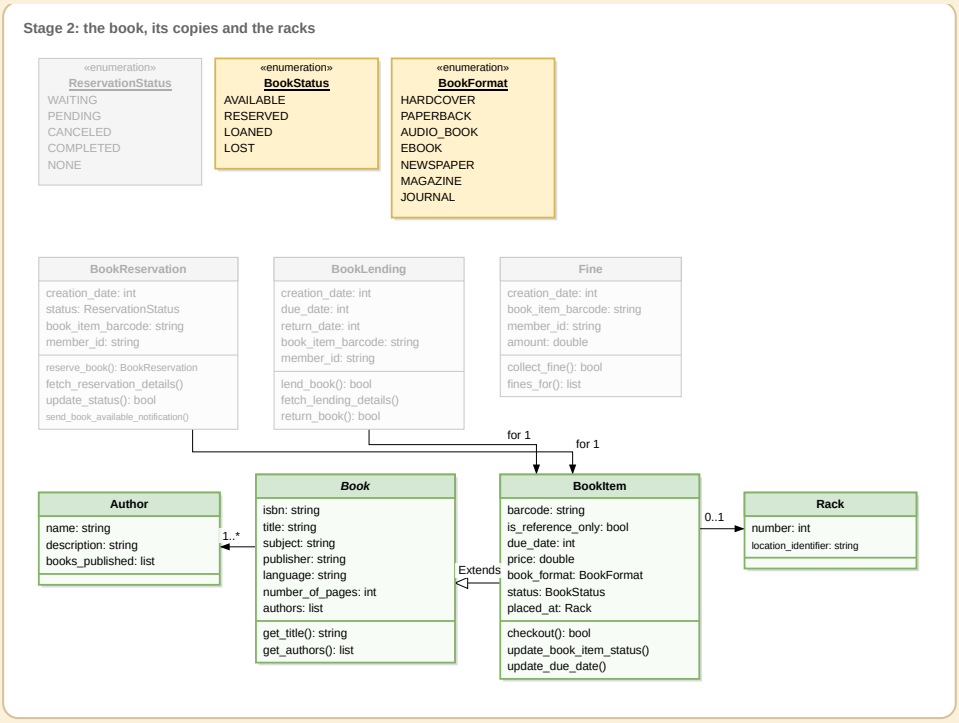


Figure — Stage 2: `Book` (the title) and `BookItem` (one copy with barcode, price and rack), plus `Author` and `Rack`; every reservation and every loan refers to one single copy (course diagram)

Stage 2 is the split that carries the whole design. In the course's diagram `BookItem` *inherits* from `Book`: the reading is that a copy carries all of the title's details and adds its own barcode, price and rack position. In our Java we use **`composition`** instead (`BookItem` *has a* `Book`) — see *Design decisions* for why.

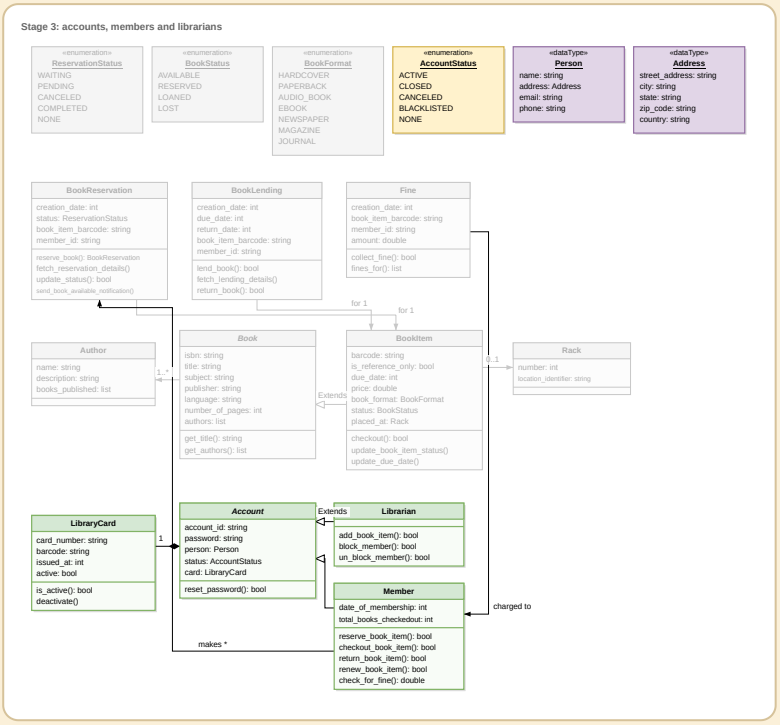


Figure — Stage 3: abstract `Account` with `Member` and `Librarian`; the account owns its `LibraryCard` (filled diamond = composition); a fine is charged to a member; a member makes reservations (course diagram)

Requirements 5 and 6 restrict *what one member may do*, so they belong to the people classes. `Account` is an **`abstract class`**: you can never create a plain `Account`, only a `Member` or a `Librarian`. The **`filled diamond`** is composition — the card has no life of its own, so closing the account destroys the card too.

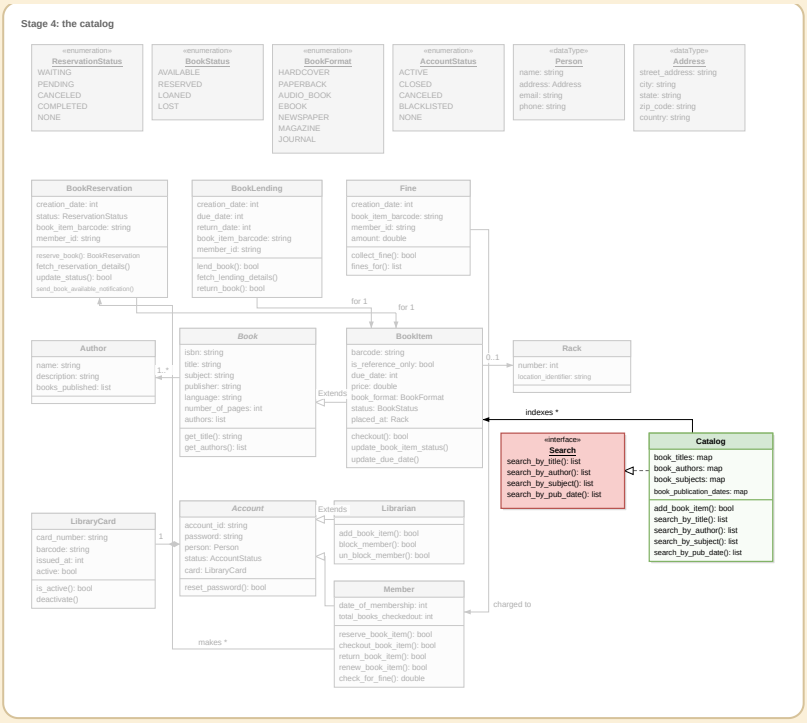


Figure — Stage 4: the `Search` interface (four methods) and `Catalog` implementing it (dashed line, hollow head), indexing book items without owning them (course diagram)

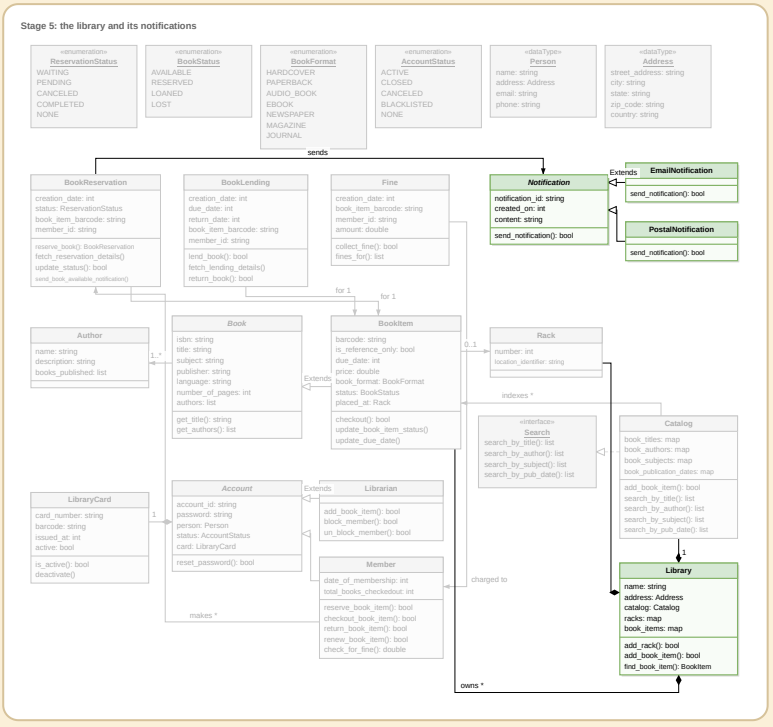


Figure — Stage 5: `Library` is the owner of the catalog, every rack and every book item; `Notification` is abstract with one subclass per delivery channel; a reservation triggers a notification when its copy comes back (course diagram)

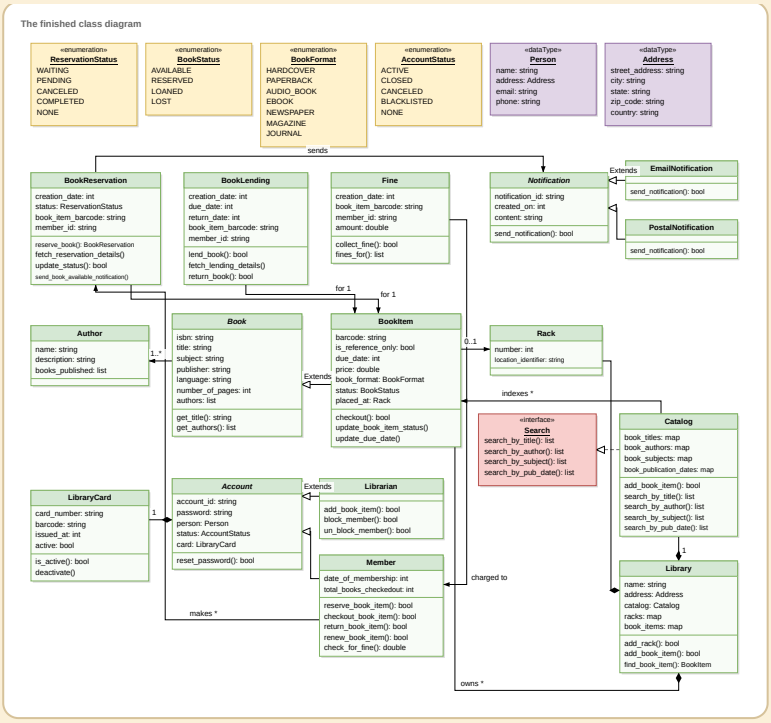
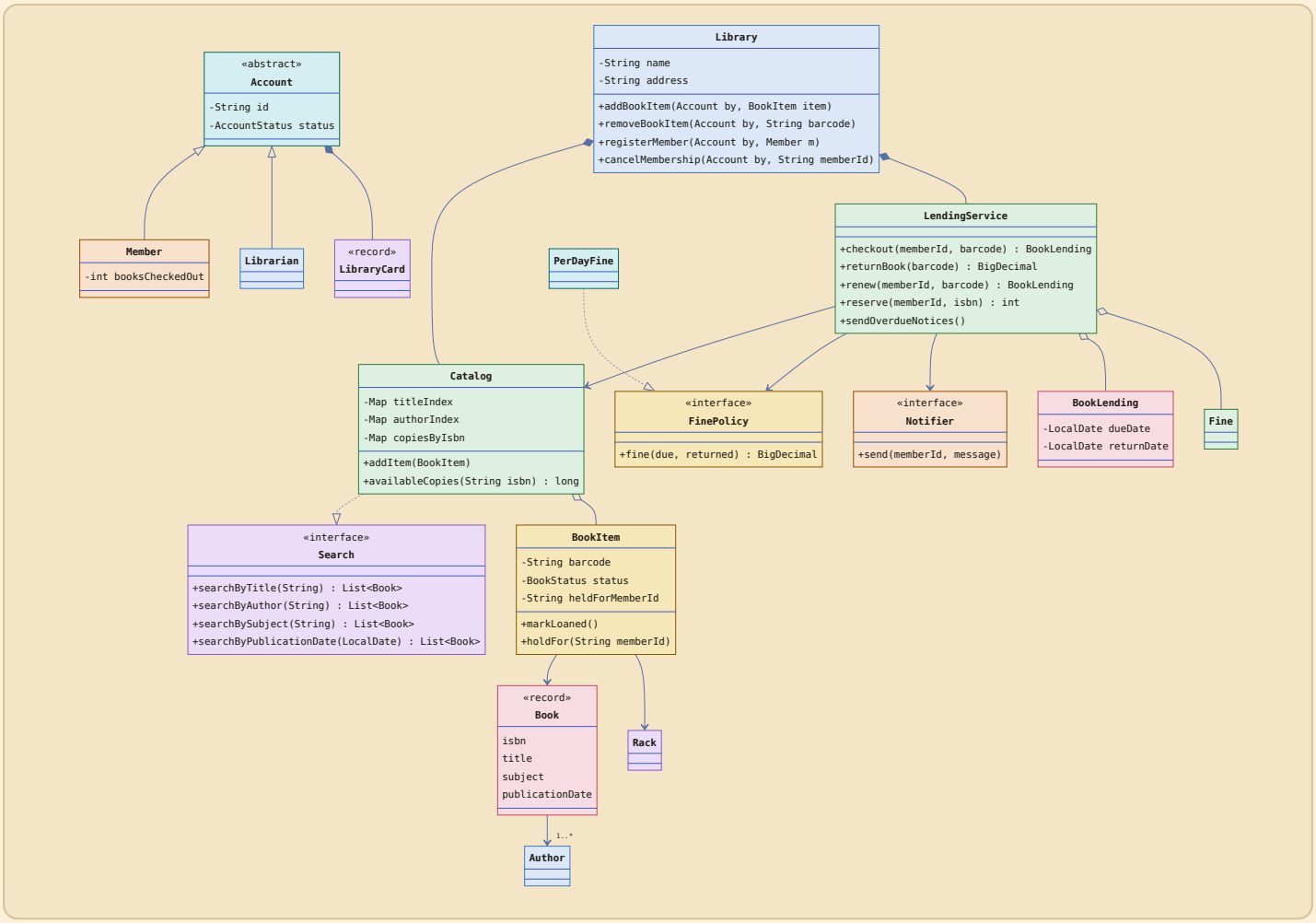


Figure — The finished library class diagram: every class from the five stages in full colour (course diagram)

Here is the class diagram of **our Java code** (the one you would draw on the whiteboard):



Key flows

The course gives three activity diagrams — check out, return and renew.

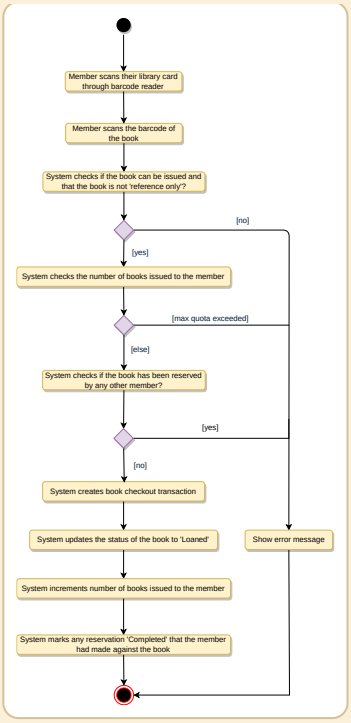


Figure — Check out: scan the book, check it can be issued (not reserved by someone else), check the member's quota, create the checkout transaction, mark the copy "Loaned", increment the member's count, complete any reservation this member had (course diagram)

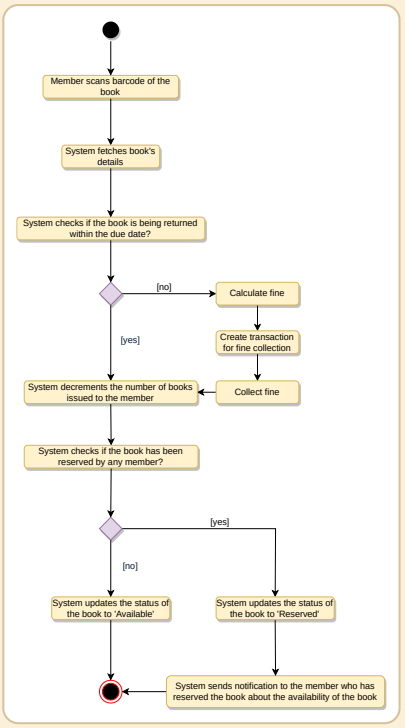


Figure — Return: scan the book, check whether it is late (if so, calculate and collect the fine), decrement the member's count, then check whether the book is reserved — if yes, notify the member who reserved it and mark it reserved, otherwise make it available (course diagram)

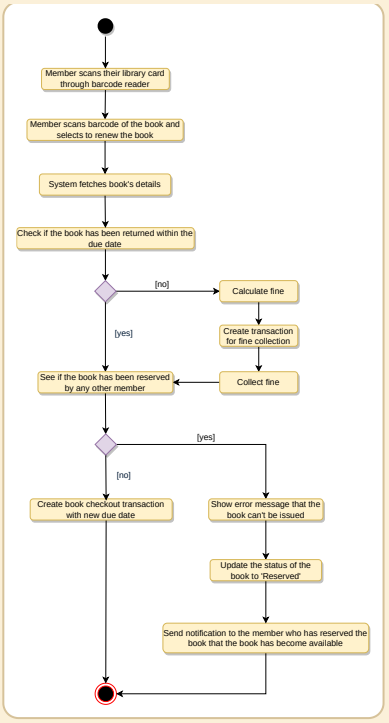
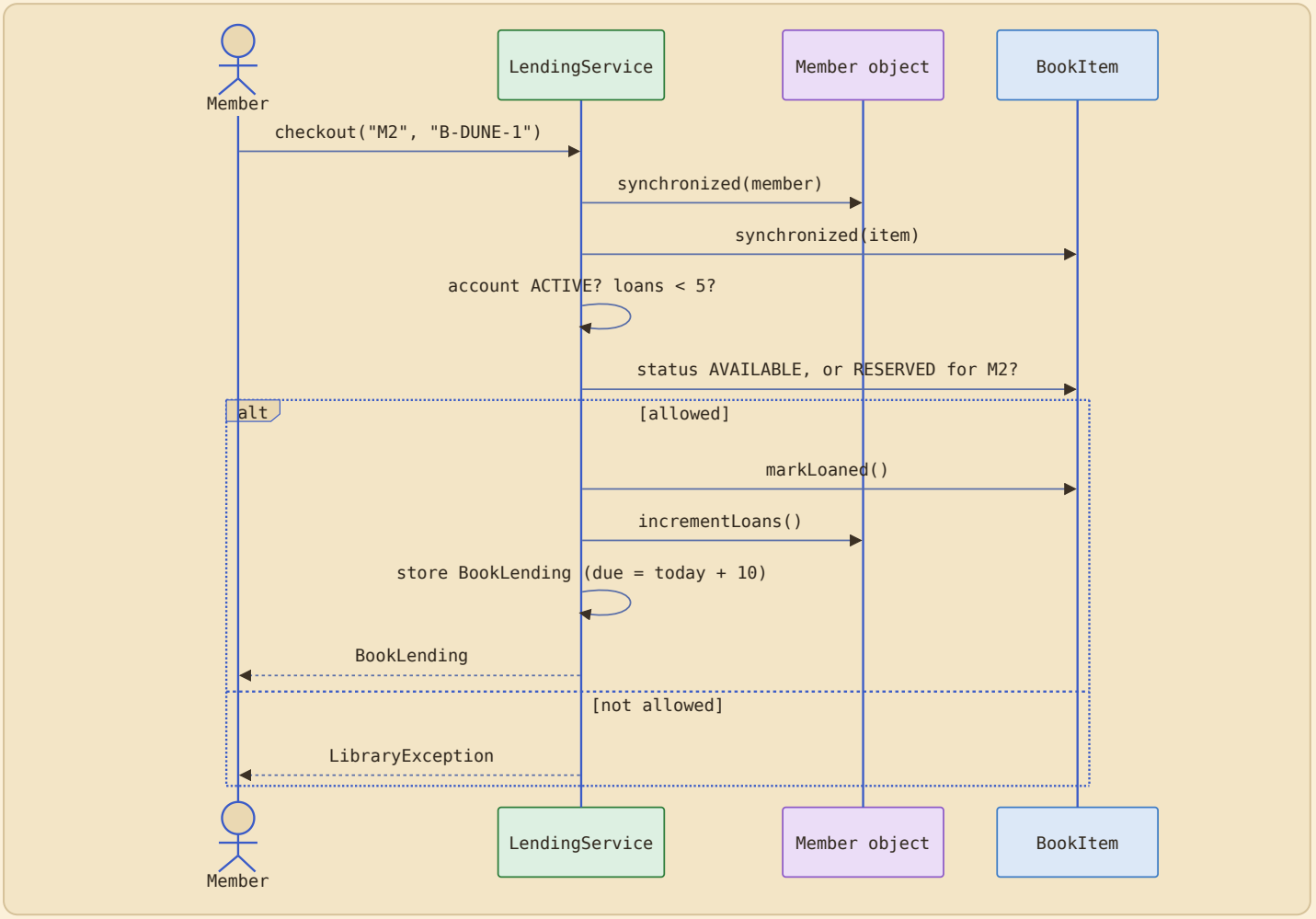
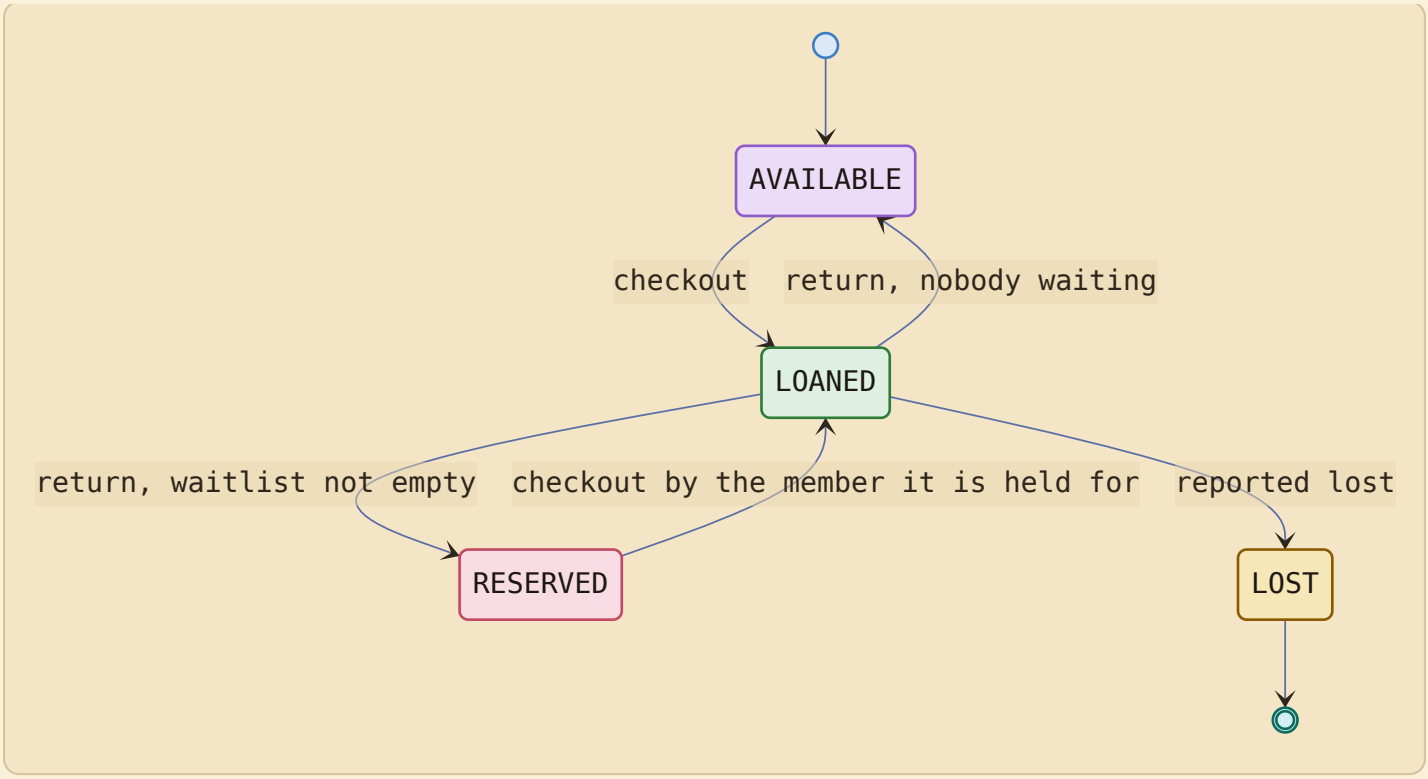


Figure — Renew: scan card and book, collect a fine if it is already late, check whether another member reserved it (if so, refuse and show an error), otherwise create a new checkout transaction with a new due date (course diagram)

The same check-out flow as a sequence diagram (messages between objects in time order, top to bottom) of our code, including the locks:



A book copy's life is a small **state machine** (a fixed set of states plus the moves allowed between them):


AVAILABLE
checkout
return, nobody waiting
LOANED
return, waitlist not empty
checkout by the member it is held for
reported lost
RESERVED
LOST

Design decisions & patterns used

Decision	Why	Alternative rejected
`Book` vs `BookItem` split	A member borrows a *copy*; search works on *titles*. Every later class depends on this split, so say it early.	One `Book` class with a `copies` counter — cannot say *which* copy is late, on which rack, or who has it.
Composition (`BookItem` has a `Book`) instead of the course's inheritance	Many copies share one immutable title; with inheritance each copy would duplicate every title field, and changing a title's subject would mean editing every copy. "A copy *is a* title" also fails a strict substitution test (the Liskov Substitution Principle: a subclass must work wherever its parent is expected): a title has no barcode or status.	`BookItem extends Book` (the course diagram) — works for a small model, and is worth mentioning as the other valid reading.
Lending rules in `LendingService`, not on `Member`	The course itself flags that a `Member` doing checkout/return/renew has *two reasons to change* (a person, and the lending rules) — a Single Responsibility violation. Moving the flows to a service keeps `Member` a plain person.	Methods on `Member` (course version) — fine for a first sketch.
Reservation = FIFO (first in, first out) waitlist per *title*	A member wants "a copy of Dune", not "copy #3". When *any* copy is returned, the first person in line gets it.	Reservation per copy (course's `BookReservation` keyed by barcode) — a member could wait on copy #3 while copy #1 sits on the shelf.
Strategy for fines (`FinePolicy`)	Fine rules change (caps, student discounts, holidays). The service calls `fine(due, returned)` and does not care how.	`if/else` inside `returnBook`.
Observer-style `Notifier`	(Observer: the sender announces events to whoever subscribed.) Lending code announces events; delivery (email, SMS, push) is someone else's job. The course's abstract `Notification` with one subclass per channel is the same idea.	Sending email directly from `LendingService`.
Facade (`Library`)	Desk software calls one object; role checks (only librarians add stock) live there.	Exposing the catalog and service directly to the UI.
Catalog keeps one index per search	Every lookup is a map read. The course names the cost: each new way of searching adds another map, whereas a single index keyed by (field, value) would stay one structure.	Scanning all books per search.

Java implementation

Package `lld.ch15.library`. Files: `Model.java` (enums, records, entities), `Catalog.java`, `Support.java` (fine policy, notifier, clock, money formatting), `LendingService.java`, `Library.java`, `Main.java`.

```java
// File: lld/ch15/library/Model.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.List;

enum BookFormat { HARDCOVER, PAPERBACK, AUDIO_BOOK, EBOOK, NEWSPAPER, MAGAZINE, JOURNAL }

enum BookStatus { AVAILABLE, RESERVED, LOANED, LOST }

enum AccountStatus { ACTIVE, CLOSED, CANCELED, BLACKLISTED }

record Author(String name) {}

/** Where a copy physically sits. */
record Rack(int number, String location) {}

/** The TITLE: one per ISBN, however many copies the library owns. Immutable. */
record Book(String isbn, String title, String subject, String publisher,
            LocalDate publicationDate, List<Author> authors) {
    Book { authors = List.copyOf(authors); }
}

/** One PHYSICAL COPY of a Book: own barcode, own rack, own status. Guarded by its own monitor. */
final class BookItem {
    private final String barcode;
    private final Book book;
    private final BookFormat format;
    private final Rack rack;
    private final BigDecimal price;          // money: BigDecimal, never double
    private BookStatus status = BookStatus.AVAILABLE;
    private String heldForMemberId;          // set only while status == RESERVED

    BookItem(String barcode, Book book, BookFormat format, Rack rack, BigDecimal price) {
        this.barcode = barcode; this.book = book; this.format = format;
        this.rack = rack; this.price = price;
    }
    String barcode()              { return barcode; }
    Book book()                   { return book; }
    Rack rack()                   { return rack; }
    BookFormat format()           { return format; }
    BigDecimal price()            { return price; }
    synchronized BookStatus status()      { return status; }
    synchronized String heldForMemberId() { return heldForMemberId; }

    // State changes are package-private: only LendingService calls them, while holding this monitor.
    void markLoaned()              { status = BookStatus.LOANED;    heldForMemberId = null; }
    void markAvailable()           { status = BookStatus.AVAILABLE; heldForMemberId = null; }
    void holdFor(String memberId)  { status = BookStatus.RESERVED;  heldForMemberId = memberId; }
    void markLost()                { status = BookStatus.LOST;      heldForMemberId = null; }
}

/** Each account owns exactly one card (composition): the card's barcode identifies the person at the desk. */
record LibraryCard(String cardNumber, String barcode, LocalDate issuedOn) {}

abstract sealed class Account permits Member, Librarian {
    private final String id;
    private final String name;
    private final LibraryCard card;
    private volatile AccountStatus status = AccountStatus.ACTIVE;

    Account(String id, String name, LibraryCard card) { this.id = id; this.name = name; this.card = card; }
    String id()               { return id; }
    String name()             { return name; }
    LibraryCard card()        { return card; }
    AccountStatus status()    { return status; }
    void setStatus(AccountStatus s) { status = s; }
}
```

```java
/** A member is a person with a quota. The count is guarded by the member's monitor. */
final class Member extends Account {
    private int booksCheckedOut;
    Member(String id, String name, LibraryCard card) { super(id, name, card); }
    int booksCheckedOut()   { return booksCheckedOut; }
    void incrementLoans()   { booksCheckedOut++; }
    void decrementLoans()   { booksCheckedOut--; }
}

final class Librarian extends Account {
    Librarian(String id, String name, LibraryCard card) { super(id, name, card); }
}

/** One checkout of one copy by one member. Answers "who has this copy?" and "what does this member have?". */
final class BookLending {
    private final String barcode;
    private final String memberId;
    private final LocalDate creationDate;
    private volatile LocalDate dueDate;      // volatile: read by the overdue job without the item lock
    private volatile LocalDate returnDate;

    BookLending(String barcode, String memberId, LocalDate creationDate, LocalDate dueDate) {
        this.barcode = barcode; this.memberId = memberId;
        this.creationDate = creationDate; this.dueDate = dueDate;
    }
    String barcode()        { return barcode; }
    String memberId()       { return memberId; }
    LocalDate creationDate(){ return creationDate; }
    LocalDate dueDate()     { return dueDate; }
    LocalDate returnDate()  { return returnDate; }
    void renewUntil(LocalDate d) { dueDate = d; }
    void close(LocalDate d)      { returnDate = d; }
}

record Fine(String memberId, String barcode, BigDecimal amount, LocalDate date) {}

final class LibraryException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    LibraryException(String msg) { super(msg); }
}
```

```java
// File: lld/ch15/library/Catalog.java
package lld.ch15.library;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The four searches the requirements ask for. */
interface Search {
    List<Book> searchByTitle(String title);
    List<Book> searchByAuthor(String author);
    List<Book> searchBySubject(String subject);
    List<Book> searchByPublicationDate(LocalDate date);
}

/**
 * Keeps one index per search so every lookup is a map read, not a scan.
 * It also knows every copy of every title, which answers "how many copies are free?".
 * Reads vastly outnumber writes in a library, so concurrent maps + copy-on-write lists fit well.
 */
final class Catalog implements Search {
    private final Map<String, List<Book>> byTitle   = new ConcurrentHashMap<>();
    private final Map<String, List<Book>> byAuthor  = new ConcurrentHashMap<>();
    private final Map<String, List<Book>> bySubject = new ConcurrentHashMap<>();
    private final Map<LocalDate, List<Book>> byDate = new ConcurrentHashMap<>();
    private final Map<String, Book> books = new ConcurrentHashMap<>();                  // isbn -> title
    private final Map<String, List<BookItem>> copies = new ConcurrentHashMap<>();       // isbn -> copies
    private final Map<String, BookItem> itemsByBarcode = new ConcurrentHashMap<>();

    void addBook(Book b) {
        if (books.putIfAbsent(b.isbn(), b) != null) return;       // already indexed
        index(byTitle, key(b.title()), b);
        b.authors().forEach(a -> index(byAuthor, key(a.name()), b));
        index(bySubject, key(b.subject()), b);
        byDate.computeIfAbsent(b.publicationDate(), d -> new CopyOnWriteArrayList<>()).add(b);
    }

    void addItem(BookItem item) {
        addBook(item.book());
        if (itemsByBarcode.putIfAbsent(item.barcode(), item) != null)
            throw new LibraryException("duplicate barcode " + item.barcode());
        copies.computeIfAbsent(item.book().isbn(), k -> new CopyOnWriteArrayList<>()).add(item);
    }

    /** Take a copy out of stock (weeded, damaged beyond repair). A copy that is out on loan or held cannot go. */
    void removeItem(String barcode) {
        BookItem item = item(barcode);
        synchronized (item) {
            if (item.status() == BookStatus.LOANED || item.status() == BookStatus.RESERVED)
                throw new LibraryException(barcode + " is " + item.status() + ", cannot remove it now");
            itemsByBarcode.remove(barcode);
            copies.getOrDefault(item.book().isbn(), List.of()).remove(item);
        }
    }

    BookItem item(String barcode) {
        BookItem i = itemsByBarcode.get(barcode);
        if (i == null) throw new LibraryException("unknown barcode " + barcode);
        return i;
    }

    List<BookItem> copiesOf(String isbn) { return copies.getOrDefault(isbn, List.of()); }

    long availableCopies(String isbn) {
        return copiesOf(isbn).stream().filter(c -> c.status() == BookStatus.AVAILABLE).count();
    }
```

```
    @Override public List<Book> searchByTitle(String t)     { return find(byTitle, key(t)); }
    @Override public List<Book> searchByAuthor(String a)    { return find(byAuthor, key(a)); }
    @Override public List<Book> searchBySubject(String s)   { return find(bySubject, key(s)); }
    @Override public List<Book> searchByPublicationDate(LocalDate d) { return List.copyOf(byDate.getOrDefault(d,
List.of())); }

    private static String key(String s) { return s.trim().toLowerCase(Locale.ROOT); }

    private static void index(Map<String, List<Book>> idx, String k, Book b) {
        idx.computeIfAbsent(k, x -> new CopyOnWriteArrayList<>()).add(b);
    }

    private static List<Book> find(Map<String, List<Book>> idx, String k) {
        return idx.getOrDefault(k, List.of()).stream().sorted(Comparator.comparing(Book::title)).toList();
    }
}
```

```java
// File: lld/ch15/library/Support.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.time.temporal.ChronoUnit;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

/** Strategy: how much a late return costs. Swappable without touching LendingService. */
interface FinePolicy {
    BigDecimal fine(LocalDate dueDate, LocalDate returnDate);
}

/** Flat amount per day late (the classic rule: $0.50 a day). */
final class PerDayFine implements FinePolicy {
    private final BigDecimal perDay;
    PerDayFine(BigDecimal perDay) { this.perDay = perDay; }
    @Override public BigDecimal fine(LocalDate due, LocalDate returned) {
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(due, returned));
        return perDay.multiply(BigDecimal.valueOf(daysLate));
    }
}

/** Observer-style outlet for "your reserved book is back" and "your book is overdue". */
interface Notifier {
    void send(String memberId, String message);
}

final class ConsoleNotifier implements Notifier {
    private final List<String> sent = new CopyOnWriteArrayList<>();
    @Override public void send(String memberId, String message) {
        String line = "  [notify " + memberId + "] " + message;
        sent.add(line);
        System.out.println(line);
    }
    List<String> sent() { return sent; }
}

/** A clock we can move forward in tests and demos, so "10 days later" needs no sleeping. */
final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}

final class Money {
    private Money() {}
    /** Money is BigDecimal (exact decimals); shown with exactly two decimal places. */
    static String fmt(BigDecimal amount) { return "$" + amount.setScale(2,
RoundingMode.UNNECESSARY).toPlainString(); }
}
```

```java
// File: lld/ch15/library/LendingService.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.LocalDate;
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

/**
 * The three core flows (check out, return, renew) plus reservations.
 * Lives apart from Member so a Member stays "a person", and the rules live in one place.
 *
 * Locking rule (prevents deadlock): always lock the MEMBER first, then the BOOK ITEM.
 */
public final class LendingService {
    static final int MAX_BOOKS_PER_MEMBER = 5;
    static final int MAX_LENDING_DAYS = 10;

    private final Catalog catalog;
    private final Clock clock;
    private final FinePolicy finePolicy;
    private final Notifier notifier;
    private final Map<String, Member> members = new ConcurrentHashMap<>();
    private final Map<String, BookLending> activeLoans = new ConcurrentHashMap<>();    // barcode -> loan
    private final Map<String, Deque<String>> waitlists = new ConcurrentHashMap<>();    // isbn -> member ids
(FIFO)
    private final List<Fine> fines = Collections.synchronizedList(new ArrayList<>());

    LendingService(Catalog catalog, Clock clock, FinePolicy finePolicy, Notifier notifier) {
        this.catalog = catalog; this.clock = clock; this.finePolicy = finePolicy; this.notifier = notifier;
    }

    void register(Member m) { members.put(m.id(), m); }

    /** Cancel a membership. Refused while the member still has books out; drops them from every waitlist. */
    void cancelMembership(String memberId) {
        Member m = member(memberId);
        synchronized (m) {
            if (m.booksCheckedOut() > 0)
                throw new LibraryException(memberId + " must return " + m.booksCheckedOut() + " book(s) first");
            m.setStatus(AccountStatus.CANCELED);
        }
        for (Deque<String> q : waitlists.values()) synchronized (q) { q.remove(memberId); }
    }

    private LocalDate today() { return LocalDate.now(clock); }

    private Member member(String id) {
        Member m = members.get(id);
        if (m == null) throw new LibraryException("unknown member " + id);
        return m;
    }

    // ---------------------------------------------------------------- check out
    BookLending checkout(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (m.status() != AccountStatus.ACTIVE)
                    throw new LibraryException(memberId + " account is " + m.status());
                if (m.booksCheckedOut() >= MAX_BOOKS_PER_MEMBER)
                    throw new LibraryException(memberId + " already has " + MAX_BOOKS_PER_MEMBER + " books");
                switch (item.status()) {
```

```
                    case AVAILABLE -> { /* ok */ }
                    case RESERVED -> {
                        if (!memberId.equals(item.heldForMemberId()))
                            throw new LibraryException(barcode + " is held for another member");
                    }
                    case LOANED, LOST -> throw new LibraryException(barcode + " is " + item.status());
                }
                item.markLoaned();
                m.incrementLoans();
                LocalDate today = today();
                BookLending loan = new BookLending(barcode, memberId, today, today.plusDays(MAX_LENDING_DAYS));
                activeLoans.put(barcode, loan);
                removeFromWaitlist(item.book().isbn(), memberId);   // their reservation is now fulfilled
                return loan;
            }
        }
    }

    // ---------------------------------------------------------------- return
    BigDecimal returnBook(String barcode) {
        BookLending loan = activeLoans.get(barcode);
        if (loan == null) throw new LibraryException(barcode + " is not checked out");
        Member m = member(loan.memberId());
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                if (activeLoans.get(barcode) != loan)               // someone returned it meanwhile
                    throw new LibraryException(barcode + " is not checked out");
                LocalDate today = today();
                loan.close(today);
                activeLoans.remove(barcode);
                m.decrementLoans();
                BigDecimal fine = chargeFineIfLate(loan, today);
                handToNextInLine(item);
                return fine;
            }
        }
    }

    // ---------------------------------------------------------------- renew
    BookLending renew(String memberId, String barcode) {
        Member m = member(memberId);
        BookItem item = catalog.item(barcode);
        synchronized (m) {
            synchronized (item) {
                BookLending loan = activeLoans.get(barcode);
                if (loan == null || !loan.memberId().equals(memberId))
                    throw new LibraryException(memberId + " does not have " + barcode);
                if (waitingCount(item.book().isbn()) > 0)
                    throw new LibraryException("cannot renew " + barcode + ": another member reserved this
title");
                LocalDate today = today();
                chargeFineIfLate(loan, today);                      // a late renewal still pays the fine
                loan.renewUntil(today.plusDays(MAX_LENDING_DAYS));
                return loan;
            }
        }
    }

    // ---------------------------------------------------------------- reserve
    /** Join the FIFO waitlist for a TITLE that has no free copy. Returns the 1-based position. */
    int reserve(String memberId, String isbn) {
        Member m = member(memberId);
        if (m.status() != AccountStatus.ACTIVE) throw new LibraryException(memberId + " account is " +
m.status());
        if (catalog.copiesOf(isbn).isEmpty()) throw new LibraryException("unknown isbn " + isbn);
        if (catalog.availableCopies(isbn) > 0)
            throw new LibraryException("a copy is on the shelf, just check it out");
        Deque<String> q = waitlist(isbn);
        synchronized (q) {
            if (!q.contains(memberId)) q.addLast(memberId);
```

```
            return new ArrayList<>(q).indexOf(memberId) + 1;
        }
    }

    // ------------------------------------------------------------------ system jobs & queries
    void sendOverdueNotices() {
        LocalDate today = today();
        activeLoans.values().stream()
                .filter(l -> l.dueDate().isBefore(today))
                .sorted((a, b) -> a.barcode().compareTo(b.barcode()))
                .forEach(l -> notifier.send(l.memberId(), "'" + catalog.item(l.barcode()).book().title()
                        + "' (" + l.barcode() + ") was due " + l.dueDate()));
    }

    String whoHas(String barcode) {
        BookLending l = activeLoans.get(barcode);
        return l == null ? null : l.memberId();
    }

    List<String> loansOf(String memberId) {
        return activeLoans.values().stream().filter(l -> l.memberId().equals(memberId))
                .map(BookLending::barcode).sorted().toList();
    }

    List<Fine> fines() { synchronized (fines) { return List.copyOf(fines); } }

    // ------------------------------------------------------------------ helpers
    private BigDecimal chargeFineIfLate(BookLending loan, LocalDate today) {
        BigDecimal amount = finePolicy.fine(loan.dueDate(), today);
        if (amount.signum() > 0) {
            fines.add(new Fine(loan.memberId(), loan.barcode(), amount, today));
            notifier.send(loan.memberId(), "fine of " + Money.fmt(amount) + " for late " + loan.barcode());
        }
        return amount;
    }

    /** Caller holds the item's monitor. Give the copy to the first waiting member, or shelve it. */
    private void handToNextInLine(BookItem item) {
        Deque<String> q = waitlist(item.book().isbn());
        String next;
        synchronized (q) { next = q.pollFirst(); }
        if (next == null) { item.markAvailable(); return; }
        item.holdFor(next);
        notifier.send(next, "'" + item.book().title() + "' copy " + item.barcode() + " is held for you");
    }

    private Deque<String> waitlist(String isbn) {
        return waitlists.computeIfAbsent(isbn, k -> new ArrayDeque<>());
    }

    int waitingCount(String isbn) {
        Deque<String> q = waitlist(isbn);
        synchronized (q) { return q.size(); }
    }

    private void removeFromWaitlist(String isbn, String memberId) {
        Deque<String> q = waitlist(isbn);
        synchronized (q) { q.remove(memberId); }
    }
}
```

```java
// File: lld/ch15/library/Library.java
package lld.ch15.library;

import java.time.Clock;

/** Facade: the one object the desk software talks to. Owns the catalog and the lending rules. */
final class Library {
    private final String name;
    private final String address;
    private final Catalog catalog = new Catalog();
    private final LendingService lending;

    Library(String name, String address, Clock clock, FinePolicy finePolicy, Notifier notifier) {
        this.name = name; this.address = address;
        this.lending = new LendingService(catalog, clock, finePolicy, notifier);
    }

    /** Only a librarian may add stock (the use-case diagram's "Add book item"). */
    void addBookItem(Account by, BookItem item) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        catalog.addItem(item);
    }

    void removeBookItem(Account by, String barcode) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        catalog.removeItem(barcode);
    }

    void registerMember(Account by, Member m) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        lending.register(m);
    }

    void cancelMembership(Account by, String memberId) {
        if (!(by instanceof Librarian)) throw new LibraryException(by.id() + " is not a librarian");
        lending.cancelMembership(memberId);
    }

    String name()            { return name; }
    String address()         { return address; }
    Catalog catalog()        { return catalog; }
    LendingService lending() { return lending; }
}
```

```java
// File: lld/ch15/library/Main.java
package lld.ch15.library;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-03-01T10:00:00Z"));
        ConsoleNotifier notifier = new ConsoleNotifier();
        Library lib = new Library("City Library", "1 Main Street", clock,
                new PerDayFine(new BigDecimal("0.50")), notifier);
        LendingService desk = lib.lending();
        LocalDate today = LocalDate.of(2026, 3, 1);

        Librarian lucy = new Librarian("L1", "Lucy", new LibraryCard("C-L1", "BC-L1", today));
        Book dune = new Book("978-0441013593", "Dune", "Science Fiction", "Ace",
                LocalDate.of(1965, 8, 1), List.of(new Author("Frank Herbert")));
        Book hobbit = new Book("978-0547928227", "The Hobbit", "Fantasy", "Mariner",
                LocalDate.of(1937, 9, 21), List.of(new Author("J.R.R. Tolkien")));
        Book lotr = new Book("978-0544003415", "The Lord of the Rings", "Fantasy", "Mariner",
                LocalDate.of(1954, 7, 29), List.of(new Author("J.R.R. Tolkien")));
        Rack r1 = new Rack(1, "Aisle A");
        lib.addBookItem(lucy, new BookItem("B-DUNE-1", dune, BookFormat.PAPERBACK, r1, new BigDecimal("9.99")));
        for (int i = 1; i <= 6; i++)
            lib.addBookItem(lucy, new BookItem("B-HOB-" + i, hobbit, BookFormat.HARDCOVER, r1, new
BigDecimal("14.99")));
        lib.addBookItem(lucy, new BookItem("B-LOTR-1", lotr, BookFormat.HARDCOVER, r1, new BigDecimal("29.99")));

        for (int i = 1; i <= 9; i++)
            lib.registerMember(lucy, new Member("M" + i, "Member " + i, new LibraryCard("C-M" + i, "BC-M" + i,
today)));

        System.out.println("== 1. Search the catalog ==");
        System.out.println("  author 'j.r.r. tolkien' -> " + titles(lib.catalog().searchByAuthor("j.r.r.
tolkien")));
        System.out.println("  subject 'Fantasy'       -> " + titles(lib.catalog().searchBySubject("Fantasy")));
        System.out.println("  published 1965-08-01    -> " +
titles(lib.catalog().searchByPublicationDate(LocalDate.of(1965, 8, 1))));
        System.out.println("  free copies of Hobbit   -> " + lib.catalog().availableCopies(hobbit.isbn()));

        System.out.println("\n== 2. Checkout limit (max 5 books per member) ==");
        for (int i = 1; i <= 6; i++) {
            try {
                BookLending l = desk.checkout("M1", "B-HOB-" + i);
                System.out.println("  M1 took " + l.barcode() + ", due " + l.dueDate());
            } catch (LibraryException e) {
                System.out.println("  REFUSED: " + e.getMessage());
            }
        }
        System.out.println("  M1 has " + desk.loansOf("M1"));

        System.out.println("\n== 3. 8 members race for the ONLY copy of Dune ==");
        AtomicInteger winners = new AtomicInteger(), losers = new AtomicInteger();
        CountDownLatch start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 2; i <= 9; i++) {
                String id = "M" + i;
                pool.submit(() -> {
                    start.await();
                    try { desk.checkout(id, "B-DUNE-1"); winners.incrementAndGet(); }
                    catch (LibraryException e) { losers.incrementAndGet(); }
                    return null;
```

```
                });
            }
            start.countDown();                  // release all threads at once
        }                                       // close() waits for every task
        String holder = desk.whoHas("B-DUNE-1");
        System.out.println("  winners=" + winners + " losers=" + losers + " (exactly one copy was lent)");
        System.out.println("  holder = " + holder + "  (which member wins varies from run to run)");

        System.out.println("\n== 4. Reserve a title with no free copy (FIFO waitlist) ==");
        String first = holder.equals("M2") ? "M3" : "M2";
        String second = holder.equals("M4") ? "M5" : "M4";
        System.out.println("  " + first + " position " + desk.reserve(first, dune.isbn()));
        System.out.println("  " + second + " position " + desk.reserve(second, dune.isbn()));
        try { desk.reserve("M6", hobbit.isbn()); }
        catch (LibraryException e) { System.out.println("  M6 reserve Hobbit REFUSED: " + e.getMessage()); }

        System.out.println("\n== 5. Renew is blocked while someone waits ==");
        try { desk.renew(holder, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }

        System.out.println("\n== 6. 13 days later: overdue notices, late return, fine, hand-over ==");
        clock.advance(Duration.ofDays(13));
        desk.sendOverdueNotices();
        BigDecimal fine = desk.returnBook("B-DUNE-1");
        System.out.println("  fine charged = " + Money.fmt(fine) + " (3 days late x $0.50)");
        try { desk.checkout(second, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  " + second + " REFUSED: " + e.getMessage()); }
        System.out.println("  " + first + " checks out held copy, due " + desk.checkout(first, "B-DUNE-
1").dueDate());
        System.out.println("  who has B-DUNE-1? " + desk.whoHas("B-DUNE-1"));

        System.out.println("\n== 7. Renew on time with nobody waiting ==");
        desk.returnBook("B-HOB-1");                     // 13 days: late, fine charged
        desk.checkout("M6", "B-HOB-1");
        clock.advance(Duration.ofDays(4));
        System.out.println("  M6 renewed B-HOB-1, new due " + desk.renew("M6", "B-HOB-1").dueDate());
        System.out.println("  fines on record: " + desk.fines().size());

        System.out.println("\n== 8. Librarian housekeeping: remove a copy, cancel a membership ==");
        try { lib.removeBookItem(lucy, "B-DUNE-1"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        lib.removeBookItem(lucy, "B-LOTR-1");
        System.out.println("  copies of LOTR left: " + lib.catalog().copiesOf(lotr.isbn()).size());
        try { lib.cancelMembership(lucy, "M6"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        lib.cancelMembership(lucy, "M8");
        try { desk.checkout("M8", "B-HOB-6"); }
        catch (LibraryException e) { System.out.println("  REFUSED: " + e.getMessage()); }
    }

    private static List<String> titles(List<Book> books) { return books.stream().map(Book::title).toList(); }
}
```

Output:

```
== 1. Search the catalog ==
  author 'j.r.r. tolkien' -> [The Hobbit, The Lord of the Rings]
  subject 'Fantasy'       -> [The Hobbit, The Lord of the Rings]
  published 1965-08-01    -> [Dune]
  free copies of Hobbit   -> 6

== 2. Checkout limit (max 5 books per member) ==
  M1 took B-HOB-1, due 2026-03-11
  M1 took B-HOB-2, due 2026-03-11
  M1 took B-HOB-3, due 2026-03-11
  M1 took B-HOB-4, due 2026-03-11
  M1 took B-HOB-5, due 2026-03-11
  REFUSED: M1 already has 5 books
  M1 has [B-HOB-1, B-HOB-2, B-HOB-3, B-HOB-4, B-HOB-5]

== 3. 8 members race for the ONLY copy of Dune ==
  winners=1 losers=7 (exactly one copy was lent)
  holder = M9  (which member wins varies from run to run)

== 4. Reserve a title with no free copy (FIFO waitlist) ==
  M2 position 1
  M4 position 2
  M6 reserve Hobbit REFUSED: a copy is on the shelf, just check it out

== 5. Renew is blocked while someone waits ==
  REFUSED: cannot renew B-DUNE-1: another member reserved this title

== 6. 13 days later: overdue notices, late return, fine, hand-over ==
  [notify M9] 'Dune' (B-DUNE-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-1) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-2) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-3) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-4) was due 2026-03-11
  [notify M1] 'The Hobbit' (B-HOB-5) was due 2026-03-11
  [notify M9] fine of $1.50 for late B-DUNE-1
  [notify M2] 'Dune' copy B-DUNE-1 is held for you
  fine charged = $1.50 (3 days late x $0.50)
  M4 REFUSED: B-DUNE-1 is held for another member
  M2 checks out held copy, due 2026-03-24
  who has B-DUNE-1? M2

== 7. Renew on time with nobody waiting ==
  [notify M1] fine of $1.50 for late B-HOB-1
  M6 renewed B-HOB-1, new due 2026-03-28
  fines on record: 2

== 8. Librarian housekeeping: remove a copy, cancel a membership ==
  REFUSED: B-DUNE-1 is LOANED, cannot remove it now
  copies of LOTR left: 0
  REFUSED: M6 must return 1 book(s) first
  REFUSED: M8 account is CANCELED
```

What to notice in the output: section 2 refuses the sixth book (requirement 5); section 3 lets exactly one of eight concurrent members borrow the only copy; section 5 refuses a renewal because someone is waiting; section 6 sends overdue notices, charges 3 days × $0.50, and hands the returned copy to the first person in the waitlist — the second person is refused because the copy is *held* for the first; section 8 covers the librarian's "remove book item" and "cancel membership" use cases — a copy that is out on loan cannot be removed, a member with a book still out cannot be cancelled, and a cancelled member can no longer borrow.

Concurrency & edge cases

- **Two members, one last copy.** `checkout` locks the copy (`synchronized (item)`), so "is it available? → mark loaned" is one atomic step. The demo's 8-thread race always has exactly one winner.
- **Two librarians scanning for the same member.** The quota check and increment happen under `synchronized (member)`, so a member can never end up with 6 books.

- **Deadlock avoidance.** (A **deadlock** is two threads each holding a lock the other needs, so both wait forever.) Every method locks **member first, then item** — a fixed global lock order. Two threads can never each hold one lock while waiting for the other. (`returnBook` reads the loan first to find the member, then locks in the same order and re-checks the loan is still active.)
- **Renew vs reserve.** Renewal is refused if the title's waitlist is non-empty. The check happens *before* charging any late fine, so a refused renewal never fines the member twice.
- **A known small window.** `reserve` checks "no free copy" and then joins the queue as two steps. If a copy is returned in between, it goes to the shelf while the member sits first in the queue; the *next* return reaches them. We accept this rather than make `reserve` take the queue lock before the copies' locks, because returns take them in the opposite order (copy, then queue) — mixing the two orders is exactly how deadlocks are born.
- **Held copy.** A returned copy with a waitlist becomes `RESERVED` for one member only; everybody else is refused at checkout.
- **Edge cases handled:** unknown barcode or member, duplicate barcodes, inactive/blacklisted/cancelled accounts, removing a copy that is on loan or held, cancelling a membership while books are still out, reserving a title that has a free copy ("just check it out"), joining the same waitlist twice (keeps your place), returning a book that is not out.
- **Across several servers** the `synchronized` blocks are not enough — the same guarantee must come from the database: e.g. `UPDATE book_item SET status='LOANED' WHERE barcode=? AND status='AVAILABLE'` and check that exactly one row changed (a *conditional update*).

Extensions & follow-up questions

- **"Two members reserve the last copy at the same moment — what happens?"** The course calls this out as unguarded in its own design. In ours, `reserve` appends to a FIFO deque (double-ended queue) under a lock, so both are queued in a definite order and the first one gets the next copy.
- **"Where do you keep the number of available copies of a title?"** Don't store a counter that can drift; derive it: `Catalog.availableCopies(isbn)` counts copies with status `AVAILABLE`. If that becomes slow, cache it (keep a ready-made copy for fast reads) and update it in the same lock as the status change.
- **"What is a `BookLending` for?"** It is the loan record: it answers "who has this copy, since when, due when", drives overdue notices and fines, and is the history of the copy.
- **"How is the fine paid?"** Add a `FinePayment` / `PaymentService` and block checkouts while unpaid fines exceed a threshold — a one-line rule in `checkout`.
- **"Hold expiry?"** A held copy should not wait forever: add `heldUntil` to `BookItem` and a daily job that passes the copy to the next person (or shelves it) when the hold lapses — exactly the TTL idea of section 15.2.
- **"E-books?"** An e-book "copy" is a licence: `BookFormat.EBOOK` items with no rack, and many libraries buy a fixed number of concurrent licences — the same copy model works.

Real-world example

Open-source integrated library systems such as **Koha** and **Evergreen** implement exactly this model: bibliographic *records* (titles) with attached *items* (copies with barcodes and shelf locations), circulation (lending and return) rules per patron (library member) category (loan period, maximum items, fine per day), a holds queue per title, and overdue/hold-available notices by email or SMS. Digital lending apps such as **OverDrive's Libby** apply the same waitlist idea to e-books: a library owns a fixed number of licences, borrowers join a holds queue, and when a licence frees up the next person is notified and given a limited window to borrow before it passes on.

In the interview

30–45 minute plan. (1) 3 min — clarify: copies, limits (5 books, 10 days), fines, reservations, notifications. (2) 5 min — actors and use cases. (3) 10 min — entities, *starting with the `Book` / `BookItem` split*, then accounts, lending records, catalog. (4) 10 min — code `checkout` and `returnBook` with the locking; mention lock ordering. (5) 5 min — the reservation hand-over and fines (Strategy). (6) Remaining — follow-ups: concurrency across servers, fine payment, hold expiry.

30-second answer: "The key split is `Book` (the title) vs `BookItem` (a physical copy with a barcode and rack). Members borrow copies; search works on titles through a `Catalog` with one index per search type. A `LendingService` enforces 5 books and 10 days, records `BookLending`s, charges fines via a `FinePolicy` strategy, and passes returned copies to a FIFO waitlist per title with a notification. Checkout locks member then copy so the last copy can't be double-lent."

Self-check

- **Q:** Why is `Book` separate from `BookItem`? **A:** Search and metadata are per title; status, rack, barcode and loans are per physical copy. One title has many copies.
- **Q:** What guarantees a member can't get a 6th book when two desks scan at once? **A:** The quota check and increment happen inside `synchronized (member)`.
- **Q:** Why a fixed lock order? **A:** Without it, thread A could hold member→wait item while B holds item→wait member: deadlock.
- **Q:** Which pattern lets the library change its fine rules without touching lending code? **A:** Strategy (`FinePolicy`).
- **Q:** When can't a book be renewed? **A:** When another member is waiting for that title.

TL;DR

Title vs copy. Limits on the member, state on the copy, history in `BookLending`. Rules in a `LendingService`; fines as a Strategy; notifications behind an interface. Lock member-then-copy so the last copy is never lent twice. Reservations are a FIFO queue per title.

15.2 Design a Movie Ticket Booking System (BookMyShow)



Figure — Cover image for the movie ticket booking case study (course diagram)

Problem & clarifying questions

An online movie ticket booking system (think **BookMyShow**, **Fandango**, **AMC's app**) lets customers see which movies are playing, pick a show, choose seats on a seat map, pay, and receive a ticket — from anywhere, at any time. It is *the* classic LLD booking question because it has a hierarchy of places (city → cinema → hall → seat), a time dimension (shows), and a genuine race (two people tapping the same seat).

Question	Answer we assume
One city or many?	Many cities with affiliate (partner) cinemas; the app lists cities.
Can a cinema have several halls? Several shows per hall?	Yes and yes — but a hall runs **one show at a time**.
What do customers search by?	Title, language, genre, release date, **city**.
Can a customer pick several seats in one booking?	Yes, from a seat map showing free vs booked seats.
How long is a seat kept while paying?	A few minutes (we use **5**). If payment does not finish, the seats go back.
Payment methods? Discounts?	Credit card or cash (cash at the front desk); optional **discount coupon**.
Refunds?	On cancellation, refund if within the allowed time (we use: up to 2 hours before the show).
Notifications?	New movie, booking made, booking cancelled.
Who else uses it?	Admins (add movies/shows, block customers), front-desk officers (book for walk-ins), guests (search only).

Requirements

Functional (the course's list, in order)

1. List the **cities** where affiliate cinemas are located.
2. Each cinema has many **halls**; each hall runs **one show at a time**.
3. Each movie has many **shows**.
4. **Search** movies by title, language, genre, release date and city.
5. After choosing a movie, show the **cinemas and shows** running it.
6. Pick a show and **book** tickets.
7. Show the **seating plan**; let the customer pick several seats.
8. Distinguish **available** seats from booked ones.
9. **Notify** on a new movie, and on booking or cancellation.
10. Pay by **credit card or cash**.
11. **No two customers can reserve the same seat.**
12. Apply a **discount coupon** to the payment.

Non-functional

- **Strong consistency on seats** (every reader sees the latest confirmed state; see System Design Fundamentals): double booking is the one bug that must never happen.
- **High read, bursty write** (writes arrive in sudden spikes): browsing dwarfs booking, but a blockbuster's first-day shows get many simultaneous seat grabs.
- **Hold expiry**: abandoned checkouts must not block seats (the course lists this as a gap in its own design; we implement it).
- **Failure-safe payments**: a customer must never pay for a seat they don't get.

Use cases

Five actors: **Admin** (add movies and shows, cancel a movie or show, block/unblock customers), **FrontDeskOfficer** (book and cancel tickets for walk-ins), **Customer** (view schedules, book, cancel), **Guest** (search only — must register to book), **System** (notifications).

Top use cases: search movies; create/modify/view booking; make payment; add a coupon to the payment; assign seats (seat map); refund payment on cancellation within the allowed time.

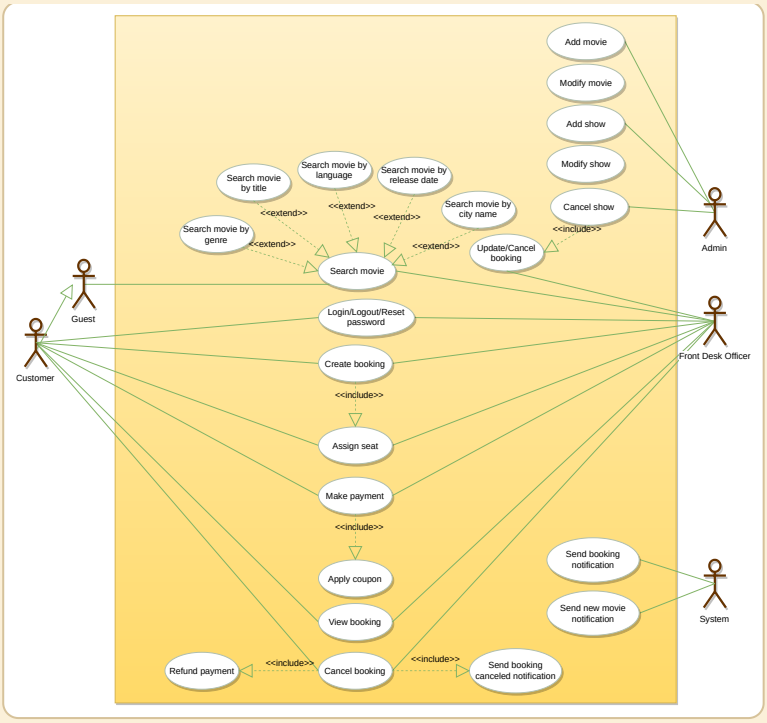


Figure — Movie booking use cases. The hollow arrow from Customer to Guest says a customer can do everything a guest can; the five "search by" use cases extend "Search movie"; "Create booking" includes "Assign seat"; "Make payment" includes "Apply coupon"; "Cancel booking" includes "Refund payment" and the cancellation notification; the admin's "Cancel show" includes updating/cancelling its bookings (course diagram)

Notice in the figure above that a **Customer is a specialised Guest**: everything a guest can do (search), a customer can do too, plus booking. That is the use-case version of the inheritance we enforce in code by giving `BookingService` a `Customer` parameter.

Core entities

Class	Responsibility
`City` (record)	Name and state. Cinemas point to it (association — the city exists without the cinema).
`Cinema`	Name, city; **owns** its halls.
`CinemaHall`	Name, physical seats; refuses an overlapping show.
`HallSeat` (record)	The seat bolted to the floor: id and `SeatType` (`REGULAR`, `PREMIUM`, `ACCESSIBLE`, `EMERGENCY_EXIT`, `OTHER`). **No booking state.**
`Movie` (record)	Title, language, genre, release date, duration.
`Show`	One screening of a movie in a hall at a time; **owns one `ShowSeat` per hall seat**.
`ShowSeat`	A seat *for this show*: price and `booked` flag. **This is what gets booked.**
`SeatLockProvider` / `InMemorySeatLockProvider`	Temporary, expiring **holds** on show seats; all-or-nothing lock, commit, release.
`Booking`	Customer, show, seat ids, status (`PENDING`, `CONFIRMED`, `CHECKED_IN`, `CANCELED`, `ABANDONED`), payment, coupon.
`BookingService`	hold → pay → commit (make final); cancel; seat map.
`Payment`, `PaymentMethod` (`CardPayment`, `CashPayment`), `PaymentGateway`	Money. The gateway (the external service, such as a card processor, that actually moves the money) is the boundary to the real provider.
`Coupon`	Percentage off, applied to the booking amount before payment.
`CancellationPolicy` / `CutoffRefundPolicy`	How much comes back on cancel (Strategy).
`MovieSearch` / `Catalog`	Five searches; "by city" is answered from shows.
`Customer`, `Guest`, `FrontDeskOfficer`, `Admin`	Actors; a `Guest` has no type that `BookingService` accepts.
`Notifier`	Messages to users: new movie (sent by `Catalog.addMovie`), booking confirmed, booking cancelled.

Class diagram

The course grows the model over **six stages**; after each one the system gains a capability it lacked:

Stage	Adds	What works afterwards
1	City, cinema, hall, physical seat	There is a building with seats in it
2	Movie, show, show seat	Every screening carries its own seat list and prices
3	Coupon, payment types, notification types	An amount can be reduced by a coupon and then paid
4	Booking and `BookingService`	One seat can never go to two customers
5	Search interface and catalog	A movie can be looked up in five different ways
6	People and accounts	Booking is restricted to registered customers

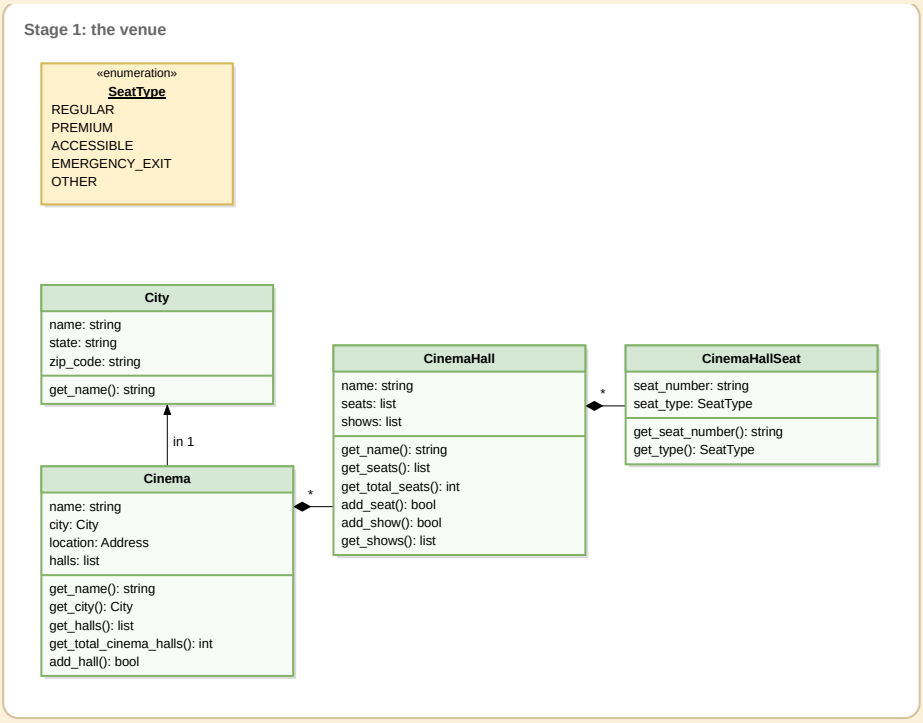


Figure — Stage 1: `City`, `Cinema`, `CinemaHall` and `CinemaHallSeat`; cinema→city is a plain association, cinema→hall and hall→seat are composition (course diagram)

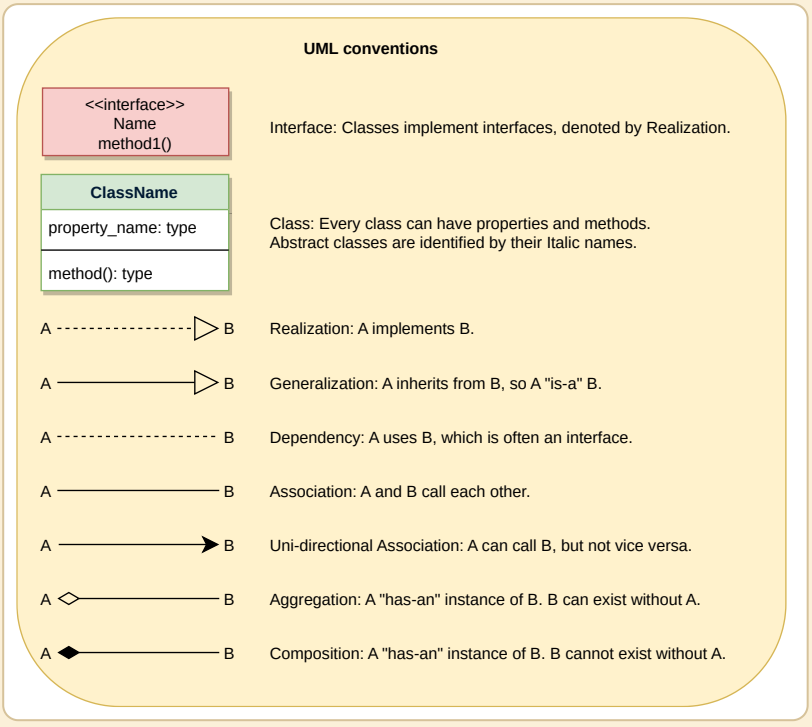


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

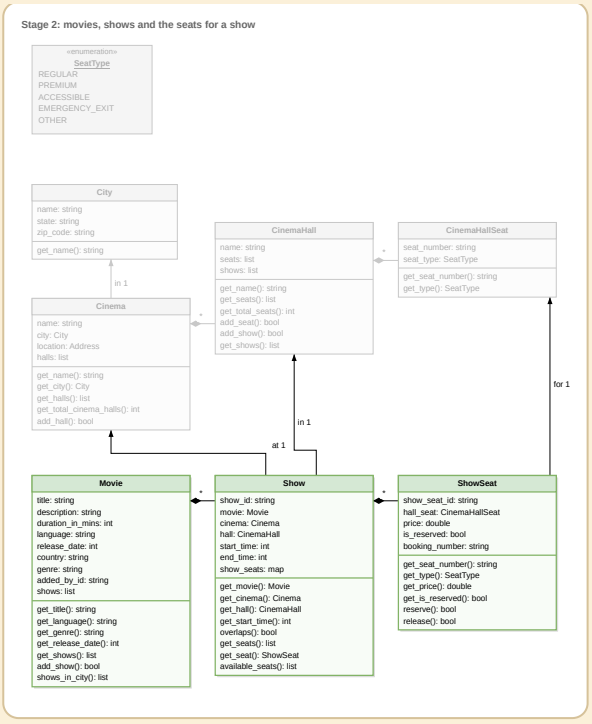


Figure — Stage 2: `Movie` owns its `Show`s; each `Show` owns one `ShowSeat` per physical seat, and each show seat points back at its `CinemaHallSeat` (course diagram)

Stage 2 is **where the design succeeds or fails**. One physical seat can be empty for the 3 pm screening and sold for the 9 pm one, so the thing you reserve cannot be the physical seat. You reserve a `ShowSeat` — one per seat per show — which also lets each show price its seats differently (a premium seat costs more).

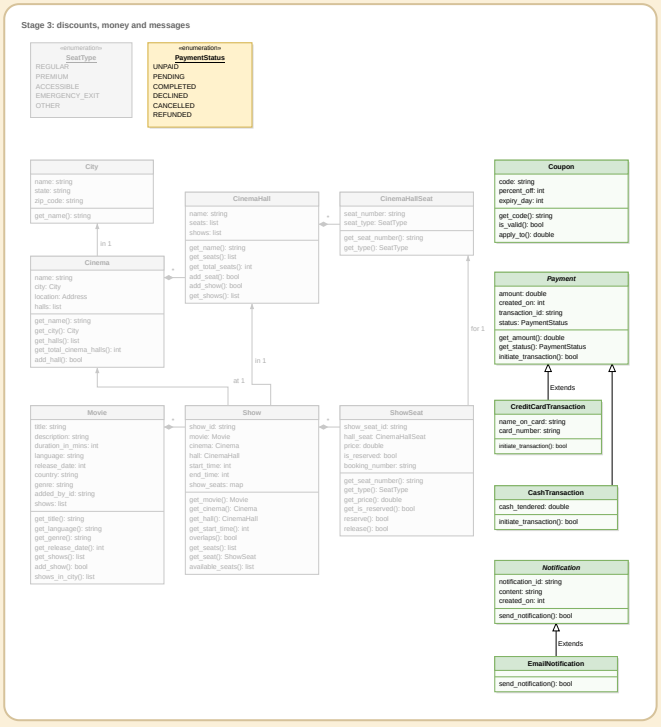


Figure — Stage 3: `Coupon`, abstract `Payment` (credit card and cash subclasses) and abstract `Notification` (one subclass per channel); none of them points at anything yet (course diagram)

The coupon reduces **the booking's amount, not the payment**: by the time any payment object is created the price is final, and no payment type needs its own discount logic.

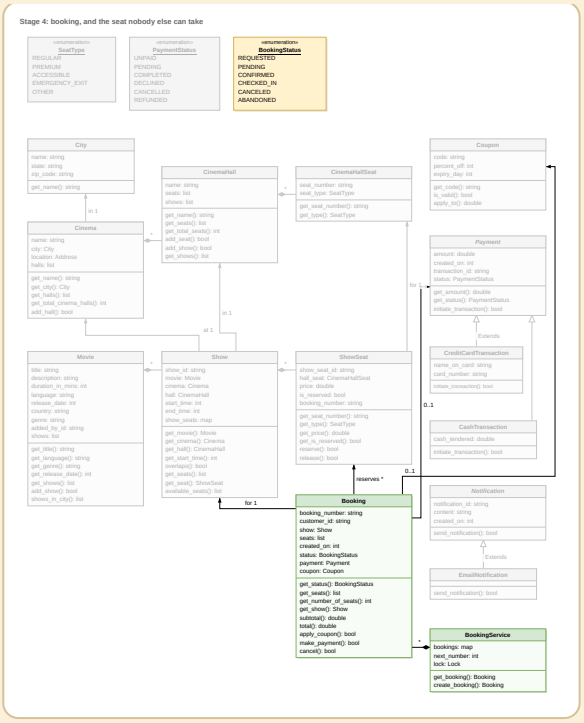


Figure — Stage 4: `Booking` points at the show, seats, payment and coupon without owning them; `BookingService` owns the bookings and is the only class holding a lock (course diagram)

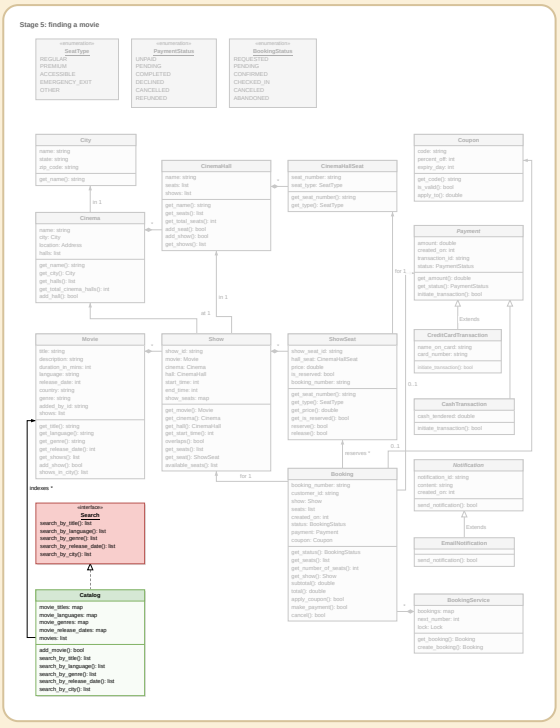


Figure — Stage 5: the `Search` interface with five methods, implemented by `Catalog`; four searches use indexes built when a movie is added, while city search is computed from the scheduled shows (course diagram)

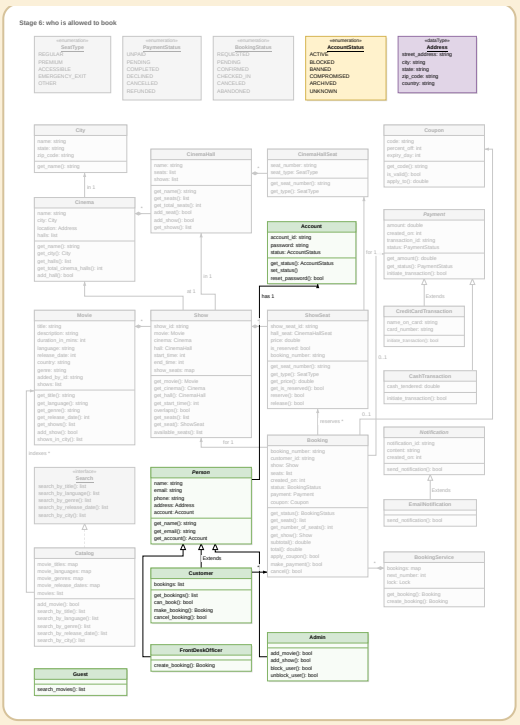


Figure — Stage 6: `Person` with `Customer`, `FrontDeskOfficer` and `Admin`; `Guest` sits outside that tree and has no booking method; a customer keeps the bookings it made (course diagram)

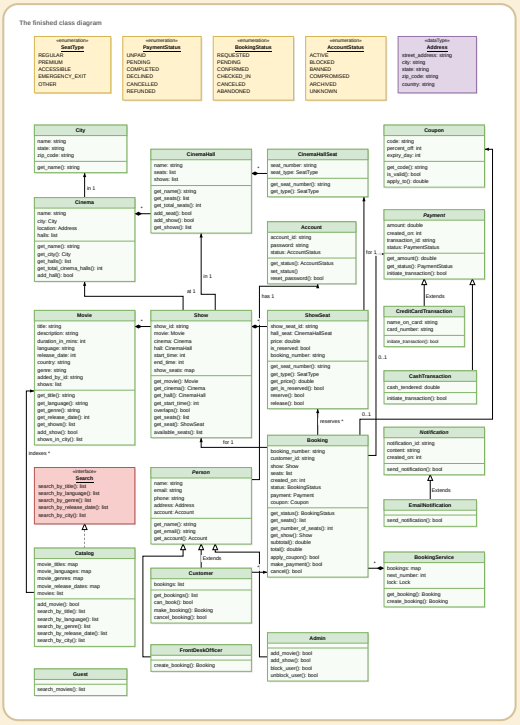
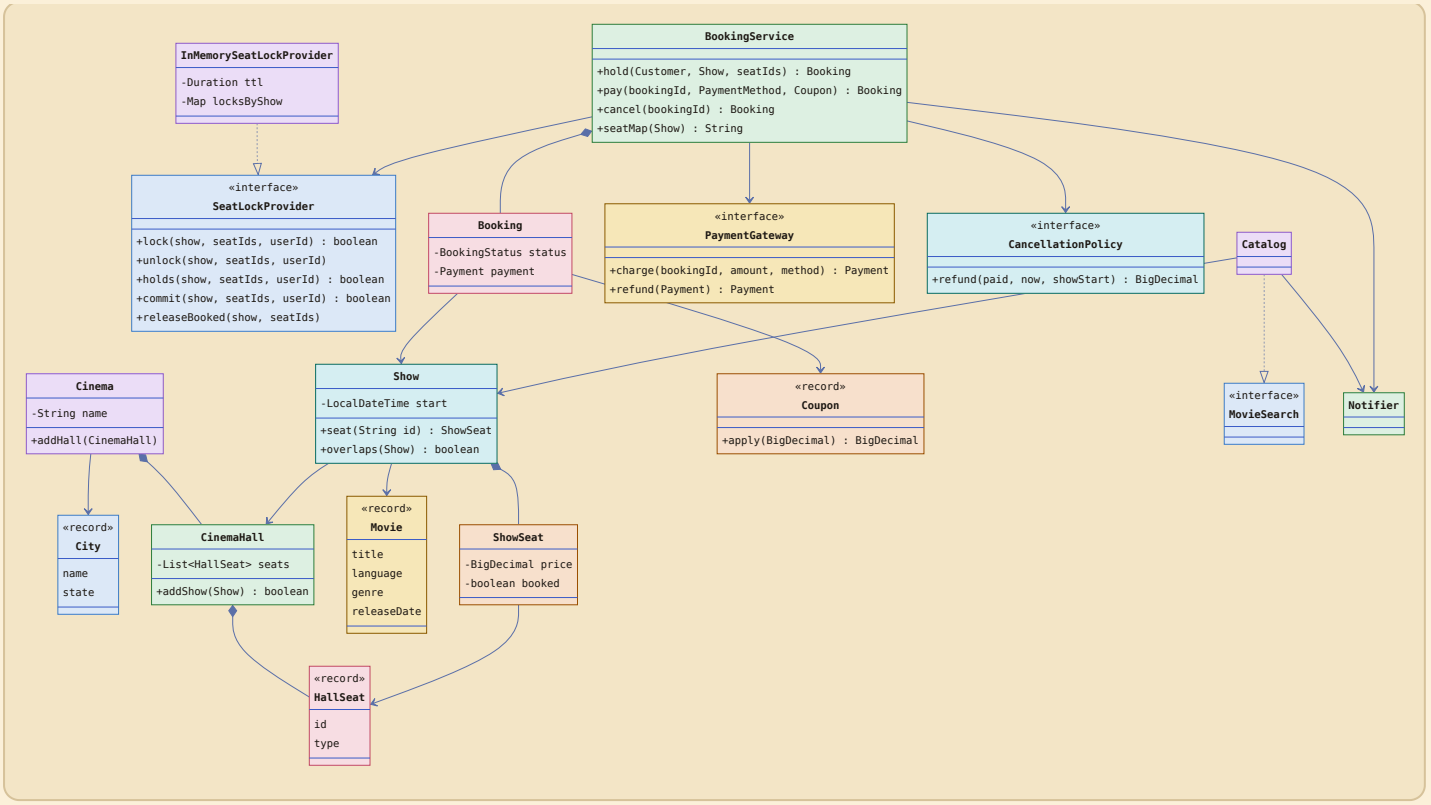


Figure — The finished movie booking class diagram (course diagram)

Our Java design (the main difference from the course: a separate `SeatLockProvider` with **TTL holds**, which the course names as its biggest omission):



Key flows

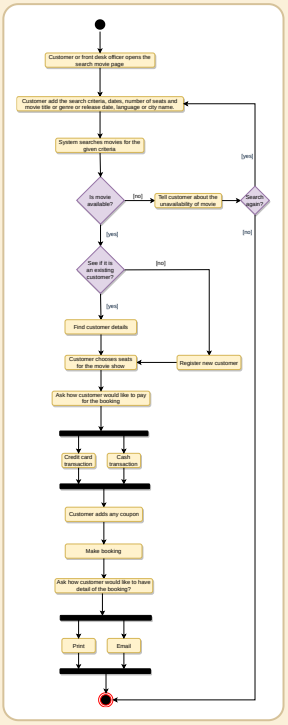


Figure — Make a booking: search movies; if available, find or register the customer, pick seats, ask how they will pay (credit card or cash), and choose how to receive the ticket (course diagram)

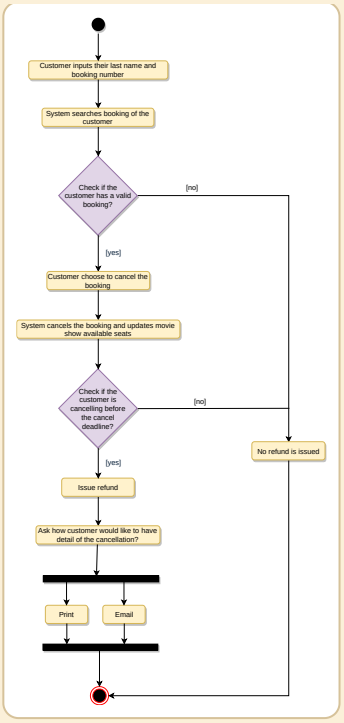
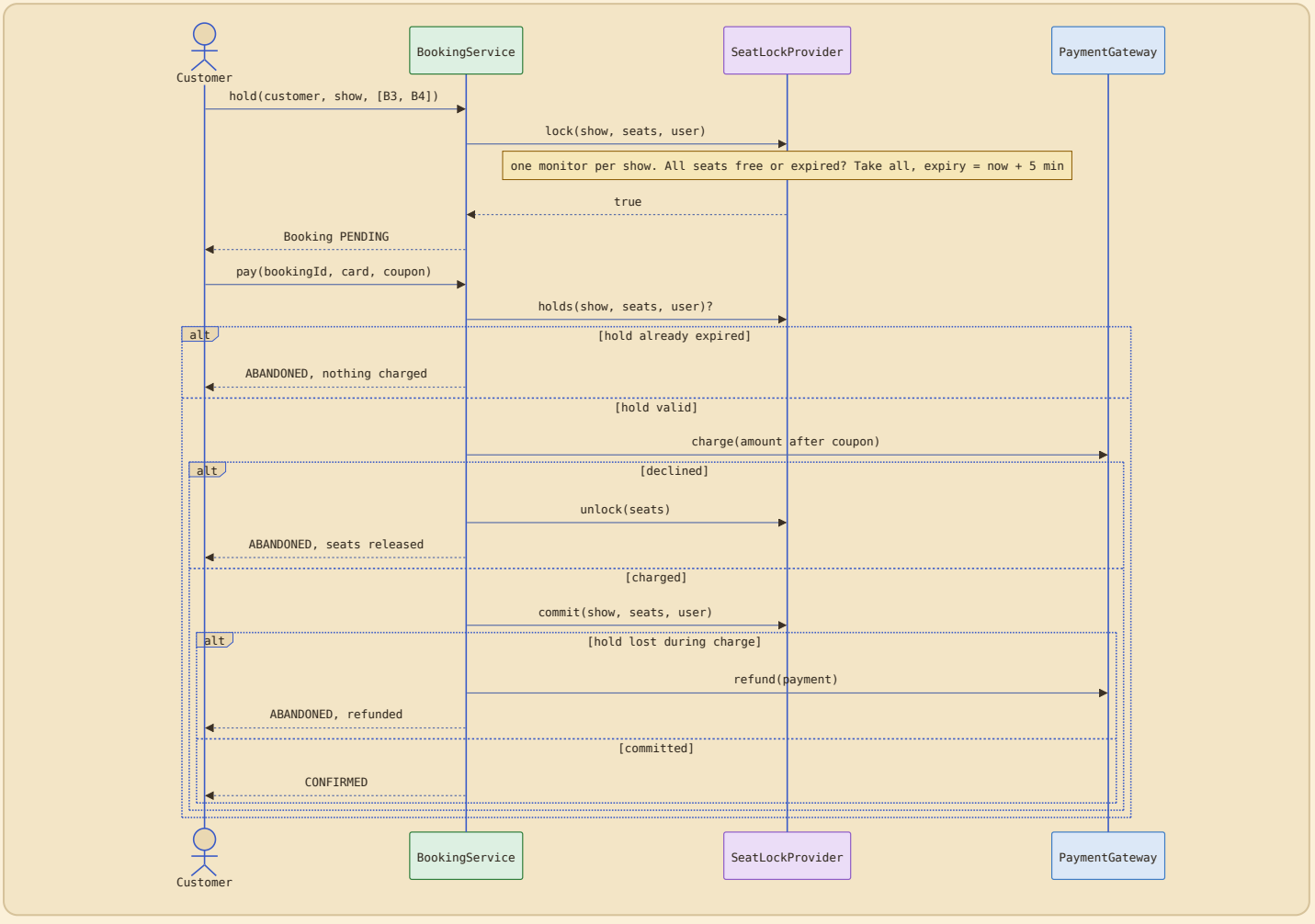


Figure — Cancel a booking: the customer gives last name and booking number; the system finds the booking, issues a refund if the cancellation is within the allowed time (otherwise no refund), cancels, updates the show's available seats, and prints or emails the details (course diagram)

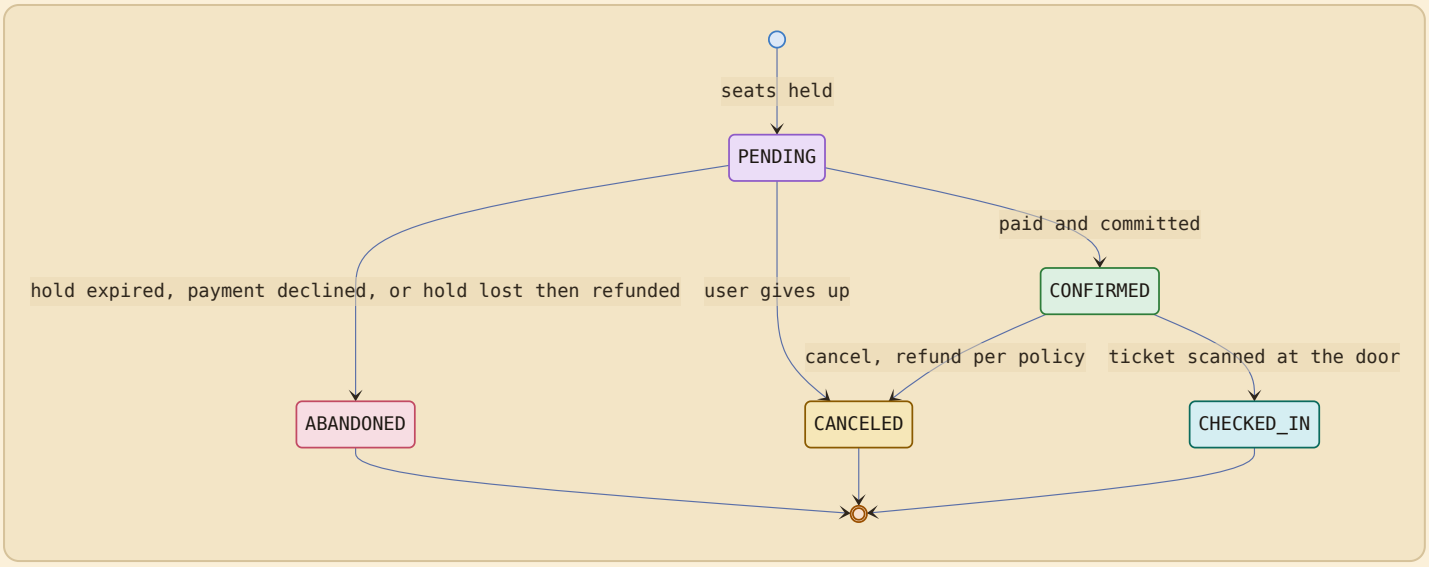
The heart of the design is a **three-step booking path**: *hold → pay → commit*.



Why `commit` must re-check inside the lock: the payment call can take seconds. If the hold expires during it and someone else grabs the seat, a naive "mark booked" would double-book. `commit` verifies "these seats are still held by me and not expired" **and** marks them booked in one atomic step. If that

fails after money was taken, we run a **compensating action** — a refund — instead of a database-style rollback (undoing unfinished changes as if they never happened), because a card charge cannot be "un-happened". This is the essence of the **Saga** idea: each step has an undo.

Booking lifecycle:



A show seat, as the customer sees it on the map: **free** (`.`), **held** by someone paying (`L`), **booked** (`X`). "Held" is not stored on the seat — it lives in the lock provider and simply disappears when it expires.

Design decisions & patterns used

Decision	Why	Alternative rejected
`ShowSeat` per show (course's key point)	The same physical seat is free in one show and taken in another; prices differ per show.	Booking `HallSeat` directly — breaks the moment there are two shows a day.
`SeatLockProvider` interface with TTL holds	Seats are kept only while the customer pays; abandoned checkouts free themselves. The interface lets you swap the in-memory version for **Redis** (a fast in-memory key-value store, often used for locks and caches; `SET key value NX PX 300000` — set only if absent, expire in 300 000 ms) or a database table with an `expires_at` column.	Holding seats until the booking is cancelled (course's version; it names this as the gap).
All-or-nothing lock	A customer asking for 4 seats together must never be left with 2. Check every seat first, then take all.	Locking seats one by one without rollback.
One monitor per show	Shows never block each other; within a show, every decision is serialized (made one at a time). Fine-grained enough for a cinema.	One global lock (the course's single-process lock) — correct but serializes the whole country's bookings.
Commit re-validates + compensation	Closes the "lock expired during payment" gap; the customer never pays for a seat they don't get.	Trusting the earlier check.
Idempotent `pay` (safe to repeat: two calls have the effect of one)	A double-click or network retry must not charge twice: paying a non-PENDING booking returns it unchanged. Real systems add an **idempotency key** per request.	Charging on every call.
Strategy for cancellation refunds and **Strategy-like `PaymentMethod`**	Refund rules and payment types vary independently of booking logic.	`if (method == CARD)` chains.
Adapter boundary (a wrapper that turns a foreign interface into ours) `PaymentGateway`	Wraps the external provider (Stripe, Razorpay…) behind our own interface; tests use `FakeGateway`.	Calling the provider's SDK (software development kit: the provider's own client library) from `BookingService`.
Coupon applied to the booking	Amount is correct before payment; payment types stay simple.	Each payment type applying coupons.
Guest has no booking method	The rule "guests must register to book" is enforced by *types*, not just written down: `hold` takes a `Customer`.	A `Guest.book()` that throws.
"Search by city" from shows	A movie is in a city only once a show is scheduled there; the other four searches are indexed when a movie is added.	Storing a city on `Movie`.

Java implementation

Package `lld.ch15.moviebooking`. Files: `Venue.java` (city, cinema, hall, seats, movie, show), `SeatLockProvider.java` (interface + in-memory TTL implementation), `Payments.java` (payment types, gateway, coupon, cancellation policy, notifier, clock), `Catalog.java` (five searches), `BookingService.java` (actors, booking, the booking path), `Main.java`.

```java
// File: lld/ch15/moviebooking/Venue.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

enum SeatType { REGULAR, PREMIUM, ACCESSIBLE, EMERGENCY_EXIT, OTHER }

record City(String name, String state) {}

/** The physical seat bolted to the floor. It has NO booking state: that depends on which show you ask about. */
record HallSeat(String id, SeatType type) {}

final class CinemaHall {
    private final String name;
    private final List<HallSeat> seats;
    private final List<Show> shows = new ArrayList<>();

    CinemaHall(String name, List<HallSeat> seats) { this.name = name; this.seats = List.copyOf(seats); }
    String name()            { return name; }
    List<HallSeat> seats()   { return seats; }

    /** A hall runs one show at a time, so an overlapping show is refused. */
    synchronized boolean addShow(Show show) {
        for (Show s : shows) if (s.overlaps(show)) return false;
        shows.add(show);
        return true;
    }
    synchronized List<Show> shows() { return List.copyOf(shows); }
}

final class Cinema {
    private final String name;
    private final City city;                      // association: the city exists without the cinema
    private final List<CinemaHall> halls = new ArrayList<>();   // composition: halls die with the cinema
    Cinema(String name, City city) { this.name = name; this.city = city; }
    String name()                 { return name; }
    City city()                   { return city; }
    void addHall(CinemaHall h)    { halls.add(h); }
    List<CinemaHall> halls()      { return Collections.unmodifiableList(halls); }
}

record Movie(String id, String title, String language, String genre, LocalDate releaseDate, int durationMinutes)
{}

/** One screening. Owns one ShowSeat per physical seat, so the same seat can be free at 6pm and taken at 9pm. */
final class Show {
    private final String id;
    private final Movie movie;
    private final Cinema cinema;
    private final CinemaHall hall;
    private final LocalDateTime start;
    private final Map<String, ShowSeat> seats = new LinkedHashMap<>();

    Show(String id, Movie movie, Cinema cinema, CinemaHall hall, LocalDateTime start, Map<SeatType, BigDecimal>
prices) {
        this.id = id; this.movie = movie; this.cinema = cinema; this.hall = hall; this.start = start;
        for (HallSeat hs : hall.seats())
            seats.put(hs.id(), new ShowSeat(hs, prices.getOrDefault(hs.type(), BigDecimal.ZERO)));
    }
    String id()                { return id; }
    Movie movie()              { return movie; }
    Cinema cinema()            { return cinema; }
    CinemaHall hall()          { return hall; }
```

```
    LocalDateTime start()       { return start; }
    LocalDateTime end()         { return start.plusMinutes(movie.durationMinutes()); }
    Map<String, ShowSeat> seats() { return Collections.unmodifiableMap(seats); }

    ShowSeat seat(String seatId) {
        ShowSeat s = seats.get(seatId);
        if (s == null) throw new IllegalArgumentException("no seat " + seatId + " in show " + id);
        return s;
    }
    boolean overlaps(Show o) { return start.isBefore(o.end()) && o.start().isBefore(end()); }
}

/** A seat for one show: its own price and its booked flag. Only SeatLockProvider flips `booked`. */
final class ShowSeat {
    private final HallSeat seat;
    private final BigDecimal price;          // money: BigDecimal, never double
    private volatile boolean booked;
    ShowSeat(HallSeat seat, BigDecimal price) { this.seat = seat; this.price = price; }
    HallSeat seat()        { return seat; }
    BigDecimal price()     { return price; }
    boolean isBooked()     { return booked; }
    void setBooked(boolean b) { booked = b; }
}
```

```java
// File: lld/ch15/moviebooking/SeatLockProvider.java
package lld.ch15.moviebooking;

import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.ConcurrentHashMap;

/**
 * Temporary, expiring holds on seats while the customer pays.
 * An interface so the in-memory version can be swapped for Redis (SET NX PX) or a DB row lock.
 */
interface SeatLockProvider {
    /** All-or-nothing: either every seat is locked for this user, or none is. */
    boolean lock(Show show, List<String> seatIds, String userId);
    /** Release this user's holds (payment failed, user gave up). */
    void unlock(Show show, List<String> seatIds, String userId);
    /** True while this user still holds every one of these seats and no hold has expired. */
    boolean holds(Show show, List<String> seatIds, String userId);
    /** Atomically: verify holds are still valid, mark seats BOOKED, drop the holds. */
    boolean commit(Show show, List<String> seatIds, String userId);
    /** Make booked seats sellable again (cancellation). */
    void releaseBooked(Show show, List<String> seatIds);
    Set<String> lockedSeats(Show show);
}

record SeatLock(String userId, Instant expiresAt) {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}

/**
 * One monitor per show: two shows never block each other, but every decision about one show's
 * seats (lock, commit, release) is made by one thread at a time. That is what makes
 * check-then-act safe and prevents double booking.
 */
final class InMemorySeatLockProvider implements SeatLockProvider {
    private final Duration ttl;
    private final Clock clock;
    private final Map<String, Map<String, SeatLock>> locksByShow = new ConcurrentHashMap<>();

    InMemorySeatLockProvider(Duration ttl, Clock clock) { this.ttl = ttl; this.clock = clock; }

    private Map<String, SeatLock> table(Show show) {
        return locksByShow.computeIfAbsent(show.id(), k -> new HashMap<>());
    }

    @Override public boolean lock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            for (String id : seatIds) {                          // 1. check every seat first
                if (show.seat(id).isBooked()) return false;
                SeatLock l = t.get(id);
                if (l != null && !l.expired(now) && !l.userId().equals(userId)) return false;
            }
            SeatLock mine = new SeatLock(userId, now.plus(ttl));
            for (String id : seatIds) t.put(id, mine);           // 2. then take them all
            return true;
        }
    }

    @Override public void unlock(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            for (String id : seatIds) {
```

```
                SeatLock l = t.get(id);
                if (l != null && l.userId().equals(userId)) t.remove(id);
            }
        }
    }

    @Override public boolean holds(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) { return holdsLocked(t, seatIds, userId, clock.instant()); }
    }

    @Override public boolean commit(Show show, List<String> seatIds, String userId) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            if (!holdsLocked(t, seatIds, userId, clock.instant())) return false;
            for (String id : seatIds) { show.seat(id).setBooked(true); t.remove(id); }
            return true;
        }
    }

    @Override public void releaseBooked(Show show, List<String> seatIds) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) { for (String id : seatIds) show.seat(id).setBooked(false); }
    }

    @Override public Set<String> lockedSeats(Show show) {
        Map<String, SeatLock> t = table(show);
        synchronized (t) {
            Instant now = clock.instant();
            Set<String> out = new TreeSet<>();
            t.forEach((id, l) -> { if (!l.expired(now)) out.add(id); });
            return out;
        }
    }

    private static boolean holdsLocked(Map<String, SeatLock> t, List<String> ids, String userId, Instant now) {
        for (String id : ids) {
            SeatLock l = t.get(id);
            if (l == null || l.expired(now) || !l.userId().equals(userId)) return false;
        }
        return true;
    }
}
```

```java
// File: lld/ch15/moviebooking/Payments.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.Set;
import java.util.concurrent.atomic.AtomicInteger;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

/** How the customer pays. Sealed: the system supports exactly these two (credit card or cash). */
sealed interface PaymentMethod permits CardPayment, CashPayment {}
record CardPayment(String cardNumberLast4) implements PaymentMethod {}
record CashPayment() implements PaymentMethod {}

record Payment(String txnId, BigDecimal amount, PaymentMethod method, PaymentStatus status) {
    Payment withStatus(PaymentStatus s) { return new Payment(txnId, amount, method, s); }
}

/** Adapter boundary to the real payment provider. */
interface PaymentGateway {
    Payment charge(String bookingId, BigDecimal amount, PaymentMethod method);
    Payment refund(Payment p);
}

/** Test double: declines listed cards, and can run a hook "while the bank is thinking". */
final class FakeGateway implements PaymentGateway {
    private final Set<String> declinedCards;
    private final AtomicInteger seq = new AtomicInteger();
    private volatile Runnable duringCharge = () -> {};

    FakeGateway(Set<String> declinedCards) { this.declinedCards = declinedCards; }
    void setDuringCharge(Runnable r) { duringCharge = r; }

    @Override public Payment charge(String bookingId, BigDecimal amount, PaymentMethod m) {
        duringCharge.run();
        String txn = "TXN-" + seq.incrementAndGet();
        boolean declined = m instanceof CardPayment c && declinedCards.contains(c.cardNumberLast4());
        return new Payment(txn, amount, m, declined ? PaymentStatus.DECLINED : PaymentStatus.COMPLETED);
    }
    @Override public Payment refund(Payment p) { return p.withStatus(PaymentStatus.REFUNDED); }
}

/** A percentage off. Applied to the booking amount BEFORE payment, so every payment type sees the right total. */
record Coupon(String code, int percentOff) {
    Coupon {
        if (percentOff < 0 || percentOff > 100) throw new IllegalArgumentException("bad percent " + percentOff);
    }
    /** Amount after the discount, rounded to whole paise/cents (half-up). */
    BigDecimal apply(BigDecimal amount) {
        BigDecimal off = amount.multiply(BigDecimal.valueOf(percentOff)).divide(BigDecimal.valueOf(100), 2,
RoundingMode.HALF_UP);
        return amount.subtract(off).setScale(2, RoundingMode.HALF_UP);
    }
}

/** Strategy: how much of the money comes back on cancellation. */
interface CancellationPolicy {
    BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart);
}

/** Full refund up to `cutoff` before the show, nothing after. */
record CutoffRefundPolicy(Duration cutoff) implements CancellationPolicy {
    @Override public BigDecimal refund(BigDecimal paid, LocalDateTime now, LocalDateTime showStart) {
```

```
        return now.plus(cutoff).isAfter(showStart) ? BigDecimal.ZERO : paid;
    }
}

interface Notifier { void send(String userId, String message); }

final class ConsoleNotifier implements Notifier {
    @Override public void send(String userId, String message) {
        System.out.println("  [notify " + userId + "] " + message);
    }
}

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}

final class Money {
    private Money() {}
    static String fmt(BigDecimal amount) { return "Rs." + amount.setScale(2,
RoundingMode.UNNECESSARY).toPlainString(); }
}
```

```java
// File: lld/ch15/moviebooking/Catalog.java
package lld.ch15.moviebooking;

import java.time.LocalDate;
import java.util.Comparator;
import java.util.List;
import java.util.Locale;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;

/** The five searches. Four are indexed when a movie is added; "by city" is answered from the shows. */
interface MovieSearch {
    List<Movie> searchByTitle(String title);
    List<Movie> searchByLanguage(String language);
    List<Movie> searchByGenre(String genre);
    List<Movie> searchByReleaseDate(LocalDate date);
    List<Movie> searchByCity(String cityName);
}

final class Catalog implements MovieSearch {
    private final Map<String, List<Movie>> byTitle = new ConcurrentHashMap<>();
    private final Map<String, List<Movie>> byLanguage = new ConcurrentHashMap<>();
    private final Map<String, List<Movie>> byGenre = new ConcurrentHashMap<>();
    private final Map<LocalDate, List<Movie>> byRelease = new ConcurrentHashMap<>();
    private final List<Show> shows = new CopyOnWriteArrayList<>();
    private final Notifier notifier;

    Catalog(Notifier notifier) { this.notifier = notifier; }

    /** Admin adds a movie: index it four ways and announce it (requirement: notify on a new movie). */
    void addMovie(Movie m) {
        put(byTitle, key(m.title()), m);
        put(byLanguage, key(m.language()), m);
        put(byGenre, key(m.genre()), m);
        byRelease.computeIfAbsent(m.releaseDate(), d -> new CopyOnWriteArrayList<>()).add(m);
        notifier.send("subscribers", "new movie: " + m.title() + " (" + m.language() + ", " + m.genre() + ")");
    }

    /** Admin schedules a show; the hall refuses it if it overlaps another. */
    boolean addShow(Show s) {
        if (!s.hall().addShow(s)) return false;
        shows.add(s);
        return true;
    }

    /** After picking a movie: which shows play in my city? */
    List<Show> showsOf(Movie m, String cityName) {
        return shows.stream()
                .filter(s -> s.movie().equals(m) && s.cinema().city().name().equalsIgnoreCase(cityName))
                .sorted(Comparator.comparing(Show::start)).toList();
    }

    @Override public List<Movie> searchByTitle(String t)    { return get(byTitle, key(t)); }
    @Override public List<Movie> searchByLanguage(String l) { return get(byLanguage, key(l)); }
    @Override public List<Movie> searchByGenre(String g)    { return get(byGenre, key(g)); }
    @Override public List<Movie> searchByReleaseDate(LocalDate d) { return List.copyOf(byRelease.getOrDefault(d,
List.of())); }
    @Override public List<Movie> searchByCity(String city) {
        return shows.stream().filter(s -> s.cinema().city().name().equalsIgnoreCase(city))
                .map(Show::movie).distinct().sorted(Comparator.comparing(Movie::title)).toList();
    }

    private static String key(String s) { return s.trim().toLowerCase(Locale.ROOT); }
    private static void put(Map<String, List<Movie>> idx, String k, Movie m) {
        idx.computeIfAbsent(k, x -> new CopyOnWriteArrayList<>()).add(m);
    }
    private static List<Movie> get(Map<String, List<Movie>> idx, String k) {
        return idx.getOrDefault(k, List.of()).stream().sorted(Comparator.comparing(Movie::title)).toList();
```

```
    }
}
```

```java
// File: lld/ch15/moviebooking/BookingService.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

enum BookingStatus { PENDING, CONFIRMED, CHECKED_IN, CANCELED, ABANDONED }

enum AccountStatus { ACTIVE, BLOCKED, BANNED, COMPROMISED, ARCHIVED, UNKNOWN }

/** Only registered customers can book. A guest has no type that BookingService accepts. */
final class Customer {
    private final String id;
    private final String name;
    private volatile AccountStatus status = AccountStatus.ACTIVE;
    Customer(String id, String name) { this.id = id; this.name = name; }
    String id()                 { return id; }
    String name()               { return name; }
    AccountStatus status()      { return status; }
    void setStatus(AccountStatus s) { status = s; }
}

record Guest(String sessionId) {
    Customer register(String id, String name) { return new Customer(id, name); }
}

record FrontDeskOfficer(String id, String name) {}

record Admin(String id, String name) {
    void block(Customer c)   { c.setStatus(AccountStatus.BLOCKED); }
    void unblock(Customer c) { c.setStatus(AccountStatus.ACTIVE); }
}

final class SeatUnavailableException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    SeatUnavailableException(String msg) { super(msg); }
}

/** A booking points at the show and seats; it owns none of them. Transitions run under its own monitor. */
final class Booking {
    private final String id;
    private final String customerId;
    private final Show show;
    private final List<String> seatIds;
    private final BigDecimal baseAmount;
    private BookingStatus status = BookingStatus.PENDING;
    private Payment payment;
    private Coupon coupon;

    Booking(String id, String customerId, Show show, List<String> seatIds, BigDecimal baseAmount) {
        this.id = id; this.customerId = customerId; this.show = show;
        this.seatIds = List.copyOf(seatIds); this.baseAmount = baseAmount;
    }
    String id()               { return id; }
    String customerId()       { return customerId; }
    Show show()               { return show; }
    List<String> seatIds()    { return seatIds; }
    BigDecimal baseAmount()   { return baseAmount; }
    synchronized BookingStatus status() { return status; }
    synchronized Payment payment()      { return payment; }
    synchronized Coupon coupon()        { return coupon; }
    void setStatus(BookingStatus s)     { status = s; }          // called with the monitor held
    void setPayment(Payment p)          { payment = p; }
    void setCoupon(Coupon c)            { coupon = c; }
```

```
    @Override public synchronized String toString() {
        return id + " " + seatIds + " " + status + (payment == null ? "" : " " + payment.status() + " " +
Money.fmt(payment.amount()));
    }
}

/**
 * The booking path: HOLD seats (TTL) -> PAY -> COMMIT seats. If the hold was lost while the bank
 * was charging, the money is refunded (compensation) instead of double-selling the seat.
 */
final class BookingService {
    private final SeatLockProvider locks;
    private final PaymentGateway gateway;
    private final CancellationPolicy cancellationPolicy;
    private final Notifier notifier;
    private final Clock clock;
    private final Map<String, Booking> bookings = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    BookingService(SeatLockProvider locks, PaymentGateway gateway, CancellationPolicy policy, Notifier notifier,
Clock clock) {
        this.locks = locks; this.gateway = gateway; this.cancellationPolicy = policy;
        this.notifier = notifier; this.clock = clock;
    }

    /** Step 1: hold the seats. All-or-nothing. Throws if any seat is booked or held by someone else. */
    Booking hold(Customer c, Show show, List<String> seatIds) {
        if (c.status() != AccountStatus.ACTIVE) throw new IllegalStateException(c.id() + " is " + c.status());
        if (seatIds.isEmpty()) throw new IllegalArgumentException("no seats");
        List<String> sorted = seatIds.stream().distinct().sorted().toList();
        if (!locks.lock(show, sorted, c.id()))
            throw new SeatUnavailableException(sorted + " not available for " + show.id());
        BigDecimal base = sorted.stream().map(id -> show.seat(id).price()).reduce(BigDecimal.ZERO,
BigDecimal::add);
        Booking b = new Booking("BK-" + seq.incrementAndGet(), c.id(), show, sorted, base);
        bookings.put(b.id(), b);
        return b;
    }

    /** The front desk books for a walk-in customer: same flow, different actor. */
    Booking holdFor(FrontDeskOfficer officer, Customer c, Show show, List<String> seatIds) {
        return hold(c, show, seatIds);
    }

    /** Step 2 + 3: pay, then commit. Idempotent: paying a confirmed booking again returns it unchanged. */
    Booking pay(String bookingId, PaymentMethod method, Coupon coupon) {
        Booking b = get(bookingId);
        synchronized (b) {
            if (b.status() != BookingStatus.PENDING) return b;
            if (!locks.holds(b.show(), b.seatIds(), b.customerId())) {       // fast fail, no charge
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired before payment, nothing charged");
                return b;
            }
            b.setCoupon(coupon);
            BigDecimal amount = coupon == null ? b.baseAmount() : coupon.apply(b.baseAmount());
            Payment p = gateway.charge(b.id(), amount, method);
            b.setPayment(p);
            if (p.status() != PaymentStatus.COMPLETED) {
                locks.unlock(b.show(), b.seatIds(), b.customerId());           // free seats at once
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " payment " + p.status() + ", seats released");
                return b;
            }
            if (!locks.commit(b.show(), b.seatIds(), b.customerId())) {       // hold lost during charge
                b.setPayment(gateway.refund(p));                              // COMPENSATION
                b.setStatus(BookingStatus.ABANDONED);
                notifier.send(b.customerId(), b.id() + " hold expired during payment, " + Money.fmt(amount) + "
refunded");
                return b;
            }
```

```
            b.setStatus(BookingStatus.CONFIRMED);
            notifier.send(b.customerId(), b.id() + " CONFIRMED " + b.seatIds() + " for '" +
b.show().movie().title()
                    + "' at " + b.show().start() + ", paid " + Money.fmt(amount));
            return b;
        }
    }

    /** Cancel: release seats and refund according to the policy. */
    Booking cancel(String bookingId) {
        Booking b = get(bookingId);
        synchronized (b) {
            switch (b.status()) {
                case PENDING -> {
                    locks.unlock(b.show(), b.seatIds(), b.customerId());
                    b.setStatus(BookingStatus.CANCELED);
                }
                case CONFIRMED -> {
                    locks.releaseBooked(b.show(), b.seatIds());
                    BigDecimal refund = cancellationPolicy.refund(b.payment().amount(),
                            LocalDateTime.now(clock), b.show().start());
                    if (refund.signum() > 0) b.setPayment(gateway.refund(b.payment()));
                    b.setStatus(BookingStatus.CANCELED);
                    notifier.send(b.customerId(), b.id() + " canceled, refund " + Money.fmt(refund));
                }
                default -> throw new IllegalStateException(b.id() + " cannot be canceled from " + b.status());
            }
            return b;
        }
    }

    Booking get(String id) {
        Booking b = bookings.get(id);
        if (b == null) throw new IllegalArgumentException("no booking " + id);
        return b;
    }

    /** "." free, "L" held by someone paying, "X" booked. */
    String seatMap(Show show) {
        Set<String> held = locks.lockedSeats(show);
        StringBuilder sb = new StringBuilder();
        char row = 0;
        for (ShowSeat s : show.seats().values()) {
            String id = s.seat().id();
            if (id.charAt(0) != row) { if (row != 0) sb.append('\n'); row = id.charAt(0); sb.append("
").append(row).append(' '); }
            sb.append(s.isBooked() ? 'X' : held.contains(id) ? 'L' : '.').append(' ');
        }
        return sb.toString();
    }
}
```

```java
// File: lld/ch15/moviebooking/Main.java
package lld.ch15.moviebooking;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-05-01T12:00:00Z"));
        FakeGateway bank = new FakeGateway(Set.of("0002"));
        SeatLockProvider locks = new InMemorySeatLockProvider(Duration.ofMinutes(5), clock);
        Notifier notifier = new ConsoleNotifier();
        BookingService svc = new BookingService(locks, bank, new CutoffRefundPolicy(Duration.ofHours(2)),
                notifier, clock);
        Catalog catalog = new Catalog(notifier);

        // ---- venue: 2 rows x 5 seats; row A regular, row B premium
        City blr = new City("Bengaluru", "KA");
        Cinema pvr = new Cinema("PVR Forum", blr);
        List<HallSeat> seats = new ArrayList<>();
        for (char row : new char[]{'A', 'B'})
            for (int n = 1; n <= 5; n++) seats.add(new HallSeat("" + row + n, row == 'A' ? SeatType.REGULAR :
SeatType.PREMIUM));
        CinemaHall audi1 = new CinemaHall("Audi 1", seats);
        pvr.addHall(audi1);

        Movie inception = new Movie("MV1", "Inception", "English", "Sci-Fi", LocalDate.of(2010, 7, 16), 148);
        Movie rrr = new Movie("MV2", "RRR", "Telugu", "Action", LocalDate.of(2022, 3, 25), 187);
        Map<SeatType, BigDecimal> prices = Map.of(SeatType.REGULAR, new BigDecimal("200.00"),
                SeatType.PREMIUM, new BigDecimal("350.00"));
        Show evening = new Show("SH1", inception, pvr, audi1, LocalDateTime.of(2026, 5, 1, 18, 0), prices);
        Show clash = new Show("SH2", rrr, pvr, audi1, LocalDateTime.of(2026, 5, 1, 19, 0), prices);

        System.out.println("== 1. Admin adds movies (subscribers told) and schedules shows; the hall refuses an
overlap ==");
        catalog.addMovie(inception);
        catalog.addMovie(rrr);
        System.out.println("  add SH1 Inception 18:00 -> " + catalog.addShow(evening));
        System.out.println("  add SH2 RRR 19:00 same hall -> " + catalog.addShow(clash));

        System.out.println("\n== 2. Search ==");
        System.out.println("  city Bengaluru -> " + titles(catalog.searchByCity("bengaluru")));
        System.out.println("  genre Action   -> " + titles(catalog.searchByGenre("action")) + " (no show yet, but
indexed)");
        System.out.println("  Inception shows in Bengaluru -> "
                + catalog.showsOf(inception, "Bengaluru").stream().map(s -> s.id() + "@" + s.start()).toList());

        System.out.println("\n== 3. 10 customers click B3+B4 at the same instant ==");
        List<Customer> crowd = new ArrayList<>();
        for (int i = 1; i <= 10; i++) crowd.add(new Customer("U" + i, "User " + i));
        ConcurrentLinkedQueue<Booking> won = new ConcurrentLinkedQueue<>();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(10)) {
            for (Customer c : crowd)
                pool.submit(() -> {
                    go.await();
                    try { won.add(svc.hold(c, evening, List.of("B3", "B4"))); }
                    catch (SeatUnavailableException ignored) { }
                    return null;
```

```
                });
            go.countDown();
        }
        Booking first = won.peek();
        System.out.println("  holds granted = " + won.size() + " (the other 9 saw 'not available')");
        System.out.println(svc.seatMap(evening));

        System.out.println("\n== 4. The winner pays by card with a 10% coupon ==");
        svc.pay(first.id(), new CardPayment("4242"), new Coupon("FIRST10", 10));
        System.out.println("  paying twice is harmless -> " + svc.pay(first.id(), new CardPayment("4242"),
null).status());
        System.out.println(svc.seatMap(evening));

        System.out.println("\n== 5. Hold expires (TTL 5 min) before paying ==");
        Customer asha = new Customer("ASHA", "Asha"), ravi = new Customer("RAVI", "Ravi");
        Booking slow = svc.hold(asha, evening, List.of("A1", "A2"));
        clock.advance(Duration.ofMinutes(6));
        Booking fast = svc.hold(ravi, evening, List.of("A1", "A2"));
        System.out.println("  Ravi re-held A1,A2 after Asha's hold expired -> " + fast.id());
        svc.pay(slow.id(), new CardPayment("4242"), null);
        svc.pay(fast.id(), new CashPayment(), null);

        System.out.println("\n== 6. Hold expires WHILE the bank is charging -> refund (compensation) ==");
        Customer meena = new Customer("MEENA", "Meena"), kiran = new Customer("KIRAN", "Kiran");
        Booking victim = svc.hold(meena, evening, List.of("A5"));
        bank.setDuringCharge(() -> {                        // bank is slow: 6 minutes pass, Kiran grabs A5
            clock.advance(Duration.ofMinutes(6));
            svc.hold(kiran, evening, List.of("A5"));
        });
        svc.pay(victim.id(), new CardPayment("4242"), null);
        bank.setDuringCharge(() -> {});
        System.out.println("  " + victim);

        System.out.println("\n== 7. Declined card releases the seats immediately ==");
        Booking declined = svc.hold(new Customer("NEHA", "Neha"), evening, List.of("B1"));
        svc.pay(declined.id(), new CardPayment("0002"), null);
        System.out.println("  B1 held by anyone now? " + locks.lockedSeats(evening).contains("B1"));

        System.out.println("\n== 8. Blocked customer, then cancellation with refund ==");
        new Admin("AD1", "Admin").block(crowd.get(9));
        try { svc.hold(crowd.get(9), evening, List.of("B5")); }
        catch (IllegalStateException e) { System.out.println("  REFUSED: " + e.getMessage()); }
        svc.cancel(first.id());                             // 12:06+ vs 18:00 show: more than 2h before
        System.out.println("  " + first);
        System.out.println(svc.seatMap(evening));
    }

    private static List<String> titles(List<Movie> ms) { return ms.stream().map(Movie::title).toList(); }
}
```

Output:

```
== 1. Admin adds movies (subscribers told) and schedules shows; the hall refuses an overlap ==
  [notify subscribers] new movie: Inception (English, Sci-Fi)
  [notify subscribers] new movie: RRR (Telugu, Action)
  add SH1 Inception 18:00 -> true
  add SH2 RRR 19:00 same hall -> false

== 2. Search ==
  city Bengaluru -> [Inception]
  genre Action   -> [RRR] (no show yet, but indexed)
  Inception shows in Bengaluru -> [SH1@2026-05-01T18:00]

== 3. 10 customers click B3+B4 at the same instant ==
  holds granted = 1 (the other 9 saw 'not available')
  A . . . . .
  B . . L L .

== 4. The winner pays by card with a 10% coupon ==
  [notify U10] BK-1 CONFIRMED [B3, B4] for 'Inception' at 2026-05-01T18:00, paid Rs.630.00
  paying twice is harmless -> CONFIRMED
  A . . . . .
  B . . X X .

== 5. Hold expires (TTL 5 min) before paying ==
  Ravi re-held A1,A2 after Asha's hold expired -> BK-3
  [notify ASHA] BK-2 hold expired before payment, nothing charged
  [notify RAVI] BK-3 CONFIRMED [A1, A2] for 'Inception' at 2026-05-01T18:00, paid Rs.400.00

== 6. Hold expires WHILE the bank is charging -> refund (compensation) ==
  [notify MEENA] BK-4 hold expired during payment, Rs.200.00 refunded
  BK-4 [A5] ABANDONED REFUNDED Rs.200.00

== 7. Declined card releases the seats immediately ==
  [notify NEHA] BK-6 payment DECLINED, seats released
  B1 held by anyone now? false

== 8. Blocked customer, then cancellation with refund ==
  REFUSED: U10 is BLOCKED
  [notify U10] BK-1 canceled, refund Rs.630.00
  BK-1 [B3, B4] CANCELED REFUNDED Rs.630.00
  A X X . . L
  B . . . . .
```

What to notice: section 1 announces each new movie to subscribers (requirement 9) and refuses an overlapping show in the same hall (requirement 2); section 3 lets exactly 1 of 10 simultaneous customers hold B3+B4; section 4 applies the 10% coupon (Rs.700 → Rs.630) and a second `pay` is a no-op; section 5 shows an expired hold being re-sold and the slow customer *not* being charged; section 6 is the compensation path — Meena's card was charged, but her hold expired during the charge and Kiran took A5, so she is refunded; section 7 releases seats immediately on a decline; section 8 blocks a customer and cancels with a full refund.

Concurrency & edge cases

- **Check-then-act is atomic.** `lock`, `holds`, `commit`, `unlock` and `releaseBooked` for one show all run inside `synchronized` on that show's lock table. "Is the seat free? → take it" can't interleave.
- **Expired holds are lazy.** There is no background thread; an expired `SeatLock` is simply treated as absent the next time anyone looks. (In Redis, the key's TTL does the same.)
- **The payment-window race** (hold expires mid-payment) is handled by commit re-validation + refund, as shown in section 6.
- **Lock granularity** (how much data one lock covers)**.** The booking's own monitor makes `pay`/`cancel` on the same booking serial (no double charge, no pay-while-cancel). The show monitor protects seats. The two are always taken in the order booking → show, never the reverse, so no deadlock.
- **Across many servers** (real BookMyShow), in-process locks are not enough. Options, from simplest:
  1. **Conditional update** (course): `UPDATE show_seat SET status='HELD', held_by=?, held_until=? WHERE show_id=? AND seat_id IN (...) AND (status='FREE' OR held_until < now())`, then check the row count equals the

number of seats; if not, roll back.

2. **Pessimistic lock** (lock first, then work, assuming conflicts are likely): `SELECT ... FOR UPDATE` on the seat rows inside a transaction (a group of database statements that succeed or fail together).
3. **Serializable transaction** (course): the result is as if each transaction had run alone, in some order — easiest to reason about, but the slowest option when many buyers collide.
4. **Optimistic locking**: a `version` column; update `WHERE version = ?`; retry on conflict. Good when conflicts are rare.
5. **Unique constraint** (a database rule that rejects a second row with the same values) on `(show_id, seat_id)` in the confirmed-tickets table as the final safety net.
6. **Distributed lock / hold store** (a lock kept in a shared store, so all servers see it) in Redis with TTL for the *hold* phase, with the database as the source of truth (the one copy that wins when copies disagree) for *confirmed* seats. The course's point, in short: the in-process lock only makes the common case quick; once there are two servers, correctness comes from the database.

- **Edge cases:** empty seat list; duplicate seat ids in one request (de-duplicated); unknown seat; blocked customer; overlapping show in a hall; cancelling an abandoned booking (rejected); cancelling too late (no refund).

Extensions & follow-up questions

- **"Show the seat map live."** Push seat changes (held/booked/released) to open seat-map pages over WebSockets or server-sent events (two ways for a server to push updates to an open browser page); the lock provider publishes events (Observer).
- **"Extend the hold when the user reaches the payment page."** Add `extend(show, seats, user, extra)` that refreshes `expiresAt` only if the caller still holds the lock.
- **"Dynamic pricing / different prices per seat type and time."** Replace the price map with a `PricingStrategy` (base by `SeatType`, surcharge for weekends or premieres).
- **"Payment succeeded but our server crashed before commit."** Store the booking as `PENDING` with the gateway's transaction id before charging; a reconciliation job (a periodic task that compares our records with the gateway's and fixes mismatches) queries the gateway for pending bookings and either commits (if the seats are still held) or refunds. Payment providers' **webhooks** (HTTP calls the provider makes to your server when something happens) notify you of the final result.
- **"Cancel a whole show."** Admin cancels → every confirmed booking of the show is cancelled with a full refund and a notification (iterate bookings by show).
- **"Loyalty points, food & beverage add-ons."** Extra line items on the booking; points as another `PaymentMethod`.
- **"Seat gaps rule (no single empty seat left between bookings)."** A validation step in `hold` before locking — a `SeatSelectionRule` chain (Chain of Responsibility).

Real-world example

On **BookMyShow**, after you pick seats the app shows a **countdown timer** on the payment screen; if it runs out the seats are released and you must start again — that timer is the seat hold's TTL. If your bank debits you but the booking does not complete, the app tells you the amount will be **refunded automatically** — that is the compensation path. Big releases put the "one monitor per show" idea under real stress: first-day-first-show bookings for a blockbuster concentrate thousands of users on a handful of shows.

In the interview

30–45 minute plan. (1) 4 min — requirements; ask about hold time, payment, refunds. (2) 4 min — actors/use cases. (3) 8 min — entities: city → cinema → hall → seat; movie → show → **show seat** (say why). (4) 12 min — the booking path: `SeatLockProvider` (all-or-nothing, TTL), `BookingService.hold/pay/cancel`; write `lock` and `commit`. (5) 5 min — payment failure and the expired-during-payment refund. (6) Remaining — multi-server concurrency (conditional update / Redis TTL), follow-ups.

> **30-second answer:** "Venue hierarchy is City → Cinema → Hall → HallSeat. A Show owns a ShowSeat per hall seat, and that's what's booked. Booking is hold → pay → commit: a SeatLockProvider takes all requested seats atomically with a 5-minute TTL; payment goes through a gateway interface; commit re-checks the hold and marks seats booked atomically, and if the hold was lost we refund. In one process a per-show monitor guarantees no double booking; across servers the database's conditional update or a unique constraint is the real guarantee, with Redis TTL keys for holds."

Follow-ups with answers: *Why not lock the whole show table?* Fine for one cinema, but a global lock serializes all shows. *Why TTL?* Without it an abandoned checkout blocks seats forever. *Optimistic or pessimistic?* Pessimistic/conditional-update for hot shows (conflicts common); optimistic when conflicts are rare.

Self-check

- **Q:** Why is `ShowSeat` separate from `HallSeat`? **A:** Booking state and price are per show; the physical seat is shared by every show in the hall.
- **Q:** What does "all-or-nothing" mean in `lock`? **A:** Check every requested seat first; take them all only if all are free — never a partial set.
- **Q:** A payment succeeded but `commit` returned false. What now? **A:** Refund (compensation) and mark the booking abandoned.
- **Q:** How does an expired hold get cleaned up here? **A:** Lazily: any lock/holds check treats an expired lock as absent.
- **Q:** What makes `pay` safe to call twice? **A:** It runs under the booking's monitor and returns immediately unless the booking is `PENDING`.

TL;DR

Book `ShowSeat`s, not seats. Hold (all-or-nothing, TTL) → pay → commit (re-validate atomically) → refund if the hold was lost. Coupon on the booking, gateway behind an interface, refund rule as a Strategy. Per-show lock in one JVM (Java Virtual Machine — one running Java process); conditional updates, unique constraints and Redis TTL keys across servers.

15.3 Design a Concert Ticket Booking System

Problem & clarifying questions

A concert ticketing system (think **Ticketmaster**, **Live Nation**, **SeatGeek**, **BookMyShow Live**) sells tickets for a single, huge, one-off event. The *objects* look like section 15.2 — an event, a venue, seats, bookings, payments — so we reuse those ideas and do **not** repeat them. The *focus* is different: **extreme contention**. A movie show has a few hundred seats and demand spread over days; a stadium tour sells 50,000 tickets in minutes to a million people who all arrive at 10:00:00. That changes three things:

1. **Fairness** — who gets to try first? (a *virtual waiting room*)
2. **Throughput under contention** (throughput = requests completed per second; see System Design Fundamentals) — locks that serialize everyone become the bottleneck (the one point that limits overall speed) (so we use *lock-free* compare-and-set)
3. **Abuse** — scalpers (people who buy tickets to resell at a markup) and bots (automated scripts) (so *per-user limits*), and what happens to fans who miss out (a *waitlist*)

Question	Answer we assume
Seat types?	Tiers: **VIP** and **PREMIUM** are numbered seats; **GENERAL_ADMISSION** (GA) is a standing area — just a *count*.
Search?	By artist, venue, date and time (same catalog idea as 15.2 — not repeated).
How do we stop a stampede?	A **virtual waiting room**: everyone gets a queue number; people are admitted in batches, in order.
Limit per person?	**4 tickets** per user per concert.
Hold time?	**10 minutes** to pay.
Sold out?	Fans can join a **waitlist** per tier; released tickets are offered to the waitlist first.
Notifications?	Confirmation and "a ticket freed up for you" (email/SMS).

Requirements

Functional — show concerts with their seating layout and availability; search by artist, venue, date, time; choose seats (or a GA quantity) and buy; **prevent double booking under concurrent requests**; **fair access** for all users; secure payments; confirmation notifications; **waitlist** for sold-out events.

Non-functional — very high write contention for a few minutes; **no overselling** ever (GA count never below zero, a seat never sold twice); fairness (first-come-first-served into the sale); bounded per-user purchases; holds that expire on their own.

Use cases

Actors: **Fan** (join queue, hold, buy, cancel, join waitlist), **Organizer/Admin** (create concert, set tiers and prices, open the sale), **System** (admit batches from the queue, expire holds, offer released tickets to the waitlist, notify), **Payment provider** (external).

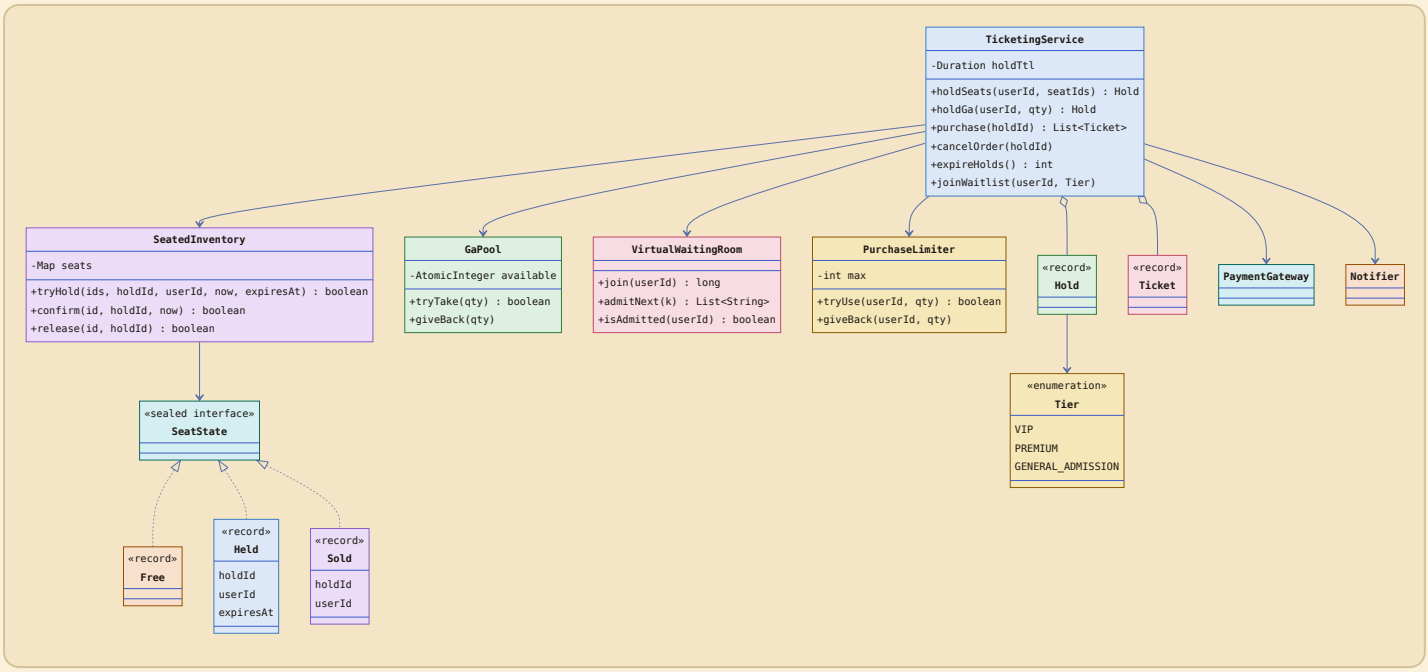
Main use cases: join the waiting room → get admitted → **hold** seats or a GA quantity → **purchase** → receive tickets; **cancel** an order (refund); **join waitlist**; system **reaps** expired holds.

Core entities

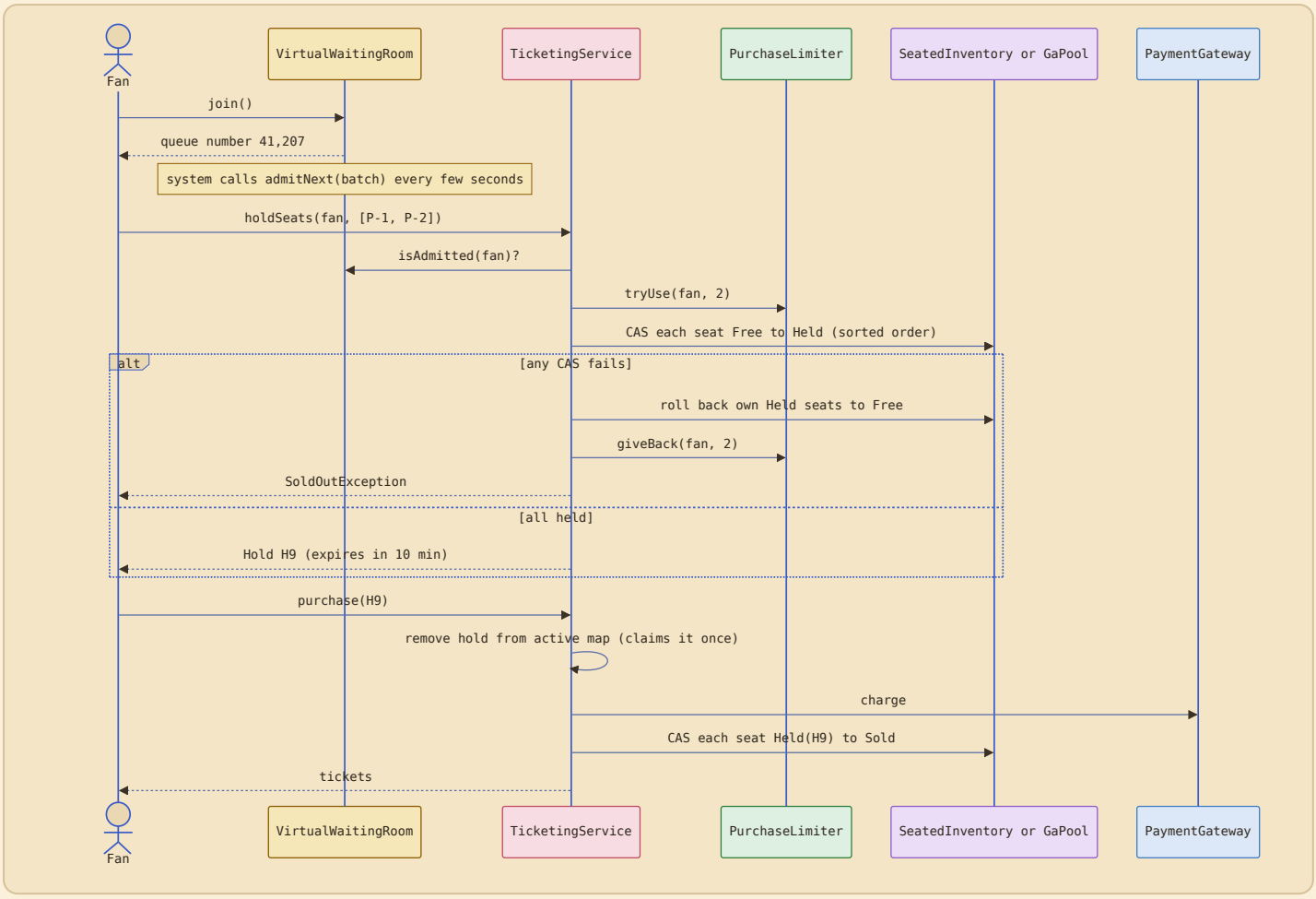
Class	Responsibility
`Concert` (record)	Artist, venue, date/time.
`Tier` (enum)	`VIP`, `PREMIUM` (seated) and `GENERAL_ADMISSION` (unseated), each with a price.
`SeatState` (sealed) → `Free`, `Held`, `Sold`	A seat's whole state as one immutable value.
`SeatedInventory`	One `AtomicReference<SeatState>` (a holder for one object reference that supports CAS) per numbered seat; claims by **CAS**.
`GaPool`	GA capacity as an `AtomicInteger` (an int with atomic updates); take/give back by CAS.
`Hold` (record)	A temporary claim: seats or a GA quantity, amount, expiry.
`Ticket` (record)	One admission: tier and seat id or "GA".
`VirtualWaitingRoom`	Queue numbers in arrival order; admits batches FIFO.
`PurchaseLimiter`	At most N tickets per user, checked and updated atomically.
`TicketingService`	hold → purchase → (cancel); reaper; waitlist offers.
`PaymentGateway`, `Notifier`	External boundaries (as in 15.2).

Class diagram

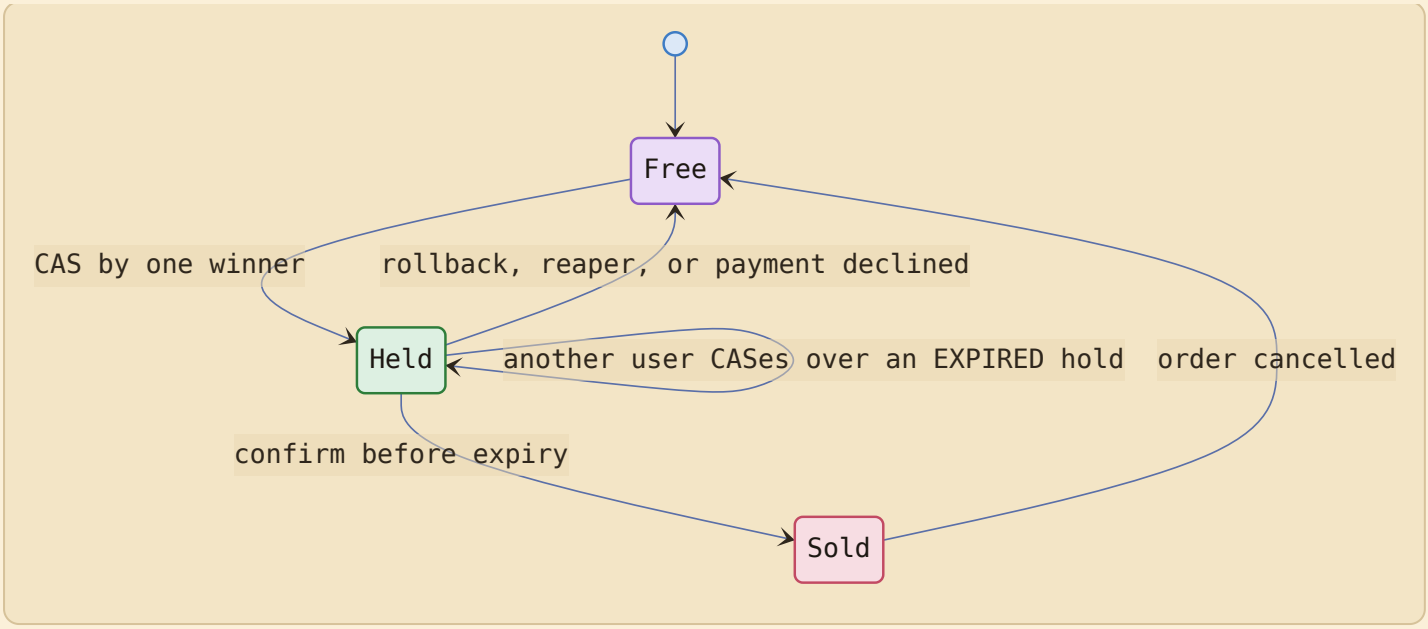
There is no course lesson for this problem; the problem statement comes from the *awesome-low-level-design* problem set. Our design:



Key flows



A numbered seat's state machine — every arrow is a single `compareAndSet`:



When inventory is released (expired hold, cancellation), `TicketingService` **offers it to the tier's waitlist first**: the next waiting fan is admitted directly and given a fresh hold, plus a notification. If they do not pay in time, the reaper releases it again and the next fan gets it.

Design decisions & patterns used

Decision	Why	Alternative rejected
Lock-free CAS per seat (`AtomicReference<SeatState>`)	Buyers of *different* seats never wait for each other; on the *same* seat, exactly one `compareAndSet` wins and the losers learn instantly. No thread is ever blocked holding a lock, so there is no deadlock.	One monitor per concert (15.2's approach) — correct, but at 50,000 simultaneous buyers every request queues behind one lock.
Immutable sealed `SeatState`	CAS compares *references* (which object it is, not what it contains); making each state a fresh immutable record means "the state I read" is exactly "the state I replace". Pattern-matching `switch` / `instanceof` on a sealed type is exhaustive and readable.	Mutable fields (`status`, `heldBy`, `expiry`) updated separately — a reader could see a half-updated seat.
Multi-seat hold in sorted order + rollback	Each seat is claimed independently; on failure we release the ones *we* took (matching our hold id). Sorted order makes two buyers of overlapping sets contend on the same first seat, so one of them fails fast.	Holding several seats with nested locks — deadlock-prone without ordering.
GA as a counter	Standing places are interchangeable — there's nothing to name. A CAS loop on an `AtomicInteger` (`if cur < qty fail; else CAS(cur, cur-qty)`) never oversells.	A `Seat` object per standing place — 40,000 objects for no benefit.
Virtual waiting room	Fairness and load control: arrival order decides who gets to *try*, and admitting in batches caps the load on inventory. Bots gain nothing by hammering "buy". `join` is `synchronized` so number order and queue order can never disagree.	"Whoever's request lands first" — rewards bots and fast connections, and melts the backend (the servers behind the app).
Per-user limit via `ConcurrentHashMap.compute` (a thread-safe map; `compute` updates one key atomically)	`compute` runs the check-and-update for one key atomically, so two parallel requests from the same user can't both slip under the limit.	`get` then `put` — a classic race.
Holds with TTL + a reaper	Seat holds are also reclaimed lazily (a CAS can overwrite an expired `Held`); GA holds have no per-item state, so a periodic `expireHolds()` returns their count. `holds.remove(id, h)` ensures the reaper and `purchase` never both process one hold.	Holds that never expire.
Waitlist offers on release	Released tickets go to the longest-waiting fan (FIFO), not to whoever refreshes fastest.	Returning tickets silently to the pool.
Compensation on lost hold	Same as 15.2: if a seat could not be confirmed after payment, release and refund.	Assuming the hold is still valid.
Singleton?	The *awesome-lld* reference solution makes the booking system a Singleton. We pass collaborators through the constructor instead (dependency injection): it is easier to test and allows one service per concert.	A global Singleton.

Java implementation

Package `lld.ch15.concert`. Files: `Model.java` (tiers, seat states, hold, ticket, interfaces, clock), `Inventory.java` (CAS seat inventory, GA pool, purchase limiter, waiting room), `TicketingService.java`, `Main.java`. The demos use Java 21 **virtual threads** (lightweight threads managed by the JVM rather than

the operating system) (`Executors.newVirtualThreadPerTaskExecutor()`), which make 1,000 concurrent buyers cheap.

```java
// File: lld/ch15/concert/Model.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.List;

/** Seated tiers have numbered seats; general admission (GA) is just a count of standing places. */
enum Tier {
    VIP(true, "5000.00"), PREMIUM(true, "2500.00"), GENERAL_ADMISSION(false, "800.00");
    final boolean seated;
    final BigDecimal price;                       // money: BigDecimal, never double
    Tier(boolean seated, String price) { this.seated = seated; this.price = new BigDecimal(price); }
    BigDecimal priceFor(int qty) { return price.multiply(BigDecimal.valueOf(qty)); }
}

record Concert(String id, String artist, String venue, LocalDateTime startsAt) {}

/** A seat's whole life in one immutable value, swapped atomically with compare-and-set (CAS). */
sealed interface SeatState permits Free, Held, Sold {}
record Free() implements SeatState {}
record Held(String holdId, String userId, Instant expiresAt) implements SeatState {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}
record Sold(String holdId, String userId) implements SeatState {}

/** A temporary claim on inventory: either specific seats or a GA quantity. */
record Hold(String id, String userId, Tier tier, List<String> seatIds, int quantity, BigDecimal amount, Instant
expiresAt) {
    boolean expired(Instant now) { return !now.isBefore(expiresAt); }
}

record Ticket(String id, String concertId, String userId, Tier tier, String seatOrGa) {}

final class SoldOutException extends RuntimeException {
    private static final long serialVersionUID = 1L;
    SoldOutException(String msg) { super(msg); }
}

interface PaymentGateway {
    boolean charge(String userId, BigDecimal amount);
    void refund(String userId, BigDecimal amount);
}

interface Notifier { void send(String userId, String message); }

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}
```

```java
// File: lld/ch15/concert/Inventory.java
package lld.ch15.concert;

import java.time.Instant;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicLong;
import java.util.concurrent.atomic.AtomicReference;

/**
 * Numbered seats, lock-free. Each seat is an AtomicReference<SeatState>; a claim is one CAS.
 * Many buyers on DIFFERENT seats never wait for each other; on the SAME seat exactly one CAS wins.
 */
final class SeatedInventory {
    private final Map<String, Tier> tierOf = new LinkedHashMap<>();
    private final Map<String, AtomicReference<SeatState>> seats = new ConcurrentHashMap<>();

    void addSeat(String id, Tier tier) {
        tierOf.put(id, tier);
        seats.put(id, new AtomicReference<>(new Free()));
    }
    Tier tierOf(String id) { return tierOf.get(id); }

    /** All-or-nothing over several seats: claim in sorted order, roll back our own claims on failure. */
    boolean tryHold(List<String> ids, String holdId, String userId, Instant now, Instant expiresAt) {
        List<String> taken = new ArrayList<>();
        for (String id : ids.stream().sorted().toList()) {
            if (!tryHoldOne(id, holdId, userId, now, expiresAt)) {
                taken.forEach(t -> release(t, holdId));
                return false;
            }
            taken.add(id);
        }
        return true;
    }

    private boolean tryHoldOne(String id, String holdId, String userId, Instant now, Instant expiresAt) {
        AtomicReference<SeatState> ref = seats.get(id);
        if (ref == null) throw new IllegalArgumentException("no seat " + id);
        while (true) {
            SeatState cur = ref.get();
            boolean claimable = cur instanceof Free || (cur instanceof Held h && h.expired(now));
            if (!claimable) return false;
            if (ref.compareAndSet(cur, new Held(holdId, userId, expiresAt))) return true;
            // CAS lost: someone changed the seat between get() and CAS. Re-read and decide again.
        }
    }

    /** Held-by-this-hold and not expired -> Sold. Fails if the hold was lost. */
    boolean confirm(String id, String holdId, Instant now) {
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        return cur instanceof Held h && h.holdId().equals(holdId) && !h.expired(now)
                && ref.compareAndSet(cur, new Sold(holdId, h.userId()));
    }

    /** Give the seat back only if it is still ours (held or sold by this hold). */
    boolean release(String id, String holdId) {
        AtomicReference<SeatState> ref = seats.get(id);
        SeatState cur = ref.get();
        boolean ours = (cur instanceof Held h && h.holdId().equals(holdId))
                || (cur instanceof Sold s && s.holdId().equals(holdId));
        return ours && ref.compareAndSet(cur, new Free());
    }
```

```
    /** Seats of a tier a fan could claim right now (Free, or Held but expired), in seat order. */
    List<String> freeSeats(Tier tier, Instant now) {
        List<String> out = new ArrayList<>();
        tierOf.forEach((id, t) -> {
            SeatState s = seats.get(id).get();
            if (t == tier && (s instanceof Free || (s instanceof Held h && h.expired(now)))) out.add(id);
        });
        return out;
    }

    long soldCount() { return seats.values().stream().filter(r -> r.get() instanceof Sold).count(); }

    SeatState stateOf(String id) { return seats.get(id).get(); }
}

/** General admission: no seat numbers, just a counter. A CAS loop means it can never go below zero. */
final class GaPool {
    private final AtomicInteger available;
    GaPool(int capacity) { available = new AtomicInteger(capacity); }

    boolean tryTake(int qty) {
        while (true) {
            int cur = available.get();
            if (cur < qty) return false;
            if (available.compareAndSet(cur, cur - qty)) return true;
        }
    }
    void giveBack(int qty) { available.addAndGet(qty); }
    int available()        { return available.get(); }
}

/** Anti-scalping: at most `max` tickets per user per concert, checked-and-updated atomically. */
final class PurchaseLimiter {
    private final int max;
    private final ConcurrentHashMap<String, Integer> used = new ConcurrentHashMap<>();
    PurchaseLimiter(int max) { this.max = max; }

    boolean tryUse(String userId, int qty) {
        boolean[] ok = {false};
        used.compute(userId, (u, cur) -> {
            int c = cur == null ? 0 : cur;
            if (c + qty > max) return cur;            // unchanged
            ok[0] = true;
            return c + qty;
        });
        return ok[0];
    }
    void giveBack(String userId, int qty) { used.computeIfPresent(userId, (u, c) -> c - qty <= 0 ? null : c -
qty); }
}

/**
 * Virtual waiting room. Everyone who arrives gets a strictly increasing number; the system admits
 * people in that order, a batch at a time. First come, first served, and bots gain nothing by
 * hammering the buy button, because only admitted users may hold inventory.
 */
final class VirtualWaitingRoom {
    private final AtomicLong nextNumber = new AtomicLong();
    private final ConcurrentLinkedQueue<String> line = new ConcurrentLinkedQueue<>();
    private final Set<String> admitted = ConcurrentHashMap.newKeySet();
    private final Map<String, Long> numberOf = new ConcurrentHashMap<>();

    /** synchronized so the number order and the line order can never disagree. */
    synchronized long join(String userId) {
        Long n = numberOf.get(userId);
        if (n != null) return n;                      // joining twice keeps your place
        long mine = nextNumber.incrementAndGet();
        numberOf.put(userId, mine);
        line.add(userId);
        return mine;
```

```
    }
    synchronized List<String> admitNext(int k) {
        List<String> out = new ArrayList<>();
        for (int i = 0; i < k; i++) {
            String u = line.poll();
            if (u == null) break;
            admitted.add(u);
            out.add(u);
        }
        return out;
    }
    boolean isAdmitted(String userId) { return admitted.contains(userId); }
    void admitDirectly(String userId) { admitted.add(userId); }
}
```

```java
// File: lld/ch15/concert/TicketingService.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.atomic.AtomicInteger;

/**
 * Flow: join waiting room -> get admitted -> HOLD (seats or GA qty, with TTL) -> PURCHASE.
 * Released inventory (expired hold, cancellation) is offered to the tier's waitlist first.
 */
final class TicketingService {
    private final Concert concert;
    private final SeatedInventory seated;
    private final GaPool ga;
    private final PurchaseLimiter limiter;
    private final VirtualWaitingRoom room;
    private final PaymentGateway payments;
    private final Notifier notifier;
    private final Clock clock;
    private final Duration holdTtl;
    private final Map<String, Hold> holds = new ConcurrentHashMap<>();           // active holds
    private final Map<String, List<Ticket>> orders = new ConcurrentHashMap<>();  // holdId -> tickets
    private final Map<Tier, Queue<String>> waitlists = new EnumMap<>(Tier.class);
    private final AtomicInteger seq = new AtomicInteger();

    TicketingService(Concert concert, SeatedInventory seated, GaPool ga, PurchaseLimiter limiter,
                     VirtualWaitingRoom room, PaymentGateway payments, Notifier notifier, Clock clock, Duration
holdTtl) {
        this.concert = concert; this.seated = seated; this.ga = ga; this.limiter = limiter; this.room = room;
        this.payments = payments; this.notifier = notifier; this.clock = clock; this.holdTtl = holdTtl;
        for (Tier t : Tier.values()) waitlists.put(t, new ConcurrentLinkedQueue<>());
    }

    // ---------------------------------------------------------------- hold
    Hold holdSeats(String userId, List<String> seatIds) {
        requireAdmitted(userId);
        if (seatIds.isEmpty()) throw new IllegalArgumentException("no seats");
        Tier tier = seated.tierOf(seatIds.get(0));
        if (seatIds.stream().anyMatch(id -> seated.tierOf(id) != tier))
            throw new IllegalArgumentException("one hold = one tier");
        if (!limiter.tryUse(userId, seatIds.size()))
            throw new IllegalStateException(userId + " would exceed the per-user ticket limit");
        Instant now = clock.instant();
        Hold h = new Hold("H" + seq.incrementAndGet(), userId, tier, List.copyOf(seatIds), seatIds.size(),
                tier.priceFor(seatIds.size()), now.plus(holdTtl));
        if (!seated.tryHold(seatIds, h.id(), userId, now, h.expiresAt())) {
            limiter.giveBack(userId, seatIds.size());
            throw new SoldOutException(seatIds + " not available");
        }
        holds.put(h.id(), h);
        return h;
    }

    Hold holdGa(String userId, int qty) {
        requireAdmitted(userId);
        if (!limiter.tryUse(userId, qty))
            throw new IllegalStateException(userId + " would exceed the per-user ticket limit");
        if (!ga.tryTake(qty)) {
            limiter.giveBack(userId, qty);
            throw new SoldOutException("general admission sold out");
```

```
        }
        Hold h = new Hold("H" + seq.incrementAndGet(), userId, Tier.GENERAL_ADMISSION, List.of(), qty,
                Tier.GENERAL_ADMISSION.priceFor(qty), clock.instant().plus(holdTtl));
        holds.put(h.id(), h);
        return h;
    }

    // ------------------------------------------------------------------ purchase
    List<Ticket> purchase(String holdId) {
        Hold h = holds.remove(holdId);                    // atomic claim: a hold is purchased at most once
        if (h == null) throw new IllegalStateException(holdId + " unknown, expired or already used");
        if (h.expired(clock.instant())) { releaseInventory(h); throw new IllegalStateException(holdId + "
expired"); }
        if (!payments.charge(h.userId(), h.amount())) { releaseInventory(h); throw new
IllegalStateException("payment declined"); }

        List<Ticket> tickets = new ArrayList<>();
        if (h.tier().seated) {
            Instant now = clock.instant();
            List<String> confirmed = new ArrayList<>();
            for (String id : h.seatIds()) {
                if (!seated.confirm(id, h.id(), now)) {   // hold lost during payment -> compensate
                    confirmed.forEach(c -> seated.release(c, h.id()));
                    h.seatIds().forEach(s -> seated.release(s, h.id()));
                    limiter.giveBack(h.userId(), h.quantity());
                    payments.refund(h.userId(), h.amount());
                    throw new IllegalStateException("seat " + id + " lost during payment, refunded");
                }
                confirmed.add(id);
                tickets.add(new Ticket("T" + seq.incrementAndGet(), concert.id(), h.userId(), h.tier(), id));
            }
        } else {
            for (int i = 0; i < h.quantity(); i++)
                tickets.add(new Ticket("T" + seq.incrementAndGet(), concert.id(), h.userId(), h.tier(), "GA"));
        }
        orders.put(h.id(), List.copyOf(tickets));
        return tickets;
    }

    // ------------------------------------------------------------------ release paths
    /** The reaper: run every few seconds in production. Returns how many holds it expired. */
    int expireHolds() {
        Instant now = clock.instant();
        int n = 0;
        for (Hold h : holds.values())
            if (h.expired(now) && holds.remove(h.id(), h)) { releaseInventory(h); n++; }
        return n;
    }

    /** Cancel a whole order (all tickets bought with one hold). */
    void cancelOrder(String holdId) {
        List<Ticket> tickets = orders.remove(holdId);
        if (tickets == null) throw new IllegalStateException("no order " + holdId);
        Tier tier = tickets.get(0).tier();
        BigDecimal refund = tier.priceFor(tickets.size());
        payments.refund(tickets.get(0).userId(), refund);
        Hold asIfHeld = new Hold(holdId, tickets.get(0).userId(), tier,
                tier.seated ? tickets.stream().map(Ticket::seatOrGa).toList() : List.of(), tickets.size(), refund,
clock.instant());
        releaseInventory(asIfHeld);
    }

    void joinWaitlist(String userId, Tier tier) { waitlists.get(tier).add(userId); }

    private void releaseInventory(Hold h) {
        limiter.giveBack(h.userId(), h.quantity());
        if (h.tier().seated) {
            h.seatIds().forEach(id -> seated.release(id, h.id()));
            h.seatIds().forEach(id -> offerSeat(h.tier(), id));
        } else {
            ga.giveBack(h.quantity());
```

```
            offerGa();
        }
    }

    /** Waitlist first: freed inventory goes to the person who has waited longest, as a fresh hold. */
    private void offerGa() {
        Queue<String> q = waitlists.get(Tier.GENERAL_ADMISSION);
        String next;
        while (ga.available() > 0 && (next = q.poll()) != null) {
            room.admitDirectly(next);
            try {
                Hold h = holdGa(next, 1);
                notifier.send(next, "a GA ticket freed up, held for you as " + h.id() + " until " +
h.expiresAt());
            } catch (RuntimeException e) { /* over their limit or lost the race: try the next person */ }
        }
    }

    private void offerSeat(Tier tier, String seatId) {
        String next = waitlists.get(tier).poll();
        if (next == null) return;
        room.admitDirectly(next);
        try {
            Hold h = holdSeats(next, List.of(seatId));
            notifier.send(next, "seat " + seatId + " freed up, held for you as " + h.id());
        } catch (RuntimeException e) { /* someone else took it first */ }
    }

    private void requireAdmitted(String userId) {
        if (!room.isAdmitted(userId)) throw new IllegalStateException(userId + " is still in the waiting room");
    }

    int gaAvailable() { return ga.available(); }

    /** What a fan sees before choosing: free places per tier (requirement: view seating and availability). */
    String availability() {
        StringBuilder sb = new StringBuilder();
        for (Tier t : Tier.values()) {
            if (t.seated) sb.append(t).append(" free ").append(seated.freeSeats(t, clock.instant())).append(", ");
            else sb.append(t).append(" free ").append(ga.available());
        }
        return sb.toString();
    }
}
```

```java
// File: lld/ch15/concert/Main.java
package lld.ch15.concert;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDateTime;
import java.util.List;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-06-01T09:00:00Z"));
        AtomicReference<BigDecimal> collected = new AtomicReference<>(BigDecimal.ZERO);
        PaymentGateway bank = new PaymentGateway() {
            public boolean charge(String u, BigDecimal a) { collected.accumulateAndGet(a, BigDecimal::add); return
true; }
            public void refund(String u, BigDecimal a)    { collected.accumulateAndGet(a, BigDecimal::subtract); }
        };
        Notifier notifier = (u, m) -> System.out.println("  [notify " + u + "] " + m);

        Concert show = new Concert("C1", "The Band", "Stadium", LocalDateTime.of(2026, 8, 15, 19, 30));
        SeatedInventory seated = new SeatedInventory();
        for (int i = 1; i <= 4; i++) seated.addSeat("VIP-" + i, Tier.VIP);
        for (int i = 1; i <= 6; i++) seated.addSeat("P-" + i, Tier.PREMIUM);
        GaPool ga = new GaPool(100);
        VirtualWaitingRoom room = new VirtualWaitingRoom();
        TicketingService svc = new TicketingService(show, seated, ga, new PurchaseLimiter(4), room,
                bank, notifier, clock, Duration.ofMinutes(10));

        System.out.println("== 1. Seating and availability, then the virtual waiting room ==");
        System.out.println("  availability: " + svc.availability());
        for (String u : List.of("ana", "ben", "cai", "dev", "eli")) System.out.println("  " + u + " got number " +
room.join(u));
        System.out.println("  ben joins again, keeps number " + room.join("ben"));
        System.out.println("  admitted batch 1: " + room.admitNext(3));
        try { svc.holdGa("eli", 1); } catch (IllegalStateException e) { System.out.println("  REFUSED: " +
e.getMessage()); }

        System.out.println("\n== 2. 40 admitted fans CAS for the same seat VIP-1 ==");
        AtomicInteger vipWins = new AtomicInteger();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 40; i++) {
                String u = "vip-fan-" + i;
                room.admitDirectly(u);
                pool.submit(() -> {
                    go.await();
                    try { svc.holdSeats(u, List.of("VIP-1")); vipWins.incrementAndGet(); }
                    catch (SoldOutException ignored) { }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("  holds on VIP-1 = " + vipWins.get());

        System.out.println("\n== 3. Flash sale: 1,000 fans, 100 GA places (98 left after test buys) ==");
        Hold anaHold = svc.holdGa("ana", 2);
        svc.purchase(anaHold.id());
        AtomicInteger sold = new AtomicInteger(), soldOut = new AtomicInteger();
        CountDownLatch go2 = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                String u = "fan-" + i;
                room.admitDirectly(u);
```

```
                pool.submit(() -> {
                    go2.await();
                    try { svc.purchase(svc.holdGa(u, 1).id()); sold.incrementAndGet(); }
                    catch (SoldOutException e) { soldOut.incrementAndGet(); }
                    return null;
                });
            }
            go2.countDown();
        }
        System.out.println("  sold=" + sold + " soldOut=" + soldOut + " gaLeft=" + svc.gaAvailable() + " (never
negative)");

        System.out.println("\n== 4. Per-user limit (4) stops a scalper ==");
        Hold p = svc.holdSeats("ben", List.of("P-1", "P-2", "P-3"));
        System.out.println("  ben holds " + p.seatIds() + " = " + p.amount());
        try { svc.holdSeats("ben", List.of("P-4", "P-5")); } catch (IllegalStateException e) {
System.out.println("  REFUSED: " + e.getMessage()); }
        List<Ticket> benTickets = svc.purchase(p.id());
        System.out.println("  ben bought " + benTickets.stream().map(Ticket::seatOrGa).toList());

        System.out.println("\n== 5. Sold out -> waitlist -> a cancellation is offered to the waitlist ==");
        try { svc.holdGa("cai", 1); } catch (SoldOutException e) { System.out.println("  cai: " + e.getMessage() +
", joins waitlist"); }
        svc.joinWaitlist("cai", Tier.GENERAL_ADMISSION);
        svc.cancelOrder(anaHold.id());                  // ana returns her 2 GA tickets
        System.out.println("  GA left after offers = " + svc.gaAvailable());

        System.out.println("\n== 6. Unpaid holds expire; the reaper returns them ==");
        System.out.println("  admitted batch 2: " + room.admitNext(3));
        Hold forgotten = svc.holdSeats("dev", List.of("P-6"));
        clock.advance(Duration.ofMinutes(11));
        System.out.println("  expired holds reaped = " + svc.expireHolds() + " (VIP-1, cai's unpaid offer, dev's
P-6)");
        try { svc.purchase(forgotten.id()); } catch (IllegalStateException e) { System.out.println("  dev: " +
e.getMessage()); }
        System.out.println("  P-6 now " + seated.stateOf("P-6") + ", GA left " + svc.gaAvailable());
        System.out.println("  availability: " + svc.availability());
        System.out.println("  money collected = " + collected.get());
    }
}
```

Output:

```
== 1. Seating and availability, then the virtual waiting room ==
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-1, P-2, P-3, P-4, P-5, P-6],
GENERAL_ADMISSION free 100
  ana got number 1
  ben got number 2
  cai got number 3
  dev got number 4
  eli got number 5
  ben joins again, keeps number 2
  admitted batch 1: [ana, ben, cai]
  REFUSED: eli is still in the waiting room

== 2. 40 admitted fans CAS for the same seat VIP-1 ==
  holds on VIP-1 = 1

== 3. Flash sale: 1,000 fans, 100 GA places (98 left after test buys) ==
  sold=98 soldOut=902 gaLeft=0 (never negative)

== 4. Per-user limit (4) stops a scalper ==
  ben holds [P-1, P-2, P-3] = 7500.00
  REFUSED: ben would exceed the per-user ticket limit
  ben bought [P-1, P-2, P-3]

== 5. Sold out -> waitlist -> a cancellation is offered to the waitlist ==
  cai: general admission sold out, joins waitlist
  [notify cai] a GA ticket freed up, held for you as H244 until 2026-06-01T09:10:00Z
  GA left after offers = 1

== 6. Unpaid holds expire; the reaper returns them ==
  admitted batch 2: [dev, eli]
  expired holds reaped = 3 (VIP-1, cai's unpaid offer, dev's P-6)
  dev: H245 unknown, expired or already used
  P-6 now Free[], GA left 2
  availability: VIP free [VIP-1, VIP-2, VIP-3, VIP-4], PREMIUM free [P-4, P-5, P-6], GENERAL_ADMISSION free 2
  money collected = 85900.00
```

What to notice: section 1 admits the first three in arrival order and refuses someone still in the queue; section 2 — 40 concurrent CAS attempts on one seat, exactly one hold; section 3 — 1,000 buyers for 98 remaining GA places sell exactly 98 and the counter stops at 0; section 4 — a fifth ticket is refused by the per-user limit; section 5 — a cancellation's released ticket goes straight to the waitlisted fan as a hold; section 6 — the reaper returns three expired holds (VIP-1, the unpaid waitlist offer, and dev's seat) and a late purchase is refused. The availability line at the end shows what is still for sale (three PREMIUM seats, four VIP seats, two GA places). Money collected is 85,900.00 = 98 × 800 + 3 × 2,500 (ana's 2 GA tickets were refunded).

Concurrency & edge cases

- **Why CAS is correct.** `compareAndSet(expected, new)` atomically replaces the value only if it is still the exact object we read. If another thread changed it in between, the CAS fails and we re-read and decide again. The loop in `tryHoldOne` only retries while the seat is still claimable; otherwise it returns `false`.
- **ABA problem?** ABA is when a value goes A → B → A and a CAS wrongly succeeds. Here every transition creates a *new* record object and `AtomicReference` compares references, so an "A" that came back is a different object and the CAS fails safely.
- **Hold claimed exactly once.** `purchase` starts with `holds.remove(holdId)`; the reaper uses `holds.remove(id, h)`. Only one of them can get the hold, so inventory is never released twice.
- **Per-user limit and failures.** Every failure path (sold out, declined, expired, lost seat) gives the limit back, so a user is never locked out by a failed attempt.
- **Lost hold after payment.** `confirm` fails if the hold expired and someone CASed over it; we release, give back the limit, and refund.
- **Fair queue scope.** The in-memory queue is fair *within one process*. At real scale the waiting room is a separate service (often at the CDN edge — a CDN, content delivery network, is a worldwide set of servers placed close to users; see System Design Fundamentals) that hands out signed, time-limited

access tokens (short strings the server can verify it issued); inventory servers only check the token.

- **Across servers.** Seat CAS maps naturally to a database conditional update (`UPDATE seat SET state='HELD', hold_id=?, expires_at=? WHERE id=? AND (state='FREE' OR expires_at < now())`) or Redis `SET NX PX`. GA counts map to Redis `DECRBY` (decrement by an amount) guarded by a Lua script (a small script Redis runs as one atomic step), or to a database row updated with `WHERE available >= ?`.

Extensions & follow-up questions

- **"Best available seats."** Instead of the user choosing, a `SeatAllocator` strategy picks the best adjacent free seats in a tier, then calls the same CAS hold.
- **"Verified fans / presale codes."** Admission to the waiting room requires a token issued to pre-registered fans; the limiter keys on a verified identity, not an email.
- **"Dynamic pricing."** Prices per tier come from a `PricingStrategy` that looks at remaining inventory and time — decided when the hold is created and frozen in the `Hold`.
- **"Resale marketplace."** A sold ticket can be listed; a buyer's purchase transfers ownership and re-issues a new barcode (so the old one stops working).
- **"Waitlist for seated tiers."** Implemented the same way for each released seat (`offerSeat`).

Real-world example

Ticketmaster uses a virtual queue ("Smart Queue") for high-demand on-sales and a **Verified Fan** presale programme that registers fans in advance to filter out bots; purchase limits per account are standard on its events. The November 2022 presale for Taylor Swift's *Eras Tour* is the well-known stress case: demand far beyond what was planned for overwhelmed the system, many fans waited for hours or were dropped, and the general public sale was cancelled — a real lesson that fairness and admission control are as important as seat-locking correctness.

In the interview

30–45 minute plan. (1) Say up front: "the entities are like movie booking; the hard part is contention and fairness." (2) Tiers: seated vs GA, and why GA is a counter. (3) Holds with TTL; code the CAS seat claim and the GA CAS loop. (4) Fairness: waiting room with batches; per-user limit with atomic `compute`. (5) Waitlist on release; reaper. (6) Scale-out: tokens from the waiting room, conditional updates/Redis.

> **30-second answer:** "Seated tiers are an `AtomicReference<SeatState>` per seat, claimed by compare-and-set — one winner, no locks. GA is an `AtomicInteger` decremented by a CAS loop so it can't go negative. Everyone enters through a virtual waiting room that admits in arrival order, and a per-user limit is enforced atomically. Holds expire after 10 minutes; a reaper returns expired GA holds; released tickets go to a FIFO waitlist first. If a hold is lost during payment we refund."

Follow-ups with answers: *Lock-free vs locks?* Lock-free wins when contention is high and critical sections are tiny (one field swap); locks are simpler when an operation must update many things together. *What if the queue service fails?* Fail closed (on failure, deny rather than allow): pause admissions, don't open the floodgates. *How do you stop one person with 50 accounts?* Verified identity, payment-card fingerprinting (recognising the same card across accounts), and limits keyed on those.

Self-check

- **Q:** Why is GA modeled as a counter, not seats? **A:** Standing places are interchangeable; a counter with CAS is simpler and can't oversell.
- **Q:** What does `compareAndSet` guarantee? **A:** The update happens only if the value is still the one you read; otherwise it fails and you retry or give up.
- **Q:** Why must `join` be synchronized? **A:** So the number handed out and the position in the line are assigned together; otherwise two joiners could get numbers in one order and queue slots in the other.
- **Q:** How is a hold prevented from being purchased *and* reaped? **A:** Both paths remove it from the `holds` map first; only one removal succeeds.

- **Q:** What happens to a released ticket when fans are waitlisted? **A:** It becomes a fresh hold for the first fan in the waitlist, who is notified.

> **TL;DR**

Same nouns as movie booking, different enemy: contention. CAS per seat, CAS counter for GA, a FIFO virtual waiting room for fairness, atomic per-user limits, TTL holds with a reaper, a waitlist that gets released tickets first, and refunds when a hold is lost.

15.4 Design a Hotel Management System



Figure — Cover image for the hotel management case study (course diagram)

Problem & clarifying questions

A **Hotel Management System** (often called a *property management system*, PMS) runs a hotel's operations in one place: it knows every room, takes bookings, checks guests in and out, issues key cards, records housekeeping and room service, and produces the bill. Management gets one portal to track available rooms, book them and generate invoices.

The inventory here is different from a cinema: a room is not sold once, it is sold **night by night**. That single fact shapes the whole design.

Question	Answer we assume
Several hotel locations?	Yes — a `Hotel` has many `HotelLocation`s, each with its own rooms.
Room types?	`STANDARD`, `DELUXE`, `FAMILY_SUITE`, `BUSINESS_SUITE`.
What does a guest search by?	Room type and **date range** (availability).
Cancellation policy?	**Full refund if cancelled more than 24 hours before check-in**; otherwise no refund (our assumption for the "otherwise" case).
Extra charges during the stay?	Room service, food (kitchen) and amenities are added to the bill.
How is the bill paid?	**Credit card, check or cash**.
Keys?	Electronic key cards with a barcode and key id; deactivated at check-out.
Housekeeping?	Every task is logged per room (a history, never overwritten).
One booking for several rooms?	No — one booking = one room (a known limitation; see follow-ups).

Requirements

Functional (course list)

1. Support booking different **room types**: standard, deluxe, family suite (and business suite).
2. Guests **search** room inventory and book any **available** room.
3. Retrieve **who booked a room** and **which rooms a customer booked**.
4. Customers can **cancel**; **full refund if more than 24 hours before check-in**.
5. **Notify** the guest shortly before check-in and shortly before check-out.
6. Keep a **housekeeping log** per room.
7. Customers can add **room services and food items**.
8. Customers can ask for **amenities**.
9. Pay bills by **credit card, check or cash**.

Non-functional — never double-book a room-night (even with concurrent bookings from the website and the front desk); the bill must always equal the sum of its lines; each actor may do only their own job; the refund rule must be changeable in one place.

Use cases

Actors (course): **Guest** (search, book), **Receptionist** (add/modify rooms, create bookings, check in/out), **System** (booking/cancellation/reminder notifications), **Manager** (adds new workers), **Housekeeper** (adds/modifies housekeeping records), **Server** (adds/modifies room-service records).

Top use cases: add/remove/edit room; search room (by type and availability); register or cancel an account; book room; **check in**; **check out** (end of booking, return of keys); add room charge; update housekeeping log.

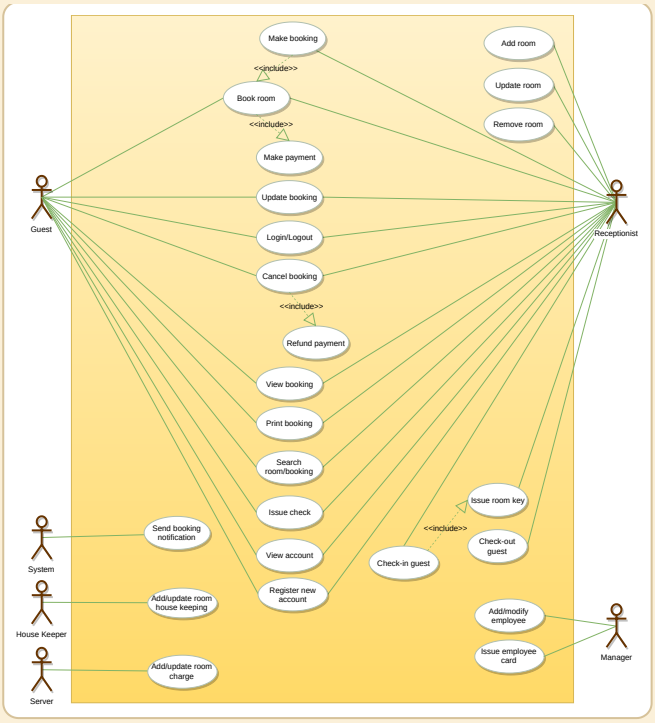


Figure — Hotel use cases: Guest and Receptionist share booking, update, cancel, view, print and search; "Make booking" includes "Book room", which includes "Make payment"; "Cancel booking" includes "Refund payment"; the receptionist's "Check-in guest" includes "Issue room key"; the System sends booking notifications; the House Keeper adds/updates housekeeping; a manager adds/modifies employees (course diagram)

Core entities

Class	Responsibility
`Hotel`	Owns its locations; searches each one in turn.
`HotelLocation` implements `Search`	Owns its rooms; answers "which rooms of this type are free for these dates?".
`Room`	Number, style, price per night; the **set of booked nights**; owns its `RoomKey`s and housekeeping log; current `RoomStatus`.
`RoomStyle`, `RoomStatus` (enums)	Type of room; what it is doing *now* (`AVAILABLE`, `RESERVED`, `OCCUPIED`, `NOT_AVAILABLE`, `BEING_SERVICED`, `OTHER`).
`RoomKey`	Key id, barcode, issue time, master flag, active flag.
`RoomHouseKeeping` (record)	One log line: when, who, what, how long.
`RoomBooking`	Reservation number, guest id, room, check-in/out dates, status, deposit; **owns its** `Invoice`; `refundDue()` holds the cancellation rule.
`BookingStatus` (enum)	`REQUESTED`, `PENDING`, `CONFIRMED`, `CHECKED_IN`, `CHECKED_OUT`, `CANCELLED`, `ABANDONED`.
`Invoice`, `InvoiceItem`	A list of lines; the total is computed from them.
`RoomCharge` (abstract) → `Amenity`, `RoomService`, `KitchenService`	A charge that knows how to write itself onto the invoice (Template Method: the parent fixes the steps, subclasses fill in one).
`Payment` (sealed, abstract) → `CreditCardTransaction`, `CheckTransaction`, `CashTransaction`	Three ways to pay with genuinely different behaviour.
`Person` (sealed) → `Guest`, `Receptionist`, `Server`, `Housekeeper`, `Manager`	Actors; service methods accept only the right actor type.
`FrontDesk`	Every flow of a stay: book, check in, add charge, check out, cancel, housekeeping, reminders, queries.
`Notifier`	Messages to guests.

Class diagram

The course's five stages:

Stage	Adds	What works afterwards
1	Invoice, payment types, notification types	A bill can be produced and settled by card, check or cash
2	Room, key card, housekeeping record	Each room can tell which of its nights are sold
3	Hotel, location, search interface	Free rooms can be listed for any date range
4	Booking and room charges	A stay can be followed from reservation to check-out
5	People and accounts	Every actor is limited to its own tasks

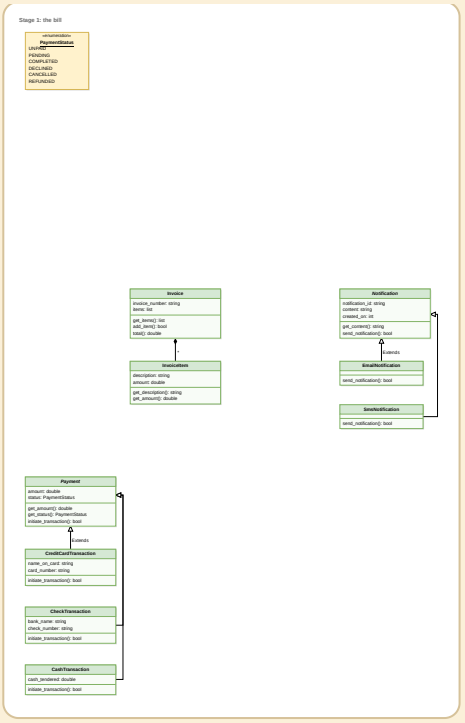


Figure — Stage 1: `Invoice` owns its `InvoiceItem`s (composition) and computes `total()`; abstract `Payment` with credit card, check and cash subclasses; abstract `Notification` with one subclass per channel (course diagram)

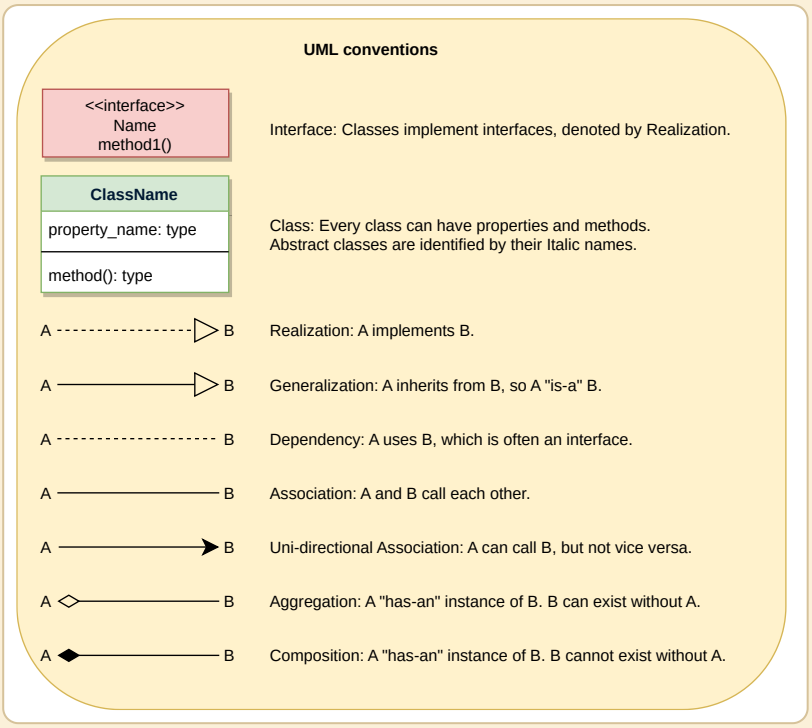


Figure — How to read a class box and the arrow styles used in these diagrams (course diagram)

Requirements 7 and 8 each put extra charges on a stay. That is why the invoice stores **individual lines** and computes its total on demand: a total that is always recomputed can never go out of step with the lines.

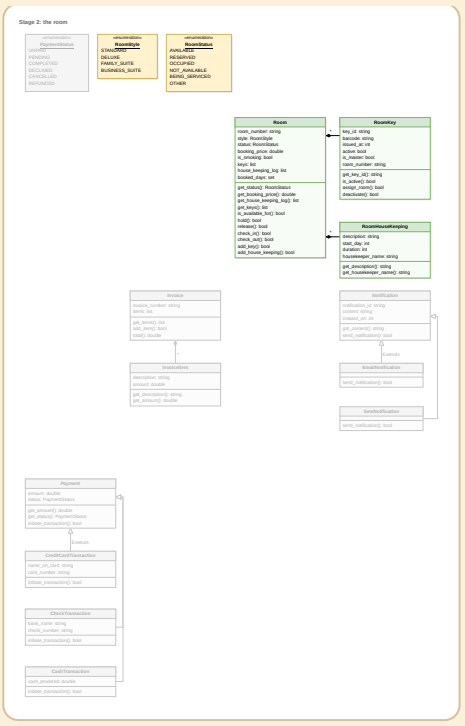


Figure — Stage 2: `Room` (number, style, status, price, and the set of nights it is committed to) owns its `RoomKey`s and its list of `RoomHouseKeeping` records (course diagram)

The key decision of the whole design sits here: **a status field is not availability.** `RoomStatus.OCCUPIED` describes the present moment only. A guest, however, asks about the future ("can I have it from the 10th to the 13th?"), and a single status value cannot answer that. So each room also stores the dates of the nights it has already sold.

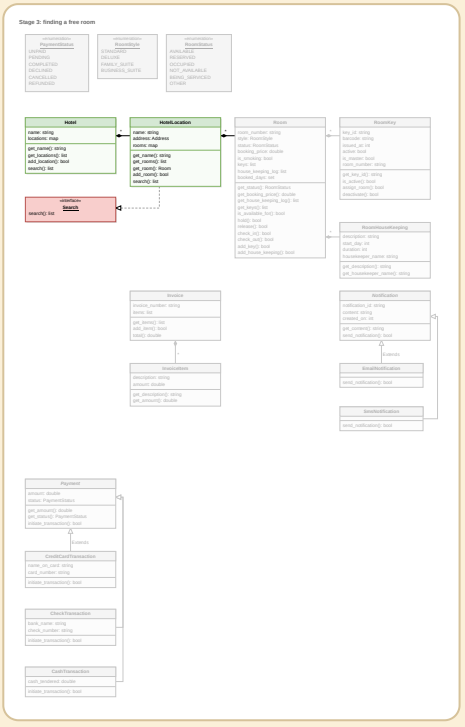


Figure — Stage 3: the `Search` interface implemented by `HotelLocation`, which owns its rooms; `Hotel` owns its locations (course diagram)

Search sits on the **location**, not on `Room`: one room has no knowledge of the other rooms, so it cannot list which of them are free. Putting `search` on `Room` is a frequent beginner mistake.

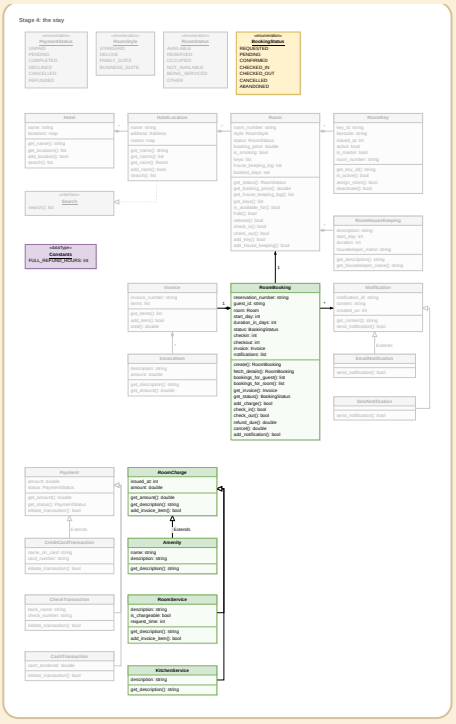


Figure — Stage 4: `RoomBooking` owns its invoice and points at its room (plain association); `RoomCharge` with `Amenity`, `RoomService` and `KitchenService` subclasses each writing an invoice line (course diagram)

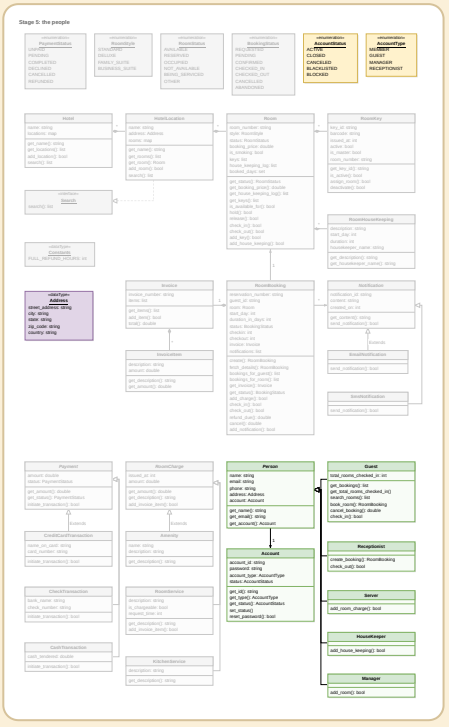


Figure — Stage 5: `Person` with `Guest`, `Receptionist`, `Server`, `HouseKeeper` and `Manager`, each with only its own methods; `Person` refers to its `Account`; there is deliberately no arrow from guest to booking (course diagram)

Notice what the figure does *not* have: an arrow between guest and booking. The booking stores only the guest's **id**, and queries filter bookings by that id — requirement 3 is met without either class holding a reference to the other.

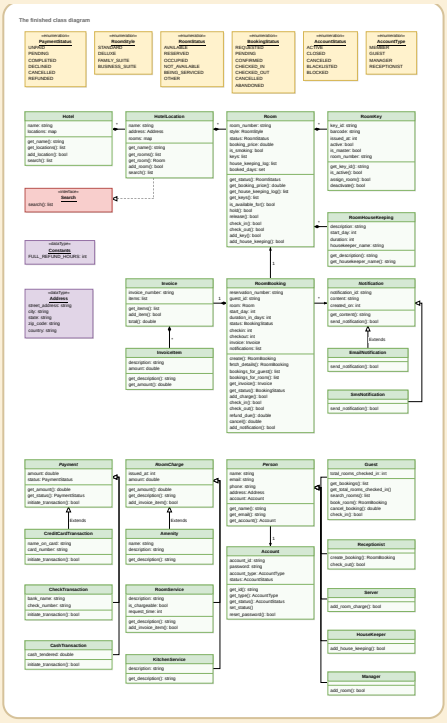
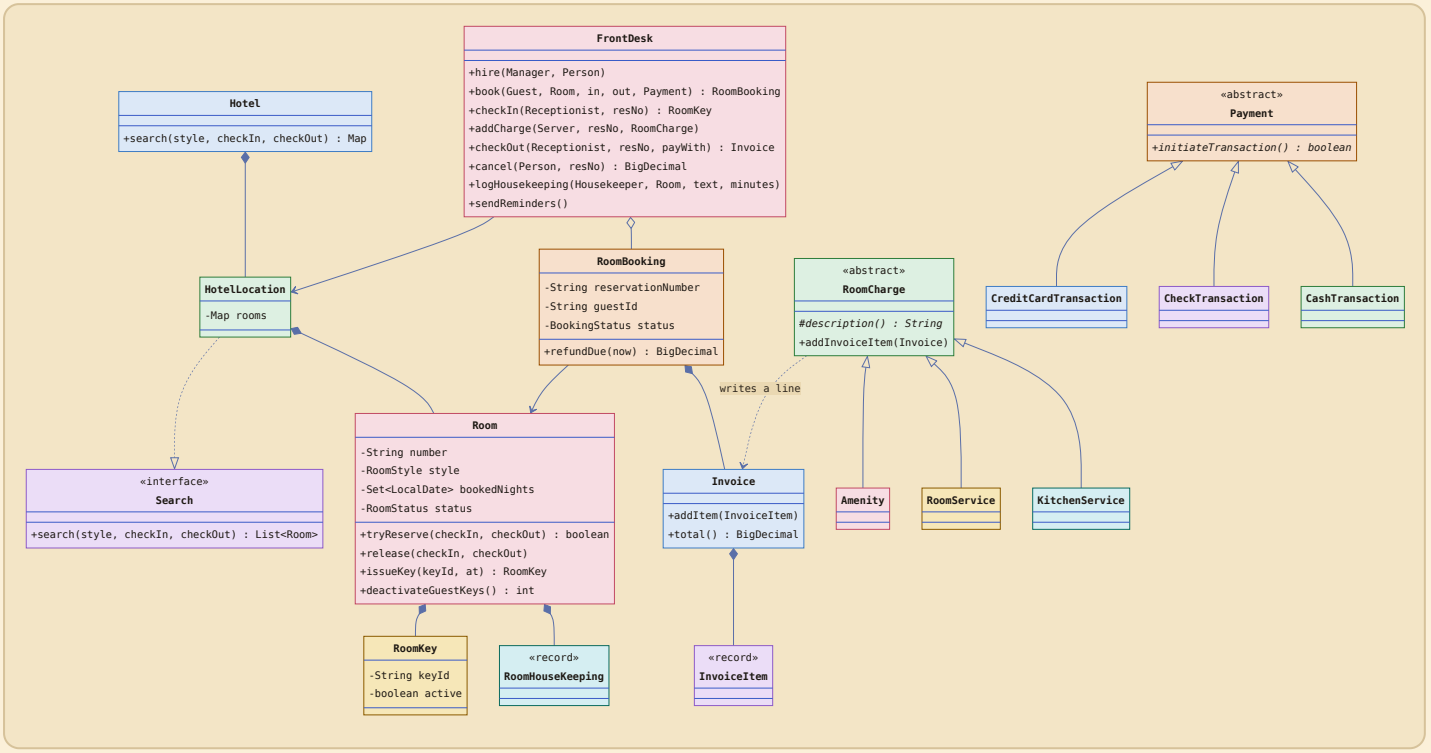


Figure — The finished hotel class diagram (course diagram)

Our Java design:



Key flows

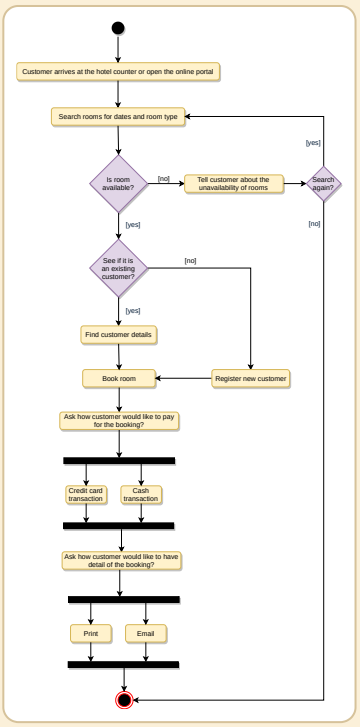


Figure — Make a room booking: search rooms for dates and room type; if none is available tell the customer; otherwise find or register the customer, book the room, ask how they will pay (credit card or cash), and how they want the booking details (course diagram)

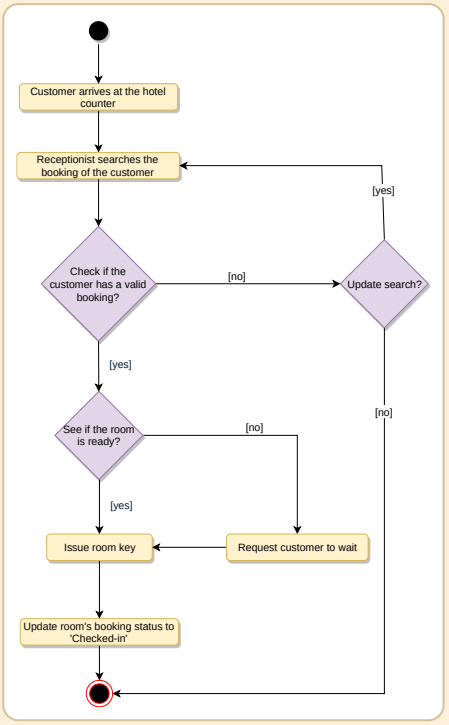


Figure — Check in: the customer arrives at the counter and the receptionist looks up the booking (searching again if it is not found); with a valid booking, check whether the room is ready — if not, the customer is asked to wait; if so, issue the room key and set the booking status to "Checked-in" (course diagram)

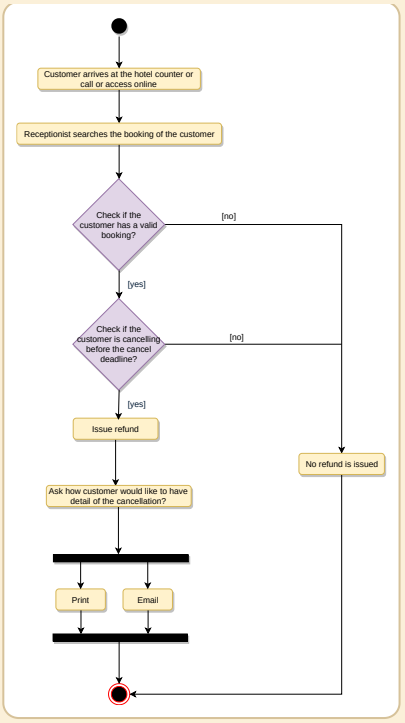
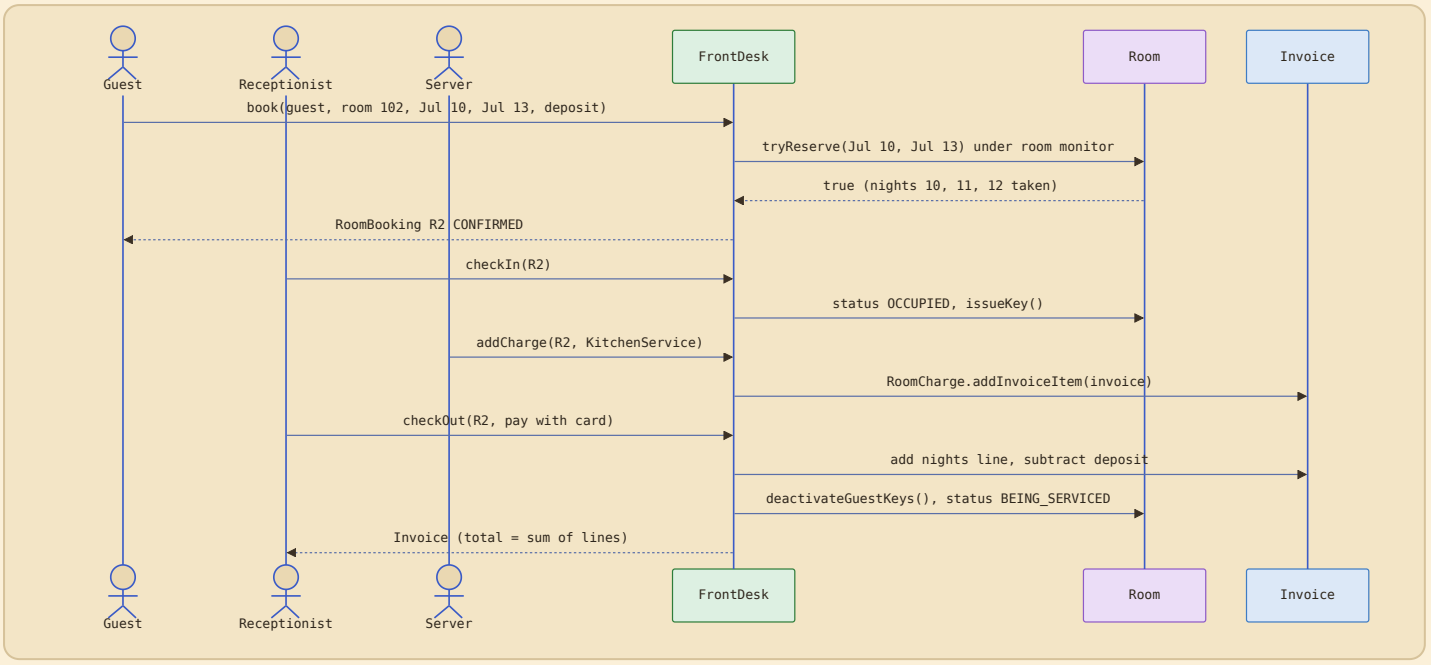
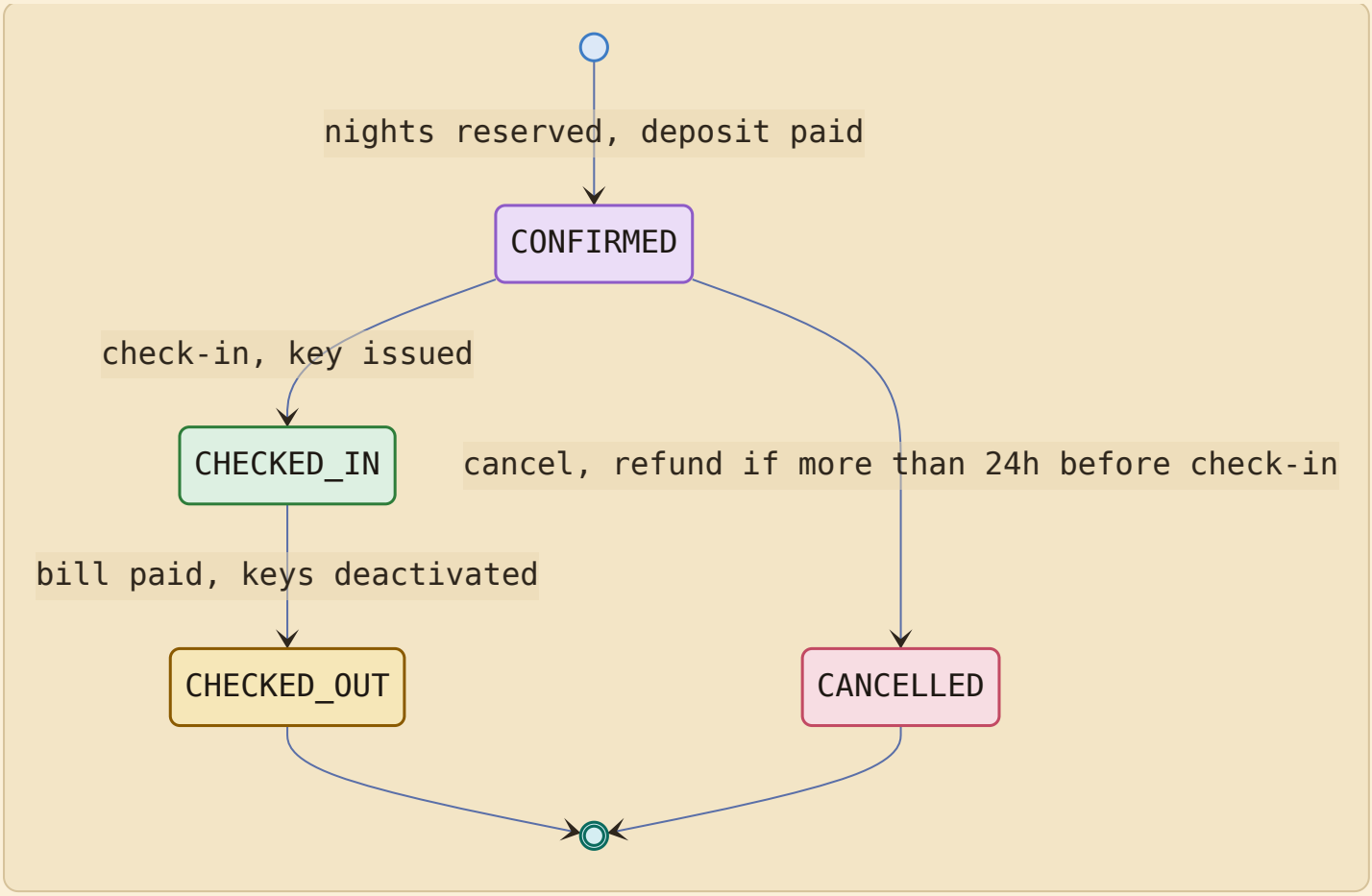


Figure — Cancel a booking: the customer comes to the counter, calls or goes online; the receptionist finds the booking and checks it is valid; if the customer is cancelling before the cancel deadline a refund is issued, otherwise no refund; the cancellation details are printed or emailed (course diagram)

A whole stay as a sequence:



Booking lifecycle:



The room's *current* status cycles independently: `AVAILABLE` → `OCCUPIED` (check-in) → `BEING_SERVICED` (check-out) → `AVAILABLE` (housekeeping logged).

Design decisions & patterns used

Decision	Why	Alternative rejected
Availability = set of booked nights (course's key point)	Answers date-range questions; nights are half-open `[checkIn, checkOut)`, so a guest leaving on the 13th and one arriving on the 13th don't clash.	Using `RoomStatus` for availability — only knows *now*.
`tryReserve` is `synchronized` on the room	"Are all these nights free? → take them" is check-then-act; the room's monitor makes it atomic. Rooms don't block each other.	One hotel-wide lock.
Search on `HotelLocation`	A room can't answer "which rooms are free?".	`Room.search()`.
Invoice = list of lines, total computed	The total can never disagree with the lines (requirements 7 and 8).	A stored running total.
Template Method: `RoomCharge.addInvoiceItem`	The parent fixes the algorithm (describe → price → write a line) and is `final`; subclasses fill in only `description()`. A new kind of charge = a new subclass, no edit to `Invoice` (Open/Closed). The course calls this the clearest Template Method example in its catalogue. Compare with Strategy, where the *whole* method is swapped rather than one step.	A `switch` on charge type inside the invoice.
Three `Payment` subclasses	They really behave differently: a card completes immediately, a **check stays `PENDING`** until it clears, cash can be declined if the amount tendered is short. Each passes the substitution test (each can be asked to take money).	One `Payment` with a `type` field and `if`s.
Refund rule in one method (`refundDue`)	When the policy changes there is exactly one place to edit. To support several policies (non-refundable rates, partial refunds), promote it to a `CancellationPolicy` strategy as in 15.2.	Refund arithmetic scattered through cancel flows.
Actor types in service signatures	`addCharge(Server, …)`, `logHousekeeping(Housekeeper, …)`, `addRoom(Manager, …)` — the compiler enforces who can do what. The receptionist booking for a guest calls the *same* booking method — two actors, one behaviour, no duplication.	Every actor able to call everything.
Booking stores guest id, not a `Guest` reference	No two-way dependency; "which rooms did this guest book?" is a query.	Bidirectional links that must be kept in sync.
Nights released on failed deposit / cancel	Reserve-then-pay needs an undo: if the deposit fails the nights are given back immediately.	Leaving phantom reservations.

Java implementation

Package `lld.ch15.hotel`. Files: `Billing.java` (invoice, payments, room charges), `Rooms.java` (room, key, housekeeping, search, hotel, clock, money), `Stay.java` (booking, actors, front desk), `Main.java`.

```java
// File: lld/ch15/hotel/Billing.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

enum PaymentStatus { UNPAID, PENDING, COMPLETED, DECLINED, CANCELLED, REFUNDED }

record InvoiceItem(String description, BigDecimal amount) {}   // money: BigDecimal, never double

/** A bill is a list of lines; the total is always computed, so it can never disagree with the lines. */
final class Invoice {
    private final String number;
    private final List<InvoiceItem> items = new ArrayList<>();
    Invoice(String number) { this.number = number; }
    String number()                        { return number; }
    synchronized void addItem(InvoiceItem i) { items.add(i); }
    synchronized List<InvoiceItem> items()   { return List.copyOf(items); }
    synchronized BigDecimal total()          { return
items.stream().map(InvoiceItem::amount).reduce(BigDecimal.ZERO, BigDecimal::add); }
}

/** Three ways to pay, which really behave differently: a check does not clear at once. */
abstract sealed class Payment permits CreditCardTransaction, CheckTransaction, CashTransaction {
    private final BigDecimal amount;
    private PaymentStatus status = PaymentStatus.UNPAID;
    Payment(BigDecimal amount) { this.amount = amount; }
    BigDecimal amount()      { return amount; }
    PaymentStatus status()   { return status; }
    protected void setStatus(PaymentStatus s) { status = s; }
    /** Take the money. Returns true when the hotel may release the guest. */
    abstract boolean initiateTransaction();
}

final class CreditCardTransaction extends Payment {
    private final String nameOnCard;
    CreditCardTransaction(BigDecimal amount, String nameOnCard) { super(amount); this.nameOnCard = nameOnCard; }
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.COMPLETED); return true; }
    @Override public String toString() { return "card(" + nameOnCard + ")"; }
}

final class CheckTransaction extends Payment {
    private final String bankName, checkNumber;
    CheckTransaction(BigDecimal amount, String bankName, String checkNumber) {
        super(amount); this.bankName = bankName; this.checkNumber = checkNumber;
    }
    @Override boolean initiateTransaction() { setStatus(PaymentStatus.PENDING); return true; }   // clears in days
    @Override public String toString() { return "check(" + bankName + " #" + checkNumber + ")"; }
}

final class CashTransaction extends Payment {
    private final BigDecimal tendered;
    CashTransaction(BigDecimal amount, BigDecimal tendered) { super(amount); this.tendered = tendered; }
    @Override boolean initiateTransaction() {
        if (tendered.compareTo(amount()) < 0) { setStatus(PaymentStatus.DECLINED); return false; }
        setStatus(PaymentStatus.COMPLETED);
        return true;
    }
    BigDecimal change() { return tendered.subtract(amount()); }
    @Override public String toString() { return "cash"; }
}

/**
 * TEMPLATE METHOD. addInvoiceItem() is the fixed algorithm (describe, price, write a line);
 * each subclass fills in only description(). A new kind of charge = a new subclass, no edit to Invoice.
 */
abstract class RoomCharge {
    private final BigDecimal amount;
    RoomCharge(BigDecimal amount) { this.amount = amount; }
```

```
    protected abstract String description();                // the one step that varies
    final void addInvoiceItem(Invoice invoice) {            // the template: final, so the order can't change
        invoice.addItem(new InvoiceItem(description(), amount));
    }
}

final class Amenity extends RoomCharge {
    private final String name;
    Amenity(String name, BigDecimal amount) { super(amount); this.name = name; }
    @Override protected String description() { return "Amenity: " + name; }
}

final class RoomService extends RoomCharge {
    private final String request;
    RoomService(String request, BigDecimal amount) { super(amount); this.request = request; }
    @Override protected String description() { return "Room service: " + request; }
}

final class KitchenService extends RoomCharge {
    private final String dish;
    KitchenService(String dish, BigDecimal amount) { super(amount); this.dish = dish; }
    @Override protected String description() { return "Kitchen: " + dish; }
}
```

```java
// File: lld/ch15/hotel/Rooms.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Clock;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;
import java.util.TreeMap;
import java.util.concurrent.ConcurrentHashMap;

enum RoomStyle { STANDARD, DELUXE, FAMILY_SUITE, BUSINESS_SUITE }

/** What the room is doing RIGHT NOW. It cannot answer "is it free for 3 nights from the 10th?". */
enum RoomStatus { AVAILABLE, RESERVED, OCCUPIED, NOT_AVAILABLE, BEING_SERVICED, OTHER }

/** An electronic key card. Deactivated at checkout so it stops opening the door. */
final class RoomKey {
    private final String keyId, barcode;
    private final LocalDateTime issuedAt;
    private final boolean master;
    private volatile boolean active = true;
    RoomKey(String keyId, String barcode, LocalDateTime issuedAt, boolean master) {
        this.keyId = keyId; this.barcode = barcode; this.issuedAt = issuedAt; this.master = master;
    }
    String keyId()        { return keyId; }
    boolean isActive()    { return active; }
    boolean isMaster()    { return master; }
    void deactivate()     { active = false; }
}

/** Append-only log line: nothing is overwritten, so a room's history can be read back. */
record RoomHouseKeeping(LocalDateTime at, String housekeeper, String description, int minutes) {}

/**
 * A room owns its keys and its housekeeping log (composition), and the SET OF NIGHTS it is committed to.
 * Reserving nights is check-then-act, so it runs under the room's monitor: two guests can never
 * both get the night of the 10th.
 */
final class Room {
    private final String number;
    private final RoomStyle style;
    private final BigDecimal pricePerNight;
    private final boolean smoking;
    private final Set<LocalDate> bookedNights = new HashSet<>();
    private final List<RoomKey> keys = new ArrayList<>();
    private final List<RoomHouseKeeping> housekeeping = new ArrayList<>();
    private RoomStatus status = RoomStatus.AVAILABLE;

    Room(String number, RoomStyle style, BigDecimal pricePerNight, boolean smoking) {
        this.number = number; this.style = style; this.pricePerNight = pricePerNight; this.smoking = smoking;
    }
    String number()             { return number; }
    RoomStyle style()           { return style; }
    BigDecimal pricePerNight()  { return pricePerNight; }

    /** Nights are [checkIn, checkOut): the checkout day itself is free for the next guest. */
    synchronized boolean isFree(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) if (bookedNights.contains(d)) return
false;
```

```
        return true;
    }
    synchronized boolean tryReserve(LocalDate checkIn, LocalDate checkOut) {
        if (!isFree(checkIn, checkOut)) return false;              // re-entrant monitor: same lock
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.add(d);
        return true;
    }
    synchronized void release(LocalDate checkIn, LocalDate checkOut) {
        for (LocalDate d = checkIn; d.isBefore(checkOut); d = d.plusDays(1)) bookedNights.remove(d);
    }

    synchronized RoomStatus status()          { return status; }
    synchronized void setStatus(RoomStatus s) { status = s; }

    synchronized RoomKey issueKey(String keyId, LocalDateTime at) {
        RoomKey k = new RoomKey(keyId, "BC-" + keyId, at, false);
        keys.add(k);
        return k;
    }
    synchronized int deactivateGuestKeys() {
        int n = 0;
        for (RoomKey k : keys) if (!k.isMaster() && k.isActive()) { k.deactivate(); n++; }
        return n;
    }
    synchronized void addHousekeeping(RoomHouseKeeping h) { housekeeping.add(h); }
    synchronized List<RoomHouseKeeping> housekeepingLog()  { return List.copyOf(housekeeping); }
}

/** Search belongs to the LOCATION: a single room only knows about itself. */
interface Search {
    List<Room> search(RoomStyle style, LocalDate checkIn, LocalDate checkOut);
}

final class HotelLocation implements Search {
    private final String name;
    private final Map<String, Room> rooms = new ConcurrentHashMap<>();
    HotelLocation(String name) { this.name = name; }
    String name()           { return name; }
    void addRoom(Room r)    { rooms.put(r.number(), r); }
    Room room(String n)     { return rooms.get(n); }

    @Override public List<Room> search(RoomStyle style, LocalDate in, LocalDate out) {
        return rooms.values().stream().filter(r -> r.style() == style && r.isFree(in, out))
                .sorted(Comparator.comparing(Room::number)).toList();
    }
}

final class Hotel {
    private final String name;
    private final List<HotelLocation> locations = new ArrayList<>();
    Hotel(String name) { this.name = name; }
    void addLocation(HotelLocation l) { locations.add(l); }
    /** Asks every location in turn. */
    Map<String, List<Room>> search(RoomStyle style, LocalDate checkIn, LocalDate checkOut) {
        Map<String, List<Room>> result = new TreeMap<>();
        for (HotelLocation l : locations) result.put(l.name(), l.search(style, checkIn, checkOut));
        return result;
    }
}

final class SimClock extends Clock {
    private volatile Instant now;
    SimClock(Instant start) { this.now = start; }
    void advance(Duration d) { now = now.plus(d); }
    @Override public ZoneId getZone() { return ZoneOffset.UTC; }
    @Override public Clock withZone(ZoneId zone) { return this; }
    @Override public Instant instant() { return now; }
}

final class Money {
    private Money() {}
```

```
    static String fmt(BigDecimal amount) {
        String digits = amount.abs().setScale(2, RoundingMode.UNNECESSARY).toPlainString();
        return (amount.signum() < 0 ? "-$" : "$") + digits;
    }
}
```

```java
// File: lld/ch15/hotel/Stay.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.time.Clock;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.function.Function;

enum BookingStatus { REQUESTED, PENDING, CONFIRMED, CHECKED_IN, CHECKED_OUT, CANCELLED, ABANDONED }

/** The actors. Each service method takes the actor type allowed to call it, so roles are enforced by types. */
sealed interface Person permits Guest, Receptionist, Server, Housekeeper, Manager { String name(); }
record Guest(String id, String name, String email) implements Person {}
record Receptionist(String id, String name) implements Person {}
record Server(String id, String name) implements Person {}
record Housekeeper(String id, String name) implements Person {}
record Manager(String id, String name) implements Person {}

interface Notifier { void send(String guestId, String message); }

/**
 * One stay in one room. Owns its invoice (composition); only points at the room (association).
 * Records the guest by id, which answers "who booked this room?" and "which rooms did this guest book?".
 */
final class RoomBooking {
    static final LocalTime CHECK_IN_TIME = LocalTime.of(14, 0);
    static final Duration FULL_REFUND_NOTICE = Duration.ofHours(24);

    private final String reservationNumber;
    private final String guestId;
    private final Room room;
    private final LocalDate checkIn, checkOut;
    private final Invoice invoice;
    private BookingStatus status = BookingStatus.CONFIRMED;
    private BigDecimal deposit = BigDecimal.ZERO;

    RoomBooking(String reservationNumber, String guestId, Room room, LocalDate checkIn, LocalDate checkOut) {
        this.reservationNumber = reservationNumber; this.guestId = guestId; this.room = room;
        this.checkIn = checkIn; this.checkOut = checkOut; this.invoice = new Invoice("INV-" + reservationNumber);
    }
    String reservationNumber() { return reservationNumber; }
    String guestId()           { return guestId; }
    Room room()                { return room; }
    LocalDate checkIn()        { return checkIn; }
    LocalDate checkOut()       { return checkOut; }
    Invoice invoice()          { return invoice; }
    long nights()              { return checkIn.until(checkOut).getDays(); }
    synchronized BookingStatus status()        { return status; }
    synchronized void setStatus(BookingStatus s) { status = s; }
    synchronized BigDecimal deposit()             { return deposit; }
    synchronized void setDeposit(BigDecimal d)    { deposit = d; }

    /** The refund rule lives in exactly ONE method: full refund if cancelled more than 24h before check-in. */
    BigDecimal refundDue(LocalDateTime now) {
        Duration notice = Duration.between(now, checkIn.atTime(CHECK_IN_TIME));
        // compareTo, not toHours(): toHours() truncates, so 24h30m would wrongly count as "24, not more than 24".
        return notice.compareTo(FULL_REFUND_NOTICE) > 0 ? deposit() : BigDecimal.ZERO;
    }
}

/** The front desk: every flow of a stay, from booking to checkout. */
```

```
final class FrontDesk {
    private final HotelLocation location;
    private final Clock clock;
    private final Notifier notifier;
    private final Map<String, RoomBooking> bookings = new ConcurrentHashMap<>();
    private final List<Person> staff = new CopyOnWriteArrayList<>();
    private final AtomicInteger seq = new AtomicInteger(), keySeq = new AtomicInteger();

    FrontDesk(HotelLocation location, Clock clock, Notifier notifier) {
        this.location = location; this.clock = clock; this.notifier = notifier;
    }

    private LocalDateTime now() { return LocalDateTime.now(clock); }

    void addRoom(Manager by, Room r) { location.addRoom(r); }

    /** The manager adds new workers (receptionists, servers, housekeepers). */
    void hire(Manager by, Person worker) {
        if (worker instanceof Guest) throw new IllegalArgumentException("a guest is not a worker");
        staff.add(worker);
    }
    List<String> staff() { return staff.stream().map(Person::name).toList(); }

    /** Guest or receptionist books: same call, two actors. The room's monitor makes it race-free. */
    RoomBooking book(Guest guest, Room room, LocalDate checkIn, LocalDate checkOut, Payment deposit) {
        if (!checkIn.isBefore(checkOut)) throw new IllegalArgumentException("check-out must be after check-in");
        if (!room.tryReserve(checkIn, checkOut))
            throw new IllegalStateException("room " + room.number() + " is not free " + checkIn + ".." +
checkOut);
        RoomBooking b = new RoomBooking("R" + seq.incrementAndGet(), guest.id(), room, checkIn, checkOut);
        if (!deposit.initiateTransaction()) {                  // payment failed: give the nights back
            room.release(checkIn, checkOut);
            throw new IllegalStateException("deposit " + deposit.status());
        }
        b.setDeposit(deposit.amount());
        bookings.put(b.reservationNumber(), b);
        notifier.send(guest.id(), b.reservationNumber() + " confirmed: room " + room.number() + " " + checkIn +
".." + checkOut);
        return b;
    }

    RoomBooking bookFor(Receptionist by, Guest guest, Room room, LocalDate in, LocalDate out, Payment deposit) {
        return book(guest, room, in, out, deposit);
    }

    RoomKey checkIn(Receptionist by, String reservationNumber) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            if (b.status() != BookingStatus.CONFIRMED) throw new IllegalStateException(reservationNumber + " is "
+ b.status());
            if (now().toLocalDate().isBefore(b.checkIn())) throw new IllegalStateException("too early to check
in");
            b.setStatus(BookingStatus.CHECKED_IN);
            b.room().setStatus(RoomStatus.OCCUPIED);
            return b.room().issueKey("K" + keySeq.incrementAndGet(), now());
        }
    }

    void addCharge(Server by, String reservationNumber, RoomCharge charge) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {                     // same monitor as checkOut: no charge can slip in after the bill
is totalled
            if (b.status() != BookingStatus.CHECKED_IN) throw new IllegalStateException("guest not in house");
            charge.addInvoiceItem(b.invoice());
        }
    }

    /** Adds the room nights, credits the deposit, takes payment for the balance, kills the keys. */
    Invoice checkOut(Receptionist by, String reservationNumber, Function<BigDecimal, Payment> payWith) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
```

```
            if (b.status() != BookingStatus.CHECKED_IN) throw new IllegalStateException(reservationNumber + " is "
+ b.status());
            Invoice inv = b.invoice();
            BigDecimal price = b.room().pricePerNight();
            List<InvoiceItem> finalLines = new ArrayList<>(List.of(
                    new InvoiceItem(b.nights() + " nights x " + Money.fmt(price),
price.multiply(BigDecimal.valueOf(b.nights()))),
                    new InvoiceItem("Deposit paid", b.deposit().negate())));
            BigDecimal balance = finalLines.stream().map(InvoiceItem::amount).reduce(inv.total(),
BigDecimal::add);
            Payment p = payWith.apply(balance);
            // Write the final lines only once payment succeeds, so a declined payment can be retried
            // without the nights and deposit lines being added to the invoice twice.
            if (!p.initiateTransaction()) throw new IllegalStateException("payment " + p.status() + ", guest still
checked in");
            finalLines.forEach(inv::addItem);
            int killed = b.room().deactivateGuestKeys();
            b.setStatus(BookingStatus.CHECKED_OUT);
            b.room().setStatus(RoomStatus.BEING_SERVICED);
            notifier.send(b.guestId(), "checked out, paid " + Money.fmt(inv.total()) + " by " + p + " [" +
p.status()
                    + "], " + killed + " key(s) deactivated");
            return inv;
        }
    }

    void logHousekeeping(Housekeeper by, Room room, String description, int minutes) {
        room.addHousekeeping(new RoomHouseKeeping(now(), by.name(), description, minutes));
        room.setStatus(RoomStatus.AVAILABLE);
    }

    BigDecimal cancel(Person by, String reservationNumber) {
        RoomBooking b = get(reservationNumber);
        synchronized (b) {
            if (b.status() != BookingStatus.CONFIRMED) throw new IllegalStateException(reservationNumber + " is "
+ b.status());
            BigDecimal refund = b.refundDue(now());
            b.room().release(b.checkIn(), b.checkOut());
            b.setStatus(BookingStatus.CANCELLED);
            notifier.send(b.guestId(), reservationNumber + " cancelled by " + by.name() + ", refund " +
Money.fmt(refund));
            return refund;
        }
    }

    /** System job: remind guests whose check-in or check-out is within the next 24 hours. */
    void sendReminders() {
        LocalDateTime now = now(), soon = now.plusHours(24);
        bookings.values().stream().sorted((x, y) ->
x.reservationNumber().compareTo(y.reservationNumber())).forEach(b -> {
            LocalDateTime in = b.checkIn().atTime(RoomBooking.CHECK_IN_TIME), out = b.checkOut().atTime(11, 0);
            if (b.status() == BookingStatus.CONFIRMED && in.isAfter(now) && !in.isAfter(soon))
                notifier.send(b.guestId(), "reminder: check-in for " + b.reservationNumber() + " at " + in);
            if (b.status() == BookingStatus.CHECKED_IN && out.isAfter(now) && !out.isAfter(soon))
                notifier.send(b.guestId(), "reminder: check-out for " + b.reservationNumber() + " at " + out);
        });
    }

    List<String> roomsBookedBy(String guestId) {
        return bookings.values().stream().filter(b -> b.guestId().equals(guestId) && b.status() !=
BookingStatus.CANCELLED)
                .map(b -> b.room().number()).sorted().toList();
    }

    List<String> whoBooked(String roomNumber) {
        return bookings.values().stream().filter(b -> b.room().number().equals(roomNumber) && b.status() !=
BookingStatus.CANCELLED)
                .map(b -> b.guestId() + "@" + b.checkIn()).sorted().toList();
    }

    RoomBooking get(String n) {
```

```
        RoomBooking b = bookings.get(n);
        if (b == null) throw new IllegalArgumentException("no booking " + n);
        return b;
    }
}
```

```java
// File: lld/ch15/hotel/Main.java
package lld.ch15.hotel;

import java.math.BigDecimal;
import java.time.Duration;
import java.time.Instant;
import java.time.LocalDate;
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        SimClock clock = new SimClock(Instant.parse("2026-07-01T09:00:00Z"));
        Notifier notifier = (g, m) -> System.out.println("  [notify " + g + "] " + m);
        Hotel hotel = new Hotel("Seaside Hotels");
        HotelLocation goa = new HotelLocation("Goa"), kochi = new HotelLocation("Kochi");
        hotel.addLocation(goa);
        hotel.addLocation(kochi);
        FrontDesk desk = new FrontDesk(goa, clock, notifier);
        Manager boss = new Manager("MG1", "Maya");
        desk.addRoom(boss, new Room("101", RoomStyle.DELUXE, usd("120.00"), false));
        desk.addRoom(boss, new Room("102", RoomStyle.DELUXE, usd("120.00"), false));
        desk.addRoom(boss, new Room("201", RoomStyle.FAMILY_SUITE, usd("250.00"), false));
        kochi.addRoom(new Room("K-1", RoomStyle.DELUXE, usd("90.00"), true));
        Room r101 = goa.room("101");
        Receptionist rita = new Receptionist("RC1", "Rita");
        desk.hire(boss, rita);
        desk.hire(boss, new Server("SV1", "Sam"));
        desk.hire(boss, new Housekeeper("HK1", "Hari"));
        System.out.println("== 0. Manager adds rooms and hires staff: " + desk.staff() + " ==\n");

        LocalDate jul10 = LocalDate.of(2026, 7, 10), jul13 = LocalDate.of(2026, 7, 13);
        System.out.println("== 1. Search DELUXE for Jul 10-13 across locations ==");
        hotel.search(RoomStyle.DELUXE, jul10, jul13).forEach((loc, rooms) ->
                System.out.println("  " + loc + " -> " + rooms.stream().map(Room::number).toList()));

        System.out.println("\n== 2. 6 guests try to book room 101 for overlapping dates at once ==");
        ConcurrentLinkedQueue<String> results = new ConcurrentLinkedQueue<>();
        CountDownLatch go = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(6)) {
            for (int i = 1; i <= 6; i++) {
                Guest g = new Guest("G" + i, "Guest " + i, "g" + i + "@mail.com");
                LocalDate out = jul10.plusDays(2 + i % 2);     // Jul 10-12 or Jul 10-13: all overlap
                pool.submit(() -> {
                    go.await();
                    try { desk.book(g, r101, jul10, out, new CreditCardTransaction(usd("120.00"), g.name()));
results.add("OK"); }
                    catch (IllegalStateException e) { results.add("NO"); }
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("  successful bookings = " + results.stream().filter("OK"::equals).count() + " of 6");
        System.out.println("  who booked 101? " + desk.whoBooked("101"));

        System.out.println("\n== 3. Back-to-back stays share the changeover day ==");
        Guest asha = new Guest("ASHA", "Asha", "asha@mail.com");
        RoomBooking stay = desk.bookFor(rita, asha, goa.room("102"), jul10, jul13, new
CashTransaction(usd("120.00"), usd("200.00")));
        RoomBooking next = desk.book(new Guest("BEN", "Ben", "ben@mail.com"), goa.room("102"), jul13,
                LocalDate.of(2026, 7, 15), new CheckTransaction(usd("120.00"), "HDFC", "000123"));
        System.out.println("  rooms booked by ASHA: " + desk.roomsBookedBy("ASHA"));

        System.out.println("\n== 4. Cancellation: >24h before check-in = full refund, later = none ==");
        RoomBooking suite = desk.book(asha, goa.room("201"), jul10, jul13, new
CreditCardTransaction(usd("250.00"), "Asha"));
        Guest carl = new Guest("CARL", "Carl", "carl@mail.com");
```

```
        RoomBooking lastMinute = desk.book(carl, goa.room("201"), LocalDate.of(2026, 7, 2), LocalDate.of(2026, 7,
3),
                new CreditCardTransaction(usd("250.00"), "Carl"));
        clock.advance(Duration.ofHours(10));              // Jul 1 19:00: Carl's check-in (Jul 2 14:00) is 19h
away
        desk.cancel(carl, lastMinute.reservationNumber());
        desk.cancel(asha, suite.reservationNumber());     // Asha's is 8+ days away

        System.out.println("\n== 5. Reminders, check-in (key), room service, check-out (invoice) ==");
        clock.advance(Duration.ofDays(8).plusHours(13));  // Jul 10 08:00
        desk.sendReminders();
        clock.advance(Duration.ofHours(6));               // Jul 10 14:00
        RoomKey key = desk.checkIn(rita, stay.reservationNumber());
        System.out.println("  issued key " + key.keyId() + " active=" + key.isActive());
        Server sam = new Server("SV1", "Sam");
        desk.addCharge(sam, stay.reservationNumber(), new RoomService("Extra towels", usd("5.00")));
        desk.addCharge(sam, stay.reservationNumber(), new KitchenService("Fish curry dinner", usd("24.00")));
        desk.addCharge(sam, stay.reservationNumber(), new Amenity("Spa access", usd("30.00")));
        clock.advance(Duration.ofDays(2).plusHours(19));  // Jul 13 09:00: checkout at 11:00 today
        desk.sendReminders();
        try { desk.checkOut(rita, stay.reservationNumber(), total -> new CashTransaction(total, usd("100.00"))); }
        catch (IllegalStateException e) { System.out.println("  cash short: " + e.getMessage()); }
        Invoice inv = desk.checkOut(rita, stay.reservationNumber(), total -> new CreditCardTransaction(total,
"Asha"));
        inv.items().forEach(i -> System.out.printf("    %-28s %10s%n", i.description(), Money.fmt(i.amount())));
        System.out.printf("    %-28s %10s%n", "TOTAL", Money.fmt(inv.total()));
        System.out.println("  key active after checkout? " + key.isActive() + ", room 102 is " +
goa.room("102").status());

        System.out.println("\n== 6. Housekeeping log ==");
        desk.logHousekeeping(new Housekeeper("HK1", "Hari"), goa.room("102"), "Full clean, linen change", 40);
        goa.room("102").housekeepingLog().forEach(h -> System.out.println("  " + h));
        System.out.println("  room 102 is " + goa.room("102").status());
    }

    private static BigDecimal usd(String amount) { return new BigDecimal(amount); }
}
```

Output:

```
== 0. Manager adds rooms and hires staff: [Rita, Sam, Hari] ==

== 1. Search DELUXE for Jul 10-13 across locations ==
  Goa -> [101, 102]
  Kochi -> [K-1]

== 2. 6 guests try to book room 101 for overlapping dates at once ==
  [notify G6] R1 confirmed: room 101 2026-07-10..2026-07-12
  successful bookings = 1 of 6
  who booked 101? [G6@2026-07-10]

== 3. Back-to-back stays share the changeover day ==
  [notify ASHA] R2 confirmed: room 102 2026-07-10..2026-07-13
  [notify BEN] R3 confirmed: room 102 2026-07-13..2026-07-15
  rooms booked by ASHA: [102]

== 4. Cancellation: >24h before check-in = full refund, later = none ==
  [notify ASHA] R4 confirmed: room 201 2026-07-10..2026-07-13
  [notify CARL] R5 confirmed: room 201 2026-07-02..2026-07-03
  [notify CARL] R5 cancelled by Carl, refund $0.00
  [notify ASHA] R4 cancelled by Asha, refund $250.00

== 5. Reminders, check-in (key), room service, check-out (invoice) ==
  [notify G6] reminder: check-in for R1 at 2026-07-10T14:00
  [notify ASHA] reminder: check-in for R2 at 2026-07-10T14:00
  issued key K1 active=true
  [notify ASHA] reminder: check-out for R2 at 2026-07-13T11:00
  [notify BEN] reminder: check-in for R3 at 2026-07-13T14:00
  cash short: payment DECLINED, guest still checked in
  [notify ASHA] checked out, paid $299.00 by card(Asha) [COMPLETED], 1 key(s) deactivated
    Room service: Extra towels        $5.00
    Kitchen: Fish curry dinner       $24.00
    Amenity: Spa access              $30.00
    3 nights x $120.00              $360.00
    Deposit paid                   -$120.00
    TOTAL                           $299.00
  key active after checkout? false, room 102 is BEING_SERVICED

== 6. Housekeeping log ==
  RoomHouseKeeping[at=2026-07-13T09:00, housekeeper=Hari, description=Full clean, linen change, minutes=40]
  room 102 is AVAILABLE
```

What to notice: section 0 — only a `Manager` can add rooms and hire staff (the course's "Manager adds new workers"); section 2 — six concurrent bookings of room 101 for overlapping dates produce exactly one booking (which guest wins varies per run); section 3 — Ben's stay starting the 13th fits right after Asha's ending the 13th (half-open ranges); section 4 — Carl cancels 19 hours before check-in and gets nothing, Asha cancels 8 days ahead and gets a full refund; section 5 — reminders fire within 24 hours of check-in/check-out, the key is issued at check-in and dead after check-out, a declined cash payment leaves the guest checked in and the invoice untouched (the retry by card does not duplicate any line), and the invoice lines add up (room service $5 + kitchen $24 + spa $30 + 3 nights $360 − $120 deposit = $299); section 6 — housekeeping is logged and the room becomes available.

Concurrency & edge cases

- **Double booking a room-night.** Prevented by `Room.tryReserve` being `synchronized`: the check and the add of every night are one atomic step. The demo's 6-thread race always has one winner.
- **Search is advisory.** `search` may show a room that is gone a millisecond later; the booking itself (`tryReserve`) is the real check — so always handle "no longer available" on book.
- **Booking-level transitions** (`checkIn`, `checkOut`, `cancel`) run under the booking's monitor, so a cancel can't interleave with a check-in. Lock order is always booking → room.
- **The 24-hour rule, exactly.** `refundDue` compares `Duration`s (`notice.compareTo(24h) > 0`). The tempting `Duration.toHours() > 24` is wrong: `toHours()` drops the minutes, so a cancellation 24 h 30 min ahead would count as "24, not more than 24" and lose its refund.
- **Checkout is retry-safe.** The nights and deposit lines are written to the invoice only after the payment succeeds, and `addCharge` takes the same booking monitor as `checkOut`, so no charge can be

added between totalling the bill and closing it.

- **Edge cases:** check-out date not after check-in; checking in before the check-in date; adding charges when the guest is not in house; cancelling after check-in; cash tendered below the amount (declined); a check payment that stays `PENDING`.
- **Chain-wide inventory.** The course notes that storing nights on each room is simple but doesn't scale to a chain selling through many channels: real systems keep availability in one place (an **inventory table** keyed by `(room_type, date)` with a count, updated with a conditional `UPDATE ... SET sold = sold + 1 WHERE sold < capacity`), so two sites or two booking channels can't sell the same night.

Extensions & follow-up questions

- **"A family wants two rooms in one booking."** The course flags this: move the dates and the invoice *up one level* to a `Reservation` that holds several `RoomStay`s (each with its room and nights). Reserving several rooms then needs all-or-nothing across rooms — lock rooms in a fixed order (by room number) to avoid deadlock, or do it in one database transaction.
- **"Book by room type, assign the room at check-in."** Many hotels sell a *type* for a date range and pick the physical room later. Keep per-type-per-night counts for selling; assign a specific room at check-in (housekeeping status permitting).
- **"Seasonal rates."** A `RatePlan` / `PricingStrategy` gives the price per night (weekends, seasons, loyalty tiers); the nights line becomes one line per night.
- **"Overbooking."** Allow selling capacity × (1 + overbooking factor) of a type; the ones who can't be housed are "walked" to a partner hotel — a business policy, pluggable as a strategy on the availability check.
- **"Separate the booker from the occupants."** The course names this as missing: add `Occupant`s to the stay (names for fire safety/registration) distinct from the paying guest.
- **"Lost key."** Deactivate all active guest keys for that room and issue a new one — `deactivateGuestKeys()` + `issueKey()`.

Real-world example

Property management systems such as **Oracle Hospitality OPERA** run the front desk this way: reservations are made against room types and dates, a specific room is assigned by check-in, the folio (the guest's bill) collects charges from restaurants, minibar and spa as separate postings, and housekeeping status (dirty, clean, inspected) controls whether a room can be assigned. At check-in, the desk encodes an RFID (radio-frequency identification, a contactless chip) key card with an electronic lock system (for example ASSA ABLOY's VingCard or dormakaba) that is valid only until the check-out time. Online, **Booking.com** shows each rate's cancellation policy (for example "free cancellation until 24 hours before arrival") — exactly the one-method refund rule of this design, but chosen per rate plan.

In the interview

30–45 minute plan. (1) Clarify: room types, date-range search, 24-hour refund, charges, payment types, keys, housekeeping. (2) Actors and use cases. (3) Entities, stating early: **"a status field is not availability"** — rooms keep booked nights. (4) Code `tryReserve` (synchronized), `book`, `checkIn`, `checkOut` with the invoice. (5) Template Method for charges; payment subclasses; refund rule. (6) Follow-ups: two rooms per booking, chain-wide inventory, overbooking.

> **30-second answer:** "A `Hotel` has `HotelLocation`s which own `Room`s and implement search by type and date range. Each room keeps the set of nights it's committed to, and reserving nights is a synchronized check-then-act so two guests can't get the same night. A `RoomBooking` owns an `Invoice` made of lines; room service, kitchen and amenities are `RoomCharge` subclasses using Template Method to add a line. Payment is credit card, check or cash subclasses. Cancellation refunds in full if more than 24 hours before check-in — one method. Check-in issues a key card; check-out totals the invoice, takes payment, kills the keys and hands the room to housekeeping."

Self-check

- **Q:** Why isn't `RoomStatus` enough to know if a room can be booked? **A:** It only says what the room is doing now; a booking asks about a range of future nights.
- **Q:** Why are nights half-open `[checkIn, checkOut)`? **A:** The checkout day's night is not used, so the next guest can check in that day.
- **Q:** Which pattern lets a new charge type be added without touching `Invoice`? **A:** Template Method (`RoomCharge.addInvoiceItem` is fixed; subclasses supply `description()`).
- **Q:** Where does the refund rule live, and why? **A:** In `RoomBooking.refundDue` — one place to change when the policy changes.
- **Q:** How is it guaranteed only a server can add a room-service charge? **A:** `addCharge` takes a `Server` parameter; other actor types don't compile.

TL;DR

Rooms are sold night by night: each room keeps its booked nights and reserves them atomically. Search lives on the location. The bill is a list of lines filled by `RoomCharge` Template Methods; three payment subclasses; one refund method (24 hours). Keys issued at check-in, killed at check-out; housekeeping is an append-only log (entries are only added, never changed or removed).

Chapter summary

Problem	Inventory unit	How a hold/booking is made safe	Payment failure handling	Cancellation
Library	`BookItem` (copy)	`synchronized` member → item (fixed order)	— (fines recorded)	Return → FIFO waitlist per title gets the copy
Movie tickets	`ShowSeat` (seat × show)	Per-show monitor; all-or-nothing lock with TTL; atomic commit	Declined → release; lost hold → refund (compensation)	Refund per `CancellationPolicy`; seats back on sale
Concert tickets	Numbered seat state / GA counter	Lock-free CAS; waiting room; per-user limit	Same compensation as movies	Released tickets offered to waitlist first
Hotel	Room × night	`synchronized` `Room.tryReserve` over a date range	Failed deposit → nights released	Full refund if > 24h before check-in

Five sentences to remember for any booking interview:

1. Name the *real* unit of inventory first (copy, show seat, room-night, GA count).
2. Every "if free then take" must be one atomic step — a monitor, a CAS, or a conditional `UPDATE`.
3. Temporary holds need an expiry, or abandoned checkouts block inventory forever.
4. Money can't be rolled back — failures after payment need a compensating refund.
5. In-process locks protect one server; with many servers, the database (constraints, conditional updates) or a TTL store like Redis is the real guarantee.

Chapter 16 — Case Studies: Booking & Inventory (Part 2)

> **Where this fits.** Chapter 15 covered the first group of booking problems. This chapter finishes the family with four more: a car rental company, a restaurant, an airline and a university's course registration. They look different on the surface, but they are the same problem underneath: **a limited resource, a calendar, and many people trying to claim it at once.** Once you see that shared skeleton, each new booking problem takes minutes to set up in an interview.

The shared skeleton of every booking problem

Idea	Car rental	Restaurant	Airline	Course registration
The thing being booked	a vehicle	a table	a seat on one flight instance	a seat in one section
The "calendar" it is booked on	days	hourly time slots	the flight instance (route + date)	the term
The check-then-claim step that must be atomic	`Vehicle.tryBook`	`Table.tryHold`	`FlightInstance.holdSeats`	`Section.enroll`
What happens when it is full	search another car	try the next table	pick another seat or flight	waitlist
Money	bill with lines + late fee	check split by seat + tax	fare per seat class, paid per itinerary	none (fees are an extension)

A **resource** is anything with a limited supply (a car, a table, a seat). A **race condition** is a bug where the result depends on which of two threads runs first — here, two people both "win" the same seat because both checked "free?" before either wrote "taken".

A **thread** is one independent line of execution inside a running program; a server handles many requests at once by running many threads side by side. A step is **atomic** when it happens as one indivisible unit: no other thread can see it half-done or slip in between its parts. **Concurrency** simply means several things are in progress at the same time.

How to run every example in this chapter

```
cd chapters/16-case-booking-2/code
javac --release 21 -d /tmp/lld_16 $(find . -name '*.java')
java -cp /tmp/lld_16 lld.ch16.carrental.Main          # or restaurant, airline, courseregistration
```

`javac` is the Java compiler (it turns `.java` source files into runnable `.class` files), `--release 21` targets Java version 21, and `-cp` (the **classpath**) tells `java` which folder to load compiled classes from. All four `Main` classes compile with Java 21 (standard library only — the classes that ship with Java itself, no extra downloads) and run. The output printed under each program is its real output. Where threads race, the *counts* are always the same, but *which* racer wins can differ from run to run.

> **Code-style note.** To keep each design readable, one file sometimes holds several small package-private types (visible only to other classes in the same package, i.e. the same folder of code) — for example `Types.java` holds the enums and records. An **enum** is a type with a fixed list of named values (`CAR`, `TRUCK` …); a **record** is a compact Java class that just holds a few fixed values and gets `equals`, `hashCode` and `toString` for free. That is legal Java. In production you would usually give each top-level type its own file.

> **Money.** All prices use `java.math.BigDecimal`, never `double`. A `double` is a binary floating-point number (it stores values in base 2 with limited precision); it cannot represent 0.1 exactly, so

> sums of prices drift by fractions of a cent. `BigDecimal` stores exact decimal digits.

Design a Car Rental System

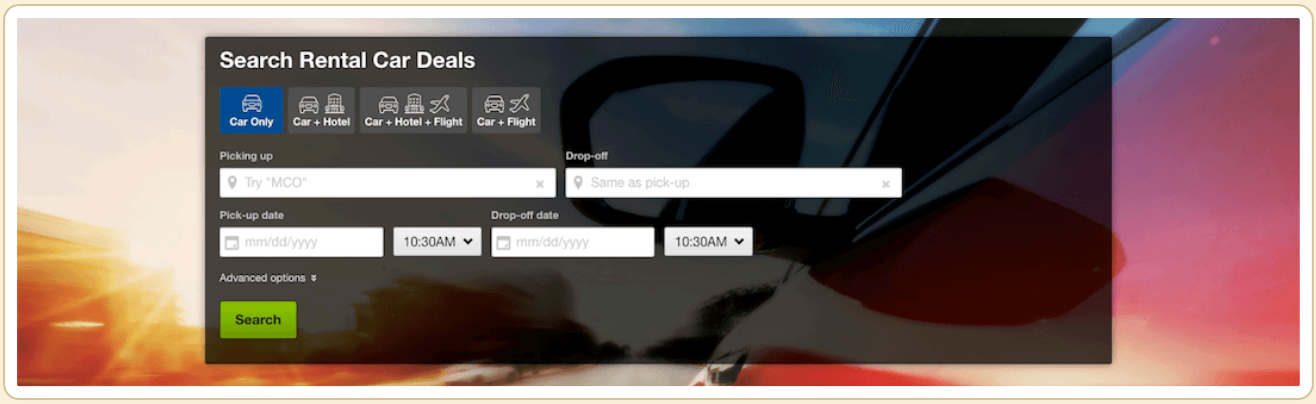


Figure — A car rental company rents vehicles from many branches (course diagram)

Problem & clarifying questions

A **car rental system** is the software a rental company uses to rent vehicles for a short time — from a few hours to a few weeks. Companies usually have many branches, often near airports and city centres, and a customer may return a car to a different branch from the one they picked it up at.

Questions worth asking in the first two minutes, with the answers we assume:

Question	Assumed answer
Which vehicle kinds?	Cars, trucks, SUVs, vans and motorcycles.
One branch or many?	Many branches, each with its own vehicles.
Can the drop-off branch differ from the pick-up branch?	Yes, the reservation records both.
How is price worked out?	A daily rate per vehicle type, plus add-ons, plus a late fee per day late.
What can be added?	Insurance, equipment (navigation, child seat, ski rack), services (roadside assistance, wifi) and additional drivers.
How are vehicles identified?	Each has a unique barcode; a barcode reader scans it at pick-up and return.
Do we need history?	Yes: a vehicle log of every event (reserved, picked up, returned, fuelling, cleaning, damage).
Concurrency?	Many members book at once; two must never get the same car on overlapping days.

Requirements

Functional

1. Rent different vehicle types: car, truck, SUV, van, motorcycle.
2. Each vehicle is added with a unique barcode and details, including the parking stall number that tells staff where it is parked.
3. Answer "which member took this vehicle?" and "which vehicles has this member rented?".
4. Charge a late fee when a vehicle comes back after its due date.
5. Members search the inventory and reserve any available vehicle.
6. Send notifications when a pick-up date is near, when a vehicle is near its due date, and when it is overdue.

7. Read vehicle barcodes.
8. Members can cancel reservations.
9. Keep a vehicle log of all events on a vehicle.
10. Members can add rental insurance.
11. Members can rent extra equipment (navigation, child seat, ski rack).
12. Members can add services (roadside assistance, additional driver, wifi).

Non-functional

- **Correctness under concurrency:** no double booking of a vehicle for overlapping days.
- **Extensibility:** adding a new kind of add-on or a new pricing rule should not change the reservation code (the **Open/Closed Principle**: code should be open to extension by adding new classes, but closed to editing what already works).
- **Auditability:** every charge is a visible bill line, every vehicle event is a log entry.

Use cases

Four **actors** (people or systems that use the software):

- **Receptionist** — adds and edits vehicles and workers; can also reserve vehicles for a customer.
- **Member** — searches the catalogue, reserves, picks up and returns a vehicle.
- **Worker** — handles returned vehicles and updates the vehicle log (cleaning, refuelling, damage).
- **System** — sends notifications (pick-up reminders, overdue alerts, cancellations).

A **use case** is one goal an actor can achieve with the system, such as "reserve a vehicle". Top use cases: add/remove/edit vehicle; search catalogue by type and availability; register or cancel membership; reserve vehicle; check out (pick up) vehicle; return vehicle; add equipment or services; update the vehicle log.

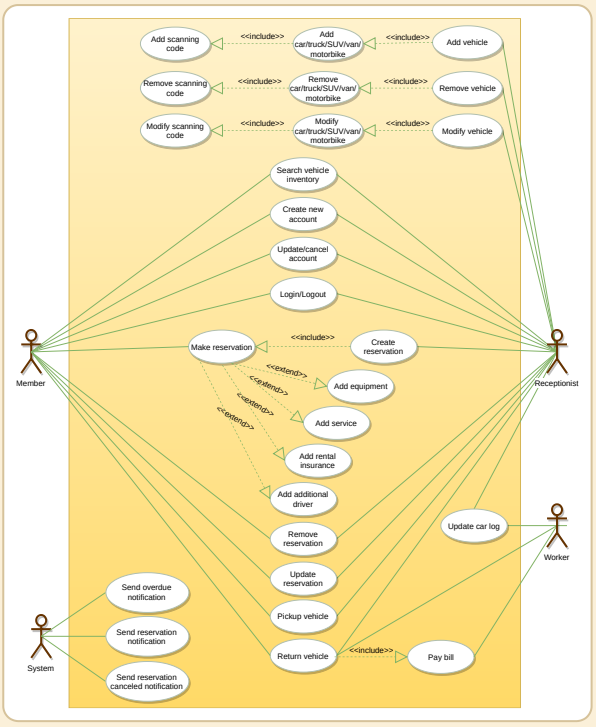


Figure — Use case diagram: receptionist, member, worker and system and the use cases each triggers (course diagram)

A **use case diagram** is a UML (Unified Modeling Language, a standard set of software diagrams) picture: stick figures are actors, ovals are use cases, and lines show who triggers what. In the figure above, notice that reserving is shared by the member and the receptionist, while updating the log belongs to the worker.

Core entities

Class	Responsibility
`CarRentalSystem`	Top-level object; owns the branches and the reservation service.
`CarRentalLocation`	One branch (name, code) with its own `VehicleInventory`.
`VehicleInventory`	Three indexes (lookup maps) over a branch's vehicles: by barcode, by type, by model. Answers "which SUVs are free on these days?".
`Vehicle`	Barcode, plate, make, model, type, seats, parking stall, mileage, **the set of booked days**, and its log.
`VehicleLogEntry`	One event on a vehicle (date, type, note).
`DateRange`	Half-open range of days `[start, end)` (includes `start`, excludes `end`); `end` is the due date.
`Reservation`	The rental: member, vehicle, dates, pick-up and return branch, add-ons, drivers, bill, status. Holds the business rules.
`ReservationAddOn` (+ `RentalInsurance`, `Equipment`, `Service`)	Anything that adds a charge to a rental.
`AdditionalDriver`	A named extra driver on the rental (no charge in our model).
`Bill`, `BillItem`	The one place that knows what a rental costs: a list of lines.
`PricingStrategy`, `LateFeePolicy`	Pluggable (swappable without code changes) rules for the base charge and the late fee.
`ReservationService`	Creates reservations atomically, answers the two "who rented what" questions, runs reminders.
`Notifier`, `PaymentMethod`	Ports (interfaces to outside systems) for sending messages and taking money.

Class diagram

A **class diagram** shows the classes, their fields and methods, and how they relate (owns, refers to, inherits from). The course builds its model in seven stages: money and messages, add-ons, vehicles, search, reservations, actors, and finally the system and its branches.

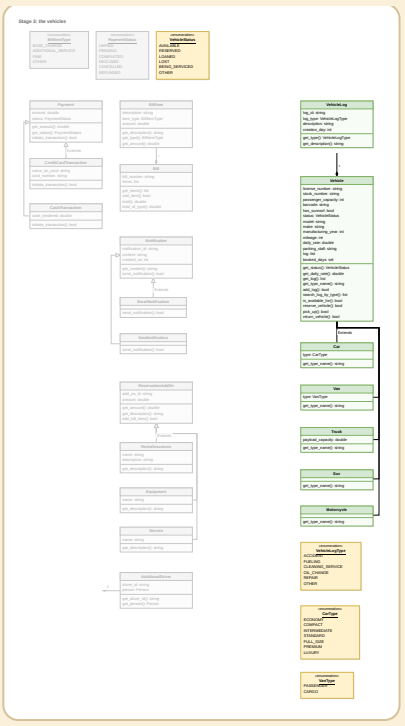


Figure — Vehicle, its log and the vehicle sub-types (course diagram)

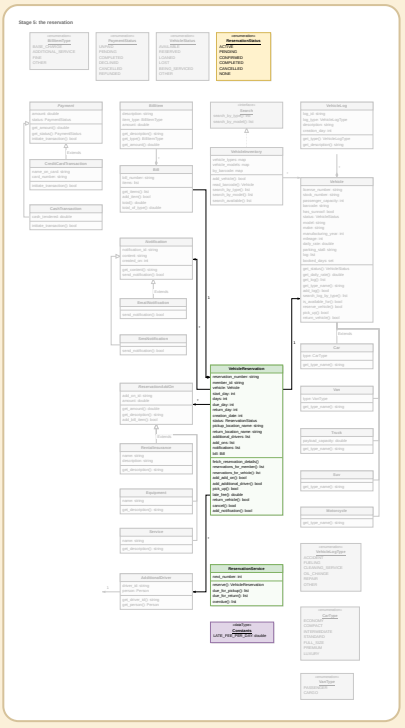


Figure — The reservation owns its bill and refers to the vehicle, add-ons, drivers and notifications (course diagram)

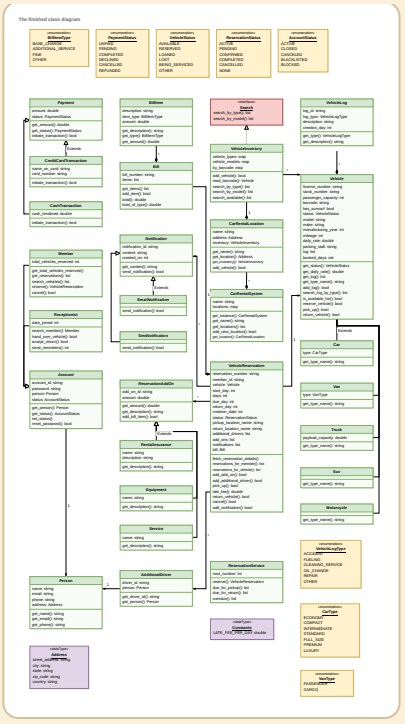
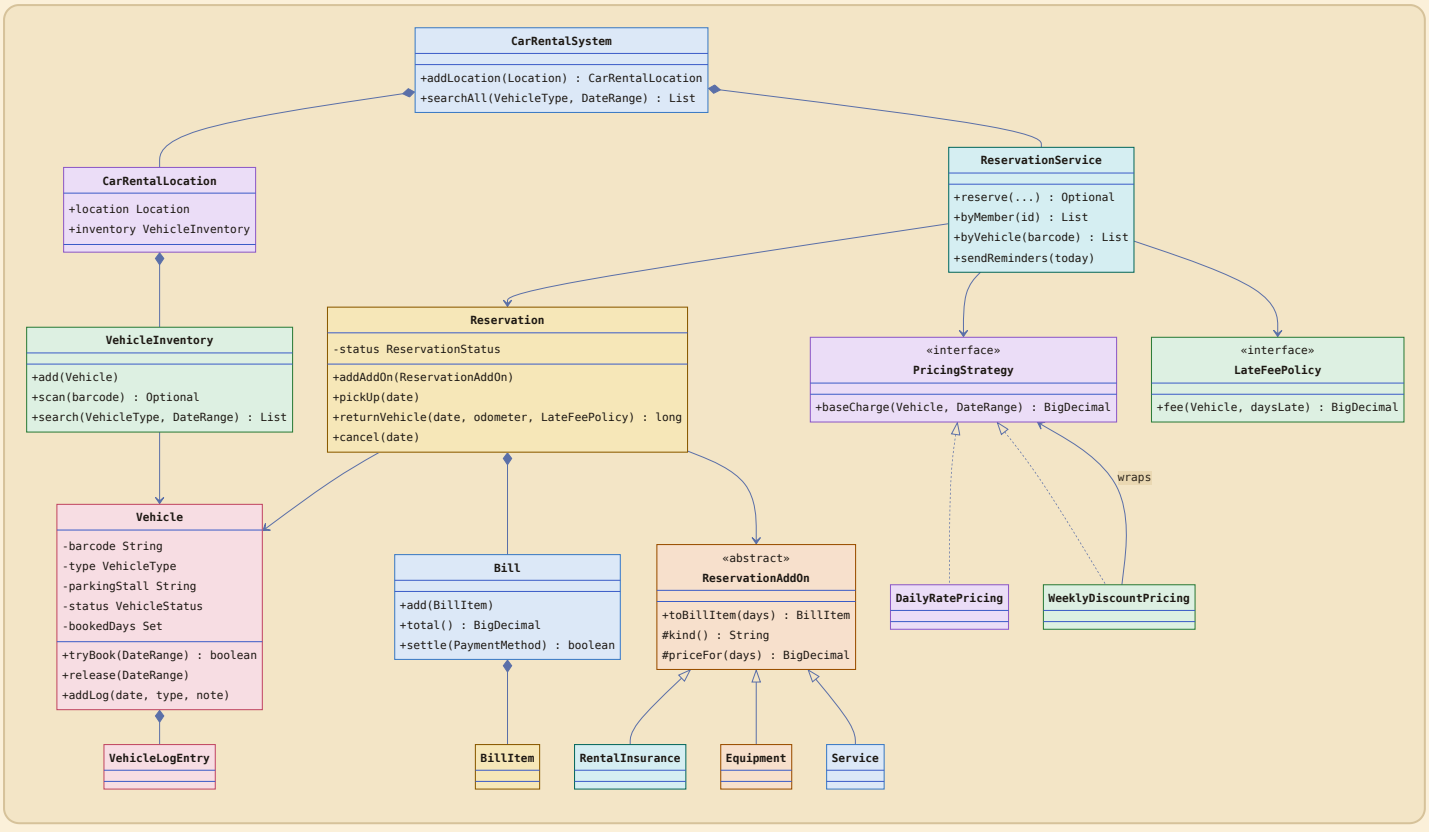


Figure — The finished class diagram of the car rental system (course diagram)

In the finished figure above, two ownership chains matter: the system owns its branches and each branch owns its inventory (filled diamonds, **composition** — the part dies with the whole), while the inventory only *refers to* vehicles (plain arrow, **association**). A reservation owns its bill but only refers to the vehicle.

Our Java design follows the same shape, with two deliberate differences: vehicle kinds are an enum (`VehicleType`) rather than five empty subclasses (classes that inherit from a parent class), because the subclasses add no behaviour; and pricing is a strategy object (an object you plug in to decide one varying rule) rather than a constant.



Key flows

The figures below are **activity diagrams** — flowcharts of one task, with diamonds for decisions. **Pick up a vehicle.** The receptionist scans the customer's licence (or searches by name), checks there is a valid reservation, scans the vehicle's barcode, and hands over the keys. If there is no reservation, the member can reserve a free vehicle on the spot.

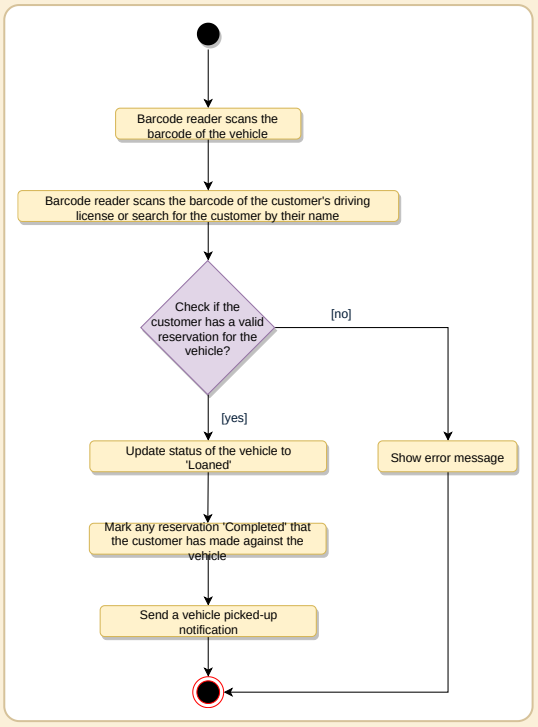


Figure — Activity diagram: picking up a vehicle (course diagram)

Return a vehicle. A worker scans the vehicle barcode; if it is past the due date, the system adds a late fee; the worker checks for damage and fuel; the bill is settled.

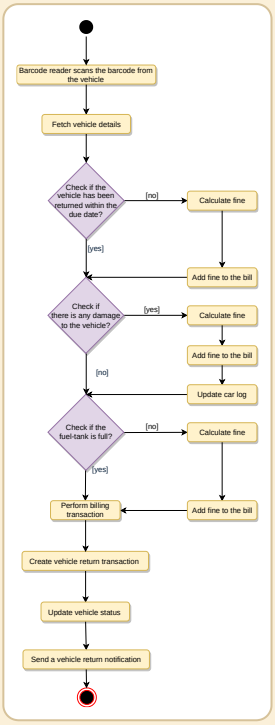
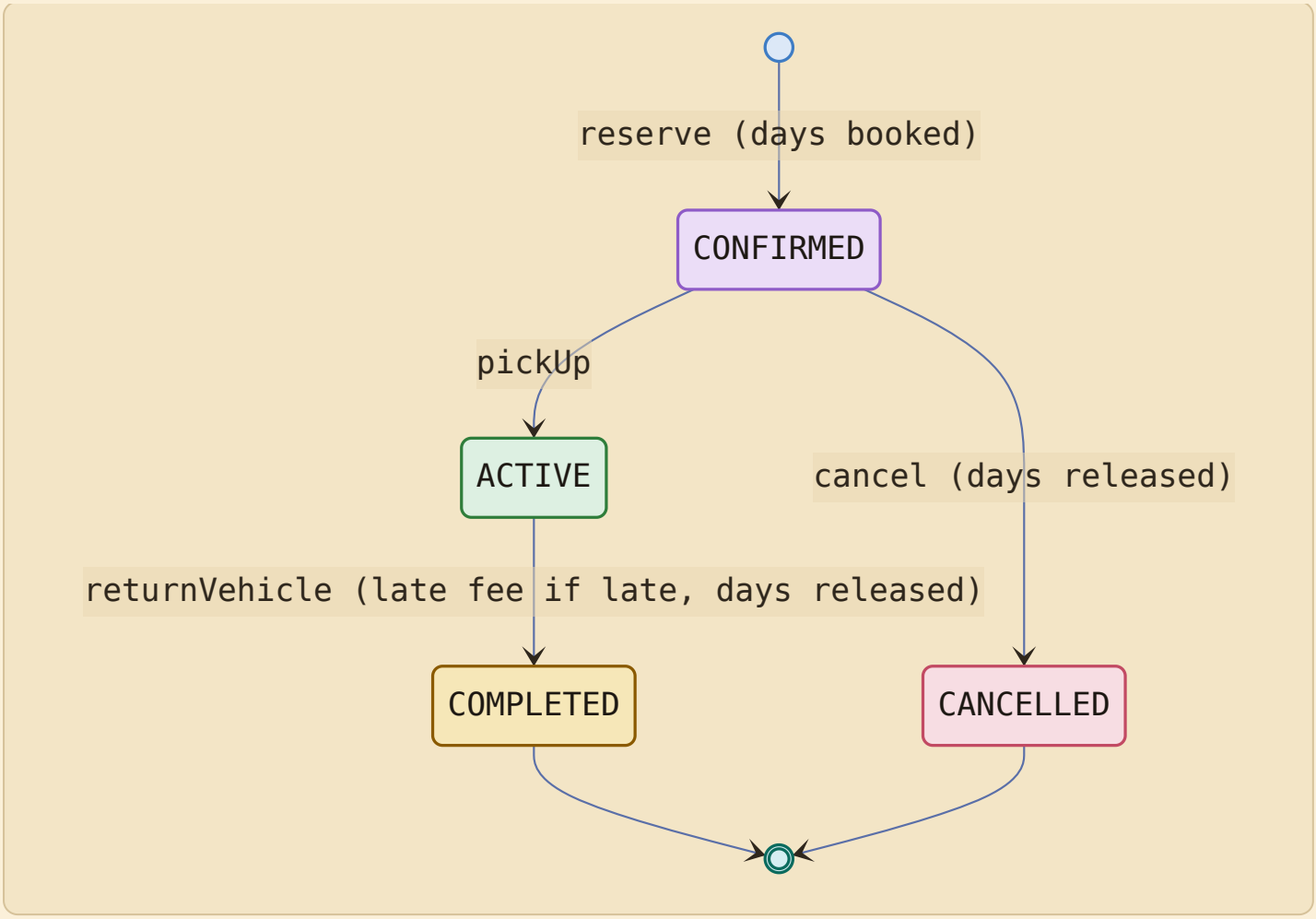


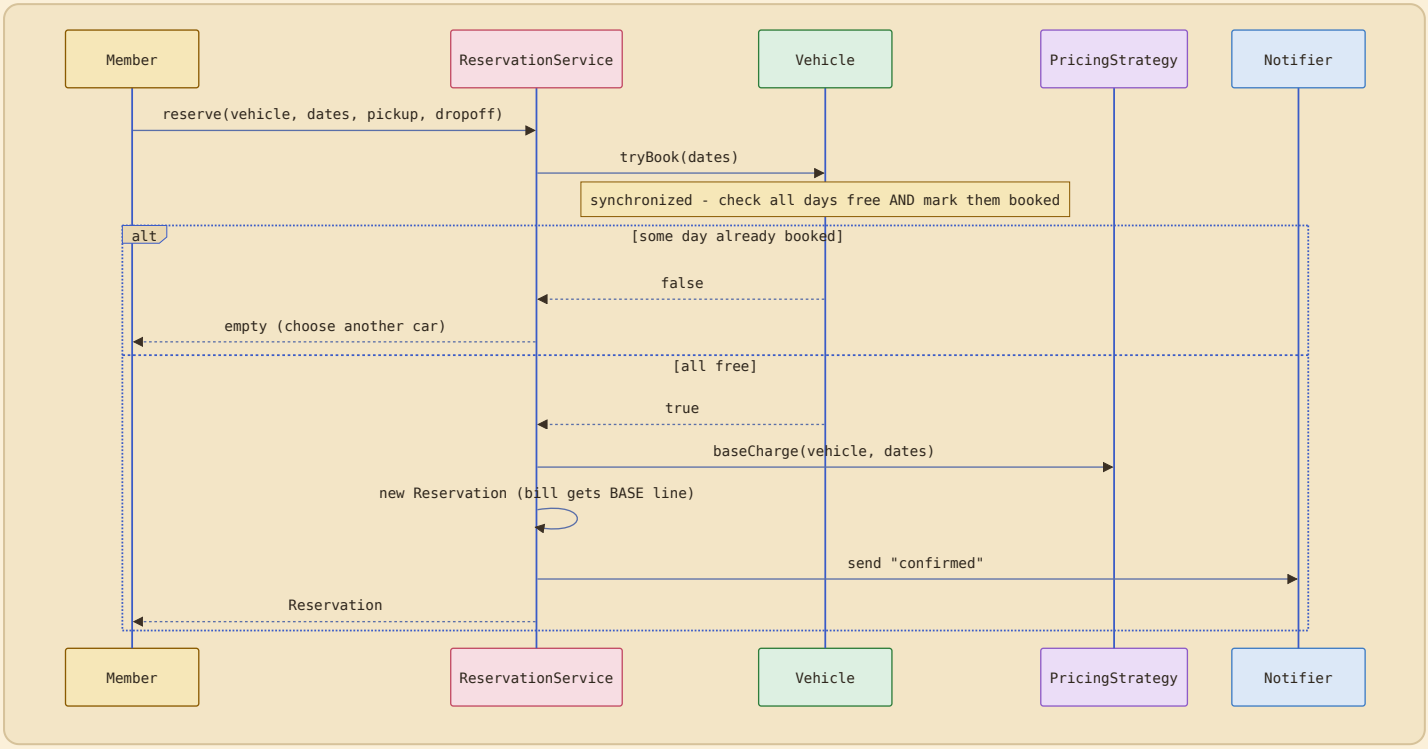
Figure — Activity diagram: returning a vehicle, with the late-fee branch (course diagram)

As the figure above shows, the late-fee decision is a branch on the return path — in our code it is one line inside `Reservation.returnVehicle`.

Reservation lifecycle (a **state machine** — a fixed set of states and the only moves allowed between them):



Reserve flow, including the race we must win (a **sequence diagram**: each column is an object, and arrows are calls in time order from top to bottom):



Design decisions & patterns used

Decision	Why	Alternative rejected
Availability comes from the days each vehicle has already sold, not from a status field	Members ask about future dates ("can I have it from the 10th to the 13th?"). A car that is out on loan right now may be back by then, and a car sitting on the lot today may already be promised to someone else next week.	A `RESERVED` status: cannot answer future-date questions, and a car can only hold one reservation.
Keep `VehicleStatus` too	Staff still need "where is the car right now" (on the lot, loaned, in service, lost).	Dropping status: loses the physical-location answer.
`Vehicle.tryBook` does check **and** book in one `synchronized` method (Java lets only one thread at a time run it on the same object)	Closes the check-then-act race (see below).	Separate `isFree()` then `book()` calls: two members can both see "free".
Template Method (a base class fixes the steps of an algorithm; subclasses fill in some steps) on `ReservationAddOn`	Billing is written once in the `final` (cannot be overridden) method `toBillItem`; subclasses only say what they are (`kind`) and may override the pricing hook (a step meant to be replaced) (`Service` is a flat fee). A fourth add-on is one new subclass.	A type enum with `switch` in the reservation: every new add-on edits the reservation.
Strategy (one interface, interchangeable implementations of a rule) for `PricingStrategy` and `LateFeePolicy`	Seasonal or length-based pricing changes one class, not the reservation.	Hard-coded rate constants.
Decorator (an object that wraps another of the same type and adds behaviour) `WeeklyDiscountPricing` wraps any `PricingStrategy`	Stacks a discount on top of whatever base rule is in place.	A subclass per combination (daily+weekly, seasonal+weekly …).
`Bill` is the only place charges live	Base, add-ons and late fees are lines on one bill; the total can never be computed in two places and disagree.	Each add-on keeping its own running total.
`ReservationService` separate from `Reservation`	Creating and indexing reservations is a different job from holding one reservation's rules (Single Responsibility: one class, one reason to change).	Static factory (a `static` method that builds objects) on `Reservation`.
Actors call the rules, they never compute them	The course's stage 6: a member calls the reservation service (search, reserve, cancel); a receptionist calls the reservation (hand over, take back, take payment, send reminders). An actor that starts working out fees is doing the reservation's job. In our code the actors are plain data (`Member`) and every rule sits in `Reservation`/`ReservationService`.	`Member.reserve()` / `Receptionist.computeLateFee()` methods.
`Bill` methods are `synchronized` and `settle` is idempotent (calling it twice has the same effect as once)	A late-fee line can never be added while the bill is being paid, and a double-clicked "pay" charges once.	Unsynchronized list + a second charge on retry.
The course makes `CarRentalSystem` a **Singleton** (a class allowed	A singleton is easy to reach but hard to replace in tests. Constructing one and passing it (dependency injection: an object receives	`static getInstance()`.

Decision	Why	Alternative rejected
only one global instance); we pass it in instead	what it needs from outside instead of creating or looking it up itself) gives the same "one per app" without a global.	

Java implementation

Package `lld.ch16.carrental`.

```java
// File: code/lld/ch16/carrental/Types.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.stream.Stream;

enum VehicleType { CAR, TRUCK, SUV, VAN, MOTORCYCLE }

/** Where the vehicle physically is RIGHT NOW. Future availability is answered by booked days, not by this. */
enum VehicleStatus { AVAILABLE, LOANED, BEING_SERVICED, LOST }

enum ReservationStatus { CONFIRMED, ACTIVE, COMPLETED, CANCELLED }

enum BillItemType { BASE_CHARGE, ADD_ON, LATE_FEE }

enum VehicleLogType { RESERVED, PICKED_UP, RETURNED, CANCELLED, FUELING, CLEANING, DAMAGE, REPAIR }

/** Half-open range [start, end): 10 Oct to 13 Oct is 3 rental days and the car is due back on 13 Oct. */
record DateRange(LocalDate start, LocalDate end) {
    DateRange {
        if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start");
    }
    long days() { return ChronoUnit.DAYS.between(start, end); }
    Stream<LocalDate> dates() { return start.datesUntil(end); }
    @Override public String toString() { return start + ".." + end; }
}

record Member(String id, String name, String email) {}

record Location(String code, String name) {}

record VehicleLogEntry(LocalDate date, VehicleLogType type, String note) {}

record AdditionalDriver(String name, String licenseNumber) {}
```

```java
// File: code/lld/ch16/carrental/Vehicle.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/** One physical vehicle. It owns its booked days and its log. */
final class Vehicle {
    private final String barcode;
    private final String licensePlate;
    private final String make;
    private final String model;
    private final VehicleType type;
    private final int passengerCapacity;
    private final String parkingStall;
    private VehicleStatus status = VehicleStatus.AVAILABLE;
    private int mileage;
    private final Set<LocalDate> bookedDays = new HashSet<>();
    private final List<VehicleLogEntry> log = new ArrayList<>();

    Vehicle(String barcode, String licensePlate, String make, String model,
            VehicleType type, int passengerCapacity, String parkingStall, int mileage) {
        this.barcode = barcode;
        this.licensePlate = licensePlate;
        this.make = make;
        this.model = model;
        this.type = type;
        this.passengerCapacity = passengerCapacity;
        this.parkingStall = parkingStall;
        this.mileage = mileage;
    }

    String barcode() { return barcode; }
    String model() { return model; }
    VehicleType type() { return type; }
    String parkingStall() { return parkingStall; }
    synchronized VehicleStatus status() { return status; }
    synchronized int mileage() { return mileage; }
    synchronized void setStatus(VehicleStatus s) { status = s; }
    synchronized void setMileage(int m) {
        if (m < mileage) throw new IllegalArgumentException("odometer cannot go backwards");
        mileage = m;
    }

    synchronized boolean isFreeFor(DateRange r) {
        return r.dates().noneMatch(bookedDays::contains);
    }

    /** Check AND book inside one lock, so two members can never both win the same days. */
    synchronized boolean tryBook(DateRange r) {
        if (!isFreeFor(r)) return false;
        r.dates().forEach(bookedDays::add);
        return true;
    }

    synchronized void release(DateRange r) { r.dates().forEach(bookedDays::remove); }

    synchronized void addLog(LocalDate date, VehicleLogType type, String note) {
        log.add(new VehicleLogEntry(date, type, note));
    }

    synchronized List<VehicleLogEntry> history() { return List.copyOf(log); }

    String describe() {
        return make + " " + model + " (" + type + ", " + passengerCapacity + " seats, plate " + licensePlate +
")";
```

```
    }
}
```

```java
// File: code/lld/ch16/carrental/Billing.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;

record BillItem(String description, BillItemType type, BigDecimal amount) {}

/**
 * The ONLY place that knows what a rental costs: every charge becomes a line here.
 * Every method is synchronized on the bill, so a line can never be added while it is being settled.
 */
final class Bill {
    private final List<BillItem> items = new ArrayList<>();
    private boolean paid;

    synchronized void add(BillItem item) {
        if (paid) throw new IllegalStateException("bill already settled");
        items.add(item);
    }
    synchronized List<BillItem> items() { return List.copyOf(items); }
    synchronized BigDecimal total() { return items.stream().map(BillItem::amount).reduce(BigDecimal.ZERO,
BigDecimal::add); }
    synchronized boolean isPaid() { return paid; }

    /** Idempotent: settling a bill that is already paid does not charge the member a second time. */
    synchronized boolean settle(PaymentMethod method) {
        if (paid) return true;
        paid = method.charge(total());
        return paid;
    }
}

/** Strategy for how money is taken. */
interface PaymentMethod { boolean charge(BigDecimal amount); }

record CardPayment(String cardToken) implements PaymentMethod {
    public boolean charge(BigDecimal amount) {
        System.out.println("    charged " + amount + " to card token " + cardToken);
        return true;
    }
}

record CashPayment(BigDecimal tendered) implements PaymentMethod {
    public boolean charge(BigDecimal amount) { return tendered.compareTo(amount) >= 0; }
}

/**
 * Template Method: toBillItem() is fixed; subclasses only say what kind of add-on they are
 * and may override how the price is computed (the hook).
 */
abstract class ReservationAddOn {
    private final String name;
    protected final BigDecimal price;

    protected ReservationAddOn(String name, BigDecimal price) {
        this.name = name;
        this.price = price;
    }

    final BillItem toBillItem(long rentalDays) {
        return new BillItem(kind() + ": " + name, BillItemType.ADD_ON, priceFor(rentalDays));
    }

    protected abstract String kind();

    /** Hook: by default an add-on is charged per rental day. */
    protected BigDecimal priceFor(long rentalDays) { return price.multiply(BigDecimal.valueOf(rentalDays)); }
}
```

```
final class RentalInsurance extends ReservationAddOn {
    RentalInsurance(String name, BigDecimal perDay) { super(name, perDay); }
    protected String kind() { return "Insurance"; }
}

final class Equipment extends ReservationAddOn {
    Equipment(String name, BigDecimal perDay) { super(name, perDay); }
    protected String kind() { return "Equipment"; }
}

/** A service such as roadside assistance or wifi is a flat fee per rental. */
final class Service extends ReservationAddOn {
    Service(String name, BigDecimal flatFee) { super(name, flatFee); }
    protected String kind() { return "Service"; }
    @Override protected BigDecimal priceFor(long rentalDays) { return price; }
}
```

```
// File: code/lld/ch16/carrental/Pricing.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.EnumMap;
import java.util.Map;

/** Strategy: how the base charge for a rental is computed. */
interface PricingStrategy { BigDecimal baseCharge(Vehicle v, DateRange period); }

final class DailyRatePricing implements PricingStrategy {
    private final Map<VehicleType, BigDecimal> dailyRate;
    DailyRatePricing(Map<VehicleType, BigDecimal> dailyRate) { this.dailyRate = new EnumMap<>(dailyRate); }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        return dailyRate.get(v.type()).multiply(BigDecimal.valueOf(p.days()));
    }
}

/** Decorator over any pricing: 10% off rentals of 7 days or more. */
final class WeeklyDiscountPricing implements PricingStrategy {
    private final PricingStrategy inner;
    WeeklyDiscountPricing(PricingStrategy inner) { this.inner = inner; }
    public BigDecimal baseCharge(Vehicle v, DateRange p) {
        BigDecimal base = inner.baseCharge(v, p);
        return p.days() >= 7 ? base.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_UP) : base;
    }
}

/** Strategy: what a late return costs. */
interface LateFeePolicy { BigDecimal fee(Vehicle v, long daysLate); }

record FlatLateFee(BigDecimal perDay) implements LateFeePolicy {
    public BigDecimal fee(Vehicle v, long daysLate) { return perDay.multiply(BigDecimal.valueOf(daysLate)); }
}

/** Observer-style port: the service announces events, the notifier decides the channel. */
interface Notifier { void send(Member to, String message); }

final class ConsoleNotifier implements Notifier {
    public void send(Member to, String message) { System.out.println("    [notify " + to.email() + "] " +
message); }
}
```

```java
// File: code/lld/ch16/carrental/Reservation.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

/** One rental. The rules (add-ons, late fee, freeing days) live here, not in the actors. */
final class Reservation {
    private final String id;
    private final Member member;
    private final Vehicle vehicle;
    private final DateRange period;
    private final Location pickupLocation;
    private final Location returnLocation;
    private final Bill bill = new Bill();
    private final List<ReservationAddOn> addOns = new ArrayList<>();
    private final List<AdditionalDriver> drivers = new ArrayList<>();
    private ReservationStatus status = ReservationStatus.CONFIRMED;

    Reservation(String id, Member member, Vehicle vehicle, DateRange period,
                Location pickup, Location dropoff, BigDecimal baseCharge) {
        this.id = id;
        this.member = member;
        this.vehicle = vehicle;
        this.period = period;
        this.pickupLocation = pickup;
        this.returnLocation = dropoff;
        bill.add(new BillItem("Base: " + vehicle.model() + " x " + period.days() + " days",
                BillItemType.BASE_CHARGE, baseCharge));
    }

    String id() { return id; }
    Member member() { return member; }
    Vehicle vehicle() { return vehicle; }
    DateRange period() { return period; }
    Location pickupLocation() { return pickupLocation; }
    Location returnLocation() { return returnLocation; }
    Bill bill() { return bill; }
    synchronized List<AdditionalDriver> drivers() { return List.copyOf(drivers); }
    synchronized ReservationStatus status() { return status; }

    private void require(ReservationStatus expected, String action) {
        if (status != expected)
            throw new IllegalStateException("cannot " + action + " reservation " + id + " in state " + status);
    }

    synchronized void addAddOn(ReservationAddOn addOn) {
        require(ReservationStatus.CONFIRMED, "add an add-on to");
        addOns.add(addOn);
        bill.add(addOn.toBillItem(period.days()));
    }

    synchronized void addDriver(AdditionalDriver d) {
        require(ReservationStatus.CONFIRMED, "add a driver to");
        drivers.add(d);
    }

    synchronized void pickUp(LocalDate on) {
        require(ReservationStatus.CONFIRMED, "pick up");
        status = ReservationStatus.ACTIVE;
        vehicle.setStatus(VehicleStatus.LOANED);
        vehicle.addLog(on, VehicleLogType.PICKED_UP, id + " by " + member.name() + " at " +
pickupLocation.code());
    }

    /** Returns the number of late days charged. The due date is period.end(). */
    synchronized long returnVehicle(LocalDate on, int odometer, LateFeePolicy lateFee) {
```

```
        require(ReservationStatus.ACTIVE, "return");
        long daysLate = Math.max(0, ChronoUnit.DAYS.between(period.end(), on));
        if (daysLate > 0) {
            bill.add(new BillItem("Late fee: " + daysLate + " day(s)", BillItemType.LATE_FEE,
                    lateFee.fee(vehicle, daysLate)));
        }
        vehicle.setMileage(odometer);
        vehicle.setStatus(VehicleStatus.AVAILABLE);
        vehicle.release(period);
        vehicle.addLog(on, VehicleLogType.RETURNED, id + " at " + returnLocation.code() + ", odometer " +
odometer);
        status = ReservationStatus.COMPLETED;
        return daysLate;
    }

    synchronized void cancel(LocalDate on) {
        require(ReservationStatus.CONFIRMED, "cancel");
        status = ReservationStatus.CANCELLED;
        vehicle.release(period);   // a cancelled rental must give its days back
        vehicle.addLog(on, VehicleLogType.CANCELLED, id);
    }
}
```

```java
// File: code/lld/ch16/carrental/RentalSystem.java
package lld.ch16.carrental;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

/** Per-branch indexes over vehicles. All three are filled by add(), so they cannot drift apart. */
final class VehicleInventory {
    private final Map<String, Vehicle> byBarcode = new ConcurrentHashMap<>();
    private final Map<VehicleType, List<Vehicle>> byType = new ConcurrentHashMap<>();
    private final Map<String, List<Vehicle>> byModel = new ConcurrentHashMap<>();

    void add(Vehicle v) {
        if (byBarcode.putIfAbsent(v.barcode(), v) != null)
            throw new IllegalArgumentException("duplicate barcode " + v.barcode());
        byType.computeIfAbsent(v.type(), t -> new CopyOnWriteArrayList<>()).add(v);
        byModel.computeIfAbsent(v.model(), m -> new CopyOnWriteArrayList<>()).add(v);
    }

    Optional<Vehicle> scan(String barcode) { return Optional.ofNullable(byBarcode.get(barcode)); }

    List<Vehicle> search(VehicleType type, DateRange period) {
        return byType.getOrDefault(type, List.of()).stream()
                .filter(v -> v.status() != VehicleStatus.LOST && v.isFreeFor(period)).toList();
    }

    List<Vehicle> searchByModel(String model, DateRange period) {
        return byModel.getOrDefault(model, List.of()).stream().filter(v -> v.isFreeFor(period)).toList();
    }
}

record CarRentalLocation(Location location, VehicleInventory inventory) {}

/** Creates and tracks reservations. Answers "what did this member rent" and "who took this car". */
final class ReservationService {
    private final PricingStrategy pricing;
    private final LateFeePolicy lateFee;
    private final Notifier notifier;
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger(1000);

    ReservationService(PricingStrategy pricing, LateFeePolicy lateFee, Notifier notifier) {
        this.pricing = pricing;
        this.lateFee = lateFee;
        this.notifier = notifier;
    }

    Optional<Reservation> reserve(Member m, Vehicle v, DateRange period, Location pickup, Location dropoff,
LocalDate today) {
        if (!v.tryBook(period)) return Optional.empty();
        var r = new Reservation("R" + seq.incrementAndGet(), m, v, period, pickup, dropoff, pricing.baseCharge(v,
period));
        reservations.put(r.id(), r);
        v.addLog(today, VehicleLogType.RESERVED, r.id() + " for " + period);
        notifier.send(m, "Reservation " + r.id() + " confirmed: " + v.model() + " " + period);
        return Optional.of(r);
    }

    Reservation get(String id) {
        var r = reservations.get(id);
        if (r == null) throw new IllegalArgumentException("no reservation " + id);
        return r;
    }
```

```
    void cancel(String id, LocalDate on) {
        var r = get(id);
        r.cancel(on);
        notifier.send(r.member(), "Reservation " + id + " cancelled");
    }

    void pickUp(String id, LocalDate on) { get(id).pickUp(on); }

    void returnVehicle(String id, LocalDate on, int odometer) {
        var r = get(id);
        long late = r.returnVehicle(on, odometer, lateFee);
        if (late > 0) notifier.send(r.member(), "Returned " + late + " day(s) late, late fee added to " + id);
    }

    List<Reservation> byMember(String memberId) {
        return reservations.values().stream().filter(r -> r.member().id().equals(memberId)).toList();
    }

    List<Reservation> byVehicle(String barcode) {
        return reservations.values().stream().filter(r -> r.vehicle().barcode().equals(barcode)).toList();
    }

    /** Requirement: remind before pick-up, before the due date, and when overdue. Run once a day by a scheduler.
*/
    void sendReminders(LocalDate today) {
        for (var r : reservations.values()) {
            var status = r.status();
            if (status == ReservationStatus.CONFIRMED && r.period().start().equals(today.plusDays(1)))
                notifier.send(r.member(), "Pick-up tomorrow for " + r.id() + " at " + r.pickupLocation().name());
            else if (status == ReservationStatus.ACTIVE && r.period().end().equals(today.plusDays(1)))
                notifier.send(r.member(), "Vehicle for " + r.id() + " is due tomorrow");
            else if (status == ReservationStatus.ACTIVE && r.period().end().isBefore(today))
                notifier.send(r.member(), "OVERDUE: " + r.id() + " was due " + r.period().end());
        }
    }
}

/** Top-level object: owns branches (each owning its inventory) and the reservation service. */
final class CarRentalSystem {
    private final Map<String, CarRentalLocation> locations = new ConcurrentHashMap<>();
    private final ReservationService reservations;

    CarRentalSystem(ReservationService reservations) { this.reservations = reservations; }

    CarRentalLocation addLocation(Location l) {
        return locations.computeIfAbsent(l.code(), c -> new CarRentalLocation(l, new VehicleInventory()));
    }
    CarRentalLocation location(String code) { return locations.get(code); }
    ReservationService reservations() { return reservations; }

    List<Vehicle> searchAll(VehicleType type, DateRange period) {
        List<Vehicle> out = new ArrayList<>();
        locations.values().forEach(l -> out.addAll(l.inventory().search(type, period)));
        return out;
    }
}
```

```java
// File: code/lld/ch16/carrental/Main.java
package lld.ch16.carrental;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    static BigDecimal usd(String s) { return new BigDecimal(s); }

    public static void main(String[] args) throws Exception {
        var pricing = new WeeklyDiscountPricing(new DailyRatePricing(Map.of(
                VehicleType.CAR, usd("50.00"), VehicleType.SUV, usd("80.00"), VehicleType.TRUCK, usd("100.00"),
                VehicleType.VAN, usd("90.00"), VehicleType.MOTORCYCLE, usd("35.00"))));
        var service = new ReservationService(pricing, new FlatLateFee(usd("40.00")), new ConsoleNotifier());
        var system = new CarRentalSystem(service);

        var sfo = new Location("SFO", "SFO Airport");
        var dtn = new Location("DTN", "Downtown");
        var sfoInv = system.addLocation(sfo).inventory();
        sfoInv.add(new Vehicle("VH-001", "7ABC123", "Toyota", "RAV4", VehicleType.SUV, 5, "A-12", 20_100));
        sfoInv.add(new Vehicle("VH-002", "7ABC124", "Honda", "CR-V", VehicleType.SUV, 5, "A-13", 15_400));
        sfoInv.add(new Vehicle("VH-003", "8XYZ001", "Honda", "Civic", VehicleType.CAR, 5, "B-02", 9_800));
        system.addLocation(dtn).inventory().add(new Vehicle("VH-010", "9VAN555", "Ford", "Transit",
VehicleType.VAN, 12, "C-01", 30_000));

        var alice = new Member("M1", "Alice", "alice@example.com");
        var today = LocalDate.of(2026, 10, 1);
        var trip = new DateRange(LocalDate.of(2026, 10, 10), LocalDate.of(2026, 10, 13));

        System.out.println("1) Search SUVs at SFO for " + trip);
        sfoInv.search(VehicleType.SUV, trip).forEach(v -> System.out.println("   " + v.barcode() + " " +
v.describe() + " stall " + v.parkingStall()));

        System.out.println("2) Alice reserves VH-001 (pick up SFO, return Downtown) and adds extras");
        var rav4 = sfoInv.scan("VH-001").orElseThrow();
        var r1 = service.reserve(alice, rav4, trip, sfo, dtn, today).orElseThrow();
        r1.addAddOn(new RentalInsurance("Collision damage waiver", usd("15.00")));
        r1.addAddOn(new Equipment("Child seat", usd("8.00")));
        r1.addAddOn(new Service("Roadside assistance", usd("25.00")));
        r1.addDriver(new AdditionalDriver("Sam", "D1234567"));

        System.out.println("3) 8 members race for VH-002 on the same dates");
        var crv = sfoInv.scan("VH-002").orElseThrow();
        var wins = new AtomicInteger();
        var winnerId = new String[1];
        var start = new CountDownLatch(1);
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int i = 0; i < 8; i++) {
                var m = new Member("M" + (10 + i), "Racer" + i, "racer" + i + "@example.com");
                pool.submit(() -> {
                    start.await();
                    service.reserve(m, crv, new DateRange(LocalDate.of(2026, 10, 11), LocalDate.of(2026, 10, 14)),
sfo, sfo, today)
                            .ifPresent(r -> { wins.incrementAndGet(); synchronized (winnerId) { winnerId[0] =
r.id(); } });
                    return null;
                });
            }
            start.countDown();
        }
        System.out.println("   successful reservations for VH-002: " + wins.get());
        System.out.println("   SUVs free for " + trip + " now: " + sfoInv.search(VehicleType.SUV, trip).size());

        System.out.println("4) Daily reminder job on 2026-10-09");
        service.sendReminders(LocalDate.of(2026, 10, 9));
```

```
        System.out.println("5) Pick up on 10 Oct, return at Downtown on 15 Oct (due 13 Oct)");
        service.pickUp(r1.id(), LocalDate.of(2026, 10, 10));
        System.out.println("   VH-001 status while out: " + rav4.status());
        service.sendReminders(LocalDate.of(2026, 10, 14));
        service.returnVehicle(r1.id(), LocalDate.of(2026, 10, 15), 20_640);
        rav4.addLog(LocalDate.of(2026, 10, 15), VehicleLogType.CLEANING, "interior clean by worker W7");

        System.out.println("6) Alice's bill");
        r1.bill().items().forEach(i -> System.out.printf("   %-40s %-10s %8s%n", i.description(), i.type(),
i.amount()));
        System.out.println("   TOTAL " + r1.bill().total());
        r1.bill().settle(new CardPayment("tok_alice_visa"));
        System.out.println("   paid: " + r1.bill().isPaid() + ", settle again (double click) -> "
                + r1.bill().settle(new CardPayment("tok_alice_visa")) + ", no second charge");

        System.out.println("7) Who rented VH-001? What has Alice rented?");
        service.byVehicle("VH-001").forEach(r -> System.out.println("   " + r.id() + " -> " + r.member().name() +
" " + r.period() + " " + r.status()));
        service.byMember("M1").forEach(r -> System.out.println("   Alice: " + r.id() + " " +
r.vehicle().model()));

        System.out.println("8) Vehicle log for VH-001");
        rav4.history().forEach(e -> System.out.println("   " + e.date() + " " + e.type() + " " + e.note()));

        System.out.println("9) The VH-002 winner cancels; the days come back");
        service.cancel(winnerId[0], LocalDate.of(2026, 10, 2));
        System.out.println("   SUVs free for " + trip + ": " + sfoInv.search(VehicleType.SUV, trip).size());

        System.out.println("10) A 7-day Civic rental gets the weekly discount");
        var week = new DateRange(LocalDate.of(2026, 11, 1), LocalDate.of(2026, 11, 8));
        var r3 = service.reserve(alice, sfoInv.scan("VH-003").orElseThrow(), week, sfo, sfo, today).orElseThrow();
        System.out.println("   base charge " + r3.bill().total() + " (7 x 50 = 350, minus 10%)");

        System.out.println("11) Rules are enforced");
        try { r1.addAddOn(new Equipment("Ski rack", usd("10.00"))); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        try { new DateRange(LocalDate.of(2026, 10, 5), LocalDate.of(2026, 10, 5)); }
        catch (IllegalArgumentException e) { System.out.println("   rejected: " + e.getMessage()); }
    }
}
```

Output:

```
1) Search SUVs at SFO for 2026-10-10..2026-10-13
   VH-001 Toyota RAV4 (SUV, 5 seats, plate 7ABC123) stall A-12
   VH-002 Honda CR-V (SUV, 5 seats, plate 7ABC124) stall A-13
2) Alice reserves VH-001 (pick up SFO, return Downtown) and adds extras
    [notify alice@example.com] Reservation R1001 confirmed: RAV4 2026-10-10..2026-10-13
3) 8 members race for VH-002 on the same dates
    [notify racer7@example.com] Reservation R1002 confirmed: CR-V 2026-10-11..2026-10-14
   successful reservations for VH-002: 1
   SUVs free for 2026-10-10..2026-10-13 now: 0
4) Daily reminder job on 2026-10-09
    [notify alice@example.com] Pick-up tomorrow for R1001 at SFO Airport
5) Pick up on 10 Oct, return at Downtown on 15 Oct (due 13 Oct)
   VH-001 status while out: LOANED
    [notify alice@example.com] OVERDUE: R1001 was due 2026-10-13
    [notify alice@example.com] Returned 2 day(s) late, late fee added to R1001
6) Alice's bill
   Base: RAV4 x 3 days                     BASE_CHARGE   240.00
   Insurance: Collision damage waiver      ADD_ON        45.00
   Equipment: Child seat                   ADD_ON        24.00
   Service: Roadside assistance            ADD_ON        25.00
   Late fee: 2 day(s)                      LATE_FEE      80.00
   TOTAL 414.00
    charged 414.00 to card token tok_alice_visa
   paid: true, settle again (double click) -> true, no second charge
7) Who rented VH-001? What has Alice rented?
   R1001 -> Alice 2026-10-10..2026-10-13 COMPLETED
   Alice: R1001 RAV4
8) Vehicle log for VH-001
   2026-10-01 RESERVED R1001 for 2026-10-10..2026-10-13
   2026-10-10 PICKED_UP R1001 by Alice at SFO
   2026-10-15 RETURNED R1001 at DTN, odometer 20640
   2026-10-15 CLEANING interior clean by worker W7
9) The VH-002 winner cancels; the days come back
    [notify racer7@example.com] Reservation R1002 cancelled
   SUVs free for 2026-10-10..2026-10-13: 2
10) A 7-day Civic rental gets the weekly discount
    [notify alice@example.com] Reservation R1003 confirmed: Civic 2026-11-01..2026-11-08
   base charge 315.00 (7 x 50 = 350, minus 10%)
11) Rules are enforced
   rejected: cannot add an add-on to reservation R1001 in state COMPLETED
   rejected: end must be after start
```

What to notice in the output: step 3 has eight threads racing for one car and exactly one wins (which racer wins varies per run); step 5 shows the overdue reminder and the 2-day late fee (2 × 40.00); step 6 shows that settling an already-paid bill does not charge again; step 9 shows cancellation giving the days back.

Concurrency & edge cases

> **Key term — lock:** A **lock** (also called a mutex) is a token only one thread can hold at a time; any other thread that wants it waits until it is released. In Java, a `synchronized` method uses the object itself as the lock. A *global lock* guards everything with one token (simple, but everyone queues), while a *per-object lock* (one per vehicle here) lets work on different objects run in parallel.

- **Double booking.** The course lists this as an open gap in its own design (checking the days and booking them are two separate steps there). Two members call `reserve` for the same car at the same moment. Because `tryBook` is `synchronized` on the vehicle, the check "are all days free?" and the write "mark them booked" happen as one step, so the second caller sees the first caller's days. Locking per vehicle (not one global lock) lets different cars be booked in parallel. In a database-backed system the same idea becomes a unique constraint on `(vehicle_id, day)` (the database refuses a second row with the same pair) or a `SELECT … FOR UPDATE` (a read that also locks the rows it returns) in one transaction (a group of database changes that succeed or fail together).

- **Search is advisory.** `search` may say a car is free and then `reserve` fails because someone else won. That is correct: the client (the app or web page the member is using) simply shows "no longer available". Never reserve based on a stale (out-of-date) search result without re-checking.
- **Late return collides with the next rental.** If Alice returns two days late and Bob booked the car from the 14th, Bob's car is not on the lot. Real systems keep a buffer (for example a cleaning day) between bookings and, when a car is overdue, reassign the next reservation to another car of the same type.
- **Early return.** We release the whole booked range; the unused days become bookable. Whether the member is refunded is a policy question.
- **Cancelling after pick-up** is rejected by the state machine (`CONFIRMED` only).
- **Add-ons after the rental ends** are rejected (see step 11 of the output).
- **Invalid ranges** (end not after start) are rejected in the `DateRange` constructor.
- **Odometer going backwards** is rejected in `Vehicle.setMileage`.
- **Different return branch.** The reservation records the return branch, but the vehicle stays in the pick-up branch's inventory. The course calls this out as a known gap: moving a car between branches is a *transfer* the design does not model yet (see Extensions).

Extensions & follow-up questions

- **"How do you handle one-way rentals?"** — On return at a different branch, remove the vehicle from the old `VehicleInventory` and add it to the new one inside `returnVehicle`, and log a `TRANSFER` event. Add a one-way fee as another `BillItem`.
- **"Seasonal or dynamic pricing?"** — Add a `SeasonalPricing` strategy (rate per date) or wrap with another decorator (`SurgePricing`). The reservation does not change.
- **"Deposits and card holds?"** — A card *hold* reserves money on the card without taking it yet. A `PaymentMethod.authorize(amount)` at pick-up and `capture(amount)` at return; the difference is released.
- **"Mileage limits, fuel level, damage claims?"** — Record odometer and fuel at pick-up and return; add `BillItem`s of type `FINE` for extra miles, missing fuel or damage (the course's enum has a `FINE` bill item type for exactly this kind of charge).
- **"How do reminders run?"** — A scheduler (for example a `ScheduledExecutorService`, Java's built-in timer for repeating tasks, or a cron job, an operating-system entry that runs a command on a fixed timetable) calls `sendReminders(today)` once a day.
- **"How would you scale this?"** — Partition vehicles by branch (split the data so each branch's vehicles live on their own database or server); the booked-days table becomes `(vehicle_id, day)` with a unique key; searches go to a read replica (a read-only copy of the database kept in sync with the main one) or a search index (a separate store built for fast lookups, such as Elasticsearch); reservations are written with a transaction. These scaling ideas are explained in System Design Fundamentals.
- **"How do workers update the log?"** — `Vehicle.addLog(date, CLEANING, note)` as in step 5; restrict who can call it by role in the service layer (the classes, like `ReservationService`, that sit between callers and the domain objects and enforce who may do what).

In the interview

30-second pitch: "Branches own inventories, inventories index vehicles by barcode, type and model. The key decision: availability is a question about *dates*, so each vehicle keeps its booked days, and `tryBook` checks and books them atomically. A reservation owns a bill; every charge — base, insurance, equipment, services, late fee — is a line on it. Add-ons share a template-method base class; pricing and late fees are strategies."

Suggested 40-minute plan:

Minutes	Do
0–5	Clarify vehicle types, branches, one-way, add-ons, late fees.
5–10	Actors and use cases; list the entities.
10–20	Class diagram. Say "availability is dates, not status" early.
20–35	Code `Vehicle.tryBook`, `Reservation` (state machine + bill + late fee), `ReservationService.reserve`.
35–40	Concurrency, late-return collision, one-way transfers.

Likely follow-ups: "Can I book it for next month while it is loaned now?" (yes — dates, not status). "Two people click reserve together?" (atomic `tryBook`). "Add a new add-on?" (one subclass).

Self-check

1. **Q:** Why is a `RESERVED` status not enough to answer "is this car free next week?" **A:** Status describes *now*. A car can be loaned today and free next week, or on the lot today but promised next week. Only per-day bookings answer date questions.
2. **Q:** What pattern does `ReservationAddOn.toBillItem` use and what is the "hook"? **A:** Template Method; `toBillItem` is the fixed algorithm and `priceFor` is the overridable hook (`Service` overrides it to a flat fee).
3. **Q:** Why is `tryBook` one method rather than `isFree` + `book`? **A:** To make check-then-act atomic under the vehicle's lock, so two threads cannot both see "free".
4. **Q:** How is the late fee computed? **A:** Days between the due date (`period.end`) and the actual return date, times the per-day fee from `LateFeePolicy`, added as a `LATE_FEE` bill line.
5. **Q:** Which class answers "who rented VH-001"? **A:** `ReservationService.byVehicle`, from the one list of reservations.

TL;DR

- Availability = booked **days** per vehicle; status = where it is now.
- `tryBook` makes check-and-book atomic per vehicle.
- A reservation owns one bill; every charge is a line. Late fee = days late × rate.
- Template Method for add-ons, Strategy (+ Decorator) for pricing.
- Known gaps to mention: branch transfers, buffer between rentals.

Real-world example. Rental companies such as Hertz and Enterprise let you book a *car class* ("intermediate SUV or similar") rather than a specific plate; the exact vehicle is assigned at the counter. That is the same design one level up: availability is counted per type per day, and a specific `Vehicle` is bound at pick-up. Their counters and return lanes scan the car and compute extra-day and fuel charges on the spot — the "return" activity above.

Design a Restaurant Management System



Figure — A restaurant with branches, tables, waiters and a kitchen (course diagram)

Problem & clarifying questions

A **restaurant management system** runs the day-to-day of a restaurant from one place: which tables are free, table reservations, taking orders, sending them to the kitchen, and producing and settling the bill.

Question	Assumed answer
One restaurant or a chain?	A restaurant with several branches; each branch has its own menu, tables and staff.
Menu structure?	Menu → sections (Starters, Mains …) → items with prices.
How are orders taken?	A waiter creates an order for a table and adds meals **per seat**; each meal has meal items, each pointing at a menu item.
Reservations?	Receptionist searches tables by date/time and party size and reserves one; customers can cancel; the system reminds them before the time.
Walk-ins?	Yes: show tables free right now.
Seating layouts?	A branch can have several seating arrangements (for example "Weekday" and "Banquet").
Payment?	Credit card, cheque or cash.
Kitchen?	Orders go to a chef queue; chefs cook them in order and the waiter is told when food is ready.

Requirements

Functional

1. The restaurant has several branches.
2. Each branch has a menu.
3. A menu has sections; sections have menu items.
4. A waiter creates an order for a table and adds meals for each seat.
5. Each meal has meal items; each meal item corresponds to a menu item.

6. Show which tables are free now for walk-in customers.
7. Support table reservations.
8. The receptionist searches free tables by date and time and reserves one.
9. Customers can cancel a reservation.
10. Send a notification when the reservation time is approaching.
11. Customers pay by credit card, cheque or cash.
12. A branch can have several seating arrangements of tables.
13. (Added) Orders go to a kitchen queue; chefs take them one at a time and the waiter is notified when an order is ready.

Non-functional

- No two parties may be given the same table for overlapping time.
- Prices live in one place (the menu item) so an order and the menu can never disagree.
- Easy to add a new payment type or a new chef-assignment rule.

Use cases

Actors:

- **Receptionist** — adds and edits tables and layouts; creates and cancels reservations; checks guests in.
- **Waiter** — takes and changes orders.
- **Manager** — hires staff and edits the menu.
- **Chef** — views and cooks orders.
- **Cashier** — produces the check (bill) and takes payment.
- **System** — sends reservation reminders and cancellations.

Top use cases: add/modify tables; search tables; place order; update order; create reservation; cancel reservation; check in; make payment.

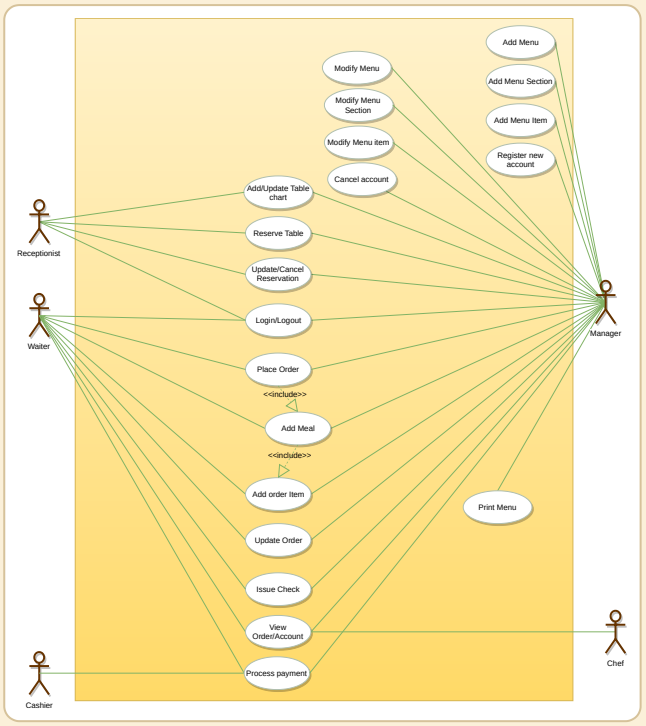


Figure — Use case diagram: receptionist, waiter, manager, chef, cashier and system (course diagram)

In the figure above, each role has a small, separate set of use cases. That is a hint for the code: each role class gets only its own methods.

Core entities

Class	Responsibility
`Restaurant`	Owns its branches. In the course the restaurant also registers its employees: closing (deactivating) the restaurant deactivates every employee account with it.
`Branch`	Owns its menu, kitchen, seating charts and reservation service; employs staff.
`Menu`, `MenuSection`, `MenuItem`	The menu tree. The **price lives only on `MenuItem`**.
`Table`, `TableSeat`	A table with a capacity and numbered seats. Keeps both its **current status** and the **time slots it is promised to**.
`TableChart`	A named seating arrangement; refers to tables (a table can appear in several charts).
`Reservation`	Customer, party size, time, table, status. `updatePeopleCount` refuses a party bigger than the reserved table seats (otherwise the booking would say six while the table holds four).
`ReservationService`	Searches tables (best fit), reserves atomically, cancels, checks in, sends reminders, lists walk-in tables.
`Order`	One per table visit: meals keyed by seat, status with allowed transitions, listeners.
`Meal`, `MealItem`	One meal per seat; a meal item = menu item × quantity.
`Kitchen`, `Chef`	Kitchen hands each order to a chef by a pluggable rule; each chef cooks from a FIFO (first in, first out) queue.
`Employee` and roles (`Receptionist`, `Waiter`, `Manager`, `Chef`, `Cashier`)	Each role's own operations.
`Check`	The bill: computed from the order (subtotal, tax, total), never stored separately.
`Payment` (`CardPayment`, `CashPayment`, `CheckPayment`)	Three ways to pay with three different behaviours.

Class diagram

The course grows the model in seven stages: payments and notifications, the menu, tables, reservations, orders, staff, and finally the branch that ties it together.

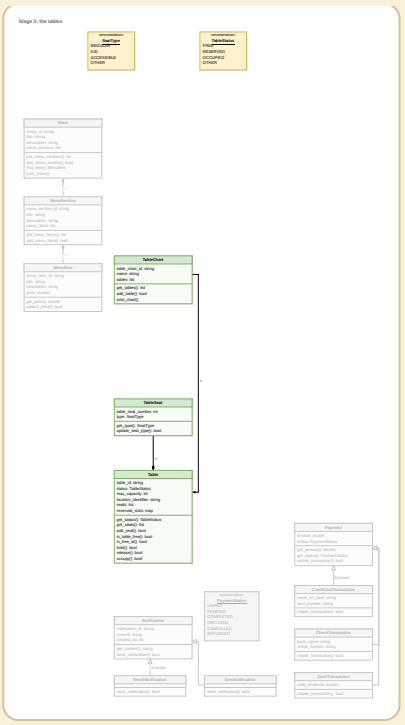


Figure — Table, TableSeat and TableChart: a table owns its seats; a chart only refers to tables (course diagram)

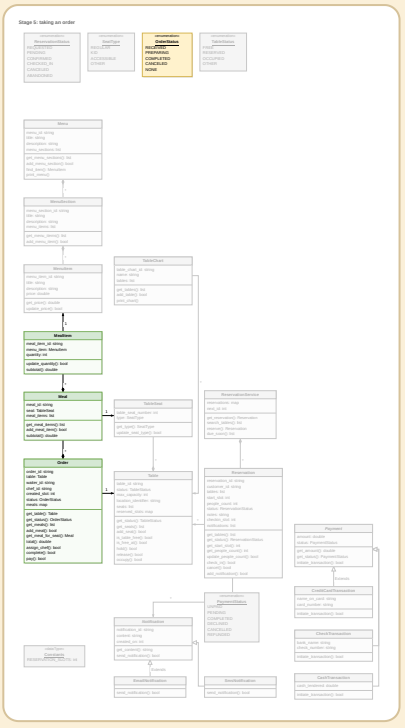


Figure — Order, Meal and MealItem: one order per table, one meal per seat (course diagram)

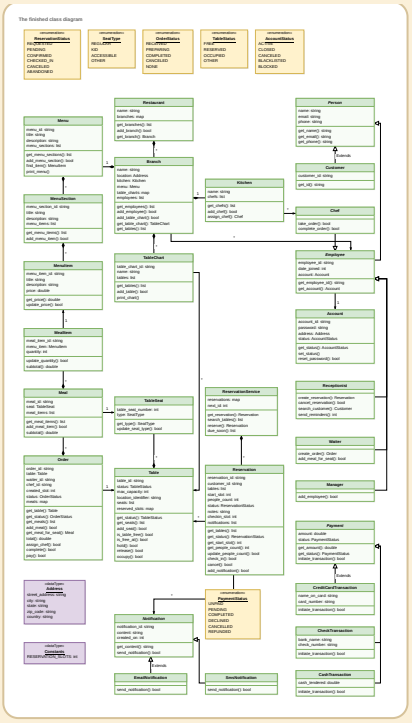
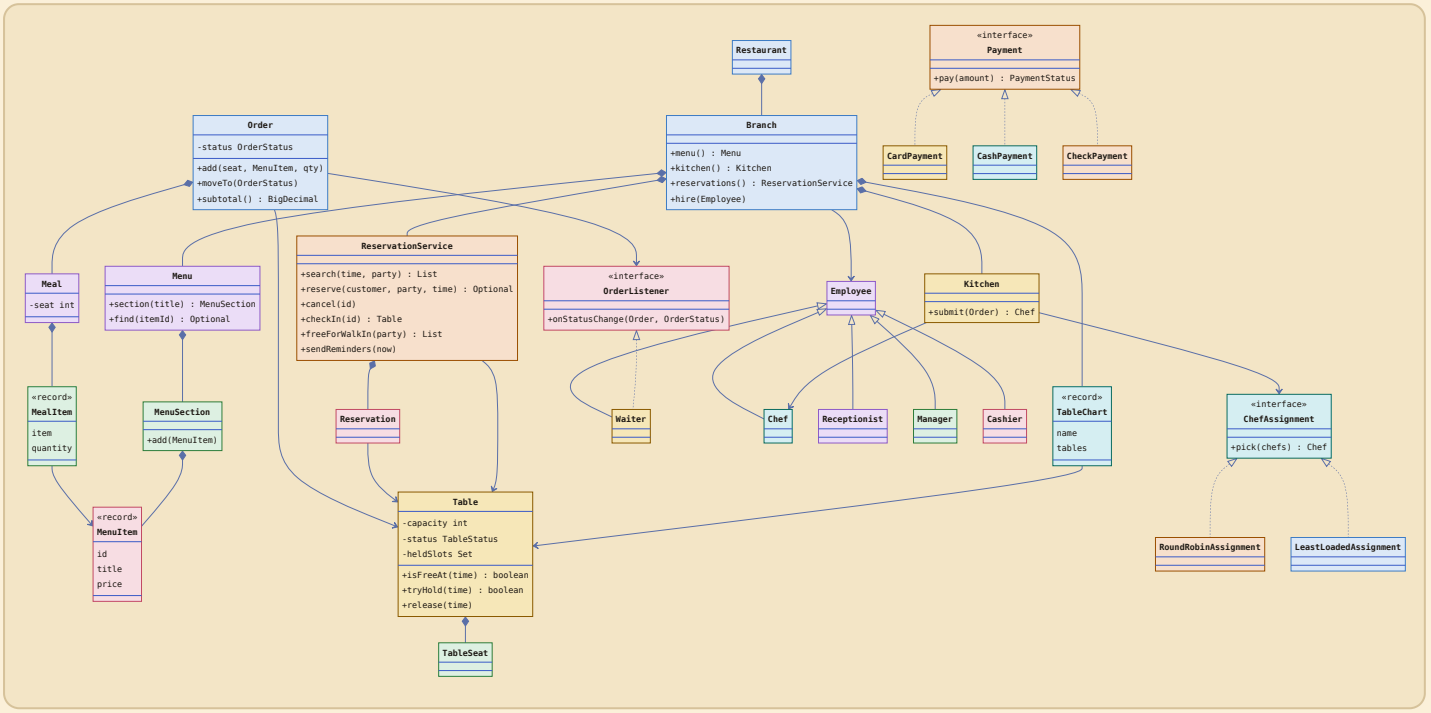


Figure — The finished class diagram of the restaurant management system (course diagram)

In the finished figure above, the branch is the hub: its menu, kitchen and seating charts are composition (they do not exist without the branch), while staff are only associated, because staff can move between branches. Note also what is *not* there: no separate `Bill` class that stores a total. The course's argument is that the order already holds every priced line, so storing the total again would create a second copy that could disagree. Our `Check` follows that rule — it is computed from the order on demand.



Key flows

Place an order. The waiter opens an order for the table, then for each seat adds a meal with its items, and sends the order to the kitchen.

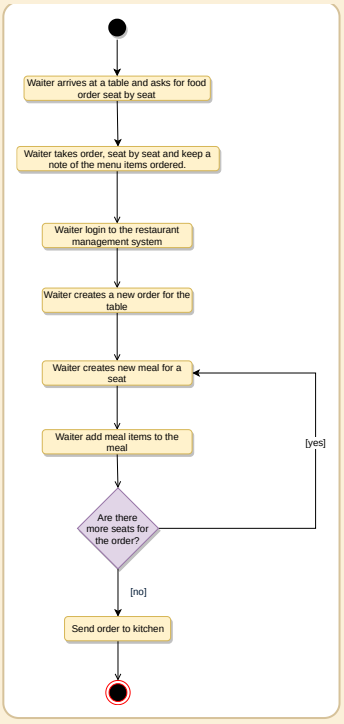


Figure — Activity diagram: placing an order seat by seat and sending it to the kitchen (course diagram)

Make a reservation. The customer calls or arrives; the receptionist searches tables for the time and party size; if one is free it is reserved and a confirmation is sent, otherwise the customer is told none is available.

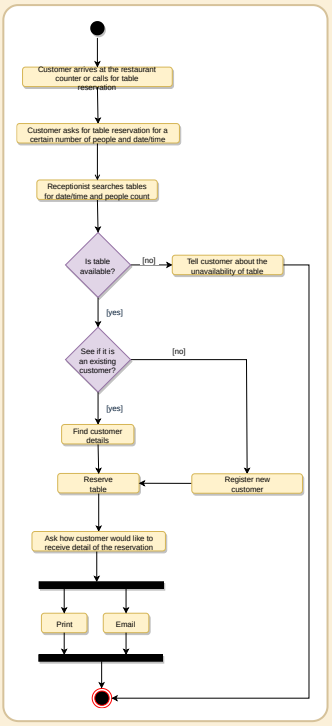


Figure — Activity diagram: making a table reservation (course diagram)

Cancel a reservation. The receptionist looks up the booking; if it is valid, it is cancelled, the table is released and the customer is notified.

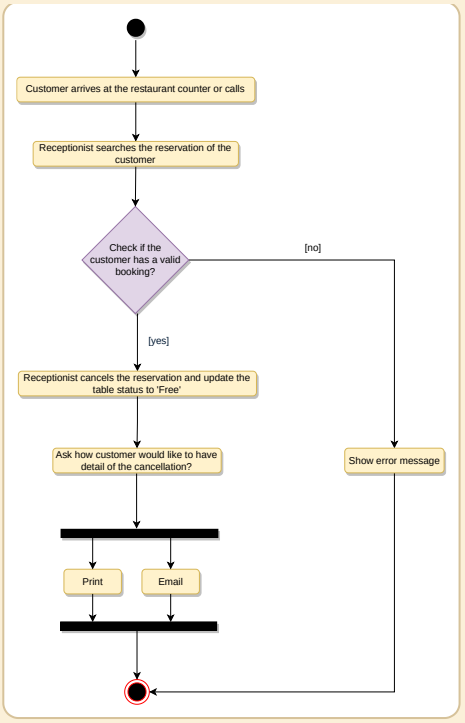
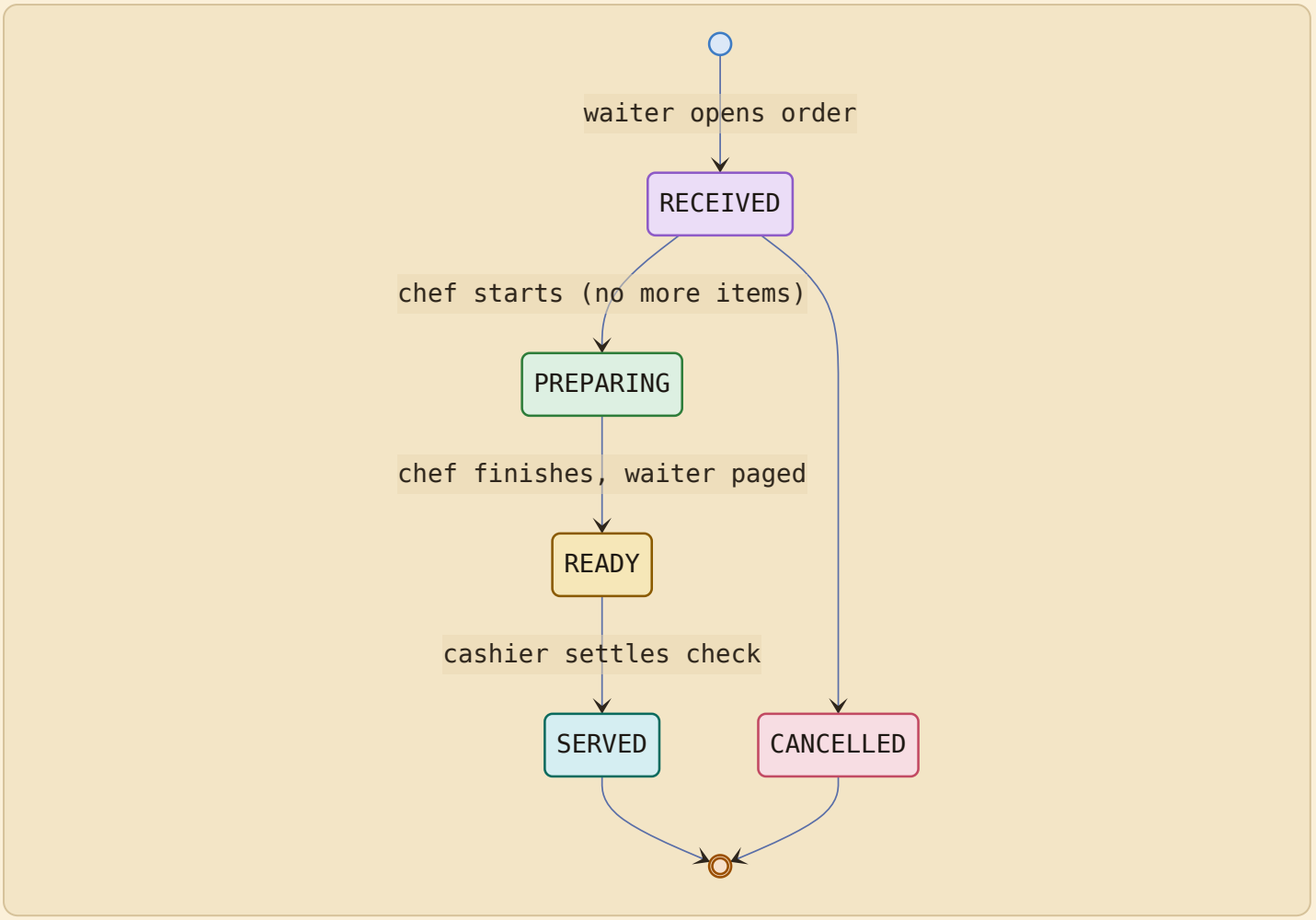
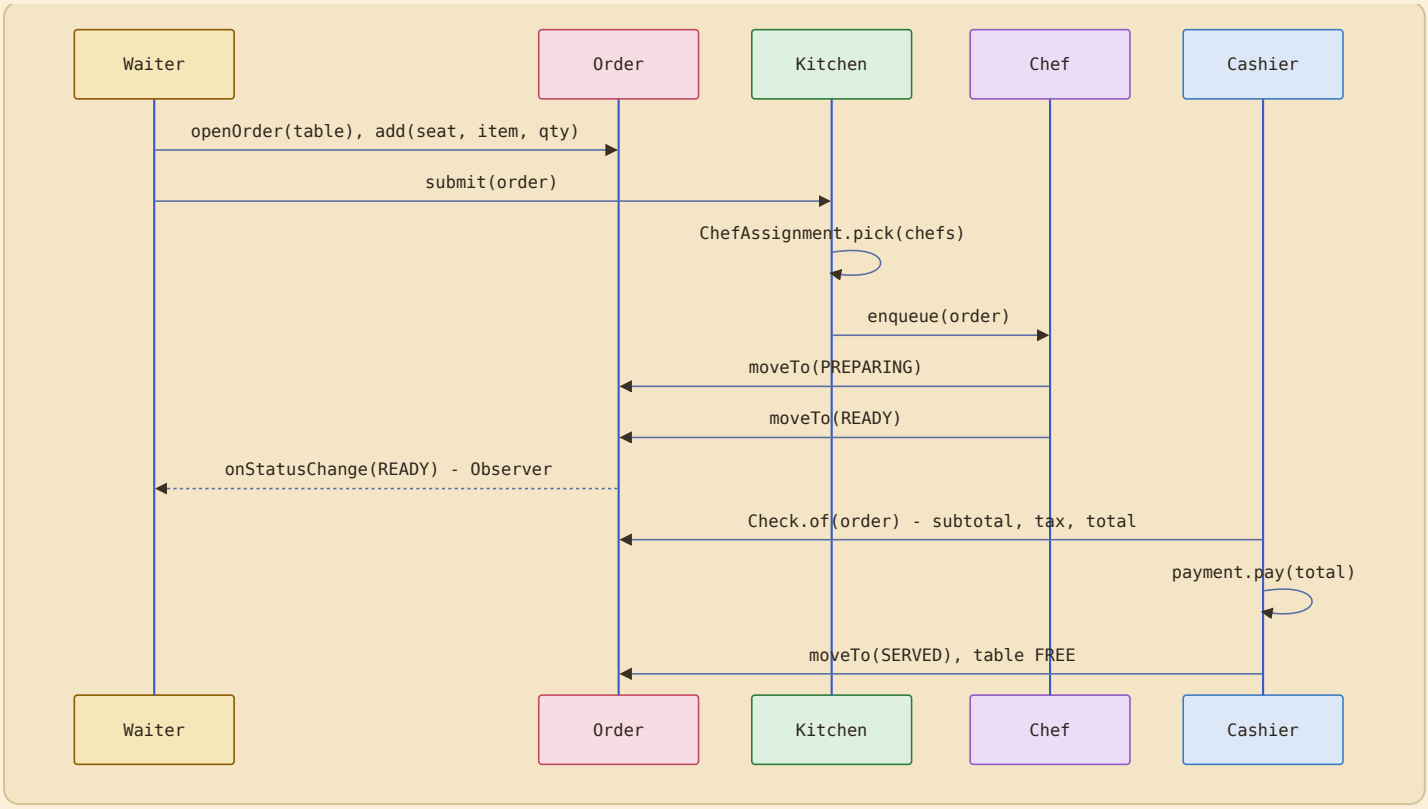


Figure — Activity diagram: cancelling a reservation (course diagram)

Order lifecycle — the transitions `Order.moveTo` allows:



From order to payment:


Waiter
Order
Kitchen
Chef
Cashier
openOrder(table), add(seat, item, qty)
submit(order)
ChefAssignment.pick(chefs)
enqueue(order)
moveTo(PREPARING)
moveTo(READY)
onStatusChange(READY) - Observer
Check.of(order) - subtotal, tax, total
payment.pay(total)
moveTo(SERVED), table FREE
Waiter
Order
Kitchen
Chef
Cashier

Design decisions & patterns used

Decision	Why	Alternative rejected
One meal per seat (`Map<seat, Meal>`)	Requirement 4 asks for it; it tells the kitchen which plate goes to which guest and makes split bills trivial.	A flat list of dishes per order.
Table keeps held time slots as well as status	Status answers "now" (walk-ins); slots answer "is it free at 19:00 on Friday" (reservations). A booking holds two hourly slots.	Status only: cannot take future bookings.
Search lives on `ReservationService`, not on `Table`	One table only knows about itself; "which tables are free at seven" needs all of them. The course notes an older version had a static search on `Table` that could not work.	Static `Table.search`.
Best fit when choosing a table	Smallest table that seats the party keeps large tables free for large parties.	First free table.
Observer (`OrderListener`; listeners register with a subject and are told when it changes)	The kitchen does not need to know who cares about "ready" — the waiter, a display screen, a delivery app can all subscribe.	Chef calling the waiter directly (tight coupling: classes that know too much about each other to change separately).
Strategy (`ChefAssignment`)	The course's kitchen rotates chefs; that is fair but not load-aware (it ignores how busy each chef already is). Swapping to `LeastLoadedAssignment` is one class, not a rewrite — the course's own point about keeping the rule in one place.	Round-robin (taking turns in a fixed rotation) hard-coded in `Kitchen`.
State machine in `Order.moveTo`	Illegal moves (adding food after cooking starts, serving before ready) fail loudly.	Free `setStatus`.
Three `Payment` classes behind a `sealed` interface (only the listed classes may implement it)	Card completes at once, cash may be short (declined) and returns change, a cheque must clear (pending). Different behaviour → different classes (Strategy).	One `Payment` class with a `type` field and `if`s.
Check computed, not stored	Prices live on menu items; the check is derived, so it cannot disagree with the order.	A stored `Bill` total.
Role classes extend `Employee`; `Customer` is not an employee	Only staff have employee ids and accounts; this keeps an account off the customer.	One `Person` class with a role flag.

Java implementation

Package `lld.ch16.restaurant`.

```java
// File: code/lld/ch16/restaurant/Types.java
package lld.ch16.restaurant;

import java.math.BigDecimal;

enum TableStatus { FREE, RESERVED, OCCUPIED }
enum SeatType { REGULAR, KID, ACCESSIBLE }
enum ReservationStatus { CONFIRMED, CHECKED_IN, CANCELLED, NO_SHOW }
enum OrderStatus { RECEIVED, PREPARING, READY, SERVED, CANCELLED }
enum PaymentStatus { COMPLETED, PENDING, DECLINED }

record MenuItem(String id, String title, BigDecimal price) {}
record Customer(String name, String phone) {}
record TableSeat(int number, SeatType type) {}
```

```java
// File: code/lld/ch16/restaurant/Menu.java
package lld.ch16.restaurant;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

/** A branch's menu: Menu -> sections -> items (composition). Prices live ONLY on MenuItem. */
final class Menu {
    private final Map<String, MenuSection> sections = new LinkedHashMap<>();

    synchronized MenuSection section(String title) { return sections.computeIfAbsent(title, MenuSection::new); }

    synchronized Optional<MenuItem> find(String itemId) {
        return sections.values().stream().flatMap(s -> s.items().stream())
                .filter(i -> i.id().equals(itemId)).findFirst();
    }

    synchronized List<MenuSection> sections() { return List.copyOf(sections.values()); }
}

final class MenuSection {
    private final String title;
    private final List<MenuItem> items = new ArrayList<>();
    MenuSection(String title) { this.title = title; }
    String title() { return title; }
    synchronized void add(MenuItem item) { items.add(item); }
    synchronized boolean remove(String itemId) { return items.removeIf(i -> i.id().equals(itemId)); }
    synchronized List<MenuItem> items() { return List.copyOf(items); }
}
```

```java
// File: code/lld/ch16/restaurant/Tables.java
package lld.ch16.restaurant;

import java.time.LocalDateTime;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;

/**
 * status answers "what is this table doing NOW" (for walk-ins).
 * heldSlots answers "is it free at 19:00 on Friday" (for reservations).
 */
final class Table {
    static final int SLOTS_PER_BOOKING = 2;   // a booking holds the table for two hourly slots

    private final String id;
    private final int capacity;
    private final List<TableSeat> seats = new ArrayList<>();
    private final Set<LocalDateTime> heldSlots = new HashSet<>();
    private TableStatus status = TableStatus.FREE;

    Table(String id, int capacity) {
        this.id = id;
        this.capacity = capacity;
        for (int i = 1; i <= capacity; i++) seats.add(new TableSeat(i, SeatType.REGULAR));
    }

    String id() { return id; }
    int capacity() { return capacity; }
    List<TableSeat> seats() { return List.copyOf(seats); }
    synchronized TableStatus status() { return status; }
    synchronized void setStatus(TableStatus s) { status = s; }

    private static List<LocalDateTime> slots(LocalDateTime start) {
        var first = start.truncatedTo(ChronoUnit.HOURS);
        List<LocalDateTime> out = new ArrayList<>();
        for (int i = 0; i < SLOTS_PER_BOOKING; i++) out.add(first.plusHours(i));
        return out;
    }

    synchronized boolean isFreeAt(LocalDateTime start) { return
slots(start).stream().noneMatch(heldSlots::contains); }

    /** Atomic check-then-hold. */
    synchronized boolean tryHold(LocalDateTime start) {
        if (!isFreeAt(start)) return false;
        heldSlots.addAll(slots(start));
        return true;
    }

    synchronized void release(LocalDateTime start) { slots(start).forEach(heldSlots::remove); }
}

/** A named seating arrangement. It REFERS to tables (association); a table can appear in several charts. */
record TableChart(String name, List<Table> tables) {}
```

```java
// File: code/lld/ch16/restaurant/Reservations.java
package lld.ch16.restaurant;

import java.time.LocalDateTime;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.atomic.AtomicInteger;

final class Reservation {
    private final String id;
    private final Customer customer;
    private int partySize;
    private final LocalDateTime time;
    private final Table table;
    private ReservationStatus status = ReservationStatus.CONFIRMED;
    private boolean reminded;

    Reservation(String id, Customer customer, int partySize, LocalDateTime time, Table table) {
        this.id = id; this.customer = customer; this.partySize = partySize; this.time = time; this.table = table;
    }
    String id() { return id; }
    Customer customer() { return customer; }
    synchronized int partySize() { return partySize; }

    /** The party grew or shrank. Refused if the reserved table cannot seat the new number. */
    synchronized void updatePeopleCount(int n) {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        if (n < 1 || n > table.capacity())
            throw new IllegalArgumentException("table " + table.id() + " seats " + table.capacity() + ", not " +
n);
        partySize = n;
    }
    LocalDateTime time() { return time; }
    Table table() { return table; }
    synchronized ReservationStatus status() { return status; }

    synchronized void cancel() {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        status = ReservationStatus.CANCELLED;
        table.release(time);
    }

    synchronized void checkIn() {
        if (status != ReservationStatus.CONFIRMED) throw new IllegalStateException(id + " is " + status);
        status = ReservationStatus.CHECKED_IN;
        table.setStatus(TableStatus.OCCUPIED);
    }

    synchronized boolean markReminded() { boolean first = !reminded; reminded = true; return first; }
}

interface Notifier { void send(Customer to, String message); }

final class SmsNotifier implements Notifier {
    public void send(Customer to, String message) { System.out.println("    [sms " + to.phone() + "] " + message);
}
}

/** Searching belongs here, not on Table: one table only knows about itself. */
final class ReservationService {
    private final List<Table> tables;
    private final Notifier notifier;
    private final Map<String, Reservation> reservations = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger();

    ReservationService(List<Table> tables, Notifier notifier) { this.tables = List.copyOf(tables); this.notifier =
notifier; }
```

```
    /** Best fit first: the smallest table that seats the party, so big tables stay free for big parties. */
    List<Table> search(LocalDateTime time, int partySize) {
        return tables.stream().filter(t -> t.capacity() >= partySize && t.isFreeAt(time))
                .sorted(Comparator.comparingInt(Table::capacity)).toList();
    }

    Optional<Reservation> reserve(Customer c, int partySize, LocalDateTime time) {
        for (Table t : search(time, partySize)) {
            if (t.tryHold(time)) {     // another receptionist may have taken it since search(): try the next one
                var r = new Reservation("RES-" + seq.incrementAndGet(), c, partySize, time, t);
                reservations.put(r.id(), r);
                notifier.send(c, "Table " + t.id() + " booked for " + partySize + " at " + time);
                return Optional.of(r);
            }
        }
        return Optional.empty();
    }

    Reservation get(String id) {
        var r = reservations.get(id);
        if (r == null) throw new IllegalArgumentException("no reservation " + id);
        return r;
    }

    void cancel(String id) {
        var r = get(id);
        r.cancel();
        notifier.send(r.customer(), "Reservation " + id + " cancelled");
    }

    Table checkIn(String id) {
        var r = get(id);
        r.checkIn();
        return r.table();
    }

    List<Table> freeForWalkIn(int partySize) {
        return tables.stream().filter(t -> t.status() == TableStatus.FREE && t.capacity() >= partySize)
                .sorted(Comparator.comparingInt(Table::capacity)).toList();
    }

    /** Run by a scheduler every few minutes: remind reservations starting within the next hour, once. */
    void sendReminders(LocalDateTime now) {
        reservations.values().stream()
                .filter(r -> r.status() == ReservationStatus.CONFIRMED)
                .filter(r -> !r.time().isBefore(now) && r.time().isBefore(now.plusHours(1)))
                .filter(Reservation::markReminded)
                .forEach(r -> notifier.send(r.customer(), "Reminder: your table " + r.table().id() + " at " +
r.time().toLocalTime()));
    }
}
```

```java
// File: code/lld/ch16/restaurant/Orders.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.TreeMap;
import java.util.concurrent.CopyOnWriteArrayList;

record MealItem(MenuItem item, int quantity) {
    MealItem {
        if (quantity <= 0) throw new IllegalArgumentException("quantity must be positive");
    }
    BigDecimal subtotal() { return item.price().multiply(BigDecimal.valueOf(quantity)); }
}

/** One meal per seat: lets the kitchen plate per guest and lets the bill be split. */
final class Meal {
    private final int seat;
    private final List<MealItem> items = new ArrayList<>();
    Meal(int seat) { this.seat = seat; }
    int seat() { return seat; }
    synchronized void add(MealItem i) { items.add(i); }
    synchronized List<MealItem> items() { return List.copyOf(items); }
    synchronized BigDecimal subtotal() { return items.stream().map(MealItem::subtotal).reduce(BigDecimal.ZERO,
BigDecimal::add); }
}

/** Observer: whoever cares about an order's progress (the waiter, a display screen). */
interface OrderListener { void onStatusChange(Order order, OrderStatus newStatus); }

final class Order {
    private final String id;
    private final Table table;
    private final Map<Integer, Meal> meals = new TreeMap<>();
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
    private OrderStatus status = OrderStatus.RECEIVED;

    Order(String id, Table table) { this.id = id; this.table = table; }
    String id() { return id; }
    Table table() { return table; }
    synchronized OrderStatus status() { return status; }
    void subscribe(OrderListener l) { listeners.add(l); }

    /** Items can only be added before the kitchen starts cooking. */
    synchronized void add(int seat, MenuItem item, int qty) {
        if (status != OrderStatus.RECEIVED) throw new IllegalStateException("order " + id + " is already " +
status);
        if (seat < 1 || seat > table.capacity()) throw new IllegalArgumentException("table " + table.id() + " has
no seat " + seat);
        meals.computeIfAbsent(seat, Meal::new).add(new MealItem(item, qty));
    }

    synchronized List<Meal> meals() { return List.copyOf(meals.values()); }

    BigDecimal subtotal() { return meals().stream().map(Meal::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add);
}

    void moveTo(OrderStatus next) {
        synchronized (this) {
            boolean ok = switch (status) {
                case RECEIVED -> next == OrderStatus.PREPARING || next == OrderStatus.CANCELLED;
                case PREPARING -> next == OrderStatus.READY;
                case READY -> next == OrderStatus.SERVED;
                case SERVED, CANCELLED -> false;
            };
            if (!ok) throw new IllegalStateException("order " + id + ": " + status + " -> " + next + " not
allowed");
            status = next;
```

```
        }
        listeners.forEach(l -> l.onStatusChange(this, next));   // notify outside the lock
    }
}
```

```
// File: code/lld/ch16/restaurant/Kitchen.java
package lld.ch16.restaurant;

import java.util.ArrayDeque;
import java.util.Comparator;
import java.util.Deque;
import java.util.List;
import java.util.concurrent.CopyOnWriteArrayList;

final class Chef extends Employee {
    private final Deque<Order> queue = new ArrayDeque<>();
    private Order current;

    Chef(String id, String name) { super(id, name); }

    synchronized void enqueue(Order o) { queue.addLast(o); }
    synchronized int load() { return queue.size() + (current == null ? 0 : 1); }

    /** Take the next order in FIFO order and start cooking it. */
    synchronized Order startNext() {
        if (current != null) throw new IllegalStateException(name() + " is still cooking " + current.id());
        current = queue.pollFirst();
        if (current != null) current.moveTo(OrderStatus.PREPARING);
        return current;
    }

    synchronized void finishCurrent() {
        if (current == null) throw new IllegalStateException(name() + " has nothing on the stove");
        current.moveTo(OrderStatus.READY);
        current = null;
    }
}

/** Strategy: which chef gets the next order. */
interface ChefAssignment { Chef pick(List<Chef> chefs); }

final class RoundRobinAssignment implements ChefAssignment {
    private int next;
    public synchronized Chef pick(List<Chef> chefs) { return chefs.get(next++ % chefs.size()); }
}

final class LeastLoadedAssignment implements ChefAssignment {
    public Chef pick(List<Chef> chefs) { return
chefs.stream().min(Comparator.comparingInt(Chef::load)).orElseThrow(); }
}

final class Kitchen {
    private final List<Chef> chefs = new CopyOnWriteArrayList<>();
    private final ChefAssignment assignment;

    Kitchen(ChefAssignment assignment) { this.assignment = assignment; }
    void addChef(Chef c) { chefs.add(c); }

    Chef submit(Order o) {
        if (chefs.isEmpty()) throw new IllegalStateException("no chef on shift");
        Chef c = assignment.pick(chefs);
        c.enqueue(o);
        return c;
    }
}
```

```java
// File: code/lld/ch16/restaurant/Staff.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

abstract class Employee {
    private final String id;
    private final String name;
    protected Employee(String id, String name) { this.id = id; this.name = name; }
    String id() { return id; }
    String name() { return name; }
}

final class Receptionist extends Employee {
    Receptionist(String id, String name) { super(id, name); }
}

/** A waiter opens orders and is told (Observer) when a dish is ready. */
final class Waiter extends Employee implements OrderListener {
    private static final AtomicInteger SEQ = new AtomicInteger();
    Waiter(String id, String name) { super(id, name); }

    Order openOrder(Table t) {
        t.setStatus(TableStatus.OCCUPIED);
        var o = new Order("O" + SEQ.incrementAndGet(), t);
        o.subscribe(this);
        return o;
    }

    public void onStatusChange(Order o, OrderStatus s) {
        if (s == OrderStatus.READY) System.out.println("    [" + name() + "'s pager] order " + o.id() + " for
table " + o.table().id() + " is READY");
    }
}

final class Manager extends Employee {
    Manager(String id, String name) { super(id, name); }
    void addMenuItem(Menu menu, String section, MenuItem item) { menu.section(section).add(item); }
    void hire(Branch b, Employee e) { b.hire(e); }
}

/** The bill ("check"). It is computed from the order's lines, never stored separately. */
record Check(Order order, BigDecimal subtotal, BigDecimal tax, BigDecimal total) {
    static final BigDecimal TAX_RATE = new BigDecimal("0.08");
    static Check of(Order o) {
        var sub = o.subtotal();
        var tax = sub.multiply(TAX_RATE).setScale(2, RoundingMode.HALF_UP);
        return new Check(o, sub, tax, sub.add(tax));
    }
}

/** Strategy: three ways to pay, three behaviours. */
sealed interface Payment permits CardPayment, CashPayment, CheckPayment { PaymentStatus pay(BigDecimal amount); }

record CardPayment(String cardToken) implements Payment {
    public PaymentStatus pay(BigDecimal amount) { return PaymentStatus.COMPLETED; }
}

record CashPayment(BigDecimal tendered) implements Payment {
    public PaymentStatus pay(BigDecimal amount) {
        if (tendered.compareTo(amount) < 0) return PaymentStatus.DECLINED;
        System.out.println("    change due: " + tendered.subtract(amount));
        return PaymentStatus.COMPLETED;
    }
}
```

```java
/** A paper check has to clear at the bank, so it is not money yet. */
record CheckPayment(String bank, String checkNumber) implements Payment {
    public PaymentStatus pay(BigDecimal amount) { return PaymentStatus.PENDING; }
}

final class Cashier extends Employee {
    Cashier(String id, String name) { super(id, name); }

    PaymentStatus settle(Check check, Payment payment) {
        // Check the order state BEFORE taking money: a refused state change after payment would leave the guest charged.
        if (check.order().status() != OrderStatus.READY)
            throw new IllegalStateException("order " + check.order().id() + " is " + check.order().status() + ", not READY");
        var status = payment.pay(check.total());
        if (status != PaymentStatus.DECLINED) {
            check.order().moveTo(OrderStatus.SERVED);
            check.order().table().setStatus(TableStatus.FREE);
        }
        return status;
    }
}

/** A branch owns its menu, kitchen and charts; staff are associated (they can move branches). */
final class Branch {
    private final String name;
    private final Menu menu = new Menu();
    private final Kitchen kitchen;
    private final List<TableChart> charts = new ArrayList<>();
    private final List<Employee> staff = new ArrayList<>();
    private final ReservationService reservations;

    Branch(String name, List<Table> tables, ChefAssignment assignment, Notifier notifier) {
        this.name = name;
        this.kitchen = new Kitchen(assignment);
        this.reservations = new ReservationService(tables, notifier);
        charts.add(new TableChart("Default", tables));
    }
    String name() { return name; }
    Menu menu() { return menu; }
    Kitchen kitchen() { return kitchen; }
    ReservationService reservations() { return reservations; }
    synchronized void addChart(TableChart c) { charts.add(c); }
    synchronized List<TableChart> charts() { return List.copyOf(charts); }
    synchronized void hire(Employee e) {
        staff.add(e);
        if (e instanceof Chef c) kitchen.addChef(c);
    }
    synchronized List<Employee> staff() { return List.copyOf(staff); }
}

final class Restaurant {
    private final String name;
    private final List<Branch> branches = new ArrayList<>();
    Restaurant(String name) { this.name = name; }
    Branch open(Branch b) { branches.add(b); return b; }
    String name() { return name; }
}
```

```java
// File: code/lld/ch16/restaurant/Main.java
package lld.ch16.restaurant;

import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.List;

public class Main {
    static MenuItem item(String id, String title, String price) { return new MenuItem(id, title, new
BigDecimal(price)); }

    public static void main(String[] args) {
        var t1 = new Table("T1", 2);
        var t2 = new Table("T2", 4);
        var t3 = new Table("T3", 4);
        var t4 = new Table("T4", 6);
        var tables = List.of(t1, t2, t3, t4);

        var restaurant = new Restaurant("Spice Route");
        var branch = restaurant.open(new Branch("Downtown", tables, new RoundRobinAssignment(), new
SmsNotifier()));
        branch.addChart(new TableChart("Banquet", List.of(t4)));

        var manager = new Manager("E1", "Maya");
        var reception = new Receptionist("E2", "Ravi");
        var wendy = new Waiter("E3", "Wendy");
        var cashier = new Cashier("E4", "Carl");
        for (var e : List.of(manager, reception, wendy, cashier, new Chef("E5", "Chen"), new Chef("E6", "Maria")))
            manager.hire(branch, e);

        System.out.println("1) Manager builds the menu");
        var menu = branch.menu();
        manager.addMenuItem(menu, "Starters", item("S1", "Tomato soup", "6.50"));
        manager.addMenuItem(menu, "Starters", item("S2", "Garlic bread", "4.00"));
        manager.addMenuItem(menu, "Mains", item("M1", "Margherita pizza", "12.00"));
        manager.addMenuItem(menu, "Mains", item("M2", "Pasta arrabbiata", "14.50"));
        manager.addMenuItem(menu, "Drinks", item("D1", "Lemonade", "3.00"));
        menu.sections().forEach(s -> System.out.println("   " + s.title() + ": " +
s.items().stream().map(MenuItem::title).toList()));
        System.out.println("   charts: " + branch.charts().stream().map(TableChart::name).toList());

        var rs = branch.reservations();
        var seven = LocalDateTime.of(2026, 10, 9, 19, 0);
        var eight = seven.plusHours(1);
        System.out.println("2) " + reception.name() + " searches tables for 4 at 19:00: "
                + rs.search(seven, 4).stream().map(Table::id).toList());
        var asha = rs.reserve(new Customer("Asha", "555-0101"), 4, seven).orElseThrow();
        var ben = rs.reserve(new Customer("Ben", "555-0102"), 4, eight).orElseThrow();
        System.out.println("   Asha -> " + asha.table().id() + ", Ben -> " + ben.table().id() + " (T2 is held
19:00-21:00)");
        System.out.println("   party of 8 at 19:00: " + rs.reserve(new Customer("Big group", "555-0199"), 8,
seven)
                .map(Reservation::id).orElse("no single table fits"));
        try { asha.updatePeopleCount(6); }
        catch (IllegalArgumentException e) { System.out.println("   Asha's party grows to 6: rejected, " +
e.getMessage()); }
        rs.cancel(ben.id());
        System.out.println("   after Ben cancels, free for 4 at 20:00: " + rs.search(eight,
4).stream().map(Table::id).toList());

        System.out.println("3) Reminder job at 18:15");
        rs.sendReminders(seven.minusMinutes(45));
        rs.sendReminders(seven.minusMinutes(30));   // second run does not spam

        System.out.println("4) Asha checks in; a couple walks in");
        var table = rs.checkIn(asha.id());
        var walkIn = rs.freeForWalkIn(2).getFirst();
        System.out.println("   Asha seated at " + table.id() + ", walk-in couple seated at " + walkIn.id());

        System.out.println("5) Wendy takes two orders, one meal per seat");
```

```
        var o1 = wendy.openOrder(table);
        o1.add(1, menu.find("S1").orElseThrow(), 1);
        o1.add(1, menu.find("M1").orElseThrow(), 1);
        o1.add(2, menu.find("M2").orElseThrow(), 1);
        o1.add(2, menu.find("D1").orElseThrow(), 2);
        o1.add(3, menu.find("S2").orElseThrow(), 1);
        var o2 = wendy.openOrder(walkIn);
        o2.add(1, menu.find("M1").orElseThrow(), 2);
        var kitchen = branch.kitchen();
        var c1 = kitchen.submit(o1);
        var c2 = kitchen.submit(o2);
        System.out.println("   " + o1.id() + " -> chef " + c1.name() + ", " + o2.id() + " -> chef " + c2.name());

        System.out.println("6) Kitchen works the queues");
        c1.startNext();
        try { o1.add(4, menu.find("D1").orElseThrow(), 1); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        c2.startNext();
        c1.finishCurrent();
        c2.finishCurrent();

        System.out.println("7) Check for " + o1.id() + " (split by seat)");
        for (var meal : o1.meals()) {
            System.out.println("   seat " + meal.seat() + ": " + meal.items().stream()
                    .map(i -> i.quantity() + "x " + i.item().title()).toList() + " = " + meal.subtotal());
        }
        var check1 = Check.of(o1);
        System.out.println("   subtotal " + check1.subtotal() + ", tax " + check1.tax() + ", total " +
check1.total());
        System.out.println("   paid cash: " + cashier.settle(check1, new CashPayment(new BigDecimal("60.00"))));
        var check2 = Check.of(o2);
        System.out.println("   " + o2.id() + " total " + check2.total() + " paid by check: " +
cashier.settle(check2, new CheckPayment("First Bank", "000123")));
        System.out.println("   table statuses: " + tables.stream().map(t -> t.id() + "=" + t.status()).toList());
    }
}
```

Output:

```
1) Manager builds the menu
   Starters: [Tomato soup, Garlic bread]
   Mains: [Margherita pizza, Pasta arrabbiata]
   Drinks: [Lemonade]
   charts: [Default, Banquet]
2) Ravi searches tables for 4 at 19:00: [T2, T3, T4]
    [sms 555-0101] Table T2 booked for 4 at 2026-10-09T19:00
    [sms 555-0102] Table T3 booked for 4 at 2026-10-09T20:00
   Asha -> T2, Ben -> T3 (T2 is held 19:00-21:00)
   party of 8 at 19:00: no single table fits
   Asha's party grows to 6: rejected, table T2 seats 4, not 6
    [sms 555-0102] Reservation RES-2 cancelled
   after Ben cancels, free for 4 at 20:00: [T3, T4]
3) Reminder job at 18:15
    [sms 555-0101] Reminder: your table T2 at 19:00
4) Asha checks in; a couple walks in
   Asha seated at T2, walk-in couple seated at T1
5) Wendy takes two orders, one meal per seat
   O1 -> chef Chen, O2 -> chef Maria
6) Kitchen works the queues
   rejected: order O1 is already PREPARING
    [Wendy's pager] order O1 for table T2 is READY
    [Wendy's pager] order O2 for table T1 is READY
7) Check for O1 (split by seat)
   seat 1: [1x Tomato soup, 1x Margherita pizza] = 18.50
   seat 2: [1x Pasta arrabbiata, 2x Lemonade] = 20.50
   seat 3: [1x Garlic bread] = 4.00
   subtotal 43.00, tax 3.44, total 46.44
    change due: 13.56
   paid cash: COMPLETED
   O2 total 25.92 paid by check: PENDING
   table statuses: [T1=FREE, T2=FREE, T3=FREE, T4=FREE]
```

Step 2 shows best fit and slot holding (Ben cannot get T2 at 20:00 because Asha's 19:00 booking holds 19:00 and 20:00). Step 6 shows the state machine rejecting a late addition and the Observer paging the waiter. Step 7 shows the per-seat split and the three payment behaviours.

Concurrency & edge cases

- **Two receptionists book the same table.** `search` is only advisory; `reserve` calls `Table.tryHold`, which is `synchronized` and checks-and-holds both slots in one step. If it fails (someone else won) the loop tries the next table in best-fit order.
- **Notifying outside the lock.** `Order.moveTo` changes state inside `synchronized` but calls listeners *after* leaving it. Calling unknown code while holding a lock is a classic way to cause deadlocks or long waits. A **deadlock** is when two threads each hold a lock the other needs, so both wait forever.
- **Reminder spam.** The reminder job may run every few minutes; `markReminded` makes it send once per reservation.
- **Party larger than any table.** Returns "no single table fits". Combining tables is out of scope (see Extensions). The course notes the design already allows it, since a reservation can hold a list of tables.
- **No-shows.** `NO_SHOW` exists as a status; a job can release tables for bookings not checked in 15 minutes after start.
- **Party grows after booking.** `updatePeopleCount(6)` on a four-seat table is rejected (step 2 of the output); the receptionist must re-book.
- **Paying for food that is not ready.** `Cashier.settle` checks the order is `READY` *before* calling the payment, so a refused state change can never happen after the guest's money has been taken.
- **Unknown reservation id** on cancel or check-in fails with a clear `IllegalArgumentException` (Java's standard "you passed a bad value" error), not a `NullPointerException` (the confusing crash you get when code uses a missing object).
- **Seat number out of range** on an order is rejected.
- **Cash short** → `DECLINED`, the order stays `READY` and the table stays occupied.
- **Cheque** → `PENDING`: the meal is served, but the money is not yet confirmed.

Extensions & follow-up questions

- **"Large party needs two tables?"** — Let a reservation hold `List<Table>`; search for combinations of adjacent tables (tag tables with a zone); hold all of them atomically by locking tables in a fixed order (for example by id) to avoid deadlock.
- **"Split the bill?"** — Already easy: `Meal.subtotal()` per seat; create one `Check` per seat group.
- **"Tips and service charge?"** — Add lines to `Check` (percentage or amount chosen at payment).
- **"Waiting list for walk-ins?"** — A FIFO queue per party size; when a table frees, notify the head.
- **"Kitchen stock?"** — An `Inventory` of ingredients; each `MenuItem` has a recipe; placing an order reserves ingredients and marks items "86'd" (unavailable) when stock runs out.
- **"Real concurrency in the kitchen?"** — Each chef becomes a worker thread (a thread that loops, picking up jobs) taking from a `BlockingQueue<Order>` (a thread-safe queue whose `take()` waits when it is empty); `take()` blocks until an order arrives.
- **"Shift schedules?"** — A `Shift` entity (employee, branch, start, end); `Kitchen` only picks chefs on shift.

In the interview

30-second pitch: "Branch owns a menu tree, a kitchen, seating charts and a reservation service. Tables keep both current status for walk-ins and held time slots for bookings; the service searches best-fit and holds slots atomically. Orders have one meal per seat, move through a strict state machine, and notify the waiter through an observer. The kitchen assigns chefs by a strategy. The check is computed from the order, and payment is a strategy with card, cash and cheque."

Minutes	Do
0–5	Clarify branches, seating layouts, per-seat meals, payment types.
5–10	Actors (six) and use cases.
10–20	Class diagram — say "meal per seat" and "status vs time slots" out loud.
20–35	Code `Table.tryHold`, `ReservationService.reserve`, `Order` state machine, `Kitchen.submit`.
35–40	Races, large parties, kitchen load balancing.

Self-check

1. **Q:** Why does a table need both a status and held slots? **A:** Status answers "what is it doing now" (walk-ins); slots answer "is it free at a future time" (reservations).
2. **Q:** Why is search on the service, not the table? **A:** A single table only knows about itself; finding all free tables needs the collection.
3. **Q:** Which pattern tells the waiter the food is ready, and why notify outside the lock? **A:** Observer; calling foreign code while locked risks deadlock and slows other threads.
4. **Q:** Why is there no stored bill total? **A:** The order's lines already hold every price; a second stored total could disagree.
5. **Q:** Why is a cheque payment `PENDING`? **A:** A cheque has to clear at the bank, so it is not money yet.

TL;DR

- Menu → section → item; price only on the item.
- Table = status (now) + held slots (later); service searches best-fit and holds atomically.
- Order → meal per seat → meal item; strict state machine; Observer for "ready".
- Kitchen assigns chefs by Strategy (round-robin or least-loaded).
- Check is computed; card/cash/cheque are three payment strategies.

Real-world example. Restaurant point-of-sale (POS — the till software where orders are entered and paid) systems such as Toast send each order to a kitchen display screen as a "ticket", show it by station and in order, and let the front of house see when it is done — the Observer and FIFO queue

above. Reservation platforms such as OpenTable model table inventory by time slot and party size, exactly like `Table.heldSlots`.

Design an Airline Management System



Figure — An airline flies passengers between airports with owned or leased aircraft (course diagram)

Problem & clarifying questions

An **airline management system** runs an airline's operations: scheduling flights, reserving tickets, cancelling flights, supporting customers and managing staff, plus daily flight updates.

Question	Assumed answer
What does a customer search by?	Date, source airport and destination airport.
One flight or a trip of several?	A customer can build an **itinerary** (the full planned trip) of several flights.
Several passengers per booking?	Yes — a reservation seats several passengers (who need not have accounts).
Seat classes?	Economy, economy plus, preferred economy, business, first; seat types regular, accessible, emergency exit, extra legroom.
Who manages flights?	Only the admin adds aircraft, flights and schedules, and cancels flights; everyone affected is notified.
Crew?	Pilots and crew are assigned to flights and can view their schedule.
Payment?	Paid per itinerary.
Notifications?	When a reservation is made or changed and when a flight's status changes.

Requirements

Functional

1. Search flights by date and by source and destination airport.
2. Reserve a ticket on any scheduled flight; build an itinerary of several flights.
3. View flight schedules: departure, arrival, available seats and other details.
4. Reserve seats for several passengers under one itinerary.
5. Only the admin adds aircraft, flights and schedules; the admin can cancel a scheduled flight and everyone affected is notified.

6. Customers can cancel a reservation or a whole itinerary.
7. Assign pilots and crew to flights.
8. Take payment for reservations.
9. Send the customer a message whenever a booking is created or modified, and whenever the status of one of their flights changes.

Non-functional

- A seat on a flight instance is sold at most once, even under heavy concurrent booking.
- Several passengers in one booking get all their seats or none (**atomic**).
- Fare rules (class multiplier, last-minute surcharge) change without touching booking code.

Use cases

Five actors:

- **Admin** — adds flights and schedules, cancels flights, manages staff and crew.
- **Front desk officer** — reserves and cancels tickets for walk-in customers.
- **Customer** — views schedules, reserves and cancels.
- **Pilot / Crew** — views assigned flights.
- **System** — sends itinerary and flight-status notifications.

Top use cases: search flights; create/modify/view reservation; assign seats to passengers; pay for a reservation; update the flight schedule; assign pilots and crew.

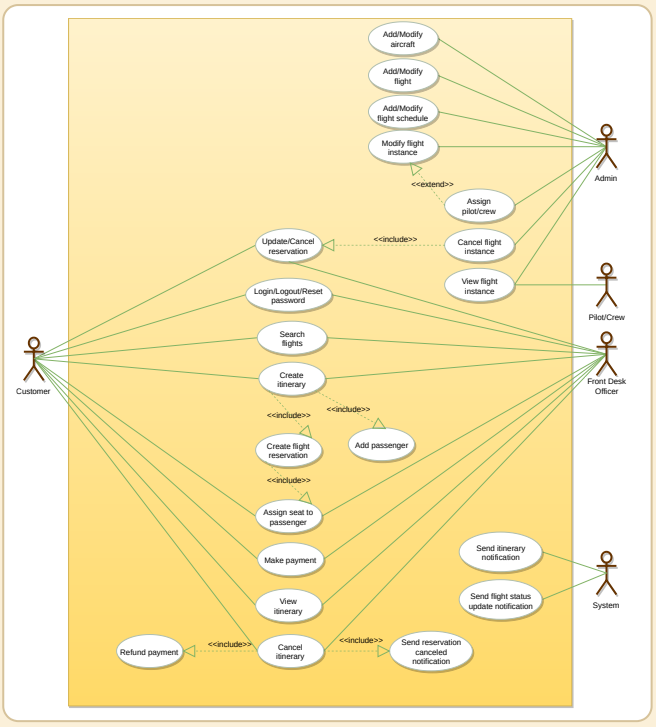


Figure — Use case diagram: admin, front desk officer, customer, crew and system (course diagram)

In the figure above, the front desk officer and the customer share the booking use cases: when an officer books for someone at the counter, it is the very same operation, just triggered by another actor.

Core entities

The one distinction that makes this design work is **four different "seat-ish" and "flight-ish" ideas**:

Idea	Class	Example
The route	`Flight`	GA212 London → Tokyo, 14 h, base fare 500
The route on one date	`FlightInstance`	GA212 on 8 Oct 2026 at 10:00
The physical seat on the aircraft	`Seat`	12A, extra legroom, economy plus
That seat on that one instance	`FlightSeat`	12A on GA212 8 Oct — booked by PNR147 (PNR = booking reference)

Class	Responsibility
`Airport`	Code and city.
`Aircraft`	Registration, model and its physical seats.
`Seat`	Number, type, class — nothing about bookings.
`Flight`	Route, duration, aircraft, base fare, weekly schedules; generates instances.
`WeeklySchedule`	"Every Tuesday at 10:00".
`FlightInstance`	Departure time, status, its `FlightSeat`s and assigned crew; holds seats atomically.
`FlightSeat`	One seat on one instance and which reservation holds it.
`Passenger`	A name on a ticket — deliberately **not** an account holder.
`Customer`	The account holder who books and pays.
`Admin`, `FrontDeskOfficer`	Staff actors. Admin-only methods on `Airline` require an `Admin` argument; the front desk books through `bookFor`.
`CrewMember`	Pilot, co-pilot or cabin crew.
`FlightReservation`	One leg: one instance, map passenger → seat, amount, status.
`Itinerary`	A trip of legs; owns the connection rule; the unit that is paid.
`FareStrategy`	Pluggable pricing: class multiplier, decorated with a last-minute surcharge.
`Airline`	Facade (one simple front object that hides many classes behind it) for admin and customer operations; keeps the index instance → reservations.

Class diagram

The course builds six stages: money and messages, people, what the airline owns, flights and the seats on one flight, reservations and itineraries, and the actors.

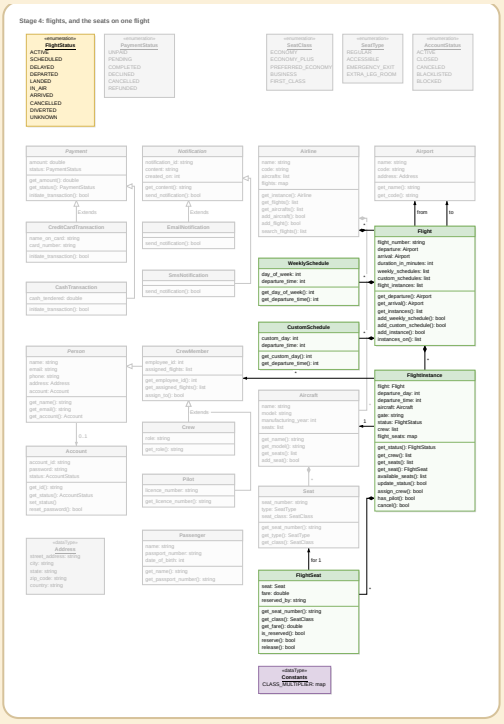


Figure — Flight, FlightInstance and FlightSeat: the route, the route on a day, and a seat on that day (course diagram)

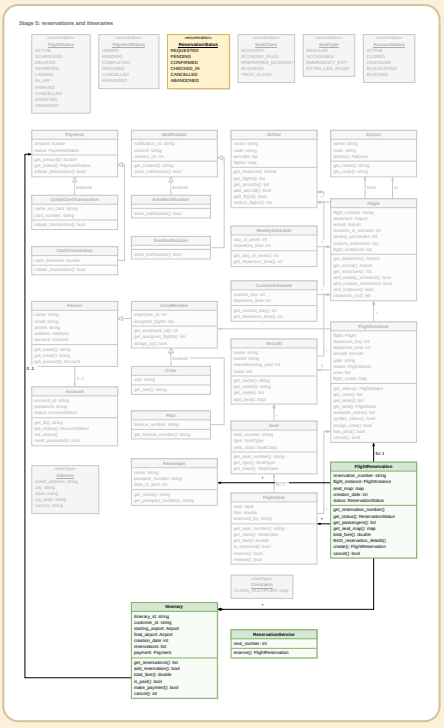


Figure — FlightReservation maps passengers to flight seats; an Itinerary owns its reservations (course diagram)

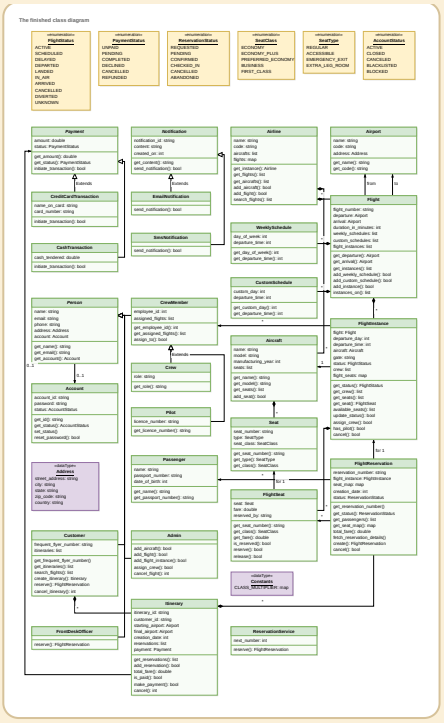
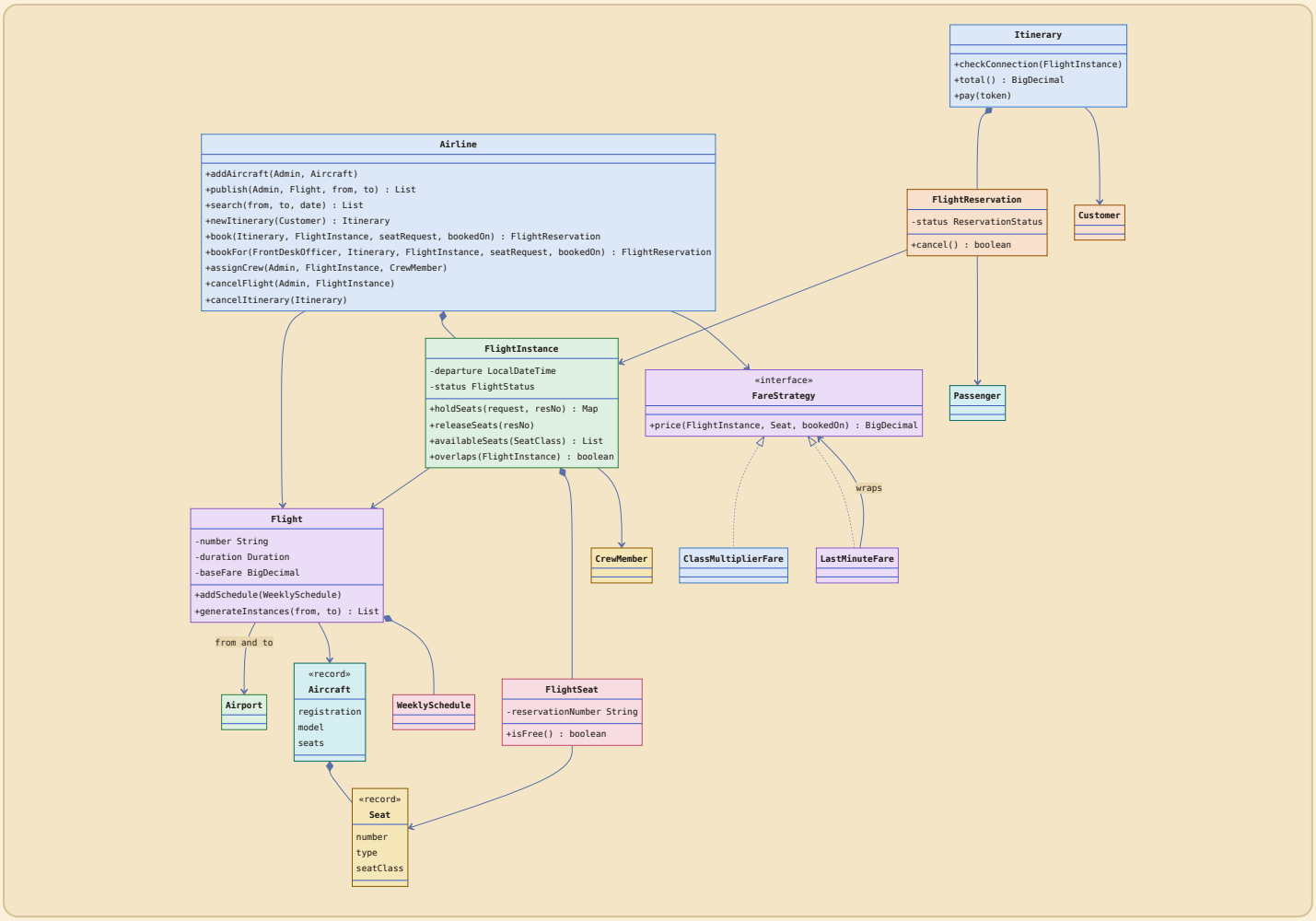


Figure — The finished class diagram of the airline management system (course diagram)

In the first figure above, all the ownership (filled-diamond) arrows run in one direction: schedules and instances belong to their flight, and flight seats belong to their instance. Airports, the aircraft and the crew are only referred to — cancelling a flight does not destroy an airport. Notice also in the finished diagram that `Passenger` sits outside the `Person`/account tree: the course's reason is that a passenger may never log in, and a customer can book for other people.



Key flows

Reserve a ticket. The customer (or front desk officer) searches; if a flight is available they choose seats and passengers, pay, and receive a confirmation; otherwise they search again.

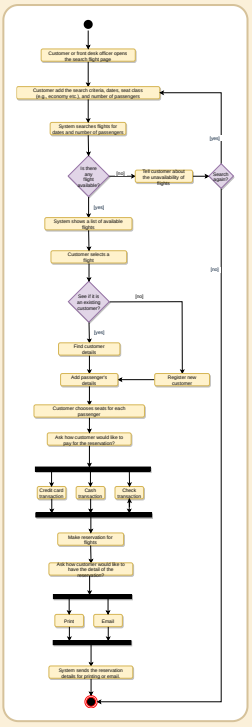


Figure — Activity diagram: reserving a ticket (course diagram)

Cancel a reservation. The system checks that the customer has a valid reservation; if so it cancels it, releases the seats and notifies the customer; if not, it reports the error.

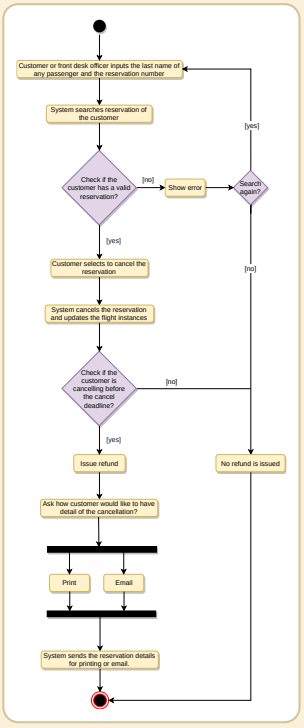
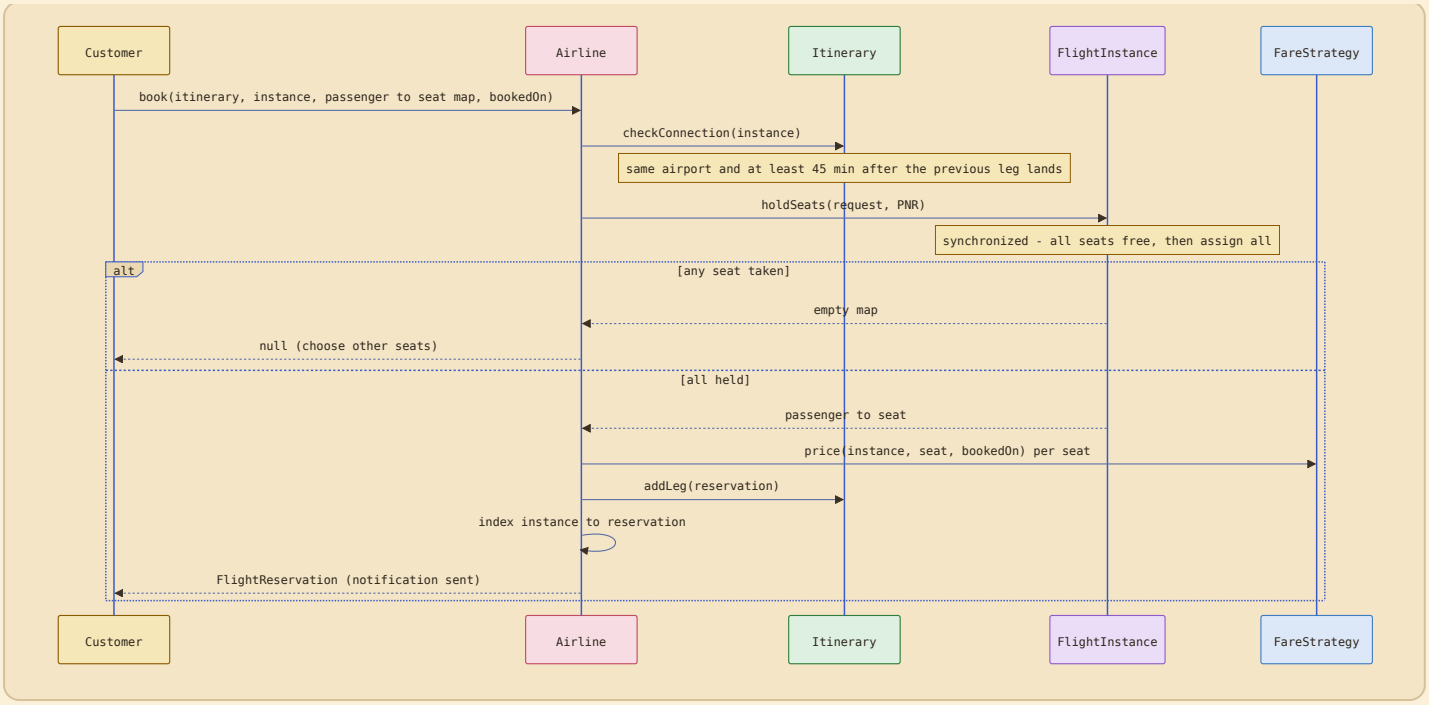
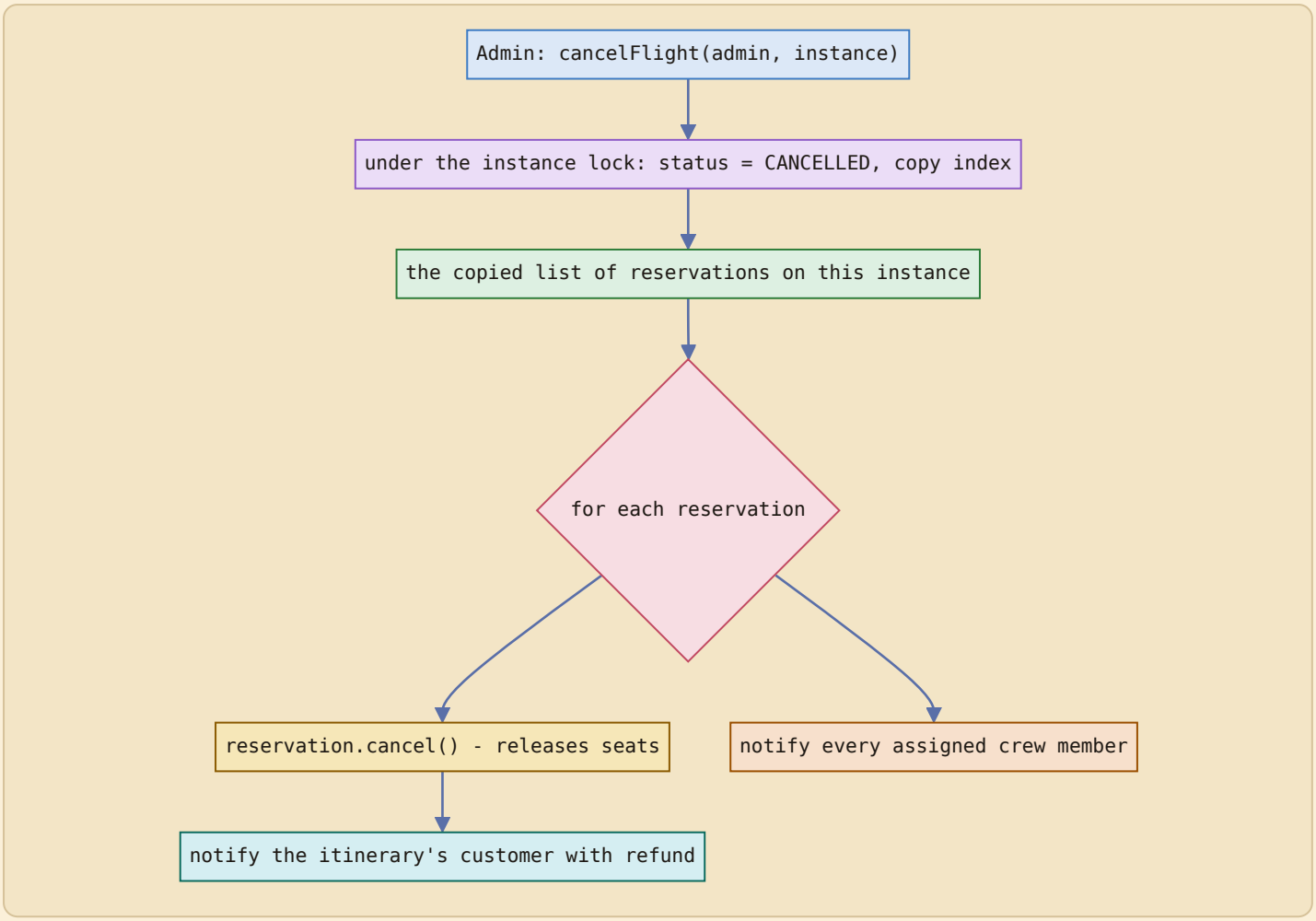


Figure — Activity diagram: cancelling a reservation (course diagram)

Booking one leg of an itinerary:



Admin cancels a flight instance:



Design decisions & patterns used

Decision	Why	Alternative rejected
Flight vs FlightInstance vs Seat vs FlightSeat	12A is free on Tuesday and taken on Thursday, so "booked" and the fare belong to the seat *on an instance*. The course says an older version of its code had drifted from its diagram here — the code is what runs.	A `booked` flag on `Seat`: one booking would block the seat forever.
`holdSeats` is **all-or-nothing** and `synchronized` per instance	A family must get all seats or none; per-instance locks let different flights sell in parallel.	Holding seats one by one (half-booked families), or one global lock (no parallelism).
`Passenger` is not an account	A passenger is a name on a ticket; a customer books for others.	`Passenger extends Person` would force an account on a child.
Itinerary is paid, not the leg	A customer pays for the trip.	Paying per reservation.
Connection rule on `Itinerary`	The course names "no connecting-time check" as a gap; the rule belongs to the thing that holds the legs. We check same airport and a 45-minute minimum.	No check (impossible trips).
Index instance → reservations	The course's other named gap: cancelling a flight told the crew but not passengers, because there was no path from instance to its reservations. The index fixes it.	Scanning every itinerary.
Strategy + Decorator for fares	The course suggests Strategy for fares (base, seasonal, last minute). `ClassMultiplierFare` uses the course's multipliers (1.0, 1.3, 1.5, 2.5, 4.0); `LastMinuteFare` wraps any fare and adds 25% within 3 days.	Price field on the seat.
Crew overlap check	A pilot cannot be on two instances whose times overlap.	Trusting the admin.
Facade (`Airline`)	One entry point for admin and customer operations. Admin-only operations (`addAircraft`, `publish`, `assignCrew`, `cancelFlight`) take an `Admin` parameter, so "only the admin" is enforced by the compiler; `bookFor(FrontDeskOfficer, …)` is the same call as `book` made by a different actor, exactly as in the course's stage 6.	Actors holding the rules.
The course makes `Airline` a Singleton	We construct it once and pass it in, to stay testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.airline`.

```java
// File: code/lld/ch16/airline/Types.java
package lld.ch16.airline;

enum FlightStatus { SCHEDULED, DELAYED, DEPARTED, LANDED, CANCELLED }
enum ReservationStatus { CONFIRMED, CANCELLED }
enum SeatClass { ECONOMY, ECONOMY_PLUS, PREFERRED_ECONOMY, BUSINESS, FIRST_CLASS }
enum SeatType { REGULAR, ACCESSIBLE, EMERGENCY_EXIT, EXTRA_LEG_ROOM }
enum CrewRole { PILOT, CO_PILOT, CABIN_CREW }

record Airport(String code, String city) {}

/** The PHYSICAL seat bolted to an aircraft. It knows nothing about bookings. */
record Seat(String number, SeatType type, SeatClass seatClass) {}

/** A name on a ticket. Not an account holder: a customer can book for other people. */
record Passenger(String name, String passportRef) {}

record Customer(String id, String name, String email) {}

record CrewMember(String id, String name, CrewRole role) {}

/** Staff actors. Admin-only operations take an Admin parameter, so the compiler enforces "only the admin". */
record Admin(String id, String name) {}
record FrontDeskOfficer(String id, String name) {}
```

```java
// File: code/lld/ch16/airline/Fleet.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.DayOfWeek;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.List;

record Aircraft(String registration, String model, List<Seat> seats) {
    Aircraft { seats = List.copyOf(seats); }
}

/** A recurring slot: "every Tuesday at 10:00". */
record WeeklySchedule(DayOfWeek day, LocalTime departure) {}

/**
 * A Flight is the ROUTE (GA212 LHR->NRT). It creates one FlightInstance per scheduled day.
 * All times are in one reference zone (UTC) to keep the example small.
 */
final class Flight {
    private final String number;
    private final Airport from;
    private final Airport to;
    private final Duration duration;
    private final Aircraft aircraft;
    private final BigDecimal baseFare;
    private final List<WeeklySchedule> schedules = new ArrayList<>();

    Flight(String number, Airport from, Airport to, Duration duration, Aircraft aircraft, BigDecimal baseFare) {
        this.number = number; this.from = from; this.to = to;
        this.duration = duration; this.aircraft = aircraft; this.baseFare = baseFare;
    }
    String number() { return number; }
    Airport from() { return from; }
    Airport to() { return to; }
    Duration duration() { return duration; }
    Aircraft aircraft() { return aircraft; }
    BigDecimal baseFare() { return baseFare; }

    void addSchedule(WeeklySchedule s) { schedules.add(s); }

    List<FlightInstance> generateInstances(LocalDate fromDate, LocalDate toDateExclusive) {
        List<FlightInstance> out = new ArrayList<>();
        fromDate.datesUntil(toDateExclusive).forEach(d -> schedules.stream()
                .filter(s -> s.day() == d.getDayOfWeek())
                .forEach(s -> out.add(new FlightInstance(this, LocalDateTime.of(d, s.departure())))));
        return out;
    }
}
```

```java
// File: code/lld/ch16/airline/FlightInstance.java
package lld.ch16.airline;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/** One seat on one flight instance: 12A can be free on Tuesday and taken on Thursday. */
final class FlightSeat {
    private final Seat seat;
    private String reservationNumber;   // null = free
    FlightSeat(Seat seat) { this.seat = seat; }
    Seat seat() { return seat; }
    String reservationNumber() { return reservationNumber; }
    boolean isFree() { return reservationNumber == null; }
    void assign(String resNo) { reservationNumber = resNo; }
    void free() { reservationNumber = null; }
}

/** The route flown on one date: owns its seats, its crew list and its status. */
final class FlightInstance {
    private final Flight flight;
    private final LocalDateTime departure;
    private final Map<String, FlightSeat> seats = new LinkedHashMap<>();
    private final List<CrewMember> crew = new ArrayList<>();
    private FlightStatus status = FlightStatus.SCHEDULED;

    FlightInstance(Flight flight, LocalDateTime departure) {
        this.flight = flight;
        this.departure = departure;
        flight.aircraft().seats().forEach(s -> seats.put(s.number(), new FlightSeat(s)));
    }

    Flight flight() { return flight; }
    LocalDateTime departure() { return departure; }
    LocalDateTime arrival() { return departure.plus(flight.duration()); }
    synchronized FlightStatus status() { return status; }
    synchronized List<CrewMember> crew() { return List.copyOf(crew); }
    String key() { return flight.number() + "@" + departure.toLocalDate(); }

    synchronized List<Seat> availableSeats(SeatClass c) {
        return seats.values().stream().filter(FlightSeat::isFree).map(FlightSeat::seat)
                .filter(s -> s.seatClass() == c).toList();
    }

    synchronized int freeSeatCount() { return (int) seats.values().stream().filter(FlightSeat::isFree).count(); }

    /** All-or-nothing: a family of three either gets all three seats or none. */
    synchronized Map<Passenger, Seat> holdSeats(Map<Passenger, String> request, String reservationNumber) {
        if (status != FlightStatus.SCHEDULED && status != FlightStatus.DELAYED)
            throw new IllegalStateException(key() + " is " + status);
        for (var seatNo : request.values()) {
            var fs = seats.get(seatNo);
            if (fs == null) throw new IllegalArgumentException("no seat " + seatNo + " on " + key());
            if (!fs.isFree()) return Map.of();
        }
        if (request.values().stream().distinct().count() != request.size())
            throw new IllegalArgumentException("two passengers asked for the same seat");
        Map<Passenger, Seat> out = new LinkedHashMap<>();
        request.forEach((p, no) -> { var fs = seats.get(no); fs.assign(reservationNumber); out.put(p, fs.seat());
});
        return out;
    }

    synchronized void releaseSeats(String reservationNumber) {
        seats.values().stream().filter(fs ->
reservationNumber.equals(fs.reservationNumber())).forEach(FlightSeat::free);
    }
```

```
    synchronized void addCrew(CrewMember m) { crew.add(m); }
    synchronized void setStatus(FlightStatus s) { status = s; }

    boolean overlaps(FlightInstance other) {
        return departure.isBefore(other.arrival()) && other.departure().isBefore(arrival());
    }
}
```

```java
// File: code/lld/ch16/airline/Booking.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.math.RoundingMode;
import java.time.Duration;
import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/** Strategy: what one seat on one flight instance costs, booked on a given day. */
interface FareStrategy { BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn); }

final class ClassMultiplierFare implements FareStrategy {
    private static final Map<SeatClass, BigDecimal> MULTIPLIER = new EnumMap<>(Map.of(
            SeatClass.ECONOMY, new BigDecimal("1.0"), SeatClass.ECONOMY_PLUS, new BigDecimal("1.3"),
            SeatClass.PREFERRED_ECONOMY, new BigDecimal("1.5"), SeatClass.BUSINESS, new BigDecimal("2.5"),
            SeatClass.FIRST_CLASS, new BigDecimal("4.0")));
    public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
        return fi.flight().baseFare().multiply(MULTIPLIER.get(seat.seatClass())).setScale(2,
RoundingMode.HALF_UP);
    }
}

/** Decorator on any fare: +25% when booked less than 3 days before departure. */
final class LastMinuteFare implements FareStrategy {
    private final FareStrategy inner;
    LastMinuteFare(FareStrategy inner) { this.inner = inner; }
    public BigDecimal price(FlightInstance fi, Seat seat, LocalDate bookedOn) {
        var p = inner.price(fi, seat, bookedOn);
        long daysBefore = ChronoUnit.DAYS.between(bookedOn, fi.departure().toLocalDate());
        return daysBefore < 3 ? p.multiply(new BigDecimal("1.25")).setScale(2, RoundingMode.HALF_UP) : p;
    }
}

/** One leg: one flight instance, several passengers, each with a seat. */
final class FlightReservation {
    private final String number;
    private final FlightInstance instance;
    private final Map<Passenger, Seat> seatMap;
    private final BigDecimal amount;
    private ReservationStatus status = ReservationStatus.CONFIRMED;

    FlightReservation(String number, FlightInstance instance, Map<Passenger, Seat> seatMap, BigDecimal amount) {
        this.number = number; this.instance = instance; this.seatMap = Map.copyOf(seatMap); this.amount = amount;
    }
    String number() { return number; }
    FlightInstance instance() { return instance; }
    Map<Passenger, Seat> seatMap() { return seatMap; }
    BigDecimal amount() { return amount; }
    synchronized ReservationStatus status() { return status; }

    /** Returns true if this call did the cancelling (idempotent). */
    synchronized boolean cancel() {
        if (status == ReservationStatus.CANCELLED) return false;
        status = ReservationStatus.CANCELLED;
        instance.releaseSeats(number);
        return true;
    }
}

/** A trip of one or more legs. The itinerary, not each leg, is what gets paid for. */
final class Itinerary {
    static final Duration MIN_CONNECTION = Duration.ofMinutes(45);

    private final String id;
    private final Customer customer;
```

```
    private final List<FlightReservation> legs = new ArrayList<>();
    private boolean paid;

    Itinerary(String id, Customer customer) { this.id = id; this.customer = customer; }
    String id() { return id; }
    Customer customer() { return customer; }
    synchronized List<FlightReservation> legs() { return List.copyOf(legs); }

    /** Rule the itinerary owns: the next leg must leave from where the last one landed, with time to change. */
    synchronized void checkConnection(FlightInstance next) {
        if (legs.isEmpty()) return;
        var last = legs.getLast().instance();
        if (!last.flight().to().equals(next.flight().from()))
            throw new IllegalArgumentException("leg must depart from " + last.flight().to().code());
        if (next.departure().isBefore(last.arrival().plus(MIN_CONNECTION)))
            throw new IllegalArgumentException("connection too short: " + next.key() + " leaves before " +
last.key() + " lands + 45 min");
    }

    synchronized void addLeg(FlightReservation r) { legs.add(r); }

    synchronized BigDecimal total() {
        return legs.stream().filter(l -> l.status() == ReservationStatus.CONFIRMED)
                .map(FlightReservation::amount).reduce(new BigDecimal("0.00"), BigDecimal::add);
    }

    /** Idempotent: a second pay() (double click, retry) does not charge again. */
    synchronized void pay(String cardToken) {
        if (paid) return;
        System.out.println("    charged " + total() + " to " + cardToken + " for itinerary " + id);
        paid = true;
    }
    synchronized boolean isPaid() { return paid; }
}
```

```java
// File: code/lld/ch16/airline/Airline.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CopyOnWriteArrayList;
import java.util.concurrent.atomic.AtomicInteger;

interface Notifier { void send(String to, String message); }

final class EmailNotifier implements Notifier {
    public void send(String to, String message) { System.out.println("    [email " + to + "] " + message); }
}

/**
 * Facade over the airline's data: admin operations, search, booking, crew and cancellation.
 * Keeps an index flight-instance -> reservations so a cancelled flight can reach every passenger.
 */
final class Airline {
    private final String name;
    private final FareStrategy fares;
    private final Notifier notifier;
    private final Map<String, Aircraft> fleet = new ConcurrentHashMap<>();
    private final List<FlightInstance> instances = new CopyOnWriteArrayList<>();
    private final Map<FlightInstance, List<FlightReservation>> byInstance = new ConcurrentHashMap<>();
    private final Map<FlightReservation, Itinerary> owner = new ConcurrentHashMap<>();
    private final AtomicInteger seq = new AtomicInteger(100);

    Airline(String name, FareStrategy fares, Notifier notifier) {
        this.name = name; this.fares = fares; this.notifier = notifier;
    }

    // ---- admin operations ----
    void addAircraft(Admin by, Aircraft a) { fleet.put(a.registration(), a); }

    List<FlightInstance> publish(Admin by, Flight f, LocalDate from, LocalDate toExclusive) {
        var created = f.generateInstances(from, toExclusive);
        instances.addAll(created);
        return created;
    }

    /** A crew member cannot be on two instances whose times overlap. */
    synchronized void assignCrew(Admin by, FlightInstance fi, CrewMember m) {
        for (var other : instances) {
            if (other != fi && other.crew().contains(m) && other.overlaps(fi))
                throw new IllegalStateException(m.name() + " already flies " + other.key() + " at that time");
        }
        if (fi.crew().contains(m)) throw new IllegalStateException(m.name() + " already on " + fi.key());
        fi.addCrew(m);
    }

    List<FlightInstance> scheduleFor(CrewMember m) {
        return instances.stream().filter(fi -> fi.crew().contains(m)).toList();
    }

    void cancelFlight(Admin by, FlightInstance fi) {
        List<FlightReservation> affected;
        synchronized (fi) {        // same lock as the hold+index step in book(): no booking can slip past the
cancel
            fi.setStatus(FlightStatus.CANCELLED);
            affected = List.copyOf(byInstance.getOrDefault(fi, List.of()));
        }
        for (var r : affected) {
            if (r.cancel()) {
                var it = owner.get(r);
                notifier.send(it.customer().email(), fi.key() + " is cancelled. Reservation " + r.number()
```

```
                        + " refunded " + r.amount() + ". Itinerary " + it.id() + " now totals " + it.total());
            }
        }
        fi.crew().forEach(c -> notifier.send(c.id() + "@crew", "Stand down: " + fi.key() + " cancelled"));
    }

    // ---- customer operations ----
    List<FlightInstance> search(Airport from, Airport to, LocalDate date) {
        return instances.stream().filter(fi -> fi.flight().from().equals(from) && fi.flight().to().equals(to)
                && fi.departure().toLocalDate().equals(date) && fi.status() != FlightStatus.CANCELLED).toList();
    }

    Itinerary newItinerary(Customer c) { return new Itinerary("IT-" + seq.incrementAndGet(), c); }

    /** Books one leg for several passengers. Throws if the connection is invalid; returns null if a seat is
taken. */
    FlightReservation book(Itinerary it, FlightInstance fi, Map<Passenger, String> seatRequest, LocalDate
bookedOn) {
        it.checkConnection(fi);
        String resNo = "PNR" + seq.incrementAndGet();
        FlightReservation r;
        synchronized (fi) {        // hold the seats AND index the reservation as one step (see cancelFlight)
            var seatMap = fi.holdSeats(seatRequest, resNo);
            if (seatMap.isEmpty()) return null;
            var amount = seatMap.values().stream().map(s -> fares.price(fi, s, bookedOn)).reduce(BigDecimal.ZERO,
BigDecimal::add);
            r = new FlightReservation(resNo, fi, seatMap, amount);
            owner.put(r, it);
            byInstance.computeIfAbsent(fi, k -> new CopyOnWriteArrayList<>()).add(r);
        }
        it.addLeg(r);
        var seatMap = r.seatMap();
        var amount = r.amount();
        notifier.send(it.customer().email(), "Booked " + fi.key() + " seats " +
seatMap.values().stream().map(Seat::number).sorted().toList() + " (" + resNo + ", " + amount + ")");
        return r;
    }

    /** The front desk books for a walk-in customer: the same call, made by a different actor. */
    FlightReservation bookFor(FrontDeskOfficer by, Itinerary it, FlightInstance fi, Map<Passenger, String>
seatRequest, LocalDate bookedOn) {
        return book(it, fi, seatRequest, bookedOn);
    }

    void cancelItinerary(Itinerary it) {
        it.legs().forEach(FlightReservation::cancel);
        notifier.send(it.customer().email(), "Itinerary " + it.id() + " cancelled");
    }

    String name() { return name; }
}
```

```java
// File: code/lld/ch16/airline/Main.java
package lld.ch16.airline;

import java.math.BigDecimal;
import java.time.DayOfWeek;
import java.time.Duration;
import java.time.LocalDate;
import java.time.LocalTime;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;
import java.util.concurrent.atomic.AtomicInteger;

public class Main {
    public static void main(String[] args) throws Exception {
        var airline = new Airline("Global Air", new LastMinuteFare(new ClassMultiplierFare()), new
EmailNotifier());
        var lhr = new Airport("LHR", "London");
        var nrt = new Airport("NRT", "Tokyo");
        var jfk = new Airport("JFK", "New York");

        var seats = List.of(
                new Seat("1A", SeatType.REGULAR, SeatClass.BUSINESS), new Seat("1B", SeatType.REGULAR,
SeatClass.BUSINESS),
                new Seat("10A", SeatType.REGULAR, SeatClass.ECONOMY), new Seat("10B", SeatType.REGULAR,
SeatClass.ECONOMY),
                new Seat("10C", SeatType.ACCESSIBLE, SeatClass.ECONOMY), new Seat("12A", SeatType.EXTRA_LEG_ROOM,
SeatClass.ECONOMY_PLUS));
        var a1 = new Aircraft("G-GAAA", "A350-900", seats);
        var a2 = new Aircraft("G-GAAB", "B787-9", seats);
        var admin = new Admin("AD1", "Olga");
        airline.addAircraft(admin, a1);
        airline.addAircraft(admin, a2);

        System.out.println("1) Admin adds flights (routes) and weekly schedules, then publishes instances");
        var ga212 = new Flight("GA212", lhr, nrt, Duration.ofHours(14), a1, new BigDecimal("500.00"));
        ga212.addSchedule(new WeeklySchedule(DayOfWeek.TUESDAY, LocalTime.of(10, 0)));
        ga212.addSchedule(new WeeklySchedule(DayOfWeek.THURSDAY, LocalTime.of(10, 0)));
        var ga100 = new Flight("GA100", jfk, lhr, Duration.ofHours(7), a2, new BigDecimal("400.00"));
        ga100.addSchedule(new WeeklySchedule(DayOfWeek.WEDNESDAY, LocalTime.of(20, 0)));
        var ga300 = new Flight("GA300", lhr, jfk, Duration.ofHours(8), a2, new BigDecimal("450.00"));
        ga300.addSchedule(new WeeklySchedule(DayOfWeek.THURSDAY, LocalTime.of(12, 0)));
        var from = LocalDate.of(2026, 10, 5);
        var to = LocalDate.of(2026, 10, 19);
        var ga212Instances = airline.publish(admin, ga212, from, to);
        airline.publish(admin, ga100, from, to);
        airline.publish(admin, ga300, from, to);
        System.out.println("   GA212 instances: " + ga212Instances.stream().map(FlightInstance::key).toList());

        System.out.println("2) Search JFK->LHR on 7 Oct and LHR->NRT on 8 Oct");
        var leg1 = airline.search(jfk, lhr, LocalDate.of(2026, 10, 7)).getFirst();
        var leg2 = airline.search(lhr, nrt, LocalDate.of(2026, 10, 8)).getFirst();
        for (var fi : List.of(leg1, leg2))
            System.out.println("   " + fi.key() + " dep " + fi.departure() + " arr " + fi.arrival() + ", free
seats " + fi.freeSeatCount());

        System.out.println("3) Priya books a two-leg itinerary for herself and her son");
        var priya = new Customer("C1", "Priya", "priya@example.com");
        var me = new Passenger("Priya Shah", "P-001");
        var son = new Passenger("Arjun Shah", "P-002");
        var bookedOn = LocalDate.of(2026, 10, 1);
        var it = airline.newItinerary(priya);
        airline.book(it, leg1, ordered(me, "10A", son, "10B"), bookedOn);
        airline.book(it, leg2, ordered(me, "1A", son, "1B"), bookedOn);
        System.out.println("   itinerary total " + it.total() + " (2 x 400 economy + 2 x 1250 business)");
        it.pay("tok_priya_card");
        it.pay("tok_priya_card");   // double click: no second charge
```

```
        System.out.println("4) Bad connection is rejected");
        var it2 = airline.newItinerary(priya);
        airline.book(it2, leg1, Map.of(me, "12A"), bookedOn);
        try { airline.book(it2, ga212Instances.getFirst(), Map.of(me, "10A"), bookedOn); }
        catch (IllegalArgumentException e) { System.out.println("   rejected: " + e.getMessage()); }
        airline.cancelItinerary(it2);

        System.out.println("5) 20 customers race for seat 10C on " + leg2.key());
        var winners = new AtomicInteger();
        var go = new CountDownLatch(1);
        var raceAirline = airline;   // effectively-final alias for the lambdas
        try (var pool = Executors.newFixedThreadPool(20)) {
            for (int i = 0; i < 20; i++) {
                int n = i;
                pool.submit(() -> {
                    go.await();
                    var c = new Customer("R" + n, "Racer" + n, "racer" + n + "@example.com");
                    var r = raceAirline.book(raceAirline.newItinerary(c), leg2, Map.of(new Passenger("Racer" + n,
"X" + n), "10C"), bookedOn);
                    if (r != null) winners.incrementAndGet();
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   successful bookings of 10C: " + winners.get() + ", free seats left: " +
leg2.freeSeatCount());

        System.out.println("6) Walk-in at the front desk, last-minute (booked 7 Oct for 8 Oct): pays +25%");
        var desk = new FrontDeskOfficer("FD1", "Tom");
        var late = airline.newItinerary(new Customer("C2", "Leo", "leo@example.com"));
        var r = airline.bookFor(desk, late, leg2, Map.of(new Passenger("Leo Park", "P-003"), "12A"),
LocalDate.of(2026, 10, 7));
        System.out.println("   economy-plus fare 500 x 1.3 = 650.00, charged " + r.amount());

        System.out.println("7) Admin assigns crew");
        var kim = new CrewMember("K1", "Capt. Kim", CrewRole.PILOT);
        var lee = new CrewMember("L1", "F/O Lee", CrewRole.CO_PILOT);
        var ana = new CrewMember("A1", "Ana", CrewRole.CABIN_CREW);
        for (var m : List.of(kim, lee, ana)) airline.assignCrew(admin, leg2, m);
        var ga300Oct8 = airline.search(lhr, jfk, LocalDate.of(2026, 10, 8)).getFirst();
        try { airline.assignCrew(admin, ga300Oct8, kim); }
        catch (IllegalStateException e) { System.out.println("   rejected: " + e.getMessage()); }
        System.out.println("   Kim's schedule: " +
airline.scheduleFor(kim).stream().map(FlightInstance::key).toList());

        System.out.println("8) Admin cancels " + leg2.key() + ": every passenger and crew member is told");
        airline.cancelFlight(admin, leg2);
        System.out.println("   search LHR->NRT on 8 Oct now returns " + airline.search(lhr, nrt,
LocalDate.of(2026, 10, 8)).size() + " flights");

        System.out.println("9) Priya cancels the rest of her trip; seats on " + leg1.key() + " come back");
        airline.cancelItinerary(it);
        System.out.println("   free economy seats on " + leg1.key() + ": " +
leg1.availableSeats(SeatClass.ECONOMY).stream().map(Seat::number).toList());
    }

    static Map<Passenger, String> ordered(Passenger a, String sa, Passenger b, String sb) {
        var m = new LinkedHashMap<Passenger, String>();
        m.put(a, sa);
        m.put(b, sb);
        return m;
    }
}
```

Output:

```
1) Admin adds flights (routes) and weekly schedules, then publishes instances
   GA212 instances: [GA212@2026-10-06, GA212@2026-10-08, GA212@2026-10-13, GA212@2026-10-15]
2) Search JFK->LHR on 7 Oct and LHR->NRT on 8 Oct
   GA100@2026-10-07 dep 2026-10-07T20:00 arr 2026-10-08T03:00, free seats 6
   GA212@2026-10-08 dep 2026-10-08T10:00 arr 2026-10-09T00:00, free seats 6
3) Priya books a two-leg itinerary for herself and her son
    [email priya@example.com] Booked GA100@2026-10-07 seats [10A, 10B] (PNR102, 800.00)
    [email priya@example.com] Booked GA212@2026-10-08 seats [1A, 1B] (PNR103, 2500.00)
   itinerary total 3300.00 (2 x 400 economy + 2 x 1250 business)
    charged 3300.00 to tok_priya_card for itinerary IT-101
4) Bad connection is rejected
    [email priya@example.com] Booked GA100@2026-10-07 seats [12A] (PNR105, 520.00)
   rejected: connection too short: GA212@2026-10-06 leaves before GA100@2026-10-07 lands + 45 min
    [email priya@example.com] Itinerary IT-104 cancelled
5) 20 customers race for seat 10C on GA212@2026-10-08
    [email racer7@example.com] Booked GA212@2026-10-08 seats [10C] (PNR115, 500.00)
   successful bookings of 10C: 1, free seats left: 3
6) Walk-in at the front desk, last-minute (booked 7 Oct for 8 Oct): pays +25%
    [email leo@example.com] Booked GA212@2026-10-08 seats [12A] (PNR147, 812.50)
   economy-plus fare 500 x 1.3 = 650.00, charged 812.50
7) Admin assigns crew
   rejected: Capt. Kim already flies GA212@2026-10-08 at that time
   Kim's schedule: [GA212@2026-10-08]
8) Admin cancels GA212@2026-10-08: every passenger and crew member is told
    [email priya@example.com] GA212@2026-10-08 is cancelled. Reservation PNR103 refunded 2500.00. Itinerary IT-101
now totals 800.00
    [email racer7@example.com] GA212@2026-10-08 is cancelled. Reservation PNR115 refunded 500.00. Itinerary IT-107
now totals 0.00
    [email leo@example.com] GA212@2026-10-08 is cancelled. Reservation PNR147 refunded 812.50. Itinerary IT-146
now totals 0.00
    [email K1@crew] Stand down: GA212@2026-10-08 cancelled
    [email L1@crew] Stand down: GA212@2026-10-08 cancelled
    [email A1@crew] Stand down: GA212@2026-10-08 cancelled
   search LHR->NRT on 8 Oct now returns 0 flights
9) Priya cancels the rest of her trip; seats on GA100@2026-10-07 come back
    [email priya@example.com] Itinerary IT-101 cancelled
   free economy seats on GA100@2026-10-07: [10A, 10B, 10C]
```

Step 3 prices 2 × 400 economy + 2 × (500 × 2.5) business = 3300, and the second `pay` call charges nothing. Step 5 has 20 threads racing for seat 10C and exactly one wins (which racer wins, and therefore the PNR and itinerary numbers printed for them in steps 5 and 8, changes from run to run). Step 6 is 500 × 1.3 = 650, plus 25% last-minute = 812.50. Step 8 reaches every passenger through the index — the gap the course points out.

Concurrency & edge cases

- **Selling one seat twice.** `holdSeats` is `synchronized` on the instance: it checks every requested seat and only then assigns them all. No other thread can slip in between.
- **Booking while the admin cancels the flight.** `book` holds the seats *and* adds the reservation to the instance → reservations index inside one `synchronized (fi)` block, and `cancelFlight` sets `CANCELLED` and copies that index under the same lock. So every reservation is either in the copy (and gets cancelled and notified) or is refused by `holdSeats` because the flight is already `CANCELLED` — none can slip through un-notified.
- **Double payment.** `Itinerary.pay` returns at once if the itinerary is already paid (step 3 calls it twice and charges once).
- **Duplicate seat in one request** (two passengers asking for 12A) is rejected.
- **Booking a cancelled or departed flight** throws.
- **Idempotent cancel.** `FlightReservation.cancel()` returns `false` if already cancelled, so a customer cancelling while the admin cancels the flight does not double-refund.
- **Crew double-booking** across overlapping instances is rejected (`assignCrew` is `synchronized` over the scan and the add).
- **Time zones.** Our example keeps all times in one reference zone. Real systems store departures as `ZonedDateTime` (a date-time plus its time zone) or `Instant` (an exact moment on the global UTC timeline) with each airport's zone, because London and Tokyo are 8–9 hours apart.

- **Seat holds during payment.** Real booking flows put a seat on a short *hold* (for example 10–15 minutes) while the customer pays, and release it automatically if payment fails. Add a `heldUntil` timestamp on `FlightSeat` and a sweeper (a background job that regularly frees expired holds).

Extensions & follow-up questions

- **"Overbooking on purpose?"** — Airlines sell a few more seats than exist because some passengers do not show up. Model a per-class `overbookLimit` and assign physical seats at check-in.
- **"Check-in and boarding passes?"** — Add `CHECKED_IN` per passenger, a `BoardingPass` (seat, gate, group), and a check-in window.
- **"Baggage?"** — `Baggage(passenger, weight, tag)` on the reservation; fees as extra lines.
- **"Change fees and fare rules?"** — Fare class with rules (refundable, change fee); cancellation refund = fare − fee.
- **"Codeshares?"** — A codeshare is one physical flight sold under several airlines' flight numbers. A `MarketingFlight` (partner's number) pointing at the operating `FlightInstance`.
- **"Delays?"** — `FlightInstance.setStatus(DELAYED)` plus a new departure; notify via the same index; re-check itinerary connections and rebook broken ones.
- **"Scale?"** — Seat inventory per instance is a hot row (one database row that very many requests try to change at once); use one row per seat with a conditional update (`UPDATE … WHERE reservation IS NULL` — it only changes the row if the condition still holds, so exactly one buyer wins) or optimistic locking (no lock is taken; each row carries a version number and a write fails if the version changed since you read it, so you retry), and a cache (a fast in-memory copy of data that is read often) for search.

In the interview

30-second pitch: "Four separate ideas carry the design. `Flight` is the published route; `FlightInstance` is one departure of it on a particular day; `Seat` is the chair fixed in the aircraft; `FlightSeat` is that chair on that one departure, and it is where the fare and the booking live. Reservations map passengers to flight seats and are held atomically per instance. An itinerary groups legs, checks connections and is what gets paid. Fares are a strategy with a last-minute decorator. An index from instance to reservations lets a cancelled flight notify every passenger."

Minutes	Do
0–5	Clarify search, itineraries, multiple passengers, crew, cancellation.
5–10	Actors and use cases.
10–20	The four-way distinction, then the class diagram.
20–35	Code `FlightInstance.holdSeats`, `Airline.book`, `Itinerary.checkConnection`, `cancelFlight`.
35–40	Seat holds during payment, overbooking, time zones.

Expected follow-up: "A flight is cancelled — how do its passengers find out?" — "Through an index from instance to reservations; without it you can only reach the crew."

Self-check

1. **Q:** Where does the "booked" flag live and why? **A:** On `FlightSeat` — a seat's booking is per instance (per date), not per aircraft.
2. **Q:** Why is `Passenger` not a subclass of `Person`/account? **A:** A passenger may never log in; a customer books for other people.
3. **Q:** How do you guarantee a family gets all seats or none? **A:** `holdSeats` checks all requested seats and assigns them in one synchronized block.
4. **Q:** What does `LastMinuteFare` add and which pattern is it? **A:** +25% within 3 days of departure; a Decorator over any `FareStrategy`.
5. **Q:** What two gaps did the course admit and how did we close them? **A:** No passenger notification on flight cancel (fixed by the instance → reservations index) and no connection-time check (fixed in `Itinerary.checkConnection`).

TL;DR

- Flight (route) ≠ FlightInstance (route on a date); Seat (physical) ≠ FlightSeat (seat on that date).
- Hold seats all-or-nothing under the instance's lock.
- Itinerary = legs + connection rule + payment.
- Fares: Strategy + Decorator. Keep an instance → reservations index for cancellations.

Real-world example. Airline systems separate a *flight number* (a published schedule) from a *flight leg on a date*; inventory is controlled per leg and per booking class, and the booking record (a PNR, passenger name record) groups passengers and segments — the same shape as `FlightReservation` and `Itinerary` here. When an airline cancels a flight, its systems find every PNR on that leg and rebook or notify passengers, which is exactly why the index matters.

Design a Course Registration System

Problem & clarifying questions

A **course registration system** lets university students sign up for classes each term. It looks simple, but it is one of the harshest concurrency problems in this chapter: at 09:00 on registration day, thousands of students click "register" for the same few popular sections within seconds.

Terms first:

- A **course** is a catalogue entry (CS201 Data Structures, 4 credits).
- A **section** is one offering of a course with its own instructor, meeting time and capacity (CS201-A, Mon/Wed 10:00–11:30, 30 seats). Students register for **sections**, not courses — this is the same split as Flight vs FlightInstance.
- A **prerequisite** is a course you must have completed first.
- A **waitlist** is a first-come-first-served queue of students who want a seat in a full section.
- A **registration window** is the period when a student may register; universities often open it earlier for seniors (priority registration).

Question	Assumed answer
Register for courses or sections?	Sections.
Prerequisites?	Must be completed (not just in progress).
Capacity?	Per section; when full, join a bounded waitlist.
Waitlist promotion?	Automatic when someone drops, in FIFO order, after re-checking the student's timetable.
Limits?	A per-student credit limit; no two sections that meet at the same time; no two sections of the same course.
Windows?	Opens by class year (seniors first); closes for everyone at one time.
Search?	By course code or title.
Concurrency?	Many students at once; a section must never exceed its capacity; one student must not double-register from two tabs.

Requirements

Functional

1. Students register for sections and view their registered sections.

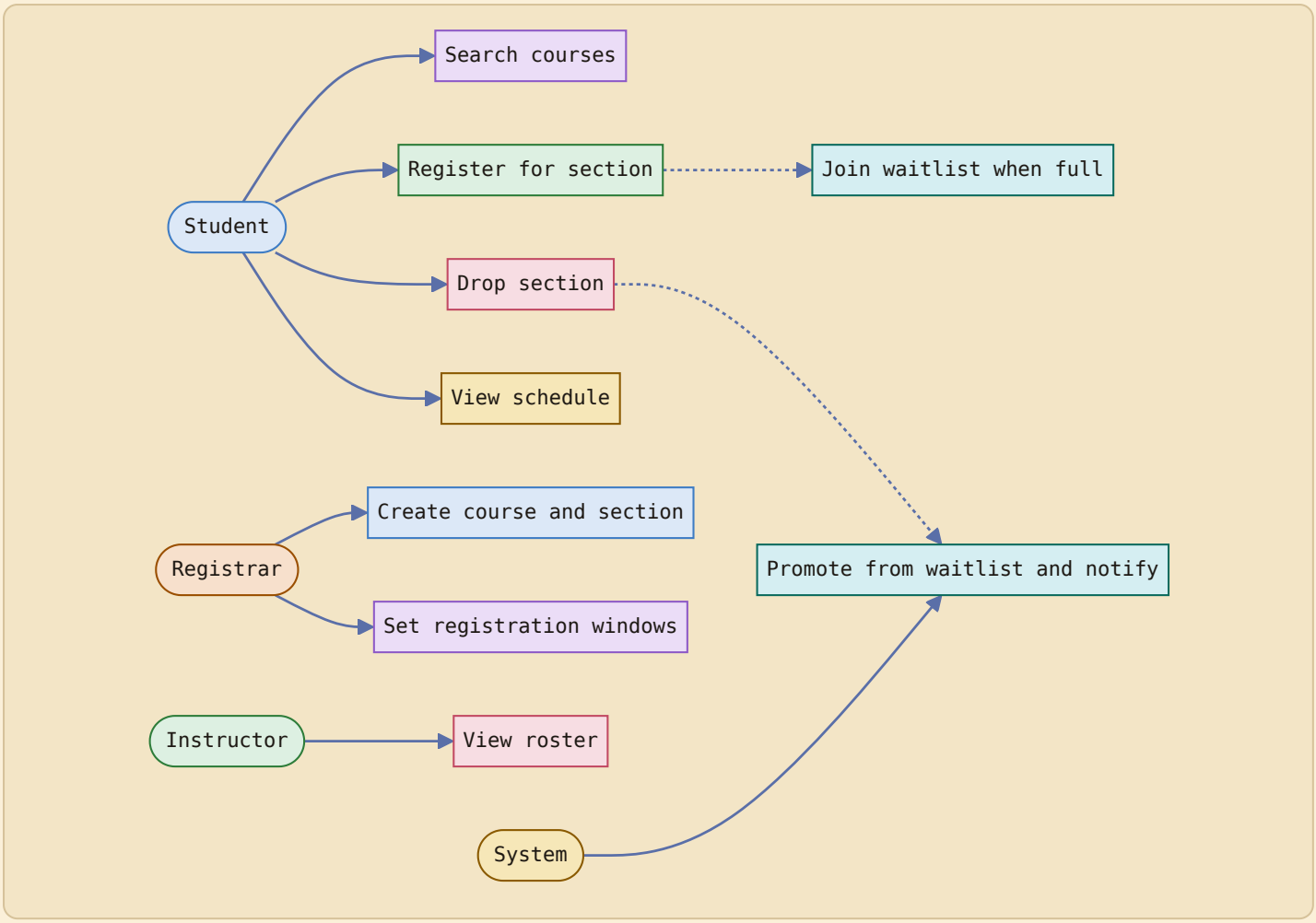
2. Each course has a code, name and credits; each section has an instructor, time slot and enrolment limit.
3. Search courses by code or name.
4. Never enrol beyond a section's capacity; when full, put the student on the waitlist (if it has room).
5. Enforce prerequisites, credit limit, time conflicts, one section per course, and the registration window.
6. On drop, promote the first student on the waitlist automatically and notify them.

Non-functional

- Handle concurrent registrations safely: no race conditions, consistent data.
- Rules must be easy to add or remove (for example "department majors only").
- Extensible for future features (fees, holds, cross-listed courses).

Use cases

- **Student** — searches courses, registers, drops, views schedule, joins a waitlist.
- **Registrar / Admin** — creates courses and sections, sets capacity and windows.
- **Instructor** — views the roster.
- **System** — enforces rules, promotes from the waitlist, sends notifications.

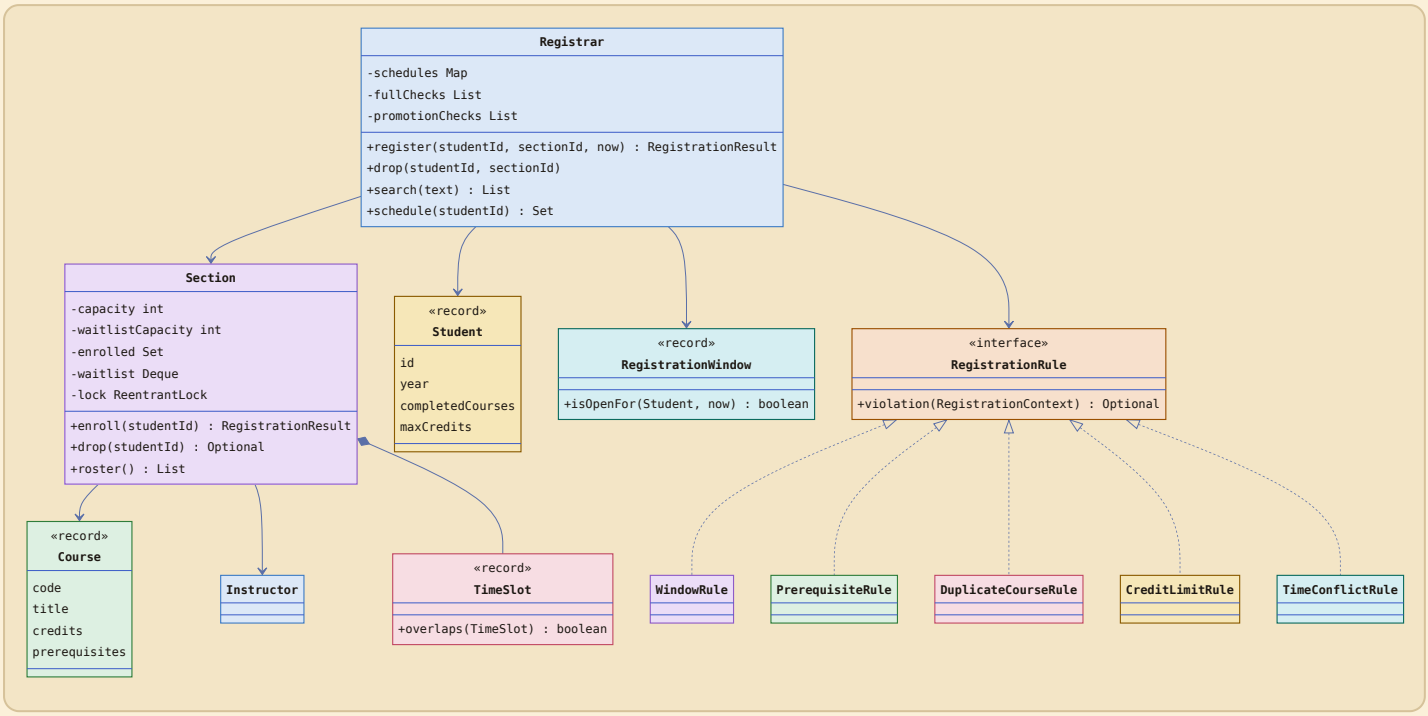


Core entities

Class	Responsibility
`Course`	Code, title, credits, prerequisite codes.
`Section`	One offering: instructor, time slot, capacity, enrolled set, waitlist. **The only place a seat is taken**, under its own lock.
`TimeSlot`	Days + start/end; knows whether it overlaps another slot.
`Instructor`	Who teaches.
`Student`	Year, completed courses, credit limit.
`RegistrationWindow`	Opening time per class year and a common closing time.
`RegistrationRule` (+ `WindowRule`, `PrerequisiteRule`, `DuplicateCourseRule`, `CreditLimitRule`, `TimeConflictRule`)	One rule each; returns the reason for a violation.
`RegistrationContext`	Everything a rule may look at.
`RegistrationResult`, `Outcome`	`ENROLLED`, `WAITLISTED` or `REJECTED` + message.
`Registrar`	The service: runs the rule chain, takes the seat, drops and promotes, keeps each student's schedule, and defines the lock order.

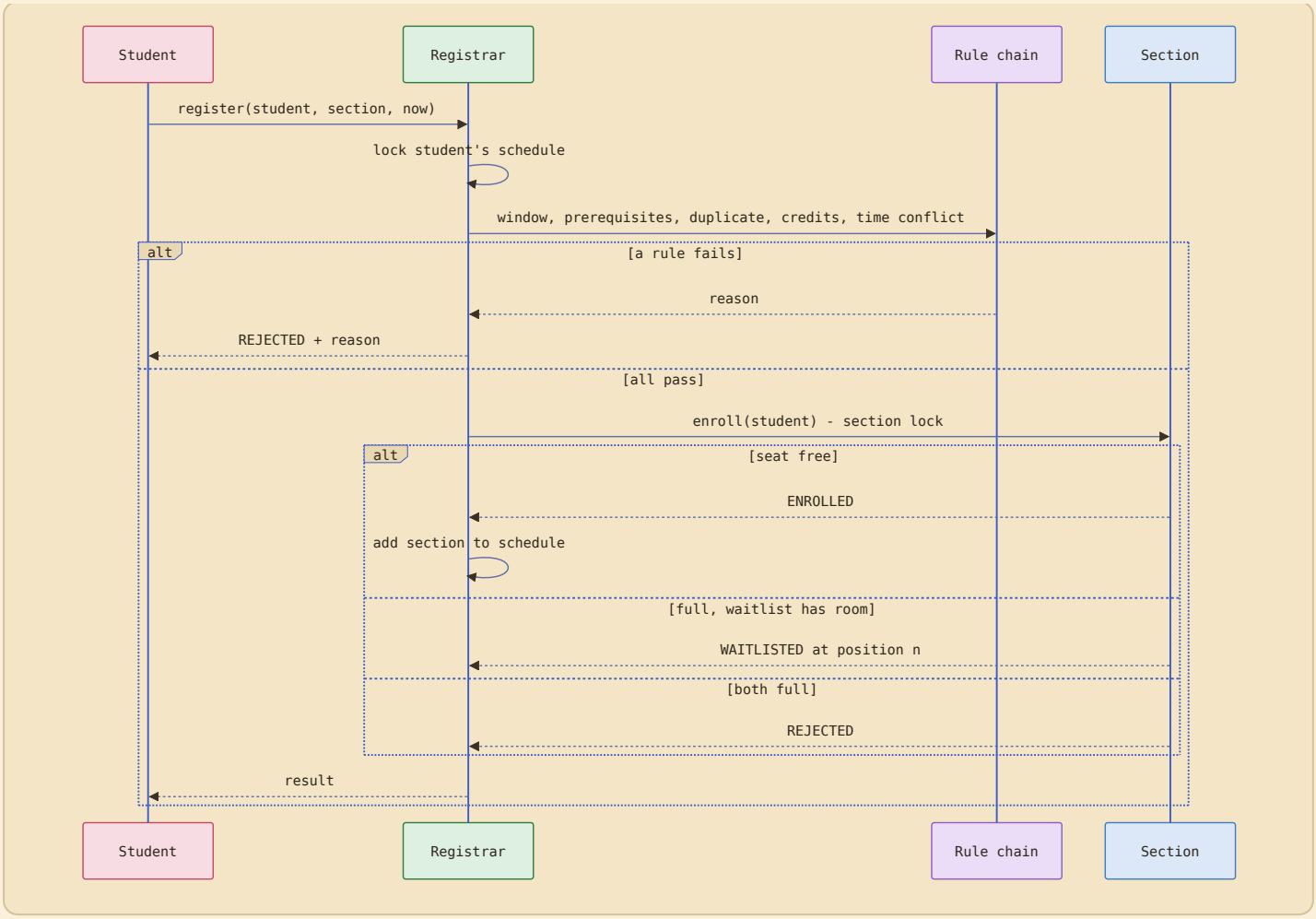
Class diagram

There is no course lesson for this problem. The problem statement comes from the *awesome-low-level-design* problem set: students register for courses and view their enrolments; a course has a code, name, instructor and maximum capacity; search by code or name; no enrolment past capacity; many students registering at once without race conditions; room to extend. Every one of those is covered above (the instructor and capacity sit on the `Section`). Here is the diagram of our Java design.

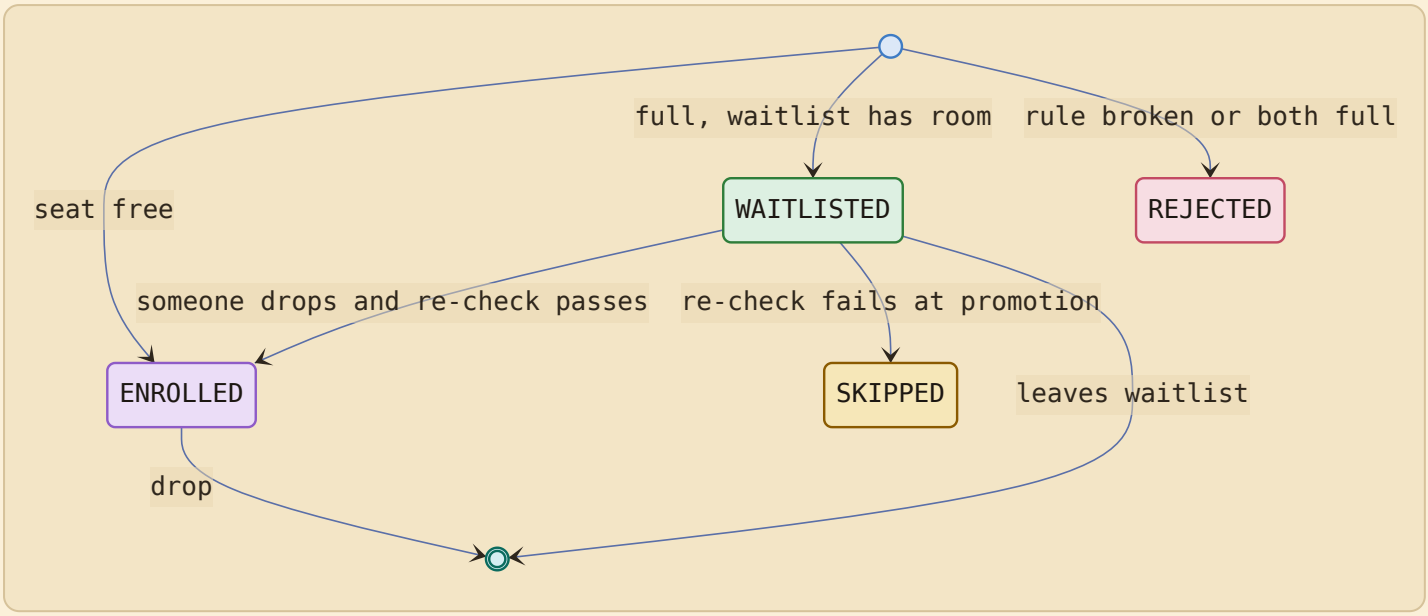


Key flows

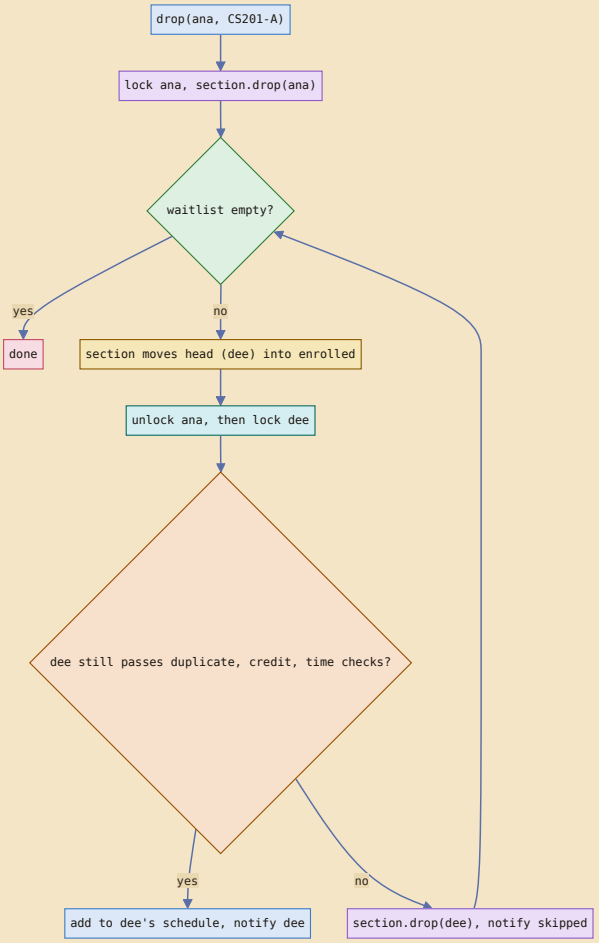
Register:



A student's status in one section:



Drop and promote:


drop(ana, CS201-A)
lock ana, section.drop(ana)
waitlist empty?
yes
no
done
section moves head (dee) into enrolled
unlock ana, then lock dee
dee still passes duplicate, credit, time checks?
yes
no
add to dee's schedule, notify dee
section.drop(dee), notify skipped

Design decisions & patterns used

Decision	Why	Alternative rejected
Course vs Section	Capacity, time and instructor belong to an offering, not the catalogue entry.	Capacity on `Course` (cannot have two sections).
Seat taken only inside `Section.enroll`, under a `ReentrantLock` (Java's explicit lock object, which the same thread may re-acquire) per section	Check-capacity-then-add is atomic; different sections do not block each other.	`synchronized` on the whole registrar (every student in the university waits in one line).
Per-student lock around rules + enroll	Credit limit and time-conflict checks read the schedule; two tabs registering the same student would both pass without it (step 6 of the output).	Lock only the section (a student could exceed credits via two sections in parallel).
Fixed lock order: student, then section	Deadlock needs two threads taking the same two locks in opposite orders. Always taking student before section rules it out. Promotion takes the next student's lock only *after* releasing the dropper's, so two student locks are never held together.	Promoting while still holding the dropper's lock (student → section → student: deadlock risk).
Rules as objects (Specification / Chain of Responsibility style: each check is its own object, and a request passes along the chain until one rejects it)	Each rule is one small class; the registrar runs them in order and stops at the first failure. A new rule ("majors only") is a new class plus one list entry — Open/Closed. The promotion path reuses a *subset* of the same rules.	One long `register` method with nested `if`s.
Waitlist is a FIFO `Deque`, bounded (a double-ended queue with a maximum size)	Fair (first come, first served) and prevents unbounded queues.	Unbounded list; priority by year (a valid extension).
Re-check on promotion	A student may have registered for a clashing section while waiting.	Promote blindly (broken timetables).
Time passed in (`now`)	Makes window logic testable without waiting for real dates.	Calling `LocalDateTime.now()` inside rules.
Singleton? The GitHub problem uses a singleton `CourseRegistrationSystem`.	We construct one `Registrar` and pass it in: one instance, still testable.	Global `getInstance()`.

Java implementation

Package `lld.ch16.courseregistration`.

```java
// File: code/lld/ch16/courseregistration/Types.java
package lld.ch16.courseregistration;

import java.time.DayOfWeek;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.EnumSet;
import java.util.Map;
import java.util.Set;

/** A catalog entry: what is taught. Prerequisites are course codes. */
record Course(String code, String title, int credits, Set<String> prerequisites) {
    Course { prerequisites = Set.copyOf(prerequisites); }
}

record Instructor(String id, String name) {}

/** When a section meets, e.g. Mon+Wed 10:00-11:30. */
record TimeSlot(Set<DayOfWeek> days, LocalTime start, LocalTime end) {
    TimeSlot {
        days = EnumSet.copyOf(days);
        if (!end.isAfter(start)) throw new IllegalArgumentException("end must be after start");
    }
    boolean overlaps(TimeSlot o) {
        boolean sharedDay = days.stream().anyMatch(o.days::contains);
        return sharedDay && start.isBefore(o.end) && o.start.isBefore(end);
    }
    @Override public String toString() { return days + " " + start + "-" + end; }
}

record Student(String id, String name, int year, Set<String> completedCourses, int maxCredits) {
    Student { completedCourses = Set.copyOf(completedCourses); }
}

/** Priority registration: each class year opens at its own time; everyone closes together. */
record RegistrationWindow(Map<Integer, LocalDateTime> opensByYear, LocalDateTime closes) {
    boolean isOpenFor(Student s, LocalDateTime now) {
        var opens = opensByYear.get(s.year());
        return opens != null && !now.isBefore(opens) && now.isBefore(closes);
    }
}

enum Outcome { ENROLLED, WAITLISTED, REJECTED }

record RegistrationResult(Outcome outcome, String message) {
    static RegistrationResult rejected(String why) { return new RegistrationResult(Outcome.REJECTED, why); }
}
```

```java
// File: code/lld/ch16/courseregistration/Section.java
package lld.ch16.courseregistration;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.locks.ReentrantLock;

/**
 * One offering of a course (CS201 section A). Capacity and waitlist live HERE and are
 * guarded by the section's own lock, so thousands of students can hit different sections in parallel.
 */
final class Section {
    private final String id;
    private final Course course;
    private final Instructor instructor;
    private final TimeSlot slot;
    private final int capacity;
    private final int waitlistCapacity;
    private final Set<String> enrolled = new LinkedHashSet<>();
    private final Deque<String> waitlist = new ArrayDeque<>();
    private final ReentrantLock lock = new ReentrantLock();

    Section(String id, Course course, Instructor instructor, TimeSlot slot, int capacity, int waitlistCapacity) {
        this.id = id; this.course = course; this.instructor = instructor; this.slot = slot;
        this.capacity = capacity; this.waitlistCapacity = waitlistCapacity;
    }

    String id() { return id; }
    Course course() { return course; }
    Instructor instructor() { return instructor; }
    TimeSlot slot() { return slot; }

    /** The only place a seat is taken: check-and-add in one critical section. */
    RegistrationResult enroll(String studentId) {
        lock.lock();
        try {
            if (enrolled.contains(studentId)) return RegistrationResult.rejected("already enrolled in " + id);
            if (waitlist.contains(studentId)) return RegistrationResult.rejected("already on the waitlist of " +
id);
            if (enrolled.size() < capacity) {
                enrolled.add(studentId);
                return new RegistrationResult(Outcome.ENROLLED, "enrolled in " + id + " (" + enrolled.size() + "/"
+ capacity + ")");
            }
            if (waitlist.size() < waitlistCapacity) {
                waitlist.addLast(studentId);
                return new RegistrationResult(Outcome.WAITLISTED, "waitlisted for " + id + " at position " +
waitlist.size());
            }
            return RegistrationResult.rejected(id + " and its waitlist are full");
        } finally {
            lock.unlock();
        }
    }

    /** Removes the student; if a seat opened, moves the head of the waitlist in and returns who it was. */
    Optional<String> drop(String studentId) {
        lock.lock();
        try {
            if (waitlist.remove(studentId)) return Optional.empty();
            if (!enrolled.remove(studentId)) throw new IllegalArgumentException(studentId + " is not in " + id);
            String next = waitlist.pollFirst();
            if (next != null) enrolled.add(next);
            return Optional.ofNullable(next);
        } finally {
            lock.unlock();
```

```
        }
    }

    boolean isEnrolled(String studentId) {
        lock.lock();
        try { return enrolled.contains(studentId); } finally { lock.unlock(); }
    }

    List<String> roster() {
        lock.lock();
        try { return List.copyOf(enrolled); } finally { lock.unlock(); }
    }

    List<String> waitlist() {
        lock.lock();
        try { return List.copyOf(waitlist); } finally { lock.unlock(); }
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Rules.java
package lld.ch16.courseregistration;

import java.time.LocalDateTime;
import java.util.Optional;
import java.util.Set;

/** What a rule may look at: who, which section, what they already have, and when. */
record RegistrationContext(Student student, Section section, Set<Section> currentSchedule, LocalDateTime now,
                           RegistrationWindow window) {}

/**
 * Specification-style rule: returns a reason when the request breaks it.
 * The registrar runs the rules as a chain and stops at the first violation.
 */
interface RegistrationRule { Optional<String> violation(RegistrationContext c); }

final class WindowRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.window().isOpenFor(c.student(), c.now()) ? Optional.empty()
                : Optional.of("registration window is closed for year " + c.student().year() + " at " + c.now());
    }
}

final class PrerequisiteRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        var missing = c.section().course().prerequisites().stream()
                .filter(p -> !c.student().completedCourses().contains(p)).sorted().toList();
        return missing.isEmpty() ? Optional.empty() : Optional.of("missing prerequisites " + missing);
    }
}

final class DuplicateCourseRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.currentSchedule().stream()
                .filter(s -> s.course().code().equals(c.section().course().code())).findFirst()
                .map(s -> "already registered for " + s.course().code() + " in section " + s.id());
    }
}

final class CreditLimitRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        int current = c.currentSchedule().stream().mapToInt(s -> s.course().credits()).sum();
        int after = current + c.section().course().credits();
        return after <= c.student().maxCredits() ? Optional.empty()
                : Optional.of("credit limit " + c.student().maxCredits() + " exceeded (" + after + ")");
    }
}

final class TimeConflictRule implements RegistrationRule {
    public Optional<String> violation(RegistrationContext c) {
        return c.currentSchedule().stream().filter(s -> s.slot().overlaps(c.section().slot())).findFirst()
                .map(s -> "time conflict with " + s.id() + " " + s.slot());
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Registrar.java
package lld.ch16.courseregistration;

import java.time.LocalDateTime;
import java.util.List;
import java.util.Map;
import java.util.Optional;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.function.Consumer;

/**
 * The service. Lock order is ALWAYS student-then-section, which rules out deadlock:
 *  - the student lock makes "check my credits/time clashes, then take the seat" atomic per student
 *    (stops one student double-registering from two browser tabs);
 *  - the section lock (inside Section) makes the seat count atomic across students.
 */
final class Registrar {
    private final Map<String, Student> students = new ConcurrentHashMap<>();
    private final Map<String, Section> sections = new ConcurrentHashMap<>();
    private final Map<String, Set<Section>> schedules = new ConcurrentHashMap<>();
    private final RegistrationWindow window;
    private final List<RegistrationRule> fullChecks;
    private final List<RegistrationRule> promotionChecks;
    private final Consumer<String> notifier;

    Registrar(RegistrationWindow window, Consumer<String> notifier) {
        this.window = window;
        this.notifier = notifier;
        this.fullChecks = List.of(new WindowRule(), new PrerequisiteRule(), new DuplicateCourseRule(),
                new CreditLimitRule(), new TimeConflictRule());
        this.promotionChecks = List.of(new DuplicateCourseRule(), new CreditLimitRule(), new TimeConflictRule());
    }

    void addStudent(Student s) { students.put(s.id(), s); schedules.put(s.id(), ConcurrentHashMap.newKeySet()); }
    void addSection(Section s) { sections.put(s.id(), s); }
    Section section(String id) { return sections.get(id); }

    List<Section> search(String text) {
        var q = text.toLowerCase();
        return sections.values().stream().filter(s -> s.course().code().toLowerCase().contains(q)
                || s.course().title().toLowerCase().contains(q)).sorted((a, b) ->
a.id().compareTo(b.id())).toList();
    }

    Set<Section> schedule(String studentId) { return Set.copyOf(schedules.get(studentId)); }

    private Optional<String> firstViolation(List<RegistrationRule> rules, RegistrationContext ctx) {
        for (var r : rules) {
            var v = r.violation(ctx);
            if (v.isPresent()) return v;
        }
        return Optional.empty();
    }

    RegistrationResult register(String studentId, String sectionId, LocalDateTime now) {
        var student = students.get(studentId);
        var section = sections.get(sectionId);
        if (student == null || section == null) return RegistrationResult.rejected("unknown student or section");
        var schedule = schedules.get(studentId);
        synchronized (schedule) {                                   // 1st lock: the student
            var ctx = new RegistrationContext(student, section, Set.copyOf(schedule), now, window);
            var v = firstViolation(fullChecks, ctx);
            if (v.isPresent()) return RegistrationResult.rejected(v.get());
            var result = section.enroll(studentId);                 // 2nd lock: the section
            if (result.outcome() == Outcome.ENROLLED) schedule.add(section);
            return result;
        }
    }
```

```
    void drop(String studentId, String sectionId) {
        var section = sections.get(sectionId);
        var schedule = schedules.get(studentId);
        Optional<String> promoted;
        synchronized (schedule) {
            promoted = section.drop(studentId);
            schedule.remove(section);
        }
        notifier.accept(studentId + " dropped " + sectionId);
        // Finish promotions OUTSIDE the dropper's lock, one student lock at a time (no nested student locks).
        while (promoted.isPresent()) promoted = finishPromotion(promoted.get(), section);
    }

    /** The promoted student's timetable may have changed while waiting, so re-check. Returns the next promotion,
if any. */
    private Optional<String> finishPromotion(String studentId, Section section) {
        var schedule = schedules.get(studentId);
        synchronized (schedule) {
            var ctx = new RegistrationContext(students.get(studentId), section, Set.copyOf(schedule), null,
window);
            var v = firstViolation(promotionChecks, ctx);
            if (v.isEmpty()) {
                schedule.add(section);
                notifier.accept(studentId + " promoted from waitlist into " + section.id());
                return Optional.empty();
            }
            notifier.accept(studentId + " skipped for " + section.id() + ": " + v.get());
            return section.drop(studentId);
        }
    }
}
```

```java
// File: code/lld/ch16/courseregistration/Main.java
package lld.ch16.courseregistration;

import java.time.DayOfWeek;
import java.time.LocalDateTime;
import java.time.LocalTime;
import java.util.EnumSet;
import java.util.Map;
import java.util.Set;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.CountDownLatch;
import java.util.concurrent.Executors;

public class Main {
    public static void main(String[] args) throws Exception {
        var window = new RegistrationWindow(Map.of(
                4, LocalDateTime.of(2026, 11, 2, 9, 0),     // seniors first
                3, LocalDateTime.of(2026, 11, 3, 9, 0),
                2, LocalDateTime.of(2026, 11, 4, 9, 0),
                1, LocalDateTime.of(2026, 11, 5, 9, 0)),
                LocalDateTime.of(2026, 11, 20, 17, 0));
        var reg = new Registrar(window, msg -> System.out.println("    [notify] " + msg));

        var cs101 = new Course("CS101", "Intro to Programming", 4, Set.of());
        var cs201 = new Course("CS201", "Data Structures", 4, Set.of("CS101"));
        var math201 = new Course("MATH201", "Discrete Math", 3, Set.of());
        var cs301 = new Course("CS301", "Algorithms", 4, Set.of("CS201", "MATH201"));
        var drLee = new Instructor("I1", "Dr. Lee");
        var mw = EnumSet.of(DayOfWeek.MONDAY, DayOfWeek.WEDNESDAY);
        var tt = EnumSet.of(DayOfWeek.TUESDAY, DayOfWeek.THURSDAY);
        reg.addSection(new Section("CS201-A", cs201, drLee, new TimeSlot(mw, LocalTime.of(10, 0), LocalTime.of(11,
30)), 2, 2));
        reg.addSection(new Section("CS201-B", cs201, drLee, new TimeSlot(tt, LocalTime.of(14, 0), LocalTime.of(15,
30)), 30, 5));
        reg.addSection(new Section("MATH201-A", math201, new Instructor("I2", "Dr. Rao"), new TimeSlot(mw,
LocalTime.of(11, 0), LocalTime.of(12, 0)), 30, 5));
        reg.addSection(new Section("CS301-A", cs301, drLee, new TimeSlot(tt, LocalTime.of(9, 0), LocalTime.of(10,
30)), 30, 5));
        reg.addSection(new Section("CS101-A", cs101, new Instructor("I3", "Dr. Kim"), new TimeSlot(tt,
LocalTime.of(11, 0), LocalTime.of(12, 0)), 30, 10));

        reg.addStudent(new Student("ana", "Ana", 4, Set.of("CS101"), 12));
        reg.addStudent(new Student("ben", "Ben", 3, Set.of("CS101"), 4));
        reg.addStudent(new Student("cy", "Cy", 2, Set.of(), 16));
        reg.addStudent(new Student("dee", "Dee", 3, Set.of("CS101"), 12));
        reg.addStudent(new Student("eve", "Eve", 3, Set.of("CS101"), 12));
        reg.addStudent(new Student("fay", "Fay", 3, Set.of("CS101"), 12));

        var nov2 = LocalDateTime.of(2026, 11, 2, 10, 0);
        var nov3 = LocalDateTime.of(2026, 11, 3, 10, 0);
        System.out.println("1) Search 'cs2': " + reg.search("cs2").stream().map(s -> s.id() + " " +
s.slot()).toList());

        System.out.println("2) Rules");
        show("ben  CS201-A on Nov 2 ", reg.register("ben", "CS201-A", nov2));
        show("ana  CS201-A on Nov 2 ", reg.register("ana", "CS201-A", nov2));
        show("ana  MATH201-A        ", reg.register("ana", "MATH201-A", nov2));
        show("ana  CS201-B          ", reg.register("ana", "CS201-B", nov2));
        show("ana  CS301-A          ", reg.register("ana", "CS301-A", nov2));
        show("cy   CS201-B on Nov 4 ", reg.register("cy", "CS201-B", LocalDateTime.of(2026, 11, 4, 9, 30)));
        show("ana  after close      ", reg.register("ana", "CS201-B", LocalDateTime.of(2026, 11, 21, 9, 0)));

        System.out.println("3) Capacity 2 + waitlist 2 on CS201-A (juniors open Nov 3)");
        show("ben  CS201-A          ", reg.register("ben", "CS201-A", nov3));
        show("dee  CS201-A          ", reg.register("dee", "CS201-A", nov3));
        show("eve  CS201-A          ", reg.register("eve", "CS201-A", nov3));
        show("fay  CS201-A          ", reg.register("fay", "CS201-A", nov3));
        show("ben  MATH201-A        ", reg.register("ben", "MATH201-A", nov3));

        System.out.println("4) Ana drops CS201-A: the head of the waitlist is promoted");
```

```
        reg.drop("ana", "CS201-A");
        var a = reg.section("CS201-A");
        System.out.println("   roster " + a.roster() + ", waitlist " + a.waitlist());

        System.out.println("5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp");
        var opening = LocalDateTime.of(2026, 11, 5, 9, 0);
        for (int i = 0; i < 50; i++) reg.addStudent(new Student("fy" + i, "Fresher" + i, 1, Set.of(), 16));
        var counts = new ConcurrentHashMap<Outcome, Integer>();
        var go = new CountDownLatch(1);
        try (var pool = Executors.newFixedThreadPool(16)) {
            for (int i = 0; i < 50; i++) {
                String id = "fy" + i;
                pool.submit(() -> {
                    go.await();
                    counts.merge(reg.register(id, "CS101-A", opening).outcome(), 1, Integer::sum);
                    return null;
                });
            }
            go.countDown();
        }
        System.out.println("   ENROLLED=" + counts.get(Outcome.ENROLLED) + " WAITLISTED=" +
counts.get(Outcome.WAITLISTED)
                + " REJECTED=" + counts.get(Outcome.REJECTED) + ", roster size " + reg.section("CS101-
A").roster().size());

        System.out.println("6) One student double-clicks: two threads, same request");
        reg.addStudent(new Student("gus", "Gus", 3, Set.of("CS101"), 12));
        var outcomes = new ConcurrentHashMap<Outcome, Integer>();
        try (var pool = Executors.newFixedThreadPool(2)) {
            for (int i = 0; i < 2; i++)
                pool.submit(() -> outcomes.merge(reg.register("gus", "CS201-B", nov3).outcome(), 1,
Integer::sum));
        }
        System.out.println("   " + new java.util.TreeMap<>(outcomes) + ", gus schedule " +
reg.schedule("gus").stream().map(Section::id).toList());
    }

    static void show(String label, RegistrationResult r) {
        System.out.println("   " + label + " -> " + r.outcome() + ": " + r.message());
    }
}
```

Output:

```
1) Search 'cs2': [CS201-A [MONDAY, WEDNESDAY] 10:00-11:30, CS201-B [TUESDAY, THURSDAY] 14:00-15:30]
2) Rules
   ben  CS201-A on Nov 2  -> REJECTED: registration window is closed for year 3 at 2026-11-02T10:00
   ana  CS201-A on Nov 2  -> ENROLLED: enrolled in CS201-A (1/2)
   ana  MATH201-A         -> REJECTED: time conflict with CS201-A [MONDAY, WEDNESDAY] 10:00-11:30
   ana  CS201-B           -> REJECTED: already registered for CS201 in section CS201-A
   ana  CS301-A           -> REJECTED: missing prerequisites [CS201, MATH201]
   cy   CS201-B on Nov 4  -> REJECTED: missing prerequisites [CS101]
   ana  after close       -> REJECTED: registration window is closed for year 4 at 2026-11-21T09:00
3) Capacity 2 + waitlist 2 on CS201-A (juniors open Nov 3)
   ben  CS201-A           -> ENROLLED: enrolled in CS201-A (2/2)
   dee  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 1
   eve  CS201-A           -> WAITLISTED: waitlisted for CS201-A at position 2
   fay  CS201-A           -> REJECTED: CS201-A and its waitlist are full
   ben  MATH201-A         -> REJECTED: credit limit 4 exceeded (7)
4) Ana drops CS201-A: the head of the waitlist is promoted
    [notify] ana dropped CS201-A
    [notify] dee promoted from waitlist into CS201-A
   roster [ben, dee], waitlist [eve]
5) 50 first-years grab 30 seats (+10 waitlist) in CS101-A at 09:00 sharp
   ENROLLED=30 WAITLISTED=10 REJECTED=10, roster size 30
6) One student double-clicks: two threads, same request
   {ENROLLED=1, REJECTED=1}, gus schedule [CS201-B]
```

Step 2 shows each rule firing with a clear reason. Step 3 fills two seats and two waitlist places. Step 4 promotes Dee automatically. Step 5 is the "09:00 rush": 50 threads, 30 seats, 10 waitlist places —

the counts are exact every run. Step 6 is the double-click: two threads for the same student and section, exactly one succeeds.

Concurrency & edge cases

- **Overselling a section** — prevented by the section lock around "size < capacity → add".
- **Double-click / two tabs** — prevented by the per-student lock; the second request sees the first enrolment and is rejected by `DuplicateCourseRule` (or by the section's "already enrolled" check).
- **Deadlock** — avoided by the fixed lock order (student → section) and by never holding two student locks at once.
- **Waitlisted in one section, enrolled in another section of the same course** — possible while waiting, because waitlist entries are not in the schedule; the promotion re-check (`DuplicateCourseRule`) catches it and skips to the next student.
- **Registration closes while on the waitlist** — promotion does not re-check the window (it is the drop that triggers it). Many universities stop automatic promotion after a deadline; add `WindowRule` to `promotionChecks` if that is the policy.
- **Dropping a section you are not in** → `IllegalArgumentException`.
- **Rules with time** — the window uses half-open time (`opens ≤ now < closes`).
- **Scaling beyond one JVM** (Java Virtual Machine — the running program that executes Java code) — the in-memory locks work for one process (one running copy of the program). With several app servers (copies of the application on different machines, usually behind a load balancer that spreads requests among them), the seat count moves to the database: `UPDATE section SET enrolled = enrolled + 1 WHERE id = ? AND enrolled < capacity` succeeds for exactly `capacity` requests (an atomic conditional update), or use optimistic locking with a version column. A queue in front of hot sections smooths the 09:00 spike. (Load balancers, queues and databases at scale are covered in System Design Fundamentals.)

Extensions & follow-up questions

- **"Co-requisites or 'prerequisite OR instructor permission'?"** — Model prerequisites as a small expression tree (a tree whose inner nodes are operators like `And` / `Or` and whose leaves are simple checks: `And`, `Or`, `CourseCompleted`, `Permission`) — the Composite pattern, where a group of objects is treated exactly like a single one — and evaluate it in `PrerequisiteRule`.
- **"Holds (unpaid fees, advising)?"** — A `HoldRule` that rejects if the student has an active hold.
- **"Priority waitlists (seniors first)?"** — Replace the `Deque` with a `PriorityQueue` (a queue that always hands out the smallest item by some ordering, not the oldest) ordered by (year, time joined).
- **"Seat reservations for majors?"** — Split capacity into buckets (majors 20, others 10) and check the bucket in `Section.enroll`.
- **"Linked sections (lecture + lab)?"** — Register both atomically: lock both sections in a fixed order (by id), check both, enrol both.
- **"Notify by email/SMS?"** — The registrar takes a `Consumer<String>` notifier (Java's built-in "function that takes a value and returns nothing"); swap for an email service or publish an event to a message queue (a service such as Kafka or RabbitMQ that stores messages until another program reads them).
- **"Course fees and billing?"** — Add a `Bill` per student per term, a line per enrolled section, recomputed on drop.

In the interview

30-second pitch: "Students register for sections, not courses. A section owns capacity and a bounded FIFO waitlist, and takes seats only inside its own lock. The registrar runs a chain of rule objects — window, prerequisites, duplicate course, credit limit, time conflict — while holding the student's lock, then asks the section for a seat. Lock order is always student then section, so no deadlocks. On drop the head of the waitlist is promoted after a re-check."

Minutes	Do
0–5	Clarify course vs section, rules, waitlist, windows, concurrency.
5–12	Entities and class diagram.
12–30	Code `Section.enroll/drop`, the rule interface and two rules, `Registrar.register`.
30–40	Concurrency: run the 50-threads demo in your head; lock order; DB conditional update for scale.

Self-check

1. **Q:** Why do students register for sections, not courses? **A:** Capacity, time and instructor belong to each offering; a course can have many sections.
2. **Q:** Which two locks does `register` take and in what order? **A:** The student's schedule lock, then the section's lock — always in that order.
3. **Q:** Why not simply make `register` `synchronized` on the registrar? **A:** It would be correct but serialise (force to run strictly one at a time) every student in the university; per-section and per-student locks let unrelated requests run in parallel.
4. **Q:** Why re-check rules on promotion? **A:** The student's timetable may have changed while waiting (a clash or the same course in another section).
5. **Q:** How do you enforce capacity with many servers? **A:** An atomic conditional update in the database (`… WHERE enrolled < capacity`) or optimistic locking.

TL;DR

- Course (catalogue) vs Section (offering with capacity, slot, waitlist).
- Rules are small objects run as a chain; add a rule without touching the registrar's logic.
- Seats are taken only inside `Section.enroll` under the section's lock; student lock first, section lock second.
- Drop → promote waitlist head → re-check → notify.
- At scale: conditional update in the database.

Real-world example. University student-information systems (for example Ellucian Banner, used by many US universities) run priority registration windows by class standing, check prerequisites and time conflicts, and keep section waitlists that notify a student when a seat opens (some schools enrol the student automatically; others give a time-limited window to claim the seat). Registration-day slowdowns at such schools are the 09:00 "thundering herd" (a huge crowd of requests all arriving at the same instant) this design addresses with per-section locking and, at scale, database conditional updates.

Chapter summary

Problem	Key insight	Atomic claim	Patterns
Car rental	Availability is a question about **dates**, not a status field	`Vehicle.tryBook(range)`	Template Method (add-ons), Strategy + Decorator (pricing), state machine
Restaurant	**Meal per seat**; table = status (now) + slots (later)	`Table.tryHold(time)`	Observer (order ready), Strategy (chef assignment, payment), state machine
Airline	**Flight ≠ FlightInstance, Seat ≠ FlightSeat**	`FlightInstance.holdSeats(map)` (all or nothing)	Strategy + Decorator (fares), Facade, index for cancellation
Course registration	**Course ≠ Section**; rules as objects	`Section.enroll` under a section lock, inside a student lock	Specification / Chain of rules, lock ordering

The four problems share one lesson: **find the calendar, find the atomic check-then-claim, and put the rules on the object that owns the data.**

Chapter 17 — Case Studies: Games & Rules

> **Where this fits.** Chapters 14–16 designed things that model the real world (an ATM, a parking lot, booking systems). Games are different: the "world" is a set of **rules**, and the rules are exact. That makes games a favourite interview topic. There is nowhere to hide a vague class, because every rule must live *somewhere*, and the interviewer will ask where.

An **LLD** (low-level design) interview asks you to design the classes, their responsibilities and some of the code for one system — as opposed to high-level design, which is about servers, databases and networks.

What games test in an LLD interview

Skill	What it means	Where you see it in this chapter
Rule encapsulation (hiding a rule inside the class that owns it)	Each rule lives in one obvious class, not spread across `if` statements everywhere.	Blackjack scoring in `BlackjackScoring`; chess legality in `Rules`; 2048 merging in `LineMerger`.
Turn management	Whose turn is it, what may they do now, and what happens when the turn ends?	A turn index (tic-tac-toe), a queue of players (snake and ladder), `turn` colour (chess), rounds (blackjack).
Polymorphic pieces	Different kinds of objects answer the same question in their own way (*polymorphism* = "many shapes": one method call, many behaviours).	Chess `Piece.targets(...)` overridden by King, Knight, Pawn and the sliding pieces.
Pluggable strategies	Behaviour that varies (how a bot plays, how dice roll, when the game ends) sits behind an interface (a list of methods with no fixed implementation; the **Strategy** pattern, Chapter 10).	`PlayerStrategy`, `DealerPolicy`, `MoveStrategy`, `Dice`, `OverflowRule`, `WinStrategy`, `TileSpawner`.
Deterministic testing (same input, same result every time)	Randomness is *injected* (handed in from outside). Pass `new Random(seed)` and the "random" sequence is the same on every run, so tests and demos repeat exactly. Or pass a scripted source (stacked shoe, loaded dice).	Every `Main` in this chapter prints the same output every time.
State and status	A game moves through states (in progress, won, draw, over). Illegal actions in the wrong state are rejected.	`GameStatus`, `GameState` enums (types with a fixed list of named values) in every game.

> **Key idea: seeded randomness.** `java.util.Random` is a *pseudo-random* generator: it computes its numbers from a starting value called the **seed**. The same seed always produces the same sequence (the algorithm is fixed by the Java specification). `Collections.shuffle(list, rnd)` is also deterministic for a given `Random`. So the rule for game code is: **never call `new Random()` or `Math.random()` inside the game. Accept a `Random` (or a `Dice` / `Spawner` interface) from outside.** Production passes an unseeded (or `SecureRandom` — a slower generator whose output cannot be predicted, used for security) source; tests pass a seed. This is plain Dependency Injection (Chapter 5): an object receives what it needs from its caller instead of creating it itself.

Problems in this chapter

#	Problem	Main ideas
17.1	Blackjack and a Deck of Cards	generic deck, shoe, soft/hard hand values, dealer rule, betting, split, double, insurance
17.2	Chess	board, polymorphic pieces, legal moves, check/checkmate/stalemate, castling, en passant, promotion, undo
17.3	Tic-Tac-Toe	N×N board, O(1) win detection with counters, pluggable players
17.4	Snake and Ladder	board config and validation, seeded dice, rules engine, pluggable win rules
17.5	2048	slide/merge logic, seeded tile spawns, game-over detection

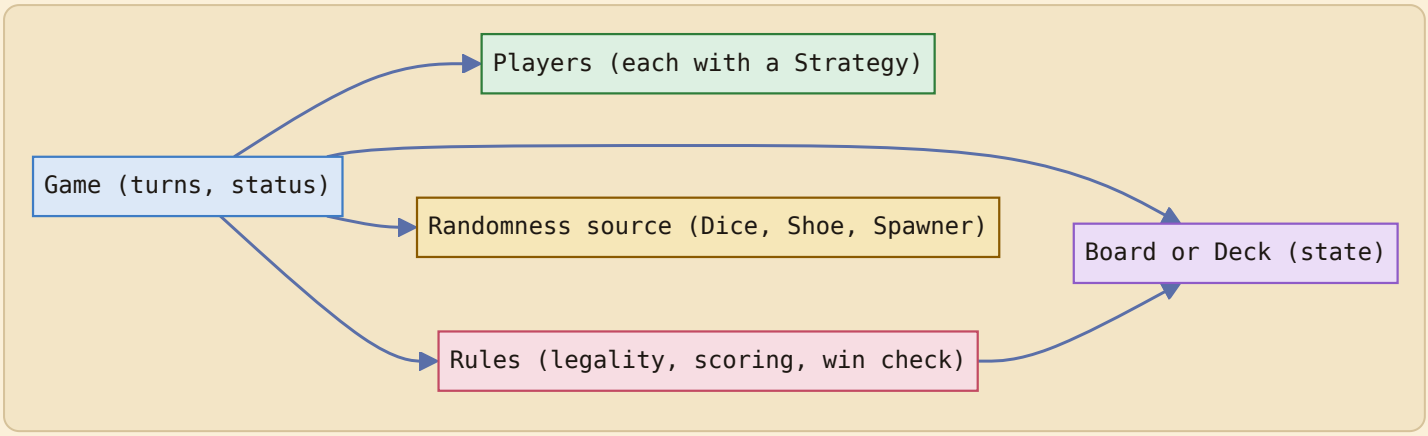
> **Key term — Big-O notation:** Expressions like O(1), O(n) and O(N²) describe how an algorithm's work grows as the input grows. **O(1)** ("constant time") means the work stays the same no matter how big the input is; **O(n)** means it grows in step with the input size n; **O(N²)** means doubling N makes the work about four times bigger. Interviewers use it to compare approaches without timing them.

How to run every example in this chapter

```
cd chapters/17-case-games/code
javac --release 21 -d /tmp/lld_17 $(find . -name '*.java')
java -cp /tmp/lld_17 lld.ch17.blackjack.Main    # or chess, tictactoe, snakeladder, game2048
```

`javac` is the Java compiler and `-cp` (the classpath) tells `java` where the compiled classes are. All five `Main` classes compile with Java 21 (standard library only — just the classes that ship with Java) and run. All randomness is seeded or scripted, so the output shown under each program is exactly what you will see.

A reusable skeleton for any board/card game. Almost every game design in an interview has the same five parts. Name them early and the rest of the interview becomes filling in details.



Part	Question it answers	Typical class names
State	What does the table look like right now?	`Board`, `Deck`, `Shoe`, `Hand`, `Cell`
Actors	Who acts, and how do they decide?	`Player` + `PlayerStrategy` / `MoveStrategy`
Rules	Is this action allowed? What does it do? Who won?	`Rules`, `DealerPolicy`, `OverflowRule`, `WinStrategy`
Controller	Whose turn? What state is the game in?	`Game`, `GameStatus`
Randomness	Where does chance come from?	`Dice`, `Shoe`, `TileSpawner` — always injected

Design Blackjack and a Deck of Cards



Figure — A blackjack table: players each compete against the dealer (course diagram)

Problem & clarifying questions

Blackjack is the casino card game played more than any other. Nobody plays against the other people at the table: every player's cards are measured only against the cards of the **dealer** (the dealer is the casino employee who deals and plays for the house). Everybody tries to get as close to **21 points** as possible without going over. Going over 21 is called **busting** and loses at once.

The interviewer often phrases it as "Design a deck of cards, then use it to build blackjack". The first half checks that you keep the deck **generic**; the second half checks rule placement.

Questions worth asking (with the answers this design assumes):

Question	Assumed answer
One deck or several?	A **shoe** holding several standard 52-card decks (casinos use 6–8). One deck is just a shoe of 1.
How many players?	One or more players, each against the dealer.
Which player options?	Hit, stand, double down, split (once), insurance. Surrender and side bets are out of scope.
Dealer rule?	Hit on 16 or less, stand on 17 or more. Some casinos also hit a "soft 17" — make it pluggable.
Payouts?	Win 1:1, blackjack 3:2, tie ("push") returns the bet, insurance 2:1 (odds like 3:2 mean you win 3 chips for every 2 bet).
Money?	Bets are counted in whole **chips** (an `int`, a whole-number type), and the ante (the starting bet) must be even. Never use `double` (an inexact binary floating-point number) for money.
Real UI (user interface — the screens), network, accounts?	Out of scope. The course model has `Account`/`Person` for sign-in; we keep `Player` minimal and say where it would attach.

Requirements

Background rules

- A standard deck has **13 ranks** (A, 2–10, J, Q, K) in **4 suits** (clubs, diamonds, hearts, spades). Blackjack uses one or more such decks.
- At the start, each player and the dealer receive **two cards**. Both player cards are face up. The dealer has one card face up (the **up card**) and one face down (the **hole card**), so players decide with incomplete information.
- **Card points:** 2–10 are worth their number. J, Q, K are worth 10. An **ace is worth 1 or 11**, whichever helps.
- **Soft vs hard total:** when an ace is counted as 11 the total is **soft** (it can still "soften" to 1 later, so one more card can never bust it). When every ace counts as 1 the total is **hard**. Example: A+5 is "soft 16" (or hard 6).
- **Blackjack** (also called a *natural*): exactly two cards totalling 21, i.e. an ace plus a 10-point card.
- **Dealer rule:** the dealer has no choices. Hit (take a card) while the hand is 16 or less; stand at 17 or more.

Functional requirements

1. A player must place an initial bet (the **ante**) to play a round.
2. Deal two cards to each player and to the dealer (one up, one hole).
3. If the dealer's up card is an **ace**, offer **insurance**: a side bet of half the ante that pays **2:1** if the dealer has blackjack. After that, the dealer checks ("peeks at") the hole card. If it is blackjack, the round ends at once: insurance wins, main bets lose, except a player blackjack, which ties.
4. A player whose hand is below 21 chooses:
    - **Hit** — take one more card, and choose again.
    - **Stand** ("stand pat") — take no more cards.
    - **Double down** — double the bet (the extra must equal the ante), take exactly **one** more card, and stop.
    - **Split** — only when the first two cards have the same rank: pay a second bet equal to the ante, split the pair into two hands, deal one card to each, and play each hand separately. No re-splitting.
5. A hand over 21 is **resolved immediately as a loss**.
6. The dealer then plays by the fixed rule.
7. **Resolution:** if the dealer busts, every non-bust hand wins. Otherwise the higher total wins; equal totals are a **push** (bet returned). A winning blackjack pays **3:2**; every other win pays **1:1**.
8. **Even money** (course requirement): if a player holds blackjack and the dealer shows an ace, the player may accept an immediate 1:1 payout instead of risking a push. (It is mathematically identical to buying insurance with a blackjack, so we explain it rather than build it — see Extensions.)

Non-functional requirements

- **Generic deck:** the `Card` / `Deck` classes must work for any card game, not just blackjack.
- **Deterministic tests:** shuffling uses an injected `Random`; a *stacked* shoe (fixed card order) makes exact scenarios reproducible.
- **Extensible rules:** dealer rule and player decision-making are pluggable.
- **Correct money:** bets and balances are whole chips held in an `int`, never a `double`. This is the one place in the handbook where money is not a `BigDecimal`, and it is deliberate: a table only accepts whole chips, and requiring an **even ante** makes every payout a whole number of chips (3:2 on 10 is 15; insurance is exactly half the ante, 5, and pays 2:1 = 10). If the table ever had to pay fractions (a 3:2 payout on an odd ante, or converting chips to currency at the cashier), switch the amounts to `BigDecimal` (Java's exact decimal number type) with an explicit `RoundingMode` (the rule for what to do with a leftover fraction, such as "round half up").

Use cases

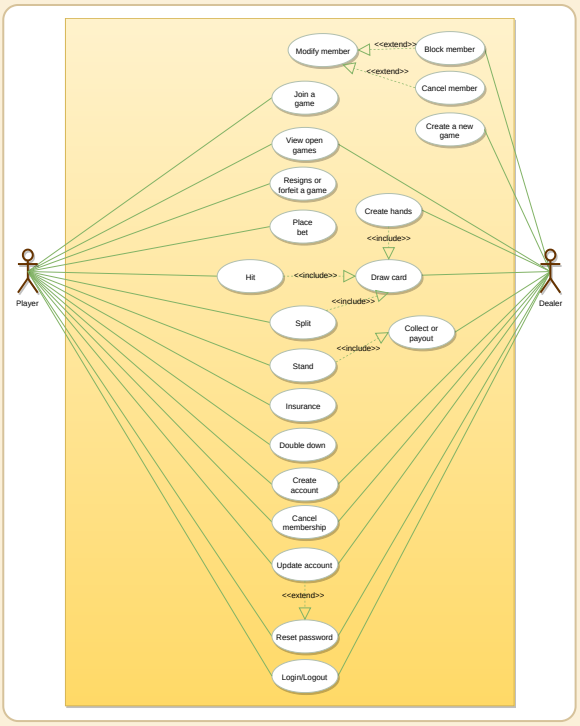


Figure — Blackjack use cases for the Player and the Dealer, including account management (course diagram)

The figure above is a **use case diagram** (UML — Unified Modeling Language — notation: stick figures are *actors*, the people or systems using the software; ovals are *use cases*, the goals they can achieve). In the figure above there are two actors:

- **Player** — places the initial bet; accepts or declines extra bets (insurance, split, double); accepts or declines an offered resolution such as even money; chooses hit, double or stand. The player also has account-level use cases (create account, log in, join or view games), which belong to a lobby service (the separate part of the app where players sign in and pick a table), not the game engine (the code that enforces the game's rules).
- **Dealer** — deals cards, plays the house hand by the fixed rule, collects losing bets and pays winners.

The main use cases (note the `<<include>>` arrows in the figure — an include arrow means one use case always runs another as part of itself: dealing and payout are part of every round):

Use case	What happens
Create hands	Deal two cards each; dealer shows one card.
Place bet	The ante; required before cards are dealt.
Player plays the hand	Hit (repeat), double (one card, done), or stand. A hand over 21 is resolved as a bust.
Dealer plays the hand	Draw while 16 or less, stop at 17 or more. A dealer bust pays every live hand.
Insurance / even money / examine hole card	Only when the up card is an ace. The hole card is checked; a 10-value hole card means dealer blackjack, insurance wins and the round ends; otherwise insurance loses and the hole card stays hidden.
Split	Pair of equal rank: second bet, two hands, one extra card each, no further splitting.
Game resolution	Compare totals; tie returns the bet; blackjack 3:2; other wins 1:1.

Activity: hit or stand

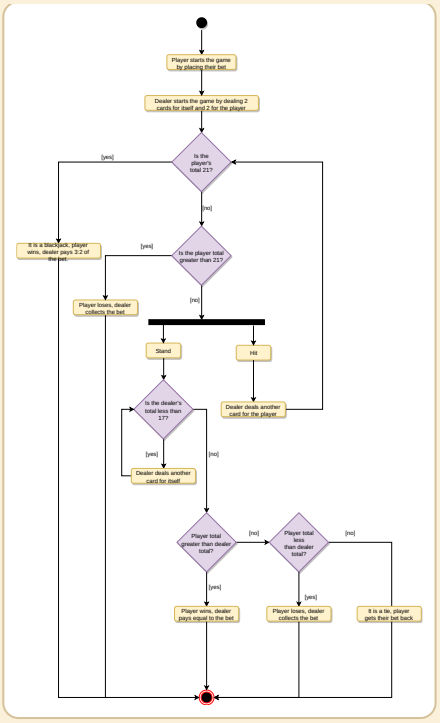


Figure — Activity diagram for one player's hit-or-stand round against the dealer (course diagram)

An **activity diagram** is a flowchart of one task, with diamonds for decisions. The figure above walks through one round: bet, deal, check for 21 (blackjack pays 3:2), check for bust, hit or stand, then the dealer draws while below 17, and finally the three outcomes — player wins 1:1, player loses, or a tie returns the bet.

Core entities

Class	Responsibility
`Suit`, `Rank` (enums)	The fixed values of a French deck (the standard 52-card deck with four suits). `Rank` stores the *face value* 1–13 (ace = 1, J = 11, Q = 12, K = 13).
`Card` (record — a compact class that just holds fixed values)	Rank + suit. Knows **nothing** about blackjack. Immutable (cannot change after creation).
`Deck<T>`	A generic (works for any type `T` you plug in), shuffleable stack of any card type `T`; `standard52()` factory (a method that builds objects) builds a French deck.
`Shoe`	Several decks shuffled together; deals one card at a time; reshuffles at the **cut card** (a marker card placed near the end of the shoe). `Shoe.stacked(...)` gives a fixed order for tests.
`BlackjackScoring`	The blackjack value of a rank (K = 10, ace = 1 hard). The *game* rule, kept off `Card`.
`Hand`	Cards + bet; computes hard/soft/best totals, bust, blackjack, can-split; splits and doubles.
`Action` (enum)	HIT, STAND, DOUBLE, SPLIT.
`PlayerStrategy` (interface)	How a player decides. `BasicStrategy` and `MimicDealerStrategy` are two bots (computer-controlled players); a UI-backed human would be a third.
`DealerPolicy` (interface)	When the dealer hits. `standOnAll17()` and `hitSoft17()`.
`Player`	Name, chip balance, strategy.
The dealer	Not a class of its own here: during a round the dealer is a `Hand` inside `BlackjackGame` plus the `DealerPolicy` that decides when it draws. (The course has a `Dealer` class next to `Player`, both under `BasePlayer`.)
`BlackjackGame`	Runs a round: bets, deal, insurance + peek, player turns, dealer turn, settlement.

Class diagram

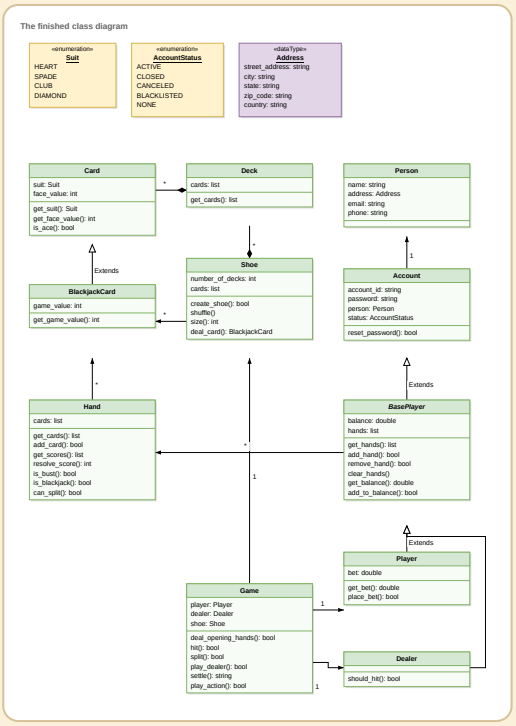
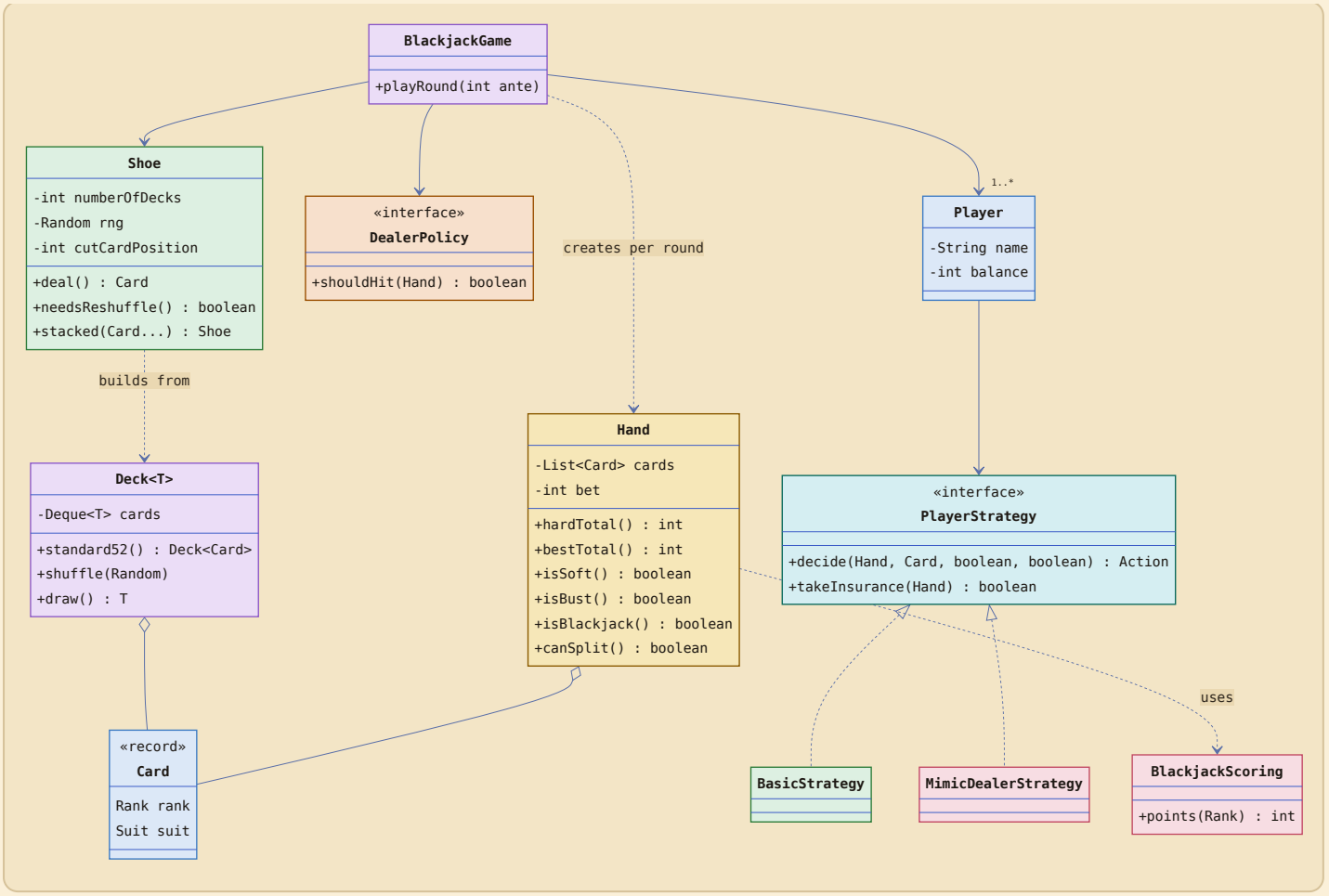


Figure — Course class diagram: Card, BlackjackCard, Deck, Shoe, Hand, BasePlayer, Player, Dealer, Game, Account (course diagram)

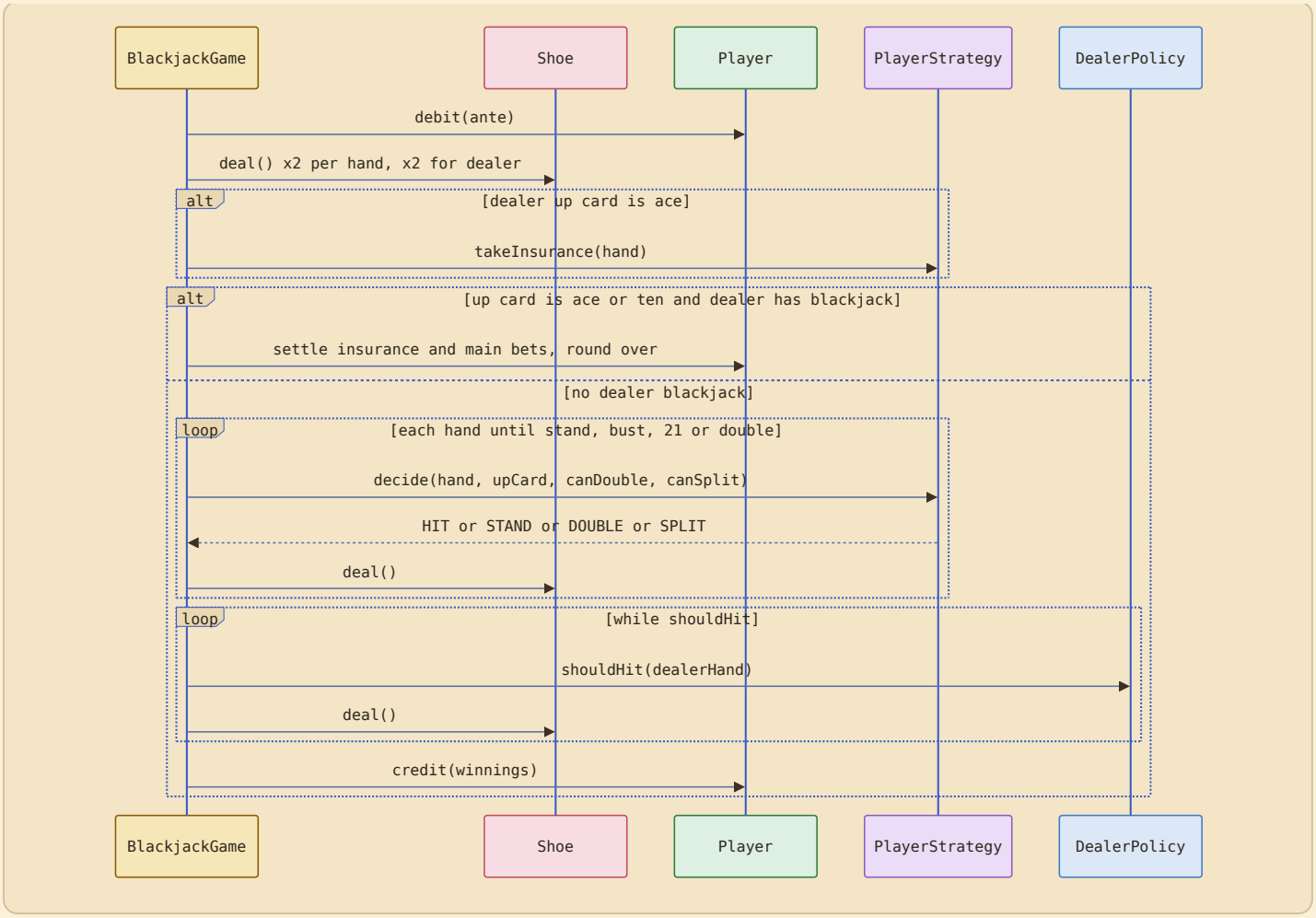
In the course model (figure above), `Card` holds a suit and a face value, and a subclass `BlackjackCard` adds the *game value* — because a king has face value 13 but is worth 10 in blackjack. A `Shoe` is composed of several `Deck`s (filled diamonds = composition: the parts belong to the whole). A plain arrow is an *association* (one class merely refers to another), and an open triangle is *inheritance* (a subclass extends a parent class). `Hand` scores itself. `Player` and `Dealer` share a `BasePlayer` (balance, hands), which extends `Account`. `Player` adds the bet; `Dealer` adds the "should I hit?" rule. `Game` points at a player, a dealer and a shoe and holds the order of play. Note the `*` from `BasePlayer` to `Hand`: splitting gives one player several hands.

Our Java design keeps the same core idea — **the game value must not live on the generic card** — but implements it with a separate scoring function instead of a subclass, and turns the two things that vary (player decisions, dealer rule) into strategies:

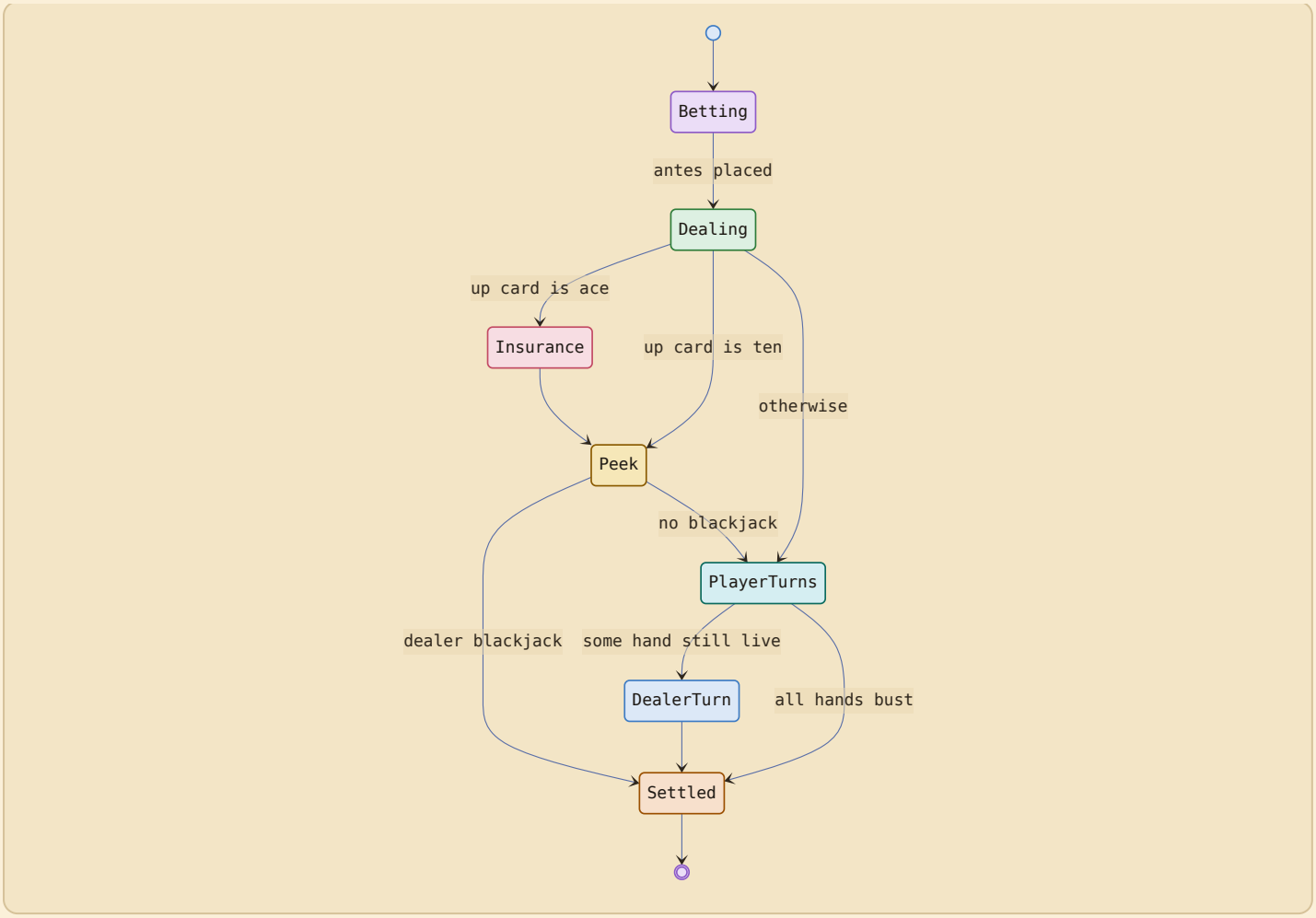


Key flows

One round (sequence)



States of one round



Scoring a hand (the ace problem). The trick is that **at most one ace can ever count as 11** (two elevens would be 22, a bust). So:

1. Add up every card with the ace as 1. This is the **hard total**.
2. If the hand has at least one ace and `hard + 10 ≤ 21`, one ace can be 11: the hand is **soft** and the best total is `hard + 10`.
3. Otherwise the best total is the hard total.

This is O(n) in the number of cards, needs no search over combinations, and gives the soft/hard flag for free (the dealer's "hit soft 17" rule needs it).

Design decisions & patterns used

Decision	Why	Alternative rejected
`Card` is a game-agnostic **record** (rank + suit).	A deck of cards is reused by poker, bridge, solitaire. A king's blackjack value (10) is a *rule of this game*, not a fact about the card.	Putting `getValue()` returning 10 for K on `Card` — breaks every other game.
Blackjack value in `BlackjackScoring`, not a `BlackjackCard` subclass.	Records cannot be extended, and the value depends only on the rank. A function is simpler and keeps one card type in the shoe.	The course's `BlackjackCard extends Card` — also valid, and also keeps `Card` clean. Say both in the interview; the important point is the same.
`Deck<T>` is **generic**.	"Design a deck of cards" follow-up: Uno or tarot cards reuse the same shuffle/draw code.	A deck hard-wired to `Card`.
Strategy for players (`PlayerStrategy`) and dealer (`DealerPolicy`).	Human UI, bots and test scripts plug in; casino variants (S17 vs H17: dealer stands on soft 17 vs hits soft 17) are one line. Open/Closed principle (add new behaviour with new classes, without editing working code).	`if (isHuman) ... else ...` and `if (casino == X)` branches inside the game.
Factory methods `Deck.standard52()`, `Shoe.stacked(...)`, `DealerPolicy.standOnAll17()`.	Named constructors make intent obvious.	Big constructors with flags.
Randomness injected (`Random` passed into `Shoe`).	Seeded shuffles make demos and tests exact.	`Collections.shuffle(list)` with a hidden random source.
`Hand` answers with `hardTotal()`, `bestTotal()` and `isSoft()`.	The course's `Hand` returns *every* possible total (A+6 is both 7 and 17) and lets the caller choose. Because at most one ace can ever count as 11, the hard total plus the soft flag already carries all of that information, and most callers just want `bestTotal()`.	Returning a list of totals that each caller must filter.
Hand stores its **own bet**.	Split and double change the bet *per hand*, not per player.	One bet per player — breaks as soon as a player splits.
The game **validates** a strategy's choice (`canDouble`, `canSplit` passed in, and re-checked).	A buggy or malicious client (the program sending the player's choices) cannot double without money.	Trusting the client.

Honest weaknesses (and what a longer design would do). `BlackjackGame.playRound` owns the whole sequence of play *and* the payout rules. Each new option (surrender, side bets) adds another branch. A larger design would model the round as a **State** machine (a fixed set of states plus the allowed moves between them; the State pattern gives each state its own class — one class per state in the diagram above, like the vending machine in Chapter 14) and move payouts into a `PayoutTable`. Say this out loud; interviewers reward knowing where your design will strain.

Java implementation

Package `lld.ch17.blackjack`.

```java
// File: code/lld/ch17/blackjack/Suit.java
package lld.ch17.blackjack;

/** The four suits. A suit never affects a blackjack score, but a generic deck needs it. */
public enum Suit {
    CLUBS('C'), DIAMONDS('D'), HEARTS('H'), SPADES('S');

    private final char letter;

    Suit(char letter) { this.letter = letter; }

    public char letter() { return letter; }
}
```

```java
// File: code/lld/ch17/blackjack/Rank.java
package lld.ch17.blackjack;

/** Thirteen ranks with their face value (ace = 1, jack = 11, queen = 12, king = 13). */
public enum Rank {
    ACE("A", 1), TWO("2", 2), THREE("3", 3), FOUR("4", 4), FIVE("5", 5), SIX("6", 6),
    SEVEN("7", 7), EIGHT("8", 8), NINE("9", 9), TEN("10", 10),
    JACK("J", 11), QUEEN("Q", 12), KING("K", 13);

    private final String symbol;
    private final int faceValue;

    Rank(String symbol, int faceValue) { this.symbol = symbol; this.faceValue = faceValue; }

    public String symbol() { return symbol; }
    public int faceValue() { return faceValue; }
}
```

```java
// File: code/lld/ch17/blackjack/Card.java
package lld.ch17.blackjack;

/** A game-agnostic playing card: just a rank and a suit. Immutable, so it is safe to share. */
public record Card(Rank rank, Suit suit) {
    @Override public String toString() { return rank.symbol() + suit.letter(); }
}
```

```java
// File: code/lld/ch17/blackjack/Deck.java
package lld.ch17.blackjack;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collection;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.NoSuchElementException;
import java.util.Random;

/**
 * A generic deck: any card type T (poker card, Uno card, tarot card...).
 * Shuffling takes a Random so that tests can pass a seeded one and get the same order every run.
 */
public final class Deck<T> {
    private final Deque<T> cards;

    public Deck(Collection<? extends T> cards) { this.cards = new ArrayDeque<>(cards); }

    /** Factory for the standard 52-card French deck (13 ranks x 4 suits). */
    public static Deck<Card> standard52() {
        List<Card> all = new ArrayList<>(52);
        for (Suit s : Suit.values()) {
            for (Rank r : Rank.values()) {
                all.add(new Card(r, s));
            }
        }
        return new Deck<>(all);
    }

    public void shuffle(Random rng) {
        List<T> list = new ArrayList<>(cards);
        Collections.shuffle(list, rng);
        cards.clear();
        cards.addAll(list);
    }

    public T draw() {
        if (cards.isEmpty()) throw new NoSuchElementException("deck is empty");
        return cards.pop();
    }

    public int size() { return cards.size(); }

    public List<T> cards() { return List.copyOf(cards); }
}
```

```java
// File: code/lld/ch17/blackjack/Shoe.java
package lld.ch17.blackjack;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Deque;
import java.util.List;
import java.util.Random;

/**
 * A casino shoe: several standard decks shuffled together. A "cut card" marks how deep we deal
 * before reshuffling (penetration). A stacked shoe (fixed order) exists for deterministic tests.
 */
public final class Shoe {
    private final int numberOfDecks;
    private final Random rng;          // null for a stacked test shoe
    private final int cutCardPosition; // reshuffle when fewer cards than this remain
    private final Deque<Card> cards = new ArrayDeque<>();

    public Shoe(int numberOfDecks, Random rng, double penetration) {
        if (numberOfDecks < 1) throw new IllegalArgumentException("need at least one deck");
        if (penetration <= 0 || penetration >= 1) throw new IllegalArgumentException("penetration must be in
(0,1)");
        this.numberOfDecks = numberOfDecks;
        this.rng = rng;
        this.cutCardPosition = (int) Math.round(numberOfDecks * 52 * (1 - penetration));
        refill();
    }

    private Shoe(List<Card> stackedOrder) {
        this.numberOfDecks = 0;
        this.rng = null;
        this.cutCardPosition = 0;
        cards.addAll(stackedOrder);
    }

    /** Cards come out in exactly the order given. Perfect for unit tests. */
    public static Shoe stacked(Card... order) { return new Shoe(List.of(order)); }

    public Card deal() {
        if (cards.isEmpty()) {
            if (rng == null) throw new IllegalStateException("stacked shoe ran out of cards");
            refill();
        }
        return cards.pop();
    }

    public boolean needsReshuffle() { return rng != null && cards.size() < cutCardPosition; }

    public void reshuffle() { refill(); }

    public int remaining() { return cards.size(); }

    private void refill() {
        List<Card> all = new ArrayList<>(numberOfDecks * 52);
        for (int i = 0; i < numberOfDecks; i++) {
            all.addAll(Deck.standard52().cards());
        }
        Collections.shuffle(all, rng);
        cards.clear();
        cards.addAll(all);
    }
}
```

```java
// File: code/lld/ch17/blackjack/BlackjackScoring.java
package lld.ch17.blackjack;

/**
 * The blackjack-specific value of a card lives here, NOT on Card.
 * A king has face value 13 in general, but it is worth 10 points in this game.
 */
public final class BlackjackScoring {
    private BlackjackScoring() {}

    /** Points with an ace counted as 1 (the "hard" value). */
    public static int points(Rank rank) {
        return switch (rank) {
            case ACE -> 1;
            case JACK, QUEEN, KING -> 10;
            default -> rank.faceValue();
        };
    }
}
```

```java
// File: code/lld/ch17/blackjack/Hand.java
package lld.ch17.blackjack;

import java.util.ArrayList;
import java.util.List;
import java.util.stream.Collectors;

/** A blackjack hand: its cards, its bet, and how to score itself (aces = 1 or 11). */
public final class Hand {
    private final List<Card> cards = new ArrayList<>();
    private int bet;
    private boolean fromSplit;
    private boolean doubled;

    public Hand(int bet) { this.bet = bet; }

    public static Hand of(Card... cs) {
        Hand h = new Hand(0);
        for (Card c : cs) h.add(c);
        return h;
    }

    void add(Card c) { cards.add(c); }

    /** Every ace counted as 1. */
    public int hardTotal() {
        int sum = 0;
        for (Card c : cards) sum += BlackjackScoring.points(c.rank());
        return sum;
    }

    private boolean hasAce() { return cards.stream().anyMatch(c -> c.rank() == Rank.ACE); }

    /** A hand is soft when one ace can count as 11 without busting. At most one ace can ever do so. */
    public boolean isSoft() { return hasAce() && hardTotal() + 10 <= 21; }

    /** The best total: the soft total if it is legal, otherwise the hard total. */
    public int bestTotal() { return isSoft() ? hardTotal() + 10 : hardTotal(); }

    public boolean isBust() { return bestTotal() > 21; }

    /** A natural: exactly two cards worth 21, and not the result of a split. */
    public boolean isBlackjack() { return !fromSplit && cards.size() == 2 && bestTotal() == 21; }

    /** Two cards of the same rank, and no re-splitting (one split per round). */
    public boolean canSplit() {
        return !fromSplit && cards.size() == 2 && cards.get(0).rank() == cards.get(1).rank();
    }

    /** Takes the second card out to start a new hand; both hands are now split hands. */
    Hand split() {
        if (!canSplit()) throw new IllegalStateException("hand cannot be split");
        Hand other = new Hand(bet);
        other.add(cards.remove(1));
        this.fromSplit = true;
        other.fromSplit = true;
        return other;
    }

    void doubleDown() { bet *= 2; doubled = true; }

    public boolean isDoubled() { return doubled; }
    public int bet() { return bet; }
    public int size() { return cards.size(); }
    public Card first() { return cards.get(0); }
    public List<Card> cards() { return List.copyOf(cards); }

    @Override public String toString() {
        String cs = cards.stream().map(Card::toString).collect(Collectors.joining(" "));
        String kind = isBlackjack() ? "BLACKJACK" : isBust() ? "bust " + bestTotal()
```

```
                : (isSoft() ? "soft " : "") + bestTotal();
        return "[" + cs + "] = " + kind;
    }
}
```

```
// File: code/lld/ch17/blackjack/Action.java
package lld.ch17.blackjack;

public enum Action { HIT, STAND, DOUBLE, SPLIT }
```

```
// File: code/lld/ch17/blackjack/PlayerStrategy.java
package lld.ch17.blackjack;

/**
 * Strategy pattern: HOW a player decides is pluggable (human via UI, basic-strategy bot, test script).
 * The game passes what is allowed, so a strategy never has to know the betting rules.
 */
public interface PlayerStrategy {
    Action decide(Hand hand, Card dealerUpCard, boolean canDouble, boolean canSplit);

    /** Offered only when the dealer shows an ace. Mathematically a bad bet for most players. */
    default boolean takeInsurance(Hand hand) { return false; }
}
```

```
// File: code/lld/ch17/blackjack/BasicStrategy.java
package lld.ch17.blackjack;

/** A simplified version of the well-known "basic strategy" table. */
public final class BasicStrategy implements PlayerStrategy {
    @Override
    public Action decide(Hand hand, Card up, boolean canDouble, boolean canSplit) {
        int dealer = BlackjackScoring.points(up.rank()) == 1 ? 11 : BlackjackScoring.points(up.rank());
        if (canSplit && (hand.first().rank() == Rank.ACE || hand.first().rank() == Rank.EIGHT)) {
            return Action.SPLIT;                                  // always split aces and eights
        }
        int total = hand.bestTotal();
        if (hand.isSoft()) {
            return total >= 18 ? Action.STAND : Action.HIT;
        }
        if (canDouble && (total == 11 || (total == 10 && dealer <= 9))) {
            return Action.DOUBLE;
        }
        if (total >= 17) return Action.STAND;
        if (total >= 13) return dealer <= 6 ? Action.STAND : Action.HIT;
        if (total == 12) return (dealer >= 4 && dealer <= 6) ? Action.STAND : Action.HIT;
        return Action.HIT;
    }
}
```

```
// File: code/lld/ch17/blackjack/MimicDealerStrategy.java
package lld.ch17.blackjack;

/** A naive player who copies the dealer (hit below 17) and always buys insurance. */
public final class MimicDealerStrategy implements PlayerStrategy {
    @Override
    public Action decide(Hand hand, Card up, boolean canDouble, boolean canSplit) {
        return hand.bestTotal() < 17 ? Action.HIT : Action.STAND;
    }

    @Override public boolean takeInsurance(Hand hand) { return true; }
}
```

```java
// File: code/lld/ch17/blackjack/DealerPolicy.java
package lld.ch17.blackjack;

/** The house rule for when the dealer draws. Casinos differ, so it is a pluggable strategy. */
@FunctionalInterface
public interface DealerPolicy {
    boolean shouldHit(Hand dealerHand);

    /** Classic rule: draw on 16 or less, stand on every 17 ("S17"). */
    static DealerPolicy standOnAll17() { return h -> h.bestTotal() <= 16; }

    /** Variant: also draw on a soft 17 such as A+6 ("H17"), which favours the house. */
    static DealerPolicy hitSoft17() { return h -> h.bestTotal() <= 16 || (h.bestTotal() == 17 && h.isSoft()); }
}
```

```java
// File: code/lld/ch17/blackjack/Player.java
package lld.ch17.blackjack;

/** A seated player: a name, a chip balance (whole chips) and a decision strategy. */
public final class Player {
    private final String name;
    private final PlayerStrategy strategy;
    private int balance;

    public Player(String name, int balance, PlayerStrategy strategy) {
        this.name = name;
        this.balance = balance;
        this.strategy = strategy;
    }

    void debit(int amount) {
        if (amount > balance) throw new IllegalStateException(name + " cannot cover " + amount);
        balance -= amount;
    }

    void credit(int amount) { balance += amount; }

    public boolean canAfford(int amount) { return balance >= amount; }
    public String name() { return name; }
    public int balance() { return balance; }
    public PlayerStrategy strategy() { return strategy; }
}
```

```java
// File: code/lld/ch17/blackjack/BlackjackGame.java
package lld.ch17.blackjack;

import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Consumer;

/**
 * Runs rounds at one table: bets, deal, insurance + dealer peek, player turns (hit/stand/double/split),
 * dealer turn, settlement. Each player plays only against the dealer.
 */
public final class BlackjackGame {
    private final Shoe shoe;
    private final DealerPolicy dealerPolicy;
    private final List<Player> players;
    private final Consumer<String> log;

    public BlackjackGame(Shoe shoe, DealerPolicy dealerPolicy, List<Player> players, Consumer<String> log) {
        this.shoe = shoe;
        this.dealerPolicy = dealerPolicy;
        this.players = List.copyOf(players);
        this.log = log;
    }

    public void playRound(int ante) {
        if (ante <= 0 || ante % 2 != 0) throw new IllegalArgumentException("ante must be a positive even number");
        if (shoe.needsReshuffle()) {
            shoe.reshuffle();
            log.accept("  (cut card reached: shoe reshuffled)");
        }

        // 1. Bets: each player who can afford the ante gets one hand.
        Map<Player, List<Hand>> hands = new LinkedHashMap<>();
        for (Player p : players) {
            if (!p.canAfford(ante)) { log.accept("  " + p.name() + " sits out (balance " + p.balance() + ")");
continue; }
            p.debit(ante);
            hands.put(p, new ArrayList<>(List.of(new Hand(ante))));
        }
        if (hands.isEmpty()) return;

        // 2. Deal two rounds: one card to each player, then one to the dealer. Dealer's 2nd card is the hole
card.
        Hand dealer = new Hand(0);
        for (int round = 0; round < 2; round++) {
            for (List<Hand> hs : hands.values()) hs.get(0).add(shoe.deal());
            dealer.add(shoe.deal());
        }
        Card up = dealer.first();
        log.accept("  Dealer shows " + up + " (hole card hidden)");
        hands.forEach((p, hs) -> log.accept("  " + p.name() + " " + hs.get(0)));

        // 3. Insurance (only when the up card is an ace), then the dealer peeks for blackjack.
        Map<Player, Integer> insurance = new LinkedHashMap<>();
        if (up.rank() == Rank.ACE) {
            hands.forEach((p, hs) -> {
                int side = ante / 2;
                if (p.strategy().takeInsurance(hs.get(0)) && p.canAfford(side)) {
                    p.debit(side);
                    insurance.put(p, side);
                    log.accept("  " + p.name() + " buys insurance for " + side);
                }
            });
        }
        boolean dealerMayHaveBlackjack = up.rank() == Rank.ACE || BlackjackScoring.points(up.rank()) == 10;
        if (dealerMayHaveBlackjack && dealer.isBlackjack()) {
            log.accept("  Dealer peeks: " + dealer + " -> round over");
            insurance.forEach((p, side) -> { p.credit(side * 3); log.accept("  " + p.name() + " insurance pays 2:1
```

```
(+" + side * 2 + ")"); });
            hands.forEach((p, hs) -> {
                Hand h = hs.get(0);
                if (h.isBlackjack()) { p.credit(h.bet()); log.accept("  " + p.name() + ": PUSH (both blackjack)");
}
                else log.accept("  " + p.name() + ": LOSE " + h.bet());
            });
            printBalances();
            return;
        }
        if (!insurance.isEmpty()) log.accept("  Dealer peeks: no blackjack, insurance bets lost");

        // 4. Player turns. Splitting inserts a new hand right after the current one.
        hands.forEach((p, hs) -> {
            for (int i = 0; i < hs.size(); i++) playHand(p, hs, i, up);
        });

        // 5. Dealer turn (skipped if every player hand is already bust).
        boolean anyLive = hands.values().stream().flatMap(List::stream).anyMatch(h -> !h.isBust());
        if (anyLive) {
            while (dealerPolicy.shouldHit(dealer)) dealer.add(shoe.deal());
            log.accept("  Dealer reveals and plays: " + dealer);
        }

        // 6. Settle every hand against the dealer.
        hands.forEach((p, hs) -> hs.forEach(h -> settle(p, h, dealer)));
        printBalances();
    }

    private void playHand(Player p, List<Hand> hs, int index, Card up) {
        Hand hand = hs.get(index);
        String label = p.name() + (hs.size() > 1 ? " hand " + (index + 1) : "");
        while (!hand.isBlackjack() && hand.bestTotal() < 21) {
            boolean canDouble = hand.size() == 2 && p.canAfford(hand.bet());
            boolean canSplit = hand.canSplit() && p.canAfford(hand.bet());
            Action a = p.strategy().decide(hand, up, canDouble, canSplit);
            if ((a == Action.DOUBLE && !canDouble) || (a == Action.SPLIT && !canSplit)) {
                throw new IllegalStateException(label + " chose a move that is not allowed: " + a);
            }
            switch (a) {
                case STAND -> { log.accept("  " + label + " stands on " + hand.bestTotal()); return; }
                case HIT -> { hand.add(shoe.deal()); log.accept("  " + label + " hits: " + hand); }
                case DOUBLE -> {
                    p.debit(hand.bet());
                    hand.doubleDown();
                    hand.add(shoe.deal());
                    log.accept("  " + label + " doubles (bet now " + hand.bet() + "): " + hand);
                    return;                                    // exactly one card after doubling
                }
                case SPLIT -> {
                    p.debit(hand.bet());
                    Hand second = hand.split();
                    hand.add(shoe.deal());
                    second.add(shoe.deal());
                    hs.add(index + 1, second);
                    label = p.name() + " hand " + (index + 1);
                    log.accept("  " + p.name() + " splits -> " + hand + " and " + second);
                }
            }
        }
        if (hand.isBust()) log.accept("  " + label + " busts");
    }

    private void settle(Player p, Hand h, Hand dealer) {
        String who = "  " + p.name() + " " + h + " vs dealer " + dealer.bestTotal() + ": ";
        if (h.isBust()) { log.accept(who + "LOSE " + h.bet()); return; }
        if (h.isBlackjack()) {                                 // dealer blackjack was handled by the peek
            p.credit(h.bet() + h.bet() * 3 / 2);
            log.accept(who + "BLACKJACK pays 3:2 (+" + h.bet() * 3 / 2 + ")");
            return;
        }
```

```
        int mine = h.bestTotal(), theirs = dealer.bestTotal();
        if (dealer.isBust() || mine > theirs) { p.credit(h.bet() * 2); log.accept(who + "WIN +" + h.bet()); }
        else if (mine == theirs) { p.credit(h.bet()); log.accept(who + "PUSH"); }
        else log.accept(who + "LOSE " + h.bet());
    }

    private void printBalances() {
        StringBuilder sb = new StringBuilder("  Balances:");
        players.forEach(p -> sb.append(' ').append(p.name()).append('=').append(p.balance()));
        log.accept(sb.toString());
    }
}
```

```
// File: code/lld/ch17/blackjack/Main.java
package lld.ch17.blackjack;

import java.util.List;
import java.util.Random;

public final class Main {
    private static Card c(Rank r, Suit s) { return new Card(r, s); }

    public static void main(String[] args) {
        System.out.println("== 1. Hand values (aces are 1 or 11) ==");
        List<Hand> samples = List.of(
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.SIX, Suit.HEARTS)),
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.SIX, Suit.HEARTS), c(Rank.TEN, Suit.CLUBS)),
                Hand.of(c(Rank.ACE, Suit.SPADES), c(Rank.ACE, Suit.HEARTS), c(Rank.NINE, Suit.CLUBS)),
                Hand.of(c(Rank.KING, Suit.SPADES), c(Rank.QUEEN, Suit.HEARTS), c(Rank.FIVE, Suit.CLUBS)),
                Hand.of(c(Rank.ACE, Suit.DIAMONDS), c(Rank.KING, Suit.CLUBS)));
        samples.forEach(h -> System.out.println("  " + h + "   (hard total " + h.hardTotal() + ")"));

        System.out.println("\n== 2. Stacked shoe: split 8s, then double both hands ==");
        Shoe stacked = Shoe.stacked(
                c(Rank.EIGHT, Suit.HEARTS), c(Rank.SIX, Suit.SPADES),     // Alice 1st card, dealer up card
                c(Rank.EIGHT, Suit.DIAMONDS), c(Rank.TEN, Suit.CLUBS),    // Alice 2nd card, dealer hole card
                c(Rank.THREE, Suit.CLUBS), c(Rank.TWO, Suit.SPADES),      // one card to each split hand
                c(Rank.NINE, Suit.HEARTS), c(Rank.FIVE, Suit.DIAMONDS),   // the two double-down cards
                c(Rank.SEVEN, Suit.SPADES));                              // dealer's hit on 16
        Player alice = new Player("Alice", 100, new BasicStrategy());
        new BlackjackGame(stacked, DealerPolicy.standOnAll17(), List.of(alice),
System.out::println).playRound(10);

        System.out.println("\n== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==");
        Shoe aceUp = Shoe.stacked(c(Rank.TEN, Suit.HEARTS), c(Rank.ACE, Suit.SPADES),
                c(Rank.NINE, Suit.CLUBS), c(Rank.KING, Suit.DIAMONDS));
        Player bob = new Player("Bob", 100, new MimicDealerStrategy());
        new BlackjackGame(aceUp, DealerPolicy.standOnAll17(), List.of(bob), System.out::println).playRound(10);

        System.out.println("\n== 4. Seeded 6-deck shoe (seed 42), two players, 5 rounds ==");
        Shoe shoe = new Shoe(6, new Random(42), 0.75);
        Player carol = new Player("Carol", 100, new BasicStrategy());
        Player dave = new Player("Dave", 100, new MimicDealerStrategy());
        BlackjackGame game = new BlackjackGame(shoe, DealerPolicy.hitSoft17(), List.of(carol, dave),
System.out::println);
        for (int round = 1; round <= 5; round++) {
            System.out.println("Round " + round + ":");
            game.playRound(10);
        }
    }
}
```

Output:

```
== 1. Hand values (aces are 1 or 11) ==
  [AS 6H] = soft 17   (hard total 7)
  [AS 6H 10C] = 17   (hard total 17)
  [AS AH 9C] = soft 21   (hard total 11)
  [KS QH 5C] = bust 25   (hard total 25)
  [AD KC] = BLACKJACK   (hard total 11)

== 2. Stacked shoe: split 8s, then double both hands ==
  Dealer shows 6S (hole card hidden)
  Alice [8H 8D] = 16
  Alice splits -> [8H 3C] = 11 and [8D 2S] = 10
  Alice hand 1 doubles (bet now 20): [8H 3C 9H] = 20
  Alice hand 2 doubles (bet now 20): [8D 2S 5D] = 15
  Dealer reveals and plays: [6S 10C 7S] = bust 23
  Alice [8H 3C 9H] = 20 vs dealer 23: WIN +20
  Alice [8D 2S 5D] = 15 vs dealer 23: WIN +20
  Balances: Alice=140

== 3. Stacked shoe: dealer shows an ace, Bob buys insurance ==
  Dealer shows AS (hole card hidden)
  Bob [10H 9C] = 19
  Bob buys insurance for 5
  Dealer peeks: [AS KD] = BLACKJACK -> round over
  Bob insurance pays 2:1 (+10)
  Bob: LOSE 10
  Balances: Bob=100

== 4. Seeded 6-deck shoe (seed 42), two players, 5 rounds ==
Round 1:
  Dealer shows AS (hole card hidden)
  Carol [8S 3S] = 11
  Dave [5D 9C] = 14
  Dave buys insurance for 5
  Dealer peeks: [AS QS] = BLACKJACK -> round over
  Dave insurance pays 2:1 (+10)
  Carol: LOSE 10
  Dave: LOSE 10
  Balances: Carol=90 Dave=100
Round 2:
  Dealer shows 6D (hole card hidden)
  Carol [2S 8H] = 10
  Dave [QS 10H] = 20
  Carol doubles (bet now 20): [2S 8H QD] = 20
  Dave stands on 20
  Dealer reveals and plays: [6D 6D 7C] = 19
  Carol [2S 8H QD] = 20 vs dealer 19: WIN +20
  Dave [QS 10H] = 20 vs dealer 19: WIN +10
  Balances: Carol=110 Dave=110
Round 3:
  Dealer shows 5C (hole card hidden)
  Carol [AD KD] = BLACKJACK
  Dave [8D AH] = soft 19
  Dave stands on 19
  Dealer reveals and plays: [5C 7D 7S] = 19
  Carol [AD KD] = BLACKJACK vs dealer 19: BLACKJACK pays 3:2 (+15)
  Dave [8D AH] = soft 19 vs dealer 19: PUSH
  Balances: Carol=125 Dave=110
Round 4:
  Dealer shows 10S (hole card hidden)
  Carol [8D 8S] = 16
  Dave [5C 9S] = 14
  Carol splits -> [8D 4S] = 12 and [8S JD] = 18
  Carol hand 1 hits: [8D 4S 7D] = 19
  Carol hand 1 stands on 19
  Carol hand 2 stands on 18
  Dave hits: [5C 9S KS] = bust 24
  Dave busts
  Dealer reveals and plays: [10S 9H] = 19
  Carol [8D 4S 7D] = 19 vs dealer 19: PUSH
  Carol [8S JD] = 18 vs dealer 19: LOSE 10
```

```
  Dave [5C 9S KS] = bust 24 vs dealer 19: LOSE 10
  Balances: Carol=115 Dave=100
Round 5:
  Dealer shows KD (hole card hidden)
  Carol [5S 8C] = 13
  Dave [8S 2C] = 10
  Carol hits: [5S 8C JS] = bust 23
  Carol busts
  Dave hits: [8S 2C 7C] = 17
  Dave stands on 17
  Dealer reveals and plays: [KD 10H] = 20
  Carol [5S 8C JS] = bust 23 vs dealer 20: LOSE 10
  Dave [8S 2C 7C] = 17 vs dealer 20: LOSE 10
  Balances: Carol=105 Dave=90
```

Reading the output. Part 1 shows the scoring: A+6 is soft 17, adding a 10 makes it a *hard* 17 (the ace dropped to 1), A+A+9 is soft 21. Part 2 uses a stacked shoe to force an exact scenario: Alice splits eights against a dealer 6, doubles both hands, and the dealer busts — she staked 40 chips and won 40. Part 3 shows insurance working as a hedge: Bob loses his main bet of 10 but the 5-chip insurance pays 10, so he breaks even. Part 4 is a seeded six-deck game: run it again and you get the same cards.

Concurrency & edge cases

- **One table = one thread.** A **thread** is one independent line of execution in a program; a server runs many side by side. A round is a strict sequence, so the simplest safe design runs each table's rounds on a single thread (or an actor/event loop — a single worker that processes incoming messages one at a time from its own queue) and funnels player commands into it. Many tables run in parallel; they share nothing except the wallet service.
- **Balances are shared across tables** in a real casino app. Move `debit`/`credit` into a wallet service with atomic operations — ones that happen as a single indivisible step no other thread can interrupt (a database transaction, i.e. a group of changes that all succeed or all fail, or `AtomicLong`/`synchronized` in memory; `synchronized` lets only one thread at a time run that code) and debit *before* dealing.
- **Odd antes and 3:2.** 3:2 on 5 chips is 7.5. We require even antes. Real casinos either round down or use smaller chip units — decide and state it.
- **Split aces.** Many casinos deal only one card to each split ace, and 21 after a split is *not* a blackjack (our `isBlackjack()` checks `fromSplit`).
- **Shoe exhaustion.** The cut card triggers a reshuffle *between* rounds; the shoe also refills if emptied mid-round. A stacked test shoe throws instead, so a wrong test fails loudly.
- **Player cannot afford a double/split.** The game passes `canDouble=false` and rejects the action if chosen anyway.
- **Dealer peek only on ace or ten.** If the up card is anything else the dealer cannot have blackjack, so there is no peek.

Extensions & follow-up questions

- **"Where would even money go?"** — It is the same as insurance for a player holding blackjack: insurance of half the ante plus a blackjack always nets exactly +1× the ante whatever the dealer has. Add `PlayerStrategy.takeEvenMoney(hand)` and, if accepted, settle that hand at 1:1 before the peek.
- **"Add surrender."** — New `Action.SURRENDER`, allowed only as the first decision; refund half the bet and end the hand.
- **"Several players at the table?"** — Already supported: the game iterates players in seat order, then the dealer plays once.
- **"Card counting detection / security?"** — Card counting is tracking which cards have gone to guess what is left. Shoe penetration (how deep into the shoe the cut card sits) already limits counting. Log every dealt card for audit (a later independent check of what happened); never send the hole card to clients until it is revealed.
- **"How would you test the dealer rule?"** — Stacked shoe + `DealerPolicy.hitSoft17()`: deal the dealer A+6 and assert it draws; with `standOnAll17()` assert it stands.
- **"Make the deck work for Uno."** — `Deck<UnoCard>` with its own factory; nothing else changes.

- **"Replace the round logic with a state machine."** — One class per state (Betting, Dealing, PlayerTurns, DealerTurn, Settled), each handling only the commands valid in that state; invalid commands are rejected by the current state.

Real-world example

Online casino tables (for example live-dealer blackjack products) separate exactly these concerns: a **shuffle/RNG service** that is certified and audited (gambling regulators, for example the UK Gambling Commission, require the random number generator to be independently tested), a **game engine** that enforces the table rules (number of decks, S17 or H17, payout 3:2 or 6:5 — printed on the physical table felt), and a **wallet** service that debits bets atomically. The rule variants are configuration (settings chosen at deploy time, not code), which is exactly why `DealerPolicy` is a strategy.

In the interview

30-second answer: "A generic `Card` (rank, suit) and a generic `Deck<T>`. A `Shoe` combines several decks and deals with an injected `Random`, plus a stacked constructor for tests. The blackjack value of a card lives in the game's scoring, not on `Card`. `Hand` scores itself: hard total with aces as 1, and it's soft if one ace can be 11 without busting. Player decisions and the dealer's hit rule are strategies. `BlackjackGame` runs the round: bets, deal, insurance and peek, player turns with hit/stand/double/split, dealer turn, settle 3:2, 1:1 or push."

How to spend 30–45 minutes: 5 min questions and rules → 5 min entities → 10 min class diagram (defend `Card` vs game value, hand-owned bet) → 15 min code `Hand` scoring and `playRound` → 5 min edge cases (split aces, insurance, money) and extensions.

Likely follow-ups with answers

- *"Why not an `int value` on `Card`?"* — Because value is game-specific; poker ranks aces high, bridge uses suits, blackjack makes K = 10.
- *"How do you compute the best hand value?"* — Hard total, then add 10 if there is an ace and it fits. At most one ace can be 11.
- *"How do you make the game testable?"* — Inject the randomness: seeded `Random` for repeatability, stacked shoe for exact scenarios.

Self-check

1. **Q:** Why does the `Hand` store the bet instead of the `Player`? **A:** After a split, one player has two hands with separate bets; after a double, only that hand's bet changes.
2. **Q:** A+A+9 — soft or hard, and what total? **A:** Hard 11 with aces as 1; one ace can be 11 → soft 21.
3. **Q:** When does the dealer peek? **A:** Only when the up card is an ace or a 10-value card — otherwise blackjack is impossible.
4. **Q:** What does insurance pay, and how much is it? **A:** 2:1. In this design (and the course) it is exactly half the ante; real casinos let you buy *up to* half.
5. **Q:** Is 21 after splitting a blackjack? **A:** No, a blackjack must be the original two-card hand.

TL;DR

- Keep `Card` / `Deck` generic; the game's value rule lives in the game (`BlackjackScoring`, or a `BlackjackCard` subclass in the course version).
- Hand value: hard total + 10 if an ace fits → soft. Blackjack = original two cards totalling 21.
- Strategies for player decisions and dealer rule; inject randomness; stacked shoe for tests.
- Round = bet → deal → (insurance + peek) → player turns → dealer turn → settle (3:2, 1:1, push).

Design Chess



Figure — A chess board in play (course diagram)

Problem & clarifying questions

Chess is a two-player strategy game on an 8×8 board of 64 squares. Each side starts with 16 pieces: **8 pawns, 2 rooks, 2 knights, 2 bishops, 1 queen, 1 king**. The goal is **checkmate**: attack the enemy king so that it has no legal escape. We design an **online** two-player chess game.

Question	Assumed answer
Full international rules?	Yes: every piece's moves, captures, check, checkmate, stalemate, castling, en passant, promotion.
How are sides chosen?	Randomly assigned white or black (`Game.withRandomSides`, with an injected `Random`); **white moves first**; players alternate.
Can players take moves back?	Not in a rated game (one that counts toward the players' official ratings; course requirement). We still support `undo()` for analysis mode and tests.
How can a game end?	Checkmate, stalemate (a draw), resignation, or forfeit.
Move log?	Yes — every move by both players is recorded.
Timers, rating, spectators, draw by repetition?	Out of scope; discussed in Extensions.
AI opponent?	Out of scope; a bot would be a `MoveStrategy` that picks from `legalMoves()`.

Requirements

Functional

1. Two players play one game; each is assigned a colour, white moves first, turns alternate.
2. A player moves one of **their own** pieces; the system rejects moves out of turn, moves a piece cannot make, and moves that would leave the mover's own king in check.
3. All international rules, including the special moves:
    - **Castling** — the king moves two squares toward a rook and the rook jumps over to the square the king crossed. Allowed only if neither piece has moved, the squares between are empty, the king is not in check, and the king does not pass through or land on an attacked square.

- **En passant** ("in passing") — if a pawn advances two squares and lands beside an enemy pawn, that enemy pawn may capture it *as if it had moved only one square*, but only on the very next move.
- **Promotion** — a pawn reaching the last rank becomes a queen, rook, bishop or knight (the player's choice).

4. Detect **check** (the king is attacked), **checkmate** (in check and no legal move — the attacker wins) and **stalemate** (not in check but no legal move — a draw).
5. Support **resignation** (the player gives up) and **forfeit** (the player loses without choosing to, for example by leaving or timing out); record the result.
6. Keep a **log** of every move.

Non-functional

- Adding a new piece type (for a chess variant) should not touch existing pieces (Open/Closed).
- Legality checks must be correct first; speed is a secondary concern for a two-human game.
- Online play: the game object must be safe when two player connections call it concurrently (at the same time, from different threads).

Use cases

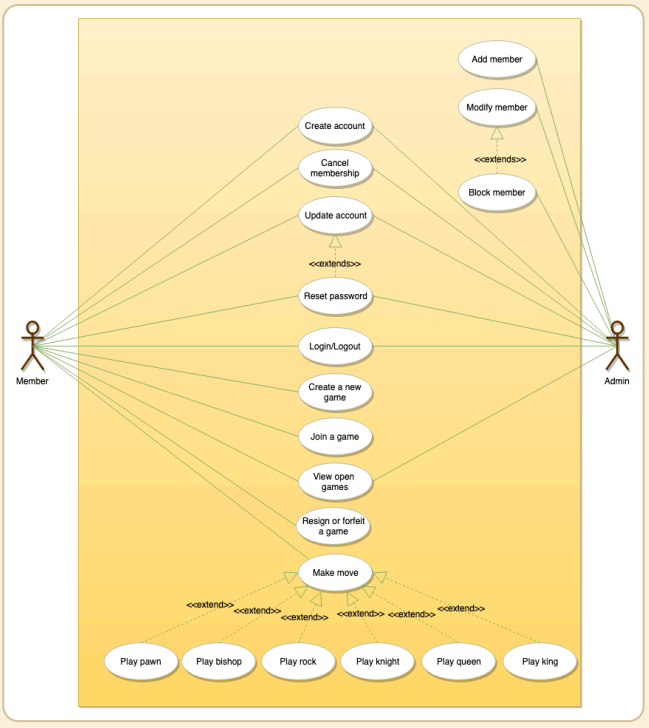


Figure — Chess use cases for Member (player) and Admin, with "Make move" extended by each piece type (course diagram)

In the figure above there are two actors:

- **Player** (a registered member) — creates or joins a game, views open games, **makes moves**, resigns or forfeits, and manages their account.
- **Admin** — adds, modifies, or blocks (bans) members.

The key use cases are **Player moves a piece** (extended per piece type — "Play pawn", "Play knight"... — which is exactly the polymorphism in our `Piece` hierarchy), **Resign or forfeit**, **Register/cancel membership**, and **Update game log** (every move is appended).

Activity: make a move

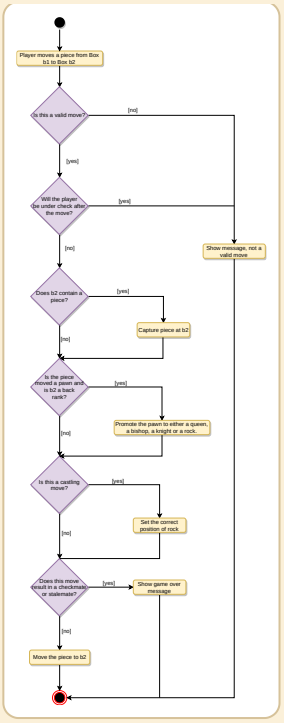


Figure — Activity diagram for making a move: validate, check self-check, capture, promotion, castling, and checkmate/stalemate (course diagram)

The figure above shows the full decision chain for one move. We follow the same order in `Game.move` and `Game.play`:

1. Is the move valid for that piece? If not, show "not a valid move".
2. Would the mover's king be in check afterwards? If yes, reject.
3. Does the destination hold a piece? Capture it.
4. Is it a pawn reaching the back rank? Promote it.
5. Is it castling? Move the rook as well.
6. Does the move produce checkmate or stalemate? Then the game is over.

Core entities

Class	Responsibility
`Color` (enum)	WHITE/BLACK and `opposite()`.
`Position` (record)	A square (file a–h, rank 1–8). Validates bounds; parses "e4". The course calls this a *Box*.
`Piece` (abstract — cannot be created directly, only through subclasses; sealed — only listed subclasses allowed)	Colour + `targets(board, from)`: where it can move by its own rules. `attacks(...)` for check detection.
`King`, `Knight`, `Pawn`	Step pieces and the pawn's special forward/diagonal rule.
`SlidingPiece` → `Rook`, `Bishop`, `Queen`	Share one "slide until blocked" loop; each subclass only lists its directions.
`Board`	8×8 grid of pieces; `copy()`, `kingOf(color)`, `isAttacked(square, byColor)`, `render()`. No turn or history knowledge.
`Move` (record) + `MoveType` (enum)	A move as data: from, to, type (NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION), promotion piece.
`CastlingRights` (record)	The four castling permissions; how they change after a move.
`Rules`	Stateless (keeps no data between calls) rules engine: pseudo-legal moves (allowed by piece movement, before the king-safety check), legal moves (king-safety filter), special moves, `apply(board, move)`, `isInCheck`.
`Game`	Turn, status, winner, en passant square, castling rights, move log, undo (Command + Memento — patterns explained below), resign, forfeit, random side assignment. Thread-safe (correct even when several threads call it at once).
`Player` (record), `GameStatus` (enum), `IllegalMoveException`	Supporting types.

Class diagram

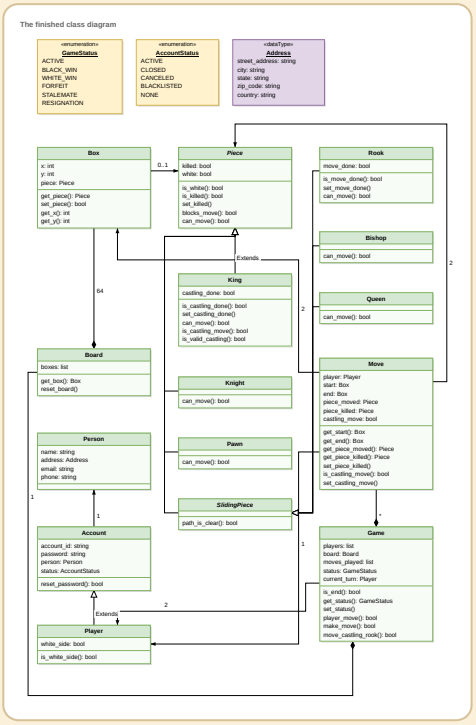
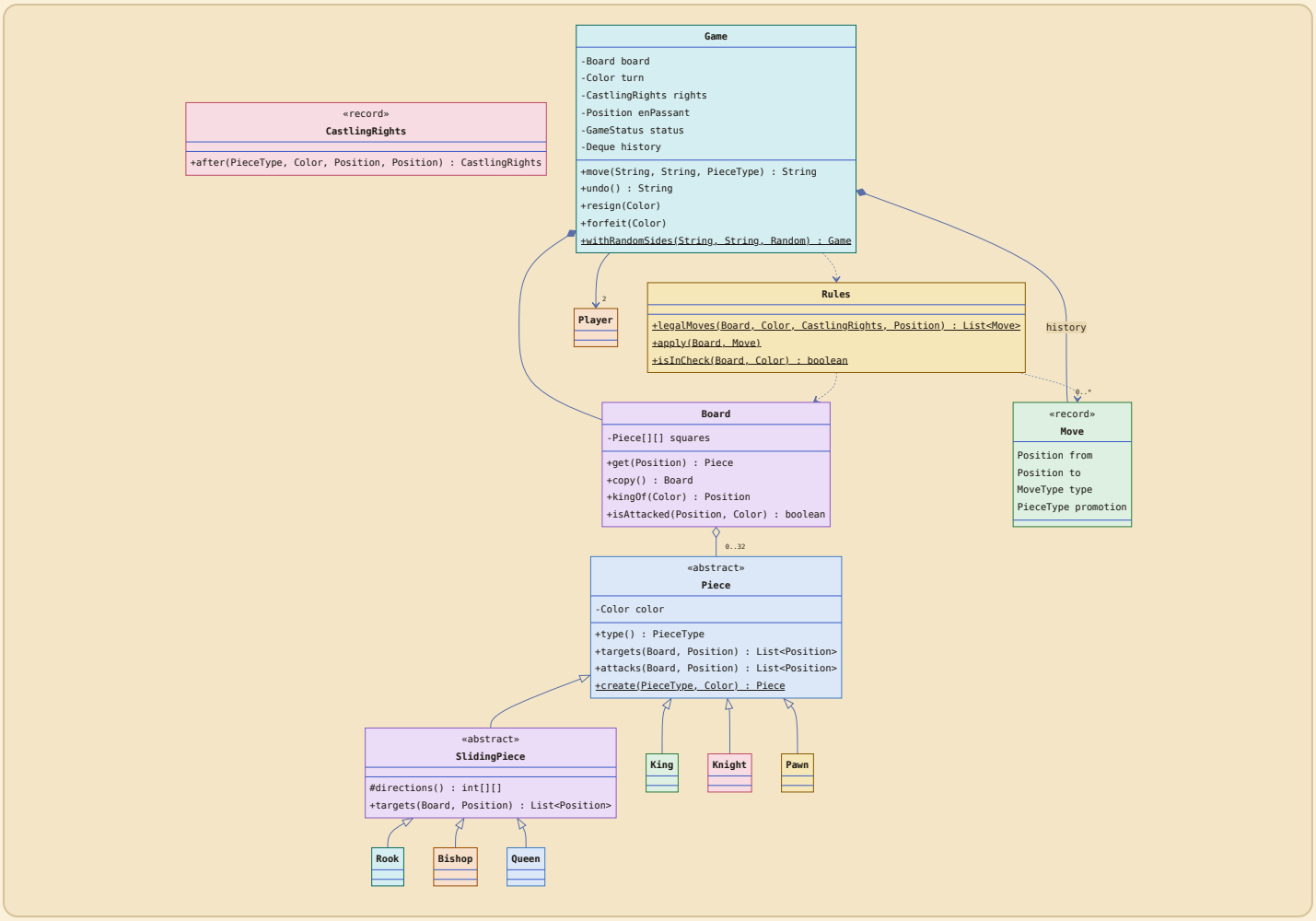


Figure — Course class diagram: Box, Board, Piece hierarchy with SlidingPiece, Move, Player, Account, Game, GameStatus (course diagram)

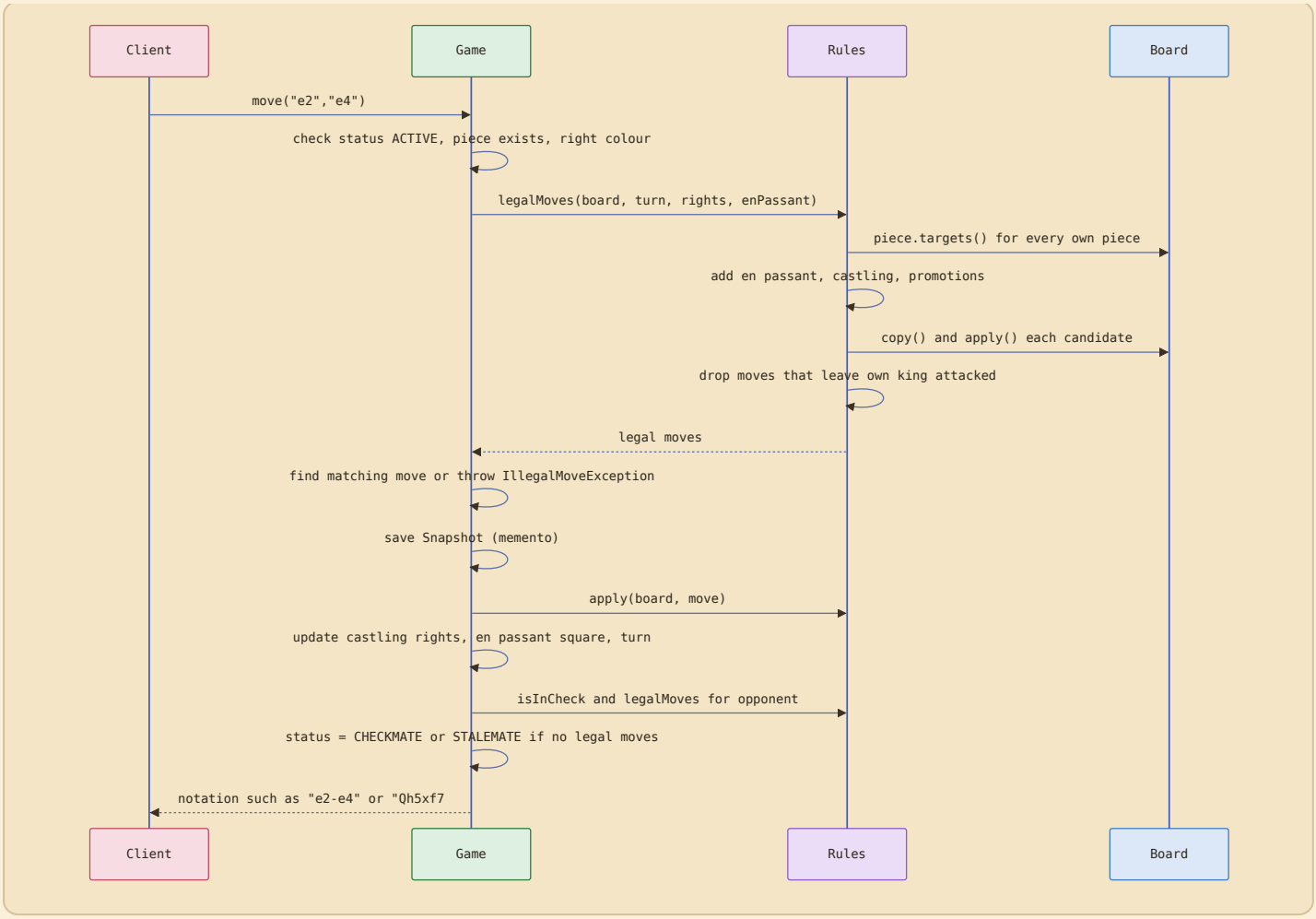
In the course model (figure above), a `Board` is **composed** of 64 `Box`es; a `Box` refers to at most one `Piece` with a *plain association* (not composition), because capturing a piece takes it off the board without destroying it — which is what makes a move log replayable. `Piece` is abstract with `can_move(board, start, end)`; `SlidingPiece` sits between `Piece` and rook/bishop/queen so the "path is clear" check is written once, while knight and king extend `Piece` directly. A `Move` is a *record* of what happened (player, start and end box, piece moved, piece captured, castling flag). `Game` owns the board and the list of moves and refers to the two players (they exist before and after the game). The course's `GameStatus` has ACTIVE, BLACK_WIN, WHITE_WIN, FORFEIT, STALEMATE, RESIGNATION.

Our Java design keeps that shape and fixes the two limits the course itself points out: (1) its pieces decide legality from the board alone, so history-based rules (castling rights, en passant) have nowhere to live, and (2) checkmate/stalemate are not detected. We add `Rules` (which sees history through `CastlingRights` and the en passant square) and make `Move` carry a `MoveType`, so a single move can move two pieces (castling) or remove a piece that is not on the target square (en passant).

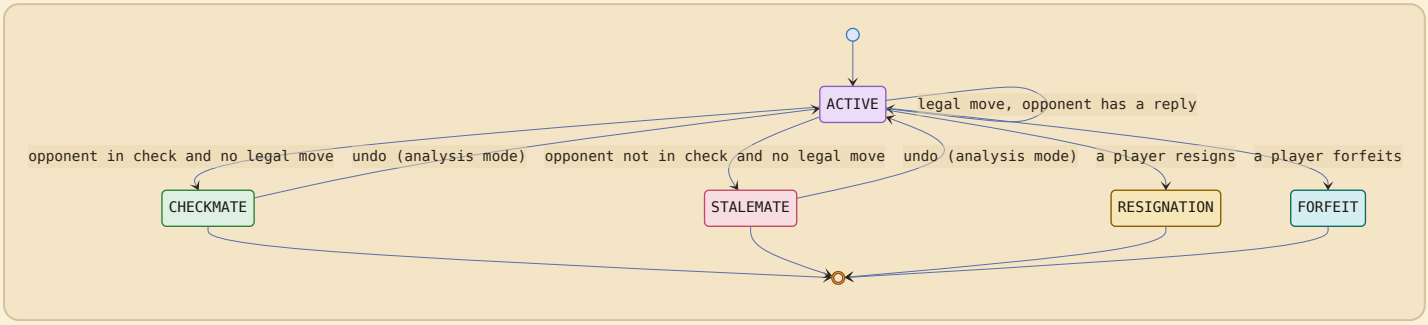


Key flows

Making a move



Game status



How check, checkmate and stalemate are detected (the "reasonable depth" answer)

1. **Pseudo-legal moves** — every move allowed by the pieces' movement rules, plus castling, en passant and promotions.
2. **Legal moves** — for each pseudo-legal move, apply it to a *copy* of the board and keep it only if the mover's king is **not attacked** afterwards. This one filter handles pins, moving into check, and ignoring a check, with no special cases.
3. **Is the king attacked?** Ask every enemy piece for the squares it attacks (`attacks(...)`); pawns attack diagonally even onto empty squares, which is why `Pawn` overrides `attacks`.
4. After a move, look at the **opponent**: if it has **zero legal moves**, it is **checkmate** when its king is in check, otherwise **stalemate**.

This is a one-ply (one move deep) search, which is all the *rules* need. Searching deeper is what a chess *engine* does to choose good moves — not required for enforcing rules.

Cost: at most ~200 pseudo-legal moves, each checked with an attack scan over ≤16 pieces — trivial for a human game. A production engine would use bitboards (each piece type's squares packed into one 64-bit number, so set operations become fast bit tricks) and incremental attack maps (attacked squares updated per move instead of recomputed); mention it only if asked.

Design decisions & patterns used

Decision	Why	Alternative rejected
Polymorphic pieces (`targets` overridden per piece), **Template-like** `SlidingPiece`.	Each piece's rule lives in its own class; the blocked-path loop is written once for three pieces.	One giant `switch (pieceType)` in the board.
Sealed hierarchy (`sealed ... permits`).	The set of chess pieces is closed; the compiler knows all subclasses, so `switch` on type can be exhaustive (the compiler checks every case is handled).	Open hierarchy — fine too for variants; say so.
Pieces are **immutable**; `Board.copy()` copies references (pointers to the same piece objects) only.	Trial moves and snapshots are cheap and safe.	Mutable pieces with `hasMoved` flags — then undo must restore every flag.
History-based rules in `Rules` / `Game`, not in pieces.	Castling rights and en passant depend on earlier moves. The course notes that a piece seeing only the board cannot know this.	Letting `King` look up history — couples a piece to the game log.
`Move` carries a `MoveType`.	One move can move two pieces (castling) or capture off-square (en passant).	Special-casing in `Game` (the course's `move_castling_rook`), which breaks "rules live in one place".
Command + Memento for undo. `Move` is the command (an action stored as an object); `Snapshot` is the memento (a saved copy of an object's state that can be restored later) of the state before it.	Restoring a snapshot is always correct, even for promotion/castling/en passant, with no inverse logic.	Pure Command with `undo()` per move type — works, but every special move needs a hand-written inverse, a common bug source.
Factory method `Piece.create(type, color)`.	Used by board set-up and promotion.	`new` calls scattered around.
`Game` methods are `synchronized`.	Two player connections may call `move` / `resign` at the same time.	No locking — race (result depends on which thread runs first) between a move and a resignation.

Memento vs incremental undo — trade-off. A snapshot of an 8×8 reference array is 64 references (~0.5 KB): nothing for a human game (a few hundred moves). An engine searching millions of positions would instead store only what the move changed (captured piece, old castling rights, old en passant square) and reverse it. Know both.

Java implementation

Package `lld.ch17.chess`.

```java
// File: code/lld/ch17/chess/Color.java
package lld.ch17.chess;

public enum Color {
    WHITE, BLACK;

    public Color opposite() { return this == WHITE ? BLACK : WHITE; }
}
```

```java
// File: code/lld/ch17/chess/PieceType.java
package lld.ch17.chess;

public enum PieceType {
    KING('K'), QUEEN('Q'), ROOK('R'), BISHOP('B'), KNIGHT('N'), PAWN('P');

    private final char letter;

    PieceType(char letter) { this.letter = letter; }

    public char letter() { return letter; }
}
```

```java
// File: code/lld/ch17/chess/Position.java
package lld.ch17.chess;

/** A square. file 0..7 = a..h, rank 0..7 = 1..8. Immutable value object. */
public record Position(int file, int rank) {
    public Position {
        if (!onBoard(file, rank)) throw new IllegalArgumentException("off board: " + file + "," + rank);
    }

    public static boolean onBoard(int file, int rank) { return file >= 0 && file < 8 && rank >= 0 && rank < 8; }

    /** Parses algebraic notation such as "e4". */
    public static Position of(String sq) {
        if (sq == null || sq.length() != 2) throw new IllegalArgumentException("bad square: " + sq);
        return new Position(sq.charAt(0) - 'a', sq.charAt(1) - '1');
    }

    @Override public String toString() { return "" + (char) ('a' + file) + (rank + 1); }
}
```

```java
// File: code/lld/ch17/chess/Piece.java
package lld.ch17.chess;

import java.util.List;

/**
 * Every piece answers "which squares can I reach from here on this board?" in its own way
 * (polymorphism). Pieces are immutable: moving a piece moves the reference, so a board copy is cheap
 * and undo can simply restore an old board. Rules that need history (castling rights, en passant)
 * live in Rules/Game, not here.
 */
public abstract sealed class Piece permits King, Knight, Pawn, SlidingPiece {
    private final Color color;

    protected Piece(Color color) { this.color = color; }

    public Color color() { return color; }

    public abstract PieceType type();

    /** Squares this piece may move to by its ordinary rules (captures included, own-king safety NOT checked). */
    public abstract List<Position> targets(Board board, Position from);

    /** Squares this piece attacks. Same as targets for every piece except the pawn. */
    public List<Position> attacks(Board board, Position from) { return targets(board, from); }

    /** Empty square or enemy piece. */
    protected boolean canLandOn(Board board, Position p) {
        Piece other = board.get(p);
        return other == null || other.color != color;
    }

    public char symbol() {
        char c = type().letter();
        return color == Color.WHITE ? c : Character.toLowerCase(c);
    }

    /** Simple factory used for set-up and promotion. */
    public static Piece create(PieceType type, Color color) {
        return switch (type) {
            case KING -> new King(color);
            case QUEEN -> new Queen(color);
            case ROOK -> new Rook(color);
            case BISHOP -> new Bishop(color);
            case KNIGHT -> new Knight(color);
            case PAWN -> new Pawn(color);
        };
    }
}
```

```java
// File: code/lld/ch17/chess/StepPieces.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Shared helper for pieces that jump a fixed set of offsets (king, knight). */
final class StepPieces {
    private StepPieces() {}

    static List<Position> step(Piece piece, Board board, Position from, int[][] offsets) {
        List<Position> out = new ArrayList<>();
        for (int[] d : offsets) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            if (!Position.onBoard(f, r)) continue;
            Position p = new Position(f, r);
            if (piece.canLandOn(board, p)) out.add(p);
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/King.java
package lld.ch17.chess;

import java.util.List;

/** One square in any direction. Castling is a two-piece move, so Rules generates it, not King. */
public final class King extends Piece {
    private static final int[][] OFFSETS = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};

    public King(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.KING; }

    @Override public List<Position> targets(Board board, Position from) {
        return StepPieces.step(this, board, from, OFFSETS);
    }
}
```

```java
// File: code/lld/ch17/chess/Knight.java
package lld.ch17.chess;

import java.util.List;

/** L-shaped jump; the only piece that ignores pieces in between. */
public final class Knight extends Piece {
    private static final int[][] OFFSETS = {{1,2},{2,1},{2,-1},{1,-2},{-1,-2},{-2,-1},{-2,1},{-1,2}};

    public Knight(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.KNIGHT; }

    @Override public List<Position> targets(Board board, Position from) {
        return StepPieces.step(this, board, from, OFFSETS);
    }
}
```

```java
// File: code/lld/ch17/chess/SlidingPiece.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Rook, bishop and queen slide until blocked. The "is the path clear" loop is written once, here. */
public abstract sealed class SlidingPiece extends Piece permits Rook, Bishop, Queen {
    protected static final int[][] STRAIGHT = {{1,0},{-1,0},{0,1},{0,-1}};
    protected static final int[][] DIAGONAL = {{1,1},{1,-1},{-1,1},{-1,-1}};

    protected SlidingPiece(Color color) { super(color); }

    protected abstract int[][] directions();

    @Override public final List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        for (int[] d : directions()) {
            int f = from.file() + d[0], r = from.rank() + d[1];
            while (Position.onBoard(f, r)) {
                Position p = new Position(f, r);
                Piece occupant = board.get(p);
                if (occupant == null) {
                    out.add(p);
                } else {
                    if (occupant.color() != color()) out.add(p);   // capture, then stop
                    break;
                }
                f += d[0];
                r += d[1];
            }
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/Rook.java
package lld.ch17.chess;

public final class Rook extends SlidingPiece {
    public Rook(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.ROOK; }
    @Override protected int[][] directions() { return STRAIGHT; }
}
```

```java
// File: code/lld/ch17/chess/Bishop.java
package lld.ch17.chess;

public final class Bishop extends SlidingPiece {
    public Bishop(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.BISHOP; }
    @Override protected int[][] directions() { return DIAGONAL; }
}
```

```java
// File: code/lld/ch17/chess/Queen.java
package lld.ch17.chess;

public final class Queen extends SlidingPiece {
    private static final int[][] ALL = {{1,0},{-1,0},{0,1},{0,-1},{1,1},{1,-1},{-1,1},{-1,-1}};
    public Queen(Color color) { super(color); }
    @Override public PieceType type() { return PieceType.QUEEN; }
    @Override protected int[][] directions() { return ALL; }
}
```

```java
// File: code/lld/ch17/chess/Pawn.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/** Moves straight forward but captures diagonally. En passant and promotion are added by Rules. */
public final class Pawn extends Piece {
    public Pawn(Color color) { super(color); }

    @Override public PieceType type() { return PieceType.PAWN; }

    private int dir() { return color() == Color.WHITE ? 1 : -1; }

    @Override public List<Position> targets(Board board, Position from) {
        List<Position> out = new ArrayList<>();
        int f = from.file(), r = from.rank() + dir();
        if (Position.onBoard(f, r) && board.get(new Position(f, r)) == null) {
            out.add(new Position(f, r));
            int startRank = color() == Color.WHITE ? 1 : 6;
            int r2 = r + dir();
            if (from.rank() == startRank && board.get(new Position(f, r2)) == null) out.add(new Position(f, r2));
        }
        for (Position p : attacks(board, from)) {
            Piece victim = board.get(p);
            if (victim != null && victim.color() != color()) out.add(p);
        }
        return out;
    }

    /** A pawn attacks both forward diagonals whether or not anything stands there. */
    @Override public List<Position> attacks(Board board, Position from) {
        List<Position> out = new ArrayList<>(2);
        int r = from.rank() + dir();
        for (int df : new int[]{-1, 1}) {
            int f = from.file() + df;
            if (Position.onBoard(f, r)) out.add(new Position(f, r));
        }
        return out;
    }
}
```

```java
// File: code/lld/ch17/chess/Board.java
package lld.ch17.chess;

/** 8x8 grid of (immutable) pieces. Knows geometry and attacks; knows nothing about turns or history. */
public final class Board {
    private final Piece[][] squares = new Piece[8][8]; // [file][rank]

    public static Board standard() {
        Board b = new Board();
        PieceType[] back = {PieceType.ROOK, PieceType.KNIGHT, PieceType.BISHOP, PieceType.QUEEN,
                            PieceType.KING, PieceType.BISHOP, PieceType.KNIGHT, PieceType.ROOK};
        for (int f = 0; f < 8; f++) {
            b.squares[f][0] = Piece.create(back[f], Color.WHITE);
            b.squares[f][1] = new Pawn(Color.WHITE);
            b.squares[f][6] = new Pawn(Color.BLACK);
            b.squares[f][7] = Piece.create(back[f], Color.BLACK);
        }
        return b;
    }

    public Piece get(Position p) { return squares[p.file()][p.rank()]; }

    void set(Position p, Piece piece) { squares[p.file()][p.rank()] = piece; }

    /** Pieces are immutable, so copying the references is a full, independent copy of the position. */
    public Board copy() {
        Board b = new Board();
        for (int f = 0; f < 8; f++) b.squares[f] = squares[f].clone();
        return b;
    }

    public Position kingOf(Color color) {
        for (int f = 0; f < 8; f++)
            for (int r = 0; r < 8; r++) {
                Piece p = squares[f][r];
                if (p != null && p.type() == PieceType.KING && p.color() == color) return new Position(f, r);
            }
        throw new IllegalStateException("no " + color + " king on board");
    }

    public boolean isAttacked(Position target, Color by) {
        for (int f = 0; f < 8; f++)
            for (int r = 0; r < 8; r++) {
                Piece p = squares[f][r];
                if (p != null && p.color() == by && p.attacks(this, new Position(f, r)).contains(target)) return
true;
            }
        return false;
    }

    public String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 7; r >= 0; r--) {
            sb.append("  ").append(r + 1).append(' ');
            for (int f = 0; f < 8; f++) {
                Piece p = squares[f][r];
                sb.append(' ').append(p == null ? '.' : p.symbol());
            }
            sb.append('\n');
        }
        return sb.append("    a b c d e f g h").toString();
    }
}
```

```java
// File: code/lld/ch17/chess/MoveType.java
package lld.ch17.chess;

public enum MoveType { NORMAL, CASTLE_KINGSIDE, CASTLE_QUEENSIDE, EN_PASSANT, PROMOTION }
```

```java
// File: code/lld/ch17/chess/Move.java
package lld.ch17.chess;

/** A move as data (the Command's payload). promotion is null unless type == PROMOTION. */
public record Move(Position from, Position to, MoveType type, PieceType promotion) {
    public static Move normal(Position from, Position to) { return new Move(from, to, MoveType.NORMAL, null); }
}
```

```java
// File: code/lld/ch17/chess/CastlingRights.java
package lld.ch17.chess;

/** Which castles are still allowed. Lost forever once the king or that rook moves (or the rook is captured). */
public record CastlingRights(boolean whiteKingSide, boolean whiteQueenSide, boolean blackKingSide, boolean
blackQueenSide) {
    public static CastlingRights all() { return new CastlingRights(true, true, true, true); }

    public boolean kingSide(Color c) { return c == Color.WHITE ? whiteKingSide : blackKingSide; }
    public boolean queenSide(Color c) { return c == Color.WHITE ? whiteQueenSide : blackQueenSide; }

    /** Returns the rights after a move from -> to by a piece of the given type. */
    public CastlingRights after(PieceType moved, Color mover, Position from, Position to) {
        boolean wk = whiteKingSide, wq = whiteQueenSide, bk = blackKingSide, bq = blackQueenSide;
        if (moved == PieceType.KING) {
            if (mover == Color.WHITE) { wk = false; wq = false; } else { bk = false; bq = false; }
        }
        for (Position p : new Position[]{from, to}) {          // a rook leaving or being captured on its corner
            if (p.equals(new Position(0, 0))) wq = false;
            if (p.equals(new Position(7, 0))) wk = false;
            if (p.equals(new Position(0, 7))) bq = false;
            if (p.equals(new Position(7, 7))) bk = false;
        }
        return new CastlingRights(wk, wq, bk, bq);
    }
}
```

```java
// File: code/lld/ch17/chess/Rules.java
package lld.ch17.chess;

import java.util.ArrayList;
import java.util.List;

/**
 * Stateless rules engine: generates legal moves (including castling, en passant, promotion)
 * and applies a move to a board. "Legal" = pseudo-legal AND does not leave your own king attacked.
 */
public final class Rules {
    private Rules() {}

    private static final PieceType[] PROMOTIONS = {PieceType.QUEEN, PieceType.ROOK, PieceType.BISHOP,
PieceType.KNIGHT};

    public static boolean isInCheck(Board board, Color side) {
        return board.isAttacked(board.kingOf(side), side.opposite());
    }

    /** Moves that follow the pieces' movement rules, before the king-safety filter. */
    public static List<Move> pseudoLegalMoves(Board board, Color side, CastlingRights rights, Position enPassant)
{
        List<Move> moves = new ArrayList<>();
        int lastRank = side == Color.WHITE ? 7 : 0;
        for (int f = 0; f < 8; f++) {
            for (int r = 0; r < 8; r++) {
                Position from = new Position(f, r);
                Piece piece = board.get(from);
                if (piece == null || piece.color() != side) continue;
                for (Position to : piece.targets(board, from)) {
                    if (piece.type() == PieceType.PAWN && to.rank() == lastRank) {
                        for (PieceType promo : PROMOTIONS) moves.add(new Move(from, to, MoveType.PROMOTION,
promo));
                    } else {
                        moves.add(Move.normal(from, to));
                    }
                }
                if (piece.type() == PieceType.PAWN && enPassant != null && piece.attacks(board,
from).contains(enPassant)) {
                    moves.add(new Move(from, enPassant, MoveType.EN_PASSANT, null));
                }
            }
        }
        addCastling(board, side, rights, moves);
        return moves;
    }

    private static void addCastling(Board board, Color side, CastlingRights rights, List<Move> moves) {
        int r = side == Color.WHITE ? 0 : 7;
        Position kingSq = new Position(4, r);
        Piece king = board.get(kingSq);
        if (king == null || king.type() != PieceType.KING || king.color() != side) return;
        Color enemy = side.opposite();
        if (board.isAttacked(kingSq, enemy)) return;                        // cannot castle out of check
        if (rights.kingSide(side) && isOwnRook(board, new Position(7, r), side)
                && empty(board, r, 5, 6)
                && !board.isAttacked(new Position(5, r), enemy)             // cannot pass through check
                && !board.isAttacked(new Position(6, r), enemy)) {
            moves.add(new Move(kingSq, new Position(6, r), MoveType.CASTLE_KINGSIDE, null));
        }
        if (rights.queenSide(side) && isOwnRook(board, new Position(0, r), side)
                && empty(board, r, 1, 2, 3)
                && !board.isAttacked(new Position(3, r), enemy)
                && !board.isAttacked(new Position(2, r), enemy)) {
            moves.add(new Move(kingSq, new Position(2, r), MoveType.CASTLE_QUEENSIDE, null));
        }
    }

    private static boolean isOwnRook(Board b, Position p, Color side) {
```

```
        Piece piece = b.get(p);
        return piece != null && piece.type() == PieceType.ROOK && piece.color() == side;
    }

    private static boolean empty(Board b, int rank, int... files) {
        for (int f : files) if (b.get(new Position(f, rank)) != null) return false;
        return true;
    }

    /** Explains why a king move of two squares (a castling attempt) is not legal right now. */
    public static String whyCannotCastle(Board board, Color side, CastlingRights rights, Position kingTo) {
        int r = side == Color.WHITE ? 0 : 7;
        if (kingTo.rank() != r || (kingTo.file() != 6 && kingTo.file() != 2)) return "that is not a castling
square";
        boolean kingSide = kingTo.file() == 6;
        if (!(kingSide ? rights.kingSide(side) : rights.queenSide(side))
                || !isOwnRook(board, new Position(kingSide ? 7 : 0, r), side))
            return "the king or that rook has already moved";
        if (!(kingSide ? empty(board, r, 5, 6) : empty(board, r, 1, 2, 3)))
            return "the squares between king and rook are not empty";
        Color enemy = side.opposite();
        if (board.isAttacked(new Position(4, r), enemy)) return "the king is in check";
        Position crossed = new Position(kingSide ? 5 : 3, r);
        if (board.isAttacked(crossed, enemy)) return "the king would pass through the attacked square " + crossed;
        if (board.isAttacked(kingTo, enemy)) return "the king would land on the attacked square " + kingTo;
        return "it is not legal in this position";
    }

    public static List<Move> legalMoves(Board board, Color side, CastlingRights rights, Position enPassant) {
        List<Move> legal = new ArrayList<>();
        for (Move m : pseudoLegalMoves(board, side, rights, enPassant)) {
            Board trial = board.copy();
            apply(trial, m);
            if (!isInCheck(trial, side)) legal.add(m);
        }
        return legal;
    }

    /** Mutates the board. Handles the extra piece movements of the special moves. */
    public static void apply(Board board, Move m) {
        Piece piece = board.get(m.from());
        board.set(m.from(), null);
        board.set(m.to(), piece);
        switch (m.type()) {
            case EN_PASSANT -> board.set(new Position(m.to().file(), m.from().rank()), null);   // the passed pawn
            case PROMOTION -> board.set(m.to(), Piece.create(m.promotion(), piece.color()));
            case CASTLE_KINGSIDE -> moveRook(board, m.from().rank(), 7, 5);
            case CASTLE_QUEENSIDE -> moveRook(board, m.from().rank(), 0, 3);
            case NORMAL -> { }
        }
    }

    private static void moveRook(Board board, int rank, int fromFile, int toFile) {
        Position from = new Position(fromFile, rank);
        board.set(new Position(toFile, rank), board.get(from));
        board.set(from, null);
    }
}
```

```
// File: code/lld/ch17/chess/GameStatus.java
package lld.ch17.chess;

public enum GameStatus { ACTIVE, CHECKMATE, STALEMATE, RESIGNATION, FORFEIT }
```

```
// File: code/lld/ch17/chess/Player.java
package lld.ch17.chess;

public record Player(String name, Color color) {}
```

```java
// File: code/lld/ch17/chess/IllegalMoveException.java
package lld.ch17.chess;

public final class IllegalMoveException extends RuntimeException {
    private static final long serialVersionUID = 1L;

    public IllegalMoveException(String message) { super(message); }
}
```

```java
// File: code/lld/ch17/chess/Game.java
package lld.ch17.chess;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;
import java.util.Optional;
import java.util.Random;

/**
 * Controls one game: whose turn it is, validates and plays moves, detects check/checkmate/stalemate,
 * keeps the move log, and supports undo.
 *
 * Undo = Command + Memento: each history entry stores the move (the command) and a snapshot of the
 * state before it (the memento). Undo restores the snapshot, which is always correct even for
 * castling, en passant and promotion.
 */
public final class Game {
    /** Memento: everything needed to restore the position exactly. */
    private record Snapshot(Board board, Color turn, CastlingRights rights, Position enPassant,
                            GameStatus status, Color winner) {}

    private record HistoryEntry(Player player, Move move, String notation, Snapshot before) {}

    private final Player white, black;
    private Board board = Board.standard();
    private Color turn = Color.WHITE;                       // white always moves first
    private CastlingRights rights = CastlingRights.all();
    private Position enPassant;                             // square a pawn just skipped over, or null
    private GameStatus status = GameStatus.ACTIVE;
    private Color winner;
    private final Deque<HistoryEntry> history = new ArrayDeque<>();

    public Game(Player white, Player black) {
        if (white.color() != Color.WHITE || black.color() != Color.BLACK) throw new
IllegalArgumentException("colors mismatch");
        this.white = white;
        this.black = black;
    }

    /** The course's rule: each player is randomly assigned a side. Pass a seeded Random for repeatable tests. */
    public static Game withRandomSides(String nameA, String nameB, Random rng) {
        boolean aIsWhite = rng.nextBoolean();
        return aIsWhite ? new Game(new Player(nameA, Color.WHITE), new Player(nameB, Color.BLACK))
                        : new Game(new Player(nameB, Color.WHITE), new Player(nameA, Color.BLACK));
    }

    public synchronized Player player(Color c) { return c == Color.WHITE ? white : black; }

    public synchronized String move(String from, String to) { return move(from, to, null); }

    public synchronized String move(String fromSq, String toSq, PieceType promotion) {
        if (status != GameStatus.ACTIVE) throw new IllegalMoveException("game is over: " + status);
        Position from = Position.of(fromSq), to = Position.of(toSq);
        Piece piece = board.get(from);
        if (piece == null) throw new IllegalMoveException("no piece on " + from);
        if (piece.color() != turn) throw new IllegalMoveException("it is " + turn + "'s turn");

        List<Move> legal = Rules.legalMoves(board, turn, rights, enPassant);
        Optional<Move> chosen = legal.stream()
                .filter(m -> m.from().equals(from) && m.to().equals(to))
                .filter(m -> m.type() != MoveType.PROMOTION
                        || m.promotion() == (promotion == null ? PieceType.QUEEN : promotion))
                .findFirst();
        if (chosen.isEmpty()) {
            if (piece.type() == PieceType.KING && Math.abs(to.file() - from.file()) == 2)
                throw new IllegalMoveException("cannot castle " + fromSq + "-" + toSq + ": "
                        + Rules.whyCannotCastle(board, turn, rights, to));
            boolean reachable = Rules.pseudoLegalMoves(board, turn, rights, enPassant).stream()
                    .anyMatch(m -> m.from().equals(from) && m.to().equals(to));
```

```
            throw new IllegalMoveException(reachable
                    ? fromSq + "-" + toSq + " would leave the " + turn + " king in check"
                    : piece.type() + " on " + fromSq + " cannot move to " + toSq);
        }
        return play(chosen.get(), piece);
    }

    private String play(Move m, Piece piece) {
        Player mover = turn == Color.WHITE ? white : black;
        Snapshot before = new Snapshot(board.copy(), turn, rights, enPassant, status, winner);
        boolean capture = board.get(m.to()) != null || m.type() == MoveType.EN_PASSANT;

        Rules.apply(board, m);
        rights = rights.after(piece.type(), turn, m.from(), m.to());
        enPassant = (piece.type() == PieceType.PAWN && Math.abs(m.to().rank() - m.from().rank()) == 2)
                ? new Position(m.from().file(), (m.from().rank() + m.to().rank()) / 2) : null;
        turn = turn.opposite();

        // End-of-game detection: the side to move has no legal move -> checkmate if in check, else stalemate.
        boolean check = Rules.isInCheck(board, turn);
        boolean noMoves = Rules.legalMoves(board, turn, rights, enPassant).isEmpty();
        if (noMoves) {
            status = check ? GameStatus.CHECKMATE : GameStatus.STALEMATE;
            winner = check ? mover.color() : null;
        }
        String notation = notation(m, piece, capture) + (check ? (noMoves ? "#" : "+") : "");
        history.push(new HistoryEntry(mover, m, notation, before));
        return notation;
    }

    private static String notation(Move m, Piece piece, boolean capture) {
        return switch (m.type()) {
            case CASTLE_KINGSIDE -> "O-O";
            case CASTLE_QUEENSIDE -> "O-O-O";
            default -> {
                String prefix = piece.type() == PieceType.PAWN ? "" : String.valueOf(piece.type().letter());
                String s = prefix + m.from() + (capture ? "x" : "-") + m.to();
                if (m.type() == MoveType.PROMOTION) s += "=" + m.promotion().letter();
                if (m.type() == MoveType.EN_PASSANT) s += " e.p.";
                yield s;
            }
        };
    }

    /** Undo the last move (e.g. for analysis mode or a take-back both players agree on). */
    public synchronized String undo() {
        if (history.isEmpty()) throw new IllegalStateException("nothing to undo");
        HistoryEntry last = history.pop();
        Snapshot s = last.before();
        board = s.board(); turn = s.turn(); rights = s.rights(); enPassant = s.enPassant();
        status = s.status(); winner = s.winner();
        return last.notation();
    }

    public synchronized void resign(Color who) {
        if (status != GameStatus.ACTIVE) throw new IllegalStateException("game is over");
        status = GameStatus.RESIGNATION;
        winner = who.opposite();
    }

    /** Forfeit: the player loses without choosing to (left the game, ran out of time, broke the rules of the
site). */
    public synchronized void forfeit(Color who) {
        if (status != GameStatus.ACTIVE) throw new IllegalStateException("game is over");
        status = GameStatus.FORFEIT;
        winner = who.opposite();
    }

    public synchronized boolean isInCheck() { return Rules.isInCheck(board, turn); }
    public synchronized int legalMoveCount() { return Rules.legalMoves(board, turn, rights, enPassant).size(); }
    public synchronized GameStatus status() { return status; }
```

```
    public synchronized Color winner() { return winner; }
    public synchronized Color turn() { return turn; }
    public synchronized String render() { return board.render(); }

    /** The move log, oldest first, numbered like a score sheet. */
    public synchronized String moveLog() {
        List<HistoryEntry> list = new java.util.ArrayList<>(history);
        java.util.Collections.reverse(list);
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < list.size(); i++) {
            if (i % 2 == 0) sb.append(i / 2 + 1).append(". ");
            sb.append(list.get(i).notation()).append(' ');
        }
        return sb.toString().trim();
    }
}
```

```java
// File: code/lld/ch17/chess/Main.java
package lld.ch17.chess;

import java.util.Random;

public final class Main {
    private static Game newGame() { return new Game(new Player("Ann", Color.WHITE), new Player("Ben",
Color.BLACK)); }

    private static void play(Game g, String... moves) {
        for (String mv : moves) {
            String[] parts = mv.split("-");
            PieceType promo = parts[1].length() == 3 ? PieceType.valueOf(switch (parts[1].charAt(2)) {
                case 'Q' -> "QUEEN"; case 'R' -> "ROOK"; case 'B' -> "BISHOP"; default -> "KNIGHT"; }) : null;
            g.move(parts[0], parts[1].substring(0, 2), promo);
        }
    }

    private static void tryMove(Game g, String from, String to, PieceType promo) {
        try {
            System.out.println("  " + from + "-" + to + " played as " + g.move(from, to, promo));
        } catch (IllegalMoveException e) {
            System.out.println("  " + from + "-" + to + " rejected: " + e.getMessage());
        }
    }

    public static void main(String[] args) {
        System.out.println("== 1. Validation, Scholar's mate, undo ==");
        Game g = newGame();
        tryMove(g, "e7", "e5", null);                // black out of turn
        tryMove(g, "e1", "e3", null);                // king cannot jump two squares
        tryMove(g, "a1", "a4", null);                // rook blocked by its own pawn
        play(g, "e2-e4", "e7-e5", "f1-c4", "b8-c6", "d1-h5", "g8-f6");
        tryMove(g, "h5", "f7", null);
        System.out.println("  Log: " + g.moveLog());
        System.out.println("  Status: " + g.status() + ", winner " + g.winner());
        System.out.println(g.render());
        System.out.println("  undo " + g.undo() + " -> status " + g.status() + ", " + g.turn() + " to move, "
                + g.legalMoveCount() + " legal moves");

        System.out.println("\n== 2. En passant, castling, promotion, and a move that ignores check ==");
        Game s = newGame();
        play(s, "e2-e4", "g8-f6", "e4-e5", "d7-d5");
        tryMove(s, "e5", "d6", null);                // en passant
        play(s, "b8-c6", "g1-f3", "a7-a5", "f1-c4", "a5-a4");
        tryMove(s, "e1", "g1", null);                // castle kingside
        play(s, "a4-a3", "d6-c7", "a3-b2");
        tryMove(s, "c7", "d8", PieceType.QUEEN);     // capture + promote, gives check
        System.out.println("  Black in check? " + s.isInCheck());
        tryMove(s, "b2", "a1", PieceType.KNIGHT);    // ignores the check -> illegal
        tryMove(s, "e8", "d8", null);                // king takes the new queen
        System.out.println("  Log: " + s.moveLog());
        System.out.println(s.render());

        System.out.println("\n== 3. Stalemate (Sam Loyd's 10-move stalemate) ==");
        Game st = newGame();
        play(st, "e2-e3", "a7-a5", "d1-h5", "a8-a6", "h5-a5", "h7-h5", "h2-h4", "a6-h6",
                 "a5-c7", "f7-f6", "c7-d7", "e8-f7", "d7-b7", "d8-d3", "b7-b8", "d3-h7",
                 "b8-c8", "f7-g6", "c8-e6");
        System.out.println("  Log: " + st.moveLog());
        System.out.println("  Status: " + st.status() + ", black in check? " + st.isInCheck()
                + ", black legal moves: " + st.legalMoveCount());
        System.out.println(st.render());

        System.out.println("\n== 4. Castling rules: through an attacked square, and out of check ==");
        Game c1 = newGame();
        play(c1, "g2-g3", "b7-b6", "f1-h3", "c8-a6", "g1-f3", "b8-c6", "e2-e4", "g8-f6");
        tryMove(c1, "e1", "g1", null);               // f1 is attacked by the bishop on a6
        Game c2 = newGame();
        play(c2, "e2-e4", "d7-d6", "d2-d4", "g8-f6", "b1-c3", "g7-g6", "g1-f3", "f8-g7", "f1-b5");
```

```
        tryMove(c2, "e8", "g8", null);               // black is in check from the b5 bishop
        tryMove(c2, "c7", "c6", null);               // block the check
        tryMove(c2, "e1", "g1", null);
        tryMove(c2, "e8", "g8", null);               // no longer in check: now legal

        System.out.println("\n== 5. Random sides, resignation and forfeit ==");
        Game r = Game.withRandomSides("Ann", "Ben", new Random(7));
        System.out.println("  white = " + r.player(Color.WHITE).name() + ", black = " +
r.player(Color.BLACK).name());
        play(r, "f2-f3", "e7-e5");
        r.resign(Color.WHITE);
        System.out.println("  Status: " + r.status() + ", winner " + r.winner());
        tryMove(r, "g2", "g4", null);
        Game f = newGame();
        f.forfeit(Color.BLACK);                       // e.g. black disconnected and did not come back
        System.out.println("  Status: " + f.status() + ", winner " + f.winner());
    }
}
```

Output:

```
== 1. Validation, Scholar's mate, undo ==
  e7-e5 rejected: it is WHITE's turn
  e1-e3 rejected: KING on e1 cannot move to e3
  a1-a4 rejected: ROOK on a1 cannot move to a4
  h5-f7 played as Qh5xf7#
  Log: 1. e2-e4 e7-e5 2. Bf1-c4 Nb8-c6 3. Qd1-h5 Ng8-f6 4. Qh5xf7#
  Status: CHECKMATE, winner WHITE
    8  r . b q k b . r
    7  p p p p . Q p p
    6  . . n . . n . .
    5  . . . . p . . .
    4  . . B . P . . .
    3  . . . . . . . .
    2  P P P P . P P P
    1  R N B . K . N R
       a b c d e f g h
  undo Qh5xf7# -> status ACTIVE, WHITE to move, 43 legal moves

== 2. En passant, castling, promotion, and a move that ignores check ==
  e5-d6 played as e5xd6 e.p.
  e1-g1 played as O-O
  c7-d8 played as c7xd8=Q+
  Black in check? true
  b2-a1 rejected: b2-a1 would leave the BLACK king in check
  e8-d8 played as Ke8xd8
  Log: 1. e2-e4 Ng8-f6 2. e4-e5 d7-d5 3. e5xd6 e.p. Nb8-c6 4. Ng1-f3 a7-a5 5. Bf1-c4 a5-a4 6. O-O a4-a3 7. d6xc7
a3xb2 8. c7xd8=Q+ Ke8xd8
    8  r . b k . b . r
    7  . p . . p p p p
    6  . . n . . n . .
    5  . . . . . . . .
    4  . . B . . . . .
    3  . . . . . N . .
    2  P p P P . P P P
    1  R N B Q . R K .
       a b c d e f g h

== 3. Stalemate (Sam Loyd's 10-move stalemate) ==
  Log: 1. e2-e3 a7-a5 2. Qd1-h5 Ra8-a6 3. Qh5xa5 h7-h5 4. h2-h4 Ra6-h6 5. Qa5xc7 f7-f6 6. Qc7xd7+ Ke8-f7 7. Qd7xb7
Qd8-d3 8. Qb7xb8 Qd3-h7 9. Qb8xc8 Kf7-g6 10. Qc8-e6
  Status: STALEMATE, black in check? false, black legal moves: 0
    8  . . . . . b n r
    7  . . . . p . p q
    6  . . . . Q p k r
    5  . . . . . . . p
    4  . . . . . . . P
    3  . . . . P . . .
    2  P P P P . P P .
    1  R N B . K B N R
       a b c d e f g h

== 4. Castling rules: through an attacked square, and out of check ==
  e1-g1 rejected: cannot castle e1-g1: the king would pass through the attacked square f1
  e8-g8 rejected: cannot castle e8-g8: the king is in check
  c7-c6 played as c7-c6
  e1-g1 played as O-O
  e8-g8 played as O-O

== 5. Random sides, resignation and forfeit ==
  white = Ann, black = Ben
  Status: RESIGNATION, winner BLACK
  g2-g4 rejected: game is over: RESIGNATION
  Status: FORFEIT, winner WHITE
```

Reading the output. Part 1 shows validation (out of turn, illegal king jump, blocked rook), then Scholar's mate (`Qh5xf7#`), and `undo()` returning the game to ACTIVE with white to move. Part 2 plays en passant (`e5xd6 e.p.` removes the black pawn from d5), kingside castling (`O-O`), and a capturing promotion that gives check (`c7xd8=Q+`); black's attempt to ignore the check is rejected, and the king captures the new queen. Part 3 is the famous 10-move stalemate composed by Sam Loyd: black is not in

check but has no legal move, so the game is a draw. Part 4 shows the castling guards: white may not castle through f1 while the a6 bishop attacks it, black may not castle out of check, and once the check is blocked both sides castle. Part 5 assigns sides at random (seeded) and shows resignation and forfeit.

Concurrency & edge cases

- **Two players, one game.** `Game`'s public methods are `synchronized`, so a move and a resignation cannot interleave (have their steps mixed together). For a server with thousands of games, each game gets its own lock (the object's monitor — the built-in lock every Java object carries, which `synchronized` uses) — games do not block each other.
- **Moving out of turn / game over / empty square** — rejected with a clear message before any rule work.
- **Pins and discovered checks** (a *pin* is a piece that cannot move because it shields its king; a *discovered check* is a check revealed when a blocking piece moves away) — handled automatically by the "apply to a copy and test the king" filter.
- **Castling through check** — explicitly tested for the squares the king crosses and lands on; the rook's crossing square on the queenside (b1/b8) may be attacked — that is legal. Section 4 of the output shows both refusals with their reason (`Rules.whyCannotCastle`): passing through an attacked f1, and castling while in check; once the check is blocked the same castle is accepted.
- **Castling rights** are lost when the king moves, when that rook moves, or when that rook is **captured** on its corner (we check both `from` and `to`).
- **En passant expires** after one move: the square is reset on every move unless a pawn just advanced two.
- **Promotion without a choice** defaults to a queen; underpromotion (knight, rook, bishop) is supported.
- **Not implemented, but should be named:** draw by threefold repetition (the same position occurs three times), the 50-move rule (50 moves each with no capture or pawn move), insufficient material (e.g. king vs king), clocks/timeouts, draw offers.

Extensions & follow-up questions

- **"Add threefold repetition."** — Keep a `Map<PositionKey, Integer>` where the key is (board, side to move, castling rights, en passant square). Production engines hash this (turn it into one compact number for quick comparison) with Zobrist hashing, which combines a random number per piece-and-square. Three occurrences → draw.
- **"Add a 50-move rule."** — A half-move (one player's move) clock reset on every pawn move or capture; at 100 half-moves either player may claim a draw.
- **"Add chess clocks."** — A `Clock` per player; the server stamps time on each move; a flag-fall (a player's time running out) ends the game (Observer — listeners register once and are told about every change — notifies both clients).
- **"Add an AI player."** — A `MoveStrategy` that picks from `legalMoves()`; a real one runs minimax (search ahead, assuming you pick your best move and the opponent picks the move worst for you) with alpha-beta pruning (skipping branches that provably cannot change the answer).
- **"Support Chess960 (a variant with a shuffled back row) or a new piece."** — New `Piece` subclass (e.g. an Archbishop = bishop + knight) and a different starting-position factory; existing pieces are untouched.
- **"Spectators and UI updates."** — Observer: `GameListener.onMove(notation)`. The course mentions a `GameController` (takes input, calls `Game`) and a `GameView` (redraws) — that is the MVC (Model–View–Controller: data, display and input handling kept in separate classes) split from Chapter 11.
- **"Why is `Position` a record?"** — It is a value: two `e4`s are equal. Records give `equals`/`hashCode` (the methods Java uses to decide if two objects are equal and to place them in hash tables) for free, which `List.contains` relies on.

Real-world example

Online chess sites such as Lichess and Chess.com treat the **server** as the source of truth (the one copy whose data wins whenever copies disagree): the board in your browser is only a view, and a move counts only after the server has accepted it as legal. Games are exported in PGN (Portable Game Notation — a

move log like ours) and positions in FEN (Forsyth–Edwards Notation — one line describing the board, side to move, castling rights and en passant square, i.e. exactly the fields of our `Snapshot`). "Takeback" is offered only when the opponent agrees, and then the server undoes the move — the same `undo()` our `Game` exposes.

In the interview

30-second answer: "`Board` is an 8×8 grid of immutable `Piece`s. `Piece` is abstract with `targets(board, from)`; each piece overrides it, and rook/bishop/queen share a `SlidingPiece` loop. A stateless `Rules` class generates pseudo-legal moves, adds castling, en passant and promotion using history kept in `Game` (castling rights, en passant square), and filters out moves that leave your king attacked. After each move, if the opponent has no legal move, it's checkmate if they're in check, otherwise stalemate. `Game` handles turns, status, a move log, and undo via Command plus Memento."

How to spend 30–45 minutes: 5 min requirements → 5 min entities → 10 min class diagram (piece hierarchy, why history lives outside pieces) → 15 min code `Piece`/`SlidingPiece`/`Pawn` and the legal-move filter → 5–10 min checkmate/stalemate, special moves, concurrency.

Likely follow-ups with answers

- *"How do you detect checkmate?"* — No legal moves and in check. Legal = pseudo-legal filtered by "king not attacked after the move".
- *"Where does castling live?"* — In `Rules`, because it moves two pieces and depends on history; `MoveType.CASTLE_*` tells `apply` to move the rook too.
- *"Why is `Box -> Piece` not composition?"* — Removing a piece from a square doesn't destroy the piece; it goes to the captured list or the log.
- *"How is undo safe with promotion?"* — The memento stores the whole prior board, so undo simply puts the old pawn back.

Self-check

1. **Q:** Why does `Pawn` override `attacks()`? **A:** A pawn moves straight but attacks diagonally, even onto empty squares; check detection needs the attack squares.
2. **Q:** What is the difference between checkmate and stalemate? **A:** Both mean no legal move; checkmate is while in check (loss), stalemate is while not in check (draw).
3. **Q:** When is en passant allowed? **A:** Only on the move immediately after an enemy pawn advanced two squares past your pawn's attack square.
4. **Q:** Name three conditions for castling. **A:** King and that rook have not moved; squares between empty; king not in check, not passing through or landing on an attacked square.
5. **Q:** Why are pieces immutable in this design? **A:** Board copies for trial moves and undo snapshots can share pieces safely.

TL;DR

- Polymorphic `Piece.targets()`; `SlidingPiece` shares the path loop; pawn overrides `attacks()`.
- `Rules`: pseudo-legal → add castling / en passant / promotion → drop moves leaving own king attacked.
- No legal moves: in check = checkmate, else stalemate.
- History-dependent rules live in `Game` / `Rules`, not in pieces. Undo = Command + Memento. `synchronized` game for online play.

Design Tic-Tac-Toe

Problem & clarifying questions

Tic-tac-toe is played on a grid. Players take turns placing their mark (X or O). The first to fill a whole **row, column or diagonal** wins; if the board fills with no winner, it is a **draw**. The classic game is 3×3 with two players; interviewers love to generalise it to **N×N** and ask for **fast win detection**.

Question	Assumed answer
Board size?	Any N ≥ 3 (classic N = 3).
Win condition?	A full line of N marks: any row, column, or one of the two main diagonals.
Players?	Two by default; the design allows more (each with its own mark).
Human or computer?	Both: each player has a pluggable **move strategy** (human input, random bot, rule-based bot, scripted test).
Invalid moves?	Rejected (occupied or off-board); the same player tries again.
UI?	Console rendering only; the engine exposes state for any UI.

Requirements

Functional

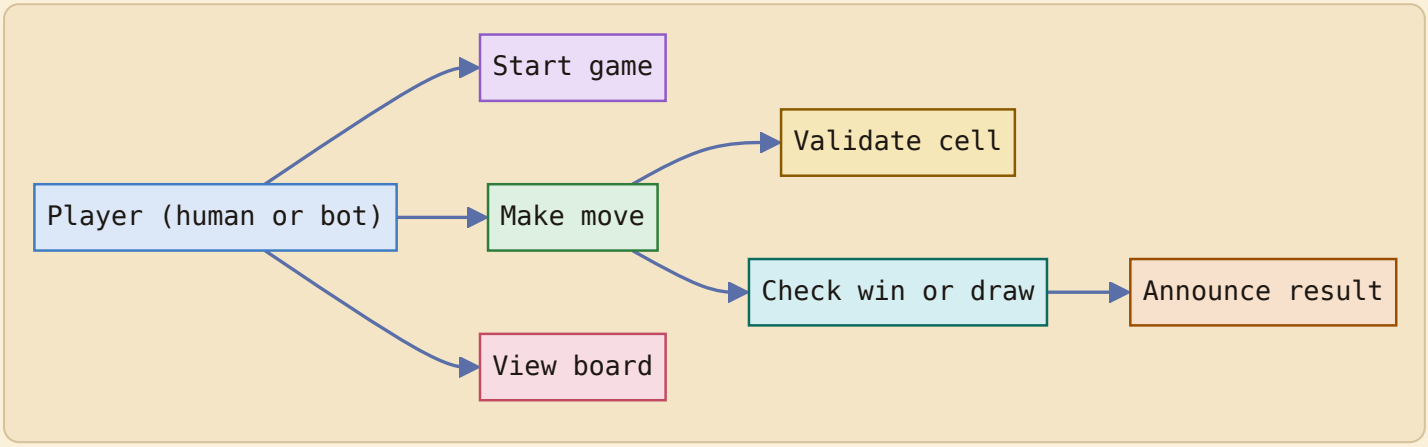
1. Create an N×N board and a game with two (or more) players, each with a unique mark.
2. Players take turns; the current player chooses an empty cell.
3. Validate moves: inside the board and empty.
4. After each move, detect a **win** (the mover completed a line) or a **draw** (board full).
5. Announce the result and show the board.

Non-functional

- Win check in **O(1)** per move (not a full O(N²) board scan).
- New player types (AI levels, network player) without touching `Game` (Open/Closed).
- Deterministic tests: random bots take a seeded `Random`; scripted players replay fixed moves.
- Separate games must not share state (many games can run at once on a server).

Use cases

Actors: **Player** (human or bot). Use cases: *start game* (choose N and players), *make move*, *view board*, *see result*. A system actor (the `Game`) validates moves, switches turns and declares the result.

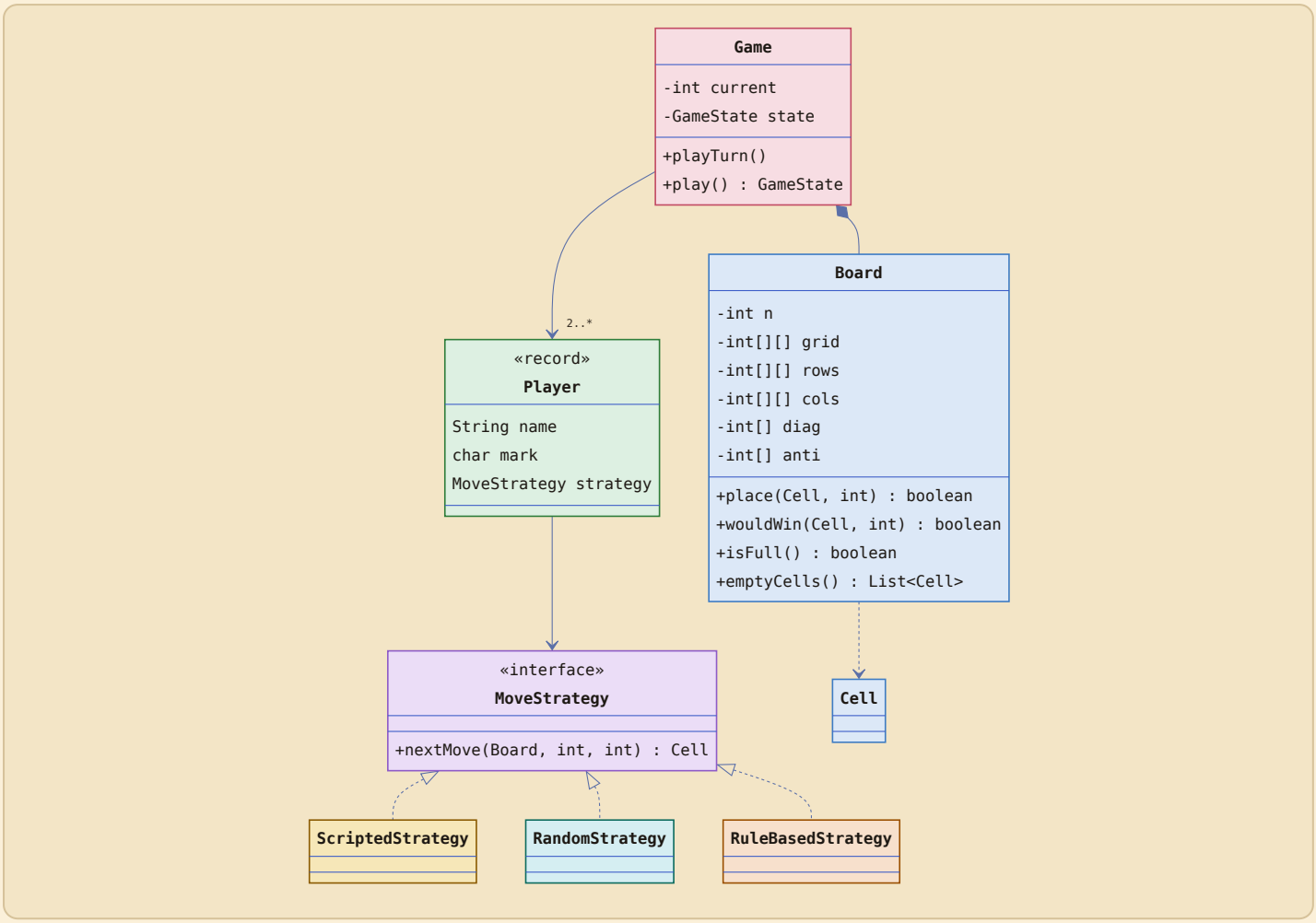


Core entities

Class	Responsibility
`Cell` (record)	Row + column of a square.
`Board`	The grid plus **per-player line counters**; `place` returns "did this win?" in O(1); `wouldWin` for bots; `isFull`, `emptyCells`, `render`.
`MoveStrategy` (interface)	Chooses the next cell. Implementations: `ScriptedStrategy`, `RandomStrategy`, `RuleBasedStrategy`.
`Player` (record)	Name, mark, strategy.
`GameState` (enum)	IN_PROGRESS, WON, DRAW.
`Game`	Turn order, move validation, state, winner, result.

Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set. That statement asks for: a 3×3 grid; two players taking turns with X and O; three in a row, column or diagonal wins; a full board with no winner is a draw; a way to display the grid and take moves; turn handling and move validation; announcing the winner or the draw. All of these are covered (the board size is generalised to N×N).



Key flows

O(1) win detection — how it works

A naive win check scans every row, column and diagonal after each move: O(N²). But a move at `(r, c)` can only complete **row r, column c, and (maybe) the two diagonals**. So we keep counters:

1. `rows[p][r]` = how many marks player `p` has in row `r`; `cols[p][c]` likewise.

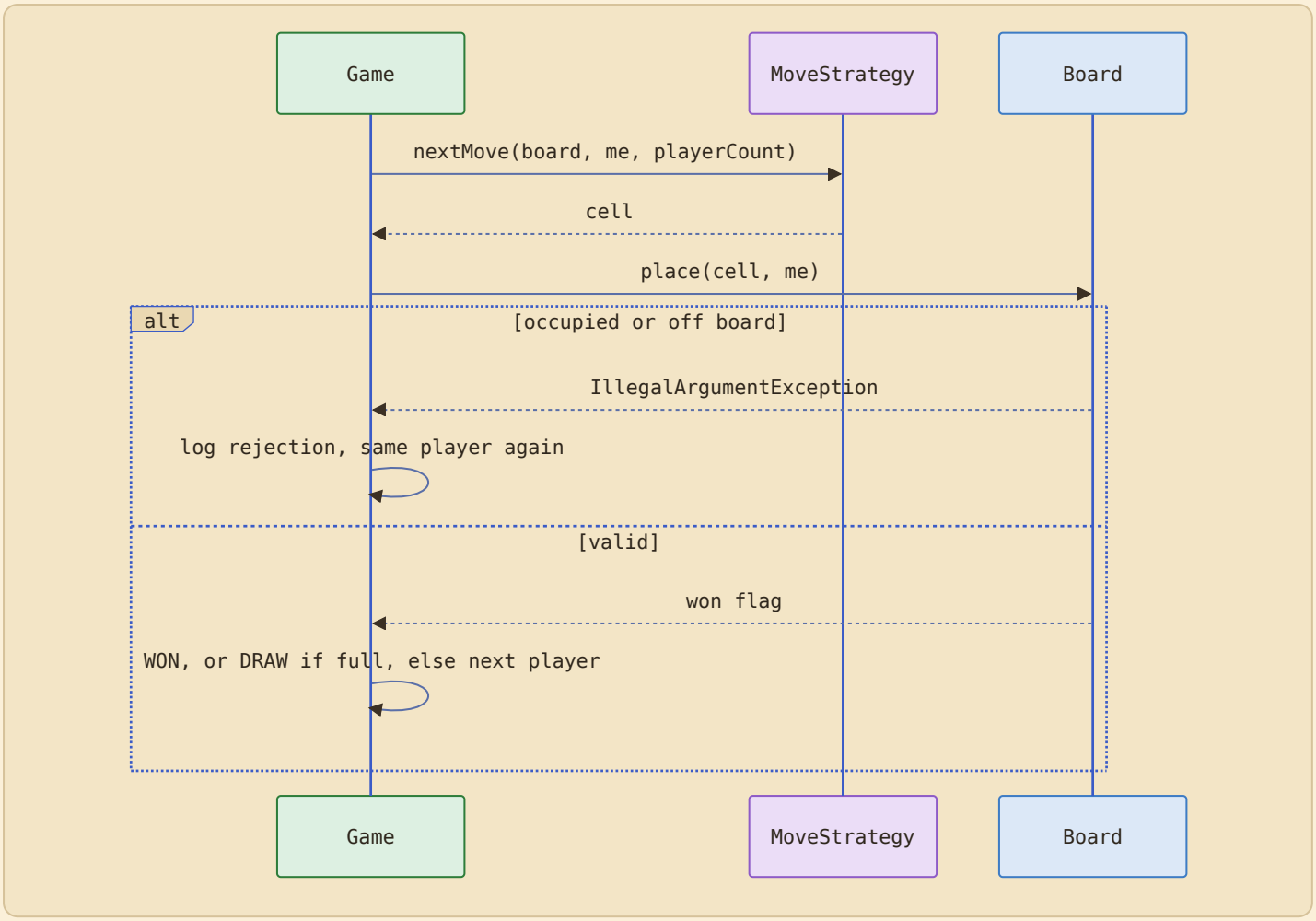
2. `diag[p]` counts marks on the main diagonal (`r == c`); `anti[p]` on the anti-diagonal (`r + c == N-1`).
3. When player `p` places at `(r, c)`, increment the counters that cell belongs to.
4. If any of them reaches **N**, player `p` owns the whole line — a win. Because only `p`'s marks increment `p`'s counters, no check for opponents' marks is needed.

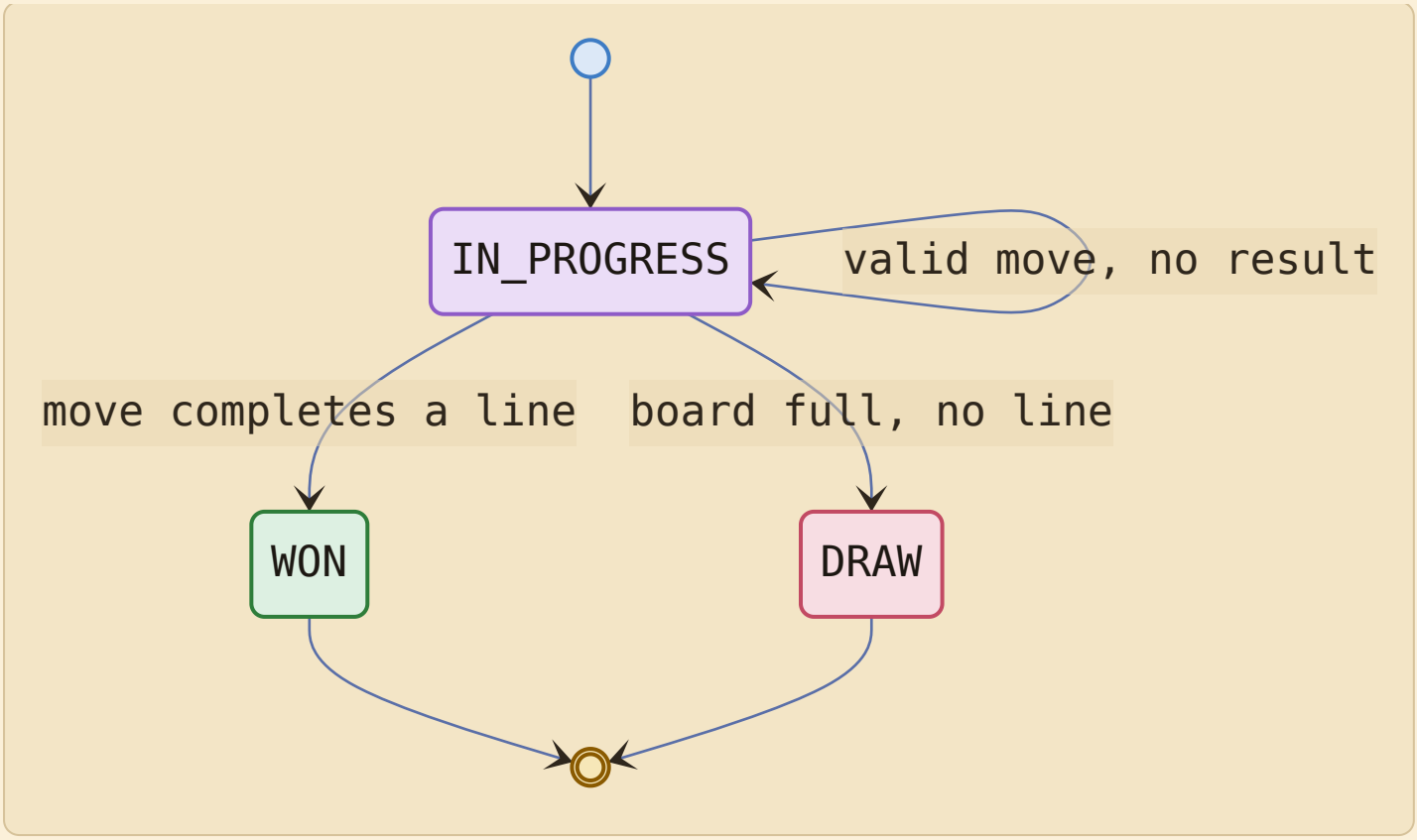
Four increments and four comparisons: **O(1) time**, **O(P·N) memory** for P players.

> **The 2-player trick you may see online:** one shared counter per line, +1 for X and −1 for O; `|count| == N` means a win. It uses half the memory but only works for exactly two players. Our per-player counters work for any number of players.

The same counters give a free **"would this move win?"** check: an empty cell wins for `p` if any of its lines already has `N-1` of `p`'s marks. The rule-based bot uses it to *win* if possible and *block* the opponent otherwise.

One turn





Design decisions & patterns used

Decision	Why	Alternative rejected
Strategy for player moves.	Human, random, rule-based, network or minimax players plug in; `Game` never changes.	`if (player.isComputer())` branches in `Game`.
Counters inside `Board` (not a separate "WinChecker" that scans).	O(1) check, kept next to the data it summarises.	Scanning all lines each move — O(N²) and duplicated for "would win".
Board returns "won?" from `place`.	The only moment a win can appear is right after a move.	Separate `checkWinner()` that the caller might forget to call.
Invalid move → exception from `Board`, caught by `Game`.	The board protects its invariant (a fact that must always stay true, e.g. one mark per cell); the game decides the policy (retry).	Silently ignoring a bad move.
Seeded `Random` injected into `RandomStrategy`.	Repeatable demos/tests.	`Math.random()` inside the bot.
Players as a list + index `current = (current+1) % size` (`%` is remainder, so the index wraps back to 0).	Works for any number of players.	Hard-coded X/O toggling.

Java implementation

Package `lld.ch17.tictactoe`.

```java
// File: code/lld/ch17/tictactoe/Cell.java
package lld.ch17.tictactoe;

public record Cell(int row, int col) {
    @Override public String toString() { return "(" + row + "," + col + ")"; }
}
```

```java
// File: code/lld/ch17/tictactoe/Board.java
package lld.ch17.tictactoe;

import java.util.ArrayList;
import java.util.List;

/**
 * N x N grid plus per-player line counters, so "did that move win?" is O(1) instead of O(N^2).
 * rows[p][r] = how many marks player p has in row r. Same idea for columns and the two diagonals.
 * A line is won when one player's count reaches N (other players' marks make that impossible).
 */
public final class Board {
    private final int n;
    private final int[][] grid;       // -1 empty, otherwise player index
    private final int[][] rows, cols;
    private final int[] diag, anti;
    private int filled;

    public Board(int n, int players) {
        if (n < 3) throw new IllegalArgumentException("board must be at least 3x3");
        this.n = n;
        grid = new int[n][n];
        for (int[] row : grid) java.util.Arrays.fill(row, -1);
        rows = new int[players][n];
        cols = new int[players][n];
        diag = new int[players];
        anti = new int[players];
    }

    public int size() { return n; }

    public boolean isEmpty(Cell c) { return inside(c) && grid[c.row()][c.col()] == -1; }

    private boolean inside(Cell c) { return c.row() >= 0 && c.row() < n && c.col() >= 0 && c.col() < n; }

    /** Places a mark and returns true if this move completed a line. O(1). */
    boolean place(Cell c, int player) {
        if (!inside(c)) throw new IllegalArgumentException("cell " + c + " is off the board");
        if (grid[c.row()][c.col()] != -1) throw new IllegalArgumentException("cell " + c + " is already taken");
        grid[c.row()][c.col()] = player;
        filled++;
        int r = c.row(), k = c.col();
        boolean win = ++rows[player][r] == n;
        win |= ++cols[player][k] == n;
        if (r == k) win |= ++diag[player] == n;
        if (r + k == n - 1) win |= ++anti[player] == n;
        return win;
    }

    /** Would placing player's mark on this empty cell win? Also O(1): a line needs exactly one more mark. */
    public boolean wouldWin(Cell c, int player) {
        int r = c.row(), k = c.col();
        return rows[player][r] == n - 1 || cols[player][k] == n - 1
                || (r == k && diag[player] == n - 1) || (r + k == n - 1 && anti[player] == n - 1);
    }

    public boolean isFull() { return filled == n * n; }

    public List<Cell> emptyCells() {
        List<Cell> out = new ArrayList<>();
        for (int r = 0; r < n; r++) for (int c = 0; c < n; c++) if (grid[r][c] == -1) out.add(new Cell(r, c));
        return out;
    }

    public String render(char[] marks) {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < n; r++) {
            sb.append("    ");
            for (int c = 0; c < n; c++) sb.append(grid[r][c] == -1 ? '.' : marks[grid[r][c]]).append(c < n - 1 ? "
" : "");
```

```
            if (r < n - 1) sb.append('\n');
        }
        return sb.toString();
    }
}
```

```
// File: code/lld/ch17/tictactoe/MoveStrategy.java
package lld.ch17.tictactoe;

/** Strategy pattern: human input, scripted test moves, random bot, rule-based bot... all plug in here. */
public interface MoveStrategy {
    Cell nextMove(Board board, int me, int playerCount);
}
```

```
// File: code/lld/ch17/tictactoe/ScriptedStrategy.java
package lld.ch17.tictactoe;

import java.util.ArrayDeque;
import java.util.Deque;
import java.util.List;

/** Plays a fixed list of moves. Stands in for a human in tests. */
public final class ScriptedStrategy implements MoveStrategy {
    private final Deque<Cell> moves;

    public ScriptedStrategy(List<Cell> moves) { this.moves = new ArrayDeque<>(moves); }

    @Override public Cell nextMove(Board board, int me, int playerCount) { return moves.poll(); }
}
```

```
// File: code/lld/ch17/tictactoe/RandomStrategy.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.Random;

/** Picks any empty cell. Takes a seeded Random so games are repeatable. */
public final class RandomStrategy implements MoveStrategy {
    private final Random rng;

    public RandomStrategy(Random rng) { this.rng = rng; }

    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        return empty.get(rng.nextInt(empty.size()));
    }
}
```

```java
// File: code/lld/ch17/tictactoe/RuleBasedStrategy.java
package lld.ch17.tictactoe;

import java.util.List;

/** Win if you can, else block an opponent's win, else take the centre, else the first corner, else anything. */
public final class RuleBasedStrategy implements MoveStrategy {
    @Override public Cell nextMove(Board board, int me, int playerCount) {
        List<Cell> empty = board.emptyCells();
        for (Cell c : empty) if (board.wouldWin(c, me)) return c;
        for (int other = 0; other < playerCount; other++) {
            if (other == me) continue;
            for (Cell c : empty) if (board.wouldWin(c, other)) return c;
        }
        int n = board.size();
        Cell centre = new Cell(n / 2, n / 2);
        if (board.isEmpty(centre)) return centre;
        for (Cell corner : List.of(new Cell(0, 0), new Cell(0, n - 1), new Cell(n - 1, 0), new Cell(n - 1, n -
1))) {
            if (board.isEmpty(corner)) return corner;
        }
        return empty.get(0);
    }
}
```

```java
// File: code/lld/ch17/tictactoe/Player.java
package lld.ch17.tictactoe;

public record Player(String name, char mark, MoveStrategy strategy) {}
```

```java
// File: code/lld/ch17/tictactoe/GameState.java
package lld.ch17.tictactoe;

public enum GameState { IN_PROGRESS, WON, DRAW }
```

```java
// File: code/lld/ch17/tictactoe/Game.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.function.Consumer;

/** Turn management and game state. Supports 2+ players on an N x N board. */
public final class Game {
    private final Board board;
    private final List<Player> players;
    private final char[] marks;
    private final Consumer<String> log;
    private int current;
    private GameState state = GameState.IN_PROGRESS;
    private Player winner;

    public Game(int n, List<Player> players, Consumer<String> log) {
        if (players.size() < 2) throw new IllegalArgumentException("need at least 2 players");
        this.players = List.copyOf(players);
        this.board = new Board(n, players.size());
        this.marks = new char[players.size()];
        for (int i = 0; i < marks.length; i++) marks[i] = players.get(i).mark();
        this.log = log;
    }

    /** Plays one turn for the current player. An invalid cell is rejected and the same player goes again. */
    public void playTurn() {
        if (state != GameState.IN_PROGRESS) throw new IllegalStateException("game is over");
        Player p = players.get(current);
        Cell cell = p.strategy().nextMove(board, current, players.size());
        boolean won;
        try {
            won = board.place(cell, current);
        } catch (IllegalArgumentException e) {
            log.accept("  " + p.name() + " tried " + cell + ": rejected (" + e.getMessage() + "), try again");
            return;
        }
        log.accept("  " + p.name() + " (" + p.mark() + ") -> " + cell);
        if (won) { state = GameState.WON; winner = p; }
        else if (board.isFull()) state = GameState.DRAW;
        else current = (current + 1) % players.size();
    }

    public GameState play() {
        while (state == GameState.IN_PROGRESS) playTurn();
        log.accept(board.render(marks));
        log.accept("  Result: " + (state == GameState.WON ? winner.name() + " wins" : "draw"));
        return state;
    }
}
```

```java
// File: code/lld/ch17/tictactoe/Main.java
package lld.ch17.tictactoe;

import java.util.List;
import java.util.Random;

public final class Main {
    public static void main(String[] args) {
        System.out.println("== 1. 3x3 scripted game (with one invalid move) ==");
        Player x = new Player("Xena", 'X', new ScriptedStrategy(List.of(
                new Cell(0, 0), new Cell(1, 1), new Cell(2, 2))));
        Player o = new Player("Omar", 'O', new ScriptedStrategy(List.of(
                new Cell(0, 0), new Cell(0, 1), new Cell(0, 2))));   // first one is already taken
        new Game(3, List.of(x, o), System.out::println).play();

        System.out.println("\n== 2. 3x3 rule-based bot vs seeded random bot (seed 7) ==");
        new Game(3, List.of(new Player("Bot", 'X', new RuleBasedStrategy()),
                new Player("Rand", 'O', new RandomStrategy(new Random(7)))), System.out::println).play();

        System.out.println("\n== 3. 4x4 rule-based vs rule-based ==");
        new Game(4, List.of(new Player("A", 'X', new RuleBasedStrategy()),
                new Player("B", 'O', new RuleBasedStrategy())), line -> { if (!line.startsWith("  A") &&
!line.startsWith("  B")) System.out.println(line); }).play();

        System.out.println("\n== 4. 5x5 with three players (two random, seeds 1 and 2; one rule-based) ==");
        new Game(5, List.of(new Player("P1", 'X', new RandomStrategy(new Random(1))),
                new Player("P2", 'O', new RandomStrategy(new Random(2))),
                new Player("P3", 'Z', new RuleBasedStrategy())), line -> { if (!line.startsWith("  P"))
System.out.println(line); }).play();
    }
}
```

Output:

```
== 1. 3x3 scripted game (with one invalid move) ==
  Xena (X) -> (0,0)
  Omar tried (0,0): rejected (cell (0,0) is already taken), try again
  Omar (O) -> (0,1)
  Xena (X) -> (1,1)
  Omar (O) -> (0,2)
  Xena (X) -> (2,2)
    X O O
    . X .
    . . X
  Result: Xena wins

== 2. 3x3 rule-based bot vs seeded random bot (seed 7) ==
  Bot (X) -> (1,1)
  Rand (O) -> (2,0)
  Bot (X) -> (0,0)
  Rand (O) -> (1,0)
  Bot (X) -> (2,2)
    X . .
    O X .
    O . X
  Result: Bot wins

== 3. 4x4 rule-based vs rule-based ==
    O O X X
    O O X X
    X X X O
    O O O X
  Result: draw

== 4. 5x5 with three players (two random, seeds 1 and 2; one rule-based) ==
    Z Z Z O O
    X O X Z X
    X X Z O O
    Z O O X O
    Z X X X Z
  Result: draw
```

Reading the output. Game 1 shows the retry after an invalid move and a diagonal win found by the O(1) counters. Game 2: the rule-based bot takes the centre, then a corner, and completes the diagonal against a seeded random bot (no block was needed: the random bot's two O's in column 0 were already blocked by the X in the corner). Game 3: two rule-based bots on 4×4 draw (the move lines are filtered out of the log for brevity). Game 4 runs the same counters with three players on 5×5 (a draw).

Concurrency & edge cases

- **Many games on one server:** each `Game` owns its own `Board`, so games share nothing. Keep them in a `ConcurrentHashMap<gameId, Game>` (a hash map that many threads can read and write safely at once) and process each game's moves one at a time (per-game lock or single-threaded actor).
- **Two clicks at once from the same game:** make `playTurn`/a `makeMove(player, cell)` method `synchronized` and check that the caller is the current player.
- **Edge cases:** move after game over (rejected with `IllegalStateException`, Java's standard "not allowed right now" error); off-board or occupied cell (rejected, same player retries); a scripted strategy running out of moves (returns `null` → fail fast, i.e. stop with a clear error at once, in tests); N = 3 minimum.
- **A cell on both diagonals** (the centre of an odd board) updates both diagonal counters — handled by two independent `if`s.

Extensions & follow-up questions

- **"Win with K in a row on an N×N board (Gomoku, K = 5)?"** — Counters no longer work because lines are shorter than the board. After each move, walk out from the new cell in the 4 directions (horizontal, vertical, two diagonals) and count consecutive own marks: O(K) per move.
- **"Unbeatable AI?"** — Minimax over the game tree (every possible sequence of moves drawn as a branching tree; tic-tac-toe 3×3 has at most 9! = 362,880 move orders, easily searched); add alpha-

beta pruning or memoisation (caching the result for each position already evaluated) for larger boards. Plug it in as another `MoveStrategy`.

- **"Undo?"** — Store moves in a stack (last in, first out); undo = clear the cell and **decrement** the counters (counters are reversible, unlike a boolean "won" flag).
- **"Online multiplayer?"** — Server holds the `Game`; clients send `makeMove(gameId, playerId, cell)`; the server broadcasts the new state (Observer).
- **"Replay a game?"** — The move list plus `ScriptedStrategy` replays it exactly.

Real-world example

Google Search has a built-in tic-tac-toe game (search "tic tac toe") with difficulty levels — different computer players behind the same game rules, which is the Strategy split in this design. Tic-tac-toe is also the classic first example for the minimax algorithm in AI courses, because the full game tree is small enough to search completely.

In the interview

30-second answer: "`Board` is N×N plus per-player counters for each row, column and the two diagonals. Placing a mark bumps at most four counters; if one hits N, that player wins — O(1) per move. `Game` holds a list of players and a turn index, validates moves, and ends on a win or a full board. Each `Player` has a `MoveStrategy` — human, random with a seeded `Random`, or rule-based that wins if it can and blocks otherwise."

Time plan: 5 min requirements and N×N/players → 5 min entities → 10 min the counter idea (draw it!) → 15 min code → 5 min extensions (K-in-a-row, minimax, undo).

Self-check

1. **Q:** Why does a move only need to check four lines? **A:** A cell belongs to exactly one row, one column, and at most the two diagonals.
2. **Q:** How do you know a cell is on the anti-diagonal? **A:** `row + col == N - 1`.
3. **Q:** Why per-player counters instead of +1/−1? **A:** They work for more than two players and give "would win" for any player.
4. **Q:** How is a draw detected? **A:** No win and the number of filled cells equals N^2.
5. **Q:** How do you test a bot that plays randomly? **A:** Inject `new Random(seed)`; the game is then identical every run.

TL;DR

- Per-player row/column/diagonal counters → O(1) win check; `N-1` → O(1) "would win".
- `Game` = players list + turn index + state enum; invalid moves rejected, same player retries.
- Players via `MoveStrategy` (human, random-seeded, rule-based, minimax later).
- K-in-a-row needs a directional walk from the last move instead.

Design Snake and Ladder

Problem & clarifying questions

Snake and Ladder is a race on a numbered board (usually 1–100). Players take turns rolling a die and move forward by the number rolled. Landing on the **foot of a ladder** takes you up to its top; landing on the **head of a snake** sends you down to its tail. The first to reach the last square wins. There are no decisions — everything is dice and rules — so the design is about a **clean rules engine**, **turn management** and **testable randomness**.

Question	Assumed answer
Board size and snakes/ladders?	Configurable (default 100 squares, a given list of snakes and ladders). Validate the config.
How many dice?	Configurable: count and faces (default one six-sided die).
How many players?	Two or more; all start off the board (square 0).
Overshooting the last square?	Configurable: **exact finish** (stay put) or **bounce back**.
Rolling a six?	Common house rule (an optional rule some players add): roll again; three sixes in a row cancels the whole turn. Configurable on/off.
When does the game end?	Configurable: **first to finish** wins, or keep playing to **rank everyone**.
Several games at once?	Yes, independent sessions (e.g. many groups online).

Requirements

Functional

1. A board of numbered cells (typically 100) with a predefined set of snakes and ladders connecting cells.
2. Multiple players, each with one token, starting off the board.
3. Players take turns rolling the dice and move forward by the total.
4. Landing on a snake head slides the token down to its tail; landing on a ladder foot climbs to its top.
5. Handle overshooting the final square (exact-finish or bounce-back rule).
6. Optional bonus roll on a six, with the three-sixes penalty.
7. The game continues until the win strategy says it is over (first to finish, or everyone ranked).
8. Support many independent game sessions at once.

Non-functional

- **Deterministic** dice for tests (seeded or loaded).
- Rules are **pluggable** without editing the game loop.
- Invalid boards are rejected **at construction**, not discovered mid-game.

Use cases

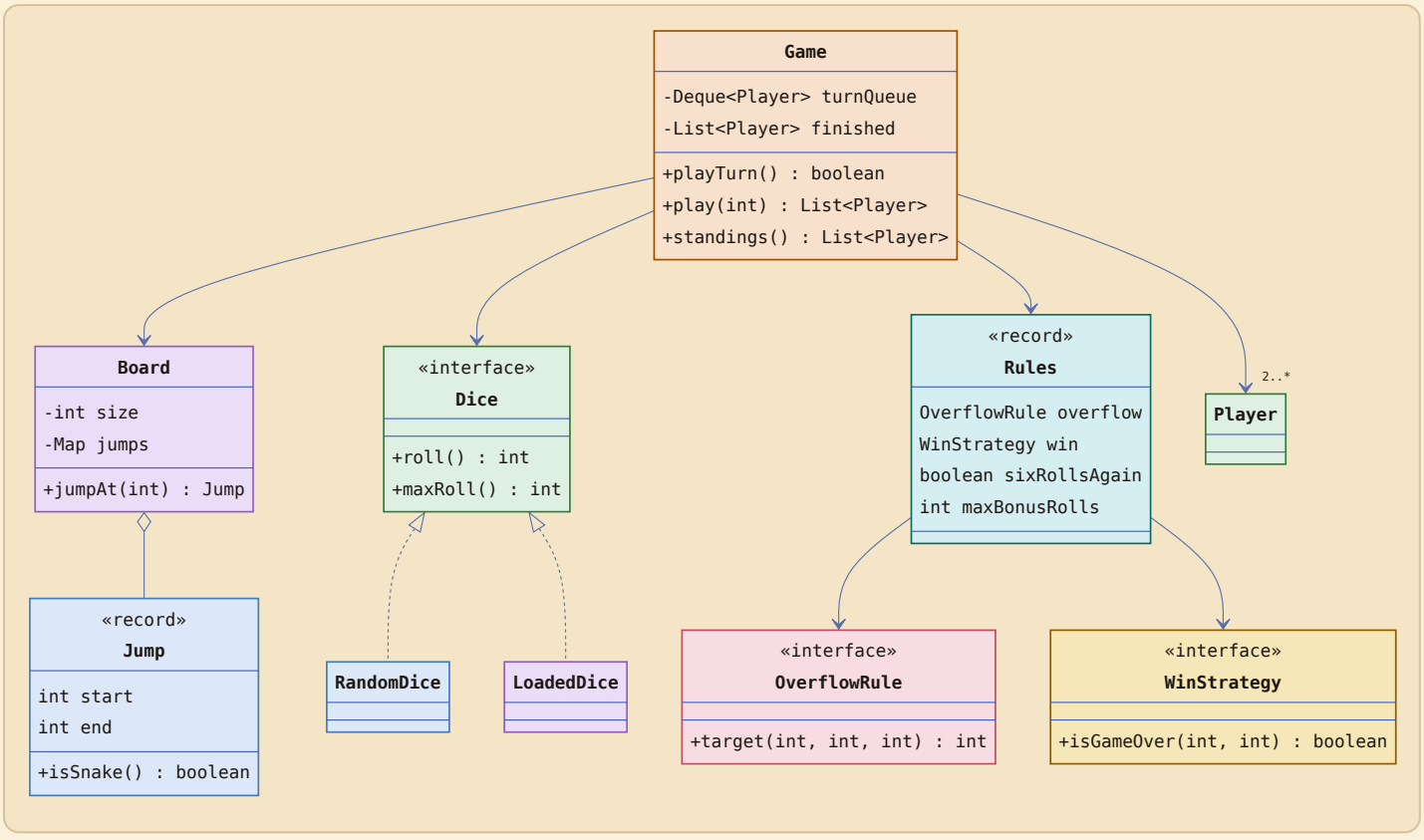
Actors: **Player** (rolls the dice — or the system rolls for them), **Game admin/host** (configures board, dice and rules, starts a game). Use cases: *configure board*, *add players*, *start game*, *take turn* (roll → move → apply snake/ladder → maybe bonus roll), *declare winner / standings*.

Core entities

Class	Responsibility
`Jump` (record)	A snake or a ladder: `start → end`. One class, because the behaviour is identical ("land on start, move to end"); `isSnake()` is just `end < start`.
`Board`	Size + `Map<start, Jump>` for O(1) lookup; validates configuration.
`Dice` (interface)	`roll()`, `maxRoll()`. `RandomDice` (count, faces, seeded `Random`), `LoadedDice` (scripted rolls for tests).
`OverflowRule` (functional interface — exactly one method, so a lambda, a short unnamed function like `x -> x + 1`, can implement it)	Where a roll past the end lands: `exactFinish()` or `bounceBack()`.
`WinStrategy` (functional interface)	When the game ends: `firstToFinish()` or `rankAll()`.
`Rules` (record)	Bundles a variant: overflow rule, win strategy, six-rolls-again flag, max bonus rolls.
`Player`	Name and position.
`Game`	Turn queue, `playTurn()`, `play(maxTurns)`, standings, event log.

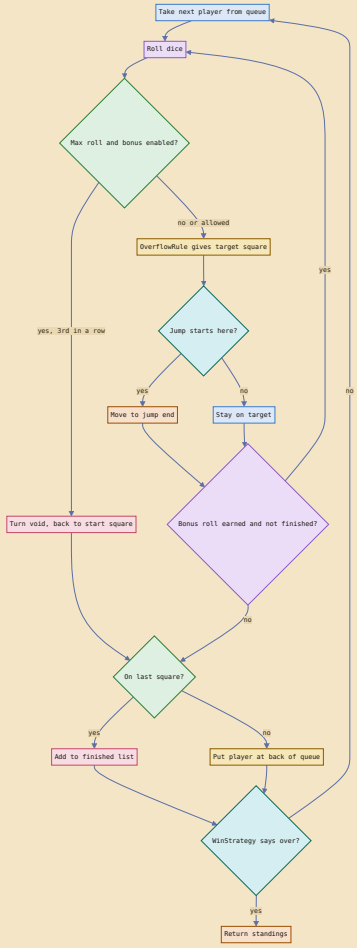
Class diagram

There is no course lesson for this problem; it comes from the *awesome-low-level-design* problem set. That statement asks for: a board of numbered cells (usually 100); predefined snakes and ladders; several players, each with a token; turn-based dice rolls; sliding down a snake's head to its tail and climbing a ladder's foot to its top; a win on reaching the last cell; and many independent game sessions running at once (its reference solution uses a singleton — a class with only one global instance — `GameManager` with one thread per session). All of these are covered; part 6 of the output runs eight sessions in parallel.



Key flows

One turn



Board validation (done once, in the constructor)

1. A jump must not start and end on the same square.
2. No jump starts on square 1 or the last square (you could never win if the last square were a snake).
3. No two jumps share a start square.
4. A jump may not end on another jump's start. This forbids chains, and therefore also forbids infinite loops (snake A → ladder B → snake A...).

Design decisions & patterns used

Decision	Why	Alternative rejected
One `Jump` type for snakes and ladders.	Identical behaviour; the direction is data.	`Snake` and `Ladder` classes with duplicate code — only worth it if they behave differently.
`Map<Integer, Jump>` lookup.	O(1) per move instead of scanning a list.	Array of size 101 — also fine; a map scales to sparse or huge boards.
Strategy for `Dice`, `OverflowRule`, `WinStrategy`.	House rules vary a lot; each is swappable. Lambdas keep them tiny.	Flags and `if`s in the game loop.
`Rules` record groups a variant.	One object = one rule set ("classic", "kids", "tournament"); easy to pass around.	Many constructor parameters on `Game`.
Queue of players.	Natural turn order; finished players simply are not re-queued.	Index arithmetic that must skip finished players.
Event log via `Consumer<String>` (a built-in "takes a value, returns nothing" function type).	The engine does not print; the caller decides (console, UI, nothing). A full design would use **Observer** (`GameListener.onMove/onSnake/onLadder/onWin`).	`System.out.println` inside the engine.
Validation in the `Board` constructor (fail fast).	The loop can trust the board.	Detecting a loop mid-game.
`play(maxTurns)` guard.	A misconfigured rule set can never hang (loop forever and never finish) a server thread.	An unbounded `while (true)`.

Java implementation

Package `lld.ch17.snakeladder`.

```java
// File: code/lld/ch17/snakeladder/Jump.java
package lld.ch17.snakeladder;

/** A snake (goes down) or a ladder (goes up). Both are just "land on start -> move to end". */
public record Jump(int start, int end) {
    public boolean isSnake() { return end < start; }
    public String kind() { return isSnake() ? "snake" : "ladder"; }
}
```

```java
// File: code/lld/ch17/snakeladder/Board.java
package lld.ch17.snakeladder;

import java.util.Collection;
import java.util.HashMap;
import java.util.Map;

/**
 * Board configuration: size plus a map start -> Jump for O(1) lookup.
 * Validates the config once, up front, so the game loop can trust it.
 */
public final class Board {
    private final int size;
    private final Map<Integer, Jump> jumps = new HashMap<>();

    public Board(int size, Collection<Jump> all) {
        if (size < 10) throw new IllegalArgumentException("board too small");
        this.size = size;
        for (Jump j : all) {
            if (j.start() == j.end()) throw new IllegalArgumentException("jump to itself: " + j);
            if (j.start() <= 1 || j.start() >= size) throw new IllegalArgumentException("jump cannot start on
first/last square: " + j);
            if (j.end() < 1 || j.end() > size) throw new IllegalArgumentException("jump ends off board: " + j);
            if (jumps.putIfAbsent(j.start(), j) != null) throw new IllegalArgumentException("two jumps start at "
+ j.start());
        }
        for (Jump j : jumps.values()) {   // no chains: an end square may not be another jump's start (also rules
out loops)
            if (jumps.containsKey(j.end())) throw new IllegalArgumentException("jump " + j + " ends on another
jump's start");
        }
    }

    public int size() { return size; }

    /** Returns the jump that starts on this square, or null. */
    public Jump jumpAt(int square) { return jumps.get(square); }
}
```

```java
// File: code/lld/ch17/snakeladder/Dice.java
package lld.ch17.snakeladder;

/** Strategy for rolling. Real dice use a seeded Random; tests use a fixed sequence. */
public interface Dice {
    int roll();
    int maxRoll();
}
```

```java
// File: code/lld/ch17/snakeladder/RandomDice.java
package lld.ch17.snakeladder;

import java.util.Random;

/** count dice with the given number of faces. Pass new Random(seed) for repeatable games. */
public final class RandomDice implements Dice {
    private final int count, faces;
    private final Random rng;

    public RandomDice(int count, int faces, Random rng) {
        this.count = count;
        this.faces = faces;
        this.rng = rng;
    }

    @Override public int roll() {
        int sum = 0;
        for (int i = 0; i < count; i++) sum += 1 + rng.nextInt(faces);
        return sum;
    }

    @Override public int maxRoll() { return count * faces; }
}
```

```java
// File: code/lld/ch17/snakeladder/LoadedDice.java
package lld.ch17.snakeladder;

/** Returns a scripted sequence of rolls (cycling). Used to test exact scenarios. */
public final class LoadedDice implements Dice {
    private final int[] rolls;
    private int i;

    public LoadedDice(int... rolls) { this.rolls = rolls.clone(); }

    @Override public int roll() { return rolls[i++ % rolls.length]; }

    @Override public int maxRoll() { return 6; }
}
```

```java
// File: code/lld/ch17/snakeladder/OverflowRule.java
package lld.ch17.snakeladder;

/** What happens when a roll would carry a token past the last square. */
@FunctionalInterface
public interface OverflowRule {
    /** Returns the new square (before snakes/ladders), given current square, roll and board size. */
    int target(int from, int roll, int size);

    /** Must land exactly on the last square, otherwise stay put. */
    static OverflowRule exactFinish() { return (from, roll, size) -> from + roll > size ? from : from + roll; }

    /** Bounce back by the extra amount (at 98 on a 100 board, a 5 goes to 100 then back to 97). */
    static OverflowRule bounceBack() { return (from, roll, size) -> from + roll > size ? size - (from + roll -
size) : from + roll; }
}
```

```java
// File: code/lld/ch17/snakeladder/WinStrategy.java
package lld.ch17.snakeladder;

/** When is the whole game over? */
@FunctionalInterface
public interface WinStrategy {
    boolean isGameOver(int finishedPlayers, int totalPlayers);

    /** Classic: the first player to reach the end wins and the game stops. */
    static WinStrategy firstToFinish() { return (finished, total) -> finished >= 1; }

    /** Play on to rank everybody; stop when only one player is left on the board. */
    static WinStrategy rankAll() { return (finished, total) -> finished >= total - 1; }
}
```

```java
// File: code/lld/ch17/snakeladder/Player.java
package lld.ch17.snakeladder;

public final class Player {
    private final String name;
    private int position;     // 0 = off the board, waiting to enter

    public Player(String name) { this.name = name; }

    public String name() { return name; }
    public int position() { return position; }
    void moveTo(int square) { position = square; }
}
```

```java
// File: code/lld/ch17/snakeladder/Rules.java
package lld.ch17.snakeladder;

/**
 * The rules engine's configuration, bundled so a game variant is one object.
 * sixRollsAgain: rolling the dice's maximum earns another roll; maxBonusRolls in a row cancels the turn.
 */
public record Rules(OverflowRule overflow, WinStrategy win, boolean sixRollsAgain, int maxBonusRolls) {
    public static Rules classic() { return new Rules(OverflowRule.exactFinish(), WinStrategy.firstToFinish(),
true, 3); }
}
```

```java
// File: code/lld/ch17/snakeladder/Game.java
package lld.ch17.snakeladder;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
import java.util.List;
import java.util.function.Consumer;

/** Turn loop: a queue of players, a dice strategy, a board and a rule set. */
public final class Game {
    private final Board board;
    private final Dice dice;
    private final Rules rules;
    private final Deque<Player> turnQueue;
    private final int totalPlayers;
    private final List<Player> finished = new ArrayList<>();
    private final Consumer<String> log;

    public Game(Board board, Dice dice, Rules rules, List<Player> players, Consumer<String> log) {
        if (players.size() < 2) throw new IllegalArgumentException("need at least 2 players");
        this.board = board;
        this.dice = dice;
        this.rules = rules;
        this.turnQueue = new ArrayDeque<>(players);
        this.totalPlayers = players.size();
        this.log = log;
    }

    public boolean isOver() { return rules.win().isGameOver(finished.size(), totalPlayers); }

    /** Plays one player's turn (including bonus rolls). Returns true if the game is now over. */
    public boolean playTurn() {
        if (isOver()) throw new IllegalStateException("game is over");
        Player p = turnQueue.poll();
        takeTurn(p);
        if (p.position() == board.size()) {
            finished.add(p);
            log.accept("  *** " + p.name() + " finishes in place " + finished.size() + " ***");
        } else {
            turnQueue.offer(p);
        }
        return isOver();
    }

    /** Plays until the win strategy says stop. Returns the standings (finishers first, in order). */
    public List<Player> play(int maxTurns) {
        for (int turns = 0; !isOver(); turns++) {
            if (turns == maxTurns) throw new IllegalStateException("no result after " + maxTurns + " turns");
            playTurn();
        }
        return standings();
    }

    public List<Player> standings() {
        List<Player> order = new ArrayList<>(finished);
        order.addAll(turnQueue);          // whoever is still on the board, in turn order
        return order;
    }

    private void takeTurn(Player p) {
        int start = p.position();
        int bonus = 0;
        StringBuilder line = new StringBuilder("  " + p.name() + " @" + start + ":");
        while (true) {
            int roll = dice.roll();
            line.append(" rolls ").append(roll);
            boolean max = rules.sixRollsAgain() && roll == dice.maxRoll();
            if (max && ++bonus == rules.maxBonusRolls()) {
                p.moveTo(start);                      // too many sixes in a row: the whole turn is cancelled
```

```
                line.append(" -> ").append(rules.maxBonusRolls()).append(" sixes in a row, turn void, back to
").append(start);
                break;
            }
            move(p, roll, line);
            if (!max || p.position() == board.size()) break;
            line.append(", bonus roll:");
        }
        log.accept(line.toString());
    }

    private void move(Player p, int roll, StringBuilder line) {
        int target = rules.overflow().target(p.position(), roll, board.size());
        if (target == p.position()) { line.append(" -> needs exact roll, stays at ").append(target); return; }
        line.append(" -> ").append(target);
        Jump j = board.jumpAt(target);
        if (j != null) {
            target = j.end();
            line.append(" (").append(j.kind()).append(" to ").append(target).append(')');
        }
        p.moveTo(target);
    }
}
```

```java
// File: code/lld/ch17/snakeladder/Main.java
package lld.ch17.snakeladder;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.Random;
import java.util.stream.Collectors;

public final class Main {
    private static final List<Jump> JUMPS = List.of(
            new Jump(4, 14), new Jump(9, 31), new Jump(21, 42), new Jump(28, 84), new Jump(51, 67),
            new Jump(71, 91), new Jump(80, 99),                                   // ladders
            new Jump(17, 7), new Jump(54, 34), new Jump(62, 19), new Jump(64, 60),
            new Jump(87, 24), new Jump(93, 73), new Jump(95, 75), new Jump(98, 79));  // snakes

    public static void main(String[] args) throws Exception {
        Board board = new Board(100, JUMPS);

        System.out.println("== 1. Loaded dice: ladder, bonus roll, snake, three sixes ==");
        Game g1 = new Game(board, new LoadedDice(4, 6, 3, 3, 6, 6, 6, 2), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob")), System.out::println);
        for (int i = 0; i < 4; i++) g1.playTurn();

        System.out.println("\n== 2. Overflow rules: token on 97 of 100, rolls a 5 ==");
        Board plain = new Board(100, List.of());
        for (String variant : List.of("Exact", "Bounce")) {
            OverflowRule rule = variant.equals("Exact") ? OverflowRule.exactFinish() : OverflowRule.bounceBack();
            Player p = new Player(variant);
            p.moveTo(97);
            Game g = new Game(plain, new LoadedDice(5), new Rules(rule, WinStrategy.firstToFinish(), false, 3),
                    List.of(p, new Player("Q")), System.out::println);
            g.playTurn();
        }

        System.out.println("\n== 3. Bad configurations are rejected up front ==");
        for (List<Jump> bad : List.of(List.of(new Jump(100, 5)), List.of(new Jump(10, 30), new Jump(30, 5)))) {
            try { new Board(100, bad); }
            catch (IllegalArgumentException e) { System.out.println("  rejected: " + e.getMessage()); }
        }

        System.out.println("\n== 4. Seeded game (seed 2024), 3 players, first to finish wins ==");
        Game g4 = new Game(board, new RandomDice(1, 6, new Random(2024)), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), System.out::println);
        System.out.println("  Winner: " + g4.play(500).get(0).name());

        System.out.println("\n== 5. Same seed, rank-everyone rule, bounce-back overflow (log hidden) ==");
        Game g5 = new Game(board, new RandomDice(1, 6, new Random(2024)),
                new Rules(OverflowRule.bounceBack(), WinStrategy.rankAll(), true, 3),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), line -> { });
        System.out.println("  Standings: " + g5.play(1000).stream().map(Player::name).collect(Collectors.joining("
> ")));

        System.out.println("\n== 6. Eight independent sessions on eight threads at once ==");
        // Each session owns its players, turn queue and dice (its own seeded Random), and shares only the
        // read-only Board. So running them in parallel must give exactly the result each one gives alone.
        Map<Integer, String> parallel = new ConcurrentHashMap<>();
        try (ExecutorService pool = Executors.newFixedThreadPool(8)) {
            for (int seed = 1; seed <= 8; seed++) {
                int s0 = seed;
                pool.submit(() -> parallel.put(s0, winnerOf(board, s0)));
            }
        }
        List<String> mismatches = new ArrayList<>();
        for (int seed = 1; seed <= 8; seed++)
            if (!parallel.get(seed).equals(winnerOf(board, seed))) mismatches.add("seed " + seed);
        System.out.println("  winners by seed: " + new java.util.TreeMap<>(parallel));
```

```
        System.out.println("  same as running each session alone? " + (mismatches.isEmpty() ? "yes" : "NO " +
mismatches));
    }

    private static String winnerOf(Board board, int seed) {
        Game g = new Game(board, new RandomDice(1, 6, new Random(seed)), Rules.classic(),
                List.of(new Player("Ann"), new Player("Bob"), new Player("Cid")), line -> { });
        return g.play(500).get(0).name();
    }
}
```

Output:

```
== 1. Loaded dice: ladder, bonus roll, snake, three sixes ==
  Ann @0: rolls 4 -> 4 (ladder to 14)
  Bob @0: rolls 6 -> 6, bonus roll: rolls 3 -> 9 (ladder to 31)
  Ann @14: rolls 3 -> 17 (snake to 7)
  Bob @31: rolls 6 -> 37, bonus roll: rolls 6 -> 43, bonus roll: rolls 6 -> 3 sixes in a row, turn void, back to
31

== 2. Overflow rules: token on 97 of 100, rolls a 5 ==
  Exact @97: rolls 5 -> needs exact roll, stays at 97
  Bounce @97: rolls 5 -> 98

== 3. Bad configurations are rejected up front ==
  rejected: jump cannot start on first/last square: Jump[start=100, end=5]
  rejected: jump Jump[start=10, end=30] ends on another jump's start

== 4. Seeded game (seed 2024), 3 players, first to finish wins ==
  Ann @0: rolls 1 -> 1
  Bob @0: rolls 3 -> 3
  Cid @0: rolls 1 -> 1
  Ann @1: rolls 5 -> 6
  Bob @3: rolls 6 -> 9 (ladder to 31), bonus roll: rolls 6 -> 37, bonus roll: rolls 6 -> 3 sixes in a row, turn
void, back to 3
  Cid @1: rolls 6 -> 7, bonus roll: rolls 1 -> 8
  Ann @6: rolls 6 -> 12, bonus roll: rolls 4 -> 16
  Bob @3: rolls 4 -> 7
  Cid @8: rolls 3 -> 11
  Ann @16: rolls 4 -> 20
  Bob @7: rolls 2 -> 9 (ladder to 31)
  Cid @11: rolls 3 -> 14
  Ann @20: rolls 4 -> 24
  Bob @31: rolls 5 -> 36
  Cid @14: rolls 6 -> 20, bonus roll: rolls 1 -> 21 (ladder to 42)
  Ann @24: rolls 4 -> 28 (ladder to 84)
  Bob @36: rolls 3 -> 39
  Cid @42: rolls 5 -> 47
  Ann @84: rolls 4 -> 88
  Bob @39: rolls 5 -> 44
  Cid @47: rolls 3 -> 50
  Ann @88: rolls 1 -> 89
  Bob @44: rolls 6 -> 50, bonus roll: rolls 4 -> 54 (snake to 34)
  Cid @50: rolls 1 -> 51 (ladder to 67)
  Ann @89: rolls 4 -> 93 (snake to 73)
  Bob @34: rolls 2 -> 36
  Cid @67: rolls 5 -> 72
  Ann @73: rolls 6 -> 79, bonus roll: rolls 2 -> 81
  Bob @36: rolls 3 -> 39
  Cid @72: rolls 3 -> 75
  Ann @81: rolls 4 -> 85
  Bob @39: rolls 5 -> 44
  Cid @75: rolls 2 -> 77
  Ann @85: rolls 5 -> 90
  Bob @44: rolls 2 -> 46
  Cid @77: rolls 2 -> 79
  Ann @90: rolls 2 -> 92
  Bob @46: rolls 6 -> 52, bonus roll: rolls 5 -> 57
  Cid @79: rolls 1 -> 80 (ladder to 99)
  Ann @92: rolls 2 -> 94
  Bob @57: rolls 4 -> 61
  Cid @99: rolls 1 -> 100
  *** Cid finishes in place 1 ***
  Winner: Cid

== 5. Same seed, rank-everyone rule, bounce-back overflow (log hidden) ==
  Standings: Cid > Ann > Bob

== 6. Eight independent sessions on eight threads at once ==
  winners by seed: {1=Ann, 2=Bob, 3=Cid, 4=Bob, 5=Cid, 6=Cid, 7=Ann, 8=Bob}
  same as running each session alone? yes
```

Reading the output. Part 1 uses loaded dice to force every rule: Ann climbs 4→14, Bob rolls a 6 and gets a bonus roll that lands on the 9→31 ladder, Ann hits the 17→7 snake, and Bob's three sixes void his turn (he goes back to 31). Part 2 compares overflow rules from square 97 with a roll of 5: exact-finish stays, bounce-back goes to 100 and back 2 to 98. Part 3 shows bad boards rejected. Part 4 is a full seeded game; Part 5 replays with different rules and ranks every player. Part 6 runs eight games with different seeds on eight threads at the same time and checks that each winner is exactly what the same seed gives when that game runs alone — proof that sessions share no mutable state (data that can be changed after creation).

Concurrency & edge cases

- **Many sessions:** each `Game` owns its players, queue and dice. Give **each game its own** `Random` (sharing one `java.util.Random` across threads is thread-safe but becomes a contention point — many threads waiting on the same shared object — and breaks per-game reproducibility). Keep games in a `ConcurrentHashMap` and run each game's turns on one thread at a time.
- **Rolling after the game ends** → `IllegalStateException`.
- **A player on the last square** is removed from the queue, so they are never asked to roll again.
- **Bonus roll that finishes the game** stops immediately — no roll after reaching 100.
- **Three sixes** → the turn is voided, and the token returns to where the turn started (even if a ladder was climbed during the turn).
- **Bounce-back landing on a snake** (e.g. bouncing to 98 on the standard board) is applied — the rule output goes through the same jump lookup.
- **Endless games** are impossible with a valid board and fair dice (probability of never finishing is 0), but `maxTurns` guards against bad custom rules.

Extensions & follow-up questions

- **"Add special cells (skip a turn, extra move, teleport)."** — Generalise `Jump` into a `CellEffect` interface (`apply(player, game)`); snakes and ladders become two implementations.
- **"Two dice, or a 1–12 die?"** — `new RandomDice(2, 6, rng)`; `maxRoll()` is 12, so the bonus rule triggers on double-max. Note that 2d6 (two six-sided dice added together) is not uniform — not every total is equally likely (7 is the most likely), which changes game balance.
- **"Players must roll a 6 to enter the board."** — A new rule strategy: `EntryRule` checked when position is 0.
- **"Pause and resume."** — Serialize (turn into bytes or text that can be saved and loaded back) the game state (positions, queue order, and the dice's state — or a seed plus number of rolls so far).
- **"Real-time UI / spectators?"** — Observer: publish `PlayerMoved`, `SnakeBitten`, `LadderClimbed`, `PlayerFinished` events.
- **"Random board generation?"** — A `BoardGenerator` with a seed, calling the same validating constructor.

Real-world example

Online multiplayer board-game apps (for example Ludo King, which also offers a Snakes & Ladders mode) have to solve the same problems as this design: every match is an isolated session with its own state, the dice must be rolled somewhere the players cannot tamper with (in a sound design, on the server, not in the client app), and each move is pushed to all players as an event. That is the "one game = isolated state + injected dice + event stream" split used here.

In the interview

> **30-second answer:** "`Board` holds size and a map from square to `Jump` (snake or ladder — same behaviour, just direction), validated on construction. `Dice` is a strategy: seeded random for play, loaded for tests. `Rules` bundles pluggable policies: overflow (exact or bounce), win strategy (first finish or rank all), and the bonus-six rule. `Game` keeps a queue of players; each turn rolls, applies the overflow rule, then any jump, and re-queues the player unless they finished."

Time plan: 5 min rules questions (overshoot, sixes, win) → 5 min entities → 10 min diagram + validation rules → 15 min code turn loop → 5 min extensions and concurrency.

Self-check

1. **Q:** Why is one `Jump` class enough for snakes and ladders? **A:** Both mean "land on start, go to end"; only the direction differs, which is data.
2. **Q:** How do you guarantee there is no infinite snake/ladder loop? **A:** Forbid any jump from ending on another jump's start.
3. **Q:** How do you test "three sixes voids the turn"? **A:** `LoadedDice(6, 6, 6)` and assert the position is unchanged.
4. **Q:** What changes to support "bounce back"? **A:** Pass `OverflowRule.bounceBack()` — no game-loop edits.
5. **Q:** Why a queue for turns? **A:** Players who finish simply are not put back; turn order stays correct.

TL;DR

- Board = size + `Map<start, Jump>`; validate once (no jump on last square, no chains/loops).
- Dice, overflow, win condition, bonus rule are all strategies; seeded or loaded dice for tests.
- Queue-based turn loop with a `maxTurns` guard; events go to a listener, not `System.out`.

Design 2048

Problem & clarifying questions

2048 is a single-player sliding puzzle on a 4×4 grid. Each move (left, right, up, down) slides every tile as far as possible in that direction. Two tiles with the **same value** that collide **merge** into one tile with double the value, and the merged value is added to the **score**. After every move that changes the board, a new tile (a 2 most of the time, sometimes a 4) appears in a random empty cell. You "win" by making a 2048 tile; the game is **over** when no move can change the board.

There is no course lesson for this problem. The problem statement is the common machine-coding (an interview round where you write working code for a small system in 1–2 hours) version (the "2048" exercise found in LLD practice lists such as Tharun's and on machine-coding sites): start with **two tiles numbered 2** at random positions; after every move insert a random tile in a random empty cell; tiles slide until stopped by another tile or the edge; two equal tiles that collide merge into their sum; if **three** equal tiles line up, the pair **nearest the wall you move toward** merges; if **four** are equal, the first two and the last two merge; win at 2048; lose when there is no empty cell and no two equal neighbours; read moves from the command line as **0 = left, 1 = right, 2 = up, 3 = down**. Every one of those rules is in the design below.

Question	Assumed answer
Board size?	Configurable N (default 4).
New tile?	One per successful move: value 2 with probability 0.9, else 4, in a uniformly random empty cell. The game starts with two 2s.
Merge rule?	Each tile merges **at most once per move**; merges happen from the wall you move toward.
Move that changes nothing?	Not a turn: no new tile, no score.
Win?	Reaching the target tile (2048, configurable). The player may continue afterwards.
Game over?	No empty cell **and** no two equal neighbours (horizontally or vertically).
Undo, high scores, animations?	Out of scope; see Extensions.

Requirements

Functional

1. Start an N×N board with two tiles numbered 2 at random positions.
2. Move in four directions: slide, merge equal neighbours once, add merged values to the score.
3. If the move changed the board, spawn one tile (2 or 4) in a random empty cell.
4. Detect reaching the target (WON, then allow continuing).
5. Detect game over (LOST) when no move is possible.
6. Accept the command-line codes 0/1/2/3 for left/right/up/down (`Direction.fromKey`).

Non-functional

- **Deterministic** spawns via an injected, seeded `Random` for tests.
- The merge logic must be written **once**, not four times (one per direction).
- The spawning rule is pluggable (e.g. an "evil" spawner for testing hard cases).

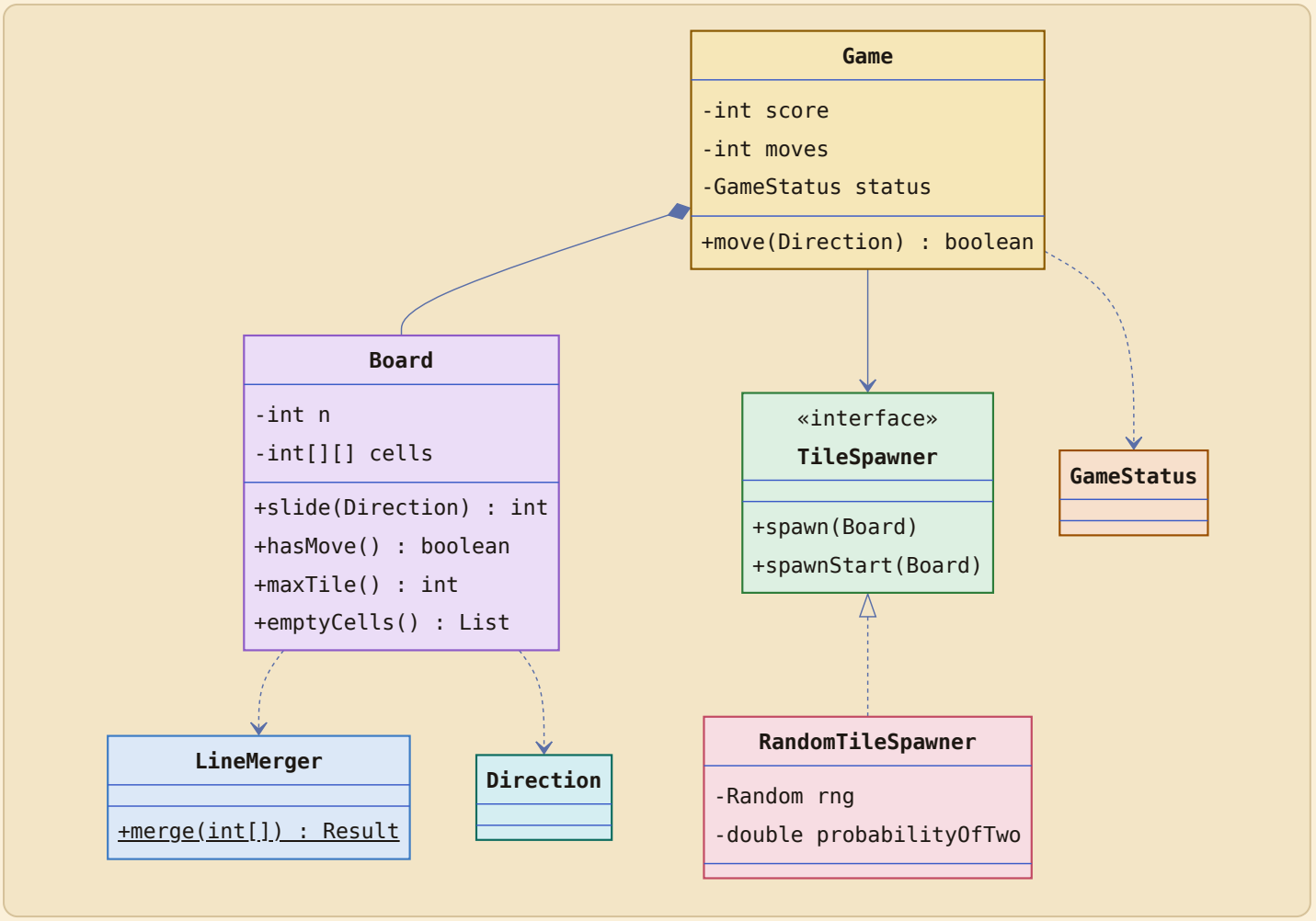
Use cases

Actor: **Player**. Use cases: *start new game*, *make a move* (includes *slide and merge*, *spawn tile*, *update score*), *see win*, *see game over*, *restart*.

Core entities

Class	Responsibility
`Direction` (enum)	LEFT, RIGHT, UP, DOWN; `fromKey(0..3)` maps the command-line codes.
`LineMerger`	Pure function (output depends only on input; changes nothing else): slide and merge **one line toward index 0**; returns the new line and points.
`Board`	N×N values; maps each direction onto lines for `LineMerger`; `slide`, `hasMove`, `maxTile`, `emptyCells`, `render`.
`TileSpawner` (interface) / `RandomTileSpawner`	Adds a new tile; seeded `Random`, 90% twos. `spawnStart` places the two starting tiles (always 2s).
`GameStatus` (enum)	PLAYING, WON, LOST.
`Game`	Score, move count, status; ties slide + spawn + status together.

Class diagram



Key flows

Merging one line (toward index 0) — the whole game in one function:

1. Walk the line from the wall outward, skipping zeros.
2. Keep one **pending** tile. If the next non-zero tile equals it, write the doubled value, add it to the points, and clear pending — so the new tile **cannot merge again this move**.
3. If it differs, write the pending tile and make the new tile pending.
4. At the end, write any pending tile; the rest of the line is zeros.

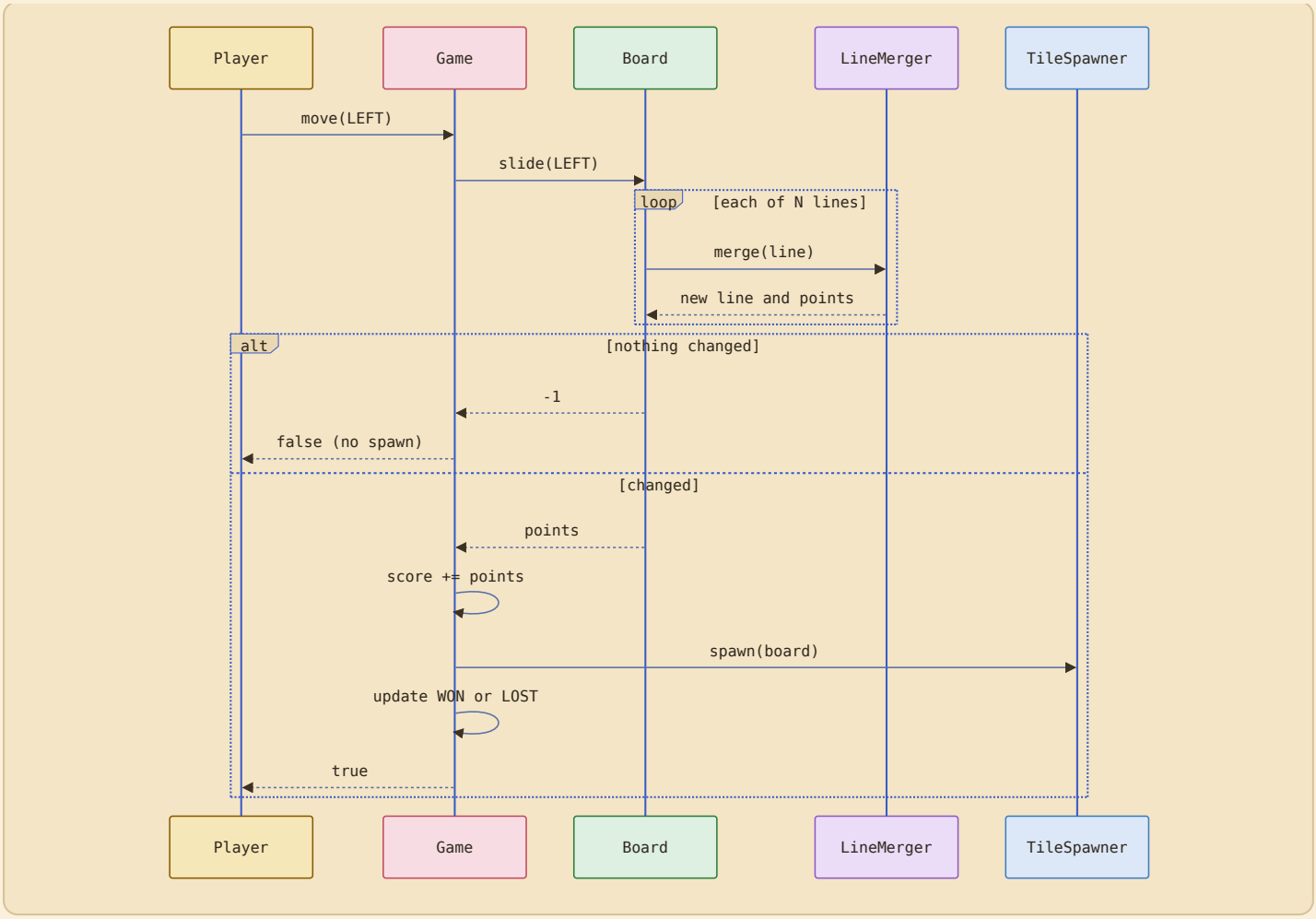
Examples (moving left): `[2,2,2,2] → [4,4,0,0]` (two merges, not one 8), `[2,2,2,0] → [4,2,0,0]` (of three equal tiles, the pair nearest the wall merges), `[2,2,4,0] → [4,4,0,0]` (the new 4 does not merge with the old 4), `[4,0,4,8] → [8,8,0,0]`.

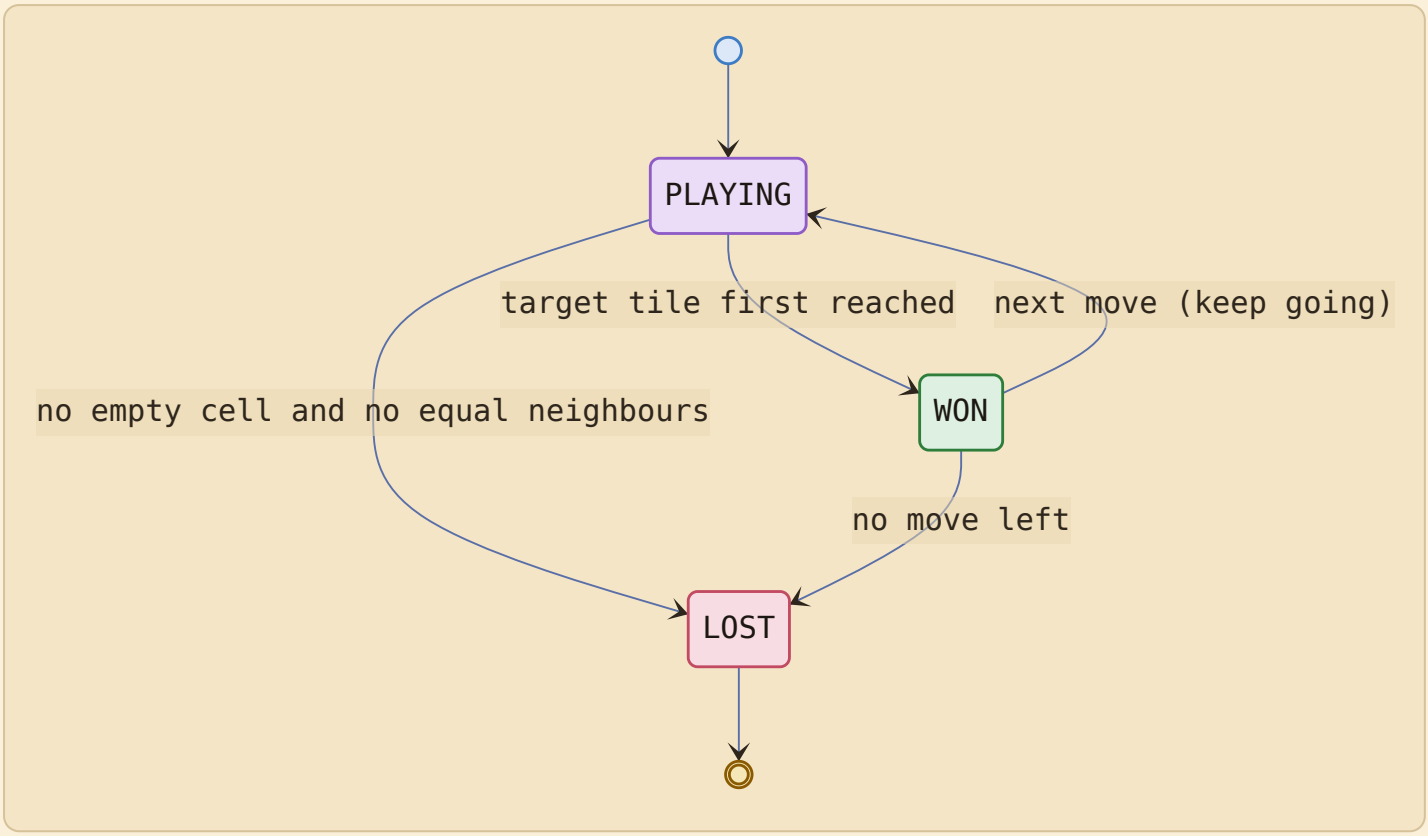
One function for four directions. `Board.coords(direction, i, j)` maps "line `i`, position `j` counted from the wall" to a real cell:

Direction	Line i is	j = 0 is
LEFT	row i	column 0
RIGHT	row i	column N-1
UP	column i	row 0
DOWN	column i	row N-1

Read each line in that order, merge it with `LineMerger`, write it back. No duplicated slide code.

A move





Game-over check. A move is possible if **any cell is empty** or **any two neighbours are equal**. It is enough to compare each cell with its right and lower neighbour (the other two directions are covered by the neighbour's own check). O(N²).

Design decisions & patterns used

Decision	Why	Alternative rejected
One pure `LineMerger` + direction→coordinates mapping.	The tricky rule is written and tested once (DRY — "Don't Repeat Yourself").	Four copies of nested loops (`moveLeft`, `moveRight`...) — classic bug farm.
`LineMerger` is a **pure function** (no state).	Trivial to unit test (test one small piece of code on its own; Part 1 of the output).	Merge logic mixed with board mutation.
Strategy for spawning (`TileSpawner`).	Seeded tests, "evil" spawner, different probabilities for variants.	`Math.random()` inside `Board`.
"No change = no turn" detected by comparing each written cell.	Matches the real game; prevents spawning on a no-op.	Always spawning — makes the game unwinnable/unfair.
`Board.of(int[][])` factory for preset boards.	Game-over and edge-case tests without playing to that position.	Only random starts — untestable edge cases.
Status updated after each move; WON only once.	The player can continue past 2048, as in the original game.	Ending the game at 2048.

Java implementation

Package `lld.ch17.game2048`.

```java
// File: code/lld/ch17/game2048/Direction.java
package lld.ch17.game2048;

public enum Direction {
    LEFT, RIGHT, UP, DOWN;

    /** Command-line input of the common machine-coding statement: 0 = left, 1 = right, 2 = up, 3 = down. */
    public static Direction fromKey(int key) {
        return switch (key) {
            case 0 -> LEFT;
            case 1 -> RIGHT;
            case 2 -> UP;
            case 3 -> DOWN;
            default -> throw new IllegalArgumentException("unknown move " + key + " (use 0-3)");
        };
    }
}
```

```java
// File: code/lld/ch17/game2048/LineMerger.java
package lld.ch17.game2048;

/**
 * The heart of 2048, written for ONE line moving toward index 0.
 * Every direction is reduced to this by reading the line in the right order.
 * Rule: slide tiles together, merge equal neighbours once per move, the merge nearest the wall first.
 */
public final class LineMerger {
    private LineMerger() {}

    /** Result: the new line and the points earned (sum of merged tile values). */
    public record Result(int[] line, int points) {}

    public static Result merge(int[] in) {
        int n = in.length;
        int[] out = new int[n];
        int write = 0, points = 0;
        int pending = 0;                        // tile waiting for a possible merge partner
        for (int v : in) {
            if (v == 0) continue;
            if (pending == 0) {
                pending = v;
            } else if (pending == v) {
                out[write++] = v * 2;           // merge, and the new tile cannot merge again this move
                points += v * 2;
                pending = 0;
            } else {
                out[write++] = pending;
                pending = v;
            }
        }
        if (pending != 0) out[write] = pending;
        return new Result(out, points);
    }
}
```

```java
// File: code/lld/ch17/game2048/TileSpawner.java
package lld.ch17.game2048;

/** Strategy for adding a new tile after each successful move. */
public interface TileSpawner {
    /** Adds one tile to a random empty cell. The board must have at least one empty cell. */
    void spawn(Board board);

    /** One of the two tiles placed when a new game starts. By default the same as spawn(). */
    default void spawnStart(Board board) { spawn(board); }
}
```

```java
// File: code/lld/ch17/game2048/RandomTileSpawner.java
package lld.ch17.game2048;

import java.util.List;
import java.util.Random;

/** Classic rule: a 2 with probability 0.9, else a 4, in a uniformly random empty cell. Seeded for tests. */
public final class RandomTileSpawner implements TileSpawner {
    private final Random rng;
    private final double probabilityOfTwo;

    public RandomTileSpawner(Random rng, double probabilityOfTwo) {
        this.rng = rng;
        this.probabilityOfTwo = probabilityOfTwo;
    }

    @Override public void spawn(Board board) {
        int[] cell = randomEmptyCell(board);
        board.set(cell[0], cell[1], rng.nextDouble() < probabilityOfTwo ? 2 : 4);
    }

    /** The problem statement starts every game with two tiles numbered 2. */
    @Override public void spawnStart(Board board) {
        int[] cell = randomEmptyCell(board);
        board.set(cell[0], cell[1], 2);
    }

    private int[] randomEmptyCell(Board board) {
        List<int[]> empty = board.emptyCells();
        if (empty.isEmpty()) throw new IllegalStateException("no empty cell");
        return empty.get(rng.nextInt(empty.size()));
    }
}
```

```java
// File: code/lld/ch17/game2048/Board.java
package lld.ch17.game2048;

import java.util.ArrayList;
import java.util.List;

/** N x N grid of tile values (0 = empty). Knows how to slide in a direction and whether any move is left. */
public final class Board {
    private final int n;
    private final int[][] cells;

    public Board(int n) {
        if (n < 2) throw new IllegalArgumentException("board too small");
        this.n = n;
        this.cells = new int[n][n];
    }

    public static Board of(int[][] values) {
        Board b = new Board(values.length);
        for (int r = 0; r < b.n; r++) b.cells[r] = values[r].clone();
        return b;
    }

    public int size() { return n; }
    public int get(int r, int c) { return cells[r][c]; }
    void set(int r, int c, int v) { cells[r][c] = v; }

    public List<int[]> emptyCells() {
        List<int[]> out = new ArrayList<>();
        for (int r = 0; r < n; r++) for (int c = 0; c < n; c++) if (cells[r][c] == 0) out.add(new int[]{r, c});
        return out;
    }

    /**
     * Slides every line toward the given wall. Returns the points earned, or -1 if nothing moved
     * (a move that changes nothing is not a turn and must not spawn a tile).
     */
    int slide(Direction d) {
        boolean changed = false;
        int points = 0;
        for (int i = 0; i < n; i++) {
            int[] line = new int[n];
            for (int j = 0; j < n; j++) line[j] = read(d, i, j);       // j = 0 is the cell next to the wall
            LineMerger.Result res = LineMerger.merge(line);
            for (int j = 0; j < n; j++) {
                if (read(d, i, j) != res.line()[j]) changed = true;
                write(d, i, j, res.line()[j]);
            }
            points += res.points();
        }
        return changed ? points : -1;
    }

    /** Maps (line i, position j from the wall) to a real cell for each direction. */
    private int[] coords(Direction d, int i, int j) {
        return switch (d) {
            case LEFT -> new int[]{i, j};
            case RIGHT -> new int[]{i, n - 1 - j};
            case UP -> new int[]{j, i};
            case DOWN -> new int[]{n - 1 - j, i};
        };
    }

    private int read(Direction d, int i, int j) { int[] rc = coords(d, i, j); return cells[rc[0]][rc[1]]; }
    private void write(Direction d, int i, int j, int v) { int[] rc = coords(d, i, j); cells[rc[0]][rc[1]] = v; }

    /** A move exists if a cell is empty or two neighbours (right or below) are equal. */
    public boolean hasMove() {
        for (int r = 0; r < n; r++)
            for (int c = 0; c < n; c++) {
```

```
                if (cells[r][c] == 0) return true;
                if (c + 1 < n && cells[r][c] == cells[r][c + 1]) return true;
                if (r + 1 < n && cells[r][c] == cells[r + 1][c]) return true;
            }
        return false;
    }

    public int maxTile() {
        int m = 0;
        for (int[] row : cells) for (int v : row) m = Math.max(m, v);
        return m;
    }

    public String render() {
        StringBuilder sb = new StringBuilder();
        for (int r = 0; r < n; r++) {
            sb.append("   ");
            for (int c = 0; c < n; c++) sb.append(String.format("%5s", cells[r][c] == 0 ? "." : cells[r][c]));
            if (r < n - 1) sb.append('\n');
        }
        return sb.toString();
    }
}
```

```
// File: code/lld/ch17/game2048/GameStatus.java
package lld.ch17.game2048;

public enum GameStatus { PLAYING, WON, LOST }
```

```java
// File: code/lld/ch17/game2048/Game.java
package lld.ch17.game2048;

/**
 * One game: board + spawner + score + status. Reaching the target tile sets WON once;
 * the player may keep going ("continue" mode). No move left sets LOST.
 */
public final class Game {
    private final Board board;
    private final TileSpawner spawner;
    private final int target;
    private int score;
    private int moves;
    private boolean reachedTarget;
    private GameStatus status = GameStatus.PLAYING;

    /** New game: an empty board with two starting tiles (two 2s with the standard spawner). */
    public Game(int size, int target, TileSpawner spawner) {
        this(new Board(size), target, spawner);
        spawner.spawnStart(board);
        spawner.spawnStart(board);
        updateStatus();
    }

    /** Game from a preset board (used by tests). */
    public Game(Board preset, int target, TileSpawner spawner) {
        this.board = preset;
        this.target = target;
        this.spawner = spawner;
        updateStatus();
    }

    /** Returns true if the move changed the board (and so counted as a turn). */
    public boolean move(Direction d) {
        if (status == GameStatus.LOST) throw new IllegalStateException("game over");
        int points = board.slide(d);
        if (points < 0) return false;          // nothing moved: no spawn, no turn
        score += points;
        moves++;
        spawner.spawn(board);                  // at least one cell is empty, because something moved or merged
        updateStatus();
        return true;
    }

    private void updateStatus() {
        if (!reachedTarget && board.maxTile() >= target) { reachedTarget = true; status = GameStatus.WON; }
        else if (status == GameStatus.WON) status = GameStatus.PLAYING;   // "keep going" after the win screen
        if (!board.hasMove()) status = GameStatus.LOST;
    }

    public GameStatus status() { return status; }
    public boolean reachedTarget() { return reachedTarget; }
    public int score() { return score; }
    public int moves() { return moves; }
    public Board board() { return board; }
}
```

```java
// File: code/lld/ch17/game2048/Main.java
package lld.ch17.game2048;

import java.util.Arrays;
import java.util.List;
import java.util.Random;

public final class Main {
    public static void main(String[] args) {
        System.out.println("== 1. Merge rules for one line moving left ==");
        for (int[] line : List.of(new int[]{2, 2, 2, 2}, new int[]{2, 2, 2, 0}, new int[]{2, 2, 4, 0}, new int[]
{4, 0, 4, 8},
                new int[]{2, 0, 0, 2}, new int[]{8, 4, 2, 2}, new int[]{2, 4, 8, 16})) {
            LineMerger.Result r = LineMerger.merge(line);
            System.out.println("  " + Arrays.toString(line) + " -> " + Arrays.toString(r.line()) + "  +" +
r.points());
        }

        System.out.println("\n== 2. A move that changes nothing does not count and spawns nothing ==");
        Game stuckLeft = new Game(Board.of(new int[][]{{2, 4, 8, 16}, {0, 0, 0, 0}, {0, 0, 0, 0}, {0, 0, 0, 0}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  move LEFT changed board? " + stuckLeft.move(Direction.LEFT) + ", moves=" +
stuckLeft.moves());
        System.out.println("  move DOWN changed board? " + stuckLeft.move(Direction.DOWN) + ", moves=" +
stuckLeft.moves());
        System.out.println(stuckLeft.board().render());

        System.out.println("\n== 3. Game-over detection ==");
        Game full = new Game(Board.of(new int[][]{{2, 4, 2, 4}, {4, 2, 4, 2}, {2, 4, 2, 4}, {4, 2, 4, 2}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  checkerboard, no empty cell, no equal neighbours -> " + full.status());
        Game almost = new Game(Board.of(new int[][]{{2, 4, 2, 4}, {4, 2, 4, 2}, {2, 4, 2, 4}, {4, 2, 4, 4}}),
                2048, new RandomTileSpawner(new Random(1), 0.9));
        System.out.println("  same but last row ends 4,4 -> " + almost.status());

        System.out.println("\n== 4. Keyboard codes from the problem statement ==");
        for (int key = 0; key < 4; key++) System.out.print("  " + key + "=" + Direction.fromKey(key));
        System.out.println();

        System.out.println("\n== 5. Seeded game (seed 42), corner strategy LEFT > DOWN > RIGHT > UP, target 128
==");
        Game g = new Game(4, 128, new RandomTileSpawner(new Random(42), 0.9));
        System.out.println("  start:\n" + g.board().render());
        List<Direction> priority = List.of(Direction.LEFT, Direction.DOWN, Direction.RIGHT, Direction.UP);
        boolean announced = false;
        while (g.status() != GameStatus.LOST) {
            for (Direction d : priority) if (g.move(d)) break;
            if (g.reachedTarget() && !announced) {
                announced = true;
                System.out.println("  reached 128 after " + g.moves() + " moves (status " + g.status() + "), score
" + g.score());
                System.out.println(g.board().render());
            }
        }
        System.out.println("  game over after " + g.moves() + " moves, score " + g.score() + ", max tile " +
g.board().maxTile());
        System.out.println(g.board().render());
    }
}
```

Output:

```
== 1. Merge rules for one line moving left ==
  [2, 2, 2, 2] -> [4, 4, 0, 0]  +8
  [2, 2, 2, 0] -> [4, 2, 0, 0]  +4
  [2, 2, 4, 0] -> [4, 4, 0, 0]  +4
  [4, 0, 4, 8] -> [8, 8, 0, 0]  +8
  [2, 0, 0, 2] -> [4, 0, 0, 0]  +4
  [8, 4, 2, 2] -> [8, 4, 4, 0]  +4
  [2, 4, 8, 16] -> [2, 4, 8, 16]  +0

== 2. A move that changes nothing does not count and spawns nothing ==
  move LEFT changed board? false, moves=0
  move DOWN changed board? true, moves=1
       .    .    .    .
       .    .    .    .
       .    2    .    .
       2    4    8   16

== 3. Game-over detection ==
  checkerboard, no empty cell, no equal neighbours -> LOST
  same but last row ends 4,4 -> PLAYING

== 4. Keyboard codes from the problem statement ==
  0=LEFT  1=RIGHT  2=UP  3=DOWN

== 5. Seeded game (seed 42), corner strategy LEFT > DOWN > RIGHT > UP, target 128 ==
  start:
       .    .    .    2
       .    .    .    .
       .    .    .    2
       .    .    .    .
  reached 128 after 85 moves (status WON), score 852
       8    2    .    .
      16    4    4    .
       2    8    4    .
     128    4    .    .
  game over after 232 moves, score 2584, max tile 256
      16    8    4    2
       8   32    8    4
      16    2   16    8
     256   64   32   16
```

Reading the output. Part 1 unit-tests the merge rule (note `[2,2,2,2]` becomes `[4,4,0,0]`, not `[8,0,0,0]`, and `[2,2,2,0]` becomes `[4,2,0,0]`). Part 2 shows that a move changing nothing does not count. Part 3 checks game-over detection on preset boards. Part 4 prints the command-line key mapping. Part 5 plays a whole seeded game that starts from two 2s, using a simple "keep tiles in the bottom-left corner" priority strategy; it reaches the 128 target after 85 moves (status `WON`) and keeps playing until the board locks after 232 moves with a 256 tile.

Concurrency & edge cases

- Single-player and single-threaded by nature. If a UI thread (the thread that draws the screen) and an input thread both call `move`, make `move` `synchronized` (or post moves to the UI event queue — the list of pending clicks, key presses and redraws that the UI thread handles one by one).
- **Key repeat / double input:** a no-op move returns `false` and changes nothing, so extra key presses are harmless.
- **Spawning needs an empty cell:** guaranteed, because a changed board after sliding always has at least one empty cell (tiles only move into gaps or merge, which frees a cell). The spawner still checks and throws if not.
- **Integer overflow** (a number growing past the largest value its type can hold, so it wraps around to a wrong value): not reachable on 4×4 (the maximum possible tile is 2^17 = 131072); a very large board would still fit an `int` for any practical game.
- **Merge order matters:** `[2,2,2]` moving left is `[4,2,0]` — the pair nearest the wall merges first.

Extensions & follow-up questions

- **"Add undo."** — Memento: push a copy of the board and score before each successful move; undo pops it (the spawned tile disappears too).
- **"Different sizes / targets?"** — Already parameters (`new Game(5, 4096, spawner)`).
- **"An AI player?"** — Expectimax search (max over your moves, average over random spawns) is the well-known approach; plug it in as a move-choosing strategy like the corner strategy in `Main`.
- **"Animations?"** — Have `LineMerger` also return where each tile came from (a list of moves/merges) and publish them to the view (Observer, MVC).
- **"High scores / leaderboard?"** — A separate service; the game only emits "game over with score X".
- **"Why is `[2,2,4]` left `[4,4,0]` and not `[8,0,0]`?"** — Each tile merges at most once per move; the new 4 was created this move.

Real-world example

2048 was created by Gabriele Cirulli in 2014 and released as open-source (its code is public for anyone to read and reuse) JavaScript; its code has the same separation: a grid model, a game manager that handles moves, merges, score and "keep playing" after a win, and a separate view that animates tiles. Its new tiles are 2 with 90% probability and 4 with 10%, the same rule our `RandomTileSpawner` uses.

In the interview

> **30-second answer:** "The core is one pure function that slides and merges a single line toward index 0: skip zeros, merge equal neighbours once, score the merged values. The board maps each direction to lines in the right order, so the function is reused four times. After a move that changes the board, a `TileSpawner` — seeded for tests — adds a 2 (90%) or a 4. The game is won when the target tile appears and over when there's no empty cell and no equal neighbours."

Time plan: 5 min rules (merge once, no-op move) → 5 min entities → 15 min `LineMerger` + direction mapping (write the tests first: `[2,2,2,2]`, `[2,2,4,0]`) → 10 min game loop, spawning, game over → 5 min extensions.

Self-check

1. **Q:** What does `[4,4,8,8]` become moving left, and how many points? **A:** `[8,16,0,0]`, 24 points.
2. **Q:** Why must a no-op move not spawn a tile? **A:** It is not a turn in the real game; spawning would punish harmless key presses and change the game's balance.
3. **Q:** How do you avoid four copies of the slide code? **A:** Map each direction to lines read from the wall, then call one line-merge function.
4. **Q:** When is the game over? **A:** No empty cell and no two equal adjacent tiles.
5. **Q:** How do you make a 2048 test repeatable? **A:** Inject `new Random(seed)` into the spawner, or start from `Board.of(...)`.

TL;DR

- One pure `LineMerger` (merge once per move, from the wall) + a direction→line mapping.
- Spawn only after a real change; 2 (90%) or 4 (10%), seeded spawner for tests.
- WON once at the target (keep playing); LOST when no empty cell and no equal neighbours.

Chapter summary

Game	Core rule object	Pluggable strategies	Randomness for tests	Signature technique
Blackjack	`Hand` scoring + `BlackjackScoring`	`PlayerStrategy`, `DealerPolicy`	Seeded `Shoe`, stacked shoe	Soft/hard totals; generic `Deck<T>`
Chess	`Rules` (legal-move filter)	(bot would be a move strategy)	None needed — deterministic game	Polymorphic pieces; checkmate = in check + no legal move; Command + Memento undo
Tic-Tac-Toe	`Board` counters	`MoveStrategy`	Seeded `RandomStrategy`, scripted players	O(1) win with row/col/diagonal counters
Snake & Ladder	`Board` + `Rules` record	`Dice`, `OverflowRule`, `WinStrategy`	Seeded `RandomDice`, `LoadedDice`	Validated config; queue-based turns
2048	`LineMerger`	`TileSpawner`	Seeded spawner, `Board.of` presets	One merge function reused for 4 directions

Patterns you used in this chapter: Strategy (everywhere), Factory Method (`Deck.standard52`, `Piece.create`, `Board.of`), Template-style shared base (`SlidingPiece`), Command + Memento (chess undo), State (game-status enums; a full State pattern is the suggested upgrade for blackjack rounds), Observer (the event log hooks; suggested for UIs and spectators).

The three habits interviewers look for in game designs:

1. **Put every rule in exactly one place** — and be able to say where.
2. **Inject randomness** — seeded or scripted — so the design is testable.
3. **Model the game's states explicitly** — and reject actions that are invalid in the current state.